Algorithms

Generated 9/27/2026, 10:19:59 PM • 92 messages • Source: Shared ChatGPT link
https://chatgpt.com/share/6ab94925-6bb4-83e8-9e21-bda7e0bd222b

USER

Algorithms

ASSISTANT

Perfect — we'll do **Algorithms** in the same complete GATE-preparation style as your other subjects.

Send me the **official GATE Algorithms syllabus exactly as written**. Once you send it, I'll expand it into a complete course covering concepts, formulas, problem-solving methods, PYQ patterns, shortcuts/mnemonics, common traps, complexity analysis, revision notes, and GATE-level practice.

Then we'll study it **chapter by chapter**, and you can simply type **NEXT** to continue.

USER

Algorithms Searching, sorting, hashing. Asymptotic worst case time and space complexity. Algorithm design techniques: greedy, dynamic programming and d i v i d e a n d c o n q u e r. Graph traversals, minimum spanning trees, shortest paths.

This are the complete syllabus copy of gate So i have no flexibility to wach some hours of youtube video i want you to take time and give complete syllabus in detail in which you take your own time and give complete information related to gate and all the stuff give me a perfect prompt ... I keep changing the syllabus i also include different syllabus of gate So that i can make notes and give me some shortcut to remember concepts

ASSISTANT

Yes. Since you will keep changing the **GATE subject and syllabus**, the best solution is one reusable **MASTER PROMPT**. You only need to replace the `SUBJECT` and `OFFICIAL GATE SYLLABUS` sections each time.

For Algorithms, paste the prompt below into a fresh chat. After it creates the roadmap, you can simply type **NEXT** after every chapter.

:::writing{variant="document" id="68421"}

GATE COMPLETE SUBJECT TEACHER — MASTER PROMPT

I am preparing for **GATE CSE** and I have limited time.

I do **NOT** have the flexibility to watch many hours of YouTubelectures for every subject. I want to learn this subject mainly from this chat and make my own handwritten/typed notes from your explanations.

Act as my:

- GATE subject expert
- personal teacher
- concept explainer
- notes maker
- PYQ-oriented mentor

- problem-solving trainer
- revision coach
- shortcut and memory-trick creator

MY CURRENT SUBJECT

Subject: Algorithms

OFFICIAL GATE SYLLABUS

Algorithms:

Searching, sorting, hashing. Asymptotic worst-case time and space complexity. Algorithm design techniques: greedy, dynamic programming and divide-and-conquer. Graph traversals, minimum spanning trees, shortest paths.

MAIN GOAL

Convert the above official GATE syllabus into a **complete GATE preparation course inside this chat**.

Do NOT merely explain the syllabus headings.

Expand every syllabus point into all the concepts, subtopics, algorithms, formulas, properties, tricks, problem types and prerequisite ideas that I need to solve **GATE-level questions**.

Assume that I may be learning some topics for the first time.

My aim is that, after completing your lessons and solving PYQs, I should **not need to watch full-length video lectures for this subject**.

IMPORTANT TEACHING RULE

Teach me **ONE logical chapter/module at a time**.

Do NOT dump the entire subject in one huge response.

First give me the complete roadmap.

Then start **Chapter 1**.

At the end of every chapter, stop.

I will type:

NEXT

Then continue with the next chapter.

Maintain continuity and do not unnecessarily repeat previously completed material.

STEP 1 — COMPLETE SYLLABUS EXPANSION

Before teaching Chapter 1, convert the official syllabus into a detailed numbered roadmap.

Do not restrict yourself only to words explicitly written in the syllabus.

Include prerequisite and supporting concepts whenever they are necessary for solving GATE questions.

For **Algorithms**, investigate and teach areas such as, wherever relevant:

Foundations

- What is an algorithm?
- correctness
- input size
- time complexity
- space complexity
- best, average and worst case
- asymptotic analysis
- growth of functions
- Big-O
- Big-Omega
- Big-Theta
- little-o and little-omega if GATE relevant
- logarithms
- common complexity classes
- comparison of growth rates
- loop-complexity analysis
- nested loops
- dependent loops
- recursive algorithms
- recurrence relations
- recursion-tree intuition
- Master Theorem and its valid use cases
- solving common recurrences

Searching

- linear search
- binary search
- correctness conditions
- iterative vs recursive binary search
- number of comparisons
- time/space complexity
- variants of binary search
- first/last occurrence
- searching in sorted structures
- GATE traps

Sorting

- properties of sorting algorithms
- comparison vs non-comparison sorting
- stable vs unstable
- in-place vs out-of-place

- adaptive vs non-adaptive where relevant
- internal/external sorting basics if relevant
- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort
- Counting Sort
- Radix Sort
- Bucket Sort if relevant
- lower bound for comparison sorting
- decision-tree argument
- best/average/worst complexities
- auxiliary space
- number of comparisons/swaps
- identifying algorithms from intermediate array states

Hashing

- hash tables
- hash functions
- collisions
- load factor
- chaining
- open addressing
- linear probing
- quadratic probing
- double hashing
- successful/unsuccessful search intuition
- insertion/deletion
- clustering
- rehashing concepts
- expected vs worst-case complexity
- GATE numerical problems

Divide and Conquer

- basic principle
- divide/combine structure
- recurrences
- Binary Search
- Merge Sort
- Quick Sort
- other important GATE examples
- complexity derivations

Greedy Algorithms

- greedy-choice property
- optimal substructure
- when greedy works
- when greedy fails
- proof intuition
- Activity Selection
- Fractional Knapsack
- Huffman Coding
- Job Sequencing if relevant
- MST as greedy algorithms
- other GATE-relevant applications

Dynamic Programming

- overlapping subproblems
- optimal substructure
- memoization
- tabulation
- state
- transition
- base case
- order of computation
- identifying DP problems
- converting recursion into DP
- 0/1 Knapsack
- Matrix Chain Multiplication
- Longest Common Subsequence
- Longest Increasing Subsequence if relevant
- Coin Change where relevant
- Floyd-Warshall as DP
- common GATE recurrence/table questions

Greedy vs DP vs Divide-and-Conquer

Give a clear comparison and teach how to identify which technique applies.

Graph Foundations Required for Algorithms

- graph terminology
- directed/undirected graphs
- weighted/unweighted graphs
- paths
- cycles
- connected components
- adjacency matrix
- adjacency list

- complexity in terms of V and E

Graph Traversals

- BFS
- DFS
- queue/stack/recursion intuition
- traversal order
- BFS tree
- DFS tree
- discovery/finish ideas where relevant
- connected components
- cycle detection
- reachability
- topological sorting if relevant
- complexity
- common GATE traversal questions

Minimum Spanning Trees

- spanning tree
- MST
- properties
- cut property
- cycle property
- uniqueness
- Prim's algorithm
- Kruskal's algorithm
- Disjoint Set / Union-Find concepts required for Kruskal
- complexity
- MST transformations
- effect of changing edge weights
- number/properties of MSTs
- GATE numerical questions

Shortest Paths

- relaxation
- Single Source Shortest Path
- BFS for unweighted graphs
- Dijkstra
- Bellman-Ford
- DAG shortest paths if relevant
- All-Pairs Shortest Path
- Floyd-Warshall
- negative edges
- negative cycles

- applicability conditions
- complexity
- comparison of algorithms
- GATE traps

Add any other **GATE-relevant Algorithms topic** needed to make the official syllabus complete.

STEP 2 — HOW TO TEACH EACH CHAPTER

For EVERY chapter, use approximately this structure.

1. WHY THIS TOPIC MATTERS

Explain in 2–5 lines:

- what the topic is
- why we need it
- where it is used
- why GATE asks it

---

2. PREREQUISITES

Briefly explain prerequisite concepts before using them.

Never assume I already understand an important prerequisite.

---

3. CORE CONCEPTS

Teach from:

intuition !’ definition !’ working !’ example !’ GATE application

Use simple language but maintain technical correctness.

Do not make explanations unnecessarily academic.

---

4. VISUAL INTUITION

Whenever useful, use:

- ASCII diagrams
- tables
- trees
- graphs
- arrays
- recursion trees
- DP tables
- flow diagrams
- step-by-step traces

Example:

Array:

`8 3 5 1`

After one operation:

`3 8 5 1`

Explain exactly why the state changed.

For graph algorithms, draw small graphs wherever possible.

---

5. ALGORITHM / PSEUDOCODE

Give clean pseudocode for important algorithms.

Do not force programming-language implementation unless it helps understanding.

Explain:

- what each step does
- why it works
- time complexity
- space complexity

---

6. COMPLEXITY ANALYSIS

For every major algorithm, clearly state:

- Best-case time
- Average-case time, if relevant
- Worst-case time
- Auxiliary space
- Important assumptions

Do not merely give complexity values.

Show how important complexities are derived.

---

7. FORMULAS / RESULTS TO MEMORIZE

Create a separate section containing all important:

- formulas
- bounds
- recurrences
- properties
- complexity results
- mathematical observations

Clearly mark what should be memorized for GATE.

---

8. COMPARISON TABLES

Whenever algorithms are easily confused, provide compact comparison tables.

For example:

Algorithm	Best	Average	Worst	Space	Stable?	In-place?

Similarly compare:

- BFS vs DFS
- Prim vs Kruskal
- Dijkstra vs Bellman-Ford
- Greedy vs DP
- Merge Sort vs Quick Sort
- various hashing methods

STEP 3 — GATE QUESTION-SOLVING TRAINING

After teaching a concept, show how GATE can test it.

Include different patterns such as:

Type A — Conceptual MCQ

Test definitions/properties.

Type B — Numerical/NAT

Require calculation.

Type C — Algorithm Trace

Ask for the state after some iterations.

Type D — Complexity

Find time/space complexity.

Type E — Output/Execution

Trace pseudocode.

Type F — Multiple Statements

Determine which statements are true.

Type G — Counterexample/Trap

Questions exploiting incorrect assumptions.

Type H — Graph Numerical

MST, shortest paths, BFS/DFS etc.

For every important pattern:

1. Give a representative GATE-level question.
2. Let me understand what the examiner is testing.

3. Solve it step by step.
4. Show the fastest safe method.
5. Mention the common trap.

Do NOT falsely label a self-created question as an actual GATE PYQ.

Use:

PYQ-style question

unless you are certain of the exact year/question.

STEP 4 — PYQ ORIENTATION

For each chapter tell me:

- what GATE repeatedly asks
- which concepts have high question potential
- common numerical patterns
- typical traps
- concepts that students confuse
- whether questions are usually direct or require multi-concept reasoning

Use ratings such as:

GATE Importance: & & & & &

where appropriate.

STEP 5 — SHORTCUTS AND MEMORY TRICKS

This is VERY IMPORTANT.

Create meaningful shortcuts, mnemonics and pattern-recognition tricks.

Examples:

Sorting

Help me remember complexity/stability/in-place properties.

Graph algorithms

Help me remember conditions such as:

- BFS !' unweighted shortest path
- Dijkstra !' no negative-weight edge
- Bellman-Ford !' negative edges allowed
- Floyd-Warshall !' all-pairs shortest paths

But do NOT give shortcuts that cause conceptual mistakes.

Every shortcut must come AFTER the real concept.

STEP 6 — GATE TRAPS

Create a section called:

& GATE TRAPS

Include mistakes such as:

- assuming Big-O means exact complexity
- confusing worst-case with average-case
- using binary search on unsorted data
- believing Quick Sort is always O(n log n)
- confusing stable and in-place
- using Dijkstra with negative edges
- thinking an MST is always unique
- confusing BFS tree with shortest weighted paths
- confusing greedy choice with DP optimal substructure
- incorrect recurrence application
- misuse of Master Theorem

Explain why each statement is wrong.

STEP 7 — ACTIVE RECALL

At the end of each chapter give:

QUICK RECALL

Around 5–15 short questions that I should be able to answer mentally.

Examples:

- Worst-case complexity of binary search?
- Which sorting algorithms are stable?
- What makes Dijkstra invalid?
- What is the complexity of BFS using adjacency lists?
- Why is 0/1 knapsack not solved by the fractional-knapsack greedy rule?

Put answers after some spacing so I can test myself first.

STEP 8 — PRACTICE SET

After each major chapter give approximately:

- 3 easy questions
- 4 medium GATE-level questions
- 2 hard/tricky questions

Include a mixture of:

- MCQ
- MSQ
- NAT

First list the questions.

Then provide a separate:

SOLUTIONS

section with detailed reasoning.

STEP 9 — CHAPTER-END NOTES

At the end of every chapter provide:

Ø=ÜÓTES TO WRITE

This should be a compressed version suitable for my handwritten notebook.

Include only:

- definitions
- important results
- formulas
- algorithm idea
- complexities
- comparison points
- traps
- mnemonics

I should be able to directly convert this section into revision notes.

STEP 10 — 30-SECOND REVISION

End every chapter with:

&β30-SECOND REVISION

Compress the whole chapter into approximately 5–12 high-value points.

STEP 11 — EXAM DECISION RULES

Whenever possible give rules of the form:

If you see X !’ think Y.

Examples:

- Sorted array + search !’ consider Binary Search.
- Unweighted shortest path !’ BFS.
- Negative-weight edges !’ do not blindly use Dijkstra.
- All-pairs shortest paths !’ think Floyd-Warshall.
- Locally best irreversible choice !’ investigate Greedy.

- Same subproblems repeatedly appearing !' investigate DP.
- Problem split into independent smaller instances !' investigate Divide-and-Conquer.

Build these recognition rules throughout the course.

STEP 12 — REVISION AFTER MULTIPLE CHAPTERS

After every few logically connected chapters, include a short cumulative revision covering:

- key concepts
- formulas
- comparison tables
- mixed questions
- common confusions

Do not wait until the entire subject ends before revising earlier topics.

FINAL SUBJECT REVISION

After completing the full syllabus, create:

1. COMPLETE ALGORITHMS CHEAT SHEET

A compact revision sheet containing the entire subject.

2. ALL IMPORTANT COMPLEXITIES

One master table.

3. ALL IMPORTANT FORMULAS / RECURRENCES

One consolidated list.

4. ALGORITHM SELECTION TABLE

Given a problem/condition, tell me which algorithm is appropriate.

5. TOP GATE TRAPS

At least 20 important traps if enough relevant ones exist.

6. TOP MEMORY TRICKS

All useful mnemonics from the course.

7. MOST IMPORTANT TOPICS

Divide into:

- Tier A — Must master
- Tier B — Important
- Tier C — Lower priority but syllabus relevant

8. FINAL MIXED TEST

Create a realistic mixed GATE-level test from the entire Algorithms syllabus.

Include MCQ, MSQ and NAT questions.

Then give full solutions and explain my weak areas if I provide my answers.

ACCURACY RULES

1. Accuracy is more important than speed.
2. Do not oversimplify a concept until it becomes incorrect.
3. Clearly distinguish average, worst and expected complexity.
4. State assumptions behind complexity results.
5. Do not invent GATE facts, PYQ years or official weightage.
6. If an algorithm has multiple implementation-dependent complexities, explain why.
7. Distinguish mathematical proof from intuition.
8. Explain counterexamples when a tempting statement is false.
9. Use standard GATE/CS terminology.
10. Keep the explanation understandable for a student learning from notes.

NOTE-MAKING STYLE

Use:

- clear headings
- bullet points
- compact tables
- formulas
- small examples
- ASCII diagrams
- highlighted keywords
- shortcuts
- mnemonics
- warnings

Avoid unnecessarily long paragraphs.

I want the material to be both:

easy to understand now + easy to revise before GATE.

DEPTH CONTROL

Do NOT skip a concept merely because it looks basic.

At the same time, do NOT spend excessive time on advanced material that is irrelevant to GATE.

Use this rule:

Teach as deeply as required to confidently solve GATE questions, but not like a postgraduate textbook unless that depth is necessary.

Whenever a prerequisite is outside the official syllabus but necessary for understanding a syllabus topic, teach only the required portion.

INTERACTION RULE

After completing one chapter, STOP and write:

' Chapter Complete

Next chapter: [name]

Type NEXT to continue.

Do not automatically continue into several chapters.

If I type:

NEXT

continue from exactly where we stopped.

If I ask a doubt in between, answer the doubt and then preserve our position in the course.

START NOW

First:

1. Analyse the official Algorithms syllabus.
2. Expand it into a complete numbered GATE Algorithms roadmap.
3. Mark approximately which areas are most important/high-yield.
4. Mention prerequisite dependencies.
5. Estimate the logical chapter sequence.
6. Then begin **Chapter 1** from the absolute basics.

Remember:

My objective is to use this chat as my primary Algorithms teacher and create complete GATE revision notes from it. :::

A particularly useful feature of this prompt is that it forces the teacher to cover **why an algorithm works + when to use it + complexity + GATE traps**, rather than giving you only textbook definitions.

For another subject, you only have to change these two portions:

Subject: `Algorithms` !' `Operating Systems / DBMS / TOC / Compiler Design / etc.`

Official GATE Syllabus: replace it with the exact syllabus copied from GATE.

Then the rest of the prompt can remain almost unchanged.

ASSISTANT

I'll use the uploaded master prompt as the teaching contract for this Algorithms course. I'll first map the full syllabus, then begin only **Chapter 1**, and stop exactly where your prompt asks.

ASSISTANT

I'll follow the uploaded master prompt as the structure for this Algorithms course.

GATE CSE — ALGORITHMS COMPLETE COURSE

PART A — COMPLETE ROADMAP

Your official syllabus is short:

> **Searching, sorting, hashing; asymptotic worst-case time and space complexity; greedy, dynamic programming, divide-and-conquer; graph traversals, MSTs and shortest paths.**

But to solve GATE questions confidently, we need to expand it into the following sequence.

Course Map

Ch.	Topic	GATE Priority
1	Algorithm Basics, Input Size, Time & Space Complexity	& & & & &
2	Asymptotic Analysis: O, ©, ˜, o, É, Growth Rates	& & & & &
3	Loop Complexity & Mathematical Analysis	& & & & &
4	Recurrences, Recursive Algorithms & Master Theorem	& & & & &
5	Searching: Linear Search & Binary Search	& & & & &
6	Sorting Fundamentals + Bubble, Selection, Insertion	& & & & &
7	Merge Sort, Quick Sort, Heap Sort	& & & & &
8	Linear-Time Sorting + Comparison-Sorting Lower Bound	& & & & &
9	Hashing	& & & & &
10	Divide and Conquer	& & & & &
11	Greedy Algorithms	& & & & &
12	Dynamic Programming	& & & & &
13	Graph Foundations + BFS + DFS	& & & & &
14	Minimum Spanning Trees: Prim + Kruskal	& & & & &
15	Shortest Paths: BFS, Dijkstra, Bellman-Ford, Floyd-Warshall	& & & & &
16	Final Mixed Revision + Master Complexity Tables + Test	& & & & &

These priority ratings are study guidance, not official GATE weightage.

Dependency Map

Think of Algorithms like this:

```text
Algorithm Basics
               !“
Complexity Analysis
               !“
Asymptotic Notations
               !“
Loops + Recurrences
               !“
  % % % % % %<% % % % % % % % % % % %
  !“          !“                        !“
Search Sorting    Hashing
               !“
Design Techniques
  % % % % % %<% % % % % % % % %
  !“          !“                   !“
D&C  Greedy      DP
               !“
Graph Algorithms
  % % % % % %<% % % % % % % % % % % %
  !“          !“                           !“
BFS/DFS MST      Shortest Paths
```

Most important prerequisite chain

You should understand:

Complexity !’ Recurrences !’ Divide & Conquer !’ Greedy/DP !’ Graph algorithms

rather than memorizing algorithms independently.

What GATE Often Tests in Algorithms

Expect questions of these broad forms:

1. Find time complexity of code.
2. Compare growth rates.
3. Solve recurrence.
4. Trace sorting algorithms.
5. Number of comparisons/swaps.
6. Determine sorting properties.
7. Binary-search calculations.
8. Hash-table insertion/search.
9. Identify whether greedy works.
10. Fill or analyze a DP table.
11. BFS/DFS traversal.
12. MST cost or MST properties.
13. Shortest-path calculations.
14. Identify which graph algorithm is applicable.
15. Statement-based conceptual traps.

Therefore, **understanding algorithm behavior is more useful than memorizing pseudocode alone**.

CHAPTER 1 — ALGORITHM BASICS, TIME & SPACE COMPLEXITY

GATE Importance: & & & & &

This chapter creates the foundation for almost everything that follows.

1. Why This Topic Matters

Suppose two programs solve exactly the same problem.

Program A takes:

```text
1 second
```

Program B takes:

```text
3 hours
```

Both may be logically correct, but they are clearly not equally useful.

Algorithms therefore involve two major questions:

```text
1. Is the solution correct?
2. Is the solution efficient?
```

GATE heavily tests the second question through **time and space complexity**.

2. What Is an Algorithm?

An **algorithm** is a finite sequence of well-defined steps used to solve a computational problem.

Example:

Problem:

> **Find the largest element in an array.**

Array:

```text
[7, 2, 15, 4, 9]
```

Possible algorithm:

```text
max = first element

for every remaining element:
```

```
    if element > max:
        max = element

return max
```

Execution:

```TEXT
Initial max = 7

2  > 7 ? No
 1 5   >   7   ?   Y e s  !’   m a x   =   1 5
4  > 15? No
9  > 15? No

Answer = 15
```

---

3. Properties of an Algorithm

An algorithm generally has:

1. Input

Zero or more inputs.

Example:

```TEXT
Array A
```

---

2. Output

At least one result.

Example:

```TEXT
Maximum element
```

---

3. Definiteness

Every instruction must be unambiguous.

Bad instruction:

```TEXT
Do something useful with x.
```

Good instruction:

```TEXT
x = x + 1
```

4. Finiteness

The algorithm must terminate after a finite number of steps.

This is not an algorithm that solves the problem:

```text
while true:
    continue
```

5. Effectiveness

Operations should be practically executable elementary steps.

4. Correctness of an Algorithm

Efficiency is meaningless if the algorithm gives a wrong answer.

We usually want:

```text
Correctness + Efficiency
```

An algorithm is **correct** if it produces the required output for every valid input.

Example:

Suppose we want maximum element.

Algorithm:

```text
max = A[0]

for i = 1 to n-1:
    if A[i] > max:
        max = A[i]
```

Why does it work?

After processing the first `k` elements:

> **`max` contains the largest element among those `k` elements.**

Eventually all `n` elements are processed.

Therefore:

> **`max` is the largest element in the entire array.**

This kind of reasoning appears later in topics such as loop invariants and algorithm correctness.

For GATE, you usually need **logical correctness intuition**, not lengthy formal proofs for every algorithm.

5. What Is Input Size?

This is one of the most important ideas.

Complexity is usually expressed as a function of **input size**.

We commonly use:

$$ n $$

to represent input size.

Example:

```text
Array = [5, 7, 2, 9]
```

Then:

$$ n=4 $$

For an array:

```text
 i n p u t   s i z e  "H   n u m b e r   o f   e l e m e n t s
```

But `n` does not always mean number of elements.

Different Meanings of Input Size

Array

$$ n = \text{number of elements} $$

String

$$ n = \text{number of characters} $$

Graph

Usually:

$$ V = \text{number of vertices} $$

$$ E = \text{number of edges} $$

Therefore graph algorithms commonly have complexities like:

$$ O(V+E) $$

rather than simply `O(n)`.

Important GATE Idea: Numerical Value vs Input Length

Suppose the input is integer:

$$ N $$

How many bits are needed to represent it?

Approximately:

$$ \log_2 N $$

Example:

$$ N=1024 $$

Since:

$$ 1024=2^{10} $$

about 11 binary digits are needed to represent 1024 itself.

So input size is:

$$ \Theta(\log N) $$

This distinction can matter in complexity questions.

& þGATE TRAP

Suppose algorithm runs:

```
TEXT
for i = 1 to N:
    ...
```

You may say:

$$ O(N) $$

operations.

That is correct with respect to the **numerical value `N`**.

But if the input itself is an integer represented using:

$$ n=\log N $$

bits, then:

$$ N=2^n $$

and the complexity becomes exponential in input length:

$$ O(2^n) $$

This distinction becomes important in theoretical algorithm analysis.

---

6. What Is Time Complexity?

Time complexity describes:

> **How the number of elementary operations performed by an algorithm grows with the input size.**

Important:

It does **not normally mean actual seconds**.

Suppose:

Algorithm A runs in:

```text
0.001 seconds
```

Algorithm B runs in:

```text
0.01 seconds
```

This depends on:

- CPU
- compiler
- programming language
- implementation
- hardware
- operating system

So theoretical algorithm analysis avoids depending primarily on wall-clock time.

Instead we count operations.

Example 1

```text
x = A[0]
```

Only one operation approximately.

Regardless of array size:

```text
n = 10
n = 100
n = 1,000,000
```

the operation count stays constant.

We call such behavior:

\[O(1) \]

The precise meaning of `O` comes in Chapter 2.

For now understand it as:

> **Constant growth.**

Example 2

```text
for i = 0 to n-1:
    print(A[i])
```

The body runs:

$$ n $$

times.

Total work grows approximately proportionally to `n`.

We describe this as:

$$ O(n) $$

Example 3

```text
for i = 1 to n:
    for j = 1 to n:
        print(i,j)
```

Outer loop:

$$ n $$

Inner loop for every outer iteration:

$$ n $$

Total:

$$ n\times n=n^2 $$

Therefore:

$$ O(n^2) $$

Key Idea

Complexity measures **growth**.

We care about what happens when:

$$ n \rightarrow \text{large} $$

---

7. Why Ignore Constants?

Suppose one algorithm executes:

$$ 3n+10 $$

operations.

Another representation might simply describe it as:

$$ O(n) $$

Why?

Because for very large `n`, linear growth dominates.

Example:

For:

$$ n=1,000,000 $$

we have:

$$ 3n+10=3,000,010 $$

The `+10` becomes insignificant compared with the part depending on `n`.

Similarly:

$$ 5n^2+20n+8 $$

is dominated for large `n` by:

$$ n^2 $$

Hence complexity focuses on **order of growth**.

We will formalize this using Big-O, ˜ and ©next chapter.

---

8. Basic Operation Counting

Consider:

```text
sum = 0

for i = 1 to n:
    sum = sum + A[i]
```

The exact operation count could include:

- initialization
- loop comparison
- increment
- indexing
- addition
- assignment

We could obtain something like:

$$ an+b $$

where `a` and `b` are constants.

For asymptotic analysis:

$$ an+b $$

has linear growth.

Therefore we normally represent it as:

$$ \Theta(n) $$

9. Time Complexity vs Running Time

Do not confuse them.

Running time

Actual execution duration:

```text
2.7 seconds
```

Time complexity

Growth as input increases:

```text
~(n)
~(n log n)
~(n²)
```

Two computers can give different running times but the same theoretical time complexity.

10. Space Complexity

Algorithms require memory as well as time.

Space complexity describes how memory usage grows with input size.

Memory may include:

- variables
- arrays
- objects/data structures
- recursion stack
- temporary storage

Example

```text
sum = 0

for i = 0 to n-1:
    sum += A[i]
```

Ignoring input array itself, extra memory is:

```text
sum
i
```

a constant number of variables.

Therefore auxiliary space:

\[O(1) \]

---

11. Total Space vs Auxiliary Space

Important distinction.

Total Space

Includes:

```text
Input storage + extra storage
```

Auxiliary Space

Only additional memory used by the algorithm beyond the input.

For GATE, questions often care about **auxiliary space**.

Example

Input:

```text
A[n]
```

Algorithm:

```text
create B[n]

copy A into B
```

Input storage:

\[O(n) \]

Extra array:

\[O(n) \]

Auxiliary space:

\[O(n) \]

Another Example

```text
reverse A in-place
```

using only:

```TEXT
i
j
temp
```

Auxiliary space:

\[O(1) \]

This distinction is important when discussing **in-place algorithms** later.

---

12. Recursion and Space Complexity

Many students forget this.

Consider:

```TEXT
fun(n):
    if n == 0:
        return
    fun(n-1)
```

Calls are:

```TEXT
fun(n)
    !“
fun(n-1)
    !“
fun(n-2)
    !“
...
    !“
fun(0)
```

There are approximately:

\[n \]

simultaneously active function calls.

Therefore recursion-stack space is:

\[O(n) \]

Even though we did not explicitly create an array.

& þGATE TRAP

Recursive code does **not automatically mean O(1) space** simply because there is no explicit array.

The call stack consumes memory.

---

13. Best Case, Average Case and Worst Case

This is extremely important.

Consider **linear search**.

Array:

```text
[4, 8, 12, 16, 20]
```

Search for some key.

Best Case

Search for:

```text
4
```

First comparison succeeds.

Number of comparisons:

$$ 1 $$

Therefore:

$$ T_{\text{best}}(n)=\Theta(1) $$

Worst Case

Search for:

```text
20
```

or a value not present.

Potential comparisons:

$$ n $$

Therefore:

$$ T_{\text{worst}}(n)=\Theta(n) $$

Average Case

Depends on probability distribution of inputs.

For example, assume:

- target is definitely present
- each position is equally likely

Comparisons could be:

$$ 1,2,3,\dots,n $$

Average:

$$ \frac{1+2+3+\cdots+n}{n} $$

Using:

$$ 1+2+\cdots+n=\frac{n(n+1)}2 $$

we get:

$$ \frac{n(n+1)}{2n} $$

$$ =\frac{n+1}{2} $$

Therefore:

$$ \Theta(n) $$

So for linear search:

Case	Complexity
Best	~ (1)
Average	~ (n)
Worst	~ (n)

Very Important

Average case is not simply halfway between best and worst.

It depends on a probability model.

---

14. Why GATE Syllabus Says “Worst Case”

Your syllabus explicitly contains:

> **“Asymptotic worst-case time and space complexity.”**

Worst-case analysis is especially useful because it provides an upper guarantee.

If:

$$ T_{\text{worst}}(n)=O(n^2) $$

then under the assumptions of the analysis, no input of size `n` makes the algorithm asymptotically worse than that bound.

---

15. Common Complexity Growth Classes

For now, learn the broad order:

$$ 1 < \log n < n < n\log n < n^2 < n^3 < 2^n < n! $$

Mnemonic:

> **Constant !’ Log !’ Linear !’ Linearithmic !’ Polynomial !’ Exponential !’ Factorial**

A shorter memory ladder:

> **C L L P E F**

```text
C = Constant
L = Logarithmic
L = Linear
P = Polynomial
E = Exponential
F = Factorial
```

We will study growth rates rigorously in Chapter 2.

Growth Example

For:

\[n=100 \]

roughly:

Complexity	Approximate work
1	1
l o g , n	~7
n	100
n l o g , n	~700
n^2	10,000
n^3	1,000,000
2 •	enormous
n!	even more enormous

This explains why algorithm choice matters greatly for large inputs.

---

16. Complexity of Sequential Statements

Suppose:

```text
for i = 1 to n:
    operation A

for j = 1 to n:
```

```
    operation B
```

First loop:

$$ n $$

Second loop:

$$ n $$

Total:

$$ n+n=2n $$

Therefore:

$$ \Theta(n) $$

Important

Sequential loops are **added**, not multiplied.

---

17. Complexity of Nested Statements

```text
for i = 1 to n:
    for j = 1 to n:
        operation
```

Total:

$$ n\times n=n^2 $$

Therefore:

$$ \Theta(n^2) $$

Memory Rule

> **One after another !' ADD**

> **One inside another !' usually MULTIPLY**

But dependent loops require more careful summation, which comes later.

Example

```text
for i = 1 to n:
    work

for i = 1 to n:
    for j = 1 to n:
        work
```

Work:

$$ n+n^2 $$

Dominant term:

$$ n^2 $$

Hence:

$$ \Theta(n^2) $$

---

18. Dominant Term Rule

Suppose:

$$ T(n)=7n^3+4n^2+100n+400 $$

For large `n`:

$$ n^3 $$

dominates all lower-order terms.

Therefore:

$$ T(n)=\Theta(n^3) $$

Shortcut:

> **Keep the fastest-growing term; ignore constant coefficient.**

Examples:

$$ 5n+100 \rightarrow \Theta(n) $$

$$ 8n^2+3n+20 \rightarrow \Theta(n^2) $$

$$ 4n\log n+10n \rightarrow \Theta(n\log n) $$

$$ 1000 \rightarrow \Theta(1) $$

---

19. Important Mathematical Results

You will repeatedly need these.

Sum of first n natural numbers

$$ 1+2+\cdots+n = \frac{n(n+1)}2 $$

Therefore:

$$ \Theta(n^2) $$

Sum of squares

$$ 1^2+2^2+\cdots+n^2 = \frac{n(n+1)(2n+1)}6 $$

Therefore:

$$ \Theta(n^3) $$

Geometric series

$$ 1+2+4+\cdots+2^k = 2^{k+1}-1 $$

This becomes very important in recurrence-tree analysis.

20. A First Look at Logarithmic Complexity

Consider:

```text
while n > 1:
    n = n / 2
```

Suppose initial value is:

$$ n $$

After 1 iteration:

$$ n/2 $$

After 2:

$$ n/2^2 $$

After `k`:

$$ n/2^k $$

Stop around:

$$ \frac{n}{2^k}=1 $$

Therefore:

$$ n=2^k $$

Taking log:

$$ k=\log_2 n $$

Thus:

$$ \Theta(\log n) $$

This idea is essential for binary search.

Memory Trick

Whenever a quantity repeatedly becomes:

```text
n/2
n/4
n/8
n/16
...
```

think:

\[\boxed{\log n} \]

Whenever it repeatedly doubles:

```text
1
2
4
8
16
...
until n
```

also think:

\[\boxed{\log n} \]

---

21. Time-Space Trade-off

Sometimes we can improve speed by using more memory.

Example idea:

Instead of repeatedly searching through a list:

```text
O(n)
```

we might build a hash table requiring extra space and obtain expected:

```text
O(1)
```

lookup.

Thus:

> **More memory may produce faster execution.**

This is called a **time-space trade-off**.

Hashing is a classic example.

---

22. Comparison Table

Term	Meaning
Input size	Size of problem instance
Running time	Actual physical execution time
Time complexity	Growth of number of operations

Term	Meaning
Space complexity	Growth of memory requirement
Auxiliary space	Extra memory excluding input
Best case	Minimum work among size-n inputs
Average case	Expected work under a probability model
Worst case	Maximum work among size-n inputs

---

23. GATE Decision Rules

If you see:

```text
one simple operation
```

Think:

\[O(1) \]

If you see:

```text
process every element once
```

Think:

\[O(n) \]

If you see:

```text
n × n nested operations
```

Think:

\[O(n^2) \]

If input repeatedly halves

Think:

\[O(\log n) \]

If two independent blocks execute sequentially

Think:

> **Add their costs, then retain dominant growth.**

If code uses recursion

Think:

> **Time recurrence + recursion-stack space**

not just visible variables.

---

24. & þGATE TRAPS

Trap 1 — Complexity = seconds

Wrong.

```text
O(n)
```

is not “n seconds”.

It describes growth.

Trap 2 — Two consecutive loops give O(n²)

Wrong.

```text
loop n times

loop n times
```

gives:

\[n+n=2n=\Theta(n) \]

They must generally be **nested** for multiplication.

Trap 3 — Ignore recursion stack

Wrong.

Recursive calls require stack frames.

Trap 4 — Average case is average of best and worst

Wrong.

Average case depends on the probability distribution of inputs.

Trap 5 — O(1) means one operation

Wrong.

Constant complexity can involve:

```text
1 operation
10 operations
1000 operations
```

provided the number does not grow with input size.

Trap 6 — Input value always equals input size

Wrong.

For an integer `N`, its binary representation has size approximately:

$$\Theta(\log N)$$

bits.

Trap 7 — Ignore lower-order terms before understanding the expression

You can asymptotically simplify:

$$n^2+100n$$

to:

$$\Theta(n^2)$$

but this doesn't mean the linear work literally disappears during execution.

It becomes asymptotically dominated.

Trap 8 — Worst case means the algorithm is always that slow

No.

Worst case describes the maximum cost among inputs of a particular size.

25. PYQ-STYLE QUESTION 1

Consider:

```text
for i = 1 to n:
    print(i)

for j = 1 to n:
    print(j)
```

What is the asymptotic running time?

Solution

First loop:

$$ n $$

Second:

$$ n $$

Total:

$$ 2n $$

Hence:

$$ \boxed{\Theta(n)} $$

Examiner trap

Thinking:

```TEXT
n × n
```

because there are two loops.

Multiplication occurs for nesting, not merely because two loops exist.

---

26. PYQ-STYLE QUESTION 2

```TEXT
for i = 1 to n:
    for j = 1 to n:
        x++
```

Number of executions of `x++`?

$$ n\times n=n^2 $$

Answer:

$$ \boxed{n^2} $$

Complexity:

$$ \boxed{\Theta(n^2)} $$

---

27. PYQ-STYLE QUESTION 3

```TEXT
i = n

while i > 1:
    i = i / 2
```

Find complexity.

After `k` iterations:

$$ i=\frac{n}{2^k} $$

Termination:

$$ \frac{n}{2^k}\le1 $$

Thus:

$$ 2^k\ge n $$

$$ k\approx\log_2n $$

Answer:

$$ \boxed{\Theta(\log n)} $$

---

28. PYQ-STYLE QUESTION 4

Suppose:

$$ T(n)=20n^2+100n+500 $$

What is its asymptotic order?

Dominating term:

$$ 20n^2 $$

Ignore constant multiplier:

$$ \boxed{\Theta(n^2)} $$

---

29. PYQ-STYLE QUESTION 5

A recursive function makes:

```text
f(n)
f(n-1)
f(n-2)
...
f(1)
```

with all calls simultaneously active before returning.

What is stack space?

Maximum recursion depth:

$$ n $$

Therefore:

$$ \boxed{\Theta(n)} $$

---

30. QUICK RECALL

Try answering these without looking below.

1. What is an algorithm?
2. What does input size mean?
3. What is time complexity?
4. Is time complexity measured in seconds?
5. Difference between total and auxiliary space?
6. Best case of linear search?
7. Worst case of linear search?
8. Average number of comparisons if every linear-search position is equally likely?
9. Complexity of two consecutive `n` loops?
10. Complexity of two `n` loops nested?
11. What growth usually appears when input is repeatedly halved?
12. What is dominant in `n³ + n² + n`?
13. Does recursion use additional memory?
14. Does O(1) necessarily mean one machine instruction?
15. What is the approximate bit-length of an integer `N`?

Answers

1. Finite, well-defined computational steps that solve a problem.
2. The measure used to quantify the size of a problem instance.
3. Growth in computational work with input size.
4. No.
5. Auxiliary space excludes input storage.
6. $\Theta(1)$.
7. $\Theta(n)$.
8. `(n+1)/2`, therefore $\Theta(n)$.
9. $\Theta(n)$.
10. $\Theta(n^2)$.
11. $\Theta(\log n)$.
12. `n³`.
13. Yes, normally through the call stack.
14. No.
15. $\Theta(\log N)$.

31. PRACTICE SET

Try these before checking solutions.

Easy

Q1 — MCQ

The complexity of:

```text
x = a + b
```

is ordinarily:

A. O(log n) B. O(1) C. O(n) D. O(n²)

Q2 — NAT

How many times does `x++` execute?

```text
for i = 1 to 20:
    x++
```

Q3 — MCQ

Two independent loops each execute `n` times. Their combined complexity is:

A. $\Theta(1)$ B. $\Theta(\log n)$ C. $\Theta(n)$ D. $\Theta(n^2)$

Medium

Q4 — MCQ

```text
for i = 1 to n:
    for j = 1 to n:
        operation
```

Complexity?

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n \log n)$ D. $\Theta(n^2)$

Q5 — NAT

For:

$$T(n)=3n^2+7n+4$$

what power of `n` determines the dominant polynomial growth?

Q6 — MSQ

Which statements are correct?

A. Worst-case complexity describes maximum cost among inputs of size `n`.

B. Average-case complexity must equal `(best + worst)/2`.

C. Recursive algorithms may consume stack memory.

D. $\Theta(1)$ means exactly one processor instruction.

Q7 — MCQ

```TEXT
i = n
while i > 1:
    i = i / 2
```

Complexity is:

A. $\Theta(1)$ B. $\Theta(\log n)$ C. $\Theta(n)$ D. $\Theta(n^2)$

Hard / Tricky

Q8 — MCQ

An algorithm performs exactly:

$$50n^2+10^9n$$

operations.

Its asymptotic growth is:

A. $\Theta(n)$ B. $\Theta(\log n)$ C. $\Theta(n^2)$ D. $\Theta(10^9 n)$

Q9 — MSQ

Suppose an algorithm takes an integer `N` as input and runs one iteration for every integer from `1` through `N`.

Which statements are reasonable?

A. The number of loop iterations is $\Theta(N)$.

B. If input size is measured in bits, it is $\Theta(\log N)$.

C. In terms of bit-length `n`, such an algorithm can require $\Theta(2^n)$ iterations for appropriately sized values.

D. Therefore the algorithm must always be considered polynomial-time in encoded input length.

SOLUTIONS

Q1

$$\boxed{B}$$

One constant amount of work.

Q2

$$\boxed{20}$$

Q3

$$n+n=2n$$

Therefore:

$$\boxed{C}$$

Q4

$$n\times n=n^2$$

Answer:

$$\boxed{D}$$

Q5

Dominant term:

$$3n^2$$

Power:

$$\boxed{2}$$

Q6

Correct:

$$\boxed{A,C}$$

B is wrong because average case requires a probability distribution.

D is wrong because constant complexity can involve any constant amount of work.

Q7

Repeated halving gives:

$$\boxed{B}$$

because:

$$k=\log_2n$$

Q8

Even though:

$$10^9n$$

has a huge coefficient, eventually n^2 grows faster than n.

Therefore:

$$\boxed{C}$$

This is a classic asymptotic-analysis trap.

Q9

Correct:

$$\boxed{A,B,C}$$

If:

$$ n\approx \log_2N $$

then:

$$ N\approx2^n $$

So ˜ (Nn)umerical-value iterations can correspond to approximately:

$$ \Theta(2^n) $$

in terms of encoded bit-length.

Therefore D is false.

Ø=ÜÓTES TO WRITE

Algorithm

Finite sequence of unambiguous steps to solve a problem.

Important qualities:

```text
Input
Output
Definiteness
Finiteness
Effectiveness
Correctness
```

Input Size

Common notation:

$$ n $$

Graphs:

$$ V=\text{vertices},\quad E=\text{edges} $$

Integer `N` requires approximately:

$$ \Theta(\log N) $$

bits.

Time Complexity

Measures growth of number of operations with input size.

Not actual seconds.

Examples:

```TEXT
constant work          !'  ~(1)
scan n elements        !'  ~(n)
n × n nested loops     !'  ~(n²)
repeated halving       !'  ~(log n)
```

Space Complexity

Memory required by algorithm.

Auxiliary space = extra memory excluding input.

Remember recursion stack.

Cases

Linear search:

```TEXT
Best      =  ~(1)
Average   =  ~(n)
Worst     =  ~(n)
```

Average case depends on probability distribution.

Dominant Term

$$ an^k+\text{lower terms} $$

usually has asymptotic growth:

$$ \Theta(n^k) $$

Examples:

$$ 5n+2 \rightarrow \Theta(n) $$

$$ 5n^2+20n+1 \rightarrow \Theta(n^2) $$

Loop Shortcut

```TEXT
Sequential !'  ADD
Nested     !'  MULTIPLY
```

subject to loop dependencies.

Important Sums

$$ 1+2+\cdots+n = \frac{n(n+1)}2 = \Theta(n^2) $$

$$ 1^2+2^2+\cdots+n^2 = \Theta(n^3) $$

$$ 1+2+4+\cdots+2^k = 2^{k+1}-1 $$

Growth Order

$$ 1<\log n<n<n\log n<n^2<n^3<2^n<n! $$

Key Traps

- Complexity "`seconds.
- Consecutive loops don't automatically multiply.
- Recursion requires stack memory.
- Average "``(best + worst)/2`.
- O(1) "`exactly one instruction.
- Numerical value and encoded input size may differ.

Ø>Ýà MEMORY TRICKS

Sequential vs Nested

> **SIDE BY SIDE !' ADD**

> **INSIDE !' MULTIPLY**

Repeated division

> **HALVING !' LOG**

```TEXT
n !' n / 2 !' n / 4 !' n / 8
```

means think:

$$ \log n $$

Space

> **R = Remember Recursion**

Whenever you see recursion, remember its stack space.

Complexity hierarchy

Think:

> **Log beats Linear; Linear beats Square; Exponential explodes.**

\[\log n<n<n^2<2^n \]

&;30-SECOND REVISION

1. Algorithm = finite, definite steps solving a problem.
2. Correctness comes before efficiency.
3. Complexity studies growth with input size.
4. Time complexity is not wall-clock seconds.
5. Auxiliary space excludes the input itself.
6. Recursion uses stack memory.
7. Linear-search best = ˜ (1), worst = ˜ (n) .
8. Sequential loops !’ add costs.
9. Nested independent loops !’ multiply costs.
10. Repeated halving !’ ˜ (l og n).
11. Ignore constants/lower-order terms asymptotically.
12. Growth ladder:

\[1<\log n<n<n\log n<n^2<n^3<2^n<n! \]

' Chapter 1 Complete

Next Chapter:

Chapter 2 — Asymptotic Analysis: Big-O, B i g - Ω, B i g -lĩttle-o, l i t t l e-ω, Growth of Functions

This is one of the most important conceptual chapters in GATE Algorithms. We will learn how statements such as

\[f(n)=O(g(n)) \]

are **actually defined**, how to compare functions, solve GATE inequalities, distinguish **O from ˜**, and avoid the classic mistake that **Big-O means worst case**.

Type `NEXT` to continue.

USER

Next

ASSISTANT

CHAPTER 2 — ASYMPTOTIC ANALYSIS

Big-O, B i g - Ω, B i g -lĩttle-o, l i t t l e-ω, Growth of Functions

GATE Importance: & & & & &

This is one of the highest-value concepts in Algorithms because almost every other topic uses asymptotic notation. Your uploaded course structure explicitly includes these foundations before moving to searching, sorting and design techniques.

---

1. WHY THIS TOPIC MATTERS

Suppose two algorithms take:

$$ T_1(n)=5n+100 $$

and

$$ T_2(n)=n^2+2 $$

For small `n`, the first algorithm may not always look dramatically better.

But for very large `n`:

$$ n^2 $$

grows much faster than:

$$ n $$

Asymptotic analysis lets us describe this growth without caring about:

- processor speed
- programming language
- small constants
- minor lower-order terms

The main notations are:

$$ O,\quad \Omega,\quad \Theta,\quad o,\quad \omega $$

---

2. FIRST INTUITION

Before definitions, remember this:

Notation	Intuition
$O(g(n))$	grows no faster than $g(n)$, up to constants
$\Omega(g(n))$	grows no slower than $g(n)$, up to constants
$\Theta(g(n))$	grows at same asymptotic rate
$o(g(n))$	grows strictly slower
$\omega(g(n))$	grows strictly faster

A useful visual memory:

```TEXT
O          !'  U p p e r   b o u n d
©          !'  L o w e r   b o u n d
```

```
~          !'  T i g h t  b o u n d

o          !'  S t r i c t l y  s m a l l e r  g r o w t h
É          !'  S t r i c t l y  l a r g e r  g r o w t h
```

3. IMPORTANT CORRECTION

A common mistake is:

> **Big-O = worst case B i g = Θbest case B i g =˜average case**

This is **wrong**.

Asymptotic notation and case analysis are different ideas.

You may write:

$$ T_{\text{worst}}(n)=\Theta(n^2) $$

or:

$$ T_{\text{best}}(n)=\Theta(n) $$

or:

$$ T_{\text{average}}(n)=O(n\log n) $$

So:

> **Case tells you which inputs you are analyzing.**

> **Asymptotic notation tells you how the resulting function grows.**

This is a very important GATE trap.

4. BIG-O NOTATION

Definition

We say:

$$ f(n)=O(g(n)) $$

if there exist positive constants:

$$ c>0,\quad n_0 $$

such that:

$$ 0\le f(n)\le c\,g(n) $$

for all:

$$ n\ge n_0 $$

Intuition

After some sufficiently large input size `n`, the function `f(n)` stays below a constant multiple of `g(n)`.

Visual intuition:

```
TEXT
c g(n)
  /
 /
/        f(n)
       /
      /
 - - - - - n € - - - - - - - - - - - - - - -!'   n
```

The exact curves are unimportant.

What matters is:

$$ f(n)\le c g(n) $$

eventually.

---

5. Example of Big-O

Let:

$$ f(n)=3n+2 $$

We want to prove:

$$ 3n+2=O(n) $$

Need:

$$ 3n+2\le cn $$

For:

$$ n\ge1 $$

we know:

$$ 2\le2n $$

Therefore:

$$ 3n+2\le3n+2n=5n $$

Choose:

$$ c=5,\quad n_0=1 $$

Therefore:

$$ \boxed{3n+2=O(n)} $$

---

6. Big-O Is Not Necessarily Tight

If:

$$ f(n)=n $$

then all of these are true:

$$ n=O(n) $$

$$ n=O(n^2) $$

$$ n=O(n^3) $$

$$ n=O(2^n) $$

because these functions eventually grow at least as fast as `n`.

But:

$$ O(n) $$

is the more informative/tighter upper bound among these.

& þGATE TRAP

If an algorithm is:

$$ \Theta(n) $$

then saying it is:

$$ O(n^2) $$

is mathematically valid.

It is simply a loose upper bound.

So a question asking:

> **Which statements are mathematically true?**

may have multiple correct answers.

---

7. BIG-OMEGA NOTATION

We say:

$$ f(n)=\Omega(g(n)) $$

if there exist positive constants:

$$ c>0,\quad n_0 $$

such that:

$$ f(n)\ge c\,g(n) $$

for all:

$$ n\ge n_0 $$

Intuition

`g(n)` provides an asymptotic **lower bound**.

Example:

$$ f(n)=3n+2 $$

Clearly:

$$ 3n+2\ge3n $$

Therefore choose:

$$ c=3 $$

Hence:

$$ \boxed{3n+2=\Omega(n)} $$

---

8. BIG-THETA NOTATION

This is the most useful notation when we know the exact asymptotic growth.

We say:

$$ f(n)=\Theta(g(n)) $$

if there exist positive constants:

$$ c_1,c_2,n_0 $$

such that:

$$ c_1g(n)\le f(n)\le c_2g(n) $$

for all:

$$ n\ge n_0 $$

In other words:

$$ f(n)=O(g(n)) $$

and

$$ f(n)=\Omega(g(n)) $$

Therefore:

$$ \boxed{\Theta = O\cap\Omega} $$

informally speaking.

Example

$$ f(n)=3n+2 $$

We already showed:

$$ 3n+2=O(n) $$

and:

$$ 3n+2=\Omega(n) $$

Hence:

$$ \boxed{3n+2=\Theta(n)} $$

MEMORY TRICK

Think of:

\[\Theta \]

as being trapped between two copies of the same growth rate:

```text
c , g ( n )
     !‘
f(n)
     !“
c • g ( n )
```

Therefore:

> **Theta = tight sandwich**

9. RELATION BETWEEN O, ©AND ˜

If:

\[f(n)=\Theta(g(n)) \]

then definitely:

\[f(n)=O(g(n)) \]

and:

\[f(n)=\Omega(g(n)) \]

But the reverse is only true if both upper and lower bounds use the same `g(n)`.

Example

Suppose:

\[f(n)=n \]

Then:

\[f(n)=O(n^2) \]

and:

\[f(n)=\Omega(1) \]

But this does **not** mean:

\[f(n)=\Theta(n^2) \]

or:

\[\Theta(1) \]

The bounds are not matching.

Correct tight bound:

$$ \Theta(n) $$

---

10. DOMINANT-TERM RULE

Suppose:

$$ f(n)=7n^3+4n^2+100n+9 $$

Highest-order term:

$$ n^3 $$

Therefore:

$$ \boxed{f(n)=\Theta(n^3)} $$

Why?

For large `n`, lower-order terms become insignificant relative to `n³`.

General Rule for Polynomials

For:

$$ a_kn^k+a_{k-1}n^{k-1}+\cdots+a_0 $$

with:

$$ a_k>0 $$

the asymptotic growth is:

$$ \boxed{\Theta(n^k)} $$

---

11. COMMON GROWTH RATES

From slower to faster:

$$ 1 < \log\log n < \log n < (\log n)^k < n^\epsilon < n < n\log n < n^2 < n^3 < c^n < n! < n^n $$

for suitable constants:

$$ k>0,\quad \epsilon>0,\quad c>1 $$

For most GATE questions, remember the simpler ladder:

$$ \boxed{ 1<\log n<n<n\log n<n^2<n^3<2^n<n!<n^n } $$

---

12. MEMORY LADDER

Use:

> **C !’ L !’ L !’ L² !’ P !’ E !’ F**

Meaning:

```text
C  = Constant
L  = Log
L  = Linear
L² = Linear-log
P  = Polynomial
E  = Exponential
F  = Factorial
```

More practically:

```text
1
log n
n
n log n
n²
 2 •
n!
```

---

13. LOGARITHM BASE IN COMPLEXITY

You may see:

$$\log_2 n$$

$$\log_{10} n$$

$$\ln n$$

Asymptotically:

$$\boxed{\log_a n=\Theta(\log_b n)}$$

for constant bases:

$$a,b>1$$

because change of base gives:

$$\log_a n=\frac{\log_b n}{\log_b a}$$

The denominator is constant.

Therefore in Big-O/Theta analysis, we usually write simply:

$$\log n$$

& þGATE TRAP

Base matters for numerical value.

But for asymptotic complexity with fixed bases greater than 1:

$$\log_2 n,\log_{10} n,\ln n$$

all belong to:

$$\Theta(\log n)$$

14. Comparing n and $\log n$

For large `n`:

$$\log n<n$$

Hence:

$$\log n=O(n)$$

But:

$$n\ne O(\log n)$$

Rather:

$$n=\Omega(\log n)$$

and in fact:

$$n=\omega(\log n)$$

which we will define shortly.

15. Comparing $n\log n$ and n^2

Consider ratio:

$$\frac{n\log n}{n^2} = \frac{\log n}{n}$$

As:

$$n\to\infty$$

we get:

$$\frac{\log n}{n}\to0$$

Therefore:

$$n\log n$$

grows strictly slower than:

$$n^2$$

Hence:

$$n\log n=O(n^2)$$

and more precisely:

$$n\log n=o(n^2)$$

16. LIMIT METHOD FOR COMPARING FUNCTIONS

This is very useful.

Consider:

$$L=\lim_{n\to\infty}\frac{f(n)}{g(n)}$$

Three important outcomes:

Case 1

If:

$$ 0<L<\infty $$

then:

$$ \boxed{f(n)=\Theta(g(n))} $$

Example:

$$ f(n)=3n^2+2n $$

$$ g(n)=n^2 $$

Then:

$$ \frac{3n^2+2n}{n^2} = 3+\frac2n $$

Limit:

$$ 3 $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

Case 2

If:

$$ L=0 $$

then `f` grows strictly slower:

$$ \boxed{f(n)=o(g(n))} $$

Example:

$$ \frac{n}{n^2} = \frac1n \to0 $$

Thus:

$$ \boxed{n=o(n^2)} $$

Case 3

If:

$$ L=\infty $$

then `f` grows strictly faster:

$$ \boxed{f(n)=\omega(g(n))} $$

Example:

$$ \frac{n^2}{n}=n\to\infty $$

Thus:

$$ \boxed{n^2=\omega(n)} $$

17. LITTLE-o NOTATION

Little-o means **strict asymptotic upper bound**.

We write:

$$ f(n)=o(g(n)) $$

if `f(n)` grows strictly more slowly than `g(n)`.

One convenient characterization is:

$$ \lim_{n\to\infty}\frac{f(n)}{g(n)}=0 $$

Example

$$ n=o(n^2) $$

because:

$$ \frac{n}{n^2} = \frac1n \to0 $$

Similarly:

$$ \log n=o(n) $$

$$ n=o(n\log n) $$

$$ n^2=o(2^n) $$

Big-O vs little-o

If:

$$ f(n)=o(g(n)) $$

then:

$$ f(n)=O(g(n)) $$

But not every Big-O relation is little-o.

Example:

$$ n=O(n) $$

but:

$$ n\ne o(n) $$

because:

$$ \frac nn=1 $$

not 0.

18. LITTLE-OMEGA

We write:

$$ f(n)=\omega(g(n)) $$

when `f(n)` grows strictly faster than `g(n)`.

Equivalent limit characterization:

$$ \lim_{n\to\infty}\frac{f(n)}{g(n)}=\infty $$

Example:

$$ n^2=\omega(n) $$

because:

$$ \frac{n^2}{n}=n\to\infty $$

---

19. Big-O vs little-ω

If:

$$ f(n)=\omega(g(n)) $$

then:

$$ f(n)=\Omega(g(n)) $$

But:

$$ n=\Omega(n) $$

while:

$$ n\ne\omega(n) $$

because they grow at the same rate.

---

20. MASTER COMPARISON TABLE

Notation	Meaning	Can growth be equal?
$O(g)$	asymptotic upper bound	Yes
$\Omega(g)$	asymptotic lower bound	Yes
$\Theta(g)$	tight bound	Yes, same order
$o(g)$	strictly slower	No
$\omega(g)$	strictly faster	No

---

21. EASY MEMORY

Think mathematically:

```TEXT
 O           "d   a s y m p t o t i c a l l y
 ©           "e   a s y m p t o t i c a l l y
 ˜            =   s a m e   o r d e r
o      < strictly
 É            >   s t r i c t l y
```

Not literal numerical inequalities, but excellent intuition.

22. Important Relationships

If:

\[f=\Theta(g) \]

then:

\[f=O(g) \]

and:

\[f=\Omega(g) \]

If:

\[f=o(g) \]

then:

\[f=O(g) \]

If:

\[f=\omega(g) \]

then:

\[f=\Omega(g) \]

And:

\[f=O(g) \iff g=\Omega(f) \]

Similarly:

\[f=o(g) \iff g=\omega(f) \]

23. TRANSITIVITY

Suppose:

\[f=O(g) \]

and:

\[g=O(h) \]

Then:

\[\boxed{f=O(h)} \]

Likewise for:

$$ \Omega,\Theta,o,\omega $$

under their usual definitions.

24. SYMMETRY

Theta is symmetric:

$$ f=\Theta(g) $$

implies:

$$ g=\Theta(f) $$

But Big-O is not symmetric.

Example:

$$ n=O(n^2) $$

but:

$$ n^2\ne O(n) $$

25. REFLEXIVITY

For a positive asymptotic function:

$$ f=O(f) $$

$$ f=\Omega(f) $$

$$ f=\Theta(f) $$

But:

$$ f\ne o(f) $$

and:

$$ f\ne\omega(f) $$

26. ADDITION RULE

If:

$$ f(n)=\Theta(n^2) $$

and:

$$ g(n)=\Theta(n) $$

then:

$$ f(n)+g(n) $$

is:

$$ \Theta(n^2+n) $$

Hence:

$$ \boxed{\Theta(n^2)} $$

General intuition:

> **When adding positive asymptotic terms, the faster-growing term dominates.**

---

27. MULTIPLICATION RULE

If:

$$ f(n)=\Theta(n) $$

and:

$$ g(n)=\Theta(\log n) $$

then:

$$ f(n)g(n) = \Theta(n\log n) $$

This appears constantly in nested-loop and sorting analysis.

---

28. CONSTANT MULTIPLIERS

For any positive constant `c`:

$$ cf(n)=\Theta(f(n)) $$

Example:

$$ 1000n=\Theta(n) $$

$$ 0.01n^2=\Theta(n^2) $$

---

29. IMPORTANT COMPARISONS TO MEMORIZE

Logarithms vs powers

For every constant:

$$ k>0 $$

and:

$$ \epsilon>0 $$

$$ (\log n)^k=o(n^\epsilon) $$

Meaning even a small positive polynomial power eventually beats any fixed power of logarithm.

Example:

$$ (\log n)^{100}=o(\sqrt n) $$

eventually.

---

30. Polynomial vs exponential

For constant:

$$ k>0,\quad c>1 $$

$$ n^k=o(c^n) $$

So:

$$ n^{100} $$

eventually grows slower than:

$$ 2^n $$

even though for smaller `n` this may not look obvious.

31. Exponential vs factorial

For fixed:

$$ c>1 $$

$$ c^n=o(n!) $$

eventually.

Thus factorial growth is extremely fast.

32. GATE FAVORITE: $(\log n)^{\log n}$

This type of expression can look scary.

Use identities.

Suppose logarithms are base 2:

$$ (\log n)^{\log n} $$

Let:

$$ x=\log n $$

Then expression becomes:

$$ x^x $$

Another useful transformation:

$$ a^{\log_b n} = n^{\log_b a} $$

For example:

$$ 2^{\log_2 n}=n $$

and:

$$ 4^{\log_2 n} = n^{\log_2 4} = n^2 $$

33. LOG IDENTITIES FOR GATE

Memorize:

$$\log(ab)=\log a+\log b$$

$$\log(a^k)=k\log a$$

$$\log_a b = \frac{\log_c b}{\log_c a}$$

$$a^{\log_a n}=n$$

and:

$$a^{\log_b n} = n^{\log_b a}$$

These are frequently useful in growth-comparison questions.

---

34. Example: Compare $2^{\log_2 n}$ and n

Using:

$$2^{\log_2 n}=n$$

Therefore:

$$\boxed{2^{\log_2 n}=\Theta(n)}$$

In fact they are exactly equal.

---

35. Example: Compare $n^{\log n}$ and 2^n

Take base-2 logarithm of both.

For:

$$f(n)=n^{\log n}$$

$$\log f(n) = (\log n)^2$$

For:

$$g(n)=2^n$$

$$\log g(n)=n$$

Since:

$$(\log n)^2=o(n)$$

we obtain:

$$\boxed{ n^{\log n}=o(2^n) }$$

Thus exponential `2^n` wins.

---

36. Example: Compare $2^{\sqrt{\log n}}$ and $(\log n)^{10}$

This is more advanced but representative of tricky ordering questions.

Let:

$$x=\log n$$

Then compare:

$$ 2^{\sqrt{x}} $$

with:

$$ x^{10} $$

Take logs:

First:

$$ \log(2^{\sqrt x})=\Theta(\sqrt x) $$

Second:

$$ \log(x^{10})=\Theta(\log x) $$

Since:

$$ \sqrt x $$

grows faster than:

$$ \log x $$

we get:

$$ 2^{\sqrt{\log n}} $$

grows faster than:

$$ (\log n)^{10} $$

You do not need to memorize this particular result. Learn the transformation technique.

---

37. ASYMPTOTIC ORDERING STRATEGY

When asked to arrange functions:

Step 1

Simplify obvious identities.

Example:

$$ 2^{\log_2n}=n $$

Step 2

Group by family:

```TEXT
logs
polynomials
exponentials
factorials
```

Step 3

Use known hierarchy.

Step 4

If functions are difficult, compare:

$$ \frac{f(n)}{g(n)} $$

or take logarithms.

38. IMPORTANT DISTINCTION: Upper Bound vs Worst Case

Suppose insertion sort has worst-case:

$$ \Theta(n^2) $$

Then we may say:

$$ T_{\text{worst}}(n)=O(n^2) $$

But the `O` does not itself mean worst case.

Likewise its best case is:

$$ T_{\text{best}}(n)=\Theta(n) $$

and therefore:

$$ T_{\text{best}}(n)=O(n) $$

The case was chosen before applying asymptotic notation.

39. SET-BASED INTERPRETATION

Technically:

$$ O(g(n)) $$

represents a set of functions.

So rigorous notation is:

$$ f(n)\in O(g(n)) $$

although computer science commonly writes:

$$ f(n)=O(g(n)) $$

Similarly:

$$ \Theta(g(n)) $$

contains all functions asymptotically bounded above and below by constant multiples of `g`.

This subtle point can help with theoretical questions.

40. GATE TRAPS

& þTrap 1 — Big-O means exact complexity

Wrong.

If:

$$ T(n)=n $$

then:

$$ T(n)=O(n) $$

but also:

$$ T(n)=O(n^2) $$

Tight complexity is:

$$ \Theta(n) $$

& þTrap 2 — Big-O means worst case

Wrong.

Big-O is an upper-bound notation.

Worst case describes which input cost is being considered.

& þTrap 3 — ©means best case

Wrong for the same reason.

& þTrap 4 — ˜is average case

Wrong.

Theta means tight asymptotic bound.

& þTrap 5 — If $f=O(g)$, then $g=O(f)$

Not necessarily.

Example:

$$ n=O(n^2) $$

but:

$$ n^2\ne O(n) $$

& þTrap 6 — If $f=o(g)$, then they may grow equally fast

Wrong.

Little-o requires **strictly slower growth**.

& þTrap 7 — Constant factors matter asymptotically

No.

$$ 1000000n=\Theta(n) $$

& þTrap 8 — Log bases change asymptotic class

Not for fixed bases > 1.

$$ \log_2n=\Theta(\log_{10}n) $$

& þTrap 9 — A large coefficient beats a higher power forever

No.

Compare:

$$ 10^{100}n $$

and:

$$ n^2 $$

Eventually:

$$ n^2 $$

wins.

& þTrap 10 — $2n$ and 2^n are similar

Completely different.

$$ 2n=\Theta(n) $$

while:

$$ 2^n $$

is exponential.

41. EXAM DECISION RULES

If polynomial appears:

Keep highest power.

$$ 7n^4+2n^2+1 \Rightarrow \Theta(n^4) $$

If logarithm bases differ:

Ignore base asymptotically.

If ratio tends to a positive finite constant:

Think:

$$\Theta$$

If ratio tends to zero:

Numerator is:

$$o(\text{denominator})$$

If ratio tends to infinity:

Numerator is:

$$\omega(\text{denominator})$$

If asked “upper bound”:

Think:

$$O$$

If asked “lower bound”:

Think:

$$\Omega$$

If asked “tight asymptotic bound”:

Think:

$$\Theta$$

---

42. PYQ-STYLE QUESTION 1

Let:

$$f(n)=5n^2+3n+1$$

Which is the tight asymptotic bound?

A. ~ (n B). ~ (n log n) C. ~ (n 2 D). ~ (2 •)

Highest power is:

$$n^2$$

Therefore:

$$\boxed{C}$$

---

43. PYQ-STYLE QUESTION 2

Which statements are true for:

$$f(n)=n$$

A.

$$ f(n)=O(n) $$

B.

$$ f(n)=O(n^2) $$

C.

$$ f(n)=\Theta(n) $$

D.

$$ f(n)=o(n) $$

Answers:

$$ \boxed{A,B,C} $$

D is false because:

$$ \frac nn=1 $$

not zero.

---

44. PYQ-STYLE QUESTION 3

Determine relationship between:

$$ f(n)=n\log n $$

and:

$$ g(n)=n^2 $$

Ratio:

$$ \frac{f(n)}{g(n)} = \frac{\log n}{n} \to0 $$

Hence:

$$ \boxed{n\log n=o(n^2)} $$

Therefore it is also true that:

$$ n\log n=O(n^2) $$

---

45. PYQ-STYLE QUESTION 4

Compare:

$$ 2^{\log_2n} $$

and:

$$ n $$

Identity:

$$ 2^{\log_2n}=n $$

Therefore:

$$ \boxed{\Theta(n)} $$

46. PYQ-STYLE QUESTION 5

Which grows faster?

$$ n^{10} $$

or:

$$ 2^n $$

Any fixed-degree polynomial grows slower than an exponential.

Therefore:

$$ \boxed{n^{10}=o(2^n)} $$

47. PYQ-STYLE QUESTION 6 — TRICK

Suppose:

$$ f(n)=\Theta(n) $$

Which are necessarily true?

A.

$$ f(n)=O(n) $$

B.

$$ f(n)=O(n^2) $$

C.

$$ f(n)=\Omega(n) $$

D.

$$ f(n)=o(n) $$

Answers:

$$ \boxed{A,B,C} $$

Why B?

Because:

$$ n=O(n^2) $$

A tight linear function is also upper bounded by a quadratic.

D is false because $\Theta(n)$ means same asymptotic order as `n`, not strictly smaller.

48. QUICK RECALL

Try mentally first.

1. What does Big-O represent?
2. What does Big-Ω represent?
3. What does Big-Θ represent?

4. Does Big-O necessarily mean worst case?
5. Is $n=O(n^2)$?
6. Is $n^2=O(n)$?
7. If $f=\Theta(g)$, is $f=O(g)$?
8. If $f=\Theta(g)$, is $f=\Omega(g)$?
9. Is $n=o(n^2)$?
10. Is $n=o(n)$?
11. Is $n^2=\omega(n)$?
12. Which grows faster: $\log n$ or n?
13. Which grows faster: n^{100} or 2^n eventually?
14. Are $\log_2 n$ and $\log_{10} n$ in the same Θ-class?
15. What does ratio limit `0` imply?

ANSWERS

1. Asymptotic upper bound.
2. Asymptotic lower bound.
3. Tight asymptotic bound.
4. No.
5. Yes.
6. No.
7. Yes.
8. Yes.
9. Yes.
10. No.
11. Yes.
12. `n`.
13. `2^n`.
14. Yes.
15. Numerator is little-o of denominator.

49. PRACTICE SET

Easy

Q1 — MCQ

$$10n+20$$

has tight asymptotic complexity:

A. $\Theta(1)$ B. $\Theta(\log n)$ C. $\Theta(n)$ D. $\Theta(n^2)$

Q2 — MSQ

For:

$$ f(n)=n^2 $$

which are true?

A. $f=O(n^2)$ B. $f=O(n^3)$ C. $f=\Omega(n)$ D. $f=o(n^2)$

Q3 — MCQ

Which notation indicates a tight bound?

A. O B. ©C. ˜D. o

Medium

Q4 — MCQ

Which grows fastest?

A. $\log n$ B. n C. $n\log n$ D. n^2

Q5 — MSQ

Which are true?

A.

$$ \log n=o(n) $$

B.

$$ n=o(n^2) $$

C.

$$ n^2=o(n) $$

D.

$$ 2^n=\omega(n^{10}) $$

Q6 — NAT

If:

$$ f(n)=4n^5+3n^2+7 $$

what is the exponent of `n` in its tight polynomial asymptotic bound?

Q7 — MCQ

$$ 4^{\log_2n} $$

is equal to:

A. n B. n^2 C. $2n$ D. 2^n

Hard / Tricky

Q8 — MSQ

Suppose:

$$ f(n)=\Theta(n\log n) $$

Which must be true?

A.

$$ f(n)=O(n^2) $$

B.

$$ f(n)=\Omega(n) $$

C.

$$ f(n)=o(n^2) $$

D.

$$ f(n)=\Theta(n) $$

Q9 — Arrange in increasing growth order

$$ \log n,\quad n,\quad n\log n,\quad n^2,\quad 2^n $$

SOLUTIONS

Q1

Dominant term:

$$ n $$

Answer:

$$ \boxed{C} $$

Q2

A true.

$$ n^2=O(n^2) $$

B true.

$$ n^2=O(n^3) $$

C true.

$$ n^2=\Omega(n) $$

D false.

$$ \frac{n^2}{n^2}=1 $$

not zero.

Answer:

$$ \boxed{A,B,C} $$

Q3

$$ \boxed{C} $$

Theta gives tight asymptotic bound.

Q4

Among the given choices:

$$ \log n<n<n\log n<n^2 $$

Answer:

$$ \boxed{D} $$

Q5

A true.

B true.

C false.

D true because exponential eventually dominates every fixed-degree polynomial.

Answer:

$$ \boxed{A,B,D} $$

Q6

Highest power:

$$ n^5 $$

Answer:

$$ \boxed{5} $$

Q7

Use:

$$ a^{\log_b n} = n^{\log_ba} $$

Therefore:

$$ 4^{\log_2n} = n^{\log_24} = n^2 $$

Answer:

$$ \boxed{B} $$

Q8

Given:

$$ f(n)=\Theta(n\log n) $$

Since:

$$ n\log n=o(n^2) $$

A is true.

Also:

$$ n\log n=\Omega(n) $$

so B is true.

C is true.

D is false.

Answer:

$$ \boxed{A,B,C} $$

Q9

$$ \boxed{ \log n < n < n\log n < n^2 < 2^n } $$

50. Ø=ÜÓTES TO WRITE

Big-O

Upper asymptotic bound:

$$ f(n)=O(g(n)) $$

if:

$$ f(n)\le cg(n) $$

for sufficiently large `n`.

B i g - ©

Lower asymptotic bound:

$$ f(n)=\Omega(g(n)) $$

if:

$$ f(n)\ge cg(n) $$

eventually.

B i g - ˜

Tight bound:

$$ c_1g(n)\le f(n)\le c_2g(n) $$

Therefore:

$$ \Theta(g)=O(g)\cap\Omega(g) $$

little-o

Strictly slower growth:

$$ f=o(g) $$

often checked using:

$$ \frac fg\to0 $$

l i t t l e - É

Strictly faster growth:

$$ f=\omega(g) $$

often checked using:

$$ \frac fg\to\infty $$

Limit Rule

$$ L=\lim\frac{f(n)}{g(n)} $$

If:

$$ 0<L<\infty $$

then:

$$ f=\Theta(g) $$

If:

$$ L=0 $$

then:

$$ f=o(g) $$

If:

$$ L=\infty $$

then:

$$ f=\omega(g) $$

Growth Order

$$ 1<\log n<n<n\log n<n^2<n^3<2^n<n!<n^n $$

Log Rule

For fixed bases:

$$ \log_an=\Theta(\log_bn) $$

Key Identity

$$ a^{\log_bn}=n^{\log_ba} $$

Special case:

$$ 2^{\log_2n}=n $$

Important Growth Facts

$$ (\log n)^k=o(n^\epsilon) $$

$$ n^k=o(c^n) $$

for:

$$ c>1 $$

and fixed positive constants.

51. MEMORY TRICKS

O

Think:

> **O = Over / Upper**

©

Think:

> **Omega = floor / Lower**

~

Think:

> **Theta = Tight**

Small letters

small-o = strictly smaller

s m a l l É = strictly bigger

Limit trick

```TEXT
r a t i o !’  0                n u m e r a t o r  l o s e s
r a t i o !’  c o n s t a n t  s a m e  g r o w t h
r a t i o !’  "                n u m e r a t o r  w i n s
```

---

52. 30-SECOND REVISION

1. `O` = asymptotic upper bound.
2. `©`= asymptotic lower bound.
3. `~`= tight bound.
4. `o` = strictly slower.
5. `É`= strictly faster.
6. Big-O does **not** mean worst case.
7. Theta does **not** mean average case.
8. \(n=\Theta(n)\) also implies \(O(n^2)\).
9. Fixed logarithm bases differ only by constants.
10. Highest polynomial power dominates.
11. \(\log n<n<n\log n<n^2<2^n<n!\).
12. Ratio !’ 0 means numerator is little-o.
13. Ratio !’ finite nonzero constant means `~` .
14. Ratio !’ " means numerator is `l i t t l e - É` .
15. \(a^{\log_bn}=n^{\log_ba}\).

' Chapter 2 Complete

Next Chapter:

Chapter 3 — Loop Complexity & Mathematical Analysis

We’ll cover:

- simple loops
- nested loops
- dependent loops
- triangular loops
- doubling/halving loops
- `i = i*2`, `i = i*i`

- loops involving `log n`
- sums such as $1+2+\cdots+n$
- tricky `for`/`while` GATE questions
- exact iteration counting
- common complexity traps

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

CHAPTER 3 — LOOP COMPLEXITY & MATHEMATICAL ANALYSIS

GATE Importance: & & & & &

This chapter is extremely important because GATE frequently gives small code fragments and asks:

- how many times a statement executes,
- what the time complexity is,
- whether loops add or multiply,
- whether the complexity is n, $n\log n$, n^2, $\log n$, or something unusual.

The main skill is:

> **Do not guess complexity from the number of loops. Count how the loop variables actually change.**

---

1. BASIC STRATEGY FOR LOOP COMPLEXITY

For any loop, ask:

Step 1

What is the starting value?

Step 2

What is the stopping condition?

Step 3

How does the loop variable change?

Examples:

```text
i = i + 1
```

usually gives linear iterations.

```text
i = i + 2
```

still gives linear iterations.

```text
i = i * 2
```

usually gives logarithmic iterations.

```text
i = i / 2
```

usually gives logarithmic iterations.

```text
i = i * i
```

can give:

$$ \Theta(\log\log n) $$

---

2. SIMPLE LINEAR LOOP

Consider:

```text
for i = 1 to n:
    operation
```

The statement executes:

$$ n $$

times.

Therefore:

$$ \boxed{\Theta(n)} $$

---

3. CONSTANT STEP SIZE

Consider:

```text
for i = 0; i < n; i = i + 2:
    operation
```

Values:

```text
0, 2, 4, 6, ...
```

Number of iterations approximately:

$$ \frac n2 $$

Therefore:

$$ T(n)=\frac n2 $$

Asymptotically:

$$ \boxed{\Theta(n)} $$

Important Rule

For any constant:

$$ c>0 $$

a loop like:

```text
i = i + c
```

usually performs:

$$ \Theta(n) $$

iterations if traversing a range of size n.

Why?

Because:

$$ \frac nc $$

differs from n only by a constant factor.

---

4. LOOP WITH DECREMENT

```text
i = n

while i > 0:
    operation
    i--
```

Values:

```text
n, n-1, n-2, ..., 1
```

Number of iterations:

$$ n $$

Therefore:

$$ \boxed{\Theta(n)} $$

---

5. LOOP RUNNING TO $n/2$

```TEXT
for i = 1 to n/2:
    operation
```

Iterations:

$$ \frac n2 $$

Therefore:

$$ \boxed{\Theta(n)} $$

Do not say:

$$ O(n/2) $$

as the final asymptotic class.

Constants are removed:

$$ \boxed{\Theta(n)} $$

---

6. LOOP RUNNING TO n^2

```TEXT
for i = 1 to n*n:
    operation
```

Iterations:

$$ n^2 $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

A single loop does **not** necessarily mean linear time.

Always inspect its bound.

& þGATE TRAP

```TEXT
for i = 1 to n*n
```

is one loop, but complexity is:

$$ \Theta(n^2) $$

So:

> **Number of loop keywords does not determine complexity.**

---

7. SEQUENTIAL LOOPS

Consider:

```text
for i = 1 to n:
    operation

for j = 1 to n:
    operation
```

First:

$$ n $$

Second:

$$ n $$

Total:

$$ n+n=2n $$

Therefore:

$$ \boxed{\Theta(n)} $$

Memory trick:

> **Side by side !' ADD**

---

8. SEQUENTIAL LOOPS OF DIFFERENT SIZES

```text
for i = 1 to n:
    operation

for j = 1 to n*n:
    operation
```

Work:

$$ n+n^2 $$

Dominant term:

$$ n^2 $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

---

9. SIMPLE NESTED LOOP

```text
for i = 1 to n:
    for j = 1 to n:
        operation
```

Outer loop:

\[n \]

For every outer iteration, inner loop:

\[n \]

Total:

\[n\times n=n^2 \]

Therefore:

\[\boxed{\Theta(n^2)} \]

Memory trick:

> **Inside independent loop !' MULTIPLY**

---

10. THREE NESTED LOOPS

```text
for i = 1 to n:
    for j = 1 to n:
        for k = 1 to n:
            operation
```

Total:

\[n\times n\times n \]

\[=n^3 \]

Therefore:

\[\boxed{\Theta(n^3)} \]

---

11. DIFFERENT NESTED BOUNDS

```text
for i = 1 to n:
    for j = 1 to n*n:
        operation
```

Outer:

\[n \]

Inner:

$$ n^2 $$

Total:

$$ n\cdot n^2 $$

$$ =n^3 $$

Therefore:

$$ \boxed{\Theta(n^3)} $$

---

12. DEPENDENT NESTED LOOPS

Now things become more interesting.

Consider:

```TEXT
for i = 1 to n:
    for j = 1 to i:
        operation
```

Inner-loop iterations depend on `i`.

When:

```TEXT
 i   =   1  !'   1   i t e r a t i o n
 i   =   2  !'   2   i t e r a t i o n s
 i   =   3  !'   3   i t e r a t i o n s
...
 i   =   n  !'   n   i t e r a t i o n s
```

Total work:

$$ 1+2+3+\cdots+n $$

Use:

$$ 1+2+\cdots+n = \frac{n(n+1)}2 $$

Therefore:

$$ T(n)=\frac{n(n+1)}2 $$

Dominant term:

$$ n^2 $$

Hence:

$$ \boxed{\Theta(n^2)} $$

---

13. TRIANGULAR LOOP

This pattern appears frequently:

```text
for i = 1 to n:
    for j = i to n:
        operation
```

Count inner iterations.

For:

```text
 i   =   1  !'   n   i t e r a t i o n s
 i   =   2  !'   n - 1
 i   =   3  !'   n - 2
...
 i   =   n  !'   1
```

Total:

$$ n+(n-1)+(n-2)+\cdots+1 $$

Same sum:

$$ \frac{n(n+1)}2 $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

Important Observation

Both:

```text
j = 1 to i
```

and:

```text
j = i to n
```

often produce triangular sums.

So think:

$$ 1+2+\cdots+n $$

which gives:

$$ \Theta(n^2) $$

---

14. LOOP WITH $j=i^2$

Consider:

```text
for i = 1 to n:
    for j = 1 to i*i:
        operation
```

For each `i`, inner work:

$$ i^2 $$

Total:

$$ 1^2+2^2+3^2+\cdots+n^2 $$

Formula:

$$ \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} $$

Highest term:

$$ \frac{2n^3}{6} $$

Therefore:

$$ \boxed{\Theta(n^3)} $$

---

15. GENERAL SUM RULE

A useful GATE result:

$$ \sum_{i=1}^{n} i^k = \Theta(n^{k+1}) $$

for fixed positive integer k.

Examples:

$$ \sum i = \Theta(n^2) $$

$$ \sum i^2 = \Theta(n^3) $$

$$ \sum i^3 = \Theta(n^4) $$

This shortcut is extremely useful.

---

16. MULTIPLICATIVE LOOP

Consider:

```text
i = 1

while i <= n:
    operation
    i = i * 2
```

Values:

```text
1, 2, 4, 8, 16, ...
```

After k iterations:

$$ i=2^k $$

Stop when:

$$ 2^k>n $$

So:

$$ k\approx\log_2 n $$

Therefore:

$$ \boxed{\Theta(\log n)} $$

---

17. MULTIPLICATION BY 3

```TEXT
i = 1

while i <= n:
    i = i * 3
```

After k iterations:

$$ i=3^k $$

Stop near:

$$ 3^k=n $$

Thus:

$$ k=\log_3n $$

Since log bases differ by constants:

$$ \boxed{\Theta(\log n)} $$

GENERAL RULE

If:

```TEXT
i = i * c
```

where:

$$ c>1 $$

then usually:

$$ \boxed{\Theta(\log n)} $$

---

18. DIVISION LOOP

```text
i = n

while i > 1:
    operation
    i = i / 2
```

Sequence:

```text
n
n/2
n/4
n/8
...
1
```

After \(k\) iterations:

\[\frac{n}{2^k}\approx1 \]

Thus:

\[2^k\approx n \]

\[k\approx\log_2n \]

Therefore:

\[\boxed{\Theta(\log n)} \]

MEMORY TRICK

> **Multiply/divide loop variable by a constant !' LOG**

Examples:

```text
i *= 2
i *= 3
i /= 2
i /= 5
```

usually mean:

\[\Theta(\log n) \]

---

19. NESTED LINEAR + LOG LOOP

Consider:

```text
for i = 1 to n:
    j = 1
```

```
    while j <= n:
        operation
        j = j * 2
```

Outer:

$$ n $$

Inner:

$$ \log n $$

Total:

$$ n\log n $$

Therefore:

$$ \boxed{\Theta(n\log n)} $$

This complexity appears frequently in efficient sorting algorithms.

---

20. TWO LOGARITHMIC NESTED LOOPS

```TEXT
i = 1

while i <= n:
    j = 1

    while j <= n:
        operation
        j *= 2

    i *= 2
```

Outer:

$$ \log n $$

Inner:

$$ \log n $$

Total:

$$ (\log n)^2 $$

Therefore:

$$ \boxed{\Theta((\log n)^2)} $$

---

21. LOOP WITH `i = i * i`

This is a famous tricky pattern.

Consider:

```text
i = 2

while i <= n:
    operation
    i = i * i
```

Values:

```text
2
4
16
256
65536
...
```

Let's derive.

Initially:

$$ i_0=2 $$

After 1 update:

$$ 2^2=2^2 $$

After 2 updates:

$$ (2^2)^2=2^4 $$

After 3:

$$ (2^4)^2=2^8 $$

After k:

$$ i=2^{2^k} $$

Termination occurs near:

$$ 2^{2^k}\ge n $$

Take log base 2:

$$ 2^k\ge\log_2 n $$

Take log again:

$$ k\ge\log_2\log_2 n $$

Therefore:

$$ \boxed{\Theta(\log\log n)} $$

GATE Shortcut

If the loop variable repeatedly **squares itself**, think:

$$ \boxed{\log\log n} $$

provided it starts above 1.

& Important

If:

```
TEXT
i = 1
i = i*i
```

then:

\[1^2=1 \]

forever.

That is an infinite loop.

So starting value matters.

---

22. LOOP WITH `i = i + i`

Consider:

```
TEXT
i = 1

while i <= n:
    i = i + i
```

But:

\[i+i=2i \]

So this is equivalent to:

```
TEXT
i *= 2
```

Therefore:

\[\boxed{\Theta(\log n)} \]

---

23. LOOP WITH `i = 2*i + 1`

```
TEXT
i = 1

while i <= n:
    i = 2*i + 1
```

Values roughly double:

```text
1, 3, 7, 15, 31, ...
```

Closed form behaves like:

$$ 2^k $$

Therefore number of iterations:

$$ \boxed{\Theta(\log n)} $$

The `+1` does not change the asymptotic growth.

---

24. LOOP WITH SQUARE ROOT

Consider:

```text
while n > 2:
    n = sqrt(n)
```

Suppose initial input is N.

After one iteration:

$$ N^{1/2} $$

After two:

$$ N^{1/4} $$

After three:

$$ N^{1/8} $$

After k:

$$ N^{1/2^k} $$

We stop when this becomes a constant.

Taking logarithms eventually gives:

$$ 2^k=\Theta(\log N) $$

Hence:

$$ \boxed{\Theta(\log\log N)} $$

---

25. LOOP WITH INNER BOUND `j < i`

We already saw:

```text
for i = 1 to n:
    for j = 1 to i:
```

gives:

$$ \Theta(n^2) $$

Now consider:

```
TEXT
for i = 1; i <= n; i *= 2:
    for j = 1; j <= i; j++:
        operation
```

This is different.

Outer `i` values:

```
TEXT
1, 2, 4, 8, ..., n
```

Inner work:

```
TEXT
1 + 2 + 4 + 8 + ... + n
```

This is a geometric series.

Sum:

$$ 1+2+4+\cdots+n $$

is approximately:

$$ 2n $$

Therefore:

$$ \boxed{\Theta(n)} $$

& ÞVERY IMPORTANT GATE TRAP

You might incorrectly calculate:

$$ \log n \times n=n\log n $$

But the inner loop does **not** execute n times for every outer iteration.

Its size changes:

$$ 1,2,4,\dots,n $$

Therefore we must **sum**, not blindly multiply:

$$ 1+2+4+\cdots+n=\Theta(n) $$

---

26. GEOMETRIC SERIES

Memorize:

$$ 1+r+r^2+\cdots+r^k = \frac{r^{k+1}-1}{r-1} $$

For $r>1$, the largest term dominates.

Example:

$$ 1+2+4+\cdots+n = \Theta(n) $$

Important shortcut:

> **In an increasing geometric sequence, the last term dominates the sum.**

---

27. REVERSE GEOMETRIC SERIES

Consider:

$$ n+\frac n2+\frac n4+\frac n8+\cdots $$

This equals approximately:

$$ 2n $$

Therefore:

$$ \boxed{\Theta(n)} $$

This pattern appears in recursive algorithm analysis and divide-and-conquer.

---

28. HARMONIC SERIES

Another very important sum:

$$ 1+\frac12+\frac13+\cdots+\frac1n $$

is:

$$ \Theta(\log n) $$

This is called the **harmonic series**.

You need this for some tricky nested-loop questions.

---

29. HARMONIC LOOP PATTERN

Consider:

```text
for i = 1 to n:
    for j = 1; j <= n; j = j + i:
        operation
```

For a fixed `i`, inner loop runs about:

$$ \frac ni $$

times.

Total:

$$ \sum_{i=1}^{n}\frac ni $$

Factor `n`:

$$ n\sum_{i=1}^{n}\frac1i $$

Harmonic sum:

$$ \Theta(\log n) $$

Therefore:

$$ \boxed{\Theta(n\log n)} $$

This is a classic GATE-style pattern.

---

30. ANOTHER TRICKY LOOP

```TEXT
for i = 1 to n:
    j = i

    while j <= n:
        operation
        j += i
```

For each `i`, values are:

```TEXT
i, 2i, 3i, ...
```

until n.

Iterations approximately:

$$ \frac ni $$

Total:

$$ \sum_{i=1}^{n}\frac ni $$

$$ =nH_n $$

Therefore:

$$ \boxed{\Theta(n\log n)} $$

---

31. LOOP WHERE INNER LOOP IS $\log i$

Consider:

```TEXT
for i = 1 to n:
    j = 1

    while j <= i:
        j *= 2
```

For fixed `i`, work:

$$ \Theta(\log i) $$

Total:

$$ \sum_{i=1}^{n}\log i $$

Using:

$$ \log1+\log2+\cdots+\log n $$

$$ =\log(n!) $$

And:

$$ \log(n!)=\Theta(n\log n) $$

Therefore:

$$ \boxed{\Theta(n\log n)} $$

---

32. IMPORTANT RESULT

Using Stirling's approximation:

$$ n! \approx\sqrt{2\pi n}\left(\frac ne\right)^n $$

Taking log:

$$ \log(n!)=\Theta(n\log n) $$

You generally do not need to memorize the full Stirling formula for basic GATE Algorithms, but remember:

$$ \boxed{\log(n!)=\Theta(n\log n)} $$

---

33. LOOP WITH `j *= 2` UP TO `i`

```text
for i = 1 to n:
    for j = 1; j <= i; j *= 2:
        operation
```

For each `i`:

$$ \Theta(\log i) $$

Total:

$$ \sum_{i=1}^{n}\log i $$

Therefore:

$$ \boxed{\Theta(n\log n)} $$

---

34. LOOP WITH `i = i + sqrt(n)`

Consider:

```text
i = 0

while i < n:
    i += sqrt(n)
```

Step size:

$$ \sqrt n $$

Number of steps:

$$ \frac n{\sqrt n} $$

$$ =\sqrt n $$

Therefore:

$$ \boxed{\Theta(\sqrt n)} $$

General idea:

If the loop increases by:

$$ f(n) $$

each time over range n, iterations are roughly:

$$ \frac n{f(n)} $$

---

35. LOOP TO $\sqrt n$

```text
for i = 1 to sqrt(n):
    operation
```

Iterations:

$$ \sqrt n $$

Therefore:

$$ \boxed{\Theta(\sqrt n)} $$

This appears in primality/divisor algorithms.

---

36. NESTED $\sqrt n$ LOOPS

```text
for i = 1 to sqrt(n):
    for j = 1 to sqrt(n):
        operation
```

Total:

$$ \sqrt n\cdot\sqrt n $$

$$ =n $$

Therefore:

$$ \boxed{\Theta(n)} $$

---

37. LOOP WITH `i*i <= n`

```text
for i = 1; i*i <= n; i++:
    operation
```

Condition:

$$ i^2\le n $$

Hence:

$$ i\le\sqrt n $$

Therefore:

$$ \boxed{\Theta(\sqrt n)} $$

This is a frequent pattern.

---

38. LOOP WITH `i*i*i <= n`

Condition:

$$ i^3\le n $$

Thus:

$$ i\le n^{1/3} $$

Therefore:

$$ \boxed{\Theta(n^{1/3})} $$

General rule:

```text
i^k <= n
```

gives:

$$ \boxed{\Theta(n^{1/k})} $$

if `i` increments by one.

---

39. LOG LOOP INSIDE LINEAR LOOP WITH DEPENDENT LIMIT

Consider:

```text
for i = 1 to n:
    j = i

    while j > 1:
        j /= 2
```

For each `i`:

$$ \Theta(\log i) $$

Therefore total:

$$ \sum_{i=1}^n\log i $$

$$ =\Theta(n\log n) $$

Hence:

$$ \boxed{\Theta(n\log n)} $$

---

40. EXACT ITERATION COUNT — LINEAR LOOP

Consider:

```text
for i = 1; i <= n; i += 3
```

Values:

$$ 1,4,7,10,\dots $$

Approximate iterations:

$$ \frac n3 $$

More exactly:

$$ \left\lfloor\frac{n-1}{3}\right\rfloor+1 $$

But asymptotically:

$$ \boxed{\Theta(n)} $$

For GATE NAT questions, exact counts can matter, so do not always immediately discard floors and constants.

---

41. EXACT ITERATION COUNT — DOUBLING

```text
i = 1

while i <= n:
    i *= 2
```

Values:

$$ 1,2,4,\dots,2^k $$

Largest k satisfying:

$$ 2^k\le n $$

is:

$$ k=\lfloor\log_2n\rfloor $$

Number of iterations:

$$ \boxed{\lfloor\log_2n\rfloor+1} $$

Asymptotically:

$$ \Theta(\log n) $$

42. WORST-CASE LOOP WITH BREAK

Consider:

```TEXT
for i = 1 to n:
    if A[i] == key:
        break
```

Best case:

first element matches:

$$ \Theta(1) $$

Worst case:

match occurs last or not at all:

$$ \Theta(n) $$

So control statements such as `break` can make best and worst cases differ.

43. CONSTANT-TIME INNER BODY CAN HIDE EXPENSIVE FUNCTIONS

Suppose:

```TEXT
for i = 1 to n:
    sort(A)
```

You cannot simply say:

$$ O(n) $$

because `sort(A)` itself has a cost.

If sorting `n` elements costs:

$$ \Theta(n\log n) $$

and it is performed n times:

$$ n\cdot n\log n $$

$$ =\boxed{\Theta(n^2\log n)} $$

Always analyze the **body cost**.

44. MATHEMATICAL SUMS TO MEMORIZE

These are extremely useful.

Constant sum

$$ \sum_{i=1}^{n}1=n $$

Natural numbers

$$ \sum_{i=1}^{n}i = \frac{n(n+1)}2 = \Theta(n^2) $$

Squares

$$ \sum_{i=1}^{n}i^2 = \frac{n(n+1)(2n+1)}6 = \Theta(n^3) $$

Cubes

$$ \sum_{i=1}^{n}i^3 = \left(\frac{n(n+1)}2\right)^2 = \Theta(n^4) $$

Powers

$$ \sum_{i=1}^{n}i^k = \Theta(n^{k+1}) $$

Geometric

$$ 1+2+4+\cdots+2^k = 2^{k+1}-1 $$

Harmonic

$$ \sum_{i=1}^{n}\frac1i = \Theta(\log n) $$

Log sum

$$ \sum_{i=1}^{n}\log i = \log(n!) = \Theta(n\log n) $$

45. LOOP-PATTERNMASTER TABLE

Pattern	Typical complexity
i++ up to n	$\Theta(n)$
i += constant	$\Theta(n)$
i *= constant	$\Theta(\log n)$
i /= constant	$\Theta(\log n)$
i = i*i, start >1	$\Theta(\log\log n)$
i <= sqrt(n)	$\Theta(\sqrt n)$
two independent nested n loops	$\Theta(n^2)$
j <= i inside i <= n	$\Theta(n^2)$
inner j *= 2, outer n	$\Theta(n\log n)$

Pattern	Typical complexity
inner work n/i	$\Theta(n\log n)$
geometric inner totals $1+2+4+\cdots+n$	$\Theta(n)$

---

46. GATE DECISION RULES

If loop variable changes by:

```text
+1
+2
+c
```

Think:

$$\Theta(n)$$

over a range of size n.

If loop variable changes by:

```text
*2
*3
/2
/3
```

Think:

$$\Theta(\log n)$$

If variable squares itself:

```text
i = i*i
```

Think:

$$\Theta(\log\log n)$$

If inner limit depends on outer variable:

Do **not** blindly multiply.

Write a summation.

If you see:

$$ 1+2+\cdots+n $$

Think:

$$ \Theta(n^2) $$

If you see:

$$ 1+2+4+\cdots+n $$

Think:

$$ \Theta(n) $$

If you see:

$$ 1+\frac12+\cdots+\frac1n $$

Think:

$$ \Theta(\log n) $$

47. & þGATE TRAPS

Trap 1 — One loop means O(n)

Wrong.

```
TEXT
for i = 1 to n*n
```

gives:

$$ \Theta(n^2) $$

Trap 2 — Two loops mean O(n²)

Wrong.

If sequential:

$$ n+n=\Theta(n) $$

Trap 3 — Every nested loop multiplies maximum bounds

Wrong for dependent loops.

Example:

```
TEXT
for i *= 2
    for j = 1 to i
```

Total:

$$ 1+2+4+\cdots+n = \Theta(n) $$

not:

$$ n\log n $$

Trap 4 — `i += 2` means logarithmic

Wrong.

It is still:

$$ \Theta(n) $$

because growth is additive, not multiplicative.

Trap 5 — `i *= 2` means linear

Wrong.

It reaches `n` after:

$$ \log_2n $$

iterations.

Trap 6 — $\sqrt n$ loop is logarithmic

Wrong.

$$ \sqrt n=n^{1/2} $$

It is polynomially larger than $\log n$.

Trap 7 — `i=i*i` always terminates

Not if `i=1`.

Trap 8 — Ignore loop body complexity

Wrong.

A function call inside a loop may itself be expensive.

Trap 9 — Harmonic sum is constant

Wrong.

$$ H_n=\Theta(\log n) $$

Trap 10 — Geometric sum has number-of-terms complexity

Not necessarily.

$$ 1+2+4+\cdots+n $$

has only $\log n$ terms but total work:

$$ \Theta(n) $$

48. PYQ-STYLE QUESTION 1

Find complexity:

```TEXT
for i = 1; i <= n; i += 5:
    x++
```

Iterations:

$$ \approx\frac n5 $$

Hence:

$$ \boxed{\Theta(n)} $$

49. PYQ-STYLE QUESTION 2

```TEXT
i = 1

while i <= n:
    i *= 4
```

After k iterations:

$$ i=4^k $$

Thus:

$$ 4^k\approx n $$

$$ k=\log_4n $$

Therefore:

$$ \boxed{\Theta(\log n)} $$

50. PYQ-STYLE QUESTION 3

```TEXT
for i = 1 to n:
    for j = 1 to i:
        x++
```

Executions:

$$ 1+2+\cdots+n $$

$$ =\frac{n(n+1)}2 $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

51. PYQ-STYLE QUESTION 4

```text
for i = 1; i <= n; i *= 2:
    for j = 1; j <= i; j++:
        x++
```

Total:

$$ 1+2+4+\cdots+n $$

$$ =\Theta(n) $$

Answer:

$$ \boxed{\Theta(n)} $$

This is a classic trap.

52. PYQ-STYLE QUESTION 5

```text
for i = 1 to n:
    for j = 1; j <= n; j += i:
        x++
```

For each `i`:

$$ \Theta(n/i) $$

Total:

$$ n\left(1+\frac12+\frac13+\cdots+\frac1n\right) $$

Therefore:

$$ \boxed{\Theta(n\log n)} $$

53. PYQ-STYLE QUESTION 6

```text
i = 2

while i <= n:
    i = i*i
```

Sequence:

$$ 2,4,16,256,\dots $$

After k:

$$ 2^{2^k} $$

Therefore:

$$ \boxed{\Theta(\log\log n)} $$

54. QUICK RECALL

Answer mentally.

1. `i++` until `n`?
2. `i += 10` until `n`?
3. `i *= 2` until `n`?
4. `i /= 2` starting from `n`?
5. `i = i*i`, starting at 2?
6. Two sequential `n` loops?
7. Two independent nested `n` loops?
8. $\sum_{i=1}^{n}i$?
9. $\sum i^2$?
10. $1+2+4+\cdots+n$?
11. $1+1/2+\cdots+1/n$?
12. Outer `n`, inner `log n`?
13. Loop until `i*i <= n`?
14. Is `n/100` asymptotically linear?
15. Does a nested loop always imply n^2?

ANSWERS

1. $\Theta(n)$
2. $\Theta(n)$
3. $\Theta(\log n)$
4. $\Theta(\log n)$
5. $\Theta(\log\log n)$
6. $\Theta(n)$
7. $\Theta(n^2)$
8. $\Theta(n^2)$
9. $\Theta(n^3)$
10. $\Theta(n)$
11. $\Theta(\log n)$
12. $\Theta(n\log n)$
13. $\Theta(\sqrt n)$
14. Yes.
15. No.

55. PRACTICE SET

Easy

Q1 — MCQ

```TEXT
for i = 1; i <= n; i += 4:
    x++
```

Complexity:

A. $\Theta(1)$ B. $\Theta(\log n)$ C. $\Theta(n)$ D. $\Theta(n^2)$

Q2 — MCQ

```TEXT
i = 1
while i <= n:
    i *= 2
```

Complexity:

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n \log n)$ D. $\Theta(n^2)$

Q3 — NAT

How many times does the inner statement execute?

```TEXT
for i = 1 to 10:
    for j = 1 to i:
        x++
```

Medium

Q4 — MCQ

```TEXT
for i = 1 to n:
    for j = 1 to i*i:
        x++
```

Complexity:

A. $\Theta(n^2)$ B. $\Theta(n^3)$ C. $\Theta(n!)$ D. $\Theta(2^n)$

Q5 — MCQ

```TEXT
for i = 1; i <= n; i *= 2:
    for j = 1; j <= i; j++:
        x++
```

Complexity:

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n \log n)$ D. $\Theta(n^2)$

Q6 — MSQ

Which loops are logarithmic?

A.

```
TEXT
i = 1
while i <= n:
    i += 2
```

B.

```
TEXT
i = 1
while i <= n:
    i *= 2
```

C.

```
TEXT
i = n
while i > 1:
    i /= 3
```

D.

```
TEXT
i = 1
while i <= sqrt(n):
    i++
```

Q7 — MCQ

```
TEXT
for i = 1 to n:
    for j = 1; j <= n; j += i:
        x++
```

Complexity:

A. $\Theta(n)$ B. $\Theta(n \log n)$ C. $\Theta(n^2)$ D. $\Theta(\log n)$

Hard / Tricky

Q8 — MCQ

```TEXT
i = 2

while i <= n:
    i = i*i
```

Complexity:

A. ~ (l o g n) B. ~ (l o g l o g n) C. ~ (\ (\ s q r t n)) D. ~ (n)

Q9 — MSQ

Which statements are true?

A.

\[\sum_{i=1}^{n}i=\Theta(n^2) \]

B.

\[\sum_{i=1}^{n}i^2=\Theta(n^3) \]

C.

\[\sum_{i=1}^{n}\frac1i=\Theta(n) \]

D.

\[1+2+4+\cdots+n=\Theta(n) \]

SOLUTIONS

Q1

Step is a fixed constant.

Iterations:

\[\approx n/4 \]

Therefore:

\[\boxed{C} \]

Q2

Repeated doubling:

\[\boxed{A} \]

Q3

Total:

\[1+2+\cdots+10 \]

\[=\frac{10\cdot11}{2} \]

$$ \boxed{55} $$

Q4

Inner work for each `i`:

$$ i^2 $$

Total:

$$ \sum_{i=1}^n i^2 = \Theta(n^3) $$

Answer:

$$ \boxed{B} $$

Q5

Total work:

$$ 1+2+4+\cdots+n $$

Therefore:

$$ \boxed{B} $$

Q6

B is logarithmic.

C is logarithmic.

A is linear.

D is:

$$ \Theta(\sqrt n) $$

Answer:

$$ \boxed{B,C} $$

Q7

For fixed `i`, iterations:

$$ n/i $$

Total:

$$ nH_n $$

$$ =\Theta(n\log n) $$

Answer:

$$ \boxed{B} $$

Q8

Repeated squaring:

$$ \boxed{B} $$

Q9

A true.

B true.

C false because harmonic sum:

$$ \Theta(\log n) $$

D true.

Answer:

$$ \boxed{A,B,D} $$

56. Ø=ÜÓTES TO WRITE

Basic Loop Rules

```TEXT
 i   + =   c o n s t a n t            !′   ˜ ( n )
 i   - =   c o n s t a n t            !′   ˜ ( n )

 i   * =   c o n s t a n t            !′   ˜ ( l o g   n )
 i   / =   c o n s t a n t            !′   ˜ ( l o g   n )

 i   =   i * i ,   s t a r t   >   1   !′   ˜ ( l o g   l o g   n )
```

Nested Loops

Independent:

$$ n\times n=\Theta(n^2) $$

Dependent:

```TEXT
for i = 1 to n
    for j = 1 to i
```

$$ 1+2+\cdots+n = \Theta(n^2) $$

Important Sums

$$ \sum_{i=1}^n1=n $$

$$ \sum i=\Theta(n^2) $$

$$ \sum i^2=\Theta(n^3) $$

$$ \sum i^k=\Theta(n^{k+1}) $$

$$ 1+2+4+\cdots+n=\Theta(n) $$

$$ \sum_{i=1}^n\frac1i=\Theta(\log n) $$

$$ \sum_{i=1}^n\log i=\Theta(n\log n) $$

Square-root Rule

```text
i*i <= n
```

implies:

$$ i\le\sqrt n $$

Hence:

$$ \Theta(\sqrt n) $$

Harmonic Pattern

If inner work is:

$$ n/i $$

then total is usually:

$$ n\sum1/i $$

$$ =\boxed{\Theta(n\log n)} $$

---

57. MEMORY TRICKS

Addition vs multiplication

> **Side by side = ADD**

> **Independent inside = MULTIPLY**

> **Dependent inside = SUM**

Loop-update shortcut

> **Add !' Linear**

> **Multiply !’ Log**

> **Square !’ Log-Log**

Sum shortcut

```TEXT
1   +   2   +   . . .   +   n           !’   n ²
1 ²   +   2 ²   +   . . .   +   n ²     !’   n ³
1   +   2   +   4   +   . . .   +   n   !’   n
1   +   1 / 2   +   . . .   +   1 / n   !’   l o g   n
```

A compact mnemonic:

> **Natural adds one power; geometric follows the largest; harmonic becomes log.**

&β30-SECOND REVISION

1. Do not count loops; count iterations.
2. `i++` or `i += constant` !’ \(\Theta(n)\).
3. `i *= c` or `i /= c` !’ \(\Theta(\log n)\).
4. `i = i*i` from a value \(>1\) !’ \(\Theta(\log\log n)\).
5. Sequential costs add.
6. Independent nested-loop costs multiply.
7. Dependent loops require summation.
8. \(1+2+\cdots+n=\Theta(n^2)\).
9. \(\sum i^2=\Theta(n^3)\).
10. \(1+2+4+\cdots+n=\Theta(n)\).
11. Harmonic sum = \(\Theta(\log n)\).
12. Inner cost \(n/i\) over `i=1...n` gives \(\Theta(n\log n)\).
13. `i*i <= n` !’ \(\Theta(\sqrt n)\).
14. Outer \(n\) × inner \(\log n\) !’ \(\Theta(n\log n)\).
15. Never blindly multiply dependent loop bounds.

' Chapter 3 Complete

Next Chapter:

Chapter 4 — Recursive Algorithms, Recurrence Relations, Recursion Trees & Master Theorem

This is another **& & & &TE topic**. We’ll learn how to analyze expressions such as:

\[T(n)=T(n/2)+1 \]

\[T(n)=2T(n/2)+n \]

\[T(n)=T(n-1)+n \]

and how to decide **when Master Theorem works, when it does not, and how GATE traps you with invalid applications**.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

CHAPTER 4 — RECURSIVE ALGORITHMS, RECURRENCES, RECURSION TREES & MASTER THEOREM

GATE Importance: & & & & &

This chapter follows the structure you gave in the uploaded Algorithms master prompt, especially the parts on recursive algorithms, recurrence relations, recursion trees and Master Theorem.

This chapter is important because GATE often gives a recurrence like:

\[T(n)=2T(n/2)+n \]

and asks for the complexity.

The main goal is:

> **Convert recursive code !' recurrence !' solve recurrence !' identify complexity.**

---

1. WHY THIS TOPIC MATTERS

Many important algorithms are recursive:

- Binary Search
- Merge Sort
- Quick Sort
- Divide-and-Conquer algorithms
- Tree traversals

For iterative code, we often count loops.

For recursive code, we usually build a **recurrence relation**.

Example:

TEXT

```
fun(n):
    if n <= 1:
        return
    fun(n/2)
```

The work can be written as:

\[T(n)=T(n/2)+\Theta(1) \]

Then we solve this recurrence.

---

2. WHAT IS RECURSION?

A function is recursive if it calls itself directly or indirectly.

Example:

```text
fun(n):
    if n == 0:
        return
    fun(n-1)
```

Call structure:

```text
fun(n)
    !“
fun(n-1)
    !“
fun(n-2)
    !“
...
    !“
fun(0)
```

Two important parts:

Base case

Stops recursion.

```text
if n == 0:
    return
```

Recursive case

Reduces the problem.

```text
fun(n-1)
```

Without a proper base case, recursion may never terminate.

---

3. WHAT IS A RECURRENCE RELATION?

A recurrence expresses the running time of a problem in terms of smaller instances.

Example:

$$ T(n)=T(n-1)+1 $$

means:

- solve a problem of size $n-1$
- perform constant additional work

Another:

$$ T(n)=2T(n/2)+n $$

means:

- solve 2 subproblems
- each of size $n/2$
- perform n extra work

---

4. HOW TO FORM A RECURRENCE

Whenever you see recursive code, identify:

1. **Number of recursive calls**
2. **Size of each recursive call**
3. **Non-recursive work**

Then write:

$$ T(n) = \text{recursive work} + \text{extra work} $$

---

5. EXAMPLE 1 — ONE CALL OF SIZE $n-1$

```text
fun(n):
    if n == 0:
        return

    operation
    fun(n-1)
```

One recursive call:

$$ T(n-1) $$

Constant extra work:

$$ \Theta(1) $$

Therefore:

$$ \boxed{T(n)=T(n-1)+\Theta(1)} $$

---

6. SOLVING BY EXPANSION

Given:

$$ T(n)=T(n-1)+1 $$

Expand:

$$ T(n-1)=T(n-2)+1 $$

Therefore:

$$ T(n)=T(n-2)+2 $$

Again:

$$ T(n)=T(n-3)+3 $$

After k expansions:

$$ T(n)=T(n-k)+k $$

Stop when:

$$ n-k=0 $$

So:

$$ k=n $$

Thus:

$$ T(n)=T(0)+n $$

Therefore:

$$ \boxed{\Theta(n)} $$

---

7. EXAMPLE 2 — ONE CALL OF SIZE $n-1$, LINEAR EXTRA WORK

```text
fun(n):
    if n <= 1:
        return

    for i = 1 to n:
        operation

    fun(n-1)
```

Extra work per call:

$$ \Theta(n) $$

Recurrence:

$$ \boxed{T(n)=T(n-1)+n} $$

Expand:

$$ T(n)=T(n-2)+(n-1)+n $$

Eventually:

$$ T(n)=1+2+\cdots+n $$

Hence:

$$ T(n)=\frac{n(n+1)}{2} $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

8. MEMORY RULE FOR $T(n)=T(n-1)+f(n)$

Usually expansion produces:

$$ T(n) = f(n)+f(n-1)+\cdots+f(1) $$

Examples:

If

$$ T(n)=T(n-1)+1 $$

then:

$$ \Theta(n) $$

If

$$ T(n)=T(n-1)+n $$

then:

$$ \Theta(n^2) $$

If

$$ T(n)=T(n-1)+n^2 $$

then:

$$ \sum_{i=1}^{n}i^2 = \Theta(n^3) $$

So:

$$ \boxed{\Theta(n^3)} $$

9. EXAMPLE 3 — HALVING

```text
fun(n):
    if n <= 1:
        return

    operation
    fun(n/2)
```

Recurrence:

$$ T(n)=T(n/2)+1 $$

Expand:

$$ T(n)=T(n/4)+2 $$

Then:

$$ T(n)=T(n/8)+3 $$

After k:

$$ T(n)=T(n/2^k)+k $$

Stop when:

$$ n/2^k=1 $$

Thus:

$$ 2^k=n $$

$$ k=\log_2n $$

Therefore:

$$ \boxed{T(n)=\Theta(\log n)} $$

This is essentially the recurrence behind Binary Search.

---

10. EXAMPLE 4 — HALVING + LINEAR WORK

Suppose:

$$ T(n)=T(n/2)+n $$

Expand:

$$ T(n) = n+\frac n2+\frac n4+\frac n8+\cdots $$

This is a geometric series.

$$ n+\frac n2+\frac n4+\cdots<2n $$

Therefore:

$$ \boxed{T(n)=\Theta(n)} $$

---

11. IMPORTANT COMPARISON

Compare:

$$ T_1(n)=T(n/2)+1 $$

and:

$$ T_2(n)=T(n/2)+n $$

Results:

$$ T_1(n)=\Theta(\log n) $$

$$ T_2(n)=\Theta(n) $$

Why?

Because the non-recursive work is very different.

---

12. MULTIPLE RECURSIVE CALLS

Consider:

```text
fun(n):
    if n <= 1:
        return

    fun(n/2)
    fun(n/2)
```

Two recursive calls, each of size \(n/2\).

Therefore:

\[T(n)=2T(n/2)+\Theta(1) \]

This creates a recursion tree.

---

13. RECURSION TREE IDEA

For:

\[T(n)=2T(n/2)+1 \]

Level 0:

```text
                  n
```

Level 1:

```text
          n/2           n/2
```

Level 2:

```text
      n/4    n/4     n/4    n/4
```

Each level doubles the number of nodes.

---

14. DEPTH OF THE TREE

Subproblem size:

\[n,\frac n2,\frac n4,\dots,1 \]

Depth:

\[\log_2n \]

At level \(i\):

Number of nodes:

\[2^i \]

Each node does constant work.

Total work at level i:

$$ 2^i $$

Last level has approximately:

$$ n $$

nodes.

Total:

$$ 1+2+4+\cdots+n $$

$$ =\Theta(n) $$

Therefore:

$$ \boxed{T(n)=\Theta(n)} $$

---

15. CLASSIC RECURRENCE — MERGE SORT

Merge Sort has:

$$ \boxed{T(n)=2T(n/2)+\Theta(n)} $$

Why?

- divide into 2 halves
- recursively sort both
- merge them in linear time

---

16. RECURSION TREE FOR MERGE SORT

At root:

Work:

$$ n $$

Level 1:

Two subproblems of size $n/2$.

Each does:

$$ n/2 $$

Total level work:

$$ 2\times n/2=n $$

Level 2:

4 subproblems of size $n/4$.

Total:

$$ 4\times n/4=n $$

So:

```text
Level 0       n
Level 1       n
Level 2       n
...
```

Number of levels:

$$ \log n $$

Therefore total:

$$ n\log n $$

Hence:

$$ \boxed{\Theta(n\log n)} $$

---

17. GENERAL DIVIDE-AND-CONQUER RECURRENCE

A very common form is:

$$ \boxed{T(n)=aT(n/b)+f(n)} $$

where:

- a = number of recursive subproblems
- n/b = size of each subproblem
- $f(n)$ = work outside recursion

This is exactly the form used by the **Master Theorem**.

---

18. UNDERSTANDING a, b, $f(n)$

Example:

$$ T(n)=4T(n/2)+n $$

Then:

$$ a=4 $$

$$ b=2 $$

$$ f(n)=n $$

Another:

$$ T(n)=3T(n/3)+n^2 $$

Then:

$$ a=3,\quad b=3,\quad f(n)=n^2 $$

---

19. MASTER THEOREM — CORE IDEA

For:

$$ T(n)=aT(n/b)+f(n) $$

calculate:

$$ n^{\log_ba} $$

Then compare:

$$ f(n) $$

with:

$$ n^{\log_ba} $$

The whole theorem is basically:

> **Compare the recursive work with the non-recursive work.**

---

20. MASTER THEOREM — CASE 1

If:

$$ f(n) $$

is polynomially smaller than:

$$ n^{\log_ba} $$

specifically:

$$ f(n)=O(n^{\log_ba-\epsilon}) $$

for some:

$$ \epsilon>0 $$

then:

$$ \boxed{ T(n)=\Theta(n^{\log_ba}) } $$

Meaning:

> **Recursive part dominates.**

---

21. CASE 1 EXAMPLE

$$ T(n)=4T(n/2)+n $$

Here:

$$ a=4,\quad b=2 $$

Calculate:

$$ n^{\log_2 4}=n^2 $$

Compare:

$$ f(n)=n $$

with:

$$ n^2 $$

Clearly:

$$ n $$

is polynomially smaller than:

$$ n^2 $$

Hence Case 1:

$$ \boxed{T(n)=\Theta(n^2)} $$

---

22. MASTER THEOREM — CASE 2

If:

$$ f(n)=\Theta(n^{\log_ba}\log^k n) $$

for suitable $k\ge0$, then:

$$ \boxed{ T(n)=\Theta(n^{\log_ba}\log^{k+1}n) } $$

The most common version for GATE:

If:

$$ f(n)=\Theta(n^{\log_ba}) $$

then:

$$ \boxed{ T(n)=\Theta(n^{\log_ba}\log n) } $$

Meaning:

> **Recursive and non-recursive work are balanced.**

---

23. CASE 2 EXAMPLE — MERGE SORT

$$ T(n)=2T(n/2)+n $$

Here:

$$ a=2,\quad b=2 $$

So:

$$ n^{\log_2 2}=n $$

Also:

$$ f(n)=n $$

They are equal in asymptotic order.

Therefore Case 2:

$$ \boxed{ T(n)=\Theta(n\log n) } $$

---

24. ANOTHER CASE 2 EXAMPLE

$$ T(n)=8T(n/2)+n^3 $$

Calculate:

$$ n^{\log_2 8}=n^3 $$

Since:

$$ f(n)=n^3 $$

same order.

Therefore:

$$ \boxed{ T(n)=\Theta(n^3\log n) } $$

---

25. MASTER THEOREM — CASE 3

If:

$$ f(n) $$

is polynomially larger than:

$$ n^{\log_ba} $$

specifically:

$$ f(n)=\Omega(n^{\log_ba+\epsilon}) $$

and regularity condition holds:

$$ a f(n/b)\le c f(n) $$

for some:

$$ c<1 $$

then:

$$ \boxed{T(n)=\Theta(f(n))} $$

Meaning:

> **Non-recursive work dominates.**

---

26. CASE 3 EXAMPLE

$$ T(n)=2T(n/2)+n^2 $$

Here:

$$ n^{\log_22}=n $$

But:

$$ f(n)=n^2 $$

which is polynomially larger.

Check regularity:

$$ a f(n/b) = 2(n/2)^2 $$

$$ =\frac{n^2}{2} $$

Thus:

$$ a f(n/b) = \frac12f(n) $$

So regularity holds with:

$$ c=1/2<1 $$

Therefore:

$$ \boxed{T(n)=\Theta(n^2)} $$

---

27. MASTER THEOREM MEMORY TRICK

Compare:

$$ f(n) $$

with:

$$ n^{\log_ba} $$

Then think:

Case 1

> **f smaller !' recursion wins**

$$ \Theta(n^{\log_ba}) $$

Case 2

> **f equal !' multiply by log**

$$ \Theta(n^{\log_ba}\log n) $$

Case 3

> **f bigger !' f wins**

$$ \Theta(f(n)) $$

Mnemonic:

> **Small !' Recursive Same !' Log extra Stronger !' f**

---

28. MASTER THEOREM TABLE

Compare $f(n)$ with $n^{\log_b a}$	Result
f polynomially smaller	$\Theta(n^{\log_ba})$
Same order	$\Theta(n^{\log_ba}\log n)$

Compare $f(n)$ with $n^{\log_b a}$	Result
f polynomially larger + regularity	$\Theta(f(n))$

29. EXAMPLE — CASE IDENTIFICATION

Solve:

$$ T(n)=9T(n/3)+n $$

Here:

$$ a=9,\quad b=3 $$

Calculate:

$$ n^{\log_3 9} = n^2 $$

Compare:

$$ f(n)=n $$

with:

$$ n^2 $$

Smaller.

Case 1.

Therefore:

$$ \boxed{\Theta(n^2)} $$

30. EXAMPLE

$$ T(n)=3T(n/3)+n $$

Here:

$$ n^{\log_33}=n $$

And:

$$ f(n)=n $$

Same.

Case 2.

Therefore:

$$ \boxed{\Theta(n\log n)} $$

31. EXAMPLE

$$ T(n)=3T(n/3)+n^2 $$

Here:

$$ n^{\log_33}=n $$

and:

$$ f(n)=n^2 $$

Larger.

Case 3.

Therefore:

$$ \boxed{\Theta(n^2)} $$

---

32. EXAMPLE WITH CONSTANT EXTRA WORK

$$ T(n)=2T(n/2)+1 $$

Here:

$$ n^{\log_22}=n $$

and:

$$ f(n)=1 $$

Since:

$$ 1=O(n^{1-\epsilon}) $$

Case 1.

Therefore:

$$ \boxed{\Theta(n)} $$

---

33. EXAMPLE WITH ONE RECURSIVE CALL

$$ T(n)=T(n/2)+1 $$

Here:

$$ a=1,\quad b=2 $$

Then:

$$ n^{\log_21}=n^0=1 $$

Also:

$$ f(n)=1 $$

same.

Case 2:

$$ \boxed{\Theta(\log n)} $$

---

34. EXAMPLE

$$ T(n)=T(n/2)+n $$

Here:

$$ a=1,\quad b=2 $$

So:

$$ n^{\log_21}=1 $$

And:

$$ f(n)=n $$

larger.

Case 3.

Therefore:

$$ \boxed{\Theta(n)} $$

---

35. MASTER THEOREM WITH LOG TERMS

Consider:

$$ T(n)=2T(n/2)+n\log n $$

We have:

$$ n^{\log_22}=n $$

And:

$$ f(n)=n\log n $$

This is:

$$ \Theta(n^{\log_ba}\log^1 n) $$

Extended Case 2:

$$ T(n) = \Theta(n\log^2n) $$

Hence:

$$ \boxed{\Theta(n\log^2n)} $$

---

36. ANOTHER EXTENDED CASE 2

$$ T(n)=2T(n/2)+n(\log n)^2 $$

Then:

$$ f(n) = \Theta(n^{\log_22}\log^2n) $$

Therefore:

$$ \boxed{ T(n)=\Theta(n\log^3n) } $$

---

37. WHEN MASTER THEOREM DOES NOT APPLY

This is extremely important.

Master Theorem applies to recurrences roughly of the form:

$$ T(n)=aT(n/b)+f(n) $$

with fixed constants:

$$ a\ge1,\quad b>1 $$

It does **not** directly apply to many other forms.

---

38. INVALID MASTER THEOREM EXAMPLE 1

$$ T(n)=T(n-1)+n $$

Why not?

Recursive subproblem is:

$$ n-1 $$

not:

$$ n/b $$

for constant $b>1$.

So Master Theorem does not apply.

Use expansion:

$$ n+(n-1)+\cdots+1 $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

---

39. INVALID EXAMPLE 2

$$ T(n)=T(n/2)+T(n/3)+n $$

There are unequal subproblem sizes:

$$ n/2,\quad n/3 $$

Standard Master Theorem does not directly apply.

Do not force it.

---

40. INVALID EXAMPLE 3

$$ T(n)=2T(n/2)+n\log\log n $$

Depending on the exact version of Master Theorem being used, the standard three-case form may not directly classify some boundary functions.

For GATE, use the theorem only when its conditions clearly hold.

Do not mechanically force every recurrence into a case.

---

41. INVALID EXAMPLE 4

$$T(n)=nT(n/2)+n$$

Here:

$$a=n$$

but Master Theorem assumes a is a constant.

So it does not apply.

---

42. INVALID EXAMPLE 5

$$T(n)=2T(\sqrt n)+n$$

The recursive problem size is:

$$\sqrt n$$

not:

$$n/b$$

for fixed b.

Standard Master Theorem does not directly apply.

---

43. GATE MASTER THEOREM CHECKLIST

Before applying Master Theorem, ask:

Q1

Can I write recurrence as:

$$aT(n/b)+f(n)?$$

Q2

Is a constant?

Q3

Is b constant and $b>1$?

Q4

Are all recursive subproblems of equal size?

Q5

Can I compare $f(n)$ with:

$$n^{\log_ba}$$

cleanly?

If any major condition fails:

> **Do not blindly apply Master Theorem.**

44. RECURSION TREE GENERAL FORM

For:

$$ T(n)=aT(n/b)+f(n) $$

at level i:

Number of subproblems:

$$ a^i $$

Size of each:

$$ \frac{n}{b^i} $$

Work per node:

$$ f(n/b^i) $$

Total work at level i:

$$ \boxed{ a^i f(n/b^i) } $$

Depth:

$$ \log_b n $$

Number of leaves:

$$ a^{\log_bn} $$

Using identity:

$$ a^{\log_bn} = n^{\log_ba} $$

So leaf count:

$$ \boxed{ n^{\log_ba} } $$

This explains why $n^{\log_ba}$ appears in the Master Theorem.

45. WHY $n^{\log_b a}$ MATTERS

Suppose every node keeps dividing until size 1.

Depth:

$$ \log_b n $$

Branching factor:

$$ a $$

Leaves:

$$ a^{\log_bn} $$

Identity:

$$ a^{\log_bn} = n^{\log_ba} $$

Therefore total leaf-level work is often:

$$ \Theta(n^{\log_ba}) $$

That is the recursive contribution against which $f(n)$ is compared.

46. RECURRENCE: $T(n)=T(n-1)+T(n-2)+1$

This resembles Fibonacci recursion.

It does not fit Master Theorem.

The number of calls grows exponentially.

A simple upper bound:

$$ T(n)=O(2^n) $$

A tighter Fibonacci-related growth is:

$$ \Theta(\phi^n) $$

up to standard modeling details, where:

$$ \phi=\frac{1+\sqrt5}{2} $$

For basic GATE purposes, recognize:

> **naive Fibonacci recursion !' exponential time.**

47. RECURSION STACK SPACE

Time complexity and recursion depth are different.

Example:

$$ T(n)=2T(n/2)+n $$

Time:

$$ \Theta(n\log n) $$

But maximum recursion depth:

$$ \Theta(\log n) $$

because along one path:

$$ n\to n/2\to n/4\to\dots\to1 $$

Thus recursion-stack space may be:

$$ \boxed{\Theta(\log n)} $$

assuming constant stack space per call, apart from other storage.

48. ANOTHER SPACE EXAMPLE

For:

$$ T(n)=T(n-1)+1 $$

recursive depth:

$$ n $$

Therefore stack:

$$ \boxed{\Theta(n)} $$

This is why recursive and iterative implementations can differ in auxiliary space.

---

49. IMPORTANT DISTINCTION

For recursive algorithms, separately ask:

Time

How many calls/work in total?

Space

What is the maximum number of calls simultaneously active?

Example:

Binary Search:

Time:

$$ \Theta(\log n) $$

Recursive auxiliary stack:

$$ \Theta(\log n) $$

Iterative Binary Search auxiliary space:

$$ \Theta(1) $$

---

50. COMMON RECURRENCES TO MEMORIZE

Recurrence	Complexity
$T(n)=T(n-1)+1$	$\Theta(n)$
$T(n)=T(n-1)+n$	$\Theta(n^2)$
$T(n)=T(n/2)+1$	$\Theta(\log n)$
$T(n)=T(n/2)+n$	$\Theta(n)$
$T(n)=2T(n/2)+1$	$\Theta(n)$
$T(n)=2T(n/2)+n$	$\Theta(n\log n)$
$T(n)=2T(n/2)+n^2$	$\Theta(n^2)$
$T(n)=4T(n/2)+n$	$\Theta(n^2)$

These are worth recognizing immediately.

---

51. PYQ-STYLE QUESTION 1

Solve:

$$T(n)=T(n-1)+1$$

Expansion:

$$T(n)=T(n-2)+2$$

Eventually:

$$T(n)=T(1)+(n-1)$$

Therefore:

$$\boxed{\Theta(n)}$$

52. PYQ-STYLE QUESTION 2

Solve:

$$T(n)=T(n-1)+n$$

Expansion:

$$n+(n-1)+(n-2)+\cdots+1$$

Therefore:

$$\boxed{\Theta(n^2)}$$

53. PYQ-STYLE QUESTION 3

Solve:

$$T(n)=T(n/2)+1$$

Depth:

$$\log_2n$$

Constant work per level.

Therefore:

$$\boxed{\Theta(\log n)}$$

54. PYQ-STYLE QUESTION 4

Solve:

$$T(n)=2T(n/2)+n$$

Master Theorem:

$$a=2,\quad b=2$$

$$n^{\log_22}=n$$

$$f(n)=n$$

Case 2.

Therefore:

$$ \boxed{\Theta(n\log n)} $$

55. PYQ-STYLE QUESTION 5

Solve:

$$ T(n)=8T(n/2)+n^2 $$

Calculate:

$$ n^{\log_28}=n^3 $$

Compare:

$$ n^2<n^3 $$

Case 1.

Therefore:

$$ \boxed{\Theta(n^3)} $$

56. PYQ-STYLE QUESTION 6

Solve:

$$ T(n)=2T(n/2)+n^3 $$

Calculate:

$$ n^{\log_22}=n $$

But:

$$ f(n)=n^3 $$

Case 3.

Therefore:

$$ \boxed{\Theta(n^3)} $$

57. PYQ-STYLE QUESTION 7 — MASTER THEOREM TRAP

Can Master Theorem solve:

$$ T(n)=T(n-1)+n $$

Answer:

$$ \boxed{\text{No}} $$

because recursive argument is:

$$ n-1 $$

rather than:

$$ n/b $$

Use expansion instead.

Result:

$$ \boxed{\Theta(n^2)} $$

---

58. PYQ-STYLE QUESTION 8

Solve:

$$ T(n)=4T(n/2)+n^2 $$

$$ n^{\log_24}=n^2 $$

Same as:

$$ f(n)=n^2 $$

Case 2.

Therefore:

$$ \boxed{\Theta(n^2\log n)} $$

---

59. QUICK RECALL

Try mentally first.

1. What is a recurrence relation?
2. Recurrence for one call on $n-1$ plus constant work?
3. $T(n)=T(n-1)+1$?
4. $T(n)=T(n-1)+n$?
5. $T(n)=T(n/2)+1$?
6. $T(n)=T(n/2)+n$?
7. General Master-Theorem form?
8. What do a and b mean?
9. What quantity do we compare with $f(n)$?
10. Case 1 meaning?
11. Case 2 meaning?
12. Case 3 meaning?
13. Can Master Theorem solve $T(n)=T(n-1)+n$?
14. Depth of repeated halving recursion?
15. Number of leaves in an a-ary divide-by-b recursion tree?

ANSWERS

1. Running time expressed using smaller problem sizes.
2. $T(n)=T(n-1)+\Theta(1)$.
3. $\Theta(n)$.

4. $\Theta(n^2)$.
5. $\Theta(\log n)$.
6. $\Theta(n)$.
7. $T(n)=aT(n/b)+f(n)$.
8. Number of subproblems and division factor.
9. $n^{\log_ba}$.
10. Recursive work dominates.
11. Balanced !' extra log factor.
12. Non-recursive $f(n)$ dominates, with conditions.
13. No.
14. $\Theta(\log n)$.
15. $n^{\log_ba}$.

---

60. PRACTICE SET

Easy

Q1 — MCQ

$$ T(n)=T(n-1)+1 $$

has complexity:

A. $\Theta(1)$ B. $\Theta(\log n)$ C. $\Theta(n)$ D. $\Theta(n^2)$

Q2 — MCQ

$$ T(n)=T(n/2)+1 $$

has complexity:

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n\log n)$ D. $\Theta(n^2)$

Q3 — MCQ

For:

$$ T(n)=2T(n/2)+n $$

the values are:

A. $a=1,b=2$ B. $a=2,b=2$ C. $a=2,b=n$ D. $a=n,b=2$

Medium

Q4 — MCQ

Solve:

$$ T(n)=4T(n/2)+n $$

A. $\Theta(n)$ B. $\Theta(n\log n)$ C. $\Theta(n^2)$ D. $\Theta(n^2\log n)$

Q5 — MCQ

Solve:

$$T(n)=2T(n/2)+n^2$$

A. $\Theta(n)$ B. $\Theta(n\log n)$ C. $\Theta(n^2)$ D. $\Theta(n^3)$

Q6 — MCQ

Solve:

$$T(n)=4T(n/2)+n^2$$

A. $\Theta(n^2)$ B. $\Theta(n^2\log n)$ C. $\Theta(n^3)$ D. $\Theta(n\log n)$

Q7 — MSQ

For which recurrences can standard Master Theorem be directly considered?

A.

$$T(n)=2T(n/2)+n$$

B.

$$T(n)=T(n-1)+n$$

C.

$$T(n)=4T(n/2)+n^2$$

D.

$$T(n)=T(n/2)+T(n/3)+n$$

Hard / Tricky

Q8 — MCQ

$$T(n)=8T(n/2)+n^3$$

has complexity:

A. $\Theta(n^3)$ B. $\Theta(n^3\log n)$ C. $\Theta(n^4)$ D. $\Theta(n^2)$

Q9 — MSQ

Which are correct?

A. Master Theorem applies to every recursive recurrence.

B. $T(n)=T(n-1)+n$ can be solved by expansion.

C. $T(n)=2T(n/2)+n$ has recursion depth $\Theta(\log n)$.

D. Time complexity and recursion-stack complexity are always equal.

SOLUTIONS

Q1

$$ T(n)=T(n-1)+1 $$

gives:

$$ 1+1+\cdots+1 $$

n times.

Answer:

$$ \boxed{C} $$

Q2

Repeated halving:

$$ \boxed{A} $$

Q3

Two subproblems each of size $n/2$.

Answer:

$$ \boxed{B} $$

Q4

$$ a=4,\quad b=2 $$

$$ n^{\log_24}=n^2 $$

$$ f(n)=n $$

Case 1.

Answer:

$$ \boxed{C} $$

Q5

$$ n^{\log_22}=n $$

$$ f(n)=n^2 $$

Case 3.

Answer:

$$ \boxed{C} $$

Q6

$$ n^{\log_24}=n^2 $$

Same as:

$$ f(n)=n^2 $$

Case 2.

Therefore:

$$ \boxed{\Theta(n^2\log n)} $$

Answer:

$$ \boxed{B} $$

Q7

A fits:

$$ aT(n/b)+f(n) $$

C also fits.

B has $(n-1)$.

D has unequal subproblem sizes.

Answer:

$$ \boxed{A,C} $$

Q8

$$ n^{\log_28} = n^3 $$

And:

$$ f(n)=n^3 $$

Case 2.

Therefore:

$$ \boxed{\Theta(n^3\log n)} $$

Answer:

$$ \boxed{B} $$

Q9

A false.

B true.

C true.

D false.

Example Merge Sort:

Time:

$$ \Theta(n\log n) $$

recursion depth:

$$ \Theta(\log n) $$

Answer:

$$ \boxed{B,C} $$

---

61. Ø=ÜÓTES TO WRITE

Recurrence

Recursive running time expressed using smaller problem sizes.

General Divide-and-Conquer form:

$$ T(n)=aT(n/b)+f(n) $$

where:

```TEXT
a = number of recursive calls
n/b = subproblem size
f(n) = outside-recursion work
```

Expansion Results

$$ T(n)=T(n-1)+1 \Rightarrow \Theta(n) $$

$$ T(n)=T(n-1)+n \Rightarrow \Theta(n^2) $$

$$ T(n)=T(n/2)+1 \Rightarrow \Theta(\log n) $$

$$ T(n)=T(n/2)+n \Rightarrow \Theta(n) $$

Recursion Tree

At level i:

$$ \text{nodes}=a^i $$

$$ \text{subproblem size}=\frac n{b^i} $$

Depth:

$$ \log_bn $$

Leaves:

$$ n^{\log_ba} $$

Master Theorem

For:

$$ T(n)=aT(n/b)+f(n) $$

calculate:

$$ n^{\log_ba} $$

Case 1

If $f(n)$ polynomially smaller:

$$ T(n)=\Theta(n^{\log_ba}) $$

Case 2

If same order:

$$ T(n)=\Theta(n^{\log_ba}\log n) $$

Extended:

$$ f(n)=\Theta(n^{\log_ba}\log^k n) $$

then:

$$ T(n)=\Theta(n^{\log_ba}\log^{k+1}n) $$

Case 3

If $f(n)$ polynomially larger + regularity:

$$ T(n)=\Theta(f(n)) $$

Master Theorem Does Not Directly Apply To

Examples:

$$ T(n)=T(n-1)+n $$

$$ T(n)=T(n/2)+T(n/3)+n $$

$$ T(n)=nT(n/2)+n $$

$$ T(n)=2T(\sqrt n)+n $$

Recursion Space

Repeated $n-1$:

$$ \Theta(n) $$

Repeated halving:

$$ \Theta(\log n) $$

Time and recursion depth need not be equal.

---

62. MEMORY TRICKS

Master Theorem

First calculate:

$$ \boxed{n^{\log_ba}} $$

Then compare with $f(n)$.

Remember:

> **Small !’ recursion wins**

> **Same !’ add a log**

> **Strong !’ f wins**

Expansion

> **Subtract-one recurrence !’ SUM**

Example:

$$ T(n)=T(n-1)+n $$

becomes:

$$ n+(n-1)+\cdots+1 $$

Halving

> **Divide by 2 each call !’ recursion depth LOG**

Space

> **Total calls "`active calls**

Time counts all work.

Stack space follows maximum depth.

& þGATE TRAPS — FINAL LIST

1. Do not apply Master Theorem to every recurrence.
2. $n-1$ is not the same as $n/2$.

3. Unequal recursive sizes do not fit standard Master Theorem.
4. a and b must be constants in the standard theorem.
5. Do not forget regularity condition in Case 3.
6. Case 2 adds a logarithmic factor.
7. Do not confuse recursion depth with total number of recursive calls.
8. Recursive auxiliary space can differ greatly from time complexity.
9. Always include non-recursive work $f(n)$.
10. Calculate $n^{\log_ba}$ before selecting a Master-Theorem case.

&β30-SECOND REVISION

1. Recursive algorithm !’ build recurrence.
2. $T(n)=T(n-1)+1=\Theta(n)$.
3. $T(n)=T(n-1)+n=\Theta(n^2)$.
4. $T(n)=T(n/2)+1=\Theta(\log n)$.
5. $T(n)=T(n/2)+n=\Theta(n)$.
6. Master form: $aT(n/b)+f(n)$.
7. Compute $n^{\log_ba}$.
8. Smaller f !’ recursive term wins.
9. Same f !’ extra $\log n$.
10. Larger f !’ $f(n)$ wins, with conditions.
11. Merge Sort: $2T(n/2)+n=\Theta(n\log n)$.
12. $T(n)=T(n-1)+n$: no Master Theorem.
13. Recursion-tree depth for divide-by-b: $\Theta(\log_b n)$.
14. Leaves: $n^{\log_ba}$.
15. Time complexity "` recursion-stack complexity.

' Chapter 4 Complete

Next Chapter:

Chapter 5 — Searching: Linear Search & Binary Search

We’ll cover:

- Linear Search
- Binary Search
- prerequisites for Binary Search
- iterative vs recursive form
- exact/maximum number of comparisons
- best/worst/average case
- first and last occurrence
- lower/upper-bound style searches
- unsuccessful searches

- binary search on answer space
- GATE numerical questions
- the classic `low`, `high`, `mid` traps

Type `NEXT` to continue.

USER

NEXT

USER

NEXT

ASSISTANT

CHAPTER 5 — SEARCHING: LINEAR SEARCH & BINARY SEARCH

GATE Importance: & & & & &

Your Algorithms syllabus explicitly includes **Searching**, and your uploaded study plan asks for linear search, binary search, correctness conditions, comparison counts, variants, and GATE traps.

The central idea is simple:

> **Linear Search checks one by one. Binary Search repeatedly discards half the search space.**

---

1. WHY SEARCHING MATTERS

Searching asks:

> **Given some data and a target value, can we find whether the target exists, and where?**

Example:

```text
A = [8, 3, 11, 6, 20]
key = 11
```

We want:

```text
index = 2
```

or simply:

```text
FOUND
```

Two fundamental algorithms are:

1. **Linear Search**
2. **Binary Search**

---

2. LINEAR SEARCH

Linear Search examines elements sequentially.

Example:

```text
A = [9, 4, 12, 7, 15]
key = 7
```

Search:

```text
 9    !'   n o
 4    !'   n o
 1 2  !'   n o
 7    !'   y e s
```

So the key is found after 4 comparisons.

---

3. LINEAR SEARCH PSEUDOCODE

```text
LinearSearch(A, n, key):

    for i = 0 to n-1:
        if A[i] == key:
            return i

    return NOT_FOUND
```

---

4. DOES LINEAR SEARCH REQUIRE SORTED DATA?

No.

It works on:

```text
sorted array
unsorted array
```

Example:

```text
[19, 2, 31, 7, 4]
```

Linear Search works normally.

That is one of its biggest advantages.

---

5. LINEAR SEARCH — BEST CASE

Suppose:

```text
A = [10, 20, 30, 40]
key = 10
```

First comparison succeeds.

Comparisons:

$$ 1 $$

Therefore:

$$ \boxed{\Theta(1)} $$

---

6. LINEAR SEARCH — WORST CASE

Suppose:

```text
A = [10, 20, 30, 40]
key = 40
```

or:

```text
key = 99
```

We may inspect every element.

Comparisons:

$$ n $$

Therefore:

$$ \boxed{\Theta(n)} $$

---

7. LINEAR SEARCH — AVERAGE CASE

Assume:

- key definitely exists,
- every array position is equally likely.

Comparisons could be:

$$ 1,2,3,\ldots,n $$

Expected comparisons:

$$ \frac{1+2+\cdots+n}{n} $$

$$ = \frac{n(n+1)/2}{n} $$

$$ = \frac{n+1}{2} $$

Therefore:

$$ \boxed{\Theta(n)} $$

---

8. LINEAR SEARCH COMPLEXITY TABLE

Case	Comparisons	Time
Best	1	$\Theta(1)$
Average	$(n+1)/2$ under equal-position assumption	$\Theta(n)$
Worst	n	$\Theta(n)$

Auxiliary space:

$$ \boxed{\Theta(1)} $$

for iterative Linear Search.

---

9. WHEN IS LINEAR SEARCH USEFUL?

Use it when:

- data is unsorted,
- dataset is small,
- only one/few searches are needed,
- sorting first is not worthwhile,
- sequential access is natural.

---

10. BINARY SEARCH

Binary Search is much faster, but requires an important condition:

> **The search space must possess an ordering/monotonic structure that lets us safely discard half.**

For ordinary array Binary Search:

$$ \boxed{\text{Array must be sorted}} $$

---

11. BINARY SEARCH INTUITION

Suppose:

```text
A = [2, 5, 8, 12, 16, 23, 38]
```

Search:

```text
key = 23
```

Initial:

```text
low = 0
high = 6
```

Middle:

```text
mid = 3
A[mid] = 12
```

Since:

\[23>12 \]

everything left of and including 12 can be discarded.

Remaining:

```text
[16, 23, 38]
```

Now middle is:

```text
23
```

Found.

12. WHY CAN WE DISCARD HALF?

Because the array is sorted.

If:

\[key>A[mid] \]

then every element at indices below `mid` is:

\[\le A[mid]<key \]

Therefore the key cannot be there.

That logical guarantee is what makes Binary Search correct.

13. BINARY SEARCH PSEUDOCODE

```text
BinarySearch(A, n, key):

    low = 0
    high = n - 1

    while low <= high:

        mid = low + (high - low) / 2

        if A[mid] == key:
            return mid

        else if key < A[mid]:
            high = mid - 1

        else:
            low = mid + 1

    return NOT_FOUND
```

14. WHY USE THIS `mid` FORMULA?

Common formula:

$$ mid=\frac{low+high}{2} $$

Mathematically fine.

But in fixed-width integer implementations:

```text
low + high
```

can overflow.

Safer:

$$ \boxed{ mid=low+\frac{high-low}{2} } $$

For GATE algorithmic reasoning, both locate the middle, but knowing the safer implementation is useful.

15. BINARY SEARCH TRACE

Search `31` in:

```text
A = [3, 8, 12, 17, 24, 31, 44, 50]
```

Indices:

```text
0  1   2   3   4   5   6   7
```

Iteration 1

```text
low = 0
high = 7
mid = 3
A[mid] = 17
```

Since:

\[31>17 \]

set:

```text
low = 4
```

Iteration 2

```text
low = 4
high = 7
mid = 5
A[mid] = 31
```

Found.

Comparisons with middle elements:

\[2 \]

---

16. BINARY SEARCH BEST CASE

If key equals the first middle element:

\[1 \]

comparison.

Therefore:

\[\boxed{\Theta(1)} \]

---

17. BINARY SEARCH WORST CASE

Each comparison cuts the remaining search space approximately in half.

Sizes:

\[n \]

\[n/2 \]

$$ n/4 $$

$$ n/8 $$

...

After k reductions:

$$ \frac{n}{2^k}\approx1 $$

Hence:

$$ 2^k\approx n $$

$$ k\approx\log_2n $$

Therefore:

$$ \boxed{\Theta(\log n)} $$

---

18. BINARY SEARCH RECURRENCE

Recursive Binary Search:

$$ T(n)=T(n/2)+\Theta(1) $$

From Chapter 4:

$$ \boxed{\Theta(\log n)} $$

---

19. EXACT WORST-CASE NUMBER OF ITERATIONS

A commonly useful bound is approximately:

$$ \boxed{\lfloor\log_2 n\rfloor+1} $$

middle-element probes for a successful search in the worst case, depending on the precise implementation and input size.

For unsuccessful search, equivalent formulations are often written using:

$$ \lceil\log_2(n+1)\rceil $$

or closely related counts depending on conventions.

For GATE NAT questions:

> **Trace the exact implementation if exact comparison count is requested.**

Do not rely only on a memorized formula without checking what is being counted.

---

20. EXAMPLE WITH $n=8$

Suppose 8 elements.

Search sizes roughly:

```text
8
4
2
1
```

Maximum successful probes can be:

$$ 4 $$

And:

$$ \log_2 8+1=4 $$

---

21. BINARY SEARCH SPACE COMPLEXITY

Iterative Binary Search

Uses:

```text
low
high
mid
```

Constant extra memory.

Therefore:

$$ \boxed{\Theta(1)} $$

Recursive Binary Search

Recursion depth:

$$ \Theta(\log n) $$

Therefore stack space:

$$ \boxed{\Theta(\log n)} $$

---

22. ITERATIVE VS RECURSIVE BINARY SEARCH

Property	Iterative	Recursive
Worst-case time	$\Theta(\log n)$	$\Theta(\log n)$
Auxiliary space	$\Theta(1)$	$\Theta(\log n)$
Uses call stack	No	Yes

For space efficiency:

$$ \boxed{\text{Iterative is better}} $$

23. LINEAR VS BINARY SEARCH

Feature	Linear Search	Binary Search
Sorted data required?	No	Yes for standard array search
Best time	$\Theta(1)$	$\Theta(1)$
Worst time	$\Theta(n)$	$\Theta(\log n)$
Iterative space	$\Theta(1)$	$\Theta(1)$
Main idea	Sequential scan	Halve search interval

24. IMPORTANT TRADE-OFF: SHOULD WE SORT FIRST?

Suppose data is unsorted.

You want to search **once**.

Option A:

Linear Search:

$$\Theta(n)$$

Option B:

Sort first:

$$\Theta(n\log n)$$

then Binary Search:

$$\Theta(\log n)$$

Total:

$$\Theta(n\log n)$$

For only one search:

$$\boxed{\text{Linear Search may be better}}$$

25. MANY SEARCH QUERIES

Suppose you have:

- n elements,
- q searches.

Without sorting:

$$\Theta(qn)$$

Using efficient sort + Binary Search:

$$\Theta(n\log n+q\log n)$$

For many queries, preprocessing by sorting may become worthwhile.

This is a common algorithm-design idea:

> **Spend preprocessing time to make future queries faster.**

---

26. BINARY SEARCH WITH DUPLICATES

Suppose:

```text
A = [2, 4, 4, 4, 7, 9]
```

Search for:

```text
4
```

Ordinary Binary Search may return:

```text
index 1
index 2
or index 3
```

depending on the implementation.

It does **not automatically return first or last occurrence**.

---

27. FIRST OCCURRENCE

To find the first occurrence of `key`:

When:

```text
A[mid] == key
```

do not stop immediately.

Store:

```text
answer = mid
```

Then continue searching left:

```text
high = mid - 1
```

---

28. FIRST OCCURRENCE PSEUDOCODE

```text
ans = -1

while low <= high:

    mid = ...

    if A[mid] == key:
        ans = mid
        high = mid - 1

    else if A[mid] < key:
        low = mid + 1

    else:
        high = mid - 1
```

Time:

$$\boxed{\Theta(\log n)}$$

29. LAST OCCURRENCE

When:

```text
A[mid] == key
```

save:

```text
ans = mid
```

but continue right:

```text
low = mid + 1
```

Again:

$$\boxed{\Theta(\log n)}$$

30. COUNTING OCCURRENCES

If array is sorted:

```text
[1, 2, 2, 2, 2, 5, 8]
```

Find:

```text
first = first occurrence of 2
last  = last occurrence of 2
```

Then:

$$ \boxed{ count=last-first+1 } $$

Both searches cost:

$$ O(\log n) $$

Therefore total:

$$ \boxed{O(\log n)} $$

---

31. LOWER-BOUND IDEA

A very common Binary Search variant asks:

> **Find the first position where value is at least x.**

Formally:

$$ A[i]\ge x $$

This is often called **lower bound**.

Example:

```text
A = [2, 5, 8, 8, 11]
x = 8
```

First element ≥8:

```text
index 2
```

---

32. UPPER-BOUND IDEA

Find first position where:

$$ A[i]>x $$

Example:

```text
A = [2, 5, 8, 8, 11]
x = 8
```

First element >8:

```text
11
```

index:

```text
4
```

These variants are often useful in GATE-style reasoning even when the terms `lower_bound` and `upper_bound` are not explicitly used.

---

33. LOWER VS UPPER BOUND

For sorted array:

```text
[2, 5, 8, 8, 8, 11]
```

For \(x=8\):

```text
 l o w e r   b o u n d  !'   f i r s t   > =   8  !'   i n d e x   2
 u p p e r   b o u n d  !'   f i r s t   >     8  !'   i n d e x   5
```

Number of `8`s:

\[5-2=3 \]

Equivalent to:

\[upperBound-lowerBound \]

---

34. UNSUCCESSFUL BINARY SEARCH

Suppose:

```text
A = [3, 6, 10, 15, 21]
key = 8
```

Binary Search keeps narrowing.

Eventually:

```text
low > high
```

This is the usual termination condition for unsuccessful search.

Time remains:

\[\boxed{\Theta(\log n)} \]

35. WHY `low <= high`?

If using a closed interval:

\[[low,high] \]

then `low == high` means one candidate still exists.

Therefore condition should be:

```text
while low <= high
```

If you use:

```text
while low < high
```

with the same update rules, you may skip the final candidate.

Binary Search has several correct interval conventions, but they must be used consistently.

36. COMMON BINARY SEARCH INTERVAL CONVENTIONS

Two popular styles:

Closed interval

\[[low,high] \]

typically:

```text
high = n - 1
while low <= high
```

Half-open interval

\[[low,high) \]

typically:

```text
high = n
while low < high
```

Both can be correct.

The danger is:

> **Mixing update rules from different conventions.**

37. BINARY SEARCH CORRECTNESS — LOOP INVARIANT INTUITION

For ordinary search:

At the start of every iteration:

> **If the key exists and has not been ruled out, it lies within `[low, high]`.**

When:

\[A[mid]<key \]

we set:

```
TEXT
low = mid + 1
```

because sorted order proves positions `<= mid` cannot contain the key.

Similarly:

\[A[mid]>key \]

allows:

```
TEXT
high = mid - 1
```

Thus every update preserves the invariant.

Eventually:

- key is found, or
- interval becomes empty.

---

38. BINARY SEARCH REQUIRES RANDOM/MIDDLE ACCESS FOR FULL BENEFIT

On arrays, accessing:

```
TEXT
A[mid]
```

is:

\[O(1) \]

So Binary Search is naturally efficient.

On a simple linked list, reaching the middle node itself costs linear time.

Therefore applying textbook Binary Search directly to a linked list does not provide the same practical advantage.

---

39. GATE TRAP — BINARY SEARCH ON UNSORTED ARRAY

Example:

```text
[10, 3, 7, 1, 20]
```

Suppose middle is:

```text
7
```

If key is:

```text
3
```

and you say:

```text
 3   <   7  !'   s e a r c h   l e f t
```

you happen to succeed here.

But there is no general guarantee that smaller elements lie left.

So Binary Search correctness collapses.

Therefore:

\[\boxed{\text{Standard Binary Search requires sorted data}} \]

---

40. SEARCH SPACE DOESN'T HAVE TO BE AN ARRAY

A deeper idea:

Binary Search can be used whenever a predicate changes monotonically.

Example conceptual predicate:

```text
Is capacity x sufficient?
```

Maybe:

```text
 x   =   1  !'   f a l s e
 x   =   2  !'   f a l s e
 x   =   3  !'   f a l s e
 x   =   4  !'   t r u e
 x   =   5  !'   t r u e
 x   =   6  !'   t r u e
```

Pattern:

```text
F F F T T T
```

Binary Search can find:

> **first `true`**

This technique is often called:

Binary Search on Answer / Search Space

It is useful algorithmically, though basic GATE questions more often focus on standard searching.

---

41. MONOTONICITY IS THE KEY

Binary Search works because a decision can eliminate one side.

Examples:

```text
F F F F T T T
```

Find first `T`.

or:

```text
T T T T F F F
```

Find last `T`.

If truth values are random:

```text
F T F T F T
```

ordinary Binary Search cannot safely eliminate half.

---

42. NUMBER OF COMPARISONS — SEARCH TREE VIEW

Binary Search comparisons can be viewed as a decision tree.

Each middle-element comparison sends search to:

```text
left
or
right
```

This is why the height is logarithmic.

A balanced binary decision structure containing \(n\) candidates has depth approximately:

\[\log_2n \]

---

43. WHY LOG BASE 2?

Because each Binary Search step retains roughly:

$$ \frac12 $$

of the elements.

After k steps:

$$ \frac{n}{2^k} $$

So base 2 naturally appears.

Asymptotically:

$$ \log_2n=\Theta(\log n) $$

so we usually write:

$$ \boxed{O(\log n)} $$

---

44. BINARY SEARCH ON 1 BILLION ELEMENTS

Suppose:

$$ n=10^9 $$

Since:

$$ 2^{30}\approx10^9 $$

Binary Search requires only about:

$$ 30 $$

comparisons.

Linear Search worst case:

$$ 1,000,000,000 $$

comparisons.

This illustrates how powerful logarithmic growth is.

---

45. PYQ-STYLE QUESTION 1

A sorted array contains:

$$ 1024 $$

elements.

Approximately how many halvings are required to reduce the search space to one element?

$$ 1024=2^{10} $$

Therefore:

$$ \boxed{10} $$

Depending on whether the question asks halvings, iterations, or element comparisons, an extra initial/final comparison may change an exact numerical answer.

Read wording carefully.

46. PYQ-STYLE QUESTION 2

Worst-case complexity of Binary Search:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n\log n)$

Answer:

$$\boxed{B}$$

47. PYQ-STYLE QUESTION 3

Which condition is fundamentally required for standard Binary Search on an array?

A. Elements must be unique B. Array must be sorted C. Array size must be power of two D. Array must contain positive values

Answer:

$$\boxed{B}$$

Duplicates are allowed.

Size need not be a power of 2.

Values may be negative.

48. PYQ-STYLE QUESTION 4

Consider sorted array:

```text
[1, 4, 4, 4, 6, 9]
```

What can ordinary Binary Search return when searching for `4`?

Potentially any of:

```text
index 1, 2, or 3
```

unless algorithm is specifically modified for first/last occurrence.

49. PYQ-STYLE QUESTION 5

Suppose you need first occurrence of key.

At:

```text
A[mid] == key
```

what should you do?

A. Immediately return B. Search only right C. Save `mid` and continue left D. Restart search

Answer:

$$\boxed{C}$$

50. PYQ-STYLE QUESTION 6

Suppose iterative Binary Search uses:

$$O(1)$$

extra space.

What about recursive Binary Search?

Maximum recursion depth:

$$\log n$$

Therefore:

$$\boxed{O(\log n)}$$

auxiliary stack space.

51. PYQ-STYLE QUESTION 7 — PREPROCESSING

You have an unsorted array of n elements and need only one search.

Compare:

Linear Search:

$$\Theta(n)$$

Sort + Binary Search:

$$\Theta(n\log n)+\Theta(\log n)$$

$$=\Theta(n\log n)$$

So asymptotically for a single query:

$$\boxed{\text{Linear Search is cheaper}}$$

assuming no other reason to sort.

52. PYQ-STYLE QUESTION 8 — MANY QUERIES

Suppose there are q searches.

Linear approach

$$\Theta(qn)$$

Sort once + Binary Search

$$ \Theta(n\log n+q\log n) $$

For sufficiently many queries, sorting first becomes advantageous.

The exact crossover depends on constants and q, but asymptotically this is the key comparison.

---

53. & ÞGATE TRAPS

Trap 1 — Binary Search works on any array

Wrong.

Standard Binary Search needs sorted/monotonic structure.

Trap 2 — Binary Search always returns first duplicate

Wrong.

Ordinary Binary Search may return any matching occurrence.

Trap 3 — Binary Search worst case is $O(n)$

Wrong for an array with constant-time middle access.

$$ \boxed{O(\log n)} $$

Trap 4 — Recursive Binary Search has $O(1)$ space

Wrong.

Call stack:

$$ \boxed{O(\log n)} $$

Trap 5 — Best-case Binary Search is $O(\log n)$

It can succeed on first probe:

$$ \boxed{\Theta(1)} $$

Trap 6 — Sorting then Binary Searching is always better

Wrong.

For one search:

$$ O(n) $$

Linear Search may beat:

$$ O(n\log n) $$

preprocessing.

Trap 7 — Binary Search requires unique values

Wrong.

Duplicates are allowed.

Only special variants are needed for first/last occurrence.

Trap 8 — Array size must be a power of two

Wrong.

Binary Search works for any positive size.

Trap 9 — `low < high` is always correct

Not without considering interval convention and update rules.

Trap 10 — Exact comparisons = simply \(\log_2 n\)

Not always.

Floors, ceilings, success/failure, and what is counted matter.

54. EXAM DECISION RULES

If data is unsorted and only one search:

Think:

> **Linear Search.**

If sorted array:

Think:

> **Binary Search candidate.**

If every iteration halves remaining possibilities:

Think:

\[\log n \]

If asked first occurrence:

On equality:

> **record + move left**

If asked last occurrence:

On equality:

> **record + move right**

If duplicate count in sorted array:

Think:

\[last-first+1 \]

or:

\[upperBound-lowerBound \]

If you see monotonic:

```
TEXT
F F F F T T T
```

Think:

> **Binary Search can locate the boundary.**

---

55. QUICK RECALL

Try without checking answers.

1. Does Linear Search require sorted data?
2. Linear Search best case?
3. Linear Search worst case?
4. Binary Search prerequisite for ordinary arrays?
5. Binary Search best case?
6. Binary Search worst case?
7. Why logarithmic?
8. Iterative Binary Search space?
9. Recursive Binary Search stack space?
10. Does Binary Search require unique elements?
11. Ordinary Binary Search on duplicates returns which occurrence?
12. What to do on equality when finding first occurrence?
13. What to do for last occurrence?
14. How to count duplicate occurrences using first and last positions?

15. Why might sorting first be bad for a single search?

ANSWERS

1. No.
2. $\Theta(1)$.
3. $\Theta(n)$.
4. Sorted/monotonic order.
5. $\Theta(1)$.
6. $\Theta(\log n)$.
7. Search space halves each iteration.
8. $\Theta(1)$.
9. $\Theta(\log n)$.
10. No.
11. Any matching occurrence depending on implementation.
12. Save answer and search left.
13. Save answer and search right.
14. `last - first + 1`.
15. Sorting costs $O(n\log n)$, while one Linear Search costs $O(n)$.

56. PRACTICE SET

Easy

Q1 — MCQ

Worst-case complexity of Linear Search:

A. $\Theta(1)$ B. $\Theta(\log n)$ C. $\Theta(n)$ D. $\Theta(n^2)$

Q2 — MCQ

Worst-case complexity of Binary Search on a sorted array:

A. $\Theta(1)$ B. $\Theta(\log n)$ C. $\Theta(n)$ D. $\Theta(n\log n)$

Q3 — MSQ

Which statements are true?

A. Linear Search can search unsorted arrays.

B. Binary Search requires all values to be unique.

C. Binary Search repeatedly reduces the search interval.

D. Iterative Binary Search uses $O(1)$ auxiliary space.

Medium

Q4 — NAT

A Binary Search begins with 1024 candidate elements.

After exactly 5 perfect halvings, how many candidates remain?

Q5 — MCQ

When finding the **first** occurrence of a duplicated key and `A[mid] == key`, we should:

A. return immediately B. store `mid` and continue left C. store `mid` and continue right D. declare failure

Q6 — MCQ

Recursive Binary Search requires what auxiliary stack space?

A. $\Theta(1)$ B. $\Theta(\log n)$ C. $\Theta(n)$ D. $\Theta(n\log n)$

Q7 — MSQ

Which properties are necessary for ordinary array Binary Search?

A. Sorted order B. Positive integers only C. Ability to compare key with middle element D. Number of elements must equal 2^k

Hard / Tricky

Q8 — MCQ

An unsorted array contains n elements. You need exactly one search.

Which has the smaller asymptotic preprocessing + search cost?

A. Sort in $O(n\log n)$, then Binary Search B. Linear Search C. Both necessarily $\Theta(\log n)$ D. Binary Search directly without sorting

Q9 — MSQ

For ordinary Binary Search:

A. Best case can be $\Theta(1)$.

B. Worst case is $\Theta(\log n)$.

C. On duplicate values it necessarily returns the first occurrence.

D. Standard correctness depends on an ordering/monotonic property.

SOLUTIONS

Q1

Worst case may inspect all n elements.

$$ \boxed{C} $$

Q2

Repeated halving:

$$ \boxed{B} $$

Q3

A true.

B false.

C true.

D true.

Answer:

$$ \boxed{A,C,D} $$

Q4

Each halving divides by 2.

$$ \frac{1024}{2^5} = \frac{1024}{32} = 32 $$

Answer:

$$ \boxed{32} $$

Q5

For first occurrence:

```
TEXT
ans = mid
high = mid - 1
```

Answer:

$$ \boxed{B} $$

Q6

Depth:

$$ \Theta(\log n) $$

Answer:

$$ \boxed{B} $$

Q7

A required.

C required.

B false.

D false.

Answer:

\[\boxed{A,C} \]

Q8

Linear Search:

\[\Theta(n) \]

Sort + search:

\[\Theta(n\log n) \]

For a single query:

\[\boxed{B} \]

Q9

A true.

B true.

C false.

D true.

Answer:

\[\boxed{A,B,D} \]

57. Ø=ÑÓTES TO WRITE

Linear Search

Sequentially inspect elements.

No sorting required.

Complexity:

```TEXT
B e s t        =   ~ ( 1 )
A v e r a g e  =   ~ ( n )
W o r s t      =   ~ ( n )
S p a c e      =   ~ ( 1 )
```

Expected comparisons, if target exists equally likely at any position:

$$ \frac{n+1}{2} $$

Binary Search

Requires sorted/monotonic search space.

Process:

```TEXT
Compare key with middle
 k e y  <  m i d d l e  !'  s e a r c h  l e f t
 k e y  >  m i d d l e  !'  s e a r c h  r i g h t
 e q u a l             !'  f o u n d
```

Search space halves each step.

Complexity:

```TEXT
 B e s t  =  ~ ( 1 )
 W o r s t  =  ~ ( l o g  n )
```

Space

Iterative:

$$ \Theta(1) $$

Recursive:

$$ \Theta(\log n) $$

Recurrence

$$ T(n)=T(n/2)+\Theta(1) $$

Therefore:

$$ \Theta(\log n) $$

First Occurrence

On equality:

```TEXT
ans = mid
high = mid - 1
```

Last Occurrence

On equality:

```text
ans = mid
low = mid + 1
```

Duplicate Count

$$ \boxed{last-first+1} $$

Lower Bound

First position satisfying:

$$ A[i]\ge x $$

Upper Bound

First position satisfying:

$$ A[i]>x $$

58. MEMORY TRICKS

Linear Search

> **LINEAR = Look one-by-one**

Binary Search

> **SORT + SPLIT**

Sorted structure allows repeated splitting.

First occurrence

> **Found? Go LEFT**

Last occurrence

> **Found? Go RIGHT**

Binary Search complexity

> **Half !’ Half !’ Half = LOG**

&β30-SECOND REVISION

1. Linear Search works on unsorted data.
2. Linear Search worst case = \(\Theta(n)\).
3. Binary Search needs sorted/monotonic structure.
4. Binary Search best = \(\Theta(1)\).
5. Binary Search worst = \(\Theta(\log n)\).
6. Halving is why logarithm appears.
7. Iterative Binary Search space = \(\Theta(1)\).
8. Recursive Binary Search stack = \(\Theta(\log n)\).
9. Duplicates are allowed.
10. Ordinary Binary Search need not return first duplicate.
11. First occurrence !’ save match and go left.
12. Last occurrence !’ save match and go right.
13. Count = `last - first + 1`.
14. One search in unsorted data may favor Linear Search over sort + Binary Search.
15. Monotonic `FFFFTTTT` patterns can also be Binary Searched.

' Chapter 5 Complete

Next Chapter:

Chapter 6 — Sorting Fundamentals + Bubble Sort, Selection Sort & Insertion Sort

We’ll cover:

- what sorting really means,
- stable vs unstable,
- in-place vs out-of-place,
- adaptive sorting,
- Bubble Sort,
- Selection Sort,
- Insertion Sort,
- exact comparison/swap behavior,
- best/average/worst complexity,
- identifying algorithms from intermediate array states,
- and the sorting properties GATE frequently tries to confuse.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

CHAPTER 6 — SORTING FUNDAMENTALS + BUBBLE SORT, SELECTION SORT & INSERTION SORT

GATE Importance: & & & & &

Sorting is explicitly part of your Algorithms syllabus, and your uploaded master prompt asks for sorting properties, major algorithms, complexity, stability, in-place behavior, and intermediate-state questions.

The core idea:

> **Sorting algorithms are not judged only by time complexity. GATE also tests stability, number of comparisons/swaps, in-place behavior, adaptiveness, and pass-by-pass behavior.**

---

1. WHAT IS SORTING?

Sorting means rearranging elements according to some order.

Example:

```text
Input:
[7, 2, 9, 1, 5]

Ascending:
[1, 2, 5, 7, 9]

Descending:
[9, 7, 5, 2, 1]
```

Most GATE discussions assume ascending order unless stated otherwise.

---

2. WHY SORTING MATTERS

Sorting helps in:

- faster searching,
- duplicate detection,
- ranking,
- merging data,
- scheduling,
- divide-and-conquer algorithms,
- many optimization problems.

Example:

Unsorted search:

$$ O(n) $$

Sorted array + Binary Search:

$$ O(\log n) $$

though sorting itself has a preprocessing cost.

---

3. IMPORTANT SORTING PROPERTIES

Before individual algorithms, understand these terms.

3.1 Stable Sorting

A sorting algorithm is **stable** if equal-key elements preserve their original relative order.

Suppose:

```text
(5,A) (2,X) (5,B)
```

Sort by number only.

Stable result:

```text
(2,X) (5,A) (5,B)
```

The two `5`s stay in original order:

```text
A before B
```

Unstable sorting may produce:

```text
(2,X) (5,B) (5,A)
```

Why Stability Matters

Suppose student records are already sorted by:

```text
name
```

and then we stably sort by:

```text
marks
```

Students with equal marks retain their previous name ordering.

Stability is useful in multi-key sorting.

MEMORY TRICK

> **Stable = Same-key order Stays**

3.2 In-Place Sorting

An algorithm is called **in-place** when it uses only a small constant amount of extra memory, usually:

\[O(1) \]

apart from recursion stack depending on convention.

Examples commonly treated as in-place:

- Bubble Sort
- Selection Sort
- Insertion Sort

Merge Sort on arrays generally requires:

\[O(n) \]

extra memory.

3.3 Adaptive Sorting

An algorithm is **adaptive** if it performs better when the input is already partially sorted.

Example:

Insertion Sort can be very fast on nearly sorted data.

Bubble Sort with an early-stop flag can also become adaptive.

Selection Sort generally does not significantly benefit from already sorted input.

3.4 Comparison Sort

A comparison-based sorting algorithm determines order through comparisons such as:

```text
A[i] < A[j]
```

Examples:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort

Later we will see a major result:

$$ \boxed{\Omega(n\log n)} $$

is the general lower bound for comparison sorting in the worst case.

3.5 Non-Comparison Sort

Some algorithms use properties of keys rather than only pairwise comparisons.

Examples:

- Counting Sort
- Radix Sort
- Bucket Sort

These can sometimes beat:

$$ O(n\log n) $$

because the comparison-sorting lower bound does not apply to them.

---

4. BUBBLE SORT — BASIC IDEA

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

Example:

```text
[5, 1, 4, 2]
```

Compare:

```text
5 and 1
```

Swap:

```text
[1, 5, 4, 2]
```

Compare:

```text
5 and 4
```

Swap:

```text
[1, 4, 5, 2]
```

Compare:

```text
5 and 2
```

Swap:

```text
[1, 4, 2, 5]
```

After one complete pass:

> **Largest element has moved to the end.**

This is why the algorithm is called Bubble Sort: large elements “bubble” toward the right.

---

5. BUBBLE SORT PASS-BY-PASS

Initial:

```text
[5, 1, 4, 2, 8]
```

Pass 1

```text
 5 , 1  !’   s w a p
[1,5,4,2,8]

 5 , 4  !’   s w a p
[1,4,5,2,8]

 5 , 2  !’   s w a p
[1,4,2,5,8]

 5 , 8  !’   n o   s w a p
[1,4,2,5,8]
```

Largest element `8` is fixed at the end.

Pass 2

```text
 1 , 4  !’   n o
 4 , 2  !’   s w a p
```

```
[1,2,4,5,8]

 4 , 5  !’   n o
```

Now last two positions are fixed.

Pass 3

No swaps required.

Sorted:

```text
[1,2,4,5,8]
```

---

6. BUBBLE SORT PSEUDOCODE

Basic version:

```text
BubbleSort(A, n):

    for i = 0 to n-2:
        for j = 0 to n-i-2:

            if A[j] > A[j+1]:
                swap(A[j], A[j+1])
```

---

7. WHY INNER LOOP SHRINKS

After first pass:

- largest element is correctly placed at last position.

After second pass:

- second-largest is correctly placed.

Therefore no need to revisit already fixed suffix.

Hence:

```text
n-1
n-2
n-3
...
1
```

comparisons.

---

8. BUBBLE SORT COMPARISONS

Worst-case/basic version:

$$ (n-1)+(n-2)+\cdots+1 $$

$$ =\frac{n(n-1)}{2} $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

---

9. BUBBLE SORT SWAPS

Worst case occurs for reverse-sorted input:

```text
[5,4,3,2,1]
```

Every inversion must eventually be removed by a swap.

Maximum swaps:

$$ \boxed{\frac{n(n-1)}{2}} $$

Thus:

$$ \Theta(n^2) $$

---

10. OPTIMIZED BUBBLE SORT

We can add a flag:

```text
swapped = false
```

If a complete pass performs no swap:

> **Array is already sorted.**

Pseudocode:

```text
for i = 0 to n-2:

    swapped = false

    for j = 0 to n-i-2:
        if A[j] > A[j+1]:
            swap
            swapped = true

    if swapped == false:
        break
```

---

11. BUBBLE SORT COMPLEXITY

Best case

Already sorted:

```text
[1,2,3,4,5]
```

With early-exit optimization:

One pass:

$$ n-1 $$

comparisons.

Therefore:

$$ \boxed{\Theta(n)} $$

Without optimization, basic Bubble Sort may still do:

$$ \Theta(n^2) $$

comparisons.

This implementation detail matters.

Average case

$$ \boxed{\Theta(n^2)} $$

Worst case

$$ \boxed{\Theta(n^2)} $$

Auxiliary space

$$ \boxed{\Theta(1)} $$

---

12. IS BUBBLE SORT STABLE?

Yes, in the standard implementation.

Why?

We swap only when:

```text
A[j] > A[j+1]
```

not when equal.

Equal elements therefore do not cross each other.

Hence:

$$ \boxed{\text{Stable}} $$

13. IS BUBBLE SORT IN-PLACE?

Yes.

Only a few variables and temporary swap storage are needed.

$$ \boxed{O(1)} $$

auxiliary space.

14. BUBBLE SORT KEY PROPERTY

After pass k:

> **the k largest elements are in their final positions at the end.**

Memory:

> **Bubble fixes the RIGHT side.**

15. SELECTION SORT — BASIC IDEA

Selection Sort repeatedly finds the minimum element from the unsorted portion and places it at the beginning.

Example:

```text
[64, 25, 12, 22, 11]
```

Find minimum:

```text
11
```

Swap with first element:

```text
[11, 25, 12, 22, 64]
```

Now first position is fixed.

16. SELECTION SORT PASS-BY-PASS

Initial:

```text
[64,25,12,22,11]
```

Pass 1

Minimum from whole array:

```text
11
```

Swap with `64`:

```text
[11,25,12,22,64]
```

Pass 2

Unsorted:

```text
[25,12,22,64]
```

Minimum:

```text
12
```

Swap:

```text
[11,12,25,22,64]
```

Pass 3

Unsorted:

```text
[25,22,64]
```

Minimum:

```text
22
```

Swap:

```text
[11,12,22,25,64]
```

Sorted.

17. SELECTION SORT PSEUDOCODE

```text
SelectionSort(A, n):

    for i = 0 to n-2:

        minIndex = i

        for j = i+1 to n-1:
            if A[j] < A[minIndex]:
                minIndex = j

        swap(A[i], A[minIndex])
```

18. SELECTION SORT COMPARISONS

Pass 1:

$$ n-1 $$

Pass 2:

$$ n-2 $$

...

Total:

$$ (n-1)+(n-2)+\cdots+1 $$

$$ = \frac{n(n-1)}{2} $$

Crucially, this does not depend significantly on initial ordering.

So:

$$ \boxed{\Theta(n^2)} $$

for best, average and worst case comparisons.

19. SELECTION SORT SWAPS

At most one swap per outer pass.

Number of passes:

$$ n-1 $$

Therefore swaps:

$$ \boxed{O(n)} $$

This is much fewer than Bubble Sort's worst-case $O(n^2)$ swaps.

This property can matter when swaps/writes are expensive.

20. SELECTION SORT COMPLEXITY

Case	Time
Best	$\Theta(n^2)$
Average	$\Theta(n^2)$
Worst	$\Theta(n^2)$

Auxiliary space:

$$ \boxed{\Theta(1)} $$

21. IS SELECTION SORT STABLE?

Standard Selection Sort is:

$$ \boxed{\text{Not stable}} $$

Example:

```
TEXT
[4A, 4B, 2]
```

Select minimum `2`.

Swap it with first element `4A`:

```
TEXT
[2, 4B, 4A]
```

Original equal-key order:

```
TEXT
4A before 4B
```

became:

```
TEXT
4B before 4A
```

Therefore stability is lost.

22. IS SELECTION SORT IN-PLACE?

Yes.

Only constant extra memory.

$$ \boxed{O(1)} $$

---

23. SELECTION SORT KEY PROPERTY

After pass k:

> **the k smallest elements are in final positions at the beginning.**

Memory:

> **Selection fixes the LEFT side.**

---

24. BUBBLE VS SELECTION — FIRST CONTRAST

Bubble

Repeatedly pushes maximum to the right.

Selection

Repeatedly chooses minimum and fixes it on the left.

Think:

```text
Bubble      !'  largest  goes  RIGHT
Selection   !'  smallest  goes  LEFT
```

---

25. INSERTION SORT — BASIC IDEA

Insertion Sort works like arranging playing cards in your hand.

Suppose:

```text
[5, 2, 4, 6, 1]
```

Treat first element as sorted:

```text
[5] | [2,4,6,1]
```

Take `2` and insert it into correct position:

```text
[2,5] | [4,6,1]
```

Take `4`:

```text
[2,4,5] | [6,1]
```

Take `6`:

```text
[2,4,5,6] | [1]
```

Take `1`:

```text
[1,2,4,5,6]
```

---

26. INSERTION SORT INVARIANT

At the start of each iteration:

> **The prefix before the current element is already sorted.**

This is the key property.

Example:

```text
[2,4,5] | 3 | [...]
```

Insert `3` into:

```text
[2,4,5]
```

Result:

```text
[2,3,4,5]
```

---

27. INSERTION SORT PSEUDOCODE

```text
InsertionSort(A, n):

    for i = 1 to n-1:

        key = A[i]
        j = i - 1

        while j >= 0 and A[j] > key:
            A[j+1] = A[j]
            j = j - 1
```

```
        A[j+1] = key
```

---

28. INSERTION SORT TRACE

Initial:

```text
[5, 2, 4, 6, 1, 3]
```

Insert 2

```text
[2,5,4,6,1,3]
```

Insert 4

Shift `5` right:

```text
[2,4,5,6,1,3]
```

Insert 6

Already correct:

```text
[2,4,5,6,1,3]
```

Insert 1

Shift:

```text
6
5
4
2
```

Result:

```text
[1,2,4,5,6,3]
```

Insert 3

Result:

```text
[1,2,3,4,5,6]
```

29. INSERTION SORT BEST CASE

Already sorted:

```text
[1,2,3,4,5]
```

Each new key only needs one comparison with previous element.

Approximately:

$$ n-1 $$

comparisons.

Therefore:

$$ \boxed{\Theta(n)} $$

This makes Insertion Sort adaptive.

30. INSERTION SORT WORST CASE

Reverse sorted:

```text
[5,4,3,2,1]
```

When inserting each element, all previous elements must shift.

Work:

$$ 1+2+3+\cdots+(n-1) $$

$$ = \frac{n(n-1)}{2} $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

31. INSERTION SORT AVERAGE CASE

On average, many elements need shifting.

Therefore:

$$ \boxed{\Theta(n^2)} $$

32. INSERTION SORT AUXILIARY SPACE

Only:

- `key`
- `i`

- `j`

are needed.

Thus:

$$ \boxed{\Theta(1)} $$

33. IS INSERTION SORT STABLE?

Yes, standard Insertion Sort is stable.

Because elements are shifted only when:

```
TEXT
A[j] > key
```

not:

```
TEXT
A[j] >= key
```

Equal elements keep relative order.

Therefore:

$$ \boxed{\text{Stable}} $$

34. IS INSERTION SORT IN-PLACE?

Yes.

$$ \boxed{\Theta(1)} $$

extra space.

35. IS INSERTION SORT ADAPTIVE?

Yes.

If array is nearly sorted, few shifts are required.

This can make Insertion Sort very efficient in practice on small/nearly sorted arrays.

36. INSERTION SORT AND INVERSIONS

An **inversion** is a pair:

$$ (i,j) $$

such that:

$$ i<j $$

but:

\[A[i]>A[j] \]

Example:

```text
[3,1,2]
```

Inversions:

```text
(3,1)
(3,2)
```

Total:

\[2 \]

Insertion Sort essentially resolves inversions through shifts.

Its running time can therefore be related to:

\[\boxed{\Theta(n+I)} \]

where \(I\) is the number of inversions, under standard operation-count assumptions.

This explains its adaptive nature.

---

37. MAXIMUM NUMBER OF INVERSIONS

Reverse-sorted array:

```text
[n, n-1, ..., 2,1]
```

Every pair is inverted.

Number of pairs:

\[\binom n2 \]

\[= \boxed{\frac{n(n-1)}2} \]

Therefore maximum inversions:

\[\Theta(n^2) \]

---

38. MINIMUM NUMBER OF INVERSIONS

Sorted array:

```text
[1,2,3,...,n]
```

No inversions.

\[\boxed{0} \]

39. WHY INSERTION SORT IS GOOD FOR NEARLY SORTED DATA

If very few inversions exist:

\[I\ll n^2 \]

then work becomes close to:

\[O(n+I) \]

So nearly sorted data can approach:

\[O(n) \]

behavior.

40. BUBBLE SORT AND INVERSIONS

Adjacent swaps in Bubble Sort also remove inversions.

Each adjacent swap changes inversion count in a controlled way, and standard Bubble Sort's swap count equals the initial number of inversions.

This gives another useful GATE relationship:

\[\boxed{\text{Bubble Sort swaps = number of inversions}} \]

for the standard adjacent-swap version.

41. COMPARISON OF THREE SIMPLE SORTS

Property	Bubble	Selection	Insertion
Best	\(\Theta(n)\)*	\(\Theta(n^2)\)	\(\Theta(n)\)
Average	\(\Theta(n^2)\)	\(\Theta(n^2)\)	\(\Theta(n^2)\)
Worst	\(\Theta(n^2)\)	\(\Theta(n^2)\)	\(\Theta(n^2)\)
Space	\(O(1)\)	\(O(1)\)	\(O(1)\)
Stable	Yes	No	Yes
In-place	Yes	Yes	Yes
Adaptive	Yes*	No	Yes
Max swaps/writes	\(O(n^2)\)	\(O(n)\) swaps	\(O(n^2)\) shifts

*Bubble Sort best-case linearity/adaptiveness assumes early-exit optimization.

42. MEMORY TABLE

Remember:

```text
Bubble:
B = Biggest bubbles right
Stable
In-place

Selection:
S = Select smallest
Fix left
Unstable
Few swaps

Insertion:
I = Insert into sorted prefix
Stable
Adaptive
Good for nearly sorted
```

---

43. IDENTIFYING BUBBLE SORT FROM INTERMEDIATE STATES

Initial:

```text
[5,1,4,2,8]
```

After one Bubble pass:

```text
[1,4,2,5,8]
```

Notice:

> **Largest element is correctly placed at end.**

After two passes:

> **Last two largest elements are fixed.**

So if GATE shows pass-by-pass states where the sorted suffix grows from the right:

> **suspect Bubble Sort.**

---

44. IDENTIFYING SELECTION SORT

Selection Sort after each pass:

> **smallest remaining element moves into the first unsorted position.**

Example:

```text
Initial:
[64,25,12,22,11]

Pass 1:
[11,25,12,22,64]

Pass 2:
[11,12,25,22,64]
```

Sorted prefix grows from the left.

But unlike Insertion Sort:

> **elements in the prefix are in final positions, because each is globally minimum among remaining elements.**

---

45. IDENTIFYING INSERTION SORT

After each insertion:

> **prefix is sorted, but its elements are not necessarily in their final global positions.**

Example:

```text
Initial:
[5,2,4,6,1]

After inserting first few:
[2,4,5,6,1]
```

Prefix:

```text
[2,4,5,6]
```

is sorted.

But once `1` is considered, all these positions shift.

So the prefix is sorted but not necessarily final.

This is a crucial distinction from Selection Sort.

---

46. SELECTION VS INSERTION PREFIX

Selection Sort

After \(k\) passes:

> **first \(k\) elements are in final positions.**

Insertion Sort

After k passes:

> **first $k+1$ elements are sorted relative to one another, but not necessarily final globally.**

Very GATE-relevant.

---

47. COMPARISON COUNT — SELECTION SORT

Selection Sort performs:

$$ \frac{n(n-1)}{2} $$

key comparisons regardless of whether input is:

```text
sorted
reverse sorted
random
```

Therefore it is not adaptive.

---

48. COMPARISON COUNT — BUBBLE SORT

Basic unoptimized Bubble Sort:

$$ \frac{n(n-1)}{2} $$

comparisons.

Optimized Bubble Sort:

Best case only:

$$ n-1 $$

comparisons before detecting no swaps.

---

49. COMPARISON COUNT — INSERTION SORT

Best:

$$ n-1 $$

Worst approximately:

$$ \frac{n(n-1)}{2} $$

key comparisons/shifts.

The exact count depends on whether failed loop-condition comparisons are included, so read exact-count questions carefully.

---

50. WHY SELECTION SORT CAN BE USEFUL

Although Selection Sort is always quadratic, it minimizes swaps.

At most:

\[n-1 \]

swaps.

If writing to memory is much more expensive than comparing, Selection Sort may be attractive.

This is a good example of:

> **asymptotic time complexity is not the only practical metric.**

---

51. WHY INSERTION SORT IS USED IN REAL ALGORITHMS

Insertion Sort is often used for:

- small subarrays,
- nearly sorted data,
- hybrid sorting algorithms.

Why?

- simple,
- low overhead,
- good cache behavior,
- adaptive,
- efficient for small `n`.

GATE may test the conceptual reason rather than implementation details.

---

52. STABILITY EXAMPLE

Suppose records:

```text
(70,A)
(50,B)
(70,C)
```

Sorted by marks:

Stable result:

```text
(50,B)
(70,A)
(70,C)
```

Original order of equal marks:

```text
A before C
```

preserved.

Standard Bubble and Insertion preserve this.

Standard Selection may not.

---

53. IN-PLACE VS STABLE ARE DIFFERENT

Do not confuse them.

An algorithm can be:

- stable and in-place,
- stable and not in-place,
- unstable and in-place,
- unstable and not in-place.

Example:

Insertion Sort:

```text
stable + in-place
```

Selection Sort:

```text
unstable + in-place
```

These are independent properties.

---

54. PYQ-STYLE QUESTION 1

Which algorithm among these performs:

$$\Theta(n^2)$$

comparisons even for already sorted input?

A. Optimized Bubble Sort B. Selection Sort C. Insertion Sort D. Binary Search

Answer:

$$\boxed{B}$$

Selection Sort still scans the unsorted portion to find minimum.

---

55. PYQ-STYLE QUESTION 2

Which standard algorithms are stable?

A. Bubble Sort B. Selection Sort C. Insertion Sort

Answer:

$$\boxed{A,C}$$

56. PYQ-STYLE QUESTION 3

Which has at most:

$$O(n)$$

swaps?

A. Bubble Sort B. Selection Sort C. Insertion Sort

Answer:

$$\boxed{B}$$

Selection Sort typically performs at most one swap per pass.

57. PYQ-STYLE QUESTION 4

Consider:

```
TEXT
[5,4,3,2,1]
```

What is number of inversions?

All pairs are inverted.

$$\binom52 = \frac{5\cdot4}{2} = \boxed{10}$$

Therefore standard Bubble Sort makes:

$$\boxed{10}$$

adjacent swaps.

58. PYQ-STYLE QUESTION 5

An array is almost sorted.

Which among these generally benefits most?

A. Selection Sort B. Insertion Sort C. Standard unoptimized Bubble Sort

Answer:

$$\boxed{B}$$

Insertion Sort is naturally adaptive.

An optimized Bubble Sort can also benefit, but Insertion Sort is the classic answer for nearly sorted data.

59. PYQ-STYLE QUESTION 6 — INTERMEDIATE STATE

Suppose after a sorting pass:

```text
[2,3,5,8 | 1,7]
```

and we know the prefix is sorted but may later shift.

Which algorithm is most consistent?

$$ \boxed{\text{Insertion Sort}} $$

The sorted prefix need not yet be globally final.

---

60. PYQ-STYLE QUESTION 7 — PROPERTY

Which statement is true after the first pass of standard Bubble Sort in ascending order?

A. Minimum reaches first position B. Maximum reaches last position C. Entire array becomes sorted D. First two elements are fixed

Answer:

$$ \boxed{B} $$

---

61. PYQ-STYLE QUESTION 8 — SELECTION

After first pass of Selection Sort in ascending order:

> **The minimum element of the entire array is placed at first position.**

Therefore first position is final.

---

62. & þGATE TRAPS

Trap 1 — Stable means in-place

Wrong.

These are independent properties.

Trap 2 — Selection Sort best case is $O(n)$

Wrong.

It still searches for the minimum every pass:

$$ \boxed{\Theta(n^2)} $$

Trap 3 — Bubble Sort best is always $O(n)$

Only with early termination optimization.

Basic fixed-pass implementation can remain:

$$ \Theta(n^2) $$

Trap 4 — Selection Sort is stable

Standard implementation is not stable.

Trap 5 — Insertion Sort prefix is final

Wrong.

The prefix is sorted, but later smaller elements can shift it.

Trap 6 — Bubble Sort fixes the smallest first

In standard ascending Bubble Sort:

> **largest moves to the end.**

Trap 7 — Selection Sort needs many swaps

Actually it performs only:

$$ O(n) $$

swaps.

Trap 8 — Insertion Sort always takes n^2

Best case:

$$ \Theta(n) $$

and nearly sorted data can also be much faster.

Trap 9 — All quadratic sorts behave identically

No.

They differ in:

- comparisons,
- swaps,
- stability,
- adaptiveness,
- behavior on nearly sorted data.

Trap 10 — Equal complexity means equal practical performance

Not necessarily.

$$O(n^2)$$

algorithms can differ substantially in constants and data movement.

63. EXAM DECISION RULES

If largest element becomes fixed at end each pass:

Think:

> **Bubble Sort**

If smallest remaining element is selected and placed at front:

Think:

> **Selection Sort**

If a sorted prefix grows by inserting one element:

Think:

> **Insertion Sort**

If input is nearly sorted:

Think:

> **Insertion Sort**

If minimum number of swaps/writes is important:

Think:

> **Selection Sort**

If stability is required among these three:

Think:

> **Bubble / Insertion**

not standard Selection.

64. QUICK RECALL

Answer mentally.

1. What does stable sorting mean?
2. What does in-place mean?
3. What does adaptive mean?
4. Bubble Sort fixes which side?
5. Selection Sort fixes which side?
6. Insertion Sort maintains what invariant?
7. Bubble best case with optimization?
8. Bubble worst case?
9. Selection best case?
10. Insertion best case?
11. Standard Selection stable?
12. Standard Bubble stable?
13. Standard Insertion stable?
14. Which has $O(n)$ swaps?
15. Maximum inversions in `n` elements?

ANSWERS

1. Equal keys retain relative order.
2. Uses only small/constant extra storage.
3. Benefits from existing order.
4. Largest elements at right.
5. Smallest elements at left.
6. Prefix is sorted.
7. $\Theta(n)$.
8. $\Theta(n^2)$.
9. $\Theta(n^2)$.
10. $\Theta(n)$.
11. No.
12. Yes.
13. Yes.
14. Selection Sort.
15. $n(n-1)/2$.

65. PRACTICE SET

Easy

Q1 — MCQ

Which standard sorting algorithm is unstable?

A. Bubble Sort B. Selection Sort C. Insertion Sort D. Both A and C

Q2 — MCQ

Best-case time of Insertion Sort:

A. $\Theta(1)$ B. $\Theta(\log n)$ C. $\Theta(n)$ D. $\Theta(n^2)$

Q3 — NAT

Maximum number of inversions in an array of 8 distinct elements?

Medium

Q4 — MSQ

Which are true for standard Selection Sort?

A. In-place B. Stable C. $\Theta(n^2)$ best case D. At most $O(n)$ swaps

---

Q5 — MCQ

Which algorithm is generally most suitable among the three for nearly sorted data?

A. Selection B. Insertion C. Basic Selection and Insertion always identical D. None

---

Q6 — NAT

How many element comparisons does standard Selection Sort make for:

$$ n=10 $$

assuming one comparison per inner-loop check between keys?

Q7 — MSQ

Which standard algorithms are both stable and in-place?

A. Bubble B. Selection C. Insertion

Hard / Tricky

Q8 — MCQ

Array:

```text
[4,3,2,1]
```

How many swaps does standard adjacent-swap Bubble Sort perform?

A. 3 B. 4 C. 6 D. 8

Q9 — MSQ

Which statements are correct?

A. Insertion Sort's sorted prefix is always in final global positions.

B. Selection Sort's sorted prefix contains elements already in final positions.

C. Bubble Sort's sorted suffix contains elements already in final positions.

D. Selection Sort can require fewer swaps than Bubble Sort.

SOLUTIONS

Q1

Standard Selection Sort is unstable.

$$\boxed{B}$$

Q2

Already sorted input:

$$n-1$$

comparisons approximately.

Therefore:

$$\boxed{C}$$

Q3

Maximum inversions:

$$\frac{n(n-1)}2$$

For $n=8$:

$$\frac{8\cdot7}{2} = \boxed{28}$$

Q4

A true.

B false.

C true.

D true.

Answer:

$$\boxed{A,C,D}$$

Q5

Nearly sorted data favors:

$$ \boxed{B} $$

Insertion Sort.

Q6

Selection Sort comparisons:

$$ \frac{10\cdot9}{2} = \boxed{45} $$

Q7

Bubble and Insertion.

$$ \boxed{A,C} $$

Q8

Reverse array has:

$$ \frac{4\cdot3}{2}=6 $$

inversions.

Bubble swaps = inversions.

Answer:

$$ \boxed{C} $$

Q9

A false.

Insertion prefix is sorted but can move later.

B true.

C true.

D true.

Answer:

$$ \boxed{B,C,D} $$

66. Ø=ÜÓTES TO WRITE

Stable

Equal-key elements preserve relative order.

Mnemonic:

> **Same keys Stay**

In-place

Uses:

$$ O(1) $$

or very small extra storage.

Adaptive

Benefits from partial ordering.

Bubble Sort

Idea:

> **Adjacent comparisons + swaps.**

After each pass:

> **largest remaining element reaches right end.**

Complexity:

```TEXT
Best        = ~(n) with early stop
Average     = ~(n²)
Worst       = ~(n²)
Space   = O(1)
```

Properties:

```TEXT
Stable
In-place
Adaptive with optimization
```

Worst comparisons:

$$ \frac{n(n-1)}2 $$

Bubble swaps:

$$ \text{number of inversions} $$

Selection Sort

Idea:

> **Find minimum of unsorted part and place it at front.**

Complexity:

```TEXT
 B e s t        =   ~ ( n ² )
 A v e r a g e  =   ~ ( n ² )
 W o r s t      =   ~ ( n ² )
Space   = O(1)
```

Properties:

```TEXT
Unstable
In-place
Not adaptive
```

Comparisons:

\[\frac{n(n-1)}2 \]

Swaps:

\[O(n) \]

Insertion Sort

Idea:

> **Insert current element into sorted prefix.**

Complexity:

```TEXT
 B e s t        =   ~ ( n )
 A v e r a g e  =   ~ ( n ² )
 W o r s t      =   ~ ( n ² )
Space   = O(1)
```

Properties:

```TEXT
Stable
In-place
Adaptive
Good for nearly sorted/small data
```

Running time related to inversions:

\[\Theta(n+I) \]

Maximum Inversions

\[\boxed{\frac{n(n-1)}2} \]

67. MEMORY TRICKS

Bubble

> **Biggest Bubbles to Back**

Selection

> **Select Smallest, Set at Start**

Insertion

> **Insert into ordered prefix**

Stability

> **BI Stable**

```text
B = Bubble
I = Insertion
```

Standard Selection is the odd one out.

Best Case

```text
 B u b b l e *        !'   n
 S e l e c t i o n    !'   n ²
 I n s e r t i o n    !'   n
```

Mnemonic:

> **B-I can improve; Selection stays Square.**

&β30-SECOND REVISION

1. Stable = equal keys preserve order.

2. In-place "`stable.
3. Bubble compares adjacent elements.
4. Bubble sends largest to right.
5. Optimized Bubble best = \(\Theta(n)\).
6. Selection repeatedly chooses minimum.
7. Selection best/average/worst = \(\Theta(n^2)\).
8. Selection has only \(O(n)\) swaps.
9. Standard Selection is unstable.
10. Insertion maintains a sorted prefix.
11. Insertion best = \(\Theta(n)\).
12. Insertion worst = \(\Theta(n^2)\).
13. Bubble and Insertion are stable.
14. All three are usually in-place.
15. Nearly sorted !' think Insertion Sort.
16. Maximum inversions = \(n(n-1)/2\).
17. Bubble adjacent swaps = inversion count.
18. Selection prefix is final; Insertion prefix is only locally sorted.

' Chapter 6 Complete

Next Chapter:

Chapter 7 — Merge Sort, Quick Sort & Heap Sort

This is one of the most important sorting chapters. We'll cover:

- Merge Sort working and recurrence,
- merge procedure,
- Quick Sort partitioning,
- pivot behavior,
- Quick Sort best/average/worst cases,
- Heap Sort,
- heap construction,
- `Build-Heap = O(n)` trap,
- stability and in-place comparison,
- exact GATE-style traces,
- and the master sorting comparison table.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

CHAPTER 7 — MERGE SORT, QUICK SORT & HEAP SORT

GATE Importance: & & & & &

These three algorithms are among the most important sorting algorithms for GATE because they connect sorting with **divide-and-conquer, recurrences, heaps, stability, in-place behavior, and worst-case analysis**. Your uploaded course plan explicitly asks for these comparisons and complexity derivations.

The three key ideas are:

> **Merge Sort !' divide, sort halves, merge**
> **Quick Sort !' partition around a pivot**
> **Heap Sort !' build heap, repeatedly remove maximum/minimum**

---

1. MERGE SORT

Core Idea

Merge Sort follows **Divide and Conquer**.

It does three things:

1. Divide array into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves.

Example:

```text
[8, 3, 2, 9, 7, 1, 5, 4]
```

Divide:

```text
[8,3,2,9]      [7,1,5,4]
```

Again:

```text
[8,3] [2,9]    [7,1] [5,4]
```

Again:

```text
[8] [3] [2] [9] [7] [1] [5] [4]
```

Single-element arrays are already sorted.

Now merge upward.

---

2. MERGING TWO SORTED ARRAYS

Suppose:

```text
L = [2,5,8]
R = [1,4,9]
```

Compare first elements:

```text
 2   v s   1  !’   t a k e   1
```

Result:

```text
[1]
```

Then:

```text
 2   v s   4  !’   t a k e   2
```

```text
[1,2]
```

Then:

```text
 5   v s   4  !’   t a k e   4
```

```text
[1,2,4]
```

Then:

```text
 5   v s   9  !’   t a k e   5
```

```text
[1,2,4,5]
```

Then:

```text
 8   v s   9  !’   t a k e   8
```

```text
[1,2,4,5,8]
```

Finally copy remaining `9`:

```text
[1,2,4,5,8,9]
```

---

3. MERGE PROCEDURE COMPLEXITY

Suppose the two sorted arrays contain:

$$ p $$

and

$$ q $$

elements.

Each element is copied/processed at most once.

Therefore:

$$ \boxed{\Theta(p+q)} $$

For two halves of total size n:

$$ \boxed{\Theta(n)} $$

---

4. MERGE SORT PSEUDOCODE

```text
MergeSort(A, l, r):

    if l >= r:
        return

    mid = (l+r)/2

    MergeSort(A, l, mid)
    MergeSort(A, mid+1, r)

    Merge(A, l, mid, r)
```

---

5. MERGE SORT RECURRENCE

Two recursive calls:

$$ 2T(n/2) $$

Merge work:

$$ \Theta(n) $$

Therefore:

$$ \boxed{ T(n)=2T(n/2)+\Theta(n) } $$

From Master Theorem:

$$ a=2,\quad b=2 $$

$$ n^{\log_2 2}=n $$

and:

$$ f(n)=n $$

Same order.

Case 2:

$$ \boxed{\Theta(n\log n)} $$

---

6. MERGE SORT RECURSION TREE

At every level, total merge work is:

$$ n $$

Levels:

$$ \log_2 n $$

So:

```TEXT
 L e v e l   0  !'   n
 L e v e l   1  !'   n
 L e v e l   2  !'   n
...
```

Total:

$$ n\log n $$

Hence:

$$ \boxed{\Theta(n\log n)} $$

---

7. MERGE SORT BEST, AVERAGE, WORST

Even if input is already sorted, the standard Merge Sort still:

- divides recursively,
- processes merging.

Therefore:

Case	Time
Best	$\Theta(n\log n)$
Average	$\Theta(n\log n)$
Worst	$\Theta(n\log n)$

Very important contrast with Quick Sort.

8. MERGE SORT SPACE

Standard array Merge Sort uses temporary arrays during merging.

Auxiliary space:

$$ \boxed{\Theta(n)} $$

Recursion stack additionally:

$$ \Theta(\log n) $$

but the temporary merge storage dominates.

So total auxiliary space is usually quoted as:

$$ \boxed{\Theta(n)} $$

9. IS MERGE SORT STABLE?

Yes, standard Merge Sort can be stable.

When equal elements occur:

```
TEXT
L[i] == R[j]
```

take from the **left half first**.

That preserves original relative order.

Therefore:

$$ \boxed{\text{Stable}} $$

10. IS MERGE SORT IN-PLACE?

For arrays, standard Merge Sort is:

$$ \boxed{\text{Not in-place}} $$

because it requires:

$$ O(n) $$

extra memory for merging.

11. IMPORTANT MERGE SORT PROPERTY

Merge Sort guarantees:

$$ \boxed{\Theta(n\log n)} $$

even in the worst case.

This makes it predictable.

12. QUICK SORT

Quick Sort is also Divide and Conquer.

But unlike Merge Sort, most work occurs during **partitioning**.

Core steps:

1. Choose pivot.
2. Partition array around pivot.
3. Recursively sort left side.
4. Recursively sort right side.

13. PARTITION IDEA

Suppose:

```text
[9, 4, 7, 3, 10, 5]
```

Choose pivot:

```text
5
```

After partitioning, one possible arrangement:

```text
[4,3] 5 [9,10,7]
```

All elements on left:

\[<5 \]

All elements on right:

\[

> **5**

\]

The pivot is now in its final sorted position.

Important:

> **Left and right parts are not necessarily internally sorted yet.**

14. QUICK SORT KEY PROPERTY

After partition:

> **Pivot is in its final position.**

Then Quick Sort recursively handles both sides.

This is different from Merge Sort, where merging produces sorted larger subarrays.

15. QUICK SORT PSEUDOCODE

```text
QuickSort(A, low, high):

    if low < high:

        p = Partition(A, low, high)

        QuickSort(A, low, p-1)
        QuickSort(A, p+1, high)
```

16. PARTITION COST

Partition scans through the subarray.

For size n:

$$\boxed{\Theta(n)}$$

The total Quick Sort complexity depends mainly on **how balanced the partitions are**.

17. QUICK SORT BEST CASE

Best case:

Pivot divides array into roughly equal halves.

Recurrence:

$$T(n)=2T(n/2)+\Theta(n)$$

Therefore:

$$\boxed{\Theta(n\log n)}$$

18. QUICK SORT AVERAGE CASE

With reasonably balanced/random partitions:

$$\boxed{\Theta(n\log n)}$$

Expected time with good random pivot selection:

$$ \boxed{\Theta(n\log n)} $$

19. QUICK SORT WORST CASE

Worst case:

Pivot is repeatedly smallest or largest element.

Example, if first/last element is selected poorly on sorted input:

```text
[1,2,3,4,5,6]
```

Partitions may become:

```text
0 and n-1
```

Then:

$$ T(n)=T(n-1)+\Theta(n) $$

Expansion:

$$ n+(n-1)+(n-2)+\cdots+1 $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

20. QUICK SORT RECURSION DEPTH

Balanced case

$$ \Theta(\log n) $$

Worst case

$$ \Theta(n) $$

This also affects recursion-stack space.

21. QUICK SORT AUXILIARY SPACE

Quick Sort is generally considered **in-place with respect to array rearrangement**, but recursion requires stack space.

Average/balanced stack

$$ \boxed{O(\log n)} $$

Worst stack

$$ \boxed{O(n)} $$

depending on implementation and recursion order.

22. IS QUICK SORT STABLE?

Standard in-place Quick Sort is:

$$ \boxed{\text{Not stable}} $$

Partitioning swaps can reorder equal elements.

23. IS QUICK SORT IN-PLACE?

Standard partition implementations use only constant extra array storage.

So it is commonly treated as:

$$ \boxed{\text{In-place}} $$

excluding recursion stack.

This convention matters in GATE questions.

24. WHY QUICK SORT IS OFTEN FAST IN PRACTICE

Even though worst case is:

$$ O(n^2) $$

Quick Sort often performs very well because:

- good cache locality,
- small constants,
- no large auxiliary merge array,
- randomized pivot reduces chance of bad splits.

Therefore practical performance can be excellent.

But GATE focuses mainly on theoretical properties.

25. PIVOT CHOICE

Common strategies:

- first element
- last element
- random pivot
- median-like choices

Bad pivot selection can create unbalanced partitions.

Randomization helps avoid consistently bad inputs.

26. RANDOMIZED QUICK SORT

Randomly choose pivot.

Worst case is still theoretically:

$$ O(n^2) $$

because bad pivots are still possible.

But expected complexity becomes:

$$ \boxed{\Theta(n\log n)} $$

Important:

> **Randomization improves expected behavior; it does not eliminate the mathematical worst case.**

---

27. QUICK SORT COMPARISON EXAMPLE

If partition sizes are:

$$ n/2,\ n/2 $$

then:

$$ T(n)=2T(n/2)+n $$

$$ =\Theta(n\log n) $$

If partition sizes are:

$$ 0,\ n-1 $$

then:

$$ T(n)=T(n-1)+n $$

$$ =\Theta(n^2) $$

So:

> **Quick Sort complexity is driven by partition balance.**

---

28. MERGE SORT VS QUICK SORT

Property	Merge Sort	Quick Sort
Best	$\Theta(n\log n)$	$\Theta(n\log n)$
Average	$\Theta(n\log n)$	$\Theta(n\log n)$
Worst	$\Theta(n\log n)$	$\Theta(n^2)$
Stable	Yes	No
In-place	Usually no	Usually yes
Main operation	Merge	Partition
Extra array space	$O(n)$	Usually $O(1)$ besides stack

29. HEAP SORT

Heap Sort uses a **binary heap**.

To understand Heap Sort, we need a minimal heap prerequisite.

30. WHAT IS A BINARY HEAP?

A binary heap is a **complete binary tree** satisfying a heap-order property.

Two types:

Max Heap

Each parent is:

$$ \ge $$

its children.

Min Heap

Each parent is:

$$ \le $$

its children.

For ascending Heap Sort, we usually use a **Max Heap**.

31. COMPLETE BINARY TREE

A complete binary tree fills levels left-to-right.

Example:

```text
        50
       /  \
     30    40
    / \    /
   10 20  35
```

This can be stored compactly in an array.

32. ARRAY REPRESENTATION OF HEAP

Using 0-based indexing:

For node at index i:

$$ \text{left}=2i+1 $$

$$ \text{right}=2i+2 $$

$$ \text{parent}=\left\lfloor\frac{i-1}{2}\right\rfloor $$

Using 1-based indexing:

$$ \text{left}=2i $$

$$ \text{right}=2i+1 $$

$$ \text{parent}=\lfloor i/2\rfloor $$

GATE may use either convention.

Check indices carefully.

---

33. MAX-HEAP PROPERTY

Example:

```text
        90
       /  \
     70    80
    / \    /
   20 40  30
```

Every parent is at least as large as children.

But note:

> **A heap is not a fully sorted tree.**

For example:

`70` and `80` do not need a particular order relative to each other.

---

34. HEAPIFY

`Heapify` repairs heap property at a node assuming its child subtrees are already heaps.

For Max Heap:

1. compare node with children,
2. swap with largest child if needed,
3. continue downward.

Height of heap:

$$ O(\log n) $$

Therefore one Heapify operation is:

$$ \boxed{O(\log n)} $$

---

35. HEAP SORT IDEA

For ascending order:

Step 1

Build a Max Heap.

Largest element is at root.

Step 2

Swap root with last element.

Largest element moves to final position.

Step 3

Reduce heap size.

Step 4

Heapify root.

Repeat.

---

36. HEAP SORT TRACE

Suppose Max Heap:

```text
        10
       /  \
      5    8
     / \
    2   3
```

Array:

```text
[10,5,8,2,3]
```

Swap root with last:

```text
[3,5,8,2,10]
```

`10` is final.

Heapify first four elements:

```text
[8,5,3,2,10]
```

Swap root:

```text
[2,5,3,8,10]
```

Heapify:

```text
[5,2,3,8,10]
```

Continue until:

```text
[2,3,5,8,10]
```

---

37. HEAP SORT PSEUDOCODE IDEA

```text
BuildMaxHeap(A)

for end = n-1 down to 1:

    swap A[0], A[end]

    heap_size--

    MaxHeapify(A, 0)
```

---

38. BUILD-HEAP COMPLEXITY — IMPORTANT TRAP

A naive thought:

There are n nodes.

Heapify costs:

$$ O(\log n) $$

Therefore Build-Heap must be:

$$ O(n\log n) $$

But this is **not tight**.

Correct Build-Heap complexity:

$$ \boxed{\Theta(n)} $$

This is a famous GATE result.

---

39. WHY BUILD-HEAP IS $O(n)$

Most nodes are near the leaves and require very little Heapify work.

Approximately:

- $n/2$ nodes have height 0
- $n/4$ nodes have height 1
- $n/8$ nodes have height 2
- ...

Total work roughly:

$$ \frac n2(0)+\frac n4(1)+\frac n8(2)+\cdots $$

Factor n:

$$ n\left(\frac14+\frac{2}{8}+\frac{3}{16}+\cdots \right) $$

The series converges to a constant.

Therefore:

$$ \boxed{\Theta(n)} $$

---

40. ALTERNATIVE HEAP CONSTRUCTION

Important distinction:

Bottom-up Build-Heap

$$ \boxed{\Theta(n)} $$

Insert n elements one-by-one into heap

Each insertion:

$$ O(\log n) $$

Total:

$$ \boxed{O(n\log n)} $$

These are not the same construction method.

---

41. HEAP SORT TIME COMPLEXITY

Build heap:

$$ \Theta(n) $$

Then perform approximately:

$$ n-1 $$

extractions.

Each extraction requires Heapify:

$$ O(\log n) $$

So:

$$ O(n\log n) $$

Total:

$$ \Theta(n)+\Theta(n\log n) $$

Therefore:

$$ \boxed{\Theta(n\log n)} $$

---

42. HEAP SORT BEST, AVERAGE, WORST

Standard Heap Sort:

Case	Time
Best	$\Theta(n\log n)$
Average	$\Theta(n\log n)$
Worst	$\Theta(n\log n)$

It does not become linear on sorted input.

43. IS HEAP SORT IN-PLACE?

Yes.

Heap can be stored directly inside array.

Additional memory:

$$\boxed{O(1)}$$

for iterative Heapify, ignoring implementation details.

Therefore Heap Sort is considered:

$$\boxed{\text{In-place}}$$

44. IS HEAP SORT STABLE?

Standard Heap Sort is:

$$\boxed{\text{Not stable}}$$

Long-distance swaps can reorder equal elements.

45. HEAP SORT KEY PROPERTY

Using Max Heap for ascending sort:

> **Maximum element is repeatedly moved to the end.**

This superficially resembles Bubble Sort, but mechanism is completely different.

Bubble:

```text
adjacent swaps
```

Heap Sort:

```text
heap root extraction
```

46. MERGE, QUICK, HEAP MASTER TABLE

Algorithm	Best	Average	Worst	Aux Space	Stable	In-place
Merge	$n\log n$	$n\log n$	$n\log n$	$O(n)$	Yes	No
Quick	$n\log n$	$n\log n$	n^2	$O(\log n)$ avg stack	No	Yes
Heap	$n\log n$	$n\log n$	$n\log n$	$O(1)$	No	Yes

47. ADD PREVIOUS SIMPLE SORTS

Sort	Best	Average	Worst	Stable	In-place
Bubble*	n	n^2	n^2	Yes	Yes
Selection	n^2	n^2	n^2	No	Yes
Insertion	n	n^2	n^2	Yes	Yes
Merge	$n\log n$	$n\log n$	$n\log n$	Yes	No
Quick	$n\log n$	$n\log n$	n^2	No	Yes
Heap	$n\log n$	$n\log n$	$n\log n$	No	Yes

*Optimized Bubble Sort.

48. MEMORY TRICK FOR STABILITY

Among the six common sorts:

Stable

```text
Bubble
Insertion
Merge
```

Mnemonic:

> **BIM is Stable**

```text
B = Bubble
I = Insertion
M = Merge
```

Usually unstable

```text
Selection
Quick
Heap
```

Mnemonic:

> **SQH = Shakes equal order**

---

49. MEMORY TRICK FOR \(n\log n\)

Three important efficient comparison sorts:

```text
Merge
Quick average
Heap
```

Mnemonic:

> **MQH !' Mostly n log n**

But remember:

> **Quick can crash to \(n^2\)**

---

50. WHY QUICK SORT WORST CASE DIFFERS

Merge and Heap guarantee:

\[\Theta(n\log n) \]

worst-case.

Quick Sort can be:

\[\Theta(n^2) \]

because its recursion tree may become highly unbalanced.

Example:

```TEXT
n
 |
n-1
 |
n-2
 |
...
```

Depth:

\[n \]

instead of:

\[\log n \]

---

51. QUICKSORT PARTITION TREE

Balanced:

```TEXT
            n
          /   \
        n/2   n/2
       / \     / \
```

Depth:

\[\log n \]

Total:

\[n\log n \]

Unbalanced:

```TEXT
n
 \
 n-1
   \
   n-2
     \
     ...
```

Depth:

\[n \]

Total:

\[n+(n-1)+\cdots+1 \]

\[=\Theta(n^2) \]

---

52. MERGE SORT INTERMEDIATE STATE

Merge Sort produces sorted runs that get progressively larger.

Example:

Initial:

```text
[8,3,6,2,7,1,5,4]
```

After sorting pairs:

```text
[3,8] [2,6] [1,7] [4,5]
```

After merging size-4 blocks:

```text
[2,3,6,8] [1,4,5,7]
```

Then final merge.

If GATE shows sorted blocks doubling in size, suspect:

\[\boxed{\text{Merge Sort}} \]

---

53. HEAP SORT INTERMEDIATE STATE

During ascending Heap Sort:

> **A sorted suffix grows at the right.**

But unlike Bubble Sort, the unsorted prefix maintains a heap structure.

So identify:

```text
sorted suffix + heap prefix
```

!' Heap Sort.

---

54. QUICK SORT INTERMEDIATE STATE

Quick Sort often shows:

> **one pivot placed correctly, with smaller elements on one side and larger on the other.**

But those sides may remain internally unsorted.

So if you see:

```text
smaller values | pivot | larger values
```

think:

$$\boxed{\text{Quick Sort partition}}$$

55. PYQ-STYLE QUESTION 1

Recurrence:

$$T(n)=2T(n/2)+n$$

Which sorting algorithm strongly matches this?

A. Merge Sort B. Selection Sort C. Linear Search D. Bubble Sort

Answer:

$$\boxed{A}$$

56. PYQ-STYLE QUESTION 2

Which algorithm has worst-case:

$$\Theta(n^2)$$

but average:

$$\Theta(n\log n)$$

among Merge, Quick and Heap?

$$\boxed{\text{Quick Sort}}$$

57. PYQ-STYLE QUESTION 3

Which standard sorting algorithm is stable and guarantees:

$$O(n\log n)$$

worst-case?

Among these three:

$$\boxed{\text{Merge Sort}}$$

58. PYQ-STYLE QUESTION 4

Bottom-up Build-Max-Heap on n arbitrary elements takes:

A. $O(\log n)$ B. $O(n)$ C. $O(n\log n)$ D. $O(n^2)$

Answer:

$$\boxed{B}$$

This is one of the classic GATE traps.

59. PYQ-STYLE QUESTION 5

Heap Sort after building heap performs approximately n Heapify operations of cost:

$$O(\log n)$$

Therefore sorting phase:

$$O(n\log n)$$

Total including Build-Heap:

$$O(n)+O(n\log n)$$

$$=\boxed{O(n\log n)}$$

60. PYQ-STYLE QUESTION 6

If Quick Sort always chooses the smallest element as pivot, partition sizes are:

$$0,\ n-1$$

Recurrence:

$$T(n)=T(n-1)+n$$

Therefore:

$$\boxed{\Theta(n^2)}$$

61. PYQ-STYLE QUESTION 7

Which are stable?

A. Merge B. Quick C. Heap

Answer:

$$\boxed{A}$$

for standard implementations.

62. PYQ-STYLE QUESTION 8

Which are commonly considered in-place?

A. Standard array Merge Sort B. Quick Sort C. Heap Sort

Answer:

$$\boxed{B,C}$$

63. PYQ-STYLE QUESTION 9 — HEAP ARRAY

For 0-based index $i=4$:

Left child:

$$ 2i+1=9 $$

Right child:

$$ 2i+2=10 $$

Parent:

$$ \left\lfloor\frac{4-1}{2}\right\rfloor = 1 $$

---

64. PYQ-STYLE QUESTION 10 — BUILD HEAP

Why isn't Build-Heap $\Theta(n\log n)$?

Because not every node travels through full height $\log n$.

Most nodes are near leaves and require:

$$ O(1) $$

or very little work.

Summing work across heights gives:

$$ \boxed{\Theta(n)} $$

---

65. & ÞGATE TRAPS

Trap 1 — Merge Sort is in-place

Standard array Merge Sort is not.

Auxiliary:

$$ O(n) $$

Trap 2 — Quick Sort always runs in $n\log n$

Wrong.

Worst:

$$ \Theta(n^2) $$

Trap 3 — Randomized Quick Sort has no worst case

Wrong.

Worst case remains:

$$ O(n^2) $$

Expected complexity improves.

Trap 4 — Build-Heap is $O(n\log n)$

Bottom-up Build-Heap:

$$\boxed{\Theta(n)}$$

Trap 5 — Heap Sort is stable

Standard Heap Sort is unstable.

Trap 6 — Heap is fully sorted

Wrong.

Heap guarantees only parent-child order.

Trap 7 — Quick Sort pivot merely approximately reaches correct position

After a correct partition, pivot is placed in its final sorted position.

Trap 8 — Merge Sort best case is $O(n)$

Standard Merge Sort:

$$\Theta(n\log n)$$

even on sorted input.

Trap 9 — Heap Sort best case is $O(n)$

Standard Heap Sort:

$$\Theta(n\log n)$$

Trap 10 — In-place means no recursion memory

Not necessarily.

Quick Sort is commonly called in-place because partitioning uses constant extra array memory, although recursion stack consumes space.

66. EXAM DECISION RULES

If you see:

```text
divide exactly into halves + merge
```

Think:

> **Merge Sort.**

If you see:

```text
pivot + partition
```

Think:

> **Quick Sort.**

If you see:

```text
complete binary tree + max/min root
```

Think:

> **Heap.**

If worst-case guarantee matters:

Among these:

> **Merge or Heap.**

If stability + \(n\log n\) worst case:

Think:

> **Merge Sort.**

If low extra array memory + good average performance:

Think:

> **Quick Sort.**

If Build-Heap:

Immediately remember:

$$\boxed{\Theta(n)}$$

If repeated heap extraction:

Each:

$$O(\log n)$$

so n extractions:

$$O(n\log n)$$

---

67. QUICK RECALL

Try mentally.

1. Merge Sort design technique?
2. Merge recurrence?
3. Merge worst-case time?
4. Merge auxiliary space?
5. Merge stable?
6. Quick Sort core operation?
7. Quick best/average?
8. Quick worst?
9. Why Quick worst can become n^2?
10. Is randomized Quick Sort worst-case still n^2?
11. Heap root of Max Heap?
12. Heapify complexity?
13. Build-Heap complexity?
14. Heap Sort total complexity?
15. Heap Sort stable?
16. Which of Merge/Quick/Heap are in-place?
17. Which guarantees $n\log n$ and is stable?
18. Heap is completely sorted — true or false?

ANSWERS

1. Divide and Conquer.
2. $2T(n/2)+\Theta(n)$.
3. $\Theta(n\log n)$.
4. $\Theta(n)$.
5. Yes.
6. Partitioning.
7. $\Theta(n\log n)$.

8. $\Theta(n^2)$.
9. Highly unbalanced partitions.
10. Yes.
11. Maximum element.
12. $O(\log n)$.
13. $\Theta(n)$.
14. $\Theta(n\log n)$.
15. No.
16. Quick and Heap are commonly considered in-place.
17. Merge Sort.
18. False.

---

68. PRACTICE SET

Easy

Q1 — MCQ

Worst-case complexity of Merge Sort:

A. $\Theta(n)$ B. $\Theta(n\log n)$ C. $\Theta(n^2)$ D. $\Theta(2^n)$

Q2 — MCQ

Worst-case complexity of Quick Sort:

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n\log n)$ D. $\Theta(n^2)$

Q3 — MCQ

Build-Max-Heap using bottom-up heap construction takes:

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n\log n)$ D. $\Theta(n^2)$

Medium

Q4 — MSQ

Which standard algorithms have worst-case:

$$\Theta(n\log n)$$

A. Merge Sort B. Quick Sort C. Heap Sort

Q5 — MSQ

Which are standard properties of Merge Sort?

A. Stable B. In-place for arrays C. Worst-case $n\log n$ D. Requires merging

---

Q6 — MCQ

Quick Sort gives recurrence:

$$ T(n)=T(n-1)+\Theta(n) $$

when:

A. partitions are balanced B. pivot always creates 0 and $n-1$ partitions C. array is merged D. heap is balanced

---

Q7 — NAT

A binary heap stored using 0-based indexing has a node at index 6.

What is its left-child index?

Hard / Tricky

Q8 — MSQ

Which statements are correct?

A. Randomized Quick Sort has expected $O(n\log n)$ time.

B. Randomized Quick Sort cannot ever take $O(n^2)$.

C. Bottom-up Build-Heap is $O(n)$.

D. Standard Heap Sort is stable.

Q9 — MCQ

Suppose Quick Sort partitions every problem into sizes approximately:

$$ n/4 $$

and:

$$ 3n/4 $$

with linear partition work.

What is the asymptotic complexity?

A. $\Theta(n)$ B. $\Theta(n\log n)$ C. $\Theta(n^2)$ D. $\Theta(\log n)$

SOLUTIONS

Q1

Merge recurrence:

$$ 2T(n/2)+n $$

Therefore:

$$ \boxed{B} $$

Q2

Worst partitions:

$$ T(n)=T(n-1)+n $$

Therefore:

$$ \boxed{D} $$

Q3

Bottom-up Build-Heap:

$$ \boxed{B} $$

Q4

Merge:

$$ \Theta(n\log n) $$

Heap:

$$ \Theta(n\log n) $$

Quick worst:

$$ \Theta(n^2) $$

Answer:

$$ \boxed{A,C} $$

Q5

A true.

B false.

C true.

D true.

Answer:

$$ \boxed{A,C,D} $$

Q6

This recurrence occurs for maximally unbalanced partitions.

Answer:

$$ \boxed{B} $$

Q7

0-based:

$$ 2i+1 $$

For:

$$ i=6 $$

$$ 2(6)+1=13 $$

Answer:

$$ \boxed{13} $$

Q8

A true.

B false.

C true.

D false.

Answer:

$$ \boxed{A,C} $$

Q9

Even though partitions are unequal, both are constant fractions of n.

Recursion depth remains:

$$ \Theta(\log n) $$

Total work per level remains approximately:

$$ \Theta(n) $$

Therefore:

$$ \boxed{\Theta(n\log n)} $$

Answer:

$$ \boxed{B} $$

---

69. Ø=ÑÓTES TO WRITE

Merge Sort

Technique:

> **Divide and Conquer.**

Recurrence:

$$ T(n)=2T(n/2)+n $$

Complexity:

```
TEXT
 B e s t        =   ~ ( n   l o g   n )
 A v e r a g e  =   ~ ( n   l o g   n )
 W o r s t      =   ~ ( n   l o g   n )
 S p a c e      =   ~ ( n )
```

Properties:

```
TEXT
Stable
Not in-place for standard arrays
```

Quick Sort

Idea:

> **Choose pivot !' partition !' recurse.**

Balanced recurrence:

$$ 2T(n/2)+n $$

Result:

$$ \Theta(n\log n) $$

Worst recurrence:

$$ T(n-1)+n $$

Result:

$$ \Theta(n^2) $$

Properties:

```
TEXT
 A v e r a g e  =   ~ ( n   l o g   n )
 W o r s t      =   ~ ( n ² )
Unstable
In-place array partitioning
```

Randomized Quick Sort:

$$ \text{Expected } \Theta(n\log n) $$

Worst still:

$$ \Theta(n^2) $$

Heap Sort

Heap:

> **Complete binary tree + heap-order property.**

0-based indexing:

$$ left=2i+1 $$

$$ right=2i+2 $$

$$ parent=\lfloor(i-1)/2\rfloor $$

Heapify:

$$ O(\log n) $$

Build-Heap:

$$ \boxed{\Theta(n)} $$

Heap Sort:

$$ \boxed{\Theta(n\log n)} $$

Properties:

```text
In-place
Unstable
Worst-case n log n
```

Master Comparison

Sort	Worst	Stable	In-place
Merge	$n\log n$	Yes	No
Quick	n^2	No	Yes
Heap	$n\log n$	No	Yes

---

70. MEMORY TRICKS

Merge

> **Split !' Sort !' Stitch**

Stitch = merge.

Quick

> **Pivot decides performance**

Balanced pivot:

$$ n\log n $$

Bad pivot:

$$ n^2 $$

Heap

> **Build once in n, remove n times in log n**

$$ n+n\log n = n\log n $$

Stability

> **BIM stable**

```text
Bubble
Insertion
Merge
```

Efficient comparison sorts

> **Merge guarantees, Heap guarantees, Quick gambles**

Merge:

$$ n\log n $$

Heap:

$$ n\log n $$

Quick:

$$ n\log n\text{ average},\ n^2\text{ worst} $$

&β30-SECOND REVISION

1. Merge Sort = divide, recursively sort, merge.
2. Merge recurrence = $2T(n/2)+n$.
3. Merge best/avg/worst = $\Theta(n\log n)$.
4. Merge is stable but standard array version uses $O(n)$ extra memory.
5. Quick Sort = pivot + partition.
6. Balanced Quick = $\Theta(n\log n)$.

7. Worst Quick = $\Theta(n^2)$.
8. Randomization gives expected $n\log n$, not worst-case guarantee.
9. Max Heap root = maximum.
10. Heapify = $O(\log n)$.
11. Bottom-up Build-Heap = $\Theta(n)$, not $n\log n$.
12. Heap Sort = $\Theta(n\log n)$ in all major cases.
13. Heap Sort is in-place but unstable.
14. Merge is stable; Quick and Heap usually are not.
15. Quick and Heap are commonly considered in-place.
16. Pivot is final after Quick partition.
17. Heap is not fully sorted.
18. Build-Heap $O(n)$ is a major GATE trap.

' Chapter 7 Complete

Next Chapter:

Chapter 8 — Comparison-Sorting Lower Bound + Counting Sort, Radix Sort & Bucket Sort

We'll cover:

- why comparison sorting cannot generally beat $\Omega(n\log n)$,
- decision-tree proof intuition,
- why Counting/Radix can beat $n\log n$,
- Counting Sort working and complexity,
- stability requirements,
- Radix Sort passes,
- Bucket Sort intuition,
- constraints behind linear-time sorting,
- and GATE traps such as **"$n\log n$ is not a universal sorting lower bound."**

Type `NEXT` to continue.

USER

NEXT

USER

NEXT

ASSISTANT

CHAPTER 8 — COMPARISON-SORTING LOWER BOUND + COUNTING SORT, RADIX SORT & BUCKET SORT

GATE Importance: & & & & &

Your uploaded Algorithms plan explicitly includes comparison vs non-comparison sorting, lower bound for comparison sorting, Counting Sort, Radix Sort and Bucket Sort.

The big idea of this chapter is:

> **Comparison sorts have a fundamental \(\Omega(n\log n)\) lower bound in the general case.**
> **Non-comparison sorts can beat it by using extra information about the keys.**

---

1. WHY THIS TOPIC MATTERS

You have already seen:

- Merge Sort !' \(O(n\log n)\)
- Heap Sort !' \(O(n\log n)\)
- Quick Sort average !' \(O(n\log n)\)

A natural question is:

> **Can we design a comparison-based sorting algorithm with worst-case \(O(n)\) time?**

For arbitrary distinct keys using only comparisons:

\[\boxed{\text{No}} \]

The lower bound is:

\[\boxed{\Omega(n\log n)} \]

But algorithms like Counting Sort can run in:

\[O(n+k) \]

Why does this not violate the lower bound?

Because Counting Sort is **not comparison-based**.

---

2. COMPARISON SORTING

A comparison sort determines relative order by asking questions like:

\[A[i]<A[j]? \]

Examples:

```text id="bqt008" Bubble Sort Selection Sort Insertion Sort Merge Sort Quick Sort Heap Sort

```
These algorithms learn the ordering through comparisons.
```
```

```
---

# 3. HOW MANY POSSIBLE SORTED ORDERS EXIST?

Suppose we have \(n\) distinct elements.

Number of possible input permutations:

\[
\boxed{n!}
\]

Example with 3 elements:

```text id="lbwe7j"
A B C
A C B
B A C
B C A
C A B
C B A
```
```

There are:

\[3!=6 \]

possible orders.

A sorting algorithm must distinguish among all these possibilities.

---

4. DECISION TREE MODEL

A comparison has two possible outcomes:

```text id="qu5dbb" Yes / No

```
So comparison sorting can be represented using a binary decision tree.

Example conceptually:

```text id="o9wr3x"
             A < B ?
            /       \
          yes       no
          /           \
      B < C ?        A < C ?
       / \             / \
      ... ...         ... ...
```
```

Each leaf represents one possible final ordering.

---

## 5. NUMBER OF LEAVES

To correctly distinguish all permutations:

\[ \text{number of leaves}\ge n! \]

For a binary tree of height \(h\):
```

$$ \text{maximum leaves}\le2^h $$

Therefore:

$$ 2^h\ge n! $$

Taking log:

$$ h\ge\log_2(n!) $$

Thus any comparison sort requires at least:

$$ \boxed{\Omega(\log(n!))} $$

comparisons in the worst case.

---

6. WHY $\log(n!)=\Theta(n\log n)$

We know:

$$ n!=1\cdot2\cdot3\cdots n $$

Upper bound:

$$ n!\le n^n $$

Taking logs:

$$ \log(n!)\le n\log n $$

For lower bound, consider roughly the last half of factors:

$$ \frac n2,\frac n2+1,\ldots,n $$

There are about $n/2$ such terms, each at least $n/2$.

Therefore:

$$ n!\ge\left(\frac n2\right)^{n/2} $$

Taking log:

$$ \log(n!) \ge \frac n2\log\frac n2 $$

which is:

$$ \Omega(n\log n) $$

Hence:

$$ \boxed{\log(n!)=\Theta(n\log n)} $$

Therefore comparison-sorting lower bound:

$$ \boxed{\Omega(n\log n)} $$

---

7. WHAT EXACTLY DOES THE LOWER BOUND MEAN?

For general comparison sorting of arbitrary keys:

$$ \boxed{\text{Worst-case comparisons}\ge\Omega(n\log n)} $$

It does **not** say:

> **Every sorting algorithm must take $n\log n$.**

And it does **not** mean:

> **Every input requires $n\log n$ comparisons.**

Example:

Insertion Sort on already sorted input:

$$ \Theta(n) $$

But worst-case comparison sorting still obeys the lower bound.

---

8. IMPORTANT GATE TRAP

Statement:

> **No sorting algorithm can run in $O(n)$.**

False.

Correct statement:

> **No general comparison-based sorting algorithm can have worst-case complexity asymptotically better than $\Omega(n\log n)$.**

Counting Sort and Radix Sort avoid this limitation.

---

9. WHY COUNTING SORT CAN BE LINEAR

Suppose keys are integers in a limited range:

$$ 0,1,\ldots,k $$

Instead of comparing elements against one another, Counting Sort counts occurrences.

Example:

```text id="am2tbr" A = [4,2,2,8,3,3,1]

```
Range:

```text id="p0zjn1"
0...8
```
```

Count frequencies.

---

## 10. COUNT ARRAY

Initialize:
```

```text id="zpk8pg" Value : 0 1 2 3 4 5 6 7 8 Count : 0 0 0 0 0 0 0 0 0

```
Process input:

```text id="kvn7o7"
 4  !'   c o u n t [ 4 ] + +
 2  !'   c o u n t [ 2 ] + +
 2  !'   c o u n t [ 2 ] + +
 8  !'   c o u n t [ 8 ] + +
 3  !'   c o u n t [ 3 ] + +
 3  !'   c o u n t [ 3 ] + +
 1  !'   c o u n t [ 1 ] + +
```

Result:

```text id="zjwg88" Value : 0 1 2 3 4 5 6 7 8 Count : 0 1 2 2 1 0 0 0 1

```
From frequencies, sorted order:

```text id="psmtjv"
[1,2,2,3,3,4,8]
```

11. SIMPLE COUNTING SORT COMPLEXITY

Let:

$$ n=\text{number of elements} $$

and:

$$ k=\text{range of possible key values} $$

Steps:

1. Initialize count array !' $O(k)$
2. Count n elements !' $O(n)$
3. Process count array !' $O(k)$

Therefore:

$$ \boxed{O(n+k)} $$

Space:

$$ \boxed{O(n+k)} $$

for a stable output-array implementation, or $O(k)$ extra for a simpler frequency reconstruction variant that may not preserve associated records stably.

12. WHEN IS COUNTING SORT EFFECTIVELY LINEAR?

If:

$$ k=O(n) $$

then:

$$ O(n+k)=O(n) $$

Example:

$$ k=2n $$

Then:

$$ O(n+2n)=O(n) $$

So Counting Sort can be linear.

---

13. WHEN IS COUNTING SORT BAD?

Suppose:

$$ n=100 $$

but possible key range:

$$ 0\ldots10^9 $$

Then count array may need roughly:

$$ 10^9 $$

positions.

Complexity:

$$ O(n+k) $$

becomes enormous.

So Counting Sort is useful when:

> **Key range is not too large compared with number of elements.**

---

14. STABLE COUNTING SORT

For applications like Radix Sort, Counting Sort must be stable.

We don't merely output values from frequencies.

Instead we compute cumulative counts.

Example frequencies:

```text id="kto9sq" Value : 0 1 2 3 4 Count : 0 1 2 2 1

```
Cumulative:

```text id="3qofax"
Value : 0 1 2 3 4
Cum   : 0 1 3 5 6
```
```

These cumulative values tell us final positions.

---

## 15. WHY PROCESS INPUT FROM RIGHT TO LEFT?
```

Suppose equal-key records:

```text id="60ee4b" (2,A), (2,B)

```
A stable Counting Sort should preserve:

```text id="nxxzcj"
A before B
```

When filling the output array using cumulative counts, processing input from **right to left** ensures the later equal element is placed later.

Thus relative order is preserved.

This is a classic stability detail.

---

16. COUNTING SORT PROPERTIES

Property	Counting Sort
Comparison-based?	No
Time	\(O(n+k)\)
Stable?	Yes, with standard stable implementation
In-place?	Usually no
Extra space	\(O(n+k)\)
Key requirement	Discrete bounded range

---

17. COUNTING SORT WITH NEGATIVE VALUES

Basic exposition often assumes:

\[0\le key\le k \]

But negative integers can also be handled using an offset.

Example:

```text id="1zxout" [-3,-1,-3,2]

```
Minimum:

\[
-3
\]

Map value \(x\) to:

\[
x-(-3)=x+3
\]
```
```

```
So range becomes nonnegative.

The real requirement is:

> Finite manageable integer range.

---

# 18. RADIX SORT

Radix Sort sorts keys **digit by digit**.

Example:

```text id="36rjfp"
170
45
75
90
802
24
2
66
```
```

For LSD Radix Sort:

1. sort by units digit,
2. then tens digit,
3. then hundreds digit.

Each digit sort must be **stable**.

---

19. WHY STABILITY IS REQUIRED IN RADIX SORT

Suppose after units-digit sorting:

```text id="qrifck" 12 22 13 23

```
Now sort by tens digit.

If the digit sort is unstable, the correct ordering established by lower digits can be destroyed.

Stable sorting preserves the result of earlier digit passes.

Therefore:

\[
\boxed{\text{LSD Radix Sort requires stable per-digit sorting}}
\]

Counting Sort is often used for this.

---

20. RADIX SORT TRACE

Input:

```text id="lmhc7g"
```

```
[170,45,75,90,802,24,2,66]
```

Pass 1 — units digit

Digits:

```text id="6euctn" 170 !’ 0 45  !’ 5 75  !’ 5 90  !’ 0 802 !’ 2 24  !’ 4 2   !’ 2 66  !’ 6

Stable order by units might become:

```text id="b19gz9"
[170,90,802,2,24,45,75,66]
```

Pass 2 — tens digit

Sort stably by tens digit.

Pass 3 — hundreds digit

After final pass:

```text id="ffms9s" [2,24,45,66,75,90,170,802]

Sorted.

---

# 21. RADIX SORT COMPLEXITY

Suppose:

- \(n\) keys,
- \(d\) digits,
- each digit ranges over \(0\ldots k-1\).

If Counting Sort is used per digit:

\[
O(n+k)
\]

per pass.

With \(d\) passes:

\[
\boxed{O(d(n+k))}
\]

If base/radix \(k\) is constant:

\[
\boxed{O(dn)}
\]

---

# 22. NUMBER OF DIGITS
```
```

```
For maximum key value \(M\) represented in base \(b\):

\[
d=\Theta(\log_b M)
\]

Therefore complexity may also be written as:

\[
O((n+b)\log_b M)
\]

depending on assumptions.

---

# 23. DOES RADIX SORT ALWAYS BEAT \(n\log n\)?

No.

Its performance depends on:

- number of digits,
- radix/base,
- key representation,
- per-digit sorting cost.

If \(d\) is large, it may not be effectively linear.

Again:

> Complexity depends on assumptions about the keys.

---

# 24. LSD VS MSD RADIX SORT

## LSD

Least Significant Digit first.

Example:

```text id="zxaxcf"
 u n i t s  !’   t e n s  !’   h u n d r e d s
```
```

Usually uses stable sort on every pass.

MSD

Most Significant Digit first.

Example:

```text id="8ybsbc" hundreds !’ tens !’ units

```
Often partitions recursively by digit.

For GATE, LSD with stable Counting Sort is the more common conceptual model.
```
```

```
---

# 25. RADIX SORT PROPERTIES

| Property | Radix Sort |
|---|---|
| Comparison-based? | No |
| Typical time | \(O(d(n+k))\) |
| Stable? | Can be, and LSD needs stable digit passes |
| In-place? | Usually not in standard form |
| Key requirement | Digit/field representation |

---

# 26. BUCKET SORT

Bucket Sort distributes elements into several buckets according to their value.

Then:

1. sort each bucket,
2. concatenate buckets.

Example values in:

\[
[0,1)
\]

```text id="am5h0d"
0.78
0.17
0.39
0.26
0.72
0.94
0.21
```
```

Possible buckets:

```text id="tyhtva" 0.0–0.1 0.1–0.2 0.2–0.3 ...

```
Distribute values.

---

27. BUCKET SORT EXAMPLE

Input:

```text id="14zy7f"
[0.78,0.17,0.39,0.26,0.72,0.94,0.21]
```
```

Buckets:

```text id="hn2evm" Bucket 1: 0.17

Bucket 2: 0.26, 0.21

Bucket 3: 0.39

Bucket 7: 0.78, 0.72

Bucket 9: 0.94

```
Sort individual buckets:

```text id="mu0tpm"
0.17
0.21,0.26
0.39
0.72,0.78
0.94
```
```

Concatenate:

```text id="11hvhr" [0.17,0.21,0.26,0.39,0.72,0.78,0.94]

```
---

28. BUCKET SORT EXPECTED COMPLEXITY

Under suitable assumptions such as:

- values approximately uniformly distributed,
- number of buckets proportional to \(n\),

expected time can be:

\[
\boxed{O(n)}
\]

But this is an **expected** result relying on distribution assumptions.

---

29. BUCKET SORT WORST CASE

Suppose every element lands in the same bucket.

Then one bucket contains all:

\[
n
\]

elements.

If that bucket uses Insertion Sort:

\[
\boxed{O(n^2)}
\]

worst case.

Therefore Bucket Sort does not generally guarantee linear time.

---

30. BUCKET SORT PROPERTIES

Property	Bucket Sort
Comparison-based overall?	Uses distribution; buckets may use comparisons internally
Expected	Often \(O(n)\) under uniform distribution

```
| Worst | Can be \(O(n^2)\) depending on bucket sort |
| Main assumption | Good distribution across buckets |
| Stable? | Depends on implementation and bucket sorter |

---

# 31. COUNTING VS RADIX VS BUCKET

| Feature | Counting | Radix | Bucket |
|---|---|---|---|
| Main idea | Count key frequencies | Sort digit by digit | Distribute into ranges |
| Typical use | Small integer range | Multi-digit integers/strings | Distributed numeric data |
| Typical time | \(O(n+k)\) | \(O(d(n+k))\) | Expected \(O(n)\) |
| Comparison sort? | No | No | Not purely comparison-based |
| Stable? | Yes if implemented carefully | Depends; LSD needs stable passes | Depends |
| Important condition | Small range | Few digits/manageable radix | Near-uniform distribution |

---

# 32. COMPARISON LOWER BOUND DOES NOT APPLY

Why can Counting Sort achieve:

\[
O(n)
\]

while comparison sorting needs:

\[
\Omega(n\log n)?
\]

Because Counting Sort obtains additional information from the numeric key itself.

It does not need to discover ordering solely through questions like:

\[
A[i]<A[j]?
\]

Therefore its computational model is different.

---

# 33. IMPORTANT DISTINCTION

### Comparison Sort

Can sort arbitrary comparable objects:

```text id="p8gzoa"
numbers
strings
records
custom objects
```
```

as long as comparisons are available.

Counting Sort

Needs structured keys:

```text id="zqf6w8" bounded integers
```

So linear time comes with stronger assumptions.

---

34. DECISION TREE LOWER-BOUND EXAMPLE

For:

$$
n=4
$$

possible permutations:

$$
4!=24
$$

Need at least 24 leaves.

Binary tree height h must satisfy:

$$
2^h\ge24
$$

Now:

$$
2^4=16<24
$$

$$
2^5=32\ge24
$$

Therefore worst-case comparison count at least:

$$
\boxed{5}
$$

for sorting 4 distinct arbitrary elements in the decision-tree model.

---

35. GENERAL EXACT LOWER BOUND

Minimum decision-tree height:

$$
\boxed{\lceil\log_2(n!)\rceil}
$$

Thus any comparison sort has worst-case comparison count at least:

$$
\boxed{\lceil\log_2(n!)\rceil}
$$

Asymptotically:

$$
\Omega(n\log n)
$$

```
---

# 36. IS MERGE SORT ASYMPTOTICALLY OPTIMAL?

Merge Sort worst-case:

\[
O(n\log n)
\]

Comparison lower bound:

\[
\Omega(n\log n)
\]

Therefore:

\[
\boxed{\text{Merge Sort is asymptotically optimal among comparison sorts}}
\]

in worst-case order of growth.

Heap Sort also matches the asymptotic lower bound.

---

# 37. WHAT ABOUT QUICK SORT?

Average:

\[
O(n\log n)
\]

so it is asymptotically optimal on average under typical assumptions.

But worst:

\[
O(n^2)
\]

so it does not give optimal worst-case guarantee.

---

# 38. STABILITY MASTER TABLE SO FAR

Standard forms:

| Algorithm | Stable? |
|---|---|
| Bubble | Yes |
| Selection | No |
| Insertion | Yes |
| Merge | Yes |
| Quick | No |
| Heap | No |
| Counting | Yes, proper implementation |
| Radix | Can be; LSD relies on stable sub-sort |
| Bucket | Depends |

Memory:
```

> **BIM + Counting** are the standard easy stable ones to remember.

---

39. IN-PLACE MASTER TABLE SO FAR

Algorithm	In-place?
Bubble	Yes
Selection	Yes
Insertion	Yes
Merge	No, standard arrays
Quick	Yes, commonly
Heap	Yes
Counting	No
Radix	Usually no
Bucket	Usually uses extra buckets

---

40. PYQ-STYLE QUESTION 1

What is the comparison-based worst-case sorting lower bound?

A. $\Omega(n)$
B. $\Omega(\log n)$
C. $\Omega(n\log n)$
D. $\Omega(n^2)$

Answer:

$$
\boxed{C}
$$

---

41. PYQ-STYLE QUESTION 2

For n distinct elements, how many possible input permutations exist?

$$
\boxed{n!}
$$

---

42. PYQ-STYLE QUESTION 3

Decision-tree height must satisfy:

$$
2^h\ge n!
$$

Therefore:

$$
h\ge\log_2(n!)
$$

Hence:

$$
\boxed{\Omega(n\log n)}
$$

```
\]

---

# 43. PYQ-STYLE QUESTION 4

Counting Sort has:

\[
n=1000,\quad k=500
\]

Time:

\[
O(n+k)
\]

\[
=O(1500)
\]

Asymptotically with \(k=O(n)\):

\[
\boxed{O(n)}
\]

---

# 44. PYQ-STYLE QUESTION 5

Suppose:

\[
n=100
\]

and:

\[
k=10^9
\]

Would standard Counting Sort be attractive?

No.

The \(O(k)\) count array dominates enormously.

---

# 45. PYQ-STYLE QUESTION 6

Why must the sorting algorithm used in LSD Radix Sort be stable?

Because previously established ordering of less-significant digits must be preserved.

\[
\boxed{\text{Stability preserves earlier digit order}}
\]

---

# 46. PYQ-STYLE QUESTION 7
```

Radix Sort processes d digits and uses Counting Sort with radix k.

Complexity:

$$
\boxed{O(d(n+k))}
$$

---

47. PYQ-STYLE QUESTION 8

Bucket Sort puts every element into one bucket and uses Insertion Sort inside it.

Worst-case:

$$
\boxed{O(n^2)}
$$

---

48. PYQ-STYLE QUESTION 9 — TRAP

Statement:

> Every sorting algorithm requires $\Omega(n\log n)$ time.

Answer:

$$
\boxed{\text{False}}
$$

Correct:

> General comparison-based sorting has an $\Omega(n\log n)$ worst-case lower bound.

---

49. PYQ-STYLE QUESTION 10

For 5 distinct elements, minimum worst-case comparisons required by decision-tree lower bound:

$$
\lceil\log_2(5!)\rceil
$$

$$
5!=120
$$

Now:

$$
2^6=64
$$

$$
2^7=128
$$

Therefore:

$$
\boxed{7}
$$

```
\]

---

 #   5 0 .  & þ    G A T E   T R A P S

## Trap 1 — \(n\log n\) lower bound applies to every sort

False.

Only comparison sorting under the standard decision-tree model.

---

## Trap 2 — Counting Sort always runs in \(O(n)\)

Wrong.

Actual:

\[
\boxed{O(n+k)}
\]

It is linear in \(n\) only when \(k=O(n)\) or otherwise suitably bounded.

---

## Trap 3 — Counting Sort is based on comparisons

False.

It uses key values as array indices/frequencies.

---

## Trap 4 — Counting Sort is automatically stable

A carefully implemented version is stable.

A simplistic frequency-output implementation may lose record ordering.

---

## Trap 5 — Radix Sort can use any unstable sort per LSD digit

Wrong.

LSD Radix Sort requires stable digit sorting.

---

## Trap 6 — Bucket Sort always runs in \(O(n)\)

Wrong.

Expected linear behavior requires suitable distribution assumptions.

---

## Trap 7 — Heap Sort beats comparison lower bound

No.

Heap Sort is:
```

```
\[
\Theta(n\log n)
\]

which matches the lower bound.

---

## Trap 8 — \(\log(n!)=\Theta(n)\)

Wrong.

\[
\boxed{\Theta(n\log n)}
\]

---

## Trap 9 — \(n!\) possible outcomes means \(n!\) comparisons

No.

Binary decision tree distinguishes them in logarithmic depth:

\[
\log_2(n!)
\]

---

## Trap 10 — Non-comparison sorts work for arbitrary objects with only `<`

Not necessarily.

They need additional structure in the key domain.

---

# 51. EXAM DECISION RULES

### If arbitrary comparable keys + only comparisons:

Think:

\[
\boxed{\Omega(n\log n)}
\]

lower bound.

---

### If small bounded integer range:

Think:

> Counting Sort.

---

### If multiple fixed-width digits:

Think:

> Radix Sort.
```

```
---

### If values distribute naturally into intervals:

Think:

> Bucket Sort.

---

### If LSD Radix Sort:

Immediately remember:

> **stable digit sort**

---

### If Counting Sort complexity:

Never say just \(O(n)\) without checking \(k\).

Write:

\[
\boxed{O(n+k)}
\]

---

### If decision tree:

Think:

\[
\text{leaves}\ge n!
\]

and:

\[
2^h\ge n!
\]

---

# 52. QUICK RECALL

Try first.

1. Number of permutations of \(n\) distinct keys?
2. Why use a decision tree?
3. Minimum leaves needed?
4. Comparison-sorting lower bound?
5. Is it universal for all sorting algorithms?
6. Counting Sort time?
7. What is \(k\) in Counting Sort?
8. When does Counting Sort become \(O(n)\)?
9. Standard Counting Sort stable?
10. Why does LSD Radix need stability?
11. Radix Sort time?
12. Bucket Sort expected time under good assumptions?
13. Bucket Sort worst case can be?
14. \(\log(n!)\) asymptotic order?
15. Does Counting Sort violate the comparison lower bound?
```

```
---

# ANSWERS

1. \(n!\)
2. To model comparison outcomes.
3. At least \(n!\).
4. \(\Omega(n\log n)\).
5. No.
6. \(O(n+k)\).
7. Key-range size.
8. When \(k=O(n)\).
9. Yes, with proper stable implementation.
10. To preserve ordering produced by earlier less-significant digits.
11. \(O(d(n+k))\).
12. \(O(n)\).
13. \(O(n^2)\), depending on implementation.
14. \(\Theta(n\log n)\).
15. No, because it is not comparison-based.

---

# 53. PRACTICE SET

## Easy

### Q1 — MCQ

Worst-case lower bound for comparison sorting is:

A. \(\Omega(n)\)
B. \(\Omega(\log n)\)
C. \(\Omega(n\log n)\)
D. \(\Omega(n^2)\)

---

### Q2 — MCQ

Counting Sort runs in:

A. \(O(\log n)\)
B. \(O(n+k)\)
C. \(O(n\log n)\) always
D. \(O(n^2)\) always

---

### Q3 — MCQ

LSD Radix Sort requires the per-digit sorting algorithm to be:

A. recursive
B. unstable
C. stable
D. in-place

---

# Medium

### Q4 — NAT

For 4 distinct elements, what is:

\[
```

```
\left\lceil\log_2(4!)\right\rceil?
\]

---

### Q5 — MSQ

Which are non-comparison sorting techniques in their standard forms?

A. Merge Sort
B. Counting Sort
C. Radix Sort
D. Heap Sort

---

### Q6 — MSQ

Which statements about Counting Sort are correct?

A. Time can be \(O(n+k)\).

B. It is appropriate even when \(k\gg n\) in all cases.

C. Stable implementation is possible.

D. It normally requires extra memory.

---

### Q7 — MCQ

Bucket Sort's expected linear behavior usually assumes:

A. every element is identical
B. approximately suitable/uniform distribution into buckets
C. no extra memory
D. comparison-sorting lower bound is false

---

# Hard / Tricky

### Q8 — MSQ

Which statements are true?

A. Merge Sort matches the asymptotic comparison-sorting lower bound.

B. Heap Sort matches the asymptotic comparison-sorting lower bound.

C. Counting Sort contradicts the lower-bound proof.

D. The lower-bound proof assumes sorting decisions are obtained through comparisons.

---

### Q9 — NAT

For 6 distinct elements, decision-tree lower bound on worst-case comparisons is:

\[
\left\lceil\log_2(6!)\right\rceil
\]

Compute it.
```

```
---

# SOLUTIONS

## Q1

\[
\boxed{C}
\]

---

## Q2

\[
\boxed{B}
\]

---

## Q3

LSD processing needs earlier digit order preserved.

\[
\boxed{C}
\]

---

## Q4

\[
4!=24
\]

\[
\log_2 24\approx4.585
\]

Ceiling:

\[
\boxed{5}
\]

---

## Q5

Counting and Radix.

\[
\boxed{B,C}
\]

---

## Q6

A true.

B false.

C true.
```

```
D true.

\[
\boxed{A,C,D}
\]

---

## Q7

\[
\boxed{B}
\]

---

## Q8

A true.

B true.

C false.

D true.

Answer:

\[
\boxed{A,B,D}
\]

---

## Q9

\[
6!=720
\]

Now:

\[
2^9=512
\]

\[
2^{10}=1024
\]

Therefore:

\[
\boxed{10}
\]

---

 #   5 4 .  Ø=ÜÝ   N O T E S   T O   W R I T E

## Comparison Sorting Lower Bound

For \(n\) distinct elements:

\[
```

```
n!
\]

possible permutations.

Binary comparison tree:

\[
2^h\ge n!
\]

Therefore:

\[
h\ge\log_2(n!)
\]

Since:

\[
\log(n!)=\Theta(n\log n)
\]

we get:

\[
\boxed{\Omega(n\log n)}
\]

worst-case comparison lower bound.

---

# Counting Sort

Requirements:

> Integer/discrete keys in manageable range.

Complexity:

\[
\boxed{O(n+k)}
\]

where:

```text id="qbhum1"
n = number of elements
k = key range
```
```

If:

\[k=O(n) \]

then:

\[O(n) \]

Stable:

> **Yes, with cumulative counts + right-to-left placement.**

Usually not in-place.

Radix Sort

Sort digit by digit.

LSD:

```text id="cbmxhm" units tens hundreds ...

```
Requires stable per-digit sorting.

Complexity:

\[
\boxed{O(d(n+k))}
\]

where:

```text id="x5ayhh"
d = number of digits
k = radix/range per digit
```
```

## Bucket Sort

Distribute into buckets !’ sort buckets !’ concatenate.

Expected:

\[ O(n) \]

under suitable distribution assumptions.

Worst:

\[ O(n^2) \]

can occur if all elements land in one bucket and quadratic sorting is used.

## 55. MEMORY TRICKS

### Lower Bound

> **n! possibilities !’ log(n!) questions !’ n log n**

### Counting

> **Small Range? Count, Don't Compare.**

### Radix
```

> **Digit by Digit — Stability Saves History.**

Bucket

> **Spread well !’ speed well.**

If values pile into one bucket, performance collapses.

&β30-SECOND REVISION

1. Comparison sorting models decisions through comparisons.
2. n distinct keys have $n!$ possible orders.
3. Decision-tree leaves "e$n!$.
4. $2^h\ge n!$.
5. Therefore $h=\Omega(n\log n)$.
6. Lower bound applies to general comparison sorting, not every sort.
7. Counting Sort = $O(n+k)$.
8. Counting Sort is good only when range k is manageable.
9. Stable Counting Sort uses cumulative counts and careful placement.
10. Radix Sort sorts digit by digit.
11. LSD Radix requires stable digit sorting.
12. Radix complexity = $O(d(n+k))$.
13. Bucket Sort can be expected $O(n)$ under good distribution.
14. Bucket worst case can become $O(n^2)$.
15. Non-comparison sorts beat $n\log n$ only by exploiting additional key structure.

' Chapter 8 Complete

Next Chapter:

Chapter 9 — Hashing

We’ll cover:

- hash tables and hash functions,
- collisions,
- load factor,
- separate chaining,
- open addressing,
- linear probing,
- primary clustering,
- quadratic probing,
- secondary clustering,

- double hashing,
- successful vs unsuccessful search,
- insertion/deletion,
- tombstones,
- expected $O(1)$ vs worst $O(n)$,
- and GATE numerical insertion-table questions.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

CHAPTER 9 — HASHING

GATE Importance: & & & & &

Hashing is explicitly part of your Algorithms syllabus, and your uploaded master prompt asks for hash tables, collisions, load factor, chaining, open addressing, probing methods, clustering, deletion, rehashing, and expected vs worst-case analysis.

The core idea is:

> **Use a key to directly compute where its value should be stored.**

This can make search, insertion and deletion **expected $O(1)$**.

---

1. WHY HASHING MATTERS

Suppose you need to store:

```text
student ID !' student record
```

If you use a normal unsorted array and search sequentially:

$$O(n)$$

But with a good hash table:

$$\boxed{\text{Expected }O(1)}$$

for operations such as:

- search
- insertion
- deletion

This is why hashing is widely used in:

- dictionaries/maps

- symbol tables
- caches
- database indexing
- compiler symbol tables
- sets

2. WHAT IS A HASH TABLE?

A **hash table** is an array-like structure where a key is converted into an index using a **hash function**.

Suppose table size:

$$ m=10 $$

Key:

$$ 42 $$

Hash function:

$$ h(k)=k\bmod10 $$

Then:

$$ h(42)=2 $$

So key `42` is stored at index:

```
TEXT
2
```

3. HASH FUNCTION

A hash function maps a key to a table index:

$$ \boxed{h(k)\rightarrow[0,m-1]} $$

where:

- k = key
- m = table size

A common example:

$$ \boxed{h(k)=k\bmod m} $$

4. EXAMPLE

Let:

$$ m=7 $$

and:

$$ h(k)=k\bmod7 $$

Keys:

```text
10, 22, 31
```

Calculate:

$$ 10\bmod7=3 $$

$$ 22\bmod7=1 $$

$$ 31\bmod7=3 $$

So:

```text
 1 0  !'   i n d e x   3
 2 2  !'   i n d e x   1
 3 1  !'   i n d e x   3
```

Problem:

`10` and `31` want the same location.

This is called a:

$$ \boxed{\text{collision}} $$

---

5. WHAT IS A COLLISION?

A collision occurs when two different keys produce the same hash-table index.

Formally:

$$ k_1\ne k_2 $$

but:

$$ h(k_1)=h(k_2) $$

Example:

$$ 10\bmod7=3 $$

$$ 31\bmod7=3 $$

Collision is unavoidable in general if key universe is larger than table size.

So we need **collision-resolution techniques**.

---

6. MAIN COLLISION-RESOLUTION METHODS

Two major families:

1. Separate Chaining

Each table position stores a collection/list of keys.

2. Open Addressing

All keys are stored directly inside the table.

If collision occurs, probe another position.

Major open-addressing techniques:

- Linear Probing
- Quadratic Probing
- Double Hashing

7. LOAD FACTOR

One of the most important hashing concepts.

Load factor:

$$\boxed{\alpha=\frac{n}{m}}$$

where:

- n = number of stored keys
- m = number of table slots

8. LOAD-FACTOR EXAMPLE

Suppose:

$$n=7$$

keys in:

$$m=10$$

slots.

Then:

$$\alpha=\frac7{10}=0.7$$

9. WHY LOAD FACTOR MATTERS

As load factor increases:

- collisions become more likely,
- probing chains become longer,
- performance tends to decrease.

In open addressing:

$$\boxed{\alpha<1}$$

must hold because every key occupies a table slot.

For chaining:

$$\alpha$$

can exceed 1 because multiple keys can be stored in each bucket.

10. SEPARATECHAINING

Suppose:

\[m=5 \]

Hash:

\[h(k)=k\bmod5 \]

Insert:

```
TEXT
10, 15, 12, 7, 25
```

Hashes:

```
TEXT
 1 0  !’   0
 1 5  !’   0
 1 2  !’   2
 7    !’   2
 2 5  !’   0
```

Table:

```
TEXT
 0  !’   1 0  !’   1 5  !’   2 5
 1  !’   N U L L
 2  !’   1 2  !’   7
 3  !’   N U L L
 4  !’   N U L L
```

Each slot stores a chain/list.

---

11. CHAINING SEARCH

To search for `25`:

1. Compute:

\[h(25)=0 \]

2. Search chain at index 0:

```
TEXT
 1 0  !’   1 5  !’   2 5
```

---

12. CHAINING COMPLEXITY

With a good hash function and relatively uniform distribution:

Expected chain length:

\[\alpha=\frac nm \]

Expected operation complexity:

$$ \boxed{O(1+\alpha)} $$

If:

$$ \alpha=O(1) $$

then expected:

$$ \boxed{O(1)} $$

---

13. CHAINING WORST CASE

Suppose all keys hash to same slot.

Example:

```TEXT
0 !' k 1 !' k 2 !' k 3 !' . . . !' k n
```

Then searching can require:

$$ \boxed{O(n)} $$

So hashing does **not** guarantee $O(1)$ worst-case under ordinary implementations.

Important:

> **Hash-table operations are usually expected/average $O(1)$, not universally worst-case $O(1)$.**

---

14. OPEN ADDRESSING

In open addressing:

> **Every key is stored directly in the hash-table array.**

No external chains.

If collision occurs, another table slot is examined.

General probe sequence:

$$ h(k,0),h(k,1),h(k,2),\dots $$

where i is probe number.

---

15. OPEN ADDRESSING REQUIREMENT

Because all keys occupy slots:

$$ n\le m $$

Therefore:

$$ \boxed{\alpha\le1} $$

For practical efficient operation, usually:

$$ \alpha<1 $$

with some empty slots remaining.

---

16. LINEAR PROBING

Linear probing uses:

$$ \boxed{h(k,i)=(h(k)+i)\bmod m} $$

So if the original position is occupied, try:

```text
next slot
next slot
next slot
...
```

---

17. LINEAR PROBING EXAMPLE

Let:

$$ m=7 $$

$$ h(k)=k\bmod7 $$

Insert:

```text
10, 17, 24, 31
```

All hash to:

$$ 3 $$

because:

$$ 10\bmod7=3 $$

$$ 17\bmod7=3 $$

etc.

Insert `10`:

```text
index 3
```

Table:

```text
0 -
1 -
2 -
3 10
4 -
5 -
```

```
6 -
```

Insert `17`:

index 3 occupied.

Try:

\[(3+1)\bmod7=4 \]

So:

```TEXT
 4  !’   1 7
```

Insert `24`:

Try:

```TEXT
3 occupied
4 occupied
5 free
```

Place at 5.

Insert `31`:

Place at 6.

Final:

```TEXT
0 -
1 -
2 -
3 10
4 17
5 24
6 31
```

This creates a long consecutive block.

---

18. PRIMARY CLUSTERING

Linear Probing suffers from:

\[\boxed{\text{primary clustering}} \]

A cluster is a consecutive block of occupied cells.

Example:

```TEXT
[ -, -, 10,17,24,31, - ]
```

New keys hashing anywhere near this cluster tend to extend it.

Cluster grows larger.

Search gets slower.

Memory:

> **Linear !' Long Lines !' Primary Clusters**

---

19. SEARCH IN LINEAR PROBING

To search key `24`:

1. Calculate home position.
2. If not equal, keep probing.
3. Stop when:
   - key found, or
   - genuinely empty slot encountered.

Why can we stop at an empty slot?

Because if the key had been inserted earlier using the same probe sequence, it would have occupied that empty position before going farther.

---

20. DELETION PROBLEM IN OPEN ADDRESSING

Suppose:

```text
i n d e x   3  !'   1 0
i n d e x   4  !'   1 7
i n d e x   5  !'   2 4
```

Delete `17`.

If we simply mark index 4 as completely EMPTY:

```text
3  !'   1 0
4  !'   E M P T Y
5  !'   2 4
```

Now search for `24` starts at 3:

```text
3  !'   n o t   2 4
4  !'   E M P T Y
```

Algorithm may incorrectly stop.

Thus ordinary empty deletion breaks the probe chain.

---

21. TOMBSTONE / DELETED MARKER

Instead of marking deleted slot as `EMPTY`, mark:

```text
DELETED
```

or:

```text
TOMBSTONE
```

Then search continues through it.

Example:

```text
3  !'   1 0
4  !'   D E L E T E D
5  !'   2 4
```

Search for `24`:

```text
3  !'   n o
4  !'   d e l e t e d ,   c o n t i n u e
5  !'   f o u n d
```

---

22. INSERTION WITH TOMBSTONES

A deleted slot may generally be reused during insertion.

But search logic must distinguish:

```text
EMPTY
```

from:

```text
DELETED
```

Because:

- EMPTY means probe sequence can stop.
- DELETED means search must continue.

---

23. QUADRATIC PROBING

Instead of checking consecutive positions:

\[h(k),h(k)+1,h(k)+2,\dots \]

Quadratic probing uses jumps based on squares.

Simple form:

$$ \boxed{ h(k,i)=(h(k)+i^2)\bmod m } $$

More general:

$$ h(k,i)= (h(k)+c_1i+c_2i^2)\bmod m $$

24. QUADRATIC PROBING EXAMPLE

Suppose:

$$ h(k)=3,\quad m=11 $$

Probe sequence:

For $i=0$:

$$ 3 $$

For $i=1$:

$$ 3+1^2=4 $$

For $i=2$:

$$ 3+4=7 $$

For $i=3$:

$$ 3+9=12\bmod11=1 $$

So:

```TEXT
 3  !′   4  !′   7  !′   1  !′   . . .
```

Instead of one long consecutive run.

25. DOES QUADRATIC PROBING REMOVE PRIMARY CLUSTERING?

Yes, it reduces primary clustering significantly because probes do not simply move one slot at a time.

But it suffers from another problem:

$$ \boxed{\text{secondary clustering}} $$

26. SECONDARY CLUSTERING

Keys with the same initial hash value:

$$ h(k_1)=h(k_2) $$

follow the same quadratic probe sequence.

Example:

Both start at 3:

```TEXT
3 !′ 4 !′ 7 !′ 1 !′ . . .
```

Therefore keys sharing the same home bucket still cluster together.

This is secondary clustering.

---

27. LINEAR VS QUADRATIC CLUSTERING

Linear Probing

Suffers from:

$$ \boxed{\text{Primary clustering}} $$

Quadratic Probing

Avoids/reduces primary clustering but suffers from:

$$ \boxed{\text{Secondary clustering}} $$

---

28. DOUBLE HASHING

Double hashing uses a second hash function to determine jump size.

General form:

$$ \boxed{ h(k,i)= (h_1(k)+i\,h_2(k))\bmod m } $$

This produces different probe sequences for different keys.

---

29. DOUBLE HASHING EXAMPLE

Let:

$$ m=11 $$

$$ h_1(k)=k\bmod11 $$

$$ h_2(k)=1+(k\bmod10) $$

Suppose:

$$ k=22 $$

Then:

$$ h_1(22)=0 $$

$$ h_2(22)=1+2=3 $$

Probe sequence:

$$ 0 $$

$$ 3 $$

$$ 6 $$

$$ 9 $$

$$ 1 $$

...

using modulo 11.

30. WHY DOUBLE HASHING IS BETTER

Different keys can have:

- different starting positions,
- different step sizes.

This greatly reduces clustering.

Among common open-addressing methods, Double Hashing generally provides better distribution.

31. IMPORTANT CONDITION FOR SECOND HASH

The step size must allow the probe sequence to reach all table positions.

A common requirement is:

$$ \gcd(h_2(k),m)=1 $$

If table size m is prime and step size is nonzero and less than m, this is easier to satisfy.

Otherwise probe sequence may cycle through only a subset of slots.

32. EXAMPLE OF BAD STEP SIZE

Suppose:

$$ m=10 $$

and:

$$ h_2(k)=2 $$

Then probe:

```
TEXT
0,2,4,6,8,0,...
```

Only even indices are visited.

Odd positions are never checked.

So even with empty slots available, insertion can fail.

Therefore step size and table size must be chosen carefully.

33. OPEN ADDRESSING COMPARISON

Method	Probe	Main issue
Linear	$h+i$	Primary clustering
Quadratic	$h+i^2$	Secondary clustering
Double Hashing	$h_1+i h_2$	Better clustering behavior

All expressions are taken modulo table size.

34. LOAD FACTOR IN OPEN ADDRESSING

As:

$$ \alpha\rightarrow 1 $$

table becomes nearly full.

Then probe lengths become large.

Therefore operation performance deteriorates sharply.

Practical implementations typically resize before reaching full occupancy.

35. EXPECTED UNSUCCESSFUL SEARCH — LINEAR PROBING

Under classical uniform-hashing approximations for linear probing, expected probes are related to load factor.

Useful textbook results:

Successful search:

$$ \boxed{ \approx \frac{1}{2} \left(1+\frac{1}{1-\alpha} \right) } $$

Unsuccessful search:

$$ \boxed{ \approx \frac{1}{2} \left(1+\frac{1}{(1-\alpha)^2} \right) } $$

These formulas apply under standard idealized assumptions.

For GATE, more important than blindly memorizing them is understanding:

> **As α approaches 1, probe cost rises rapidly.**

36. UNIFORM HASHING APPROXIMATION

For open addressing under uniform hashing:

Expected unsuccessful probes are roughly:

$$ \boxed{\frac{1}{1-\alpha}} $$

Expected successful search has a related logarithmic expression:

$$ \boxed{ \frac{1}{\alpha}\ln\frac{1}{1-\alpha} } $$

These depend on the model.

Important:

> **Do not mix formulas for linear probing with formulas for uniform hashing.**

If an exact formula is needed, inspect the assumptions in the question.

---

37. HASHING EXPECTED VS WORST CASE

With a good hash function and controlled load factor:

```TEXT
S e a r c h       !'  e x p e c t e d  O ( 1 )
I n s e r t       !'  e x p e c t e d  O ( 1 )
D e l e t e       !'  e x p e c t e d  O ( 1 )
```

Worst case:

$$ \boxed{O(n)} $$

Example:

- all keys collide,
- long chain,
- long probe sequence.

---

38. WHAT MAKES A GOOD HASH FUNCTION?

A good hash function should:

1. be fast to compute,
2. distribute keys uniformly,
3. minimize systematic collisions,
4. use information from the whole key where relevant.

A bad hash function can destroy hash-table performance.

---

39. MODULO METHOD

Common:

$$ h(k)=k\bmod m $$

Choice of m matters.

Prime table sizes are often useful because they can reduce undesirable patterns, especially with structured keys and probing schemes.

But:

> **“Prime is always mandatory” is too strong.**

It depends on the hashing method.

40. DIRECT ADDRESSING VS HASHING

Suppose keys are only:

$$ 0,1,2,\ldots,999 $$

You could use an array of 1000 slots directly:

```text
index = key
```

This is **direct addressing**.

No collisions.

But if universe is enormous:

$$ 0\ldots10^{12} $$

and only 100 keys are present, direct-address table wastes huge space.

Hashing maps large key universe into smaller table.

41. HASH TABLE VS DIRECT ADDRESS TABLE

Feature	Direct Addressing	Hashing
Index	key itself	hash(key)
Collisions	No	Possible
Space	proportional to universe	proportional to table size
Useful when	key universe small	universe large

42. CHAINING VS OPEN ADDRESSING

Property	Chaining	Open Addressing
Storage	buckets + lists	table itself
$\alpha>1$?	Yes	No
Deletion	Easier	Tombstones needed
Extra pointers/storage	Yes	Less external structure
Cache locality	Often poorer	Often better
Collision handling	chain	probing

43. REHASHING / RESIZING

When load factor becomes too high:

1. allocate larger table,
2. choose table size/hash setup,
3. insert all existing keys again.

This is called:

$$ \boxed{\text{rehashing}} $$

Important:

> **You cannot usually just copy entries into identical indices because new table size changes hash values.**

---

44. WHY REHASH?

To reduce:

- collisions,
- long probe sequences,
- long chains.

Although one rehash can cost:

$$ O(n) $$

it happens occasionally.

This can still support efficient **amortized** operation costs.

---

45. HASHING WITH DUPLICATE KEYS

Whether duplicates are allowed depends on data structure semantics.

Set

Usually one copy only.

Map

One key usually maps to one value; inserting same key may update value.

Multimap/multiset

Duplicates may be stored.

This is implementation-specific, not a core hash-function property.

---

46. HASHING AND ORDER

A standard hash table generally does **not** preserve sorted key order.

Therefore operations such as:

> **find smallest key**

are not naturally $O(1)$.

Hashing is optimized for key-based lookup, not ordering.

---

47. GATE NUMERICAL — LINEAR PROBING

Let:

$$ m=10 $$

$$ h(k)=k\bmod10 $$

Insert:

```text
23, 43, 13, 27
```

23

$$ 23\bmod10=3 $$

Place:

```text
 3  !′   2 3
```

43

$$ 43\bmod10=3 $$

3 occupied.

Try 4:

```text
 4  !′   4 3
```

13

Home:

$$ 3 $$

Probe:

```text
3 occupied
4 occupied
5 free
```

Place:

```TEXT
 5  !′   1 3
```

27

\[27\bmod10=7 \]

Place at 7.

Final:

```TEXT
0 -
1 -
2 -
3 23
4 43
5 13
6 -
7 27
8 -
9 -
```

---

48. GATE NUMERICAL — QUADRATIC PROBING

Let:

\[m=11 \]

\[h(k)=k\bmod11 \]

Probe rule:

\[(h(k)+i^2)\bmod11 \]

Insert:

```TEXT
22, 33, 44
```

All have home:

\[0 \]

22

\[i=0\rightarrow0 \]

Place at 0.

33

0 occupied.

\[i=1: 0+1=1 \]

Place at 1.

44

0 occupied.

$i=1$: 1 occupied.

$i=2$:

$$ 0+4=4 $$

Place at 4.

Final:

```TEXT
0  !'   2 2
1  !'   3 3
4  !'   4 4
```

---

49. GATE NUMERICAL — DOUBLE HASHING

Suppose:

$$ m=7 $$

$$ h_1(k)=k\bmod7 $$

$$ h_2(k)=1+(k\bmod5) $$

Insert `10`.

$$ h_1(10)=3 $$

Place at 3.

Now insert `17`.

$$ h_1(17)=3 $$

collision.

Second hash:

$$ h_2(17)=1+(17\bmod5) $$

$$ =1+2=3 $$

Next probe:

$$ (3+1\cdot3)\bmod7=6 $$

Place at 6.

---

50. SUCCESSFUL SEARCH COST

Suppose linear probing table:

```TEXT
i n d e x   3  !'   2 3
i n d e x   4  !'   4 3
i n d e x   5  !'   1 3
```

Search for `13`.

Home:

\[3 \]

Probes:

```
TEXT
 3  !′   2 3
 4  !′   4 3
 5  !′   1 3
```

Number of probes:

\[\boxed{3} \]

GATE may ask exactly this.

---

51. UNSUCCESSFUL SEARCH COST

Same table.

Search for `53`.

Home:

\[3 \]

Probe:

```
TEXT
3 occupied
4 occupied
5 occupied
6 EMPTY
```

Stop at empty.

Probes:

\[\boxed{4} \]

depending on whether the final empty-slot probe is counted—which it usually is when counting table probes.

---

52. CLUSTERING EXAMPLE

Suppose linear probing:

```
TEXT
keys: 10,20,30,40
m = 10
```

All hash:

\[0 \]

They occupy:

```text
0,1,2,3
```

Now a new key hashes to 1.

It joins the same block.

Thus cluster grows even though its home location differs from original collision group.

That is why it is called **primary** clustering.

---

53. WHY QUADRATIC STILL CLUSTERS

If two keys have the same home:

$$ h(k_1)=h(k_2)=3 $$

they both use:

```text
3
4
7
1
...
```

Thus same-home keys follow same path.

That's:

$$ \boxed{\text{secondary clustering}} $$

---

54. PYQ-STYLE QUESTION 1

If:

$$ n=80,\quad m=100 $$

load factor is:

$$ \alpha=\frac{80}{100} $$

$$ \boxed{0.8} $$

---

55. PYQ-STYLE QUESTION 2

Can separate chaining have:

$$ \alpha>1? $$

Yes.

Example:

$$ n=20,\quad m=10 $$

$$ \alpha=2 $$

Possible because each slot can hold multiple keys.

56. PYQ-STYLE QUESTION 3

Can open addressing have:

$$ n>m? $$

No.

All keys must occupy table cells.

Therefore:

$$ \boxed{\alpha\le1} $$

57. PYQ-STYLE QUESTION 4

Which suffers from primary clustering?

A. Chaining B. Linear probing C. Quadratic probing D. Double hashing

Answer:

$$ \boxed{B} $$

58. PYQ-STYLE QUESTION 5

Which suffers mainly from secondary clustering?

$$ \boxed{\text{Quadratic probing}} $$

59. PYQ-STYLE QUESTION 6

Why is simple deletion unsafe in linear probing?

Because replacing an occupied slot with ordinary EMPTY may terminate later searches prematurely.

Use a:

$$ \boxed{\text{tombstone/deleted marker}} $$

60. PYQ-STYLE QUESTION 7

Hash table search time is often described as:

$$ O(1) $$

What is more precise?

$$ \boxed{\text{Expected }O(1)} $$

under good hashing/load assumptions.

Worst case:

$$ \boxed{O(n)} $$

61. PYQ-STYLE QUESTION 8

Which technique generally gives the best resistance to clustering among:

- linear probing,
- quadratic probing,
- double hashing?

$$ \boxed{\text{Double hashing}} $$

assuming good hash functions and table parameters.

62. & þGATE TRAPS

Trap 1 — Hashing guarantees $O(1)$ worst-case

Wrong for standard hash tables.

Expected:

$$ O(1) $$

Worst:

$$ O(n) $$

Trap 2 — Collisions mean hash function is invalid

Wrong.

Collisions are normal and often unavoidable.

Trap 3 — Open addressing allows $\alpha>1$

Impossible.

$$ n\le m $$

Trap 4 — Chaining requires $\alpha<1$

Wrong.

Chains allow:

$$ \alpha>1 $$

Trap 5 — Linear probing has secondary clustering only

Main issue:

$$ \boxed{\text{primary clustering}} $$

Trap 6 — Quadratic probing has no clustering

It suffers from:

$$ \boxed{\text{secondary clustering}} $$

Trap 7 — Delete by simply making slot empty

Can break probe sequence.

Use tombstone.

Trap 8 — Empty and deleted mean same thing

No.

`EMPTY` can terminate unsuccessful search.

`DELETED` cannot.

Trap 9 — Double-hashing step size can be arbitrary

Bad step size may visit only part of table.

Need suitable relationship with table size.

Trap 10 — Hash tables maintain keys in sorted order

Standard hash tables do not.

63. EXAM DECISION RULES

If asked load factor:

Immediately write:

$$ \boxed{\alpha=n/m} $$

If collision + linked list:

Think:

> **Separate Chaining.**

If:

```text
h+i
```

Think:

> **Linear Probing.**

If:

\[h+i^2 \]

Think:

> **Quadratic Probing.**

If:

\[h_1+i h_2 \]

Think:

> **Double Hashing.**

If linear probing:

Remember:

> **Primary clustering**

If quadratic:

Remember:

> **Secondary clustering**

If deletion in open addressing:

Remember:

> **Tombstone**

If expected complexity:

Good hash + controlled load factor:

\[O(1) \]

Worst:

$$ O(n) $$

64. QUICK RECALL

Try mentally.

1. What is hashing?
2. What is a collision?
3. Formula for load factor?
4. Can chaining have $\alpha>1$?
5. Can open addressing have $\alpha>1$?
6. Linear probing formula?
7. Main problem with linear probing?
8. Quadratic probing's main clustering issue?
9. Double hashing formula?
10. Why use tombstones?
11. Expected search complexity under good hashing?
12. Worst-case search complexity?
13. What does rehashing mean?
14. Does hash table naturally preserve sorted order?
15. Why must double-hash step size be chosen carefully?

ANSWERS

1. Mapping keys to table indices using hash function.
2. Different keys map to same slot.
3. $\alpha=n/m$.
4. Yes.
5. No.
6. $(h(k)+i)\bmod m$.
7. Primary clustering.
8. Secondary clustering.
9. $(h_1(k)+ih_2(k))\bmod m$.
10. To preserve search probe chains after deletion.
11. Expected $O(1)$.
12. $O(n)$.
13. Resize/rebuild table and hash entries again.
14. No.
15. To ensure probe sequence reaches enough/all table cells.

65. PRACTICE SET

Easy

Q1 — NAT

A hash table has:

$$ m=20 $$

slots and contains:

$$ n=15 $$

keys.

Find load factor.

Q2 — MCQ

Which collision-resolution method uses linked lists at each table position?

A. Linear probing B. Quadratic probing C. Chaining D. Double hashing

---

Q3 — MCQ

Primary clustering is mainly associated with:

A. Linear probing B. Quadratic probing C. Separate chaining D. Binary Search

---

Medium

Q4 — MSQ

Which statements are correct?

A. Chaining can have $\alpha>1$.

B. Open addressing requires enough empty slots for keys.

C. Standard hashing guarantees $O(1)$ worst-case.

D. Double hashing uses two hash functions.

Q5 — NAT

Let:

$$ m=10 $$

$$ h(k)=k\bmod10 $$

Using linear probing, insert:

```text
12, 22, 32
```

At what index is `32` stored?

Q6 — MCQ

In open addressing, why is a tombstone used?

A. To sort keys B. To mark a slot permanently unusable C. To allow searches to continue through a deleted position D. To eliminate collisions

---

Q7 — MSQ

Which can have worst-case search:

$$O(n)?$$

A. Chaining B. Linear probing C. Quadratic probing D. Double hashing

---

Hard / Tricky

Q8 — NAT

Let:

$$m=11$$

$$h(k)=k\bmod11$$

Quadratic probing uses:

$$h(k,i)=(h(k)+i^2)\bmod11$$

Keys `22` and `33` are already inserted at indices 0 and 1.

At what index will `44` be placed?

Q9 — MSQ

Which statements are true?

A. EMPTY and DELETED markers have identical search meaning.

B. A deleted slot may often be reused during insertion.

C. A search through open addressing may continue past DELETED.

D. An unsuccessful search can generally stop at a truly EMPTY slot.

SOLUTIONS

Q1

$$\alpha=\frac{15}{20}$$

$$\boxed{0.75}$$

Q2

$$\boxed{C}$$

Q3

$$\boxed{A}$$

Q4

A true.

B true.

C false.

D true.

Answer:

$$\boxed{A,B,D}$$

Q5

All hash to index 2.

`12`:

```
TEXT
2
```

`22`:

```
TEXT
3
```

`32`:

```
TEXT
4
```

Answer:

$$\boxed{4}$$

Q6

$$\boxed{C}$$

Q7

All four can degrade to:

$$O(n)$$

under bad conditions.

Answer:

$$\boxed{A,B,C,D}$$

Q8

`44` hashes to:

$$0$$

Probe:

$i=0$:

$$0$$

occupied.

$i=1$:

$$1$$

occupied.

$i=2$:

$$0+4=4$$

Answer:

$$\boxed{4}$$

Q9

A false.

B true.

C true.

D true.

Answer:

$$\boxed{B,C,D}$$

66. Ø=ÜÓTES TO WRITE

Hashing

Hash function:

$$h(k)\rightarrow[0,m-1]$$

Common:

$$h(k)=k\bmod m$$

Collision:

$$k_1\ne k_2$$

but:

$$ h(k_1)=h(k_2) $$

Load Factor

$$ \boxed{\alpha=\frac nm} $$

Chaining:

$$ \alpha $$

may exceed 1.

Open addressing:

$$ \alpha\le1 $$

Separate Chaining

Each table index stores a chain.

Expected:

$$ O(1+\alpha) $$

Worst:

$$ O(n) $$

Linear Probing

$$ \boxed{ h(k,i)=(h(k)+i)\bmod m } $$

Problem:

$$ \boxed{\text{Primary clustering}} $$

Quadratic Probing

$$ \boxed{ h(k,i)=(h(k)+i^2)\bmod m } $$

Problem:

$$ \boxed{\text{Secondary clustering}} $$

Double Hashing

$$ \boxed{ h(k,i)= (h_1(k)+ih_2(k))\bmod m } $$

Better clustering behavior.

Choose step size so probe sequence covers table effectively.

Deletion

Open addressing should use:

\[\boxed{\text{DELETED/TOMBSTONE}} \]

not ordinary EMPTY.

Complexity

Good hashing:

```TEXT
 S e a r c h    !'   e x p e c t e d   O ( 1 )
 I n s e r t    !'   e x p e c t e d   O ( 1 )
 D e l e t e    !'   e x p e c t e d   O ( 1 )
```

Worst:

\[\boxed{O(n)} \]

Rehashing

When load factor is high:

```TEXT
Create larger table
Recompute hashes
Reinsert keys
```

---

67. MEMORY TRICKS

Load Factor

> **Load = keys / slots**

\[\alpha=n/m \]

Collision Methods

> **L-Q-D**

```TEXT
Linear
Quadratic
Double
```

Clustering

> **Linear = Primary**

> **Quadratic = Secondary**

Deletion

> **Don't Empty — Tombstone**

Complexity

> **Hashing hopes for 1, can fall to n.**

Expected:

\[O(1) \]

Worst:

\[O(n) \]

&β30-SECOND REVISION

1. Hashing maps key !’ table index.
2. Collision = two keys map to same slot.
3. Load factor = \(n/m\).
4. Chaining can have \(\alpha>1\).
5. Open addressing cannot store more keys than slots.
6. Linear probing = \(h+i\).
7. Linear probing !’ primary clustering.
8. Quadratic probing = \(h+i^2\).
9. Quadratic !’ secondary clustering.
10. Double hashing = \(h_1+i h_2\).
11. Double hashing usually reduces clustering best.
12. Open-address deletion needs tombstone.
13. EMPTY stops search; DELETED does not.
14. Good hashing gives expected \(O(1)\).
15. Worst case remains \(O(n)\).
16. High load factor hurts performance.
17. Rehashing rebuilds into larger table.
18. Hash tables do not naturally maintain sorted order.

' Chapter 9 Complete

Next Chapter:

Chapter 10 — Divide and Conquer as an Algorithm Design Technique

We'll connect everything you already learned and cover:

- divide–conquer–combine,
- recognizing D&C problems,
- Binary Search,
- Merge Sort,
- Quick Sort,
- recurrence formation,
- recursion-tree interpretation,
- when Divide and Conquer works,
- when it does not,
- D&C vs Dynamic Programming,
- and GATE algorithm-design identification questions.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

CHAPTER 10 — DIVIDE AND CONQUER

GATE Importance: & & & & &

Your uploaded Algorithms course plan explicitly includes **Divide and Conquer** as one of the required design techniques, along with Binary Search, Merge Sort, Quick Sort, recurrences, and complexity derivations.

The core idea is:

> **Break one big problem into smaller similar problems, solve them recursively, then combine their answers.**

---

1. WHY THIS TOPIC MATTERS

Divide and Conquer is not one single algorithm.

It is an **algorithm-design technique**.

Many famous algorithms use it:

- Binary Search
- Merge Sort
- Quick Sort

- Maximum/minimum using recursion
- Some multiplication and geometric algorithms

It is also the foundation for understanding:

- recurrence relations
- recursion trees
- Master Theorem

---

2. THE THREE STEPS

A Divide-and-Conquer algorithm usually has:

1. Divide

Split the problem into smaller subproblems.

2. Conquer

Solve those subproblems recursively.

3. Combine

Combine the smaller answers to obtain the original answer.

Memory trick:

> **DCC = Divide !' Conquer !' Combine**

---

3. SIMPLE EXAMPLE

Suppose we want to sort:

```text
[8, 3, 6, 2]
```

Merge Sort:

Divide

```text
[8,3]      [6,2]
```

Again:

```text
[8] [3]    [6] [2]
```

Conquer

Single-element lists are already sorted.

Combine

```text
[3,8]

[2,6]
```

Then:

```text
[2,3,6,8]
```

This is classic Divide and Conquer.

---

4. GENERAL RECURRENCE

A common Divide-and-Conquer recurrence is:

$$ \boxed{T(n)=aT(n/b)+f(n)} $$

where:

$$ a=\text{number of subproblems} $$

$$ n/b=\text{size of each subproblem} $$

$$ f(n)=\text{divide/combine work} $$

This is exactly the form you studied for Master Theorem.

---

5. EXAMPLE

Suppose:

$$ T(n)=2T(n/2)+n $$

Interpretation:

- 2 subproblems
- each size $n/2$
- n work outside recursion

This is the Merge Sort recurrence.

---

6. BINARY SEARCH AS DIVIDE AND CONQUER

Binary Search:

1. Look at middle element.
2. Decide which half may contain the key.
3. Recur/search only that half.

Example:

```text
[2,5,8,12,16,20,25]
```

Search `20`.

Middle:

```text
12
```

Since:

$$ 20>12 $$

discard left half.

Remaining:

```text
[16,20,25]
```

This is division of the search space.

---

7. BINARY SEARCH RECURRENCE

Only one half is recursively explored.

Therefore:

$$ T(n)=T(n/2)+\Theta(1) $$

Result:

$$ \boxed{\Theta(\log n)} $$

---

8. IMPORTANT POINT

Binary Search divides into two halves physically, but recursively solves only **one** of them.

So recurrence is:

$$ T(n)=T(n/2)+1 $$

not:

$$ 2T(n/2)+1 $$

This is a common conceptual trap.

---

9. MERGE SORT AS DIVIDE AND CONQUER

Merge Sort:

Divide

Split into two halves.

Conquer

Sort both halves recursively.

Combine

Merge two sorted halves.

Recurrence:

$$ T(n)=2T(n/2)+\Theta(n) $$

Therefore:

$$ \boxed{\Theta(n\log n)} $$

---

10. QUICK SORT AS DIVIDE AND CONQUER

Quick Sort:

Divide

Partition around pivot.

Conquer

Sort left and right partitions recursively.

Combine

Essentially nothing substantial is required after recursive calls because partitioning already places pivot correctly.

---

11. QUICK SORT RECURRENCE

If pivot creates sizes:

$$ k $$

and:

$$ n-k-1 $$

then:

$$ \boxed{ T(n)=T(k)+T(n-k-1)+\Theta(n) } $$

Partition work:

$$ \Theta(n) $$

---

12. BALANCED QUICK SORT

If:

$$ k\approx n/2 $$

then:

$$ T(n)=2T(n/2)+n $$

Therefore:

$$ \boxed{\Theta(n\log n)} $$

13. UNBALANCED QUICK SORT

If pivot repeatedly creates:

$$ 0 $$

and:

$$ n-1 $$

then:

$$ T(n)=T(n-1)+n $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

So Divide and Conquer does **not automatically imply**:

$$ O(n\log n) $$

Balance matters.

14. DIVIDE AND CONQUER RECURSION TREE

For:

$$ T(n)=2T(n/2)+n $$

tree:

```TEXT
                    n
                 /     \
              n/2       n/2
             /  \       /  \
           n/4 n/4    n/4 n/4
```

Each level total work:

$$ n $$

Depth:

$$ \log n $$

Total:

$$ \boxed{n\log n} $$

15. DIVIDE SIZE MATTERS

Suppose:

$$ T(n)=T(n/2)+1 $$

Depth:

$$ \log n $$

Now compare:

$$ T(n)=T(n-1)+1 $$

Depth:

$$ n $$

This shows why reducing problem by a **constant fraction** is much more powerful than reducing it by only 1.

---

16. DECREASE-BY-ONE VS DIVIDE-BY-TWO

Decrease by one

```text
n
n-1
n-2
...
1
```

Number of levels:

$$ \Theta(n) $$

Divide by two

```text
n
n/2
n/4
...
1
```

Number of levels:

$$ \Theta(\log n) $$

Memory:

> **Subtract !' linear depth**

> **Divide !' logarithmic depth**

---

17. WHEN DIVIDE AND CONQUER WORKS WELL

It is especially useful when:

1. Problem can be split into smaller similar instances.
2. Subproblems can be solved mostly independently.
3. Combination work is manageable.
4. Recursive subdivision reduces size significantly.

18. INDEPENDENT SUBPROBLEMS

This is a key idea.

Suppose a problem splits into:

```text
left half
right half
```

and solving left does not repeatedly require recomputing right.

That is a natural Divide-and-Conquer situation.

Merge Sort is a perfect example.

19. DIVIDE AND CONQUER VS DYNAMIC PROGRAMMING — FIRST LOOK

This is extremely important for later chapters.

Divide and Conquer

Subproblems are generally:

$$ \boxed{\text{independent/non-overlapping}} $$

Dynamic Programming

Subproblems generally:

$$ \boxed{\text{overlap}} $$

and are solved repeatedly unless stored.

20. EXAMPLE — MERGE SORT

Split:

```text
left half
right half
```

They contain different elements.

Subproblems do not overlap.

Therefore:

> **Divide and Conquer.**

21. EXAMPLE — NAIVE FIBONACCI

$$ F(n)=F(n-1)+F(n-2) $$

To compute:

\[F(n-1) \]

we calculate:

\[F(n-2) \]

again.

Call tree:

```TEXT
            F(n)
          /       \
      F(n-1)      F(n-2)
      /   \
 F(n-2)  F(n-3)
```

Notice:

\[F(n-2) \]

appears repeatedly.

This is **overlapping subproblems**.

That strongly suggests:

> **Dynamic Programming.**

---

22. D&C VS DP

Feature	Divide & Conquer	Dynamic Programming
Subproblems	Usually independent	Overlapping
Repeated computation	Usually little	Often large without storage
Main technique	recursive splitting	store subproblem answers
Examples	Merge, Quick, Binary Search	Knapsack, LCS, MCM

We will study DP properly later.

---

23. DIVIDE AND CONQUER VS GREEDY

Greedy usually:

> **Makes one locally best choice and commits to it.**

Divide and Conquer:

> **Splits into smaller problem instances and solves them.**

Example:

Merge Sort

Does not choose “locally best element.”

It recursively solves both halves.

Activity Selection

Chooses an activity greedily.

Different design philosophy.

---

24. MAXIMUM ELEMENT USING DIVIDE AND CONQUER

Suppose:

```text
A = [8, 3, 11, 2, 7, 15]
```

Split into two halves.

Find maximum in left:

```text
[8,3,11]
```

Result:

$$ 11 $$

Find maximum in right:

```text
[2,7,15]
```

Result:

$$ 15 $$

Combine:

$$ \max(11,15)=15 $$

---

25. RECURRENCE FOR MAXIMUM

If divide into two halves:

$$ T(n)=2T(n/2)+\Theta(1) $$

Using Master Theorem:

$$ n^{\log_2 2}=n $$

and:

$$ f(n)=1 $$

Case 1:

$$\boxed{\Theta(n)}$$

No asymptotic improvement over simple linear scan.

---

26. IMPORTANT LESSON

Using Divide and Conquer does not automatically make an algorithm faster.

Maximum-finding still needs:

$$\Omega(n)$$

because every element must effectively be considered.

So:

> **Design technique "`automatic speed improvement.**

---

27. FIND MINIMUM AND MAXIMUM TOGETHER

A classic Divide-and-Conquer problem is finding both:

$$\min$$

and:

$$\max$$

of an array.

Naive approach:

For every element:

```text
compare with min
compare with max
```

Approximately:

$$2n-2$$

comparisons.

---

28. PAIRING / DIVIDE IDEA

Compare elements in pairs.

For each pair:

1. compare pair members,
2. smaller compared with global minimum,
3. larger compared with global maximum.

Approximately 3 comparisons for each pair.

For even n:

$$ \boxed{\frac{3n}{2}-2} $$

comparisons.

This is a famous result.

---

29. MIN-MAX COMPARISON RESULT

For even n:

$$ \boxed{\frac{3n}{2}-2} $$

comparisons can find both minimum and maximum.

Compared with naive:

$$ 2n-2 $$

This is an important comparison-count optimization.

---

30. WHY?

Suppose pair:

```
TEXT
(a,b)
```

First compare:

$$ a<b? $$

Then only:

- smaller candidate needs comparison with global min,
- larger candidate needs comparison with global max.

So each pair uses:

$$ 3 $$

comparisons.

There are:

$$ n/2 $$

pairs.

With initialization adjustments:

$$ \frac{3n}{2}-2 $$

for even n.

---

31. MULTIPLICATION EXAMPLE — CONCEPTUAL

Classical multiplication of two n-digit numbers uses roughly:

$$ O(n^2) $$

digit operations.

Some advanced Divide-and-Conquer multiplication algorithms can improve this by splitting numbers into parts.

A famous example is Karatsuba:

$$ T(n)=3T(n/2)+O(n) $$

Using Master Theorem:

$$ n^{\log_2 3} $$

approximately:

$$ n^{1.585} $$

So:

$$ \boxed{O(n^{1.585})} $$

This is useful as an example of D&C, though not a central GATE Algorithms syllabus item.

---

32. MASTER THEOREM CONNECTION

For Divide and Conquer:

$$ T(n)=aT(n/b)+f(n) $$

three things determine complexity:

1. Branching factor

$$ a $$

2. Shrink factor

$$ b $$

3. Extra work

$$ f(n) $$

These together determine total recursion-tree work.

---

33. EXAMPLE

$$ T(n)=4T(n/2)+n $$

We know:

$$ n^{\log_2 4}=n^2 $$

So recursion dominates.

Result:

$$ \boxed{\Theta(n^2)} $$

Even though each level divides by 2.

This shows that many recursive branches can overwhelm the benefit of shrinking.

34. EXAMPLE

$$ T(n)=T(n/2)+n $$

Only one branch.

Expansion:

$$ n+\frac n2+\frac n4+\cdots $$

Total:

$$ \boxed{\Theta(n)} $$

Again, D&C complexity depends on both branching and combination work.

35. EXAMPLE

$$ T(n)=8T(n/2)+n^2 $$

Calculate:

$$ n^{\log_2 8}=n^3 $$

Since:

$$ n^2 $$

is smaller:

$$ \boxed{\Theta(n^3)} $$

Recursive branching dominates.

36. COMBINE COST CAN DOMINATE

Suppose:

$$ T(n)=2T(n/2)+n^2 $$

Recursive threshold:

$$ n^{\log_2 2}=n $$

But combine/divide work:

$$ n^2 $$

is larger.

Result:

$$ \boxed{\Theta(n^2)} $$

So inefficient combining can destroy advantages.

37. BALANCED VS UNBALANCED DIVISION

Balanced:

```TEXT
n
% % %    n / 2
% % %    n / 2
```

Depth:

\[O(\log n) \]

Very unbalanced:

```TEXT
n
% % %    1
% % %    n - 1
```

Depth:

\[O(n) \]

Balanced recursion is often desirable.

This is exactly why Quick Sort pivot quality matters.

---

38. DOES EVERY D&C ALGORITHM USE TWO SUBPROBLEMS?

No.

It can use:

- one
- two
- three
- many

Examples:

Binary Search:

\[1 \]

recursive subproblem.

Merge Sort:

\[2 \]

Karatsuba:

\[3 \]

General recurrence:

\[aT(n/b)+f(n) \]

---

39. BASE CASE

Every recursive D&C algorithm needs a base case.

Examples:

Merge Sort

```text
if array size <= 1:
    return
```

Binary Search

```text
if low > high:
    not found
```

Base case prevents infinite recursion.

---

40. RECURSION OVERHEAD

Divide and Conquer can introduce:

- function-call overhead,
- stack usage,
- temporary storage.

Therefore for very small subproblems, practical implementations sometimes switch to a simpler algorithm like Insertion Sort.

This is why hybrid algorithms exist.

---

41. PARALLELISM

An advantage of independent subproblems:

> **They may be solved in parallel.**

Example Merge Sort:

```text
sort left half
sort right half
```

These can potentially execute simultaneously.

This is another conceptual advantage of D&C.

Not every D&C algorithm provides perfect parallelism, though.

---

42. DIVIDE AND CONQUER RECOGNITION PATTERN

If question says:

> **Split problem into similar smaller problems, solve recursively, combine.**

Think:

$$\boxed{\text{Divide and Conquer}}$$

43. RECOGNITION — BINARY SEARCH

If:

> **sorted data + discard half each comparison**

Think:

$$\boxed{\text{Divide and Conquer / Binary Search}}$$

44. RECOGNITION — MERGE SORT

If:

> **divide into equal halves + merge sorted results**

Think:

$$\boxed{\text{Merge Sort}}$$

45. RECOGNITION — QUICK SORT

If:

> **pivot + partition + recursive sorting**

Think:

$$\boxed{\text{Quick Sort}}$$

46. GATE TRAP — QUICK SORT COMBINE

In Quick Sort:

- expensive work is mainly **before recursion** during partition.
- combine stage after recursion is almost trivial.

In Merge Sort:

- divide stage is simple.
- expensive combine stage is merging.

Remember:

```TEXT
 M e r g e  !'  w o r k  w h i l e  C O M B I N I N G
 Q u i c k  !'  w o r k  w h i l e  P A R T I T I O N I N G
```

47. RECURRENCE IDENTIFICATION

Suppose code:

```TEXT
fun(n):
    if n <= 1:
        return

    fun(n/2)
    fun(n/2)

    for i = 1 to n:
        work
```

Recurrence:

\[\boxed{T(n)=2T(n/2)+\Theta(n)} \]

Result:

\[\boxed{\Theta(n\log n)} \]

48. ANOTHER CODE

```TEXT
fun(n):
    if n <= 1:
        return

    fun(n/2)

    for i = 1 to n:
        work
```

Recurrence:

\[T(n)=T(n/2)+n \]

Result:

\[\boxed{\Theta(n)} \]

49. ANOTHER CODE

```TEXT
fun(n):
    if n <= 1:
        return
```

```
    fun(n/2)
    fun(n/2)
```

Recurrence:

$$ T(n)=2T(n/2)+1 $$

Result:

$$ \boxed{\Theta(n)} $$

---

50. WHY NOT $\Theta(\log n)$?

There are:

$$ \log n $$

levels.

But there are many nodes at each level.

Total calls:

$$ 1+2+4+\cdots+n $$

$$ =\Theta(n) $$

This is another classic trap:

> **Recursion depth "`total work.**

---

51. GATE-STYLE DESIGN-TECHNIQUE COMPARISON

Algorithm	Technique
Binary Search	Divide & Conquer
Merge Sort	Divide & Conquer
Quick Sort	Divide & Conquer
Fractional Knapsack	Greedy
Activity Selection	Greedy
0/1 Knapsack	Dynamic Programming
Matrix Chain Multiplication	Dynamic Programming

This recognition table will become very useful after the next chapters.

---

52. DIVIDE AND CONQUER VS BRUTE FORCE

Brute force often:

> **directly tries everything or scans everything.**

D&C tries to exploit structure by splitting.

Example:

Searching sorted array.

Linear Search:

$$ O(n) $$

Binary Search:

$$ O(\log n) $$

Because sorting/order lets us discard half.

---

53. D&C DOES NOT ALWAYSIMPROVE COMPLEXITY

Maximum finding:

Simple:

$$ O(n) $$

Divide-and-Conquer:

$$ O(n) $$

Same asymptotic complexity.

Therefore use D&C because it fits the problem structure, not because it always produces lower complexity.

---

54. GATE TRAPS

& þTrap 1 — D&C always means two recursive calls

False.

Binary Search uses one.

& þTrap 2 — Every D&C algorithm is $O(n\log n)$

False.

Examples:

Binary Search:

$$ O(\log n) $$

Max finding:

$$ O(n) $$

Bad Quick Sort:

$$ O(n^2) $$

& þTrap 3 — Recursion depth determines total time

False.

For:

\[2T(n/2)+1 \]

depth is:

\[\log n \]

but time:

\[n \]

& þTrap 4 — Merge and Quick do the same combine work

False.

Merge performs expensive merge.

Quick performs expensive partition.

& þTrap 5 — D&C and DP are the same

No.

Key distinction:

```text
D & C  !'  i n d e p e n d e n t  s u b p r o b l e m s
D P    !'  o v e r l a p p i n g  s u b p r o b l e m s
```

& þTrap 6 — Quick Sort always makes equal halves

No.

Partition sizes depend on pivot.

& þTrap 7 — Binary Search solves both halves

No.

It discards one half and solves only one.

& þTrap 8 — Any recursive algorithm is Divide and Conquer

Not necessarily.

Recursion is an implementation technique.

Divide and Conquer is an algorithm-design strategy.

Naive Fibonacci is recursive, but its overlapping structure is better handled with DP.

& þTrap 9 — Subtracting 1 and dividing by 2 give similar depth

No.

$$ n\rightarrow n-1 $$

gives:

$$ \Theta(n) $$

depth.

$$ n\rightarrow n/2 $$

gives:

$$ \Theta(\log n) $$

& þTrap 10 — Master Theorem applies to every D&C recurrence

No.

Example:

$$ T(n)=T(n/3)+T(2n/3)+n $$

does not fit the standard equal-subproblem form directly.

---

55. EXAM DECISION RULES

If subproblems are independent:

Think:

> **Divide and Conquer.**

If same subproblems keep reappearing:

Think:

> **Dynamic Programming candidate.**

If one locally optimal choice is made:

Think:

> **Greedy candidate.**

If problem size repeatedly halves:

Think:

$$\log n$$

depth.

If both halves are recursively solved:

Think:

$$2T(n/2)$$

If only one half is selected:

Think:

$$T(n/2)$$

If partition is balanced:

Usually expect logarithmic recursion depth.

If partition becomes $n-1$:

Expect:

$$O(n)$$

depth and possibly:

$$O(n^2)$$

time.

---

56. PYQ-STYLE QUESTION 1

Which technique does Merge Sort use?

A. Greedy B. Dynamic Programming C. Divide and Conquer D. Backtracking

Answer:

$$\boxed{C}$$

---

57. PYQ-STYLE QUESTION 2

Binary Search recurrence:

A.

$$T(n)=2T(n/2)+1$$

B.

$$T(n)=T(n/2)+1$$

C.

$$ T(n)=T(n-1)+1 $$

D.

$$ T(n)=nT(n/2) $$

Answer:

$$ \boxed{B} $$

58. PYQ-STYLE QUESTION 3

Which recurrence corresponds to balanced Merge Sort?

$$ \boxed{T(n)=2T(n/2)+\Theta(n)} $$

Complexity:

$$ \boxed{\Theta(n\log n)} $$

59. PYQ-STYLE QUESTION 4

Quick Sort partitions into:

$$ n/2,\ n/2 $$

each time.

Complexity:

$$ \boxed{\Theta(n\log n)} $$

60. PYQ-STYLE QUESTION 5

Quick Sort partitions into:

$$ 0,\ n-1 $$

each time.

Recurrence:

$$ T(n)=T(n-1)+n $$

Therefore:

$$ \boxed{\Theta(n^2)} $$

61. PYQ-STYLE QUESTION 6

Which technique is more appropriate when subproblems overlap heavily?

A. Divide and Conquer B. Dynamic Programming C. Linear Search D. Hashing

Answer:

$$ \boxed{B} $$

62. PYQ-STYLE QUESTION 7

Recurrence:

$$ T(n)=2T(n/2)+1 $$

Depth:

$$ \Theta(\log n) $$

But number of leaves:

$$ n $$

Therefore time:

$$ \boxed{\Theta(n)} $$

63. PYQ-STYLE QUESTION 8 — MIN/MAX

For even n, minimum comparisons to find both min and max using efficient pairwise method:

$$ \boxed{\frac{3n}{2}-2} $$

Example:

$$ n=8 $$

comparisons:

$$ 12-2 $$

$$ \boxed{10} $$

64. QUICK RECALL

Try without looking.

1. What are three stages of D&C?
2. General recurrence form?
3. Binary Search recurrence?
4. Merge Sort recurrence?
5. Quick Sort's main divide step?
6. Merge Sort's main combine step?
7. Does Binary Search recursively solve both halves?
8. Balanced Quick Sort complexity?
9. Worst Quick Sort complexity?
10. D&C subproblems usually overlap or not?
11. DP subproblems usually overlap or not?
12. Divide-by-two recursion depth?
13. Subtract-one recursion depth?
14. Does recursion automatically mean D&C?
15. Efficient min+max comparisons for even n?

ANSWERS

1. Divide, Conquer, Combine.
2. $aT(n/b)+f(n)$.
3. $T(n/2)+\Theta(1)$.
4. $2T(n/2)+\Theta(n)$.
5. Partition around pivot.
6. Merge sorted halves.
7. No.
8. $\Theta(n\log n)$.
9. $\Theta(n^2)$.
10. Usually independent/non-overlapping.
11. Usually overlapping.
12. $\Theta(\log n)$.
13. $\Theta(n)$.
14. No.
15. $\frac{3n}{2}-2$.

65. PRACTICE SET

Easy

Q1 — MCQ

Which is **not** one of the classic Divide-and-Conquer stages?

A. Divide B. Conquer C. Combine D. Memorize

Q2 — MCQ

Binary Search solves how many recursive halves after each comparison?

A. 0 B. 1 C. 2 D. n

Q3 — MCQ

Balanced Merge Sort recurrence is:

A. $T(n)=T(n-1)+n$

B. $T(n)=T(n/2)+1$

C. $T(n)=2T(n/2)+n$

D. $T(n)=nT(n/2)$

Medium

Q4 — MSQ

Which use Divide and Conquer?

A. Binary Search B. Merge Sort C. Quick Sort D. Fractional Knapsack

---

Q5 — MCQ

A recurrence:

$$ T(n)=2T(n/2)+1 $$

has complexity:

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n\log n)$ D. $\Theta(n^2)$

Q6 — MCQ

Which statement best separates D&C and DP?

A. D&C always uses loops; DP always uses recursion.

B. D&C usually has independent subproblems; DP exploits overlapping subproblems.

C. D&C always has lower complexity.

D. There is no difference.

Q7 — NAT

For even:

$$ n=20 $$

how many comparisons are needed by the efficient pairwise min-max method?

Hard / Tricky

Q8 — MSQ

Which statements are true?

A. Every recursive algorithm is Divide and Conquer.

B. Binary Search has logarithmic recursion depth.

C. Merge Sort's combine work is linear.

D. Quick Sort's partition balance affects its complexity.

Q9 — MCQ

Suppose:

$$ T(n)=4T(n/2)+n $$

What is the complexity?

A. $\Theta(n)$ B. $\Theta(n\log n)$ C. $\Theta(n^2)$ D. $\Theta(n^3)$

SOLUTIONS

Q1

`Memorize` is not a classic D&C stage.

$$\boxed{D}$$

Q2

Binary Search chooses only one half.

$$\boxed{B}$$

Q3

$$\boxed{C}$$

Q4

Binary Search, Merge Sort and Quick Sort.

$$\boxed{A,B,C}$$

Fractional Knapsack is Greedy.

Q5

Master Theorem:

$$a=2,b=2$$

$$n^{\log_2 2}=n$$

$$f(n)=1$$

Therefore:

$$\boxed{\Theta(n)}$$

Answer:

$$\boxed{B}$$

Q6

$$\boxed{B}$$

Q7

$$\frac{3n}{2}-2$$

$$=\frac{3(20)}{2}-2$$

$$ =30-2 $$

$$ \boxed{28} $$

Q8

A false.

B true.

C true.

D true.

Answer:

$$ \boxed{B,C,D} $$

Q9

$$ a=4,\quad b=2 $$

$$ n^{\log_2 4}=n^2 $$

$$ f(n)=n $$

Recursive term dominates.

Therefore:

$$ \boxed{\Theta(n^2)} $$

Answer:

$$ \boxed{C} $$

---

66. Ø=ÜÓTES TO WRITE

Divide and Conquer

Three stages:

```text
DIVIDE
CONQUER
COMBINE
```

General recurrence:

$$ \boxed{T(n)=aT(n/b)+f(n)} $$

Binary Search

Only one half is solved:

$$ T(n)=T(n/2)+1 $$

$$ \boxed{\Theta(\log n)} $$

Merge Sort

Two halves + linear merge:

$$ T(n)=2T(n/2)+n $$

$$ \boxed{\Theta(n\log n)} $$

Quick Sort

General:

$$ T(n)=T(k)+T(n-k-1)+n $$

Balanced:

$$ \Theta(n\log n) $$

Worst:

$$ \Theta(n^2) $$

D&C vs DP

```TEXT
D & C  !'  i n d e p e n d e n t / n o n - o v e r l a p p i n g  s u b p r o b l e m s
D P    !'  o v e r l a p p i n g  s u b p r o b l e m s
```

Recursion Depth

Divide by constant:

$$ \Theta(\log n) $$

Decrease by 1:

$$ \Theta(n) $$

Min + Max

For even n:

$$ \boxed{\frac{3n}{2}-2} $$

comparisons.

67. MEMORY TRICKS

D&C

> **Split !’ Solve !’ Stitch**

```text
Split   = Divide
Solve   = Conquer
Stitch  = Combine
```

Binary vs Merge

> **Binary chooses ONE half.**

> **Merge solves BOTH halves.**

Merge vs Quick

> **Merge works while joining.**

> **Quick works while partitioning.**

DP difference

> **D&C divides cleanly. DP sees duplicates.**

“Duplicates” here means repeated subproblems.

&β30-SECOND REVISION

1. Divide and Conquer = divide, conquer, combine.
2. General recurrence = \(aT(n/b)+f(n)\).
3. Binary Search solves only one half.
4. Binary Search = \(\Theta(\log n)\).
5. Merge solves both halves.
6. Merge recurrence = \(2T(n/2)+n\).
7. Merge = \(\Theta(n\log n)\).
8. Quick uses pivot + partition.
9. Balanced Quick = \(n\log n\).
10. Worst Quick = \(n^2\).
11. D&C usually has independent subproblems.
12. DP is useful for overlapping subproblems.
13. Recursion itself does not imply D&C.

14. Divide-by-two depth = $\log n$.
15. Recursion depth is not necessarily total running time.
16. D&C does not automatically improve complexity.
17. Balanced partitions are usually desirable.
18. Efficient min+max comparison count = $3n/2-2$ for even n.

' Chapter 10 Complete

Next Chapter:

Chapter 11 — Greedy Algorithms

We'll cover:

- greedy-choice property,
- optimal substructure,
- how to recognize Greedy problems,
- when Greedy fails,
- Activity Selection,
- Fractional Knapsack,
- Huffman Coding,
- Job Sequencing where relevant,
- proof/exchange-argument intuition,
- Greedy vs DP,
- and GATE traps such as **why Fractional Knapsack is Greedy but 0/1 Knapsack is not**.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

CHAPTER 11 — GREEDY ALGORITHMS

GATE Importance: & & & & &

Your uploaded Algorithms plan explicitly includes **Greedy Algorithms**, including greedy-choice property, optimal substructure, Activity Selection, Fractional Knapsack, Huffman Coding, Job Sequencing where relevant, and comparison with Dynamic Programming.

The core idea is:

> **At each step, choose what looks best right now — and never go back.**

But the important warning is:

> **A locally best choice does not always lead to a globally optimal answer.**

So the real skill is knowing **when Greedy is valid**.

---

1. WHAT IS A GREEDY ALGORITHM?

A Greedy algorithm constructs a solution step by step.

At each step:

1. choose the locally best option,
2. commit to that choice,
3. continue,
4. do not reconsider previous choices.

Example idea:

Suppose you want to maximize the number of non-overlapping activities.

A greedy strategy is:

> **always choose the activity that finishes earliest.**

That choice leaves maximum room for remaining activities.

---

2. GREEDY DOES NOT MEAN “JUST PICK THE BIGGEST”

Greedy does **not** always mean:

```text id="gd1a4a" pick maximum value

```

```
The greedy criterion depends on the problem.

Examples:

### Activity Selection

Choose:

\[
\boxed{\text{earliest finishing activity}}
\]

### Fractional Knapsack

Choose highest:

\[
\boxed{\text{value/weight ratio}}
\]

### Kruskal MST

Choose:

\[
```

```
\boxed{\text{smallest safe edge}}
\]

### Prim MST

Choose:

\[
\boxed{\text{minimum edge connecting tree to outside}}
\]

So:

> Greedy = make the best **problem-specific local choice**.

---

# 3. TWO IMPORTANT PROPERTIES

A problem is a strong Greedy candidate when it has:

## 1. Greedy-choice property

A globally optimal solution can be reached by making a locally optimal choice first.

## 2. Optimal substructure

An optimal solution contains optimal solutions to its subproblems.

---

# 4. GREEDY-CHOICE PROPERTY

Suppose a problem has choices:

```text id="vyjywc"
A B C D
```
```

If choosing the locally best one now can always be part of some optimal solution, then greedy choice is safe.

This is the hard part of proving a Greedy algorithm.

---

5. OPTIMAL SUBSTRUCTURE

Suppose after making one choice, the remaining problem is smaller.

If an optimal solution to the original problem contains an optimal solution to the remaining problem, then it has optimal substructure.

Both Greedy and DP often have optimal substructure.

The difference is:

> **Greedy commits immediately.**

> **DP explores/stores many possible subproblem states before deciding.**

---

6. GREEDY VS DYNAMIC PROGRAMMING

Feature	Greedy	Dynamic Programming
Decision	Locally best now	Compare many subproblem possibilities
Reconsider choice?	No	Effectively yes through stored states
Subproblems	Often simple	Often overlapping
Space	Usually less	Often more
Proof needed	Greedy choice must be safe	Recurrence correctness
Example	Fractional Knapsack	0/1 Knapsack

---

7. HOW TO RECOGNIZE A GREEDY PROBLEM

Look for phrases such as:

```text id="7k7na6" choose maximum possible choose minimum possible earliest finishing lowest cost safe choice highest ratio take best currently available

```
But don't conclude Greedy immediately.

Ask:

> If I make this choice now, can it block a better future solution?

If yes, Greedy may fail.

---

8. ACTIVITY SELECTION PROBLEM

We have activities with:

\[
(start_i, finish_i)
\]

Goal:

> Select maximum number of mutually non-overlapping activities.

Example:

Activity	Start	Finish
A1	1	2
A2	3	4
A3	0	6
A4	5	7
A5	8	9
A6	5	9

---

9. GREEDY RULE FOR ACTIVITY SELECTION
```
```

```
Choose:

\[
\boxed{\text{activity with earliest finish time}}
\]

Then choose next activity whose start time is at least the finish time of the last selected
activity.

---

# 10. WHY EARLIEST FINISH?

If an activity finishes early:

> It leaves as much remaining time as possible for future activities.

That maximizes opportunity for additional selections.

---

# 11. ACTIVITY SELECTION TRACE

Activities sorted by finish time:

```text id="xtn6up"
A1 (1,2)
A2 (3,4)
A3 (0,6)
A4 (5,7)
A5 (8,9)
A6 (5,9)
```
```

Choose:

```text id="xpn7rg" A1

```
Last finish:

\[
2
\]

Next start \(\ge2\):

```text id="7htdmf"
A2 (3,4)
```
```

Choose A2.

Last finish:

\[ 4 \]

Next compatible:

```text id="9x39dj" A4 (5,7)

```
Choose A4.

Last finish:
```

```
\[
7
\]

Next compatible:

```text id="hcl09f"
A5 (8,9)
```
```

Final:

```text id="0jtr2x" A1, A2, A4, A5

```
Number:

\[
\boxed{4}
\]

---

12. ACTIVITY SELECTION PSEUDOCODE

Assuming activities already sorted by finish time:

```text id="80piiy"
select first activity
last = first

for each remaining activity i:
    if start[i] >= finish[last]:
        select i
        last = i
```
```

If already sorted:

\[ \boxed{O(n)} \]

If sorting is required:

\[ \boxed{O(n\log n)} \]

overall.

---

## 13. ACTIVITY SELECTION TRAPS

Do **not** greedily choose:

- earliest starting activity,
- shortest-duration activity,
- latest starting activity,
- activity with most free time after it,

unless separately proven.

The standard optimal criterion is:

\[ \boxed{\text{earliest finish time}} \]
```

14. FRACTIONAL KNAPSACK

You have a knapsack of capacity:

\[W \]

Each item has:

- value \(v_i\),
- weight \(w_i\).

In **Fractional Knapsack**, you are allowed to take part of an item.

Goal:

> **maximize total value.**

15. EXAMPLE

Capacity:

\[W=50 \]

Items:

Item	Value	Weight	\(v/w\)
A	60	10	6
B	100	20	5
C	120	30	4

Greedy rule:

> **choose highest value per unit weight first.**

Order:

```text id="5l8obm" A !' B !' C

```

```
---

# 16. FRACTIONAL KNAPSACK TRACE

Take A completely:

Weight:

\[
10
\]

Value:
```

```
\[
60
\]

Remaining capacity:

\[
40
\]

Take B:

Weight:

\[
20
\]

Value total:

\[
160
\]

Remaining:

\[
20
\]

C weighs 30, but only 20 capacity remains.

Take:

\[
\frac{20}{30}=\frac23
\]

of C.

Value obtained:

\[
120\times\frac23=80
\]

Total value:

\[
60+100+80
\]

\[
\boxed{240}
\]

---

# 17. FRACTIONAL KNAPSACK GREEDY RULE

Compute:

\[
\boxed{\frac{value}{weight}}
\]
```

```
Sort descending.

Take items in that order.

If next item does not fit:

> take the required fraction.

---

# 18. FRACTIONAL KNAPSACK COMPLEXITY

Compute ratios:

\[
O(n)
\]

Sort:

\[
O(n\log n)
\]

Scan:

\[
O(n)
\]

Total:

\[
\boxed{O(n\log n)}
\]

assuming general comparison sorting.

---

# 19. WHY GREEDY WORKS HERE

If one item gives more value per unit of weight than another, replacing lower-ratio weight by
higher-ratio weight can only improve value.

Because fractions are allowed, there is no indivisibility restriction.

That is exactly why Greedy works.

---

# 20. 0/1 KNAPSACK

Now change one rule:

> You must either take an entire item or leave it.

No fractions.

This is **0/1 Knapsack**.

Greedy by value/weight ratio can fail.

---

# 21. GREEDY FAILURE IN 0/1 KNAPSACK
```

```
Use same example:

Capacity:

\[
50
\]

| Item | Value | Weight | Ratio |
|---|---:|---:|---:|
| A | 60 | 10 | 6 |
| B | 100 | 20 | 5 |
| C | 120 | 30 | 4 |

Greedy ratio chooses:

```text id="ey14ff"
A + B
```
```

Weight:

\[30 \]

Value:

\[160 \]

Remaining:

\[20 \]

Cannot take C.

Greedy answer:

\[160 \]

But choose:

```text id="spu7qv" B + C

```
```
Weight:

\[
50
\]

Value:

\[
220
\]

Thus:

\[
\boxed{220>160}
\]

Greedy fails.

---
```

```
# 22. KEY GATE RULE

### Fractional Knapsack

\[
\boxed{\text{Greedy}}
\]

### 0/1 Knapsack

\[
\boxed{\text{Dynamic Programming}}
\]

Memory:

 >   * * F r a c t i o n   a l l o w e d  !’   G r e e d y * *

 >   * * W h o l e   i t e m   o n l y  !’   D P * *

---

# 23. HUFFMAN CODING

Huffman Coding creates an optimal **prefix code** for symbols based on their frequencies.

More frequent symbols get shorter codes.

Less frequent symbols get longer codes.

Used in compression.

---

# 24. PREFIX CODE

A prefix code has the property:

> No codeword is a prefix of another codeword.

Example valid:

```text id="en7c9d"
 A  !’   0
 B  !’   1 0
 C  !’   1 1 0
 D  !’   1 1 1
```
```

`0` is not a prefix of another code beginning with 0 because none does.

This makes decoding unambiguous.

---

25. HUFFMAN GREEDY RULE

Repeatedly:

1. choose two symbols/subtrees with smallest frequencies,
2. combine them,
3. insert combined frequency back,
4. repeat.

That is the greedy choice.

26. HUFFMAN EXAMPLE

Frequencies:

```text id="s391vf" A = 5 B = 9 C = 12 D = 13 E = 16 F = 45

```
Choose two smallest:

\[
5+9=14
\]

Now:

```text id="k2o4fh"
12,13,14,16,45
```

Choose:

\[12+13=25 \]

Now:

```text id="9zdi83" 14,16,25,45

```
Choose:

\[
14+16=30
\]

Now:

```text id="4qgb45"
25,30,45
```

Choose:

\[25+30=55 \]

Now:

```text id="5jqd6c" 45,55

```
Combine:

\[
100
\]

Tree complete.

---

27. HUFFMAN TREE

Conceptually:
```
```

```text id="u3ioam"
                100
              /     \
            45       55
                    /  \
                  25    30
                 / \    / \
               12 13   14 16
                       / \
                      5   9
```

Assign:

```text id="1sm0da" left  = 0 right = 1

Codes follow root-to-leaf paths.

Exact 0/1 orientation can differ, but total encoded cost stays optimal.

---

# 28. HUFFMAN COST

Weighted path length:

\[
\boxed{
\sum_i f_i \cdot depth_i
}
\]

where:

- \(f_i\) = frequency,
- `depth` = code length.

Goal is to minimize this sum.

---

# 29. WHY HUFFMAN USES TWO SMALLEST

Low-frequency symbols should be deeper because their longer codes contribute less to total cost.

Thus two least frequent symbols can safely be combined deepest.

This greedy choice can be proved optimal.

---

# 30. HUFFMAN COMPLEXITY

Using a min-priority queue / min-heap:

- Build heap: \(O(n)\)
- Perform \(n-1\) merge steps
- each involving heap operations \(O(\log n)\)

Total:

\[
\boxed{O(n\log n)}
\]
```

If frequencies are already sorted, specialized approaches can do better, but \(O(n\log n)\) is the standard result.

---

31. JOB SEQUENCING WITH DEADLINES

A classic Greedy problem:

Each job has:

- deadline,
- profit,
- unit execution time.

Goal:

> schedule jobs before deadlines to maximize profit.

Example:

Job	Deadline	Profit
J1	2	100
J2	1	19
J3	2	27
J4	1	25
J5	3	15

---

32. GREEDY RULE FOR JOB SEQUENCING

Sort jobs by:

\[
\boxed{\text{decreasing profit}}
\]

For each job:

> place it in the latest available slot before or at its deadline.

Why latest?

Because earlier slots remain available for jobs with tighter deadlines.

---

33. JOB SEQUENCING TRACE

Profit order:

```text id="5ao3nb"
J1 = 100, d=2
J3 = 27,  d=2
J4 = 25,  d=1
J2 = 19,  d=1
J5 = 15,  d=3
```

Slots:

```text id="zkjrs1" 1 2 3
```

```
J1 deadline 2:

Place as late as possible:

```text id="xqm9lw"
 s l o t   2  !’   J 1
```
```

J3 deadline 2:

slot 2 occupied.

Try slot 1:

```text id="btdl90" slot 1 !’ J3

```
J4 deadline 1:

slot 1 occupied.

Cannot schedule.

J2 same.

J5 deadline 3:

```text id="eocxll"
 s l o t   3  !’   J 5
```
```

Final:

```text id="unj9og" Slot 1 !’ J3 Slot 2 !’ J1 Slot 3 !’ J5

```
Profit:

\[
27+100+15
\]

\[
\boxed{142}
\]

---

34. WHY “LATEST AVAILABLE SLOT”?

Suppose a job has deadline 3.

It could occupy:

```text id="h2kaht"
1,2,or3
```
```

Putting it at slot 3 preserves:

```text id="yf61x8" 1 and 2

```
for jobs that may have earlier deadlines.
```
```

Hence the greedy placement.

---

35. JOB SEQUENCING COMPLEXITY

Simple implementation:

- sort by profit: $O(n\log n)$
- search backwards through slots for each job: up to $O(n)$

Total naive:

$$
O(n^2)
$$

depending on deadlines and implementation.

More efficient DSU-based implementations can improve slot finding, but the simple greedy principle is what matters most here.

---

36. GREEDY PROOF — EXCHANGE ARGUMENT

A common way to prove Greedy correctness is an **exchange argument**.

Idea:

1. Start with some optimal solution.
2. Show its first choice can be replaced with the greedy choice.
3. Show this replacement does not make the solution worse.
4. Therefore some optimal solution begins with the greedy choice.
5. Repeat recursively/inductively.

---

37. EXCHANGE ARGUMENT — ACTIVITY SELECTION

Suppose an optimal solution starts with activity A.

Greedy chooses activity G with earliest finishing time.

Since:

$$
finish(G)\le finish(A)
$$

replace A with G.

This cannot reduce the room available for later activities.

Therefore an optimal solution exists that starts with G.

So greedy choice is safe.

---

38. GREEDY STAYS AHEAD

Another proof technique:

> After every step, show greedy solution is at least as good as any competing partial solution

```
according to some measure.

This is sometimes called a **greedy stays ahead** argument.

---

# 39. GREEDY FAILS — COIN CHANGE

Suppose coin denominations:

```text id="8r9tqe"
1, 3, 4
```

Need amount:

\[6 \]

Greedy chooses largest possible coin:

```text id="dnerzf" 4
```

```
Remaining:

\[
2
\]

Then:

```text id="a87127"
1 + 1
```

Total coins:

\[3 \]

But optimal:

```text id="6ces33" 3 + 3
```

```
uses:

\[
2
\]

coins.

Thus Greedy fails.

---

# 40. IMPORTANT LESSON

Greedy coin-change works for some coin systems, but not arbitrary denominations.

Therefore:

> Never assume a plausible local rule is globally optimal without proof/property.

---
```

41. GREEDY FAILS — 0/1 KNAPSACK

Already seen.

Highest ratio can block a better combination.

Hence:

$$
\boxed{\text{Greedy-choice property does not hold}}
$$

for general 0/1 Knapsack.

---

42. GREEDY AND MST

Minimum Spanning Tree algorithms are also Greedy.

We will study them in detail later.

Kruskal

Repeatedly choose:

> smallest edge that does not create cycle.

Prim

Repeatedly choose:

> smallest edge that connects current tree to a new vertex.

Both rely on MST cut/cycle properties.

---

43. GREEDY AND SHORTEST PATHS

Dijkstra is also greedy.

It repeatedly finalizes the currently smallest tentative distance.

But it only works correctly under appropriate edge-weight assumptions:

> no negative-weight edges.

We will study this later.

---

44. IMPORTANT GREEDY ALGORITHM MAP

Problem	Greedy Criterion
Activity Selection	Earliest finish time
Fractional Knapsack	Highest value/weight
Huffman Coding	Two lowest frequencies
Job Sequencing	Highest profit, latest feasible slot
Kruskal MST	Smallest safe edge
Prim MST	Cheapest crossing edge
Dijkstra	Smallest tentative distance

This is an important GATE revision table.

```
---

# 45. GREEDY VS DP — EXAMPLE

## Fractional Knapsack

Can take fractions.

Local ratio decision can always be safely extended.

\[
\boxed{\text{Greedy}}
\]

## 0/1 Knapsack

Cannot split items.

Local ratio decision can block optimal combination.

\[
\boxed{\text{DP}}
\]

---

# 46. GREEDY VS D&C

### Greedy

Chooses one locally best option.

### Divide and Conquer

Splits into smaller independent problems and solves them.

Example:

```text id="icq3iv"
 M e r g e   S o r t  !’   D & C
 A c t i v i t y   S e l e c t i o n  !’   G r e e d y
```
```

47. GREEDY DOES NOT NECESSARILY USE RECURSION

Greedy is a design strategy, not a recursion pattern.

Many Greedy algorithms are iterative.

Example Activity Selection can simply scan sorted activities.

48. GREEDY DOES NOT NECESSARILY MEAN FASTEST POSSIBLE

Greedy may be efficient, but its defining property is **choice strategy**, not complexity.

A greedy algorithm could still involve sorting:

\[O(n\log n) \]

or other expensive operations.

49. GATE TRAP — OPTIMAL SUBSTRUCTURE ALONE

If a problem has optimal substructure, does that guarantee Greedy?

$$\boxed{\text{No}}$$

DP problems also have optimal substructure.

Greedy additionally needs the:

$$\boxed{\text{greedy-choice property}}$$

50. GATE TRAP — LOCAL OPTIMUM

A local optimum does not automatically imply global optimum.

Greedy is correct only when problem structure guarantees this.

51. GATE TRAPS

& þTrap 1 — Greedy always gives optimal answer

False.

& þTrap 2 — Optimal substructure means Greedy works

False.

Need greedy-choice property too.

& þTrap 3 — Activity Selection chooses shortest activity

False.

Standard rule:

$$\boxed{\text{earliest finish}}$$

& þTrap 4 — Fractional Knapsack chooses maximum value

Not necessarily.

Choose maximum:

$$\boxed{value/weight}$$

ratio.

& þTrap 5 — Same Greedy method solves 0/1 Knapsack

False.

& þTrap 6 — Huffman combines highest frequencies first

False.

It combines:

\[\boxed{\text{two smallest}} \]

& þTrap 7 — Job Sequencing schedules high-profit job as early as possible

Wrong.

Place it:

\[\boxed{\text{as late as possible before deadline}} \]

& þTrap 8 — Greedy and DP are equivalent

No.

Greedy commits immediately; DP preserves alternatives through subproblem states.

& þTrap 9 — Greedy algorithm must be recursive

False.

& þTrap 10 — Greedy choice doesn't need proof

Wrong.

Correctness often depends on exchange/cut/stays-ahead arguments.

52. EXAM DECISION RULES

If activity scheduling asks maximum number:

Think:

\[\boxed{\text{earliest finish}} \]

If items can be divided:

Think:

\[\boxed{\text{Fractional Knapsack !' ratio}} \]

If items cannot be divided:

Think:

\[\boxed{\text{0/1 Knapsack !' DP candidate}} \]

If compression with symbol frequencies:

Think:

\[\boxed{\text{Huffman !' combine two minimum frequencies}} \]

If jobs have deadlines/profits and unit time:

Think:

> **High profit first, place as late as possible.**

If repeated local choice can block future optimum:

Be suspicious of Greedy.

53. PYQ-STYLE QUESTION 1

Activity Selection is optimally solved by repeatedly selecting:

A. earliest start B. shortest duration C. earliest finish D. latest finish

Answer:

\[\boxed{C} \]

54. PYQ-STYLE QUESTION 2

Fractional Knapsack prioritizes:

A. maximum weight B. minimum weight C. maximum value D. maximum \(value/weight\)

Answer:

\[\boxed{D} \]

55. PYQ-STYLE QUESTION 3

Capacity:

\[50 \]

Items:

```text id="j8092s" (v,w): (60,10) (100,20) (120,30)

```
Ratios:

\[
6,5,4
\]
```
```

```
Optimal fractional value:

\[
60+100+\frac{20}{30}(120)
\]

\[
=160+80
\]

\[
\boxed{240}
\]

---

# 56. PYQ-STYLE QUESTION 4

For the same items, 0/1 Knapsack optimum:

Choose:

```text id="hgmlx0"
(100,20) + (120,30)
```
```

Weight:

\[50 \]

Value:

\[\boxed{220} \]

Ratio-greedy gives only:

\[160 \]

Thus Greedy fails.

---

57. PYQ-STYLE QUESTION 5

Huffman frequencies:

```text id="lfprml" 2,3,7,9

```
```
First combination?

Two minimum:

\[
2+3
\]

\[
\boxed{5}
\]

New frequencies:

```text id="90l2hw"
5,7,9
```
```

58. PYQ-STYLE QUESTION 6

A problem has optimal substructure.

Can we conclude Greedy works?

\[\boxed{\text{No}} \]

It also needs greedy-choice property.

59. PYQ-STYLE QUESTION 7

Coin denominations:

```text id="nfj5o9" 1,3,4

Amount:

\[
6
\]

Greedy:

\[
4+1+1
\]

3 coins.

Optimal:

\[
3+3
\]

2 coins.

Therefore generic coin change is a counterexample to naive Greedy.

---

# 60. QUICK RECALL

Answer mentally.

1. What does Greedy do at each step?
2. Does it normally reconsider previous choices?
3. Two properties relevant to Greedy correctness?
4. Activity Selection criterion?
5. Fractional Knapsack criterion?
6. Fractional or 0/1 Knapsack uses Greedy?
7. What does Huffman combine?
8. What is prefix-code property?
9. Job Sequencing chooses jobs in what order?
10. Where is a selected job placed?
11. Does optimal substructure alone imply Greedy?
12. What is exchange argument?
13. Kruskal is Greedy or DP?
14. Dijkstra is Greedy or DP?
```

15. Why can Greedy fail?

ANSWERS

1. Makes locally best available choice.
2. No.
3. Greedy-choice property and optimal substructure.
4. Earliest finish.
5. Highest value/weight ratio.
6. Fractional.
7. Two lowest-frequency nodes.
8. No codeword is a prefix of another.
9. Decreasing profit.
10. Latest feasible slot before deadline.
11. No.
12. Replace part of an optimal solution by greedy choice without worsening it.
13. Greedy.
14. Greedy.
15. Local optimum may block global optimum.

---

61. PRACTICE SET

Easy

Q1 — MCQ

Which is Greedy?

A. 0/1 Knapsack
B. Fractional Knapsack
C. Matrix Chain Multiplication
D. LCS

Q2 — MCQ

Activity Selection chooses:

A. earliest start
B. latest finish
C. earliest finish
D. highest duration

Q3 — MCQ

Huffman Coding repeatedly combines:

A. two highest frequencies
B. two lowest frequencies
C. random frequencies
D. first two frequencies

Medium

Q4 — MSQ

Which problems are classically solved using Greedy techniques?

```
A. Fractional Knapsack
B. Activity Selection
C. Huffman Coding
D. 0/1 Knapsack

---

### Q5 — NAT

Capacity:

\[
W=20
\]

Items:

| Value | Weight |
|---:|---:|
| 40 | 10 |
| 30 | 5 |
| 50 | 10 |

Find optimal fractional-knapsack value.

---

### Q6 — MSQ

Which statements are true?

A. Greedy decisions are generally irreversible.

B. Optimal substructure alone guarantees Greedy correctness.

C. Greedy may fail for arbitrary coin systems.

D. Fractional Knapsack is optimally solved by value/weight ratio.

---

### Q7 — MCQ

In Job Sequencing with deadlines, after sorting by profit, a job should ideally be placed:

A. earliest possible slot
B. latest possible slot not exceeding deadline
C. exactly at deadline even if occupied
D. random slot

---

# Hard / Tricky

### Q8 — MCQ

Coin system:

```text id="l8dgqc"
1,4,6
```
```

Amount:

\[8 \]

Largest-coin greedy gives:

```text id="9z89qg" 6+1+1

```
Optimal gives:

```text id="jo1m0b"
4+4
```

What does this show?

A. Greedy always works B. Coin Change never has optimal substructure C. Naive largest-coin Greedy can fail D. Dynamic Programming always takes \(O(1)\)

Q9 — MSQ

Which are valid Greedy proof ideas?

A. Exchange argument B. Greedy stays ahead C. Show local choice can be part of an optimal solution D. Assume local optimum equals global optimum without proof

---

SOLUTIONS

Q1

Fractional Knapsack.

\[\boxed{B} \]

Q2

\[\boxed{C} \]

Q3

\[\boxed{B} \]

Q4

Fractional Knapsack, Activity Selection, Huffman.

\[\boxed{A,B,C} \]

Q5

Ratios:

\[40/10=4 \]

\[30/5=6 \]

\[50/10=5 \]

Order:

```text id="r0b6ti" (30,5) (50,10) (40,10)

```
Take first two fully.

Used weight:

\[
15
\]

Value:

\[
80
\]

Remaining:

\[
5
\]

Take half of first remaining item:

\[
40\times\frac5{10}=20
\]

Total:

\[
\boxed{100}
\]

Q6

A true.

B false.

C true.

D true.

Answer:

\[
\boxed{A,C,D}
\]

Q7

\[
\boxed{B}
\]

```
```

```
## Q8

It proves naive largest-coin Greedy can fail.

\[
\boxed{C}
\]

---

## Q9

A true.

B true.

C true.

D false.

Answer:

\[
\boxed{A,B,C}
\]

---

 #   6 2 .  Ø=ÜÝ   N O T E S   T O   W R I T E

# Greedy Algorithm

At every step:

\[
\boxed{\text{choose locally best option and commit}}
\]

Important properties:

```text id="jb5s2r"
Greedy-choice property
Optimal substructure
```
```

Optimal substructure alone is not enough.

Activity Selection

Goal:

> **maximum number of non-overlapping activities.**

Rule:

\[\boxed{\text{earliest finish time}} \]

Sorted input scan:

\[O(n) \]

Including sorting:

$$ O(n\log n) $$

Fractional Knapsack

Fractions allowed.

Choose highest:

$$ \boxed{\frac{value}{weight}} $$

Time:

$$ O(n\log n) $$

because of sorting.

Greedy optimal.

0/1 Knapsack

Whole item only.

Ratio Greedy can fail.

Use:

$$ \boxed{\text{DP}} $$

Huffman Coding

Repeatedly combine:

$$ \boxed{\text{two minimum frequencies}} $$

Creates optimal prefix code.

Cost:

$$ \sum f_i\cdot depth_i $$

Using min-heap:

$$ O(n\log n) $$

Job Sequencing

Sort by:

$$ \boxed{\text{descending profit}} $$

Place each job:

$$ \boxed{\text{latest free slot }\le deadline} $$

Greedy Proof

Common:

```text id="mktufc" Exchange argument Greedy stays ahead

```
---

63. MEMORY TRICKS

Greedy

> **Grab best now, don't go back.**

Activity Selection

 > * * F i n i s h F a s t !’ F i t M o r e * *

Fractional Knapsack

 > * * F r a c t i o n !’ R a t i o * *

0/1 Knapsack

 > * * W h o l e !’ D P * *

Huffman

> **Small + Small, repeatedly**

Job Sequencing

> **Profit first, schedule late**

 # &¡ 3 0 - S E C O N D R E V I S I O N

1. Greedy makes locally optimal irreversible choices.
2. Greedy needs greedy-choice property.
3. Greedy problems often also have optimal substructure.
4. Optimal substructure alone does not guarantee Greedy.
 5 . A c t i v i t y S e l e c t i o n !’ e a r l i e s t f i n i s h .
 6 . F r a c t i o n a l K n a p s a c k !’ h i g h e s t v a l u e / w e i g h t .
7. Fractional Knapsack is Greedy.
8. 0/1 Knapsack is not solved by ratio Greedy.
9. Huffman repeatedly combines two smallest frequencies.
10. Huffman gives optimal prefix coding.
 1 1 . J o b S e q u e n c i n g !’ h i g h e s t p r o f i t f i r s t .
12. Place selected job as late as possible before deadline.
13. Greedy can fail on arbitrary coin denominations.
14. Exchange argument is a standard correctness proof.
15. Kruskal, Prim and Dijkstra are also Greedy algorithms.
 1 6 . G r e e d y "` r e c u r s i o n .
 1 7 . L o c a l o p t i m u m "` g l o b a l o p t i m u m u n l e s s p r o b l e m s t r u c

 # ' C h a p t e r 1 1 C o m p l e t e
```
```

```
## Next Chapter:

# **Chapter 12 — Dynamic Programming**

 This is another **&&&&& GATE chapter**. We'll cover:

- overlapping subproblems,
- optimal substructure,
- memoization vs tabulation,
- state, transition and base case,
- converting recursion to DP,
- Fibonacci intuition,
- 0/1 Knapsack,
- Longest Common Subsequence,
- Matrix Chain Multiplication,
- LIS/Coin Change where relevant,
- DP table questions,
- and how to recognize **Greedy vs DP vs Divide-and-Conquer**.

**Type `NEXT` to continue.**
```

USER

NEXT

ASSISTANT

CHAPTER 12 — DYNAMIC PROGRAMMING

GATE Importance: & & & & &

Your uploaded Algorithms plan explicitly includes Dynamic Programming, including overlapping subproblems, optimal substructure, memoization, tabulation, state, transition, base cases, 0/1 Knapsack, Matrix Chain Multiplication, LCS, LIS, Coin Change and Floyd-Warshall.

The core idea is:

> **Solve repeated subproblems once, store their answers, and reuse them.**

---

1. WHY DYNAMIC PROGRAMMING MATTERS

Many recursive problems repeatedly solve the same smaller problems.

Example:

\[F(n)=F(n-1)+F(n-2) \]

Naive recursion:

TEXT

```
            F(5)
         /        \
      F(4)        F(3)
```

```
      /   \        /   \
   F(3)  F(2)   F(2)  F(1)
```

Notice:

\[F(3),F(2) \]

are calculated repeatedly.

Dynamic Programming says:

> **Compute them once and remember them.**

---

2. TWO KEY PROPERTIES

A problem is a strong DP candidate when it has:

1. Overlapping Subproblems

The same smaller problems appear repeatedly.

2. Optimal Substructure

The optimal solution to the large problem can be constructed using optimal solutions to smaller problems.

---

3. DP VS DIVIDE AND CONQUER

Very important.

Divide and Conquer

Subproblems are usually independent.

Example:

Merge Sort:

```text
left half
right half
```

No meaningful overlap.

Dynamic Programming

Subproblems overlap.

Example Fibonacci:

```text
F(5)
% % %    F ( 4 )
%       % % %    F ( 3 )
```

```
%        % % %    F ( 2 )
% % %    F ( 3 )
```

`F(3)` repeats.

So:

Feature	D&C	DP
Subproblems	Usually independent	Overlapping
Store results?	Usually unnecessary	Yes
Example	Merge Sort	0/1 Knapsack

Memory:

> **D&C divides. DP remembers duplicates.**

---

4. DP VS GREEDY

Greedy

Makes one locally best choice and commits.

DP

Considers alternatives systematically through stored states.

Example:

Fractional Knapsack:

\[\boxed{\text{Greedy}} \]

0/1 Knapsack:

\[\boxed{\text{DP}} \]

Why?

Because in 0/1 Knapsack, a locally attractive item may block a better combination.

---

5. THREE MOST IMPORTANT DP TERMS

Every DP problem should make you identify:

State

What smaller problem are we storing?

Transition

How is current answer computed from previous states?

Base Case

What smallest cases do we already know?

Memory:

> **S-T-B**

```text
State
Transition
Base
```

---

6. MEMOIZATION VS TABULATION

There are two main DP styles.

Memoization — Top Down

Start with recursive solution.

Whenever a state is solved:

> **store the answer.**

If requested again:

> **return stored value.**

Structure:

```text
recursive function
      +
memo/cache
```

Tabulation — Bottom Up

Start from smallest known states.

Build table iteratively until desired answer is reached.

Structure:

```text
base cases
!'   s m a l l   s t a t e s
!'   l a r g e r   s t a t e s
!'   f i n a l   s t a t e
```

7. MEMOIZATION VS TABULATION

Property	Memoization	Tabulation
Direction	Top-down	Bottom-up
Recursion	Usually yes	Usually no
Computes	Needed states	Often all states
Stack overhead	Yes	No recursion stack
Easy from recurrence?	Often yes	Requires order
Storage	Cache	Table

Both are Dynamic Programming.

8. FIBONACCI — NAIVE RECURSION

\[F(n)=F(n-1)+F(n-2) \]

Base:

\[F(0)=0 \]

\[F(1)=1 \]

Naive recursion:

```text
Fib(n):
    if n <= 1:
        return n

    return Fib(n-1) + Fib(n-2)
```

Time grows exponentially.

Roughly:

\[O(2^n) \]

because the call tree contains enormous repeated computation.

9. FIBONACCI WITH MEMOIZATION

```text
Fib(n):

    if n <= 1:
        return n

    if memo[n] exists:
        return memo[n]
```

```
    memo[n] = Fib(n-1) + Fib(n-2)

    return memo[n]
```

Each state:

$$ 0,1,2,\ldots,n $$

is computed once.

Number of states:

$$ n+1 $$

Work per state:

$$ O(1) $$

Therefore:

$$ \boxed{O(n)} $$

Time.

Space:

$$ O(n) $$

for memo + recursion stack.

---

10. FIBONACCI TABULATION

```TEXT
dp[0] = 0
dp[1] = 1

for i = 2 to n:
    dp[i] = dp[i-1] + dp[i-2]
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ O(n) $$

---

11. SPACE OPTIMIZATION

To calculate:

$$ F(i) $$

we need only:

$$ F(i-1),F(i-2) $$

So instead of entire array:

```text
prev2
prev1
current
```

Then auxiliary space:

$$ \boxed{O(1)} $$

This is called **space optimization**.

---

12. DP COMPLEXITY SHORTCUT

A very useful rule:

$$ \boxed{ \text{DP time} \approx \text{number of states} \times \text{work per state} } $$

Example Fibonacci:

States:

$$ n $$

Work per state:

$$ O(1) $$

Therefore:

$$ O(n) $$

This rule will help greatly in GATE.

---

13. 0/1 KNAPSACK

We already saw why Greedy fails.

Each item has:

- weight w_i
- value v_i

Knapsack capacity:

$$ W $$

Each item can be either:

```text
take completely
or
do not take
```

Goal:

> **maximize total value without exceeding capacity.**

---

14. 0/1 KNAPSACK STATE

A classic state:

$$ dp[i][w] $$

meaning:

> **maximum value obtainable using first i items with capacity w.**

This is extremely important.

---

15. TWO CHOICES

For item i:

Don't take it

Value:

$$ dp[i-1][w] $$

Take it

Possible only if:

$$ w_i\le w $$

Then:

$$ v_i+dp[i-1][w-w_i] $$

Choose maximum.

---

16. KNAPSACK TRANSITION

If:

$$ w_i>w $$

then:

$$ \boxed{ dp[i][w]=dp[i-1][w] } $$

Otherwise:

$$ \boxed{ dp[i][w] = \max \left(dp[i-1][w], v_i+dp[i-1][w-w_i] \right) } $$

This recurrence is worth memorizing.

---

17. KNAPSACK BASE CASE

With zero items:

$$ dp[0][w]=0 $$

With zero capacity:

$$ dp[i][0]=0 $$

18. KNAPSACK EXAMPLE

Capacity:

$$ W=5 $$

Items:

Item	Weight	Value
1	2	12
2	1	10
3	3	20
4	2	15

Try important combinations.

Items 1 + 2 + 4:

Weight:

$$ 2+1+2=5 $$

Value:

$$ 12+10+15=37 $$

Items 3 + 4:

Weight:

$$ 5 $$

Value:

$$ 35 $$

So better:

$$ \boxed{37} $$

19. KNAPSACK COMPLEXITY

States:

$$ n\times W $$

Work per state:

$$ O(1) $$

Therefore:

$$ \boxed{O(nW)} $$

Space:

$$ \boxed{O(nW)} $$

for full table.

Can be optimized to:

$$ \boxed{O(W)} $$

space.

---

20. IMPORTANT GATE TRAP — PSEUDO-POLYNOMIAL

0/1 Knapsack DP:

$$ O(nW) $$

looks polynomial.

But input size for capacity W is approximately:

$$ \log W $$

bits.

Therefore:

$$ O(nW) $$

is **pseudo-polynomial**, not polynomial in encoded input size.

This is an important theoretical distinction.

---

21. WHY 1D KNAPSACK ITERATES BACKWARDS

Space-optimized DP:

```text
for each item:
    for w = W down to item_weight:
        dp[w] = max(dp[w],
                    value + dp[w-item_weight])
```

Why backwards?

Because each item can be used only once.

If you iterate forward, the updated value may reuse the same item again.

Backward traversal preserves:

$$ \boxed{\text{0/1 behavior}} $$

---

22. 0/1 VS UNBOUNDED KNAPSACK

0/1

Each item once.

Capacity loop:

> **backwards.**

Unbounded

Each item can be used repeatedly.

Capacity loop can often go:

> **forwards.**

This loop-direction distinction is very useful.

---

23. LONGEST COMMON SUBSEQUENCE — LCS

Given two sequences, find the longest sequence that appears in both in the same relative order, not necessarily contiguously.

Example:

```text
X = ABCDGH
Y = AEDFHR
```

One LCS:

```text
ADH
```

Length:

\[3 \]

---

24. SUBSEQUENCE VS SUBSTRING

Important.

Subsequence

Elements need not be adjacent.

Example:

```text
ABCDE
```

`ACE` is a subsequence.

Substring

Must be contiguous.

`BCD` is substring.

`ACE` is not.

GATE loves this distinction.

25. LCS STATE

Let:

$$ dp[i][j] $$

mean:

> **LCS length of first i characters of X and first j characters of Y.**

26. LCS TRANSITION — MATCH

If:

$$ X[i-1]=Y[j-1] $$

then that character can extend the LCS:

$$ \boxed{ dp[i][j]=1+dp[i-1][j-1] } $$

27. LCS TRANSITION — NO MATCH

If:

$$ X[i-1]\ne Y[j-1] $$

then:

$$ \boxed{ dp[i][j] = \max(dp[i-1][j], dp[i][j-1]) } $$

28. LCS BASE CASE

If either string is empty:

$$ dp[0][j]=0 $$

$$ dp[i][0]=0 $$

29. LCS EXAMPLE

```text
X = ABC
Y = AC
```

DP table:

	""	A	C
""	0	0	0

	""	A	C
A	0	1	1
B	0	1	1
C	0	1	2

Therefore:

$$ \boxed{LCS=2} $$

Sequence:

```text
AC
```

---

30. LCS COMPLEXITY

If:

$$ m=|X| $$

$$ n=|Y| $$

states:

$$ mn $$

Work per state:

$$ O(1) $$

Therefore:

$$ \boxed{O(mn)} $$

Space:

$$ O(mn) $$

or:

$$ O(\min(m,n)) $$

for length-only space optimization.

---

31. MATRIX CHAIN MULTIPLICATION— MCM

Suppose matrices:

$$ A_1,A_2,\ldots,A_n $$

Matrix multiplication is associative:

$$ (A_1A_2)A_3 = A_1(A_2A_3) $$

But the **number of scalar multiplications** can differ dramatically depending on parenthesization.

Goal:

> **find parenthesization with minimum multiplication cost.**

32. MCM EXAMPLE

Matrices:

$$ A:10\times30 $$

$$ B:30\times5 $$

$$ C:5\times60 $$

Two choices.

$(AB)C$

First:

$$ 10\cdot30\cdot5=1500 $$

Result dimension:

$$ 10\times5 $$

Then multiply with C:

$$ 10\cdot5\cdot60=3000 $$

Total:

$$ \boxed{4500} $$

$A(BC)$

First:

$$ 30\cdot5\cdot60=9000 $$

Result:

$$ 30\times60 $$

Then:

$$ 10\cdot30\cdot60=18000 $$

Total:

$$ 27000 $$

So optimal:

$$ \boxed{4500} $$

---

33. MCM DIMENSION ARRAY

If:

$$ A_i $$

has dimensions:

$$ p_{i-1}\times p_i $$

then matrices are represented using:

$$ p_0,p_1,\ldots,p_n $$

Example:

```
TEXT
A1 = 10×30
A2 = 30×5
A3 = 5×60
```

Dimension array:

$$ [10,30,5,60] $$

---

34. MCM STATE

$$ dp[i][j] $$

means:

> **minimum scalar multiplications needed for matrices $A_i\ldots A_j$.**

---

35. MCM TRANSITION

Try every split point:

$$ k=i,\ldots,j-1 $$

Cost:

$$ dp[i][k] + dp[k+1][j] + p_{i-1}p_kp_j $$

Therefore:

$$ \boxed{ dp[i][j] = \min_{i\le k<j} \left[dp[i][k] + dp[k+1][j] + p_{i-1}p_kp_j \right] } $$

Important recurrence.

---

36. MCM BASE CASE

One matrix requires no multiplication:

$$ \boxed{dp[i][i]=0} $$

---

37. MCM COMPLEXITY

States:

$$ O(n^2) $$

For each state, try:

$$ O(n) $$

split points.

Therefore:

$$ \boxed{O(n^3)} $$

Space:

$$ \boxed{O(n^2)} $$

---

38. LONGEST INCREASING SUBSEQUENCE — LIS

Given:

```text
[10,22,9,33,21,50,41,60]
```

One LIS:

```text
10,22,33,50,60
```

Length:

$$ 5 $$

---

39. SIMPLE DP FOR LIS

Define:

$$ dp[i] $$

as:

> **length of LIS ending exactly at index i.**

Then:

$$ dp[i] = 1+ \max(dp[j]) $$

for all:

$$ j<i $$

such that:

$$ A[j]<A[i] $$

If no such j:

$$ dp[i]=1 $$

Answer:

$$ \max_i dp[i] $$

---

40. LIS DP COMPLEXITY

For every `i`, scan previous `j`.

\[O(n^2) \]

There is a more advanced:

\[O(n\log n) \]

method using binary search, but for DP understanding the quadratic version is fundamental.

---

41. COIN CHANGE — MINIMUM COINS

Given coin denominations and target amount.

Example:

```text
coins = {1,3,4}
amount = 6
```

Greedy failed:

```text
 4 + 1 + 1  !'   3   c o i n s
```

DP finds:

```text
 3 + 3  !'   2   c o i n s
```

---

42. COIN CHANGE STATE

\[dp[x] \]

means:

> **minimum coins required to form amount \(x\).**

Base:

\[dp[0]=0 \]

Transition:

\[\boxed{ dp[x] = 1+\min_{c\le x}dp[x-c] } \]

over available coins \(c\).

---

43. COIN CHANGE COMPLEXITY

If:

- target = A,
- number of coin types = k,

then typical DP:

$$\boxed{O(Ak)}$$

Again pseudo-polynomial because amount is numeric value.

---

44. COUNTING NUMBER OF WAYS — DIFFERENT DP

Coin Change can also ask:

> **How many ways can amount be formed?**

That is a different state/transition.

Important:

> **Same problem data can lead to different DP depending on what is being asked.**

Always define state carefully.

---

45. FLOYD-WARSHALL — DP PREVIEW

This will be studied properly in shortest paths.

Floyd-Warshall computes:

> **all-pairs shortest paths.**

DP idea:

Let:

$$D^{(k)}[i][j]$$

represent shortest path from i to j using only vertices:

$$1,\ldots,k$$

as possible intermediate vertices.

Transition:

$$\boxed{ D^{(k)}[i][j] = \min \left(D^{(k-1)}[i][j], D^{(k-1)}[i][k]+D^{(k-1)}[k][j] \right) }$$

Complexity:

$$\boxed{O(V^3)}$$

This is a beautiful graph application of DP.

---

46. HOW TO DESIGN A DP SOLUTION

Use this sequence:

Step 1 — Define state

Ask:

> **What information uniquely describes a smaller problem?**

Step 2 — Write recurrence/transition

Ask:

> **How can current state use smaller states?**

Step 3 — Set base cases

Step 4 — Decide computation order

Step 5 — Return correct final state

Step 6 — Analyze states × work per state

---

47. DP STATE EXAMPLES

Fibonacci

$$ dp[i] $$

= Fibonacci value i.

Knapsack

$$ dp[i][w] $$

= best value using first i items and capacity w.

LCS

$$ dp[i][j] $$

= LCS of prefixes of lengths i,j.

MCM

$$ dp[i][j] $$

= best multiplication cost from matrix i to j.

The state definition is often the hardest and most important part.

---

48. TOP-DOWN CALL GRAPH VS CALL TREE

Naive recursion may have a huge **call tree**.

Memoization transforms repeated nodes into reused results.

Conceptually, it reduces computation from a tree to a DAG of unique states.

Example:

```text
F(5)
!“
unique states:
F(5),F(4),F(3),F(2),F(1),F(0)
```

Only:

\[O(n) \]

unique states.

---

49. DP DOES NOT MEAN “USE A 2D TABLE”

Wrong.

DP can use:

- one variable,
- 1D array,
- 2D table,
- 3D table,
- hash map,
- recursion + memoization.

What makes it DP is:

> **storing/reusing solutions to subproblems.**

---

50. DP DOES NOT ALWAYS REQUIRE OPTIMIZATION PROBLEM

DP can compute:

- counts,
- boolean feasibility,
- minimum,
- maximum,
- sequence lengths,
- shortest paths.

Examples:

```text
C a n  s u b s e t  s u m  b e  f o r m e d ? !’  b o o l e a n
H o w  m a n y  w a y s ?                     !’  c o u n t
M i n i m u m  c o i n s ?                     !’  m i n i m u m
L C S  l e n g t h ?                           !’  m a x i m u m
```

---

51. DP AND SHORTEST PATHS

Some shortest-path algorithms can be viewed through DP.

Examples:

- Bellman-Ford
- Floyd-Warshall
- DAG shortest paths conceptually

But Dijkstra is primarily Greedy.

This distinction will matter later.

---

52. GREEDY VS DP VS D&C MASTER TABLE

Feature	Greedy	DP	Divide & Conquer
Main idea	Best choice now	Store repeated states	Split independent problems
Reconsider possibilities	Usually no	Yes through states	Solve all needed subproblems
Overlap	Not defining feature	Yes	Usually no
Storage	Usually low	Important	Usually recursion
Example	Fractional Knapsack	0/1 Knapsack	Merge Sort

---

53. RECOGNITION RULES

If same recursive states repeat:

Think:

\[\boxed{\text{DP}} \]

If locally best choice can be safely committed:

Think:

\[\boxed{\text{Greedy}} \]

If independent smaller instances appear:

Think:

\[\boxed{\text{Divide and Conquer}} \]

---

54. DP COMPLEXITY — NUMBER OF STATES

Example:

\[dp[i][j] \]

where:

$$ 1\le i\le n $$

and:

$$ 1\le j\le m $$

Number of states:

$$ nm $$

If each transition is:

$$ O(1) $$

then:

$$ \boxed{O(nm)} $$

If each state tries n possibilities:

$$ O(n^2m) $$

This is the fastest way to analyze many DP algorithms.

---

55. WHY MCM IS $O(n^3)$

States:

$$ (i,j) $$

Number:

$$ O(n^2) $$

For each state, try every split:

$$ O(n) $$

Therefore:

$$ \boxed{O(n^3)} $$

---

56. WHY LCS IS $O(mn)$

States:

$$ (i,j) $$

Number:

$$ mn $$

Transition:

$$ O(1) $$

Therefore:

$$ \boxed{O(mn)} $$

---

57. WHY KNAPSACK IS $O(nW)$

States:

$$ (i,w) $$

Number:

$$ nW $$

Transition:

$$ O(1) $$

Therefore:

$$ \boxed{O(nW)} $$

---

58. DP TABLE FILLING ORDER

Transition dependencies determine table order.

Example LCS:

$$ dp[i][j] $$

depends on:

```TEXT
dp[i-1][j]
dp[i][j-1]
dp[i-1][j-1]
```

So fill:

> **top-to-bottom, left-to-right.**

MCM is different.

It depends on shorter intervals.

Therefore fill by:

> **increasing chain length.**

This is why computation order matters.

---

59. MCM FILL ORDER

First:

$$ dp[i][i]=0 $$

Then chains of length 2.

Then length 3.

Then length 4.

...

Because larger intervals depend on smaller intervals.

60. MEMOIZATION CAN AVOID UNUSED STATES

Suppose a DP table theoretically contains many possible states but actual recursion reaches only some.

Memoization may compute only reachable states.

Tabulation often computes all table states.

This is one potential advantage of memoization.

61. TABULATION ADVANTAGES

Tabulation often:

- avoids recursion overhead,
- avoids stack overflow,
- can be easier to space-optimize,
- has predictable computation order.

Neither approach is universally superior.

62. GATE TRAPS

& þTrap 1 — Any recursion is DP

False.

DP requires stored/reused overlapping states.

& þTrap 2 — Optimal substructure alone implies DP

Not necessarily.

Greedy and D&C problems may also have optimal substructure.

& þTrap 3 — Memoization and tabulation are different algorithm paradigms

No.

Both are DP techniques.

& þTrap 4 — DP always uses \(O(n^2)\) space

False.

State dimension determines space.

& þTrap 5 — 0/1 Knapsack uses ratio Greedy

False.

& þTrap 6 — Fractional Knapsack requires DP

False.

Greedy solves it optimally.

& þTrap 7 — LCS means longest common substring

Wrong.

Subsequence does not require contiguity.

& þTrap 8 — MCM changes order of matrices

No.

Matrix order stays:

\[A_1A_2\ldots A_n \]

Only parenthesization changes.

& þTrap 9 — \(O(nW)\) means strongly polynomial in input length

No.

It is pseudo-polynomial.

& þTrap 10 — Forward loop is safe for 1D 0/1 Knapsack

It can reuse the same item.

Use backward capacity traversal.

63. EXAM DECISION RULES

Same states repeat?

\[\boxed{\text{DP}} \]

Independent halves?

\[\boxed{\text{D&C}} \]

Safe local choice?

\[\boxed{\text{Greedy}} \]

0/1 Knapsack?

$$ \boxed{\text{DP}} $$

Fractional Knapsack?

$$ \boxed{\text{Greedy}} $$

Two strings + longest common subsequence?

$$ \boxed{O(mn)\text{ DP}} $$

Matrix parenthesization?

$$ \boxed{\text{MCM DP}} $$

DP complexity?

Think:

$$ \boxed{\text{states × transition cost}} $$

---

64. PYQ-STYLE QUESTION 1

Naive Fibonacci:

$$ F(n)=F(n-1)+F(n-2) $$

Why does DP help?

A. Independent subproblems B. Repeated overlapping subproblems C. Sorting required D. Greedy-choice property

Answer:

$$ \boxed{B} $$

---

65. PYQ-STYLE QUESTION 2

0/1 Knapsack transition:

$$ dp[i][w] = \max(dp[i-1][w], v_i+dp[i-1][w-w_i]) $$

when:

$$ w_i\le w $$

Correct.

It represents:

```text
skip item
vs
take item
```

---

66. PYQ-STYLE QUESTION 3

LCS:

```text
X = ABC
Y = AC
```

LCS length:

$$ \boxed{2} $$

67. PYQ-STYLE QUESTION 4

LCS transition when characters match:

$$ \boxed{ 1+dp[i-1][j-1] } $$

When they do not match:

$$ \boxed{ \max(dp[i-1][j],dp[i][j-1]) } $$

68. PYQ-STYLE QUESTION 5

Matrices:

$$ 10\times30,\quad30\times5,\quad5\times60 $$

Optimal multiplication cost:

$$ \boxed{4500} $$

from:

$$ (AB)C $$

69. PYQ-STYLE QUESTION 6

MCM has:

$$ O(n^2) $$

states and:

$$ O(n) $$

split choices per state.

Therefore:

$$ \boxed{O(n^3)} $$

70. PYQ-STYLE QUESTION 7

0/1 Knapsack with n items and capacity W:

Time:

$$ \boxed{O(nW)} $$

Full-table space:

$$ \boxed{O(nW)} $$

71. PYQ-STYLE QUESTION 8 — TRAP

Is:

$$ O(nW) $$

polynomial in the bit-length of W?

No.

Bit-length:

$$ \Theta(\log W) $$

Thus this is:

$$ \boxed{\text{pseudo-polynomial}} $$

72. QUICK RECALL

Try mentally.

1. Two main DP properties?
2. Memoization direction?
3. Tabulation direction?
4. Three things to identify in a DP?
5. Fibonacci DP time?
6. 0/1 Knapsack state?
7. 0/1 Knapsack time?
8. Fractional vs 0/1?
9. LCS state?
10. LCS time?
11. MCM goal?
12. MCM time?
13. LIS simple DP time?
14. Why 1D 0/1 Knapsack goes backward?
15. DP complexity shortcut?

ANSWERS

1. Overlapping subproblems + optimal substructure.
2. Top-down.
3. Bottom-up.
4. State, transition, base case.
5. $O(n)$.
6. $dp[i][w]$.
7. $O(nW)$.

8. Fractional Greedy; 0/1 DP.
9. $dp[i][j]$ for prefixes.
10. $O(mn)$.
11. Minimize scalar multiplications through parenthesization.
12. $O(n^3)$.
13. $O(n^2)$.
14. To avoid reusing same item.
15. States × work per state.

73. PRACTICE SET

Easy

Q1 — MCQ

Which property most directly distinguishes DP from basic Divide and Conquer?

A. Sorting B. Overlapping subproblems C. Recursion D. Graphs

Q2 — MCQ

0/1 Knapsack is classically solved using:

A. Greedy ratio B. Dynamic Programming C. Binary Search D. Selection Sort

Q3 — MCQ

Memoization is:

A. bottom-up only B. top-down caching C. greedy selection D. sorting

Medium

Q4 — NAT

For strings of lengths:

$$ m=20,\quad n=30 $$

how many states are there in a straightforward LCS DP table, ignoring the extra zero row/column for this count?

Q5 — MSQ

Which are classic DP problems?

A. 0/1 Knapsack B. LCS C. Matrix Chain Multiplication D. Fractional Knapsack

Q6 — MCQ

If:

$$ X[i-1]=Y[j-1] $$

then LCS transition is:

A. $dp[i-1][j]$

B. $dp[i][j-1]$

C. $1+dp[i-1][j-1]$

D. 0

Q7 — MCQ

Simple MCM DP complexity:

A. $O(n)$ B. $O(n^2)$ C. $O(n^3)$ D. $O(2^n)$

Hard / Tricky

Q8 — MSQ

Which statements are correct?

A. Every DP problem must use a 2D array.

B. Memoization stores recursive results.

C. Optimal substructure appears in both Greedy and DP problems.

D. 0/1 Knapsack's $O(nW)$ algorithm is pseudo-polynomial.

Q9 — MCQ

In space-optimized 0/1 Knapsack, iterating capacity from low to high may:

A. make Binary Search fail B. allow one item to be reused in same iteration C. improve stability D. convert it to Merge Sort

---

SOLUTIONS

Q1

$$\boxed{B}$$

Q2

$$\boxed{B}$$

Q3

$$\boxed{B}$$

Q4

$$ 20\times30 = \boxed{600} $$

Q5

0/1 Knapsack, LCS, MCM.

$$ \boxed{A,B,C} $$

Fractional Knapsack is Greedy.

Q6

$$ \boxed{C} $$

Q7

$$ \boxed{C} $$

Q8

A false.

B true.

C true.

D true.

Answer:

$$ \boxed{B,C,D} $$

Q9

Forward traversal can use the same newly updated state and effectively reuse the item.

$$ \boxed{B} $$

74. Ø=ÜÓTES TO WRITE

Dynamic Programming

Use when:

```text
Overlapping subproblems
+
Optimal substructure
```

Main concepts:

```text
STATE
TRANSITION
BASE CASE
```

Memoization

Top-down.

```text
Recursion + cache
```

Tabulation

Bottom-up.

```text
 B a s e   c a s e s  !'   l a r g e r   s t a t e s
```

Complexity Rule

$$ \boxed{ \text{Time} = \text{number of states} \times \text{work per state} } $$

Fibonacci

Naive:

$$ O(2^n) $$

DP:

$$ O(n) $$

Space can be:

$$ O(1) $$

0/1 Knapsack

State:

$$ dp[i][w] $$

Transition:

$$ dp[i][w] = \max(dp[i-1][w], v_i+dp[i-1][w-w_i]) $$

Time:

$$ \boxed{O(nW)} $$

Pseudo-polynomial.

1D optimization:

> **iterate capacity backward.**

LCS

If characters equal:

$$ dp[i][j] = 1+dp[i-1][j-1] $$

Else:

$$ dp[i][j] = \max(dp[i-1][j],dp[i][j-1]) $$

Time:

$$ \boxed{O(mn)} $$

Matrix Chain Multiplication

State:

$$ dp[i][j] $$

Transition:

$$ dp[i][j] = \min_k \{ dp[i][k] + dp[k+1][j] + p_{i-1}p_kp_j \} $$

Time:

$$ \boxed{O(n^3)} $$

Space:

$$ O(n^2) $$

LIS

Simple DP:

$$ O(n^2) $$

Coin Change

Minimum coins:

$$ dp[x] = 1+\min dp[x-c] $$

Typical:

$$ O(Ak) $$

---

75. MEMORY TRICKS

DP

> **Don't Process twice — store it.**

DP recipe

> **S-T-B**

```text
State
Transition
Base
```

Paradigm recognition

> **Independent !’ D&C**

> **Repeated !’ DP**

> **Local safe choice !’ Greedy**

Knapsack

> **Fraction = Greedy**

> **0/1 = DP**

LCS

> **Match !’ diagonal + 1**

> **Mismatch !’ max(top, left)**

MCM

> **Try every split.**

&β30-SECOND REVISION

1. DP = solve repeated subproblems once and reuse answers.
2. DP needs overlapping subproblems + optimal substructure.
3. Memoization = top-down.
4. Tabulation = bottom-up.
5. DP recipe = State, Transition, Base.
6. Complexity "Η states × transition work.
7. Fibonacci naive exponential !’ DP \(O(n)\).
8. 0/1 Knapsack !’ DP.
9. Knapsack time = \(O(nW)\), pseudo-polynomial.
10. 1D 0/1 Knapsack capacity goes backward.
11. LCS "`substring.
12. LCS match !’ diagonal + 1.
13. LCS mismatch !’ max(top, left).
14. LCS time = \(O(mn)\).
15. MCM changes parenthesization, not matrix order.
16. MCM time = \(O(n^3)\).
17. LIS simple DP = \(O(n^2)\).
18. Floyd-Warshall is a graph DP with \(O(V^3)\).
19. Greedy commits; DP keeps alternatives through states.
20. D&C subproblems usually don't overlap.

' Chapter 12 Complete

Next Chapter:

Chapter 13 — Graph Foundations + BFS + DFS

We’ll cover:

- graph terminology,
- directed vs undirected,
- weighted vs unweighted,
- path, cycle, connected component,
- adjacency matrix vs adjacency list,
- BFS,
- DFS,
- traversal order,
- BFS/DFS trees,
- reachability,
- connected components,
- cycle detection,
- topological sorting where relevant,
- and the crucial complexity:

$$\boxed{O(V+E)}$$

for adjacency-list traversals.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

CHAPTER 13 — GRAPH FOUNDATIONS + BFS + DFS

GATE Importance: & & & & &

Your uploaded Algorithms plan explicitly includes graph terminology, graph representations, BFS, DFS, reachability, connected components, cycle detection, traversal order, and related GATE questions.

The core idea:

> **A graph represents relationships. BFS explores level by level; DFS explores as deep as possible before backtracking.**

---

1. WHAT IS A GRAPH?

A graph is written as:

$$\boxed{G=(V,E)}$$

where:

- V = set of vertices/nodes
- E = set of edges

Example:

```
A ----- B
|       |
|       |
C ----- D
```

Vertices:

$$V=\{A,B,C,D\}$$

Edges:

$$E=\{AB,AC,BD,CD\}$$

---

2. IMPORTANT GRAPH TERMINOLOGY

Vertex

A node.

Example:

```text
A
```

Edge

A connection between two vertices.

Example:

$$ (A,B) $$

Adjacent Vertices

Two vertices connected directly by an edge.

If:

$$ (A,B)\in E $$

then A and B are adjacent.

Degree

For an undirected graph:

> **Degree of a vertex = number of incident edges.**

Notation:

$$ deg(v) $$

Example:

```text
    B
    |
A---C---D
```

For C:

$$ deg(C)=3 $$

3. HANDSHAKING LEMMA

For an undirected graph:

$$ \boxed{ \sum_{v\in V}deg(v)=2|E| } $$

Why?

Every edge touches exactly two endpoints.

Example:

If:

$$ |E|=5 $$

then sum of all degrees:

$$ \boxed{10} $$

---

4. IMPORTANT CONSEQUENCE

The number of vertices having **odd degree** in an undirected graph is always:

$$ \boxed{\text{even}} $$

This is a classic graph fact that GATE may use indirectly.

---

5. DIRECTED GRAPH

In a directed graph, edges have direction.

Example:

```text
 A   !’    B
!’          |
 |        !“
 D   !•    C
```

Edge:

$$ (A,B) $$

does not automatically imply:

$$ (B,A) $$

---

6. INDEGREE AND OUTDEGREE

For directed graph:

Indegree

Number of incoming edges.

$$ deg^-(v) $$

Outdegree

Number of outgoing edges.

$$ deg^+(v) $$

---

7. DIRECTED GRAPH DEGREE RESULTS

For any directed graph:

$$\boxed{ \sum indegree = |E| }$$

and:

$$\boxed{ \sum outdegree = |E| }$$

Therefore:

$$\boxed{ \sum indegree = \sum outdegree }$$

---

8. WEIGHTED GRAPH

Edges may have costs/weights.

Example:

```text
A --5-- B
|       |
2       7
|       |
C --3-- D
```

Weights may represent:

- distance
- time
- price
- cost
- capacity

These become important for:

- MST
- shortest paths

---

9. UNWEIGHTED GRAPH

Edges have no explicit numerical cost.

Conceptually, each edge can often be treated as equal cost.

For shortest path in an unweighted graph:

$$\boxed{\text{BFS}}$$

will be extremely important.

---

10. PATH

A path is a sequence of vertices connected by edges.

Example:

```TEXT
 A  !′   B  !′   C  !′   D
```

Path:

\[A,B,C,D \]

Length in an unweighted graph is usually number of edges:

\[3 \]

---

11. SIMPLE PATH

A path with no repeated vertices.

Example:

```TEXT
 A  !′   B  !′   C
```

Simple.

But:

```TEXT
 A  !′   B  !′   C  !′   B  !′   D
```

is not simple because B repeats.

---

12. CYCLE

A path that starts and ends at the same vertex.

Example:

```TEXT
 A  !′   B
!′      !“
 D  !•   C
```

Cycle:

\[A\to B\to C\to D\to A \]

---

13. CONNECTED GRAPH

An undirected graph is connected if every vertex can reach every other vertex.

Example:

```TEXT
A---B---C
    |
```

```
    D
```

All vertices are connected.

14. DISCONNECTED GRAPH

Example:

```text
A---B       C---D
```

There are two connected components:

```text
{A,B}
{C,D}
```

15. CONNECTED COMPONENT

A connected component is a maximal connected subgraph.

For:

```text
A---B      C---D      E
```

components:

\[\{A,B\} \]

\[\{C,D\} \]

\[\{E\} \]

Number:

\[\boxed{3} \]

16. COMPLETE GRAPH

A complete undirected graph has an edge between every pair of distinct vertices.

Notation:

\[K_n \]

Number of edges:

\[\boxed{ \frac{n(n-1)}2 } \]

Example:

\[K_4 \]

has:

$$ \frac{4\cdot3}{2}=6 $$

edges.

17. COMPLETE DIRECTED GRAPH

If every ordered pair of distinct vertices has an edge:

$$ \boxed{n(n-1)} $$

directed edges.

No self-loops assumed.

18. TREE

A tree is an undirected graph that is:

```TEXT
connected
+
acyclic
```

For n vertices, every tree has:

$$ \boxed{n-1} $$

edges.

This fact is extremely important for MST later.

19. TREE EQUIVALENT PROPERTIES

For an undirected graph with n vertices, several equivalent properties are useful.

A tree:

- is connected and has $n-1$ edges,
- is acyclic and has $n-1$ edges,
- has exactly one simple path between every pair of vertices.

20. GRAPH REPRESENTATIONS

Two main representations:

1. Adjacency Matrix
2. Adjacency List

The representation strongly affects algorithm complexity.

21. ADJACENCY MATRIX

For V vertices, use:

$$ V\times V $$

matrix.

Example graph:

```text
A---B
|   |
C---D
```

Matrix:

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

---

22. MATRIX PROPERTY FOR UNDIRECTED GRAPH

Adjacency matrix is symmetric:

$$ \boxed{A[i][j]=A[j][i]} $$

because if:

$$ i\leftrightarrow j $$

then both directions are represented.

---

23. ADJACENCY MATRIX SPACE

$$ \boxed{\Theta(V^2)} $$

regardless of how many edges exist.

Thus it can waste space for sparse graphs.

---

24. ADJACENCY MATRIX EDGE CHECK

To ask:

> **Is edge (u,v) present?**

Simply check:

$$ A[u][v] $$

Time:

$$\boxed{O(1)}$$

Excellent advantage.

25. ADJACENCY LIST

Store neighbors of each vertex.

Example:

```TEXT
A  !'   B ,   C
B  !'   A ,   D
C  !'   A ,   D
D  !'   B ,   C
```

For an undirected graph, each edge appears twice.

26. ADJACENCY LIST SPACE

For undirected graph:

$$\boxed{\Theta(V+E)}$$

because:

- one list per vertex,
- approximately $2E$ adjacency entries.

For directed graph also:

$$\Theta(V+E)$$

27. SPARSE VS DENSE GRAPHS

Sparse

Relatively few edges.

Typically:

$$E\ll V^2$$

Adjacency list is usually preferable.

Dense

Edges near maximum:

$$E=\Theta(V^2)$$

Adjacency matrix can be reasonable.

28. REPRESENTATION COMPARISON

Operation/Property	Matrix	List
Space	$O(V^2)$	$O(V+E)$
Check edge u,v	$O(1)$	$O(\deg(u))$ typically
Iterate neighbors	$O(V)$	$O(\deg(u))$
Good for sparse graphs	No	Yes
Good for dense graphs	Often	Yes, but matrix may simplify

---

29. GRAPH TRAVERSAL

Traversal means systematically visiting graph vertices.

Two fundamental algorithms:

$$\boxed{\text{BFS}}$$

and:

$$\boxed{\text{DFS}}$$

Both can be used for:

- reachability
- connected components
- traversal trees
- cycle-related logic
- many graph algorithms

---

30. BFS — BREADTH-FIRST SEARCH

BFS explores:

> **level by level.**

It uses a:

$$\boxed{\text{queue}}$$

Memory:

> **BFS = Broad First = Queue**

---

31. BFS INTUITION

Graph:

```text
      A
    / | \
   B  C  D
  / \    |
 E   F   G
```

Starting from A:

Level 0:

```text
A
```

Level 1:

```text
B C D
```

Level 2:

```text
E F G
```

Possible BFS:

```text
A, B, C, D, E, F, G
```

assuming adjacency order shown.

---

32. BFS PSEUDOCODE

```text
BFS(G, s):

    mark s visited
    enqueue s

    while queue not empty:

        u = dequeue

        for each neighbor v of u:

            if v not visited:
                mark v visited
                enqueue v
```

---

33. WHY MARK WHEN ENQUEUING?

Important.

Correct approach:

> **mark a vertex visited as soon as it is discovered/enqueued.**

If we wait until dequeue, multiple neighbors may enqueue the same vertex repeatedly.

That creates unnecessary duplicates.

---

34. BFS TRACE

Graph:

```text
A---B---D
|   |
C---E
```

Adjacency order alphabetically.

Start:

\[A \]

Queue:

```text
[A]
```

Remove A.

Discover:

```text
B, C
```

Queue:

```text
[B,C]
```

Remove B.

Neighbors:

```text
A,D,E
```

A visited.

Add:

```text
D,E
```

Queue:

```text
[C,D,E]
```

C produces no new node.

Then D, E.

Traversal:

\[\boxed{A,B,C,D,E} \]

---

35. BFS TREE

When a vertex is discovered first, the edge used becomes a tree edge.

From previous graph:

```text
A
% % %    B
%       % % %    D
%       % % %    E
% % %    C
```

This is a BFS tree.

---

36. BFS SHORTEST PATH PROPERTY

For an **unweighted graph**, BFS finds shortest paths from source in terms of:

\[\boxed{\text{minimum number of edges}} \]

Why?

Because BFS visits vertices by increasing distance:

```text
distance 0
distance 1
distance 2
...
```

This is one of the most important GATE facts.

---

37. BFS DISTANCE

Set:

\[dist[s]=0 \]

When discovering \(v\) from \(u\):

\[\boxed{ dist[v]=dist[u]+1 } \]

The first discovery gives shortest unweighted distance.

38. BFS DOES NOT GENERALLY SOLVE WEIGHTED SHORTEST PATH

Suppose:

```text
A --100-- B
 \       /
  1     1
   \   /
     C
```

From A to B:

Direct edge:

\[100 \]

Two-edge path:

\[A\to C\to B \]

cost:

\[2 \]

BFS prefers fewer edges, not smaller weighted cost.

Therefore:

> **Standard BFS is shortest path for unweighted/equal-weight edges, not arbitrary weighted graphs.**

39. BFS COMPLEXITY — ADJACENCY LIST

Each vertex is processed once:

\[O(V) \]

Every edge is examined once for directed, twice for undirected, still:

\[O(E) \]

Therefore:

\[\boxed{O(V+E)} \]

This is a must-memorize result.

40. BFS COMPLEXITY — ADJACENCY MATRIX

For every vertex, scanning its whole row costs:

\[O(V) \]

for \(V\) vertices:

\[\boxed{O(V^2)} \]

So representation matters.

41. BFS SPACE

Visited array:

\[O(V) \]

Queue:

\[O(V) \]

Hence:

\[\boxed{O(V)} \]

auxiliary space.

42. DFS — DEPTH-FIRST SEARCH

DFS explores:

> **as deep as possible along one path before backtracking.**

Uses:

- recursion stack, or
- explicit stack.

Memory:

> **DFS = Dive First = Stack**

43. DFS INTUITION

Graph:

```text
      A
    / | \
   B  C  D
  / \
 E   F
```

Possible DFS:

```text
A
!'   B
!'   E
!•   b a c k
!'   F
!•   b a c k
!•   A
```

```
!’   C
!•
!’   D
```

Traversal:

\[\boxed{A,B,E,F,C,D}\]

depending on adjacency order.

---

44. DFS PSEUDOCODE

Recursive:

```text
DFS(u):

    mark u visited

    for each neighbor v of u:

        if v not visited:
            DFS(v)
```

---

45. DFS ITERATIVE

Use stack:

```text
push source

while stack not empty:
    pop vertex
    process unvisited neighbors
```

But exact traversal order can differ from recursive DFS depending on push order.

---

46. DFS TREE

The recursive discovery edges form a DFS tree.

Example:

```text
A
% % %    B
         % % %    E
         % % %    F
```

Then after backtracking, other branches continue.

If graph is disconnected, repeated DFS produces a:

\[\boxed{\text{DFS forest}} \]

47. DFS COMPLEXITY — ADJACENCY LIST

Every vertex:

$$ O(V) $$

Every edge examined:

$$ O(E) $$

Thus:

$$ \boxed{O(V+E)} $$

Same asymptotic time as BFS.

48. DFS COMPLEXITY — MATRIX

$$ \boxed{O(V^2)} $$

because each adjacency row may be scanned completely.

49. DFS AUXILIARY SPACE

Visited:

$$ O(V) $$

Recursion/stack depth worst case:

$$ O(V) $$

Therefore:

$$ \boxed{O(V)} $$

50. BFS VS DFS

Property	BFS	DFS
Main structure	Queue	Stack/recursion
Exploration	Level-by-level	Deep then backtrack
Unweighted shortest path	Yes	No guarantee
Time with list	$O(V+E)$	$O(V+E)$
Space worst case	$O(V)$	$O(V)$
Connected components	Yes	Yes
Reachability	Yes	Yes

51. TRAVERSAL ORDER IS NOT ALWAYS UNIQUE

Suppose:

```text
A connected to B,C,D
```

If adjacency order is:

```text
B,C,D
```

BFS may begin:

```text
A,B,C,D
```

If adjacency order is:

```text
D,C,B
```

BFS may begin:

```text
A,D,C,B
```

Therefore:

> **Traversal order depends on neighbor ordering unless explicitly specified.**

Very important for GATE.

---

52. REACHABILITY

To determine whether \(t\) is reachable from \(s\):

Run BFS or DFS from \(s\).

If \(t\) becomes visited:

\[\boxed{\text{reachable}} \]

Time:

\[O(V+E) \]

with adjacency list.

---

53. CONNECTED COMPONENTS USING DFS/BFS

For an undirected graph:

```text
count = 0

for every vertex v:
    if v unvisited:
        count++
        DFS(v)
```

Each new DFS marks one entire connected component.

---

54. CONNECTED COMPONENT COMPLEXITY

Every vertex and edge is processed across all traversals only once overall.

Therefore:

$$ \boxed{O(V+E)} $$

not:

$$ O(V(V+E)) $$

This can be a trap.

---

55. EXAMPLE COMPONENTS

Graph:

```text
A---B    C---D    E
```

Start A:

```text
visit A,B
```

Component count:

$$ 1 $$

Next C:

```text
visit C,D
```

count:

$$ 2 $$

Next E:

```text
visit E
```

count:

$$ 3 $$

Answer:

$$ \boxed{3} $$

56. CYCLE DETECTION — UNDIRECTED GRAPH

Using DFS:

If while exploring vertex u, you find a visited neighbor v that is **not the parent** of u, then a cycle exists.

Why ignore parent?

Because in an undirected graph, the edge back to parent appears in the adjacency list naturally.

57. UNDIRECTED CYCLE EXAMPLE

```text
A---B
|   |
D---C
```

DFS:

```text
 A  !′   B  !′   C  !′   D
```

From D, neighbor A is already visited and is not D's parent.

Therefore:

$$ \boxed{\text{cycle exists}} $$

58. CYCLE DETECTION — DIRECTED GRAPH

Directed graphs need different logic.

A common DFS method uses:

- `visited`
- `recursionStack` / color states

If DFS encounters a vertex currently on the active recursion path, that is a:

$$ \boxed{\text{back edge}} $$

and implies a directed cycle.

59. DFS COLORS

Useful conceptual model:

```text
WHITE = unvisited
GRAY  = currently active in DFS
BLACK = finished
```

If an edge goes to a:

\[\boxed{\text{GRAY vertex}} \]

there is a directed cycle.

---

60. DFS DISCOVERY AND FINISH TIMES

DFS can assign:

\[d[u]=\text{discovery time} \]

\[f[u]=\text{finish time} \]

A vertex is discovered when first entered and finished after all descendants are processed.

Example nesting:

```text
discover A
  discover B
    finish B
  discover C
    finish C
finish A
```

Intervals of descendants nest inside ancestors.

This becomes useful for:

- edge classification,
- topological sorting,
- graph reasoning.

---

61. TOPOLOGICAL SORTING

A topological ordering of a directed graph is an ordering of vertices such that every directed edge:

\[u\to v \]

places:

\[u \]

before:

\[v \]

It exists only for a:

\[\boxed{\text{DAG}} \]

Directed Acyclic Graph.

---

62. TOPOLOGICAL ORDER EXAMPLE

Dependencies:

```text
A  !'   C
B  !'   C
C  !'   D
```

Possible topological order:

```text
A, B, C, D
```

or:

```text
B, A, C, D
```

Not necessarily unique.

63. DFS TOPOLOGICAL SORT

Run DFS.

When a vertex finishes, push it onto a stack.

Reverse finish order gives a topological ordering.

Why?

If:

\[u\to v \]

in a DAG, DFS finishing structure ensures appropriate precedence in reverse finish order.

64. TOPOLOGICAL SORT USING INDEGREE

Another method is Kahn's algorithm:

1. Compute indegrees.
2. Put all indegree-0 vertices in queue.
3. Repeatedly remove one.
4. Delete its outgoing edges conceptually.
5. Newly indegree-0 vertices enter queue.

Time:

\[\boxed{O(V+E)} \]

with adjacency list.

65. TOPOLOGICAL SORT AND CYCLES

If a directed graph has a cycle:

```text
 A  !′   B  !′   C  !′   A
```

No vertex arrangement can satisfy all precedence directions.

Thus:

\[\boxed{\text{DAG iff topological ordering exists}} \]

---

66. BFS TREE LEVEL PROPERTY

In BFS tree rooted at \(s\):

\[level(v)=dist(s,v) \]

for unweighted graph.

For every graph edge \((u,v)\) in an undirected graph, BFS levels differ by at most 1:

\[\boxed{|level(u)-level(v)|\le1} \]

This is a useful property.

---

67. DFS DOES NOT GUARANTEE SHORTEST PATH

Example:

```text
A---B
|   |
C---D---E
```

Depending on neighbor order, DFS may reach E through a long route even if a shorter route exists.

Therefore:

\[\boxed{\text{DFS is not an unweighted shortest-path algorithm}} \]

---

68. BFS FOR BIPARTITE CHECK — USEFUL SUPPORTING IDEA

A graph is bipartite if vertices can be colored using two colors so no edge joins same-colored vertices.

BFS can color levels alternately:

```text
0,1,0,1,...
```

If an edge joins vertices with same color:

\[\boxed{\text{not bipartite}} \]

Equivalent famous result:

> **An undirected graph is bipartite iff it contains no odd cycle.**

This is useful graph knowledge for GATE.

69. DFS/BFS ON DISCONNECTED GRAPH

Starting BFS/DFS from one source only visits that source's connected/reachable region.

To traverse the **whole graph**:

```text
for every vertex:
    if unvisited:
        BFS/DFS(vertex)
```

This creates a forest for disconnected graphs.

70. GRAPH EDGE LIMITS

For simple undirected graph with V vertices:

Maximum edges:

$$\boxed{\frac{V(V-1)}{2}}$$

Minimum edges for connected graph:

$$\boxed{V-1}$$

71. FOR DIRECTED SIMPLE GRAPH

Maximum directed edges without self-loops:

$$\boxed{V(V-1)}$$

72. TREE EDGE RESULT

If graph is connected and:

$$E=V-1$$

then it is a tree.

If graph is acyclic and:

$$E=V-1$$

and contains all V vertices, it is connected and hence a tree.

73. FOREST

An acyclic undirected graph may have multiple components.

If a forest has:

- \(V\) vertices
- \(C\) connected components

then:

\[\boxed{E=V-C} \]

For a tree:

\[C=1 \]

thus:

\[E=V-1 \]

---

74. BFS/DFS GATE NUMERICAL STRATEGY

When asked traversal order:

1. identify starting vertex,
2. note exact adjacency/tie-breaking order,
3. maintain queue for BFS,
4. maintain recursion/stack for DFS,
5. mark discovered vertices immediately,
6. do not assume alphabetical order unless specified.

---

75. BFS QUEUE TRACE TEMPLATE

Use:

Step	Removed	Newly Added	Queue
1	A	B,C	B,C
2	B	D	C,D
3	C	E	D,E

This prevents mistakes.

---

76. DFS TRACE TEMPLATE

Write:

```text
A
  % !′   B
          % !′   D
                  % !′   . . .
```

and explicitly backtrack.

DFS questions become much easier when you draw the recursion path.

77. PYQ-STYLE QUESTION 1

An undirected graph has:

$$ 10 $$

edges.

Sum of all degrees:

$$ 2E $$

$$ =20 $$

Answer:

$$ \boxed{20} $$

78. PYQ-STYLE QUESTION 2

A complete undirected graph has:

$$ n=8 $$

vertices.

Edges:

$$ \frac{8\cdot7}{2} $$

$$ \boxed{28} $$

79. PYQ-STYLE QUESTION 3

A tree has:

$$ 25 $$

vertices.

Edges:

$$ 25-1 $$

$$ \boxed{24} $$

80. PYQ-STYLE QUESTION 4

Which data structure is naturally used by BFS?

A. Stack B. Queue C. Heap D. Hash Table

Answer:

$$ \boxed{B} $$

81. PYQ-STYLE QUESTION 5

Which data structure is naturally used by DFS?

$$\boxed{\text{Stack / recursion}}$$

82. PYQ-STYLE QUESTION 6

BFS complexity with adjacency list:

$$\boxed{O(V+E)}$$

With adjacency matrix:

$$\boxed{O(V^2)}$$

83. PYQ-STYLE QUESTION 7

Which traversal guarantees shortest path in an unweighted graph?

$$\boxed{\text{BFS}}$$

84. PYQ-STYLE QUESTION 8

A graph has:

$$V=12$$

vertices and:

$$C=4$$

components and is a forest.

Edges:

$$E=V-C$$

$$=12-4$$

$$\boxed{8}$$

85. PYQ-STYLE QUESTION 9

Can a directed graph containing a cycle have a topological ordering?

$$\boxed{\text{No}}$$

Topological ordering exists iff graph is a DAG.

86. PYQ-STYLE QUESTION 10

Suppose an undirected graph has 7 vertices and 6 edges and is connected.

What can you conclude?

Since:

$$ E=V-1 $$

and graph is connected:

$$ \boxed{\text{It is a tree}} $$

---

87. & þGATE TRAPS

Trap 1 — BFS always gives cheapest weighted path

False.

BFS minimizes number of edges, not arbitrary weight.

Trap 2 — DFS gives shortest unweighted path

No guarantee.

Trap 3 — BFS and DFS always have unique order

False.

Neighbor/tie ordering matters.

Trap 4 — BFS complexity is always $O(V^2)$

Depends on representation.

Adjacency list:

$$ O(V+E) $$

Trap 5 — Adjacency matrix is always space efficient

No.

It always uses:

$$ O(V^2) $$

Trap 6 — An undirected edge appears once in adjacency lists

Typically appears twice:

```text
u lists v
v lists u
```

Still total:

$$ O(V+E) $$

Trap 7 — Any connected graph with $V-1$ edges might have a cycle

No.

A connected undirected graph with $V-1$ edges is a tree.

Trap 8 — Topological sorting works on any directed graph

Only DAGs.

Trap 9 — Visited parent in undirected DFS proves a cycle

No.

Parent edge is expected.

Need visited neighbor that is not parent.

Trap 10 — One BFS from any vertex visits a disconnected graph fully

False.

Only its component.

88. EXAM DECISION RULES

Unweighted shortest path?

$$ \boxed{\text{BFS}} $$

Need deep exploration/backtracking?

$$ \boxed{\text{DFS}} $$

Sparse graph?

Think:

$$ \boxed{\text{Adjacency List}} $$

Need constant-time edge existence check?

Think:

$$ \boxed{\text{Adjacency Matrix}} $$

Need connected-component count?

Run BFS/DFS from each unvisited vertex.

Directed prerequisites/dependencies?

Think:

$$\boxed{\text{DAG + Topological Sort}}$$

BFS?

Remember:

> **Queue + Levels + Unweighted shortest path**

DFS?

Remember:

> **Stack + Depth + Backtracking**

---

89. QUICK RECALL

Try mentally.

1. Graph notation?
2. Sum of degrees in undirected graph?
3. Number of edges in K_n?
4. Tree with n vertices has how many edges?
5. Adjacency matrix space?
6. Adjacency-list space?
7. BFS structure?
8. DFS structure?
9. BFS list complexity?
10. DFS list complexity?
11. BFS shortest path works for what type of graph?
12. Can topological sort exist with a directed cycle?
13. Connected components found using?
14. Forest formula with C components?
15. Is traversal order always unique?

ANSWERS

1. $G=(V,E)$.
2. $2E$.
3. $n(n-1)/2$.

4. $n-1$.
5. $O(V^2)$.
6. $O(V+E)$.
7. Queue.
8. Stack/recursion.
9. $O(V+E)$.
10. $O(V+E)$.
11. Unweighted/equal-edge-weight graph.
12. No.
13. BFS or DFS.
14. $E=V-C$.
15. No.

90. PRACTICE SET

Easy

Q1 — NAT

An undirected graph has 15 edges.

What is the sum of degrees of all vertices?

Q2 — MCQ

BFS uses:

A. Stack B. Queue C. Priority Queue D. Recursion only

Q3 — MCQ

DFS most naturally uses:

A. Queue B. Stack C. Hash function D. Sorting

Medium

Q4 — NAT

How many edges does a complete undirected graph with 10 vertices contain?

Q5 — MSQ

Which statements are correct?

A. BFS finds shortest number-of-edge paths in unweighted graphs.

B. DFS always gives shortest path.

C. BFS and DFS using adjacency lists are $O(V+E)$.

D. Adjacency matrix requires $O(V^2)$ space.

Q6 — MCQ

An undirected connected graph has:

$$V=20,\quad E=19$$

It must be:

A. complete B. tree C. disconnected D. directed

Q7 — NAT

A forest has:

$$V=18$$

vertices and:

$$C=5$$

components.

How many edges?

Hard / Tricky

Q8 — MSQ

Which statements are true?

A. A topological ordering can exist for a directed cyclic graph.

B. A DAG always has at least one topological ordering.

C. BFS traversal order may depend on adjacency-list order.

D. DFS traversal order may depend on adjacency-list order.

Q9 — MCQ

In DFS of an undirected graph, seeing a visited adjacent vertex proves a cycle if:

A. it is the parent B. it is not the parent C. it has degree 1 D. it is source

---

SOLUTIONS

Q1

$$2E=2(15)=\boxed{30}$$

Q2

$$\boxed{B}$$

Q3

$$\boxed{B}$$

Q4

$$\frac{10\cdot9}{2} = \boxed{45}$$

Q5

A true.

B false.

C true.

D true.

Answer:

$$\boxed{A,C,D}$$

Q6

Connected graph with:

$$E=V-1$$

is a tree.

$$\boxed{B}$$

Q7

Forest:

$$E=V-C$$

$$=18-5$$

$$\boxed{13}$$

Q8

A false.

B true.

C true.

D true.

Answer:

$$\boxed{B,C,D}$$

Q9

Visited neighbor other than parent indicates cycle.

$$\boxed{B}$$

91. Ø=ÑÓTES TO WRITE

Graph

$$\boxed{G=(V,E)}$$

Undirected:

$$\sum deg(v)=2E$$

Directed:

$$\sum indegree = \sum outdegree = E$$

Complete undirected:

$$\boxed{\frac{V(V-1)}{2}}$$

Tree:

$$\boxed{E=V-1}$$

Forest with C components:

$$\boxed{E=V-C}$$

Representations

Adjacency Matrix

Space:

$$O(V^2)$$

Edge check:

$$O(1)$$

Adjacency List

Space:

$$O(V+E)$$

Good for sparse graphs.

BFS

Uses:

$$ \boxed{\text{Queue}} $$

Explores:

$$ \boxed{\text{level by level}} $$

Adjacency-list time:

$$ \boxed{O(V+E)} $$

Space:

$$ O(V) $$

Key property:

$$ \boxed{\text{shortest unweighted path}} $$

DFS

Uses:

$$ \boxed{\text{Stack/recursion}} $$

Explores:

> **deep then backtrack.**

Time:

$$ \boxed{O(V+E)} $$

Space:

$$ O(V) $$

Useful for:

```text
reachability
components
cycle detection
topological sorting
```

Topological Sort

Exists iff graph is:

$$ \boxed{\text{DAG}} $$

Time:

$$ O(V+E) $$

---

92. MEMORY TRICKS

BFS

> **B = Broad = Breadth = Queue**

and:

> **BFS = Best Fewest Steps**

for unweighted shortest paths.

DFS

> **D = Dive Deep = Stack**

Matrix vs List

> **Matrix spends \(V^2\), List spends \(V+E\).**

Tree

> **V vertices !’ V" 1edges**

Topological Sort

> **Direction without cycle !’ ordering possible.**

&¡30-SECOND REVISION

1. Graph = \(G=(V,E)\).
2. Undirected degree sum = \(2E\).
3. Complete graph edges = \(V(V-1)/2\).
4. Tree = connected + acyclic.
5. Tree edges = \(V-1\).
6. Forest edges = \(V-C\).
7. Matrix space = \(O(V^2)\).
8. List space = \(O(V+E)\).
9. BFS uses queue.
10. DFS uses stack/recursion.
11. BFS explores levels.
12. DFS explores depth first.
13. BFS/DFS with list = \(O(V+E)\).
14. BFS finds shortest unweighted paths.

15. DFS does not guarantee shortest paths.
16. Connected components !' repeated BFS/DFS.
17. Undirected cycle DFS: visited non-parent neighbor.
18. Directed cycle DFS: edge to active/GRAY node.
19. Topological sorting exists only for DAG.
20. BFS/DFS order may not be unique without tie order.

' Chapter 13 Complete

Next Chapter:

Chapter 14 — Minimum Spanning Trees: Prim, Kruskal & Union-Find

We'll cover:

- spanning trees,
- number of edges in a spanning tree,
- minimum spanning tree,
- cut property,
- cycle property,
- uniqueness of MST,
- Prim's algorithm,
- Kruskal's algorithm,
- Disjoint Set / Union-Find,
- path compression + union by rank,
- MST numerical problems,
- effect of changing edge weights,
- and major GATE traps.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

CHAPTER 14 — MINIMUM SPANNING TREES: PRIM, KRUSKAL & UNION-FIND

GATE Importance: & & & & &

Your uploaded Algorithms study plan explicitly includes **Minimum Spanning Trees, Prim's algorithm, Kruskal's algorithm, Union-Find, cut/cycle properties, uniqueness, complexity, and edge-weight effects**.

The core idea is:

> **An MST connects all vertices with minimum total edge weight, without creating cycles.**

1. WHAT IS A SPANNING TREE?

Given a connected, undirected graph:

$$ G=(V,E) $$

a **spanning tree** is a subgraph that:

1. contains all vertices,
2. is connected,
3. has no cycles.

For:

$$ |V|=n $$

every spanning tree has exactly:

$$ \boxed{n-1} $$

edges.

2. EXAMPLE

Graph:

```text
A ----- B
| \     |
|  \    |
C ----- D
```

A spanning tree could be:

```text
A ----- B
|
|
C ----- D
```

All 4 vertices are connected.

Edges:

$$ 4-1=3 $$

No cycle.

So it is a spanning tree.

3. WHAT IS A MINIMUM SPANNING TREE?

If graph edges have weights, among all spanning trees choose one having minimum total weight.

That is a:

$$ \boxed{\text{Minimum Spanning Tree (MST)}} $$

4. EXAMPLE

Suppose:

```text
A --1-- B
|       |
4       2
|       |
C --3-- D
```

Possible spanning tree:

```text
AB = 1
BD = 2
CD = 3
```

Total:

$$ 1+2+3=6 $$

Another:

```text
AB = 1
AC = 4
CD = 3
```

Total:

$$ 8 $$

So the first is better.

5. MST EXISTS WHEN?

For a connected undirected weighted graph:

$$ \boxed{\text{An MST exists}} $$

If graph is disconnected:

> **there is no spanning tree covering all vertices.**

Instead, we get a:

$$ \boxed{\text{minimum spanning forest}} $$

6. MST HAS HOW MANY EDGES?

For V vertices:

$$ \boxed{V-1} $$

This is always true for any spanning tree.

Very important GATE fact.

---

7. MST VS SHORTEST PATHTREE

These are different.

MST

Goal:

> **minimize total weight of all selected tree edges.**

Shortest Path Tree

Goal:

> **minimize distance from one source to every vertex.**

They need not be the same tree.

---

8. SIMPLE DIFFERENCE

Suppose:

```text
A --10-- B
 \      /
  1    1
   \  /
    C
```

From source A:

Shortest path to B:

\[A\to C\to B \]

cost:

\[2 \]

An MST may also choose AC and CB here, but in general shortest-path optimization and total-tree optimization are different goals.

Do not confuse them.

---

9. TWO MAIN MST ALGORITHMS

The two most important are:

\[\boxed{\text{Prim's Algorithm}} \]

and:

$$ \boxed{\text{Kruskal's Algorithm}} $$

Both are:

$$ \boxed{\text{Greedy}} $$

---

10. PRIM'S ALGORITHM — CORE IDEA

Prim grows **one tree**.

Start from any vertex.

Repeatedly choose:

> **minimum-weight edge connecting a vertex already in the tree to a vertex outside the tree.**

Memory:

> **Prim = grow from a Point**

---

11. PRIM EXAMPLE

Graph:

```text
       2
A -------- B
|          |
3          1
|          |
C -------- D
     4
```

Also edge:

```text
A --5-- D
```

Start from A.

Current tree:

```text
{A}
```

Available edges:

```text
AB = 2
AC = 3
AD = 5
```

Choose:

$$ AB=2 $$

Tree:

```text
{A,B}
```

Now crossing edges:

```text
AC = 3
AD = 5
BD = 1
```

Choose:

$$ BD=1 $$

Tree:

```text
{A,B,D}
```

Remaining vertex C.

Crossing edges:

```text
AC = 3
CD = 4
```

Choose:

$$ AC=3 $$

MST weight:

$$ 2+1+3 $$

$$ \boxed{6} $$

---

12. IMPORTANT PRIM RULE

At every step, choose the smallest edge that crosses from:

$$ \boxed{\text{inside current tree}} $$

to:

$$ \boxed{\text{outside}} $$

Do not simply choose globally smallest unused edge.

That would be closer to Kruskal.

---

13. PRIM PSEUDOCODE IDEA

Using keys:

```text
key[source] = 0
all others = infinity

while some vertex remains outside MST:

    choose vertex u with minimum key

    include u in MST

    for every neighbor v of u:
        if v outside MST and weight(u,v) < key[v]:
            key[v] = weight(u,v)
            parent[v] = u
```

---

14. WHAT DOES `key[v]` MEAN?

\[key[v] \]

means:

> **cheapest edge currently known that can connect vertex \(v\) to the growing MST.**

Important:

It is **not** shortest-path distance from source.

This is a common Prim vs Dijkstra confusion.

---

15. PRIM WITH ADJACENCY MATRIX

If graph uses adjacency matrix:

Selecting minimum key can cost:

\[O(V) \]

and done \(V\) times.

Updating neighbors also scans \(V\).

Total:

\[\boxed{O(V^2)} \]

This is suitable for dense graphs.

---

16. PRIM WITH BINARY HEAP + ADJACENCY LIST

Using min-priority queue:

Each edge may cause heap updates.

Typical complexity:

$$\boxed{O(E\log V)}$$

More precisely often:

$$O((V+E)\log V)$$

which simplifies to:

$$O(E\log V)$$

for connected graphs because:

$$E\ge V-1$$

17. PRIM WITH FIBONACCI HEAP

Theoretical complexity can be:

$$\boxed{O(E+V\log V)}$$

This is useful conceptual knowledge but binary heap complexity is more commonly tested.

18. KRUSKAL'S ALGORITHM — CORE IDEA

Kruskal grows a **forest** initially.

It sorts edges globally by increasing weight.

Then repeatedly:

> **choose the smallest edge that does not create a cycle.**

Memory:

> **Kruskal = Keep smallest safe edge**

19. KRUSKAL EXAMPLE

Suppose edges:

Edge	Weight
AB	1
BC	2
AC	3
CD	4
BD	5

Sort:

```text
AB(1)
BC(2)
AC(3)
CD(4)
BD(5)
```

Choose AB:

```text
{A,B}
```

Choose BC:

```text
{A,B,C}
```

Now AC would create cycle:

```text
A-B-C-A
```

Skip AC.

Choose CD.

Now all 4 vertices connected.

Selected:

```text
AB, BC, CD
```

Weight:

\[1+2+4=7 \]

---

20. KRUSKAL PSEUDOCODE

```text
sort all edges by nondecreasing weight

for each vertex:
    make-set(vertex)

for each edge (u,v) in sorted order:

    if find(u) != find(v):

        select edge

        union(u,v)

    stop after V-1 selected edges
```

21. WHY DO WE NEED UNION-FIND?

Kruskal repeatedly asks:

> **Are \(u\) and \(v\) already connected by selected edges?**

If yes:

adding edge creates cycle.

If no:

safe to connect components.

Union-Find efficiently answers this.

22. DISJOINT SET / UNION-FIND

Union-Find maintains disjoint groups.

Main operations:

MAKE-SET(x)

Create a set containing only \(x\).

FIND(x)

Return representative of set containing \(x\).

UNION(x,y)

Merge sets containing \(x\) and \(y\).

23. KRUSKAL CYCLE TEST

Suppose selected edges have formed:

```text
A---B---C
```

Then:

```text
find(A) = find(C)
```

If candidate edge AC appears:

Both endpoints are already in same set.

Therefore adding AC creates cycle.

Skip it.

24. UNION-FIND FOREST REPRESENTATION

Each set can be represented by a rooted tree.

Example:

```text
      A
     / \
    B   C
       /
      D
```

A is representative/root.

Then:

```text
 f i n d ( D )  !’   A
```

---

25. PATHCOMPRESSION

Suppose:

```text
 D  !’   C  !’   B  !’   A
```

After:

```text
find(D)
```

path compression can change to:

```text
 D  !’   A
 C  !’   A
 B  !’   A
```

This makes later `find` operations much faster.

---

26. UNION BY RANK / SIZE

When merging two trees:

> **attach smaller/shallower tree under larger/deeper tree.**

This keeps tree height small.

Two common heuristics:

```text
union by rank
union by size
```

27. UNION-FIND COMPLEXITY

With:

- path compression
- union by rank/size

a sequence of operations takes nearly constant amortized time.

Per operation:

$$\boxed{O(\alpha(V))}$$

where:

$$\alpha$$

is inverse Ackermann function.

It grows extraordinarily slowly.

For all practical input sizes:

$$\alpha(V)$$

is tiny.

28. KRUSKAL COMPLEXITY

Main cost:

> **sorting edges.**

Sorting:

$$O(E\log E)$$

Union-Find work:

$$O(E\alpha(V))$$

So total:

$$\boxed{O(E\log E)}$$

Since for simple graphs:

$$E\le V^2$$

we have:

$$\log E=O(\log V)$$

Thus also commonly written:

$$\boxed{O(E\log V)}$$

29. PRIM VS KRUSKAL

Property	Prim	Kruskal
Growth style	One tree	Forest/components
Main choice	Cheapest crossing edge	Globally cheapest safe edge
Main DS	Priority queue	Union-Find
Cycle test explicitly?	Usually not needed	Yes
Good for	Often dense graphs	Often sparse graphs
Greedy?	Yes	Yes

30. PRIM VISUAL INTUITION

```text
Start:
A

Grow:
A---B

Grow:
A---B---C

Grow:
A---B---C
        |
        D
```

Always one connected component.

31. KRUSKAL VISUAL INTUITION

Initially:

```text
A   B   C   D   E
```

Then:

```text
A---B   C---D   E
```

Then:

```text
A---B---C---D   E
```

Eventually:

```text
A---B---C---D---E
```

Multiple components may exist until late.

---

32. CUT OF A GRAPH

A **cut** divides vertices into two disjoint sets:

\[S \]

and:

\[V-S \]

Example:

```text
S = {A,B}
V-S = {C,D}
```

An edge with one endpoint in each side is called a:

\[\boxed{\text{crossing edge}} \]

---

33. CUT PROPERTY

One of the most important MST properties.

For any cut that respects the currently selected MST edges:

> **a minimum-weight edge crossing the cut is safe to include in an MST.**

Informally:

\[\boxed{\text{Lightest crossing edge is safe}} \]

This is the foundation of Prim and Kruskal.

---

34. CUT PROPERTY INTUITION

Suppose an MST did not contain the cheapest crossing edge \(e\).

Adding \(e\) to the MST creates a cycle.

That cycle contains another edge crossing the same cut.

Replace that edge with \(e\).

Since e is no heavier:

> **total weight does not increase.**

Therefore some MST contains e.

That is the exchange-style proof idea.

---

35. CYCLE PROPERTY

Another key property:

> **If an edge is strictly heavier than every other edge in a cycle, that edge cannot belong to any MST.**

Memory:

$$ \boxed{\text{Heaviest unique cycle edge is unsafe}} $$

---

36. CYCLE PROPERTY EXAMPLE

Cycle:

```text
A --1-- B
|       |
4       2
|       |
C --3-- D
```

Suppose cycle edges are:

$$ 1,2,3,4 $$

The unique heaviest:

$$ 4 $$

cannot be in an MST.

Why?

Removing it keeps cycle vertices connected with lower total weight.

---

37. IMPORTANT WORD — UNIQUE HEAVIEST

If several edges tie for maximum weight in a cycle:

> **you cannot automatically say each one is excluded from every MST.**

The strict form is important.

---

38. MST UNIQUENESS

A very important sufficient condition:

If all edge weights are distinct:

$$\boxed{\text{MST is unique}}$$

Why?

Every cut has a unique lightest crossing edge structure sufficient to force decisions.

---

39. CONVERSE IS FALSE

If graph has repeated edge weights:

> **MST may still be unique.**

So:

$$\text{all weights distinct} \Rightarrow \text{unique MST}$$

but:

$$\text{unique MST} \not\Rightarrow \text{all weights distinct}$$

Very important GATE logic.

---

40. EXAMPLE WITH REPEATED WEIGHTS BUT UNIQUE MST

Suppose triangle:

```text
A --1-- B
 \      |
  1     5
   \    |
     C
```

Weights:

```text
AB=1
AC=1
BC=5
```

Repeated `1`.

Only cheapest spanning tree:

```text
AB + AC
```

weight:

$$2$$

Unique MST despite repeated weights.

41. MULTIPLE MSTs

Triangle:

```text
A --1-- B
 \      /
  1    1
   \  /
     C
```

Any two edges form spanning tree.

All have total weight:

\[2 \]

Number of MSTs:

\[3 \]

So equal weights can lead to multiple MSTs.

42. MST WEIGHT VS MST STRUCTURE

Even if multiple MSTs exist:

> **all MSTs have the same total weight.**

By definition, each is minimum.

Their edge sets can differ.

43. NEGATIVE EDGE WEIGHTS

Can MST algorithms handle negative edges?

\[\boxed{\text{Yes}} \]

Prim and Kruskal work perfectly with negative weights.

Why?

MST does not depend on path-distance assumptions like Dijkstra.

A negative edge is simply very attractive.

44. ZERO-WEIGHT EDGES

Also fine.

MST algorithms can use:

- negative,
- zero,
- positive

edge weights.

45. DIRECTED GRAPH MST?

Standard MST is defined for:

$$\boxed{\text{undirected graphs}}$$

Directed analogues involve different concepts such as minimum arborescences.

Do not apply standard Prim/Kruskal directly to directed graphs.

46. MST OF AN ALREADY TREE GRAPH

Suppose graph is already a tree.

It has:

$$V-1$$

edges and no cycles.

There is exactly one spanning tree:

> **the graph itself.**

Therefore:

$$\boxed{\text{That tree is automatically the MST}}$$

regardless of weights.

47. COMPLETE GRAPH DOES NOT MEAN UNIQUE MST

Even if graph is complete:

> **equal weights can produce many MSTs.**

Completeness alone says nothing about uniqueness.

48. EFFECT OF DECREASING AN MST EDGE WEIGHT

Suppose edge e belongs to an MST T.

If its weight decreases:

> **T remains an MST.**

Its total only becomes even smaller relative to alternatives containing/not containing e.

This is a useful sensitivity result.

49. EFFECT OF INCREASING AN MST EDGE WEIGHT

If an MST edge weight increases:

> **current tree may or may not remain minimum.**

You need to consider alternative edges that can reconnect the cut created by removing that edge.

So no unconditional guarantee.

50. EFFECT OF DECREASING A NON-MST EDGE

Suppose edge e is not in current MST.

Decrease its weight.

It may become beneficial.

Adding e to MST creates a cycle.

If e becomes lighter than a heavier edge on that cycle, replacing that heavier edge can yield a new MST.

51. EFFECT OF INCREASING A NON-MST EDGE

If a non-MST edge's weight increases:

> **the existing MST remains an MST.**

Because that edge was not used, making it more expensive cannot create a better alternative through that edge.

52. IMPORTANT EDGE-SENSITIVITY TABLE

Change	Existing MST guaranteed to remain MST?
Decrease weight of MST edge	Yes
Increase weight of MST edge	Not necessarily
Decrease weight of non-MST edge	Not necessarily
Increase weight of non-MST edge	Yes

Very useful.

53. KRUSKAL WITH EQUAL WEIGHTS

If equal-weight edges exist:

> **their processing order may change which MST is produced.**

But total weight remains minimum.

Different tie orders can generate different valid MSTs.

---

54. PRIM START VERTEX

If MST is unique:

> **Prim produces the same MST regardless of starting vertex.**

If multiple MSTs exist:

> **start vertex and tie-breaking can produce different MSTs.**

Still all have same total minimum weight.

---

55. PRIM DOES NOT SELECT LOWEST GLOBAL EDGE

Example:

Current tree contains:

```text
{A,B}
```

Suppose globally smallest unused edge is:

```text
C-D = 1
```

but neither C nor D is in current tree.

Prim cannot choose it now.

Prim only chooses edges crossing current cut.

Kruskal can choose it.

---

56. KRUSKAL DOES NOT NEED START VERTEX

Because it processes edges globally.

There is no starting vertex.

This is an easy identification clue.

---

57. MST AND CYCLES

A spanning tree cannot contain a cycle.

If selected edges ever form cycle:

> **remove at least one edge.**

Kruskal explicitly prevents cycles.

Prim implicitly avoids cycles by adding a new outside vertex each step.

---

58. WHY PRIM CANNOT FORM A CYCLE

Every selected Prim edge connects:

$$\text{inside vertex}$$

to:

$$\text{outside vertex}$$

So a new vertex is introduced each time.

Therefore cycle cannot appear.

---

59. MINIMUM NUMBER OF EDGES FOR CONNECTED GRAPH

For:

$$V$$

vertices:

$$\boxed{V-1}$$

Any connected graph must have at least this many edges.

A spanning tree uses exactly this minimum.

---

60. SPANNING TREE COUNT FOR COMPLETE GRAPH

A famous theorem:

For complete graph K_n, number of spanning trees is:

$$\boxed{n^{n-2}}$$

This is **Cayley's formula**.

Example:

$$K_4$$

has:

$$4^{4-2}=16$$

spanning trees.

This is useful supporting graph knowledge.

61. IF ALL EDGES HAVE SAME WEIGHT

Then every spanning tree has:

$$ (V-1)\times w $$

same total weight.

Therefore:

$$ \boxed{\text{Every spanning tree is an MST}} $$

For complete graph K_n, number of MSTs then is:

$$ n^{n-2} $$

62. PRIM TRACE QUESTION STRATEGY

When solving manually:

1. circle current tree vertices,
2. list only crossing edges,
3. choose minimum,
4. add new vertex,
5. repeat until $V-1$ edges.

Do not inspect irrelevant edges entirely inside/outside the tree.

63. KRUSKAL TRACE QUESTION STRATEGY

1. Sort all edges ascending.
2. Start with each vertex separate.
3. Scan edges.
4. If endpoints in different components !' take.
5. Otherwise !' skip.
6. Stop after $V-1$ chosen edges.

64. KRUSKAL EXAMPLE WITH UNION-FIND

Edges:

```text
AB=1
CD=2
BC=3
AD=4
AC=5
```

Initially:

```text
{A} {B} {C} {D}
```

Take AB:

```text
{A,B} {C} {D}
```

Take CD:

```text
{A,B} {C,D}
```

Take BC:

Different components.

Union:

```text
{A,B,C,D}
```

Selected edges:

\[3=V-1 \]

Stop.

Total:

\[1+2+3 \]

\[\boxed{6} \]

---

65. GATE-STYLE CUT QUESTION

Suppose cut:

\[S=\{A,B\} \]

\[V-S=\{C,D\} \]

Crossing edges:

```text
AC = 5
AD = 2
BC = 4
BD = 7
```

Unique lightest crossing edge:

\[AD=2 \]

By cut property:

\[\boxed{AD\text{ belongs to every MST}} \]

because it is uniquely lightest across that cut.

66. UNIQUE LIGHT EDGE VS SAFE EDGE

If multiple crossing edges tie for minimum:

> **each minimum edge is safe in the sense that some MST can contain it.**

But an individual tied edge need not be in **every** MST.

Important distinction.

67. GATE-STYLE CYCLE QUESTION

Cycle edge weights:

$$ 2,4,6,10 $$

Unique heaviest edge:

$$ 10 $$

Therefore:

$$ \boxed{\text{Edge weight 10 cannot appear in any MST}} $$

68. SECOND-BEST MST IDEA

Not a core requirement here, but useful intuition.

Given an MST, add one non-tree edge.

A cycle forms.

Removing the heaviest edge on that cycle gives another spanning tree.

This replacement idea is fundamental to:

- MST sensitivity,
- second-best spanning tree,
- correctness proofs.

69. PRIM VS DIJKSTRA — VERY IMPORTANT

These two can look similar because both use a priority queue.

But their keys mean different things.

Prim

$$ key[v]= $$

minimum edge weight connecting v to current MST.

Dijkstra

\[dist[v]= \]

shortest known total path distance from source.

Memory:

> **Prim minimizes one connecting EDGE.**

> **Dijkstra minimizes whole PATH distance.**

---

70. EXAMPLE PRIM VS DIJKSTRA

Suppose:

```text
S --10-- A
 \       |
  6      1
   \     |
      B
```

Prim from S might first choose:

\[SB=6 \]

then:

\[BA=1 \]

MST total:

\[7 \]

Dijkstra from S:

Distance to A:

Direct:

\[10 \]

via B:

\[6+1=7 \]

So shortest path also uses B here.

But this agreement is accidental; objectives differ.

---

71. GATE TRAPS

& þTrap 1 — MST minimizes source-to-all distances

False.

That's shortest path tree.

& þTrap 2 — Prim chooses globally smallest unused edge

False.

It chooses minimum **crossing** edge.

& þTrap 3 — Kruskal grows one connected tree from the beginning

False.

It can maintain several components.

& þTrap 4 — Prim requires Union-Find

Normally no.

Kruskal does.

& þTrap 5 — Negative weights break MST algorithms

False.

Negative weights are fine.

& þTrap 6 — Equal weights imply multiple MSTs

Not necessarily.

They make multiple MSTs possible, not guaranteed.

& þTrap 7 — Unique MST implies all edge weights distinct

False.

Distinct weights are sufficient, not necessary.

& þTrap 8 — MST on \(V\) vertices has \(V\) edges

Wrong.

$$\boxed{V-1}$$

& þTrap 9 — Standard Prim/Kruskal apply directly to directed graphs

No.

MST is standardly for undirected graphs.

& þTrap 10 — Increasing a non-MST edge can invalidate current MST

No.

Making an unused alternative more expensive cannot make current MST worse relative to it.

72. EXAM DECISION RULES

Need MST with global edge sorting?

\[\boxed{\text{Kruskal}} \]

Need MST by growing from a starting vertex?

\[\boxed{\text{Prim}} \]

Need cycle checking among selected edges?

\[\boxed{\text{Union-Find}} \]

Unique lightest crossing edge?

\[\boxed{\text{Must belong to every MST}} \]

Unique heaviest edge in cycle?

\[\boxed{\text{Cannot belong to any MST}} \]

All edge weights distinct?

\[\boxed{\text{MST unique}} \]

Graph is already a tree?

\[\boxed{\text{That graph itself is MST}} \]

73. PYQ-STYLE QUESTION 1

A connected weighted graph has:

\[12 \]

vertices.

How many edges does every MST contain?

\[12-1 \]

\[\boxed{11} \]

74. PYQ-STYLE QUESTION 2

Which algorithm uses Union-Find?

A. Prim B. Kruskal C. BFS D. Dijkstra

Answer:

$$\boxed{B}$$

75. PYQ-STYLE QUESTION 3

Which MST algorithm requires sorting edges globally?

$$\boxed{\text{Kruskal}}$$

76. PYQ-STYLE QUESTION 4

Prim with adjacency matrix:

$$\boxed{O(V^2)}$$

Prim with binary heap + adjacency list:

$$\boxed{O(E\log V)}$$

77. PYQ-STYLE QUESTION 5

Kruskal:

Sorting:

$$O(E\log E)$$

Union-Find:

near linear.

Total:

$$\boxed{O(E\log E)}$$

78. PYQ-STYLE QUESTION 6

If all edge weights are distinct, number of MSTs?

$$\boxed{1}$$

79. PYQ-STYLE QUESTION 7

If all edges in a connected graph have equal weight:

Every spanning tree has same total.

Therefore:

$$\boxed{\text{Every spanning tree is an MST}}$$

80. PYQ-STYLE QUESTION 8

A cycle has edge weights:

$$ 1,3,4,9,2 $$

Can the edge of weight 9 appear in an MST?

It is unique heaviest on that cycle.

$$ \boxed{\text{No}} $$

81. PYQ-STYLE QUESTION 9

Suppose current MST edge weight decreases.

Can current tree cease to be an MST solely because of that decrease?

$$ \boxed{\text{No}} $$

It remains an MST.

82. PYQ-STYLE QUESTION 10

A non-MST edge's weight increases.

Does current MST remain valid?

$$ \boxed{\text{Yes}} $$

83. QUICK RECALL

Try mentally.

1. Spanning tree definition?
2. Edges in spanning tree with V vertices?
3. MST applies to directed or undirected graphs?
4. Prim's greedy choice?
5. Kruskal's greedy choice?
6. Which uses Union-Find?
7. Prim matrix complexity?
8. Prim binary-heap complexity?
9. Kruskal complexity?
10. Cut property?
11. Cycle property?
12. Distinct edge weights imply?
13. Repeated weights imply multiple MSTs always?
14. Can MST contain negative edges?
15. Prim vs Dijkstra key meaning?

ANSWERS

1. Connected acyclic subgraph containing all vertices.
2. $V-1$.
3. Standard MST: undirected.
4. Cheapest crossing edge.
5. Globally cheapest edge that doesn't create cycle.
6. Kruskal.
7. $O(V^2)$.
8. $O(E\log V)$.
9. $O(E\log E)$.
10. Lightest crossing edge is safe.
11. Unique heaviest cycle edge cannot belong to MST.
12. Unique MST.
13. No.
14. Yes.
15. Prim uses connecting-edge weight; Dijkstra uses source-path distance.

84. PRACTICE SET

Easy

Q1 — NAT

A spanning tree contains 20 vertices.

How many edges?

Q2 — MCQ

Which is a Greedy MST algorithm?

A. Prim B. Kruskal C. Both D. Neither

Q3 — MCQ

Kruskal primarily uses:

A. Queue B. Union-Find C. DP table D. Stack

Medium

Q4 — MSQ

Which statements are true?

A. Every MST has $V-1$ edges.

B. Prim can work with negative edge weights.

C. Kruskal can work with negative edge weights.

D. Standard MST requires a directed graph.

Q5 — MCQ

All edge weights in a connected graph are distinct.

Then:

A. MST may not exist B. MST is unique C. every spanning tree is MST D. Prim fails

---

Q6 — MCQ

The unique heaviest edge in a cycle:

A. must be in every MST B. cannot be in any MST C. must be chosen by Prim D. must be chosen by Kruskal

---

Q7 — NAT

For complete graph K_5, how many spanning trees exist?

Use Cayley's formula.

Hard / Tricky

Q8 — MSQ

Which changes guarantee an existing MST remains an MST?

A. Decrease an MST edge weight.

B. Increase an MST edge weight.

C. Increase a non-MST edge weight.

D. Decrease a non-MST edge weight.

Q9 — MCQ

Suppose a cut has crossing-edge weights:

$$ 2,2,5,7 $$

Which statement is safest?

A. Both weight-2 edges must belong to every MST.

B. At least one minimum crossing edge can be safely chosen for some MST.

C. Weight-7 edge must be in every MST.

D. No MST exists.

SOLUTIONS

Q1

$$ V-1 = 20-1 $$

$$ \boxed{19} $$

Q2

Both are Greedy.

$$ \boxed{C} $$

Q3

$$ \boxed{B} $$

Q4

A true.

B true.

C true.

D false.

Answer:

$$ \boxed{A,B,C} $$

Q5

Distinct weights imply:

$$ \boxed{\text{unique MST}} $$

Answer:

$$ \boxed{B} $$

Q6

Unique heaviest edge on a cycle:

$$ \boxed{\text{cannot belong to any MST}} $$

Answer:

$$ \boxed{B} $$

Q7

Cayley:

$$ n^{n-2} $$

For:

$$ n=5 $$

$$ 5^3 = \boxed{125} $$

Q8

Decrease MST edge:

Current MST remains optimal.

Increase non-MST edge:

Current MST remains optimal.

The others can potentially change optimal tree.

Answer:

$$ \boxed{A,C} $$

Q9

Because two crossing edges tie for minimum:

an individual one need not occur in every MST.

But a minimum crossing edge is safe.

Answer:

$$ \boxed{B} $$

---

85. Ø=ÜÓTES TO WRITE

Spanning Tree

Contains all vertices.

Connected.

Acyclic.

Edges:

$$ \boxed{V-1} $$

Minimum Spanning Tree

Spanning tree with minimum:

$$ \boxed{\text{total edge weight}} $$

Standard setting:

```text
connected
undirected
weighted graph
```

Prim

Grow one tree.

Choose:

$$\boxed{\text{minimum crossing edge}}$$

Complexity:

Matrix:

$$\boxed{O(V^2)}$$

Binary heap + list:

$$\boxed{O(E\log V)}$$

Kruskal

Sort edges ascending.

Take:

$$\boxed{\text{smallest edge that does not create cycle}}$$

Uses:

$$\boxed{\text{Union-Find}}$$

Complexity:

$$\boxed{O(E\log E)}$$

Union-Find

Operations:

```text
MAKE-SET
FIND
UNION
```

Optimizations:

```text
Path Compression
Union by Rank/Size
```

Amortized:

$$ O(\alpha(V)) $$

Cut Property

$$ \boxed{\text{Lightest crossing edge is safe}} $$

Unique lightest across cut:

> **belongs to every MST.**

Cycle Property

$$ \boxed{\text{Unique heaviest cycle edge is in no MST}} $$

Uniqueness

All edge weights distinct:

$$ \boxed{\text{unique MST}} $$

Converse false.

Edge Weight Changes

```TEXT
D e c r e a s e   M S T   e d g e            !'  M S T   r e m a i n s
I n c r e a s e   M S T   e d g e            !'  m a y b e   c h a n g e s
D e c r e a s e   n o n - M S T   e d g e    !'  m a y b e   c h a n g e s
I n c r e a s e   n o n - M S T   e d g e    !'  M S T   r e m a i n s
```

86. MEMORY TRICKS

Prim

> **Prim = Plant one tree and grow it.**

Kruskal

> **Kruskal = Keep smallest safe edge.**

Cut Property

> **Across a cut, light is right.**

Cycle Property

> **Heavy in cycle? Throw it away.**

when uniquely heaviest.

Union-Find

> **Find = same component? Union = merge components.**

Prim vs Kruskal

> **Prim grows. Kruskal joins.**

&β30-SECOND REVISION

1. Spanning tree = connected + acyclic + all vertices.
2. Every spanning tree has \(V-1\) edges.
3. MST minimizes total tree weight.
4. Prim and Kruskal are Greedy.
5. Prim grows one connected tree.
6. Prim chooses lightest crossing edge.
7. Prim matrix = \(O(V^2)\).
8. Prim heap = \(O(E\log V)\).
9. Kruskal sorts all edges.
10. Kruskal takes smallest cycle-safe edge.
11. Kruskal uses Union-Find.
12. Kruskal = \(O(E\log E)\).
13. Path compression + union by rank gives near-constant DSU operations.
14. Cut property !’ minimum crossing edge is safe.
15. Unique minimum crossing edge belongs to every MST.
16. Unique heaviest cycle edge belongs to no MST.
17. Distinct weights imply unique MST.
18. Equal weights do not necessarily mean multiple MSTs.
19. Negative edge weights are allowed in MST.
20. Prim key "`Dijkstra distance.
21. Increase non-MST edge !’ current MST stays.
22. Decrease MST edge !’ current MST stays.

' Chapter 14 Complete

Next Chapter:

Chapter 15 — Shortest Paths: BFS, Dijkstra, Bellman-Ford, DAG Shortest Path & Floyd-Warshall

This is the final major Algorithms chapter. We'll cover:

- relaxation,
- single-source vs all-pairs shortest paths,
- BFS for unweighted graphs,
- Dijkstra,
- why Dijkstra fails with negative edges,
- Bellman-Ford,
- negative-cycle detection,
- shortest paths in DAGs,
- Floyd-Warshall,
- algorithm-selection tables,
- complexity,
- and GATE numerical relaxation questions.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

CHAPTER 15 — SHORTEST PATHS: BFS, DIJKSTRA, BELLMAN-FORD, DAG SHORTEST PATH & FLOYD-WARSHALL

GATE Importance: & & & & &

This is the final major Algorithms chapter in your current syllabus sequence.

The big goal is:

> **Given a graph, find the minimum-cost path between vertices.**

The most important skill for GATE is not just knowing the algorithms — it is knowing:

> **Which shortest-path algorithm is valid for which graph.**

---

1. WHAT IS A SHORTEST PATH?

For a path:

\[v_0\to v_1\to v_2\to\cdots\to v_k \]

its cost is the sum of its edge weights:

\[\boxed{ w(v_0,v_1)+w(v_1,v_2)+\cdots+w(v_{k-1},v_k) } \]

A shortest path between two vertices is a path with minimum total weight.

---

2. UNWEIGHTED GRAPH

If every edge effectively has the same cost, shortest path means:

\[\boxed{\text{minimum number of edges}} \]

Use:

\[\boxed{\text{BFS}} \]

---

3. WEIGHTED GRAPH

If edges have different weights:

```text id="sp1" A --5-- B \  / 1  2 \ / C

```
 D i r e c t   A!’ B :

\[
5
\]

Via C:

\[
1+2=3
\]

Shortest:

\[
\boxed{A\to C\to B}
\]

---

4. SINGLE-SOURCE SHORTEST PATH

Given source:

\[
s
\]

find shortest distances from \(s\) to every vertex.

Abbreviation:

\[
\boxed{\text{SSSP}}
\]

Algorithms:
```
```

```
- BFS
- Dijkstra
- Bellman-Ford
- DAG shortest path

depending on graph type.

---

# 5. ALL-PAIRS SHORTEST PATH

Find shortest path between every pair:

\[
(i,j)
\]

Algorithm:

\[
\boxed{\text{Floyd-Warshall}}
\]

is the classic GATE answer.

---

# 6. MOST IMPORTANT CONCEPT — RELAXATION

Relaxation is the heart of shortest-path algorithms.

Suppose edge:

\[
u\to v
\]

with weight:

\[
w(u,v)
\]

Current known distances:

\[
dist[u]
\]

and:

\[
dist[v]
\]

If going through \(u\) improves \(v\):

\[
dist[u]+w(u,v)<dist[v]
\]

then update:

\[
\boxed{
```

```
dist[v]=dist[u]+w(u,v)
}
\]

and often:

\[
parent[v]=u
\]

---

# 7. RELAXATION EXAMPLE

Suppose:

\[
dist[A]=4
\]

\[
dist[B]=10
\]

Edge:

\[
A\to B
\]

weight:

\[
3
\]

Candidate:

\[
4+3=7
\]

Since:

\[
7<10
\]

update:

\[
\boxed{dist[B]=7}
\]

---

# 8. RELAXATION DOES NOT ALWAYS CHANGE DISTANCE

Suppose:

\[
dist[A]=4
\]

\[
dist[B]=6
```

```
\]

edge weight:

\[
3
\]

Candidate:

\[
7
\]

Since:

\[
7>6
\]

do nothing.

---

# 9. INITIALIZATION

For source \(s\):

\[
dist[s]=0
\]

For all others:

\[
dist[v]=\infty
\]

This is standard initialization.

---

# 10. BFS FOR UNWEIGHTED SHORTEST PATH

From Chapter 13:

BFS explores:

```text id="sp2"
distance 0
distance 1
distance 2
distance 3
...
```
```

Therefore the first time a vertex is discovered gives the minimum number of edges from source.

---

11. BFS DISTANCE RULE

When discovering \(v\) from \(u\):

\[\boxed{ dist[v]=dist[u]+1 } \]

Time with adjacency list:

\[\boxed{O(V+E)} \]

---

12. BFS WHEN EDGE WEIGHTS ARE ALL EQUAL

If every edge has weight:

\[c>0 \]

then minimum-weight path is also minimum-edge-count path.

So BFS is still sufficient.

---

13. DIJKSTRA'S ALGORITHM

Dijkstra solves single-source shortest paths when edge weights are:

\[\boxed{\text{non-negative}} \]

That is:

\[w(e)\ge0 \]

for every edge.

---

14. DIJKSTRA CORE IDEA

Maintain tentative distances.

Repeatedly:

1. choose unprocessed vertex with smallest tentative distance,
2. finalize it,
3. relax its outgoing edges.

This is Greedy.

Memory:

> **Dijkstra = choose closest unfinished vertex**

---

15. DIJKSTRA EXAMPLE

Graph:

```text id="sp3" 4 A -------- B

1 1
```

C -------- D 2

```
Also:

```text id="sp4"
B --5-- D
```
```

Source:

\[A \]

Initialize:

```text id="sp5" A = 0 B = "  C = "  D = "

```
---

16. FIRST STEP

Choose A.

Relax:

\[
A\to B=4
\]

so:

\[
dist[B]=4
\]

\[
A\to C=1
\]

so:

\[
dist[C]=1
\]

Now:

```text id="sp6"
A=0
B=4
C=1
 D ="
```
```

---

## 17. SECOND STEP

Choose smallest unfinished:

\[ C=1 \]

Relax:

\[ C\to D=2 \]
```

Candidate:

\[1+2=3 \]

Thus:

\[dist[D]=3 \]

Now:

```text id="sp7" A=0 C=1 D=3 B=4

```
---

18. THIRD STEP

Choose:

\[
D=3
\]

If edge:

\[
D\to B=1
\]

candidate:

\[
3+1=4
\]

same as current B.

No improvement.

Final distances:

\[
\boxed{
A=0,\ C=1,\ D=3,\ B=4
}
\]

---

19. DIJKSTRA GREEDY PROPERTY

Once vertex \(u\) is selected as minimum tentative distance:

> its distance is finalized.

Why?

With non-negative edges, any future path reaching \(u\) through some unprocessed vertex cannot become cheaper.

This logic fails with negative edges.

---

20. WHY DIJKSTRA FAILS WITH NEGATIVE EDGES

```
Consider:

```text id="sp8"
S --2--> A
 \       ^
  5      |
   \     -4
    v    |
     B ---
```
```

Edges:

\[S\to A=2 \]

\[S\to B=5 \]

\[B\to A=-4 \]

Dijkstra may finalize A with:

\[2 \]

before processing B.

But actual shortest path:

\[S\to B\to A \]

cost:

\[5-4=1 \]

So finalized value was wrong.

Therefore:

\[\boxed{\text{Dijkstra cannot handle negative edge weights in general}} \]

---

21. IMPORTANT WORDING

Dijkstra does **not** require strictly positive edges.

Zero-weight edges are fine.

Requirement:

\[\boxed{w(e)\ge0} \]

---

22. DIJKSTRA COMPLEXITY — SIMPLE ARRAY/MATRIX

Selecting minimum-distance vertex:

\[O(V) \]

done \(V\) times.

Total:

\[\boxed{O(V^2)} \]

This works well for dense graphs.

---

23. DIJKSTRA WITH BINARY HEAP

Using:

- adjacency list
- min-priority queue

Typical complexity:

$$\boxed{O((V+E)\log V)}$$

For connected/sufficiently edge-containing graph, commonly:

$$\boxed{O(E\log V)}$$

---

24. DIJKSTRA WITH FIBONACCI HEAP

Theoretical:

$$\boxed{O(E+V\log V)}$$

Useful for theory; binary heap version is more commonly relevant.

---

25. DIJKSTRA VS PRIM

They look extremely similar.

Both:

- use a priority queue,
- repeatedly choose minimum key vertex.

But meanings differ.

Prim

$$key[v]=\text{cheapest edge connecting v to MST}$$

Dijkstra

$$dist[v]=\text{cheapest total path from source}$$

Memory:

> **Prim minimizes connecting EDGE.**

> **Dijkstra minimizes full PATH.**

---

26. BELLMAN-FORD

Bellman-Ford solves SSSP even when graph contains:

$$\boxed{\text{negative edge weights}}$$

provided no reachable negative-weight cycle makes the shortest distance undefined.

It can also:

\[\boxed{\text{detect negative cycles}} \]

---

27. BELLMAN-FORD CORE IDEA

Relax **every edge repeatedly**.

Why?

A shortest simple path can contain at most:

\[V-1 \]

edges.

Therefore relaxing all edges:

\[V-1 \]

times is sufficient.

---

28. BELLMAN-FORD PSEUDOCODE

```text id="sp9" dist[source] = 0 all others = infinity

repeat V-1 times:

for every edge (u,v,w):

if dist[u] + w < dist[v]:

dist[v] = dist[u] + w

```

```
Then perform one more pass.

If some distance still improves:

\[
\boxed{\text{reachable negative cycle exists}}
\]

---

# 29. WHY \(V-1\) PASSES?

Any shortest simple path can visit at most:

\[
V
\]

vertices.

Thus it has at most:

\[
V-1
\]

edges.
```

```
After pass 1:

> shortest paths using at most 1 edge can propagate.

After pass 2:

> up to 2 edges.

...

After:

\[
V-1
\]

passes:

> all simple shortest paths have had enough opportunities.

---

# 30. BELLMAN-FORD EXAMPLE

Edges:

```text id="sp10"
 S  !’   A   :   4
 S  !’   B   :   5
 A  !’   B   :   - 2
```
```

Initialize:

```text id="sp11" S=0  A =" B ="

```
 R e l a x   S!’ A :

\[
A=4
\]

 R e l a x   S!’ B :

\[
B=5
\]

 R e l a x   A!’ B :

\[
4+(-2)=2
\]

So:

\[
B=2
\]

Final:

\[
```
```

```
\boxed{S=0,\ A=4,\ B=2}
\]

Dijkstra's restrictions are avoided.

---

# 31. NEGATIVE CYCLE

Example:

```text id="sp12"
 A  !’   B   :   1
 B  !’   C   :   - 3
 C  !’   A   :   1
```
```

Cycle weight:

\[1-3+1=-1 \]

Negative.

You can traverse cycle repeatedly:

\[-1,-2,-3,\ldots \]

Thus path cost can decrease without bound.

There is no finite shortest-path value for vertices reachable through that cycle.

---

32. BELLMAN-FORD NEGATIVE CYCLE TEST

After the normal:

\[V-1 \]

passes, perform one additional relaxation pass.

If:

\[dist[u]+w(u,v)<dist[v] \]

still holds for any reachable edge:

\[\boxed{\text{negative cycle reachable from source}} \]

---

33. BELLMAN-FORD COMPLEXITY

There are:

\[V-1 \]

passes.

Each scans:

\[E \]

edges.

Therefore:

$$ \boxed{O(VE)} $$

Space:

$$ \boxed{O(V)} $$

excluding graph storage.

34. DIJKSTRA VS BELLMAN-FORD

Property	Dijkstra	Bellman-Ford
Negative edges	No	Yes
Negative cycle detection	No	Yes
Strategy	Greedy	Repeated relaxation / DP-like
Typical time	$O(E\log V)$	$O(VE)$
Faster normally?	Yes	No
SSSP	Yes	Yes

35. IMPORTANT DECISION RULE

Non-negative weighted graph

Use:

$$ \boxed{\text{Dijkstra}} $$

Negative edges may exist

Use:

$$ \boxed{\text{Bellman-Ford}} $$

Need negative-cycle detection

Use:

$$ \boxed{\text{Bellman-Ford}} $$

36. SHORTEST PATHIN A DAG

A DAG has no directed cycles.

This allows an efficient shortest-path algorithm even with:

$$ \boxed{\text{negative edges}} $$

because there are no cycles, so no negative cycles.

37. DAG SHORTEST PATH ALGORITHM

1. Topologically sort vertices.
2. Initialize distances.
3. Process vertices in topological order.
4. Relax outgoing edges once.

Time:

\[\boxed{O(V+E)} \]

---

38. WHY TOPOLOGICAL ORDER WORKS

If:

\[u\to v \]

then \(u\) occurs before \(v\).

So by the time we process \(v\), all possible predecessors that could improve its distance have already been processed.

No repeated passes are needed.

---

39. DAG SHORTEST PATH EXAMPLE

Suppose:

```text id="sp13" A !’ B : 3 A !’ C : 2 B !’ D : -4 C !’ D : 5

```
Topological:

```text id="sp14"
A, B, C, D
```
```

Source A:

\[ A=0 \]

Relax A:

\[ B=3 \]

\[ C=2 \]

Process B:

\[ D=3-4=-1 \]

Process C:

Candidate:

\[ 2+5=7 \]

No improvement.

Final:
```

$$ \boxed{D=-1} $$

Negative edge worked perfectly because graph is acyclic.

40. DAG SHORTEST PATH COMPLEXITY

Topological sort:

$$ O(V+E) $$

Relax all edges:

$$ O(E) $$

Total:

$$ \boxed{O(V+E)} $$

Excellent result.

41. FLOYD-WARSHALL

Floyd-Warshall computes:

$$ \boxed{\text{all-pairs shortest paths}} $$

It is a Dynamic Programming algorithm.

It can handle:

- positive weights,
- zero weights,
- negative edges,

but not meaningful finite shortest paths when negative cycles are involved.

42. FLOYD-WARSHALL DP IDEA

Let:

$$ D^{(k)}[i][j] $$

be shortest distance from i to j using only vertices:

$$ 1,2,\ldots,k $$

as possible intermediate vertices.

At step k:

Either shortest path:

1. does not use k, or
2. uses k.

Therefore:

$$ \boxed{ D^{(k)}[i][j] = \min \left(D^{(k-1)}[i][j], D^{(k-1)}[i][k]+D^{(k-1)}[k][j] \right) } $$

43. PRACTICAL FLOYD-WARSHALL UPDATE

Common in-place form:

\[\boxed{ D[i][j] = \min(D[i][j],D[i][k]+D[k][j]) } \]

Triple loops:

```text id="sp15" for k: for i: for j: D[i][j] = min(D[i][j], D[i][k] + D[k][j])

```
---

44. LOOP ORDER MATTERS

The canonical Floyd-Warshall loop order is:

```text id="sp16"
k outermost
i middle
j inner
```

Why?

Because \(k\) represents:

> **which intermediate vertices are currently allowed.**

Putting \(k\) outermost preserves the DP interpretation.

This is important.

45. FLOYD-WARSHALL INITIAL MATRIX

For:

\[i=j \]

\[D[i][i]=0 \]

If edge \(i\to j\) exists:

\[D[i][j]=w(i,j) \]

If no edge:

\[D[i][j]=\infty \]

46. FLOYD-WARSHALL EXAMPLE

Initial:

\[D= \begin{bmatrix} 0 & 3 & \infty\\ \infty & 0 & 1\\ 2 & \infty & 0 \end{bmatrix} \]

Consider path:

\[1\to2\to3 \]

Cost:

$$ 3+1=4 $$

So:

$$ D[1][3] $$

can improve from:

$$ \infty $$

to:

$$ 4 $$

---

47. FLOYD-WARSHALL COMPLEXITY

Three nested loops over V:

$$ \boxed{O(V^3)} $$

Space:

$$ \boxed{O(V^2)} $$

---

48. FLOYD-WARSHALL NEGATIVE CYCLE DETECTION

Initially:

$$ D[i][i]=0 $$

After algorithm, if:

$$ \boxed{D[i][i]<0} $$

for some vertex i, then a negative cycle exists involving/reachable within that path structure.

This is a useful GATE fact.

---

49. FLOYD-WARSHALL VS RUNNING DIJKSTRA MANY TIMES

For all-pairs shortest paths:

Floyd-Warshall

$$ O(V^3) $$

Dijkstra from every vertex

Using binary heap:

$$ O(VE\log V) $$

Which is preferable depends on graph density and constraints.

For dense graphs:

$$ E=\Theta(V^2) $$

repeated Dijkstra:

$$ O(V^3\log V) $$

while Floyd-Warshall:

$$ O(V^3) $$

may be simpler/better asymptotically.

50. SHORTEST-PATHALGORITHM MASTER TABLE

Graph Type	Algorithm	Time
Unweighted	BFS	$O(V+E)$
Weighted, non-negative	Dijkstra	$O(E\log V)$ typical
Negative edges allowed	Bellman-Ford	$O(VE)$
DAG, even negative edges	Topological relaxation	$O(V+E)$
All-pairs	Floyd-Warshall	$O(V^3)$

This table is a must-memorize GATE table.

51. NEGATIVE EDGE VS NEGATIVE CYCLE

Do not confuse them.

Negative edge

Example:

$$ u\to v=-5 $$

Bellman-Ford can handle it.

DAG shortest path can handle it.

Floyd-Warshall can handle it if no problematic negative cycle.

Negative cycle

Total cycle weight:

$$ <0 $$

Then shortest distance may become:

$$ -\infty $$

because cycle can be repeated.

52. DIJKSTRA AND NEGATIVE CYCLES

If a graph has a negative cycle, it certainly has a negative edge.

So Dijkstra is already invalid before discussing negative-cycle behavior.

53. BELLMAN-FORD EARLY STOP OPTIMIZATION

If during one complete pass:

> **no distance changes,**

then algorithm can stop early.

Why?

No future pass can improve anything.

Worst-case still:

\[O(VE) \]

but practical performance may improve.

---

54. SHORTEST PATH TREE

For SSSP, parent pointers can define a shortest-path tree rooted at source, if reachable shortest paths are well-defined.

For each vertex:

```text id="sp17" parent[v]

```
stores predecessor used in shortest path.

---

55. RECONSTRUCTING A PATH

Suppose:

```text id="sp18"
parent[D] = C
parent[C] = A
parent[A] = S
```

To reconstruct S!' D :

follow backwards:

```text id="sp19" D !' C !' A !' S

```
reverse:

```text id="sp20"
 S  !'   A  !'   C  !'   D
```

---

56. MULTIPLE SHORTEST PATHS

There may be multiple shortest paths with equal cost.

Example:

```text id="sp21"  A!’ B!’ Ð 5  A!’ C!’ Ð 5

Both are shortest.

A shortest-path tree may select one depending on tie-breaking.

---

# 57. SHORTEST PATH NEED NOT BE UNIQUE

Distinct edge weights do not automatically guarantee unique shortest paths.

Different combinations can still produce equal total cost.

Important distinction from MST uniqueness results.

---

# 58. SIMPLE PATH AND NEGATIVE CYCLES

If there are no negative cycles affecting the route:

> a shortest path can be taken to be simple.

Why?

If a path repeats a non-negative cycle, removing that cycle does not increase cost.

---

# 59. TRIANGLE INEQUALITY FOR SHORTEST DISTANCES

Shortest-path distances satisfy:

\[
\boxed{
\delta(u,v)
\le
\delta(u,x)+\delta(x,v)
}
\]

whenever distances are well-defined.

Going through \(x\) gives one candidate route, so true shortest path cannot be more expensive than that candidate.

---

# 60. RELAXATION PROPERTY

During shortest-path algorithms:

\[
dist[v]
\]

is often an upper bound on the true shortest distance:

\[
dist[v]\ge\delta(s,v)
\]

Relaxation progressively lowers this bound.
```

```
At termination, correct algorithms achieve equality.

---

# 61. DIJKSTRA FINALIZATION PROPERTY

For non-negative edges:

When Dijkstra extracts minimum-distance vertex \(u\):

\[
\boxed{
dist[u]=\delta(s,u)
}
\]

That is the core correctness fact.

---

# 62. BELLMAN-FORD PASS PROPERTY

After \(k\) full relaxation passes:

> distances are correct for shortest paths that use at most \(k\) edges.

This is the key induction idea.

---

# 63. FLOYD-WARSHALL DP PROPERTY

After outer-loop iteration \(k\):

> \(D[i][j]\) is shortest distance using intermediate vertices only from:

\[
\{1,\dots,k\}
\]

This is the key invariant.

---

# 64. BFS AS DIJKSTRA SPECIAL CASE

If all edge weights are identical:

Dijkstra's priority ordering essentially corresponds to BFS levels.

So BFS can be viewed as a more efficient special case for equal-weight edges.

---

# 65. 0–1 EDGE WEIGHTS — SUPPORTING NOTE

If weights are only:

\[
0\text{ or }1
\]

there is a specialized algorithm called **0-1 BFS** using a deque, running in:

\[
O(V+E)
```

```
\]

This is useful knowledge, though not central to the official basic syllabus.

---

# 66. GATE TRAP — DIJKSTRA ON UNDIRECTED NEGATIVE EDGE

Suppose undirected edge:

\[
u-v
\]

has weight:

\[
-2
\]

In an undirected graph, traversing:

\[
u\to v\to u
\]

costs:

\[
-4
\]

This itself forms a negative cycle of length 2 conceptually.

So shortest paths become problematic.

Negative-weight undirected edges are especially dangerous.

---

# 67. GATE TRAP — BFS ON WEIGHTED GRAPH

If weights are:

```text id="sp22"
1,1,1,...
```
```

BFS works.

If arbitrary:

```text id="sp23" 1,7,2,100,...

```
BFS does not minimize weight.

---

68. GATE TRAP — BELLMAN-FORD NEEDS \(V\) PASSES

Normal shortest-distance phase needs:

\[
\boxed{V-1}
\]
```
```

```
passes.

The extra \(V\)-th style pass is only to detect negative cycles.

---

# 69. GATE TRAP — FLOYD-WARSHALL IS GREEDY

False.

It is:

\[
\boxed{\text{Dynamic Programming}}
\]

---

# 70. GATE TRAP — DIJKSTRA IS DP

Its standard classification is:

\[
\boxed{\text{Greedy}}
\]

---

# 71. GATE TRAP — BELLMAN-FORD FAILS WITH NEGATIVE EDGES

False.

That is exactly where it is useful.

It fails to produce finite shortest paths only when reachable negative cycles make them undefined.

---

# 72. GATE TRAP — DAG SHORTEST PATH REQUIRES NON-NEGATIVE EDGES

False.

DAG shortest path allows negative edges because there are no cycles.

---

# 73. GATE TRAP — FLOYD-WARSHALL REQUIRES NON-NEGATIVE EDGES

False.

Negative edges are allowed.

Negative cycles are the real issue.

---

# 74. GATE TRAP — MST AND SHORTEST PATH SAME

False.

MST:

\[
\text{minimize total tree weight}
\]
```

Shortest path:

$$
\text{minimize source-to-destination path weight}
$$

Different optimization objectives.

---

75. EXAM DECISION TREE

When shortest-path question appears, ask:

Q1. Unweighted?

Use:

$$
\boxed{\text{BFS}}
$$

Q2. Weighted and all weights non-negative?

Use:

$$
\boxed{\text{Dijkstra}}
$$

Q3. Negative edges?

Use:

$$
\boxed{\text{Bellman-Ford}}
$$

unless DAG.

Q4. DAG?

Use:

$$
\boxed{\text{Topological shortest path}}
$$

even with negative weights.

Q5. Need all-pairs?

Use:

$$
\boxed{\text{Floyd-Warshall}}
$$

---

76. PYQ-STYLE QUESTION 1

Which algorithm finds shortest paths in an unweighted graph?

A. DFS

```
B. BFS
C. Prim
D. Kruskal

Answer:

\[
\boxed{B}
\]

---

# 77. PYQ-STYLE QUESTION 2

Which algorithm requires non-negative edge weights?

A. Bellman-Ford
B. Floyd-Warshall
C. Dijkstra
D. DAG shortest path

Answer:

\[
\boxed{C}
\]

---

# 78. PYQ-STYLE QUESTION 3

Which algorithm detects reachable negative cycles?

\[
\boxed{\text{Bellman-Ford}}
\]

---

# 79. PYQ-STYLE QUESTION 4

How many main relaxation rounds in Bellman-Ford?

\[
\boxed{V-1}
\]

Then one extra check for negative cycle.

---

# 80. PYQ-STYLE QUESTION 5

Bellman-Ford complexity:

\[
\boxed{O(VE)}
\]

---

# 81. PYQ-STYLE QUESTION 6

Floyd-Warshall complexity:

\[
```

```
\boxed{O(V^3)}
\]

Space:

\[
\boxed{O(V^2)}
\]

---

# 82. PYQ-STYLE QUESTION 7

A DAG has negative edge weights.

Which is valid?

A. BFS only
B. Dijkstra necessarily
C. Topological shortest path
D. Kruskal

Answer:

\[
\boxed{C}
\]

---

# 83. PYQ-STYLE QUESTION 8

If after Floyd-Warshall:

\[
D[i][i]<0
\]

for some \(i\), what does it indicate?

\[
\boxed{\text{negative cycle}}
\]

---

# 84. PYQ-STYLE QUESTION 9

Relax edge:

\[
u\to v
\]

Given:

\[
dist[u]=7
\]

\[
w(u,v)=-3
\]

\[
dist[v]=8
```

```
\]

Candidate:

\[
7-3=4
\]

Since:

\[
4<8
\]

new:

\[
\boxed{dist[v]=4}
\]

---

# 85. PYQ-STYLE QUESTION 10

Given:

\[
dist[u]=5
\]

\[
w(u,v)=4
\]

\[
dist[v]=7
\]

Candidate:

\[
9
\]

No improvement.

So:

\[
\boxed{dist[v]=7}
\]

---

# 86. QUICK RECALL

Try mentally.

1. What is relaxation?
2. BFS shortest path works for what?
3. Dijkstra edge-weight condition?
4. Dijkstra design technique?
5. Dijkstra typical heap complexity?
6. Bellman-Ford handles negative edges?
7. Bellman-Ford passes?
8. Bellman-Ford complexity?
```

```
9. How detect negative cycle?
10. DAG shortest path complexity?
11. Can DAG shortest path handle negative edges?
12. Floyd-Warshall solves what?
13. Floyd-Warshall technique?
14. Floyd-Warshall time?
15. Floyd negative cycle test?

---

# ANSWERS

1. Improve \(dist[v]\) using \(dist[u]+w(u,v)\).
2. Unweighted/equal-weight graphs.
3. Non-negative weights.
4. Greedy.
5. \(O(E\log V)\).
6. Yes.
7. \(V-1\).
8. \(O(VE)\).
9. Extra relaxation pass still improves distance.
10. \(O(V+E)\).
11. Yes.
12. All-pairs shortest paths.
13. Dynamic Programming.
14. \(O(V^3)\).
15. Some \(D[i][i]<0\).

---

# 87. PRACTICE SET

## Easy

### Q1 — MCQ

Shortest path in an unweighted graph:

A. DFS
B. BFS
C. Kruskal
D. Prim

---

### Q2 — MCQ

Dijkstra works correctly when:

A. all edges are negative
B. all edge weights are non-negative
C. graph is necessarily a tree
D. graph must be complete

---

### Q3 — MCQ

Bellman-Ford time complexity:

A. \(O(V+E)\)
B. \(O(E\log V)\)
C. \(O(VE)\)
D. \(O(V^3)\)

---
```

Medium

Q4 — NAT

A graph has:

\[
V=8
\]

vertices.

How many main edge-relaxation passes does Bellman-Ford perform before the extra negative-cycle check?

Q5 — MSQ

Which algorithms can handle negative edges under their proper conditions?

A. Dijkstra
B. Bellman-Ford
C. Floyd-Warshall
D. DAG shortest path

Q6 — MCQ

Floyd-Warshall is:

A. Greedy
B. Divide and Conquer
C. Dynamic Programming
D. Hashing

Q7 — MCQ

A weighted DAG has some negative edges. Best standard algorithm:

A. Dijkstra
B. topological-order relaxation
C. BFS
D. Prim

Hard / Tricky

Q8 — MSQ

Which statements are true?

A. Dijkstra can safely finalize the minimum tentative vertex with non-negative weights.

B. Bellman-Ford can detect reachable negative cycles.

C. Floyd-Warshall requires all weights positive.

D. BFS minimizes number of edges in an unweighted graph.

```
### Q9 — NAT

Relax edge:

\[
A\to B
\]

Given:

\[
dist[A]=9
\]

\[
w(A,B)=-5
\]

\[
dist[B]=7
\]

What is the new value of \(dist[B]\)?

---

# SOLUTIONS

## Q1

\[
\boxed{B}
\]

---

## Q2

\[
\boxed{B}
\]

---

## Q3

\[
\boxed{C}
\]

---

## Q4

\[
V-1=8-1
\]

\[
\boxed{7}
\]

---

## Q5
```

```
Bellman-Ford: yes.

Floyd-Warshall: yes, assuming no problematic negative cycle.

DAG shortest path: yes.

Dijkstra: no in general.

Answer:

\[
\boxed{B,C,D}
\]

---

## Q6

\[
\boxed{C}
\]

---

## Q7

Topological relaxation.

\[
\boxed{B}
\]

---

## Q8

A true.

B true.

C false.

D true.

Answer:

\[
\boxed{A,B,D}
\]

---

## Q9

Candidate:

\[
9-5=4
\]

Current:

\[
7
\]
```

Since:

\[
4<7
\]

update:

\[
\boxed{4}
\]

---

8 8 . Ø=ÜÝ N O T E S T O W R I T E

Relaxation

For edge:

\[
u\to v
\]

if:

\[
dist[u]+w(u,v)<dist[v]
\]

then:

\[
\boxed{
dist[v]=dist[u]+w(u,v)
}
\]

BFS

Use for:

\[
\boxed{\text{unweighted/equal-weight shortest path}}
\]

Time:

\[
\boxed{O(V+E)}
\]

Dijkstra

Use when:

\[
\boxed{w(e)\ge0}
\]

Greedy.

```
Typical heap time:

\[
\boxed{O(E\log V)}
\]

Matrix:

\[
O(V^2)
\]

Does not handle negative edges in general.

---

# Bellman-Ford

Handles:

\[
\boxed{\text{negative edges}}
\]

Relax all edges:

\[
V-1
\]

times.

Time:

\[
\boxed{O(VE)}
\]

Extra pass with improvement:

\[
\boxed{\text{negative cycle}}
\]

---

# DAG Shortest Path

Topological order + relax edges once.

Allows:

\[
\boxed{\text{negative edges}}
\]

Time:

\[
\boxed{O(V+E)}
\]

---

# Floyd-Warshall
```

```
All-pairs shortest paths.

DP recurrence:

\[
\boxed{
D[i][j]
=
\min(D[i][j],D[i][k]+D[k][j])
}
\]

Time:

\[
\boxed{O(V^3)}
\]

Space:

\[
\boxed{O(V^2)}
\]

Negative-cycle clue:

\[
\boxed{D[i][i]<0}
\]

---

# 89. MASTER SHORTEST-PATH TABLE TO MEMORIZE

| Situation | Algorithm |
|---|---|
| Unweighted graph | BFS |
| Non-negative weighted graph | Dijkstra |
| Negative edges | Bellman-Ford |
| DAG with weights | Topological relaxation |
| All pairs | Floyd-Warshall |
| Need negative-cycle detection | Bellman-Ford / Floyd clues |

---

# 90. MEMORY TRICKS

### Relaxation

> **Can U make V cheaper? Update V.**

---

### BFS

 >  * * F e w e s t  e d g e s  !’  B F S * *

---

### Dijkstra

 >  * * N o  N e g a t i v e  !’  D i j k s t r a * *

---
```

Bellman-Ford

> **Bad/negative edges? Bellman handles them.**

Bellman passes

> **V vertices !' V" 1 path edges**

DAG

> **No cycles !' one topological sweep**

Floyd-Warshall

> **Every pair through every possible middle**

Think:

\[
i\to k\to j
\]

&¡ 30-SECOND REVISION

1. Shortest path minimizes total path cost.
2. Relaxation tests whether path via \(u\) improves \(v\).
3. BFS = unweighted shortest path.
4. BFS = \(O(V+E)\).
5. Dijkstra = non-negative weights.
6. Dijkstra is Greedy.
7. Dijkstra with heap "H \(O(E\log V)\).
8. Negative edge can break Dijkstra.
9. Bellman-Ford supports negative edges.
10. Bellman-Ford performs \(V-1\) main passes.
11. Extra improving pass !' negative cycle.
12. Bellman-Ford = \(O(VE)\).
13. DAG shortest path uses topological ordering.
14. DAG shortest path allows negative edges.
15. DAG shortest path = \(O(V+E)\).
16. Floyd-Warshall = all-pairs shortest path.
17. Floyd-Warshall is DP.
18. Floyd update = `min(current, via k)`.
19. Floyd = \(O(V^3)\).
20. \(D[i][i]<0\) after Floyd indicates negative cycle.
21. MST and shortest path are different problems.
22. Prim key = edge cost; Dijkstra key = total source distance.

' Chapter 15 Complete

Next Chapter:

Chapter 16 — FINAL ALGORITHMS REVISION + MASTER COMPLEXITY TABLE + GATE MIXED TEST

This will consolidate the entire syllabus:

- Searching
- Sorting
- Hashing

```
- Asymptotic analysis
- Loop & recurrence complexity
- Divide and Conquer
- Greedy
- Dynamic Programming
- BFS / DFS
- MST
- Shortest Paths

I'll give you:

- one final master complexity sheet,
- sorting-property master table,
- Greedy-vs-DP-vs-D&C decision map,
- graph algorithm-selection table,
- highest-value GATE traps,
- formulas and mnemonics,
- and a mixed GATE-style final practice test.

**Type `NEXT` to continue.**
```

USER

NEXT

ASSISTANT

CHAPTER 16 — FINAL ALGORITHMS REVISION + MASTER COMPLEXITY TABLE + GATE MIXED TEST

GATE Importance: & & & & &

This final chapter consolidates the complete Algorithms syllabus you gave and follows the final-revision structure requested in your uploaded master prompt.

Your official coverage is now complete:

- Searching
- Sorting
- Hashing
- Asymptotic worst-case time and space complexity
- Divide and Conquer
- Greedy
- Dynamic Programming
- Graph Traversals
- Minimum Spanning Trees
- Shortest Paths

This chapter is for **revision, recall and question selection**.

PART 1 — MASTER COMPLEXITY LADDER

Memorize this order:

\[\boxed{ 1 < \log n < \sqrt n < n < n\log n < n^2 < n^3 < 2^n < n! < n^n } \]

More generally:

\[(\log n)^k=o(n^\epsilon) \]

for constants:

\[k>0,\epsilon>0 \]

and:

\[n^k=o(c^n) \]

for constant:

\[c>1 \]

---

1. ASYMPTOTIC NOTATIONS

Big-O

Upper bound:

\[f(n)=O(g(n)) \]

Big-\(\Omega\)

Lower bound:

\[f(n)=\Omega(g(n)) \]

Big-\(\Theta\)

Tight bound:

\[\boxed{ f(n)=\Theta(g(n)) } \]

iff:

\[f(n)=O(g(n)) \]

and:

\[f(n)=\Omega(g(n)) \]

---

2. CRITICAL GATE CORRECTION

Do **not** memorize:

```text id="final1" O = worst case ©= best case ˜= average case

```
That is wrong.
```
```

```
Correct:

> Best/worst/average describe **input cases**.

> \(O,\Omega,\Theta\) describe **asymptotic bounds**.

---

# 3. LIMIT SHORTCUT

Consider:

\[
L=\lim_{n\to\infty}\frac{f(n)}{g(n)}
\]

### If:

\[
L=0
\]

then:

\[
f=o(g)
\]

---

### If:

\[
0<L<\infty
\]

then:

\[
f=\Theta(g)
\]

---

### If:

\[
L=\infty
\]

then:

\[
f=\omega(g)
\]

Memory:

 >  * * 0  !’  n u m e r a t o r  l o s e s * *

 >  * * f i n i t e  n o n z e r o  !’  s a m e * *

 >  * *"  !’  n u m e r a t o r  w i n s * *

---
```

```
# PART 2 — LOOP COMPLEXITY MASTER SHEET

# 4. BASIC LOOP RULES

```text id="final2"
 i + +                          !'   ~ ( n )
 i   + =   c o n s t a n t      !'   ~ ( n )

 i   * =   c o n s t a n t      !'   ~ ( l o g   n )
 i   / =   c o n s t a n t      !'   ~ ( l o g   n )

 i   =   i * i                  !'   ~ ( l o g   l o g   n ) ,   s t a r t   >   1
```

---

5. SQUARE-ROOT PATTERN

```text id="final3" for i = 1; i*i <= n; i++

Condition:

\[
i\le\sqrt n
\]

Therefore:

\[
\boxed{\Theta(\sqrt n)}
\]

---

# 6. NESTED LOOPS

Independent:

```text id="final4"
for i = 1 to n
    for j = 1 to n
```

\[\boxed{\Theta(n^2)} \]

Dependent:

```text id="final5" for i = 1 to n for j = 1 to i

\[
1+2+\cdots+n
\]

\[
=\boxed{\Theta(n^2)}
\]

---

# 7. IMPORTANT SUMS
```

```
\[
\boxed{
\sum_{i=1}^{n}1=n
}
\]

\[
\boxed{
\sum_{i=1}^{n}i=\Theta(n^2)
}
\]

\[
\boxed{
\sum_{i=1}^{n}i^2=\Theta(n^3)
}
\]

\[
\boxed{
\sum_{i=1}^{n}i^k=\Theta(n^{k+1})
}
\]

---

Geometric:

\[
1+2+4+\cdots+n
=
\boxed{\Theta(n)}
\]

---

Harmonic:

\[
1+\frac12+\frac13+\cdots+\frac1n
=
\boxed{\Theta(\log n)}
\]

---

Log sum:

\[
\sum_{i=1}^{n}\log i
=
\log(n!)
=
\boxed{\Theta(n\log n)}
\]

---

# 8. LOOP DECISION TRICK

Remember:

 >  * * A D D  !'  L i n e a r * *

 >  * * M U L T I P L Y  !'  L o g * *
```

```
 >  * * S Q U A R E !'  L o g - L o g * *

 >  * * D e p e n d e n t  i n n e r  b o u n d !'  S U M  I T * *

---

# PART 3 — RECURRENCES MASTER SHEET

# 9. COMMON RECURRENCES

\[
T(n)=T(n-1)+1
\]

\[
\boxed{\Theta(n)}
\]

---

\[
T(n)=T(n-1)+n
\]

\[
\boxed{\Theta(n^2)}
\]

---

\[
T(n)=T(n/2)+1
\]

\[
\boxed{\Theta(\log n)}
\]

---

\[
T(n)=T(n/2)+n
\]

\[
\boxed{\Theta(n)}
\]

---

\[
T(n)=2T(n/2)+1
\]

\[
\boxed{\Theta(n)}
\]

---

\[
T(n)=2T(n/2)+n
\]

\[
\boxed{\Theta(n\log n)}
```

```
\]

---

\[
T(n)=2T(n/2)+n^2
\]

\[
\boxed{\Theta(n^2)}
\]

---

# 10. MASTER THEOREM

For:

\[
T(n)=aT(n/b)+f(n)
\]

first compute:

\[
\boxed{n^{\log_ba}}
\]

Then compare \(f(n)\).

---

## Case 1

\(f(n)\) polynomially smaller:

\[
\boxed{T(n)=\Theta(n^{\log_ba})}
\]

---

## Case 2

Same order:

\[
f(n)=\Theta(n^{\log_ba})
\]

then:

\[
\boxed{
T(n)=\Theta(n^{\log_ba}\log n)
}
\]

Extended:

\[
f(n)=\Theta(n^{\log_ba}\log^k n)
\]

gives:
```

```
\[
\boxed{
\Theta(n^{\log_ba}\log^{k+1}n)
}
\]

---

## Case 3

\(f(n)\) polynomially larger and regularity holds:

\[
\boxed{T(n)=\Theta(f(n))}
\]

---

# 11. MASTER THEOREM MEMORY

 >  * * S m a l l  !’  r e c u r s i o n  w i n s * *

 >  * * S a m e  !’  a d d  l o g * *

 >  * * S t r o n g  !’  \ ( f \ )  w i n s * *

---

# 12. MASTER THEOREM DOES NOT DIRECTLY APPLY TO

\[
T(n)=T(n-1)+n
\]

\[
T(n)=T(n/2)+T(n/3)+n
\]

\[
T(n)=2T(\sqrt n)+n
\]

\[
T(n)=nT(n/2)+n
\]

Do not force it.

---

# PART 4 — SEARCHING MASTER TABLE

# 13. LINEAR SEARCH

| Case | Time |
|---|---:|
| Best | \(\Theta(1)\) |
| Average | \(\Theta(n)\) |
| Worst | \(\Theta(n)\) |
| Space | \(\Theta(1)\) |

Does not require sorted data.

---

# 14. BINARY SEARCH
```

Requires:

$$
\boxed{\text{sorted/monotonic search space}}
$$

Case	Time
Best	$\Theta(1)$
Worst	$\Theta(\log n)$

Iterative space:

$$
\boxed{\Theta(1)}
$$

Recursive space:

$$
\boxed{\Theta(\log n)}
$$

Recurrence:

$$
T(n)=T(n/2)+1
$$

---

15. BINARY SEARCH VARIANTS

First occurrence:

```
 >   F o u n d  !’   s a v e  !’   g o   l e f t .
```

Last occurrence:

```
 >   F o u n d  !’   s a v e  !’   g o   r i g h t .
```

Count duplicates:

$$
\boxed{last-first+1}
$$

Lower bound:

$$
\boxed{\text{first }A[i]\ge x}
$$

Upper bound:

$$
\boxed{\text{first }A[i]>x}
$$

PART 5 — SORTING MASTER TABLE

16. COMPLETE SORTING TABLE

```
| Algorithm | Best | Average | Worst | Stable | In-place |
|---|---:|---:|---:|---|---|
| Bubble* | \(n\) | \(n^2\) | \(n^2\) | Yes | Yes |
| Selection | \(n^2\) | \(n^2\) | \(n^2\) | No | Yes |
| Insertion | \(n\) | \(n^2\) | \(n^2\) | Yes | Yes |
| Merge | \(n\log n\) | \(n\log n\) | \(n\log n\) | Yes | No |
| Quick | \(n\log n\) | \(n\log n\) | \(n^2\) | No | Yes |
| Heap | \(n\log n\) | \(n\log n\) | \(n\log n\) | No | Yes |
| Counting | \(n+k\) | \(n+k\) | \(n+k\) | Yes** | No |
| Radix | \(d(n+k)\) | \(d(n+k)\) | \(d(n+k)\) | Depends on stable digit sort | Usually no |
| Bucket | expected \(n\) | expected \(n\) | can be \(n^2\) | Depends | Depends |

\*Optimized Bubble Sort.

\**Proper stable Counting Sort implementation.

---

# 17. STABLE SORT MEMORY TRICK

Standard stable:

```text id="final6"
Bubble
Insertion
Merge
Counting
```
```

Mnemonic:

> **BIM + Count**

Standard unstable:

```text id="final7" Selection Quick Heap

```
```
---

# 18. BUBBLE SORT

Core:

> Adjacent comparisons.

After each pass:

> largest remaining element reaches the right.

Worst comparisons:

\[
\boxed{\frac{n(n-1)}2}
\]

Bubble swaps:

\[
\boxed{\text{number of inversions}}
\]

---
```

```
# 19. SELECTION SORT

Core:

> Select minimum and put at beginning.

Comparisons:

\[
\boxed{\frac{n(n-1)}2}
\]

all major cases.

Swaps:

\[
\boxed{O(n)}
\]

Standard:

\[
\boxed{\text{unstable}}
\]

---

# 20. INSERTION SORT

Core:

> Insert next element into sorted prefix.

Best:

\[
\boxed{\Theta(n)}
\]

Worst:

\[
\boxed{\Theta(n^2)}
\]

Excellent for:

\[
\boxed{\text{nearly sorted data}}
\]

Running time related to inversions:

\[
\boxed{\Theta(n+I)}
\]

---

# 21. MERGE SORT

Recurrence:

\[
T(n)=2T(n/2)+n
```

```
\]

\[
\boxed{\Theta(n\log n)}
\]

all major cases.

Stable:

\[
\boxed{\text{Yes}}
\]

Standard array auxiliary space:

\[
\boxed{O(n)}
\]

---

# 22. QUICK SORT

Balanced:

\[
\boxed{\Theta(n\log n)}
\]

Worst:

\[
\boxed{\Theta(n^2)}
\]

Worst recurrence:

\[
T(n)=T(n-1)+n
\]

Randomized Quick Sort:

\[
\boxed{\text{Expected }\Theta(n\log n)}
\]

but worst remains:

\[
\Theta(n^2)
\]

---

# 23. HEAP SORT

Heapify:

\[
\boxed{O(\log n)}
\]

Bottom-up Build-Heap:
```

```
\[
\boxed{\Theta(n)}
\]

Heap Sort:

\[
\boxed{\Theta(n\log n)}
\]

Stable:

\[
\boxed{\text{No}}
\]

In-place:

\[
\boxed{\text{Yes}}
\]

---

# 24. COMPARISON-SORTING LOWER BOUND

There are:

\[
n!
\]

possible orders.

Decision tree requires:

\[
2^h\ge n!
\]

Thus:

\[
h\ge\log_2(n!)
\]

and:

\[
\log(n!)=\Theta(n\log n)
\]

Therefore:

\[
\boxed{\Omega(n\log n)}
\]

worst-case comparison lower bound.

---

# 25. CRITICAL TRAP

This lower bound does **not** apply to all sorting algorithms.
```

Counting/Radix exploit extra information about keys.

PART 6 — HASHING MASTER SHEET

26. HASH FUNCTION

$$
h(k)\rightarrow[0,m-1]
$$

Common:

$$
h(k)=k\bmod m
$$

Collision:

$$
k_1\neq k_2
$$

but:

$$
h(k_1)=h(k_2)
$$

---

27. LOAD FACTOR

$$
\boxed{\alpha=\frac nm}
$$

Chaining:

$$
\alpha>1
$$

is possible.

Open addressing:

$$
\alpha\le1
$$

---

28. COLLISION METHODS

Chaining

Expected:

$$
\boxed{O(1+\alpha)}
$$

Worst:

```
\[
O(n)
\]

---

## Linear Probing

\[
\boxed{
h(k,i)=(h(k)+i)\bmod m
}
\]

Problem:

\[
\boxed{\text{Primary clustering}}
\]

---

## Quadratic Probing

\[
\boxed{
h(k,i)=(h(k)+i^2)\bmod m
}
\]

Problem:

\[
\boxed{\text{Secondary clustering}}
\]

---

## Double Hashing

\[
\boxed{
h(k,i)=(h_1(k)+ih_2(k))\bmod m
}
\]

Usually better distribution.

---

# 29. DELETION IN OPEN ADDRESSING

Use:

\[
\boxed{\text{TOMBSTONE}}
\]

not ordinary EMPTY.

Why?

EMPTY can terminate search.

Tombstone means:
```

```
> keep probing.

---

# 30. HASHING COMPLEXITY

Good hashing:

```text id="final8"
 S e a r c h    !’   E x p e c t e d   O ( 1 )
 I n s e r t    !’   E x p e c t e d   O ( 1 )
 D e l e t e    !’   E x p e c t e d   O ( 1 )
```
```

Worst:

\[\boxed{O(n)} \]

Memory:

> **Hashing hopes for 1, can fall to n.**

PART 7 — DESIGN TECHNIQUE MASTER MAP

---

31. DIVIDE AND CONQUER

Pattern:

> **Split into smaller mostly independent problems.**

Examples:

- Binary Search
- Merge Sort
- Quick Sort

General:

\[aT(n/b)+f(n) \]

Memory:

> **Split !’ Solve !’ Stitch**

---

32. GREEDY

Pattern:

> **Make locally best safe choice and commit.**

Needs:

- greedy-choice property

- optimal substructure

Examples:

- Activity Selection
- Fractional Knapsack
- Huffman
- Prim
- Kruskal
- Dijkstra

Memory:

> **Best now, no going back.**

---

33. DYNAMIC PROGRAMMING

Pattern:

> **Overlapping subproblems + optimal substructure.**

Main concepts:

```text id="final9" State Transition Base Case

```

```
Memory:

> **Don't Process twice.**

---

# 34. GREEDY VS DP VS D&C

| Question Feature | Think |
|---|---|
| Independent smaller problems | D&C |
| Same states repeat | DP |
| Safe locally best choice | Greedy |

---

# 35. CLASSIFICATION TABLE

| Problem | Technique |
|---|---|
| Binary Search | D&C |
| Merge Sort | D&C |
| Quick Sort | D&C |
| Fractional Knapsack | Greedy |
| Activity Selection | Greedy |
| Huffman | Greedy |
| Prim | Greedy |
| Kruskal | Greedy |
| Dijkstra | Greedy |
| 0/1 Knapsack | DP |
| LCS | DP |
| MCM | DP |
```

```
| Floyd-Warshall | DP |

---

# PART 8 — GREEDY MASTER SHEET

# 36. ACTIVITY SELECTION

Goal:

> maximum number of non-overlapping activities.

Greedy rule:

\[
\boxed{\text{earliest finish time}}
\]

Memory:

 >  * * F i n i s h  F a s t  !'  F i t  M o r e * *

---

# 37. FRACTIONAL KNAPSACK

Choose highest:

\[
\boxed{\frac{value}{weight}}
\]

Fractions allowed.

Greedy works.

---

# 38. 0/1 KNAPSACK

Whole item or nothing.

Ratio Greedy can fail.

Use:

\[
\boxed{\text{DP}}
\]

Memory:

 >  * * F r a c t i o n  !'  G r e e d y * *

 >  * * W h o l e  !'  D P * *

---

# 39. HUFFMAN CODING

Repeatedly combine:

\[
\boxed{\text{two smallest frequencies}}
\]
```

```
Using min-heap:

\[
\boxed{O(n\log n)}
\]

---

# 40. JOB SEQUENCING

Sort jobs by:

\[
\boxed{\text{decreasing profit}}
\]

Place each:

\[
\boxed{\text{latest available slot }\le deadline}
\]

---

# PART 9 — DP MASTER SHEET

# 41. DP COMPLEXITY SHORTCUT

\[
\boxed{
\text{Time}
=
\text{number of states}
\times
\text{work per state}
}
\]

This single rule solves many complexity questions.

---

# 42. FIBONACCI

Naive recursion:

\[
O(2^n)
\]

DP:

\[
\boxed{O(n)}
\]

Space optimized:

\[
\boxed{O(1)}
\]

---

# 43. 0/1 KNAPSACK
```

```
State:

\[
dp[i][w]
\]

Transition:

\[
\boxed{
dp[i][w]
=
\max(
dp[i-1][w],
v_i+dp[i-1][w-w_i]
)
}
\]

when item fits.

Time:

\[
\boxed{O(nW)}
\]

Pseudo-polynomial.

---

# 44. LCS

If characters match:

\[
\boxed{
dp[i][j]=1+dp[i-1][j-1]
}
\]

Else:

\[
\boxed{
dp[i][j]
=
\max(dp[i-1][j],dp[i][j-1])
}
\]

Time:

\[
\boxed{O(mn)}
\]

Remember:

 >  * * M a t c h  !'  d i a g o n a l  +  1 * *

 >  * * M i s m a t c h  !'  m a x ( t o p , l e f t ) * *

---

# 45. MATRIX CHAIN MULTIPLICATION
```

```
State:

\[
dp[i][j]
\]

Transition:

\[
\boxed{
dp[i][j]
=
\min_k
\{
dp[i][k]+dp[k+1][j]+p_{i-1}p_kp_j
\}
}
\]

Time:

\[
\boxed{O(n^3)}
\]

Space:

\[
\boxed{O(n^2)}
\]

---

# 46. LIS

Classic DP:

\[
\boxed{O(n^2)}
\]

More advanced method:

\[
O(n\log n)
\]

---

# PART 10 — GRAPH FOUNDATION MASTER SHEET

# 47. BASIC FORMULAS

Graph:

\[
G=(V,E)
\]

Undirected degree sum:

\[
\boxed{\sum deg(v)=2E}
\]
```

```
Directed:

\[
\boxed{
\sum indegree
=
\sum outdegree
=
E
}
\]

---

# 48. COMPLETE GRAPH

Undirected \(K_V\):

\[
\boxed{\frac{V(V-1)}2}
\]

Directed without self-loops:

\[
\boxed{V(V-1)}
\]

---

# 49. TREE

A tree is:

\[
\boxed{\text{connected + acyclic}}
\]

Edges:

\[
\boxed{V-1}
\]

Forest with \(C\) components:

\[
\boxed{E=V-C}
\]

---

# 50. GRAPH REPRESENTATION

| Representation | Space | Edge Check |
|---|---:|---:|
| Adjacency Matrix | \(O(V^2)\) | \(O(1)\) |
| Adjacency List | \(O(V+E)\) | \(O(deg(u))\) typical |

Sparse graph:

> adjacency list.

---

# PART 11 — BFS / DFS MASTER SHEET
```

```
# 51. BFS

Uses:

\[
\boxed{\text{Queue}}
\]

Explores:

> level by level.

Adjacency-list time:

\[
\boxed{O(V+E)}
\]

Key property:

\[
\boxed{\text{shortest path in unweighted graph}}
\]

Memory:

> **BFS = Best Fewest Steps**

---

# 52. DFS

Uses:

\[
\boxed{\text{Stack/recursion}}
\]

Time:

\[
\boxed{O(V+E)}
\]

Useful for:

- reachability
- connected components
- cycle detection
- topological sorting

Memory:

> **DFS = Dive First**

---

# 53. TOPOLOGICAL SORT

Exists only for:

\[
\boxed{\text{DAG}}
\]
```

```
Time:

\[
\boxed{O(V+E)}
\]

A directed graph has a topological order:

\[
\iff
\]

it is acyclic.

---

# PART 12 — MST MASTER SHEET

# 54. SPANNING TREE

Contains all vertices.

Connected.

Acyclic.

Edges:

\[
\boxed{V-1}
\]

---

# 55. PRIM

Grow one tree.

Choose:

\[
\boxed{\text{lightest crossing edge}}
\]

Matrix:

\[
\boxed{O(V^2)}
\]

Binary heap + list:

\[
\boxed{O(E\log V)}
\]

---

# 56. KRUSKAL

Sort edges ascending.

Choose:

\[
\boxed{\text{smallest edge that does not create cycle}}
```

```
\]

Uses:

\[
\boxed{\text{Union-Find}}
\]

Time:

\[
\boxed{O(E\log E)}
\]

---

# 57. UNION-FIND

Operations:

```text id="final10"
MAKE-SET
FIND
UNION
```
```

Optimizations:

- path compression
- union by rank/size

Amortized:

\[\boxed{O(\alpha(V))} \]

---

58. MST CUT PROPERTY

> **Minimum crossing edge is safe.**

If uniquely minimum:

\[\boxed{\text{belongs to every MST}} \]

---

59. MST CYCLE PROPERTY

> **Unique heaviest edge in a cycle belongs to no MST.**

---

60. MST UNIQUENESS

If all edge weights are distinct:

\[\boxed{\text{MST unique}} \]

Converse:

\[\boxed{\text{False}} \]

Repeated weights can still have unique MST.

PART 13 — SHORTEST PATH MASTER SHEET

61. RELAXATION

For edge:

$$ u\to v $$

if:

$$ dist[u]+w(u,v)<dist[v] $$

then:

$$ \boxed{ dist[v]=dist[u]+w(u,v) } $$

62. ALGORITHM SELECTION TABLE

Situation	Algorithm	Time
Unweighted	BFS	$O(V+E)$
Non-negative weighted	Dijkstra	$O(E\log V)$ typical
Negative edges	Bellman-Ford	$O(VE)$
Weighted DAG	Topological relaxation	$O(V+E)$
All-pairs	Floyd-Warshall	$O(V^3)$

This table is worth memorizing exactly.

63. DIJKSTRA

Condition:

$$ \boxed{w(e)\ge0} $$

Strategy:

$$ \boxed{\text{Greedy}} $$

Negative edge?

> **Do not use Dijkstra in general.**

64. BELLMAN-FORD

Relax every edge:

$$ \boxed{V-1} $$

times.

Extra improving pass:

$$\boxed{\text{negative cycle}}$$

Time:

$$\boxed{O(VE)}$$

---

65. DAG SHORTEST PATH

Topological sort.

Relax once in order.

Allows:

$$\boxed{\text{negative edges}}$$

because there are no cycles.

Time:

$$\boxed{O(V+E)}$$

---

66. FLOYD-WARSHALL

All-pairs shortest path.

DP update:

$$\boxed{ D[i][j] = \min(D[i][j],D[i][k]+D[k][j]) }$$

Time:

$$\boxed{O(V^3)}$$

Space:

$$\boxed{O(V^2)}$$

Negative cycle:

$$\boxed{D[i][i]<0}$$

for some i.

PART 14 — PRIM VS DIJKSTRA

This is a major GATE confusion.

Prim	Dijkstra
MST	Shortest paths
minimizes connecting edge	minimizes complete source path
key[v] = minimum crossing edge	dist[v] = minimum source distance

Prim	Dijkstra
negative edges allowed	negative edges invalid in general

Memory:

> **Prim !’ EDGE**

> **Dijkstra !’ PATH**

PART 15 — 30 HIGHEST-VALUEGATE TRAPS

67. TRAP LIST

1.

Big-O does not mean worst case.

2.

Two loops do not automatically mean \(n^2\).

3.

Dependent nested loops need summation.

4.

`i*=2` !’ \(\log n\), not \(n\).

5.

`i=i*i` from \(i>1\) !’ \(\log\log n\).

6.

Master Theorem does not apply to \(T(n-1)\) recurrence.

7.

Binary Search requires sorted/monotonic structure.

8.

Ordinary Binary Search does not guarantee first duplicate.

9.

Selection Sort best case remains \(\Theta(n^2)\).

10.

Optimized Bubble best = \(\Theta(n)\), basic fixed-pass version may not.

11.

Selection Sort is unstable in standard form.

12.

Insertion Sort prefix is sorted but not necessarily final.

13.

Merge Sort uses $O(n)$ auxiliary space for arrays.

14.

Quick Sort worst = $\Theta(n^2)$.

15.

Randomized Quick Sort still has n^2 worst case.

16.

Bottom-up Build-Heap = $\Theta(n)$, not $n\log n$.

17.

Comparison-sorting lower bound does not apply to Counting Sort.

18.

Counting Sort is $O(n+k)$, not always $O(n)$.

19.

Linear probing !’ primary clustering.

20.

Quadratic probing !’ secondary clustering.

21.

Open-address deletion needs tombstone.

22.

Hashing expected $O(1)$ does not mean worst $O(1)$.

23.

Optimal substructure alone does not prove Greedy.

24.

Fractional Knapsack = Greedy; 0/1 Knapsack = DP.

25.

BFS shortest path only for unweighted/equal-weight edges.

26.

Traversal order may depend on neighbor ordering.

27.

Equal MST edge weights do not automatically imply multiple MSTs.

28.

Prim and Dijkstra are different objectives.

29.

Dijkstra does not support negative edges in general.

30.

Floyd-Warshall is DP, not Greedy.

PART 16 — ULTRA-SHORT MEMORY SHEET

68. ONE-LINE MNEMONICS

Asymptotics

> **O upper, Ω lower, Θ tight**

Loops

> **Add !' n, Multiply !' log, Square !' log-log**

D&C

> **Split !' Solve !' Stitch**

Greedy

> **Grab best safe choice now**

DP

> **Don't Process twice**

Bubble

> **Biggest Bubbles Back**

Selection

> **Select Smallest, Set at Start**

Insertion

> **Insert into sorted prefix**

Merge

> **Split and Stitch**

Quick

> **Pivot decides performance**

Heap

> **Build in n, remove n times in log n**

Activity Selection

> **Finish Fast !' Fit More**

Fractional Knapsack

> **Fraction !' Ratio**

Huffman

> **Small + Small repeatedly**

BFS

> **Queue + Levels + Fewest edges**

DFS

> **Stack + Dive + Backtrack**

Prim

> **Plant one tree and grow**

Kruskal

> **Keep smallest safe edge**

Dijkstra

> **No Negative !' nearest unfinished**

Bellman-Ford

> **Negative? Relax everything \(V-1\) times**

Floyd

> **Every pair through every middle**

PART 17 — GATE ALGORITHM SELECTION IN 10 SECONDS

69. DECISION MAP

Question says:

Search unsorted?

\[\boxed{\text{Linear Search}} \]

Search sorted?

\[\boxed{\text{Binary Search}} \]

Nearly sorted?

\[\boxed{\text{Insertion Sort}} \]

Stable \(n\log n\) comparison sort?

\[\boxed{\text{Merge Sort}} \]

Low extra array memory + good average sorting?

\[\boxed{\text{Quick Sort}} \]

Worst-case \(n\log n\) + in-place?

\[\boxed{\text{Heap Sort}} \]

Small integer key range?

\[\boxed{\text{Counting Sort}} \]

Repeated subproblems?

\[\boxed{\text{DP}} \]

Safe local choice?

\[\boxed{\text{Greedy}} \]

Independent recursive halves?

\[\boxed{\text{D\&C}} \]

Unweighted graph shortest path?

\[\boxed{\text{BFS}} \]

Reachability/components?

\[\boxed{\text{BFS/DFS}} \]

MST growing from vertex?

\[\boxed{\text{Prim}} \]

MST sorting edges?

\[\boxed{\text{Kruskal}} \]

Non-negative weighted shortest path?

\[\boxed{\text{Dijkstra}} \]

Negative edges?

\[\boxed{\text{Bellman-Ford}} \]

Weighted DAG?

\[\boxed{\text{Topological relaxation}} \]

All-pairs shortest paths?

\[\boxed{\text{Floyd-Warshall}} \]

PART 18 — FINAL MIXED GATE-STYLE TEST

Try these before checking the solutions.

Q1 — MCQ

Complexity:

```text id="final11" i = 1 while i <= n: i *= 3

```

```
A. \(\Theta(n)\)
B. \(\Theta(\log n)\)
```

```
C. \(\Theta(\sqrt n)\)
D. \(\Theta(n\log n)\)

---

## Q2 — MCQ

Complexity:

```text id="final12"
for i = 1 to n:
    for j = 1 to i:
        operation
```
```

A. \(\Theta(n)\) B. \(\Theta(n\log n)\) C. \(\Theta(n^2)\) D. \(\Theta(n^3)\)

Q3 — MCQ

Solve:

\[T(n)=4T(n/2)+n \]

A. \(\Theta(n)\) B. \(\Theta(n\log n)\) C. \(\Theta(n^2)\) D. \(\Theta(n^3)\)

Q4 — MSQ

Which standard sorting algorithms are stable?

A. Bubble B. Selection C. Insertion D. Merge

Q5 — MCQ

Bottom-up Build-Heap complexity:

A. \(O(\log n)\) B. \(O(n)\) C. \(O(n\log n)\) D. \(O(n^2)\)

Q6 — MCQ

Worst-case comparison sorting lower bound:

A. \(\Omega(n)\) B. \(\Omega(\log n)\) C. \(\Omega(n\log n)\) D. \(\Omega(n^2)\)

Q7 — NAT

A hash table has:

\[n=75 \]

keys and:

\[m=100 \]

slots.

Find load factor.

Q8 — MCQ

Primary clustering occurs mainly in:

A. Separate chaining B. Linear probing C. Quadratic probing D. Double hashing

Q9 — MCQ

Which problem is classically Greedy?

A. 0/1 Knapsack B. Matrix Chain Multiplication C. Fractional Knapsack D. LCS

Q10 — MCQ

Activity Selection chooses:

A. earliest start B. earliest finish C. shortest duration D. maximum duration

Q11 — MCQ

Simple DP complexity of LCS for string lengths (m,n):

A. $O(m+n)$ B. $O(mn)$ C. $O(m^2n^2)$ D. $O(\log mn)$

Q12 — NAT

A tree contains:

$$ 40 $$

vertices.

Number of edges?

Q13 — MCQ

BFS uses:

A. Stack B. Queue C. Heap D. Union-Find

Q14 — MSQ

Which have adjacency-list traversal time:

$$ O(V+E) $$

A. BFS B. DFS C. Selection Sort D. Binary Search

Q15 — MCQ

Which MST algorithm uses Union-Find?

A. Prim B. Kruskal C. Dijkstra D. Floyd-Warshall

Q16 — MSQ

Which statements are true?

A. All distinct edge weights imply unique MST.

B. Equal edge weights always imply multiple MSTs.

C. Prim is Greedy.

D. Kruskal is Greedy.

Q17 — MCQ

Shortest path in graph with arbitrary negative edges but no reachable negative cycle:

A. Dijkstra B. Bellman-Ford C. Prim D. BFS

Q18 — MCQ

Floyd-Warshall complexity:

A. $O(V+E)$ B. $O(E\log V)$ C. $O(VE)$ D. $O(V^3)$

Q19 — MCQ

If after Floyd-Warshall:

$$ D[i][i]<0 $$

then:

A. graph disconnected B. MST is unique C. negative cycle exists D. graph is DAG

Q20 — MSQ

Which algorithms can correctly handle negative edges under proper conditions?

A. Bellman-Ford B. DAG shortest path C. Floyd-Warshall D. Dijkstra

---

PART 19 — FINAL TEST SOLUTIONS

Q1

Repeated multiplication:

$$ \boxed{\Theta(\log n)} $$

Answer:

$$ \boxed{B} $$

Q2

$$ 1+2+\cdots+n $$

$$ =\Theta(n^2) $$

Answer:

$$ \boxed{C} $$

Q3

$$ a=4,b=2 $$

$$ n^{\log_24}=n^2 $$

$$ f(n)=n $$

Case 1.

$$ \boxed{\Theta(n^2)} $$

Answer:

$$ \boxed{C} $$

Q4

Stable standard algorithms:

- Bubble
- Insertion
- Merge

Answer:

$$ \boxed{A,C,D} $$

Q5

$$ \boxed{B} $$

Bottom-up Build-Heap:

$$ \Theta(n) $$

Q6

$$ \boxed{C} $$

Q7

$$ \alpha=\frac{75}{100} $$

$$ \boxed{0.75} $$

Q8

$$ \boxed{B} $$

Linear probing.

Q9

$$ \boxed{C} $$

Fractional Knapsack.

Q10

$$ \boxed{B} $$

Earliest finish.

Q11

States:

$$ m\times n $$

Each $O(1)$.

$$ \boxed{O(mn)} $$

Answer:

$$ \boxed{B} $$

Q12

$$ 40-1 = \boxed{39} $$

Q13

$$ \boxed{B} $$

Queue.

Q14

BFS and DFS.

$$\boxed{A,B}$$

Q15

$$\boxed{B}$$

Kruskal.

Q16

A true.

B false.

C true.

D true.

Answer:

$$\boxed{A,C,D}$$

Q17

Negative edges:

$$\boxed{\text{Bellman-Ford}}$$

Answer:

$$\boxed{B}$$

Q18

$$\boxed{D}$$

$$O(V^3)$$

Q19

$$\boxed{C}$$

Negative cycle.

Q20

Bellman-Ford:

Yes.

DAG shortest path:

Yes.

Floyd-Warshall:

Yes, assuming shortest paths are well-defined/no problematic negative cycles.

Dijkstra:

No in general.

Answer:

\[\boxed{A,B,C} \]

PART 20 — Ø=ÜÍFINAL NOTES TO WRITE ON ONE PAGE

If you make only **one final revision page**, write this:

```text id="final13" ALGORITHMS — GATE LAST REVISION

Growth: 1 < log n < " n< n < nlogn < n² < n³ < 2 •< n!

Loops: i++ !’ n i=*2 !’ log n i=i* !’ loglog n £ i !’ n² £ i ² !’ n³ £ 1 / i !’ log n 1+2+4... !’ n

Master: T(n)=aT(n/b)+f(n) compare f with n^(log_b a)

Search: Linear !’ worst n Binary !’ worst log n, sorted

Stable: Bubble, Insertion, Merge, Counting

Selection: n² all cases, O(n) swaps

Insertion: best n, nearly sorted

Merge: nlogn all cases, stable, space n

Quick: avg nlogn, worst n²

Heap: BuildHeap n HeapSort nlogn

Comparison sort lower bound: © ( n l o g n )

Hash: ± = n /m Linear !’ primary clustering Quadratic !’ secondary Double hashing !’ best spread Expected O(1), worst O(n)

D&C: Binary, Merge, Quick

Greedy: Activity !’ earliest finish Fractional K !’ max v/w Huffman !’ two smallest Prim/Kruskal/Dijkstra

DP: 0/1 K !’ nW LCS !’ mn MCM !’ n³

Graph: £ d e g r e e T r e e E=V-1 Forest E=V-C

BFS: Queue V+E unweighted shortest

DFS: Stack V+E

MST: Prim !’ cheapest crossing edge Kruskal !’ smallest safe edge + DSU MST edges = V-1 distinct weights !’ unique MST

Shortest: Unweighted !’ BFS Nonnegative !’ Dijkstra Negative !’ Bellman-Ford DAG !’ Topological All-pairs !’ Floyd

Bellman: V-1 passes VE extra improvement !’ negative cycle

Floyd: D[i][j]=min(D[i][j],D[i][k]+D[k][j]) V³ D[i][i]<0 !’ negative cycle
```

```
---

 #  &¡   F I N A L   6 0 - S E C O N D   R E V I S I O N

If you are outside the exam hall and have only one minute:

1. Binary Search = \(\log n\), sorted.
2. Merge = \(n\log n\), stable, extra \(n\) space.
3. Quick = average \(n\log n\), worst \(n^2\).
4. Heap = Build-Heap \(n\), sort \(n\log n\).
5. Comparison lower bound = \(\Omega(n\log n)\).
6. Hash expected = \(O(1)\), worst = \(O(n)\).
7. Linear probing = primary clustering.
8. Greedy: Fractional Knapsack.
9. DP: 0/1 Knapsack.
10. BFS = queue + unweighted shortest path.
11. DFS = stack + deep traversal.
12. BFS/DFS = \(O(V+E)\).
13. Tree edges = \(V-1\).
14. Prim = grow one MST tree.
15. Kruskal = sort edges + Union-Find.
 1 6 .   D i s t i n c t   e d g e   w e i g h t s   !’   u n i q u e   M S T .
17. Dijkstra = non-negative weights.
18. Bellman-Ford = negative edges + \(V-1\) relaxations.
19. Floyd-Warshall = all pairs + \(V^3\).
20. DAG shortest paths can handle negative edges in \(O(V+E)\).

 #  '   C h a p t e r   1 6   C o m p l e t e

 #  '   C O M P L E T E   G A T E   A L G O R I T H M S   S Y L L A B U S   F I N I S H E D

You have now completed the full structured Algorithms course for the syllabus you supplied:

**Searching, Sorting, Hashing, Asymptotic Worst-Case Time and Space Complexity, Greedy, Dynamic
Programming, Divide-and-Conquer, Graph Traversals, Minimum Spanning Trees and Shortest Paths.**

## Recommended next action

Do **not** immediately reread all 16 chapters from the beginning.

Use this order:

 * * 1 .   R e v i s e   C h a p t e r   1 6   o n c e   !’   2 .   S o l v e   A l g o r i t h m s   P Y Q s
 r e t u r n   o n l y   t o   t h a t   s p e c i f i c   c h a p t e r / c o n c e p t   !’   4 .   M a i n t

That will convert these notes into actual GATE marks much more efficiently than repeatedly
rereading everything.
```

USER

NEXT

ASSISTANT

PHASE 2 — GATE ALGORITHMS PYQ-ORIENTED PRACTICE

You have finished the **complete theory syllabus**, so from this point onward, continuing chapter-style theory would mostly create repetition.

The best next phase is:

$$\boxed{\text{Concept Recall} \rightarrow \text{Mixed Problems} \rightarrow \text{Mistake Analysis}}$$

Your original study plan is PYQ-oriented, so we'll now convert the theory into **GATE problem-solving ability**.

PRACTICE SET 1 — COMPLEXITY + SEARCHING + SORTING

Difficulty: Easy !' GATE-level tricky **Questions:** 15 **Types:** MCQ + MSQ + NAT

Important: Try each question before looking for a solution. I am intentionally **not giving the solutions in this message** so that this works like an actual test.

Q1 — NAT: Loop Complexity

Consider:

```text
count = 0

for i = 1; i <= n; i = i * 2:
    for j = 1; j <= i; j++:
        count++
```

Find the asymptotic value of `count`.

Write your answer in $\Theta(\cdot)$ form.

Q2 — MCQ: Nested Loop Trap

```text
for i = 1 to n:
    j = 1

    while j <= n:
        j = j * 2
```

Time complexity is:

A. $\Theta(n)$ B. $\Theta(\log n)$ C. $\Theta(n\log n)$ D. $\Theta(n^2)$

Q3 — MCQ: Log-Log

```text
i = 2

while i <= n:
    i = i * i
```

Time complexity:

A. $\Theta(\log n)$ B. $\Theta(\sqrt n)$ C. $\Theta(\log\log n)$ D. $\Theta(n)$

Q4 — MSQ: Asymptotic Relations

Which are true?

A.

$$ n=O(n^2) $$

B.

$$ n^2=O(n) $$

C.

$$ n\log n=o(n^2) $$

D.

$$ 2^n=o(n^3) $$

Q5 — MCQ: Recurrence

Consider:

$$ T(n)=2T(n/2)+n $$

Then:

A. $\Theta(n)$ B. $\Theta(n\log n)$ C. $\Theta(n^2)$ D. $\Theta(\log n)$

Q6 — MCQ: Recurrence Trap

$$ T(n)=T(n/2)+n $$

has complexity:

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n\log n)$ D. $\Theta(n^2)$

Q7 — NAT: Binary Search

A sorted array contains:

$$ 1024 $$

elements.

Suppose standard Binary Search repeatedly halves the search interval.

What is the logarithmic scale:

$$ \log_2 1024 $$

relevant to the number of halvings?

Enter the numeric value.

Q8 — MSQ: Binary Search

Which statements are correct?

A. Standard Binary Search requires an ordered/monotonic search space.

B. Binary Search always returns the first occurrence of a duplicate key.

C. Iterative Binary Search can use $O(1)$ auxiliary space.

D. Recursive Binary Search can use $O(\log n)$ stack space.

Q9 — MCQ: Sorting Identification

After one complete pass of an ascending sorting algorithm, the **largest remaining element is guaranteed to be in the last position**.

The algorithm most directly described is:

A. Insertion Sort B. Selection Sort C. Bubble Sort D. Counting Sort

---

Q10 — MSQ: Sorting Properties

Which standard algorithms are stable?

A. Bubble Sort B. Selection Sort C. Insertion Sort D. Merge Sort E. Heap Sort

---

Q11 — NAT: Selection Sort

Classic Selection Sort operates on:

$$ n=20 $$

elements.

Number of key comparisons is:

$$ \frac{n(n-1)}2 $$

Compute the answer.

Q12 — NAT: Inversions

Consider:

```
TEXT
[5, 4, 3, 2, 1]
```

How many inversions are present?

Q13 — MCQ: Nearly Sorted Input

Among the following, which is generally the best simple sorting algorithm for a **nearly sorted** array?

A. Selection Sort B. Insertion Sort C. Heap Sort D. Standard unoptimized Bubble Sort

---

Q14 — MSQ: Merge / Quick / Heap

Which statements are correct?

A. Standard Merge Sort has worst-case $\Theta(n\log n)$.

B. Standard Quick Sort has worst-case $\Theta(n^2)$.

C. Bottom-up Build-Heap takes $\Theta(n\log n)$.

D. Heap Sort has worst-case $\Theta(n\log n)$.

E. Standard array Merge Sort is in-place using $O(1)$ auxiliary memory.

Q15 — HARD: Comparison-Sorting Lower Bound

Suppose a comparison-based sorting algorithm sorts:

$$ 5 $$

distinct elements.

A binary decision tree representing the comparisons must have at least:

$$ 5!=120 $$

leaves.

Therefore its worst-case depth must be at least:

$$ \left\lceil\log_2 120\right\rceil $$

What is the minimum integer value?

#ñþHOW TO ATTEMPT THIS SET

For every question, train yourself to first identify the **pattern**, not calculate blindly:

```TEXT
Loop:
 a d d i t i v e   u p d a t e            !’   u s u a l l y   l i n e a r
 m u l t i p l i c a t i v e   u p d a t e  !’   u s u a l l y   l o g a r i t h m i c
 i   =   i ²                               !’   l o g   l o g

Recurrence:
identify a, b, f(n)

Binary Search:
 h a l f  !’   l o g

Bubble:
largest moves right

Selection:
select minimum

Insertion:
sorted prefix

Merge:
stable + n log n + extra memory
```

```
Quick:
pivot + possible n² worst case

Heap:
Build-Heap = n

Comparison sorting:
 n !   l e a v e s  !’   l o g ( n ! )  !’   n   l o g   n
```

& þTHREE QUESTIONS YOU SHOULD BE ESPECIALLY CAREFUL WITH

Q1: Do not say \(n\log n\) just because one loop is logarithmic and another looks linear. The inner-loop work changes with `i`.

Q6: A recurrence with logarithmic depth does **not** necessarily have logarithmic total work.

Q14: Remember the famous distinction:

\[\boxed{\text{Heapify}=O(\log n)} \]

but:

\[\boxed{\text{Bottom-up Build-Heap}=\Theta(n)} \]

YOUR TARGET

Before moving ahead, a strong score is:

\[\boxed{13/15\text{ or better}} \]

But wrong answers are useful here—the goal is to expose exactly which concepts need revision.

' GATE Algorithms Practice Set 1 Ready

Next:

Solutions + Mistake-Trap Analysis + Practice Set 2 — Hashing, Greedy & Dynamic Programming

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PRACTICE SET 1 — SOLUTIONS + MISTAKE-TRAP ANALYSIS

You should not just check the final option. For GATE, the important part is learning **why the wrong options are tempting**.

Q1 — LOOP COMPLEXITY

```text
for i = 1; i <= n; i = i * 2:
    for j = 1; j <= i; j++:
        count++
```

Outer values:

$$ 1,2,4,8,\ldots,n $$

Inner work:

$$ 1+2+4+\cdots+n $$

Geometric series:

$$ =2n-1 $$

Therefore:

$$ \boxed{\Theta(n)} $$

Major trap

Do **not** multiply:

$$ \log n\times n $$

because the inner loop does not execute n times for every outer iteration.

Q2

Outer:

$$ n $$

Inner:

$$ \log n $$

Independent work:

$$ \boxed{\Theta(n\log n)} $$

Answer:

$$ \boxed{C} $$

Q3

```text
i = 2
i = i*i
```

Values:

$$ 2,\ 4,\ 16,\ 256,\ldots $$

After k iterations:

$$ i=2^{2^k} $$

Need:

$$ 2^{2^k}\le n $$

Take log:

$$ 2^k\le\log_2 n $$

Again log:

$$ k\le\log_2\log_2 n $$

Therefore:

$$ \boxed{\Theta(\log\log n)} $$

Answer:

$$ \boxed{C} $$

Q4 — ASYMPTOTIC RELATIONS

A:

$$ n=O(n^2) $$

True.

B:

$$ n^2=O(n) $$

False.

C:

$$ n\log n=o(n^2) $$

True because:

$$ \frac{n\log n}{n^2} = \frac{\log n}{n} \to0 $$

D:

$$ 2^n=o(n^3) $$

False. Exponential eventually dominates polynomial.

Answer:

$$ \boxed{A,C} $$

Q5

$$ T(n)=2T(n/2)+n $$

Master Theorem:

$$ a=2,\quad b=2 $$

$$ n^{\log_22}=n $$

Since:

$$ f(n)=n $$

same order.

Therefore:

$$ \boxed{\Theta(n\log n)} $$

Answer:

$$ \boxed{B} $$

Q6

$$ T(n)=T(n/2)+n $$

Expansion:

$$ n+\frac n2+\frac n4+\frac n8+\cdots $$

Geometric sum:

$$ <2n $$

Therefore:

$$ \boxed{\Theta(n)} $$

Answer:

$$ \boxed{B} $$

Trap

Recursion depth is:

$$ \Theta(\log n) $$

but total running time is:

$$ \Theta(n) $$

Depth "`total work.

Q7

$$ \log_2 1024 $$

Since:

$$ 1024=2^{10} $$

Answer:

$$ \boxed{10} $$

Note: exact Binary Search comparison counts depend on successful/unsuccessful case and implementation; this question only asked the logarithmic halving scale.

Q8

A. Ordered/monotonic search space !’ true.

B. Standard Binary Search always gives first duplicate !’ false.

C. Iterative auxiliary space $O(1)$!’ true.

D. Recursive stack $O(\log n)$!’ true.

Answer:

$$\boxed{A,C,D}$$

Q9

Largest element bubbles toward the end after a full pass.

Answer:

$$\boxed{C\text{ — Bubble Sort}}$$

Q10

Standard stable algorithms among given choices:

- Bubble
- Insertion
- Merge

Selection and Heap are normally unstable.

Answer:

$$\boxed{A,C,D}$$

Q11

Selection Sort comparisons:

$$\frac{n(n-1)}2$$

For:

$$n=20$$

$$\frac{20\times19}{2} = 10\times19 = \boxed{190}$$

Q12

Reverse-sorted array of 5 elements has maximum inversions:

$$\frac{5(4)}2 = \boxed{10}$$

Q13

Nearly sorted data strongly favors:

$$ \boxed{\text{Insertion Sort}} $$

Answer:

$$ \boxed{B} $$

Q14

A. Merge worst $n\log n$!' true.

B. Quick worst n^2 !' true.

C. Build-Heap $n\log n$!' false.

Correct:

$$ \Theta(n) $$

D. Heap worst $n\log n$!' true.

E. Standard array Merge Sort $O(1)$ space !' false.

Correct auxiliary:

$$ O(n) $$

Answer:

$$ \boxed{A,B,D} $$

Q15

$$ 5!=120 $$

Need:

$$ \lceil\log_2 120\rceil $$

$$ 2^6=64 $$

$$ 2^7=128 $$

Therefore:

$$ \boxed{7} $$

PRACTICE SET 1 — ANSWER KEY

Q	Answer
1	$\Theta(n)$
2	C
3	C

Q	Answer
4	A,C
5	B
6	B
7	10
8	A,C,D
9	C
10	A,C,D
11	190
12	10
13	B
14	A,B,D
15	7

---

5 HIGH-VALUE MISTAKES TO NEVER REPEAT

Mistake 1

Nested loops:

> **blindly multiply bounds.**

Instead:

> **if inner work depends on outer variable, sum it.**

Mistake 2

Recursion depth:

$$\log n$$

therefore total time:

$$\log n$$

Wrong.

Count work at every level.

Mistake 3

All stable algorithms?

Remember:

> **BIM**

$$\boxed{\text{Bubble, Insertion, Merge}}$$

Mistake 4

Heapify vs Build-Heap

$$\boxed{\text{Heapify}=O(\log n)}$$

but:

$$\boxed{\text{Build-Heap}=\Theta(n)}$$

Mistake 5

Comparison sorting:

$$\Omega(n\log n)$$

does **not** mean every sorting algorithm needs $n\log n$.

PRACTICE SET 2 — HASHING + GREEDY + DYNAMIC PROGRAMMING

Questions: 18 **Level:** Easy !' tricky GATE-style **Types:** MCQ + MSQ + NAT

Try them before checking solutions in the next phase.

HASHING

Q1 — NAT

A hash table has:

$$m=50$$

slots and:

$$n=35$$

stored keys.

Find load factor:

$$\alpha$$

Q2 — MCQ

Which collision-resolution method suffers primarily from **primary clustering**?

A. Separate chaining B. Linear probing C. Quadratic probing D. Double hashing

---

Q3 — MCQ

Quadratic probing is mainly associated with:

A. no clustering at all B. primary clustering only C. secondary clustering D. linked lists

Q4 — MSQ

Which statements are true?

A. Separate chaining can have \(\alpha>1\).

B. Open addressing can store more keys than table slots.

C. Hash-table search is expected \(O(1)\) under good assumptions.

D. Hash-table search can degrade to \(O(n)\).

Q5 — NAT: LINEAR PROBING

Let:

\[m=10 \]

\[h(k)=k\bmod10 \]

Insert in order:

```text
23, 43, 13, 27, 33
```

using Linear Probing.

At what index is `33` stored?

Q6 — MCQ

Why should a deleted slot in open addressing generally be marked `DELETED` rather than ordinary `EMPTY`?

A. To improve sorting B. To preserve probe sequences during search C. To increase hash value D. To prevent insertion forever

GREEDY

Q7 — MCQ

For Activity Selection, the standard optimal Greedy rule is:

A. earliest starting activity B. shortest duration C. earliest finishing activity D. highest starting time

Q8 — NAT: FRACTIONAL KNAPSACK

Knapsack capacity:

\[W=20 \]

Items:

Item	Value	Weight
A	60	10
B	100	20
C	120	30

Find the maximum value using **Fractional Knapsack**.

Q9 — MSQ

Which are classically Greedy algorithms/problems?

A. Fractional Knapsack B. Huffman Coding C. Prim's Algorithm D. 0/1 Knapsack E. Kruskal's Algorithm

---

Q10 — MCQ

Huffman Coding repeatedly combines:

A. two maximum frequencies B. two minimum frequencies C. minimum and maximum frequency D. arbitrary symbols

---

Q11 — MCQ

Which statement is correct?

A. Optimal substructure alone guarantees Greedy correctness.

B. Every Greedy problem is also solved by Binary Search.

C. Greedy correctness generally requires a safe local-choice property.

D. Greedy always explores every possible solution.

DYNAMIC PROGRAMMING

Q12 — MCQ

Which is the clearest indicator that Dynamic Programming may help?

A. subproblems never repeat B. overlapping subproblems C. graph is complete D. array is sorted

---

Q13 — MSQ

Which statements are correct?

A. Memoization is usually top-down.

B. Tabulation is usually bottom-up.

C. Every DP solution must use a 2D matrix.

D. DP often stores solutions to repeated states.

Q14 — NAT: 0/1 KNAPSACK

Capacity:

$$ W=5 $$

Items:

Weight	Value
2	12
1	10
3	20
2	15

What is the maximum 0/1 Knapsack value?

Q15 — MCQ

Standard 0/1 Knapsack DP complexity with n items and capacity W is:

A. $O(n+W)$ B. $O(nW)$ C. $O(2^W)$ always D. $O(\log W)$

Q16 — NAT: LCS

Find the LCS length for:

```text
X = ABCD
Y = ACBD
```

Q17 — MCQ

If:

$$ X[i-1]=Y[j-1] $$

LCS transition is:

A.

$$ dp[i][j]=dp[i-1][j] $$

B.

$$ dp[i][j]=1+dp[i-1][j-1] $$

C.

\[dp[i][j]=dp[i][j-1] \]

D.

\[dp[i][j]=0 \]

Q18 — HARD MSQ

Which statements are correct?

A. Fractional Knapsack is solved optimally by ratio Greedy.

B. Ratio Greedy always solves 0/1 Knapsack optimally.

C. Matrix Chain Multiplication has standard DP time \(O(n^3)\).

D. LCS has standard DP time \(O(mn)\).

E. \(O(nW)\) 0/1 Knapsack is pseudo-polynomial.

&#BEFORE YOU MOVE ON — REMEMBER THESE 9 RULES

TEXT

```
Hash:
 ±  =  n / m
 L i n e a r  !'  P r i m a r y
 Q u a d r a t i c  !'  S e c o n d a r y
 D e l e t i o n  !'  T o m b s t o n e
Expected O(1), Worst O(n)

Greedy:
 A c t i v i t y  !'  e a r l i e s t  f i n i s h
 F r a c t i o n a l  K  !'  m a x  v a l u e / w e i g h t
 H u f f m a n  !'  t w o  s m a l l e s t

DP:
 R e p e a t e d  s t a t e s  !'  s t o r e  t h e m
 0 / 1  K  !'  D P
 L C S  m a t c h  !'  d i a g o n a l  +  1
```

' Practice Set 2 Ready

Next:

Practice Set 2 Solutions + Practice Set 3 — BFS, DFS, MST & Shortest Paths

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PRACTICE SET 2 — SOLUTIONS + MISTAKE-TRAP ANALYSIS

Now check not only the answers, but also the **reasoning pattern** you should recognize in a GATE question.

Q1 — LOAD FACTOR

$$\alpha=\frac{n}{m}$$

Given:

$$n=35,\quad m=50$$

$$\alpha=\frac{35}{50}$$

$$\boxed{0.7}$$

Q2 — PRIMARY CLUSTERING

Primary clustering is mainly caused by:

$$\boxed{\text{Linear Probing}}$$

Answer:

$$\boxed{B}$$

Memory:

> **Linear !' Long consecutive cluster !' Primary**

Q3 — QUADRATIC PROBING

Quadratic probing reduces primary clustering but keys with the same home position still follow the same probe sequence.

Therefore:

$$\boxed{\text{Secondary clustering}}$$

Answer:

$$\boxed{C}$$

Q4 — HASHING STATEMENTS

A. Chaining can have $\alpha>1$

True.

Multiple keys can occupy one bucket through a chain.

B. Open addressing can store more keys than slots

False.

All keys must live inside the table.

$$ n\le m $$

C. Hash search expected $O(1)$

True under:

- good hash function,
- controlled load factor,
- suitable assumptions.

D. Hash search can degrade to $O(n)$

True.

Example: all keys collide.

Answer:

$$ \boxed{A,C,D} $$

Q5 — LINEAR PROBING

$$ m=10 $$

$$ h(k)=k\bmod10 $$

Insert:

```
TEXT
23, 43, 13, 27, 33
```

23

$$ 23\bmod10=3 $$

Place at:

$$ 3 $$

43

Home 3 occupied.

Place at:

$$ 4 $$

13

3 occupied.

4 occupied.

Place at:

$$ 5 $$

27

$$ 27\bmod10=7 $$

Place at:

$$ 7 $$

33

Home:

$$ 3 $$

Probe:

```TEXT
3 occupied
4 occupied
5 occupied
6 free
```

Therefore:

$$ \boxed{6} $$

Q6 — DELETION IN OPEN ADDRESSING

If deleted position is marked ordinary `EMPTY`, later searches may terminate there incorrectly.

Use:

$$ \boxed{\text{DELETED/TOMBSTONE}} $$

so searches continue.

Answer:

$$ \boxed{B} $$

Q7 — ACTIVITY SELECTION

Standard Greedy criterion:

$$ \boxed{\text{Earliest finishing activity}} $$

Answer:

$$ \boxed{C} $$

Memory:

> **Finish Fast !' Fit More**

Q8 — FRACTIONAL KNAPSACK

Capacity:

$$ W=20 $$

Items:

Item	Value	Weight	Ratio
A	60	10	6
B	100	20	5
C	120	30	4

Choose highest ratio first.

Take A fully.

Weight used:

$$ 10 $$

Value:

$$ 60 $$

Remaining capacity:

$$ 10 $$

Next item B weighs 20.

Take:

$$ \frac{10}{20} = \frac12 $$

Value obtained:

$$ 100\times\frac12=50 $$

Total:

$$ 60+50 $$

$$ \boxed{110} $$

Q9 — GREEDY PROBLEMS

Fractional Knapsack

Greedy.

Huffman

Greedy.

Prim

Greedy.

0/1 Knapsack

DP.

Kruskal

Greedy.

Answer:

$$\boxed{A,B,C,E}$$

Q10 — HUFFMAN CODING

Repeatedly combine:

$$\boxed{\text{two smallest frequencies}}$$

Answer:

$$\boxed{B}$$

Q11 — GREEDY CORRECTNESS

A Greedy algorithm works only when its local choice can be shown to be safe.

Answer:

$$\boxed{C}$$

A useful proof may use:

- exchange argument,
- cut property,
- greedy-stays-ahead argument.

Q12 — DP INDICATOR

The strongest clue is:

$$\boxed{\text{Overlapping subproblems}}$$

Answer:

$$\boxed{B}$$

Q13 — MEMOIZATION / TABULATION

A. Memoization !' top-down.

True.

B. Tabulation !' bottom-up.

True.

C. Every DP requires 2D matrix.

False.

D. DP stores repeated states.

True.

Answer:

$$\boxed{A,B,D}$$

Q14 — 0/1 KNAPSACK

Capacity:

$$W=5$$

Items:

Weight	Value
2	12
1	10
3	20
2	15

Try useful combinations.

Items 1 + 2 + 4

Weight:

$$2+1+2=5$$

Value:

$$12+10+15 = 37$$

Items 3 + 4

Weight:

$$3+2=5$$

Value:

$$20+15=35$$

Items 1 + 3

Weight:

$$5$$

Value:

$$32$$

Maximum:

$$\boxed{37}$$

Q15 — KNAPSACK COMPLEXITY

Classic DP:

$$ n\times W $$

states.

Constant work per state.

Therefore:

$$ \boxed{O(nW)} $$

Answer:

$$ \boxed{B} $$

Remember:

> **This is pseudo-polynomial.**

Q16 — LCS

```text
X = ABCD
Y = ACBD
```

Possible common subsequences:

```text
ABD
ACD
```

Each length:

$$ 3 $$

Can we get length 4?

No, because B and C appear in opposite relative order:

```text
ABCD
ACBD
```

Therefore:

$$ \boxed{3} $$

Q17 — LCS MATCH TRANSITION

If:

$$ X[i-1]=Y[j-1] $$

then:

$$ \boxed{ dp[i][j] = 1+dp[i-1][j-1] } $$

Answer:

$$ \boxed{B} $$

Memory:

> **Match !' diagonal + 1**

Q18 — HARD MSQ

A. Fractional Knapsack by ratio Greedy.

True.

B. Ratio Greedy always solves 0/1 Knapsack.

False.

C. MCM standard DP:

$$ O(n^3) $$

True.

D. LCS:

$$ O(mn) $$

True.

E. 0/1 Knapsack $O(nW)$ is pseudo-polynomial.

True.

Answer:

$$ \boxed{A,C,D,E} $$

PRACTICE SET 2 — ANSWER KEY

Q	Answer
1	0.7
2	B
3	C
4	A,C,D
5	6
6	B

Q	Answer
7	C
8	110
9	A,B,C,E
10	B
11	C
12	B
13	A,B,D
14	37
15	B
16	3
17	B
18	A,C,D,E

---

7 MISTAKES YOU SHOULD NEVER REPEAT

1. Load factor

\[\boxed{\alpha=n/m} \]

Do not reverse it.

---

2. Clustering

```text
L i n e a r        !'   P r i m a r y
Q u a d r a t i c  !'   S e c o n d a r y
```

---

3. Open-address deletion

> **EMPTY stops. DELETED continues.**

---

4. Activity Selection

Do not choose shortest duration.

Choose:

\[\boxed{\text{earliest finish}} \]

---

5. Knapsack

```TEXT
F r a c t i o n a l  !'  G r e e d y
0 / 1                !'  D P
```

---

6. DP

DP does not mean:

> **"make a 2D table."**

It means:

> **store and reuse subproblem results.**

---

7. LCS

```TEXT
M a t c h       !'  d i a g o n a l  +  1
M i s m a t c h !'  m a x ( t o p , l e f t )
```

PRACTICE SET 3 — BFS, DFS, MST & SHORTEST PATHS

Questions: 20 **Difficulty:** Basic !' GATE-tricky **Types:** MCQ + MSQ + NAT

Try these before reading solutions in the next `NEXT`.

PART A — GRAPH BASICS + BFS/DFS

Q1 — NAT

An undirected graph has:

\[E=18 \]

edges.

What is the sum of degrees of all vertices?

Q2 — NAT

A complete undirected graph contains:

\[V=12 \]

vertices.

How many edges does it contain?

Q3 — NAT

A tree has:

$$ 31 $$

vertices.

How many edges?

Q4 — MCQ

Which data structure is naturally used by BFS?

A. Stack B. Queue C. Min-Heap D. Union-Find

Q5 — MCQ

Which is naturally associated with DFS?

A. Queue B. Stack/recursion C. Priority queue only D. Hash probing

Q6 — MSQ

Which statements are correct?

A. BFS using adjacency lists takes $O(V+E)$.

B. DFS using adjacency lists takes $O(V+E)$.

C. BFS always produces the same traversal order independent of adjacency order.

D. DFS can be used for reachability.

Q7 — MCQ

Which algorithm guarantees shortest paths in an **unweighted graph**?

A. DFS B. BFS C. Prim D. Kruskal

Q8 — NAT

A forest contains:

$$ V=20 $$

vertices and:

$$ C=6 $$

connected components.

How many edges does it contain?

PART B — MST

Q9 — MCQ

Prim's algorithm chooses:

A. globally smallest unused edge regardless of current tree

B. smallest edge crossing from current MST vertices to outside vertices

C. highest-weight edge

D. shortest complete source path

Q10 — MCQ

Kruskal's algorithm selects:

A. smallest edge that does not create a cycle

B. shortest source-to-vertex path

C. highest-weight edge

D. only edges adjacent to one starting vertex

Q11 — NAT

A connected weighted graph has:

$$V=17$$

vertices.

How many edges does every MST contain?

Q12 — MSQ

Which statements are correct?

A. Prim is Greedy.

B. Kruskal is Greedy.

C. Kruskal commonly uses Union-Find.

D. Prim and Kruskal require all edge weights to be non-negative.

Q13 — MCQ

If all edge weights in a connected undirected graph are distinct, then:

A. no MST exists B. MST is unique C. every spanning tree is an MST D. Kruskal fails

Q14 — MSQ: MST PROPERTIES

Which statements are true?

A. A unique lightest edge crossing a cut belongs to every MST.

B. A unique heaviest edge in a cycle belongs to no MST.

C. Equal edge weights always imply multiple MSTs.

D. Negative edge weights can appear in an MST.

PART C — SHORTEST PATHS

Q15 — MCQ

For a weighted graph with all edge weights non-negative, the standard SSSP algorithm is:

A. Prim B. Dijkstra C. Kruskal D. DFS

Q16 — MCQ

For a graph that may contain negative edges, but no reachable negative cycle, use:

A. Dijkstra B. Bellman-Ford C. Prim D. BFS

Q17 — NAT

A graph has:

$$V=9$$

vertices.

How many main edge-relaxation passes does Bellman-Ford perform before the extra negative-cycle check?

Q18 — MSQ

Which statements are correct?

A. Dijkstra is Greedy.

B. Bellman-Ford can detect reachable negative cycles.

C. Floyd-Warshall computes all-pairs shortest paths.

D. Weighted DAG shortest paths require all weights to be non-negative.

Q19 — NAT: RELAXATION

Suppose:

$$dist[u]=8$$

$$dist[v]=12$$

and edge:

\[u\rightarrow v \]

has weight:

\[-3 \]

After relaxing the edge, what is the new value of:

\[dist[v]? \]

Q20 — HARD MSQ

Which statements are true?

A. BFS finds shortest paths when all edges have equal weight.

B. Dijkstra works correctly with arbitrary negative edges.

C. A weighted DAG can be solved in \(O(V+E)\) using topological order.

D. Floyd-Warshall has time complexity \(O(V^3)\).

E. If after Floyd-Warshall some \(D[i][i]<0\), a negative cycle exists.

STOP AND RECALL THESE BEFORE SOLVING

```text
Undirected degree sum = 2E

Complete undirected:
E = V(V-1)/2

Tree:
E = V-1

Forest:
E = V-C

BFS:
Queue
O(V+E)
Unweighted shortest path

DFS:
Stack
O(V+E)

Prim:
Grow one tree
Minimum crossing edge

Kruskal:
Sort edges
Smallest cycle-safe edge
Union-Find

MST:
V-1 edges
 D i s t i n c t   w e i g h t s  !’   u n i q u e   M S T
```

```
Dijkstra:
Non-negative weights
Greedy

Bellman-Ford:
Negative edges
V-1 passes
O(VE)

DAG shortest:
Topological order
O(V+E)

Floyd:
All pairs
O(V³)
 D [ i ] [ i ]  <  0  !'  n e g a t i v e  c y c l e
```

' Practice Set 3 Ready

Next:

Practice Set 3 Solutions + Final GATE Algorithms Weak-Area Test

That next test will deliberately mix concepts so the question **does not tell you which chapter it belongs to** — much closer to actual GATE reasoning.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PRACTICE SET 3 — SOLUTIONS + MISTAKE-TRAP ANALYSIS

This set covered **Graph Basics, BFS/DFS, MST, and Shortest Paths**. The key goal is to recognize the right algorithm/property quickly.

Q1 — DEGREE SUM

For an undirected graph:

$$\sum deg(v)=2E$$

Given:

$$E=18$$

$$2E=36$$

$$\boxed{36}$$

Q2 — COMPLETE UNDIRECTED GRAPH

$$E=\frac{V(V-1)}{2}$$

For:

$$V=12$$

$$E=\frac{12\cdot 11}{2}$$

$$=66$$

$$\boxed{66}$$

Q3 — TREE EDGES

For a tree:

$$E=V-1$$

$$=31-1$$

$$\boxed{30}$$

Q4 — BFS DATA STRUCTURE

BFS explores level by level.

It uses:

$$\boxed{\text{Queue}}$$

Answer:

$$\boxed{B}$$

Q5 — DFS DATA STRUCTURE

DFS uses:

$$\boxed{\text{Stack / recursion}}$$

Answer:

$$\boxed{B}$$

Q6 — BFS / DFS STATEMENTS

A. BFS with adjacency list:

$$O(V+E)$$

True.

B. DFS with adjacency list:

$$ O(V+E) $$

True.

C. BFS order is independent of adjacency order.

False.

Neighbor ordering can change traversal order.

D. DFS can test reachability.

True.

Answer:

$$ \boxed{A,B,D} $$

Q7 — UNWEIGHTED SHORTEST PATH

BFS visits vertices in increasing number of edges from source.

Therefore:

$$ \boxed{\text{BFS}} $$

Answer:

$$ \boxed{B} $$

Q8 — FOREST

For a forest:

$$ E=V-C $$

Given:

$$ V=20,\quad C=6 $$

$$ E=20-6 $$

$$ \boxed{14} $$

Q9 — PRIM

Prim grows one tree and chooses:

> **minimum-weight edge crossing from the current tree to an outside vertex.**

Answer:

$$ \boxed{B} $$

Q10 — KRUSKAL

Kruskal sorts edges and repeatedly chooses:

> **smallest edge that does not create a cycle.**

Answer:

$$\boxed{A}$$

Q11 — MST EDGE COUNT

Every spanning tree on V vertices contains:

$$V-1$$

Given:

$$V=17$$

$$\boxed{16}$$

Q12 — MST STATEMENTS

A. Prim is Greedy.

True.

B. Kruskal is Greedy.

True.

C. Kruskal commonly uses Union-Find.

True.

D. Prim and Kruskal require non-negative weights.

False.

Negative edge weights are allowed.

Answer:

$$\boxed{A,B,C}$$

Q13 — DISTINCT EDGE WEIGHTS

For a connected undirected graph:

$$\boxed{\text{all edge weights distinct} \Rightarrow \text{unique MST}}$$

Answer:

$$\boxed{B}$$

Important:

The converse is false.

Q14 — MST PROPERTIES

A. Unique lightest edge crossing a cut belongs to every MST.

True.

B. Unique heaviest edge in a cycle belongs to no MST.

True.

C. Equal weights always imply multiple MSTs.

False.

D. Negative weights can appear in MST.

True.

Answer:

$$\boxed{A,B,D}$$

Q15 — NON-NEGATIVE SHORTEST PATHS

For weighted SSSP with:

$$w(e)\ge0$$

use:

$$\boxed{\text{Dijkstra}}$$

Answer:

$$\boxed{B}$$

Q16 — NEGATIVE EDGES

If negative edges may exist and there is no reachable negative cycle:

$$\boxed{\text{Bellman-Ford}}$$

Answer:

$$\boxed{B}$$

Q17 — BELLMAN-FORD PASSES

Main relaxation rounds:

$$V-1$$

Given:

$$V=9$$

$$9-1 = \boxed{8}$$

Then one extra pass may be used for negative-cycle detection.

Q18 — SHORTEST-PATHSTATEMENTS

A. Dijkstra is Greedy.

True.

B. Bellman-Ford detects reachable negative cycles.

True.

C. Floyd-Warshall computes all-pairs shortest paths.

True.

D. Weighted DAG shortest paths require non-negative weights.

False.

A DAG can contain negative edges.

Answer:

$$\boxed{A,B,C}$$

Q19 — RELAXATION

Given:

$$dist[u]=8$$

$$w(u,v)=-3$$

Candidate:

$$8+(-3)=5$$

Current:

$$dist[v]=12$$

Since:

$$5<12$$

update:

$$\boxed{dist[v]=5}$$

Q20 — HARD MSQ

A. BFS works when all edges have equal weight.

True.

B. Dijkstra works with arbitrary negative edges.

False.

C. Weighted DAG shortest path can be:

$$O(V+E)$$

True.

D. Floyd-Warshall:

$$O(V^3)$$

True.

E. After Floyd:

$$ D[i][i]<0 $$

indicates a negative cycle.

True.

Answer:

$$ \boxed{A,C,D,E} $$

PRACTICE SET 3 — ANSWER KEY

Q	Answer
1	36
2	66
3	30
4	B
5	B
6	A,B,D
7	B
8	14
9	B
10	A
11	16
12	A,B,C
13	B
14	A,B,D
15	B
16	B
17	8
18	A,B,C
19	5
20	A,C,D,E

8 GRAPH MISTAKES TO NEVER REPEAT

1. BFS vs DFS

```text
B F S  !'  Q u e u e
D F S  !'  S t a c k
```

2. BFS shortest path

BFS gives shortest path only for:

$$\boxed{\text{unweighted/equal-weight edges}}$$

not arbitrary weighted graphs.

3. Traversal order

BFS/DFS traversal order is **not necessarily unique**.

Adjacency/tie-breaking order matters.

4. Tree

$$\boxed{E=V-1}$$

5. Prim vs Kruskal

```text
P r i m       !'  g r o w  o n e  t r e e
K r u s k a l !'  j o i n  c o m p o n e n t s
```

6. MST negative edges

Negative edge weights are completely valid.

Do not confuse MST with Dijkstra.

7. Dijkstra

Requirement:

$$\boxed{w(e)\ge0}$$

Zero is allowed.

Negative is the problem.

8. Bellman-Ford

$$ V-1 $$

normal passes.

One extra improving pass:

$$ \boxed{\text{negative cycle}} $$

FINAL GATE ALGORITHMS WEAK-AREA TEST

Now we remove chapter labels.

The question will **not tell you whether it is Sorting, DP, Graphs, Complexity, Greedy, etc.**

That is closer to how GATE tests you.

Questions: 25 **Difficulty:** Mixed, with deliberate traps **Do not check answers immediately.**

Q1 — MCQ

Consider:

```text
i = 1

while i <= n:
    j = 1

    while j <= i:
        j++

    i *= 2
```

Time complexity is:

A. $\Theta(\log n)$ B. $\Theta(n)$ C. $\Theta(n\log n)$ D. $\Theta(n^2)$

Q2 — MCQ

$$ T(n)=3T(n/2)+n $$

has asymptotic complexity:

A. $\Theta(n)$

B. $\Theta(n\log n)$

C. $\Theta(n^{\log_2 3})$

D. $\Theta(n^2)$

Q3 — MSQ

Which statements are correct?

A.

$$ n\log n=O(n^2) $$

B.

$$ n^3=o(2^n) $$

C.

$$ 2^n=O(n^{10}) $$

D.

$$ \log(n!)=\Theta(n\log n) $$

Q4 — NAT

Classic Selection Sort is applied to:

$$ n=15 $$

elements.

How many key comparisons does it perform?

Q5 — MCQ

An algorithm sorts an array by repeatedly choosing a pivot and arranging smaller keys on one side and larger keys on the other.

The algorithm is:

A. Heap Sort B. Merge Sort C. Quick Sort D. Selection Sort

Q6 — MSQ

Which standard sorting algorithms have worst-case:

$$ \Theta(n\log n) $$

A. Merge Sort B. Heap Sort C. Quick Sort D. Selection Sort

Q7 — MCQ

A sorting algorithm has:

- best case $\Theta(n)$,
- worst case $\Theta(n^2)$,
- stable,
- in-place,
- performs very well on nearly sorted input.

Most likely:

A. Selection Sort B. Insertion Sort C. Heap Sort D. Merge Sort

Q8 — NAT

A reverse-sorted array contains:

$$ 9 $$

distinct elements.

How many inversions?

Q9 — MSQ

Which statements are true?

A. Bottom-up Build-Heap is $\Theta(n)$.

B. Heapify is $O(\log n)$.

C. Standard Heap Sort is stable.

D. Heap Sort has worst-case $\Theta(n\log n)$.

Q10 — MCQ

A hash table uses:

$$ h(k)=k\bmod 7 $$

and Linear Probing.

Keys are inserted:

```text
10, 17, 24
```

At what index is `24` placed?

A. 3 B. 4 C. 5 D. 6

Q11 — MSQ

Which statements are correct?

A. Chaining can have load factor greater than 1.

B. Linear probing suffers from primary clustering.

C. Quadratic probing completely eliminates clustering.

D. Hash search can have worst-case $O(n)$.

Q12 — MCQ

A problem requires selecting the maximum number of mutually compatible intervals.

Which rule is the standard optimal one?

A. Earliest starting interval B. Shortest interval C. Earliest finishing interval D. Largest interval

Q13 — NAT

Knapsack capacity:

$$ W=15 $$

Items:

Value	Weight
40	5
50	10
60	15

Using Fractional Knapsack, find the maximum value.

Q14 — MCQ

Which condition most strongly suggests Dynamic Programming?

A. The graph is dense.

B. The same subproblems occur repeatedly.

C. Every edge is positive.

D. Input is sorted.

Q15 — MSQ

Which are standard DP problems?

A. LCS B. Matrix Chain Multiplication C. 0/1 Knapsack D. Fractional Knapsack

Q16 — NAT

For strings of lengths:

$$ 25 $$

and:

$$ 40 $$

a straightforward LCS DP has how many (i,j) states if the extra zero row/column is ignored?

Q17 — MCQ

A connected undirected graph has:

$$ V=14,\quad E=13 $$

What must it be?

A. Complete graph B. Tree C. Directed acyclic graph D. Disconnected forest

Q18 — NAT

An undirected graph has degree sequence:

```text
3, 3, 2, 2, 2, 2
```

How many edges does it have?

Q19 — MSQ

Which statements are correct?

A. BFS uses a queue.

B. DFS uses a stack/recursion.

C. BFS finds minimum-weight paths in every weighted graph.

D. Both BFS and DFS can test reachability.

Q20 — MCQ

A directed graph admits a topological ordering if and only if:

A. it is complete B. it is connected C. it is acyclic D. all weights are positive

Q21 — MSQ

Which statements about MSTs are true?

A. Every MST has $V-1$ edges.

B. Prim and Kruskal are Greedy.

C. If all weights are distinct, the MST is unique.

D. Negative edges make MST impossible.

Q22 — MCQ

Which algorithm repeatedly selects the globally smallest edge that joins two currently different components?

A. Prim B. Kruskal C. Dijkstra D. Bellman-Ford

Q23 — MCQ

A graph has some negative edge weights and may contain cycles, but no reachable negative cycle.

Which algorithm is appropriate for SSSP?

A. Dijkstra B. Bellman-Ford C. BFS D. Prim

Q24 — MSQ

Which statements are correct?

A. Dijkstra is Greedy.

B. Floyd-Warshall is Dynamic Programming.

C. A DAG shortest-path algorithm can handle negative edges.

D. Bellman-Ford normally performs \(E-1\) relaxation rounds.

Q25 — HARD NAT

Suppose a comparison-based sorting algorithm sorts:

\[7 \]

distinct elements.

The decision tree must have at least:

\[7! \]

leaves.

Find:

\[\left\lceil\log_2(7!)\right\rceil \]

DO NOT JUST CALCULATE — IDENTIFY THE SIGNAL

Before answering, mentally classify:

```text
Repeated multiplication? ! logarithmic loop
Dependent loop? ! sum work
aT(n/b)+f(n)? ! Master Theorem

Nearly sorted? ! Insertion
Pivot? ! Quick
Build-Heap? ! ~(n)
n! leaves? ! comparison lower bound

Load factor/clustering? ! Hashing

Intervals? ! Greedy
Repeated states? ! DP

Queue? ! BFS
Stack? ! DFS
```

```
V - 1  e d g e s ?  !'  T r e e / M S T

G l o b a l  s a f e  s m a l l e s t  e d g e ?  !'  K r u s k a l
G r o w i n g  o n e  t r e e ?  !'  P r i m

N o - n e g a t i v e  s h o r t e s t  p a t h ?  !'  D i j k s t r a
N e g a t i v e  e d g e s ?  !'  B e l l m a n - F o r d
D A G ?  !'  T o p o l o g i c a l  r e l a x a t i o n
A l l  p a i r s ?  !'  F l o y d
```

TARGET SCORE

For this final mixed test:

\[\boxed{22/25} \]

is a very strong target.

More important than the score:

> **Any question you miss should become one line in your Algorithms Mistake Sheet.**

' Final Weak-Area Test Ready

Next:

Final Weak-Area Test Solutions + Personal Weak-Topic Revision Map

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

FINAL WEAK-AREA TEST — SOLUTIONS + WEAK-TOPIC REVISION MAP

This completes the **Algorithms theory !' practice !' mixed-test cycle**.

Since you continued directly with `NEXT` rather than sending your attempted answers, I cannot truthfully label any topic as *your* weakness yet. So after the solutions, I'll give you an exact **question-number !' weak-topic map**: whatever you got wrong tells you exactly what to revise.

Q1 — LOOP COMPLEXITY

TEXT

```
i = 1
```

```
while i <= n:
    j = 1
    while j <= i:
        j++

    i *= 2
```

Outer values:

$$ 1,2,4,8,\ldots,n $$

Work:

$$ 1+2+4+8+\cdots+n $$

Geometric sum:

$$ \Theta(n) $$

Therefore:

$$ \boxed{B\ —\ \Theta(n)} $$

Trap

Do not multiply:

$$ n\times\log n $$

The inner loop does **not** run n times on every iteration.

Q2 — RECURRENCE

$$ T(n)=3T(n/2)+n $$

Master Theorem:

$$ a=3,\quad b=2 $$

Compute:

$$ n^{\log_2 3} $$

Since:

$$ \log_2 3\approx1.585 $$

and:

$$ f(n)=n $$

is polynomially smaller.

Therefore:

$$ \boxed{ T(n)=\Theta(n^{\log_2 3}) } $$

Answer:

$$ \boxed{C} $$

Q3 — ASYMPTOTIC RELATIONS

A

$$ n\log n=O(n^2) $$

True.

B

$$ n^3=o(2^n) $$

True.

Exponential eventually dominates every fixed-degree polynomial.

C

$$ 2^n=O(n^{10}) $$

False.

D

$$ \log(n!)=\Theta(n\log n) $$

True.

Answer:

$$ \boxed{A,B,D} $$

Q4 — SELECTION SORT COMPARISONS

$$ \frac{n(n-1)}2 $$

For:

$$ n=15 $$

$$ \frac{15\times14}{2} = 15\times7 = \boxed{105} $$

Q5 — PIVOT SORTING

Pivot + partition:

$$ \boxed{\text{Quick Sort}} $$

Answer:

$$ \boxed{C} $$

Q6 — WORST-CASE $n\log n$

Merge Sort

$$ \Theta(n\log n) $$

Yes.

Heap Sort

$$\Theta(n\log n)$$

Yes.

Quick Sort

Worst:

$$\Theta(n^2)$$

No.

Selection Sort

$$\Theta(n^2)$$

No.

Answer:

$$\boxed{A,B}$$

Q7 — NEARLY SORTED ARRAY

Properties:

- stable,
- in-place,
- best $\Theta(n)$,
- worst $\Theta(n^2)$,
- excellent on nearly sorted data.

This describes:

$$\boxed{\text{Insertion Sort}}$$

Answer:

$$\boxed{B}$$

Q8 — INVERSIONS

Reverse-sorted array with 9 distinct elements:

$$I_{\max} = \frac{n(n-1)}2$$

$$= \frac{9\times8}{2}$$

$$=\boxed{36}$$

Q9 — HEAPS

A. Bottom-up Build-Heap:

$$\Theta(n)$$

True.

B. Heapify:

$$ O(\log n) $$

True.

C. Heap Sort stable.

False.

D. Heap Sort worst:

$$ \Theta(n\log n) $$

True.

Answer:

$$ \boxed{A,B,D} $$

Q10 — LINEAR PROBING

$$ h(k)=k\bmod7 $$

Keys:

```text
10,17,24
```

All hash to:

$$ 3 $$

because:

$$ 10\bmod7=3 $$

$$ 17\bmod7=3 $$

$$ 24\bmod7=3 $$

Insert:

```text
 1 0  !'   3
 1 7  !'   4
 2 4  !'   5
```

Therefore:

$$ \boxed{5} $$

Answer:

$$ \boxed{C} $$

Q11 — HASHING

A. Chaining can have:

$$ \alpha>1 $$

True.

B. Linear probing !’ primary clustering.

True.

C. Quadratic probing eliminates all clustering.

False.

It can have:

$$ \boxed{\text{secondary clustering}} $$

D. Hash search worst-case:

$$ O(n) $$

True.

Answer:

$$ \boxed{A,B,D} $$

Q12 — ACTIVITY SELECTION

Standard Greedy rule:

$$ \boxed{\text{earliest finishing interval}} $$

Answer:

$$ \boxed{C} $$

Memory:

> **Finish Fast !’ Fit More**

Q13 — FRACTIONAL KNAPSACK

Capacity:

$$ W=15 $$

Ratios:

Value	Weight	Ratio
40	5	8
50	10	5
60	15	4

Take first item fully:

Weight:

$$ 5 $$

Value:

$$ 40 $$

Remaining:

$$ 10 $$

Take second item fully:

Weight:

$$ 10 $$

Value:

$$ 50 $$

Total capacity:

$$ 15 $$

Total value:

$$ 40+50 = \boxed{90} $$

Q14 — DP IDENTIFICATION

The strongest signal:

> **same subproblems occur repeatedly.**

Therefore:

$$ \boxed{\text{Dynamic Programming}} $$

Answer:

$$ \boxed{B} $$

Q15 — DP PROBLEMS

LCS

DP.

Matrix Chain Multiplication

DP.

0/1 Knapsack

DP.

Fractional Knapsack

Greedy.

Answer:

$$ \boxed{A,B,C} $$

Q16 — LCS STATES

Lengths:

$$ 25,\quad40 $$

Ignoring extra zero row/column:

$$ 25\times40 $$

$$ =\boxed{1000} $$

Q17 — CONNECTED GRAPH

Given:

$$ V=14,\quad E=13 $$

and graph is connected.

Since:

$$ E=V-1 $$

it must be a:

$$ \boxed{\text{Tree}} $$

Answer:

$$ \boxed{B} $$

Q18 — HANDSHAKING LEMMA

Degree sequence:

```text
3,3,2,2,2,2
```

Sum:

$$ 3+3+2+2+2+2 = 14 $$

For undirected graph:

$$ \sum degree=2E $$

Therefore:

$$ 2E=14 $$

$$ \boxed{E=7} $$

Q19 — BFS / DFS

A. BFS uses queue.

True.

B. DFS uses stack/recursion.

True.

C. BFS finds minimum-weight paths in every weighted graph.

False.

D. Both test reachability.

True.

Answer:

$$ \boxed{A,B,D} $$

Q20 — TOPOLOGICAL ORDER

A directed graph admits topological ordering iff it is:

$$ \boxed{\text{acyclic}} $$

That is, a DAG.

Answer:

$$ \boxed{C} $$

Q21 — MST

A. Every MST has $V-1$ edges.

True.

B. Prim and Kruskal are Greedy.

True.

C. Distinct edge weights imply unique MST.

True.

D. Negative edges make MST impossible.

False.

Answer:

$$ \boxed{A,B,C} $$

Q22 — GLOBAL SMALLEST SAFE EDGE

This describes:

$$ \boxed{\text{Kruskal}} $$

Answer:

$$ \boxed{B} $$

Q23 — NEGATIVE EDGE SSSP

Negative edges may exist and graph can contain cycles.

Use:

$$\boxed{\text{Bellman-Ford}}$$

Answer:

$$\boxed{B}$$

Q24 — ALGORITHM CLASSIFICATION

A. Dijkstra is Greedy.

True.

B. Floyd-Warshall is DP.

True.

C. DAG shortest path can handle negative edges.

True.

D. Bellman-Ford performs $E-1$ rounds.

False.

Correct:

$$\boxed{V-1}$$

main passes.

Answer:

$$\boxed{A,B,C}$$

Q25 — COMPARISON DECISION TREE

$$7!=5040$$

We need:

$$\left\lceil\log_2 5040\right\rceil$$

Check powers:

$$2^{12}=4096$$

$$2^{13}=8192$$

Since:

$$4096<5040\le8192$$

therefore:

$$\boxed{13}$$

FINAL ANSWER KEY

Q	Answer
1	B — \(\Theta(n)\)
2	C
3	A,B,D
4	105
5	C
6	A,B
7	B
8	36
9	A,B,D
10	C / index 5
11	A,B,D
12	C
13	90
14	B
15	A,B,C
16	1000
17	B
18	7
19	A,B,D
20	C
21	A,B,C
22	B
23	B
24	A,B,C
25	13

HOW TO SCORE YOURSELF

23–25 correct

$$\boxed{\text{Very strong}}$$

You should stop rereading basic Algorithms theory and focus heavily on actual GATE PYQs.

20–22 correct

$$\boxed{\text{Good}}$$

Theory is mostly solid. Revise only missed areas and then solve PYQs.

16–19 correct

$$\boxed{\text{Moderate}}$$

Do targeted revision before heavy PYQ practice.

Below 16

Do not restart the entire subject.

Identify your weak clusters below and revise only those chapters.

PERSONAL WEAK-TOPIC MAP

Use the question numbers you got wrong.

If Q1–Q3 were wrong

Revise:

Complexity & Recurrences

Go back to:

- Chapter 2 — Asymptotics
- Chapter 3 — Loops
- Chapter 4 — Recurrences

Must know:

$$1+2+4+\cdots+n=\Theta(n)$$

$$T(n)=aT(n/b)+f(n)$$

$$n^{\log_ba}$$

Priority:

$$\boxed{\text{VERY HIGH}}$$

If Q4–Q9 were wrong

Revise:

Sorting

Go back to Chapters 6–8.

Must memorize:

Sort	Key fact
Bubble	adjacent swaps
Selection	\(n(n-1)/2\) comparisons
Insertion	nearly sorted
Merge	stable, \(n\log n\), space \(n\)
Quick	pivot, worst \(n^2\)
Heap	Build-Heap \(n\), sort \(n\log n\)

Also:

\[I_{\max} = \frac{n(n-1)}2 \]

Priority:

\[\boxed{\text{VERY HIGH}} \]

If Q10–Q11 were wrong

Revise:

Hashing

Return to Chapter 9.

Write this from memory:

\[\alpha=\frac nm \]

```text
Linear       !'  Primary  clustering
Quadratic    !'  Secondary  clustering
Double       !'  Better  spread
```

Expected:

\[O(1) \]

Worst:

\[O(n) \]

Priority:

\[\boxed{\text{HIGH}} \]

If Q12–Q16 were wrong

Revise:

Greedy + DP

The central distinction:

```
TEXT
 S a f e   l o c a l   c h o i c e            !'  G r e e d y
 R e p e a t e d   s u b p r o b l e m s      !'  D P
 I n d e p e n d e n t   s u b p r o b l e m s !'  D i v i d e   &   C o n q u e r
```

Must know:

```
TEXT
 A c t i v i t y   S e l e c t i o n !'  e a r l i e s t   f i n i s h
 F r a c t i o n a l   K            !'  v a l u e / w e i g h t
 0 / 1   K                          !'  D P
 H u f f m a n                      !'  t w o   m i n i m u m
 L C S                                !'  D P
 M C M                                !'  D P
```

Complexities:

\[0/1K=O(nW) \]

\[LCS=O(mn) \]

\[MCM=O(n^3) \]

Priority:

\[\boxed{\text{VERY HIGH}} \]

If Q17–Q20 were wrong

Revise:

Graph Foundations + BFS/DFS

Must instantly know:

\[\sum deg(v)=2E \]

\[Tree:\ E=V-1 \]

\[Forest:\ E=V-C \]

```
TEXT
 B F S  !'  Q u e u e
 D F S  !'  S t a c k
```

Both with adjacency list:

\[O(V+E) \]

BFS:

\[\boxed{\text{unweighted shortest path}} \]

Topological sort:

\[\boxed{\text{DAG only}} \]

Priority:

$$ \boxed{\text{VERY HIGH}} $$

If Q21–Q22 were wrong

Revise:

MST

Must distinguish:

Prim

> **Grow one tree using cheapest crossing edge.**

Kruskal

> **Globally smallest safe edge + Union-Find.**

MST:

$$ V-1 $$

edges.

Distinct weights:

$$ \boxed{\text{unique MST}} $$

Negative edges:

$$ \boxed{\text{allowed}} $$

Priority:

$$ \boxed{\text{HIGH}} $$

If Q23–Q24 were wrong

Revise:

Shortest Paths

Memorize this exact table:

Situation	Algorithm
Unweighted	BFS
Non-negative	Dijkstra
Negative edges	Bellman-Ford
Weighted DAG	Topological relaxation
All pairs	Floyd-Warshall

And:

\[Bellman-Ford=O(VE) \]

with:

\[V-1 \]

main passes.

\[Floyd=O(V^3) \]

Priority:

\[\boxed{\text{VERY HIGH}} \]

If Q25 was wrong

Revise:

Comparison-Sorting Lower Bound

Only remember this chain:

\[n!\text{ possible orders} \]

\[2^h\ge n! \]

\[h\ge\log_2(n!) \]

\[\log(n!)=\Theta(n\log n) \]

Therefore:

\[\boxed{\Omega(n\log n)} \]

for general comparison sorting.

YOUR ALGORITHMS MISTAKE SHEET

Make one page titled:

`ALGORITHMS — MISTAKES I MUST NOT REPEAT`

Do **not** copy all notes.

Whenever you get a PYQ wrong, write only:

```text
Question/topic:
What I thought:
Correct idea:
Why I was wrong:
One-line memory rule:
```

Example:

```text
Topic: Build-Heap

Wrong thought:
```

```
n nodes × log n heapify = n log n

Correct:
Most nodes are near leaves.

Answer:
 B u i l d - H e a p   =   ~ ( n )

Memory:
Heapify log n, Build-Heap n.
```

This will become far more useful than rereading dozens of pages.

THE 20 FACTS I WOULD LOCK BEFORE GATE

1. \(O\neq\) automatically worst case.
2. Multiply-update loop !' \(\log n\).
3. Dependent loops !' sum.
4. Master Theorem compares \(f(n)\) with \(n^{\log_ba}\).
5. Binary Search !' sorted/monotonic + \(\log n\).
6. Selection !' \(n(n-1)/2\) comparisons.
7. Insertion !' nearly sorted.
8. Merge !' stable \(n\log n\), \(O(n)\) extra space.
9. Quick !' average \(n\log n\), worst \(n^2\).
10. Build-Heap !' \(n\).
11. Comparison sort lower bound !' \(\Omega(n\log n)\).
12. Hash !' expected \(O(1)\), worst \(O(n)\).
13. Fractional Knapsack !' Greedy; 0/1 !' DP.
14. BFS !' queue + unweighted shortest path.
15. DFS !' stack/recursion.
16. BFS/DFS !' \(O(V+E)\).
17. Prim !' crossing edge; Kruskal !' globally smallest safe edge.
18. Dijkstra !' non-negative.
19. Bellman-Ford !' negative edges + \(V-1\) passes.
20. Floyd-Warshall !' all pairs + \(O(V^3)\).

' GATE ALGORITHMS COMPLETE

At this point, **do not continue creating more theory chapters for Algorithms**.

The highest-value next step is:

\[\boxed{ \text{Actual GATE PYQs} \rightarrow \text{check mistakes} \rightarrow \text{revise only weak concept} \rightarrow \text{repeat} } \]

For Algorithms, this is the point where solving questions becomes more valuable than reading additional explanation.

USER

For placement preparation also give preparation content

ASSISTANT

Absolutely. Your **GATE Algorithms preparation and placement preparation should not be identical**.

For GATE, you mainly need to **analyze algorithms and solve conceptual/numerical questions**.

For placements, you need to be able to **write code, recognize patterns, explain your approach, optimize it, and handle follow-up questions in interviews**.

So from this point, we can add a separate track:

Ø=Ñ PLACEMENT PREPARATION — DSA + ALGORITHMS

I'll teach it in the same simple, step-by-step way, but focused on **coding rounds + technical interviews**.

---

1. WHAT YOU NEED FOR PLACEMENTS

For placements, prepare these six layers:

Layer 1 — Programming foundations

- variables
- loops
- functions
- arrays
- strings
- recursion
- time/space complexity

Layer 2 — Core data structures

- Arrays
- Strings
- Linked Lists
- Stacks
- Queues
- Hashing
- Trees
- BST
- Heaps
- Graphs

Layer 3 — Problem-solving patterns

- Two Pointers
- Sliding Window
- Prefix Sum

- Binary Search
- Hashing/Frequency counting
- Fast/Slow pointers
- Monotonic Stack
- Recursion/Backtracking
- Greedy
- Dynamic Programming
- BFS/DFS

Layer 4 — Important algorithms

- Sorting
- Binary Search
- BFS
- DFS
- Heap operations
- Topological Sort
- MST basics
- Dijkstra
- DP classics

Layer 5 — Interview explanation

For every coding problem, you must explain:

1. Brute-force solution
2. Why it is slow
3. Better approach
4. Data structure used
5. Time complexity
6. Space complexity
7. Edge cases

Layer 6 — Practice

Eventually:

\[\boxed{\text{Concept !' Pattern !' Coding !' Interview explanation}} \]

COMPLETE PLACEMENT DSA ROADMAP

I recommend this order.

Module	Topic	Placement Priority
1	Complexity + Coding Foundations	& & & & &
2	Arrays	& & & & &
3	Strings	& & & & &
4	Hashing	& & & & &

Module	Topic	Placement Priority
5	Two Pointers	& & & & &
6	Sliding Window	& & & & &
7	Prefix Sum	& & & & &
8	Binary Search	& & & & &
9	Sorting + Custom Sorting	& & & & &
10	Linked Lists	& & & & &
11	Stack	& & & & &
12	Queue + Deque	& & & & &
13	Recursion	& & & & &
14	Backtracking	& & & & &
15	Trees	& & & & &
16	BST	& & & & &
17	Heap / Priority Queue	& & & & &
18	Greedy	& & & & &
19	Graph BFS/DFS	& & & & &
20	Topological Sort	& & & & &
21	Shortest Path	& & & & &
22	Dynamic Programming Foundations	& & & & &
23	1D DP	& & & & &
24	2D DP / Knapsack / LCS	& & & & &
25	Mixed Interview Patterns	& & & & &
26	Mock Coding Interviews	& & & & &

VERY IMPORTANT: GATE VS PLACEMENT

Suppose question is Binary Search.

GATE may ask:

$$T(n)=T(n/2)+1$$

Find complexity.

Answer:

$$O(\log n)$$

Placement interview may ask:

> **Find the first position of `8` in:**

```text
[1,3,5,8,8,8,10]
```

Now you must actually write code.

And the interviewer may ask:

> **Can you do it in \(O(\log n)\)?**

Then:

> **What about last occurrence?**

Then:

> **What if the answer does not exist?**

Then:

> **Can Binary Search work on the answer rather than an array?**

That is placement preparation.

MODULE 1 — PLACEMENT CODING FOUNDATIONS

Before DSA, lock these concepts.

---

1. ALWAYSTHINK IN THIS ORDER

Whenever you see a coding question:

Step 1 — Understand input/output

Ask:

> **What exactly am I given?**

> **What exactly must I return?**

Step 2 — Try a small example manually

Example:

```text
Input:
[2,7,11,15]
target = 9
```

Manually:

\[2+7=9 \]

So answer indices:

```text
0,1
```

Step 3 — Think brute force

Try every pair:

```text
for i:
    for j:
```

Time:

\[O(n^2) \]

Step 4 — Look for optimization pattern

Possible clues:

- lookup !’ Hashing
- sorted !’ Binary Search / Two Pointers
- contiguous subarray !’ Sliding Window / Prefix Sum
- repeated subproblem !’ DP
- hierarchy !’ Tree
- connectivity !’ Graph/BFS/DFS

Step 5 — Write code

Step 6 — Check edge cases

Examples:

- empty array
- one element
- duplicate values
- negative values
- already sorted
- reverse sorted
- target absent

Step 7 — State complexity

Always say:

> **Time Complexity: \(O(...)\)**

> **Space Complexity: \(O(...)\)**

---

2. THE MOST IMPORTANT PLACEMENT QUESTION

Interviewers often ask:

> **Can you optimize it?**

Therefore train yourself to think:

\[\boxed{\text{Brute Force !' Better !' Optimal}} \]

Example:

Find whether duplicate exists

Brute force:

compare every pair.

\[O(n^2) \]

Better:

sort.

\[O(n\log n) \]

Then compare adjacent elements.

Optimal expected:

use HashSet.

\[O(n) \]

expected time.

This progression is extremely important during interviews.

---

3. ARRAY FOUNDATION

An array stores elements in contiguous/indexed positions.

Example:

```text
A = [10,20,30,40,50]
```

Indices:

```text
0   1   2   3   4
```

Access:

```text
A[2] = 30
```

Time:

$$ \boxed{O(1)} $$

---

4. ARRAY OPERATION COMPLEXITIES

Operation	Complexity
Access by index	$O(1)$
Search unsorted	$O(n)$
Append	usually amortized $O(1)$
Insert at beginning	$O(n)$
Delete at beginning	$O(n)$
Binary Search sorted	$O(\log n)$

These are placement interview basics.

---

5. FIRST IMPORTANT PLACEMENT PATTERN— TWO POINTERS

Suppose sorted array:

```text
[1,2,4,6,10]
```

Target:

$$ 8 $$

Need two numbers whose sum is 8.

Instead of checking every pair:

$$ O(n^2) $$

Use:

```text
left = 0
right = n-1
```

Calculate:

\[A[left]+A[right] \]

If sum too large:

```text
right--
```

If sum too small:

```text
left++
```

If equal:

> **found.**

Time:

\[\boxed{O(n)} \]

Space:

\[\boxed{O(1)} \]

---

6. WHY TWO POINTERS WORK

Sorted array:

```text
1 2 4 6 10
!'               !'
L       R
```

Suppose:

\[1+10=11 \]

target:

\[8 \]

11 is too large.

Increasing left would make sum even larger.

Therefore reduce right:

```text
1 2 4 6 10
!'          !'
```

```
L       R
```

Now:

\[1+6=7 \]

too small.

Increase left.

Eventually:

\[2+6=8 \]

found.

---

7. INTERVIEW VERSION

If interviewer asks:

> **Find pair with target sum in sorted array.**

Your explanation should sound like:

> **"The brute-force approach checks every pair and takes \(O(n^2)\). Because the input is sorted, I can use two pointers. I keep one pointer at each end. If the sum is too small I move the left pointer forward, and if it is too large I move the right pointer backward. This gives \(O(n)\) time and \(O(1)\) extra space."**

That explanation matters almost as much as code.

---

8. SECOND IMPORTANT PATTERN — HASHING

Suppose array is not sorted:

```text
[2,7,11,15]
```

target:

\[9 \]

While scanning `x`, look for:

\[target-x \]

For `2`:

\[9-2=7 \]

Remember 2.

When 7 appears:

\[9-7=2 \]

2 already exists.

Found pair.

Expected:

$$ \boxed{O(n)} $$

time.

---

9. WHEN TO THINK HASHING

Words such as:

- duplicate
- frequency
- occurrence
- pair sum
- lookup
- seen before
- count characters
- unique element

should trigger:

$$ \boxed{\text{HashMap / HashSet}} $$

---

10. THIRD IMPORTANT PATTERN— SLIDING WINDOW

Sliding Window is extremely important for placements.

Typical clue:

> **Find something about a contiguous subarray or substring.**

Example:

```
TEXT
[2,1,5,1,3,2]
```

Find maximum sum of subarray of size:

$$ k=3 $$

Brute force:

calculate every window separately.

Better:

First window:

$$ 2+1+5=8 $$

Next window:

remove 2, add 1:

$$ 8-2+1=7 $$

Next:

remove 1, add 3:

$$ 7-1+3=9 $$

Next:

remove 5, add 2:

$$ 9-5+2=6 $$

Maximum:

$$ \boxed{9} $$

Time:

$$ \boxed{O(n)} $$

---

11. SLIDING WINDOW SIGNAL

If question contains:

- subarray
- substring
- contiguous
- longest
- shortest
- maximum/minimum window

think:

$$ \boxed{\text{Sliding Window}} $$

Not always, but it is one of the first patterns to test.

---

12. FOURTH PATTERN— PREFIX SUM

Array:

```text
[2,4,1,5,3]
```

Prefix:

```text
[2,6,7,12,15]
```

Meaning:

$$ prefix[i] = A[0]+A[1]+\cdots+A[i] $$

Now range sum from index l to r:

$$ \boxed{ prefix[r]-prefix[l-1] } $$

instead of scanning the entire range.

13. PREFIX-SUM SIGNAL

Think Prefix Sum when:

- many range-sum queries
- subarray sums
- cumulative totals
- equilibrium/pivot index
- sum between indices

14. FIVE CORE ARRAY PATTERNS

For placements, memorize:

```text
Sorted + pair            !' Two Pointers
Contiguous range         !' Sliding Window
Repeated lookup          !' Hashing
Many range sums          !' Prefix Sum
Sorted search space      !' Binary Search
```

This is much more valuable than memorizing hundreds of individual solutions.

15. TOP ARRAY QUESTIONS YOU SHOULD SOLVE

Start with these:

Easy

1. Find maximum/minimum
2. Reverse Array
3. Second Largest
4. Remove Duplicates from Sorted Array
5. Move Zeroes
6. Missing Number
7. Contains Duplicate
8. Two Sum
9. Best Time to Buy and Sell Stock
10. Majority Element

Medium

11. Maximum Subarray
12. Product of Array Except Self
13. Rotate Array
14. Sort Colors
15. 3Sum
16. Longest Consecutive Sequence

17. Subarray Sum Equals K
18. Container With Most Water
19. Merge Intervals
20. Next Permutation

Don't memorize 20 codes.

Learn the patterns behind them.

---

16. TOP STRING QUESTIONS

Prepare:

1. Reverse String
2. Valid Palindrome
3. Anagram
4. First Unique Character
5. Longest Common Prefix
6. String Compression
7. Longest Substring Without Repeating Characters
8. Valid Parentheses
9. Group Anagrams
10. Minimum Window-type problems later

Patterns:

```text
Character frequency !' Hashing
Palindrome !' Two pointers
Substring !' Sliding window
Pattern matching !' String algorithms if required
```

---

17. LINKED LIST PLACEMENT QUESTIONS

Must know:

1. Reverse Linked List
2. Middle of Linked List
3. Detect Cycle
4. Merge Two Sorted Lists
5. Remove Nth Node from End
6. Palindrome Linked List
7. Intersection of Two Linked Lists

Main pattern:

\[\boxed{\text{Fast + Slow pointers}} \]

---

18. FAST/SLOW POINTER IDEA

Example:

```text
1  !'  2  !'  3  !'  4  !'  5
```

Slow moves:

```text
1 step
```

Fast moves:

```text
2 steps
```

When fast reaches end:

> **slow is near middle.**

Used for:

- middle node
- cycle detection
- palindrome techniques

19. STACK PLACEMENT QUESTIONS

Must know:

- Valid Parentheses
- Min Stack
- Next Greater Element
- Daily Temperatures
- Largest Rectangle basics
- Expression evaluation basics

Key pattern:

$$\boxed{\text{Monotonic Stack}}$$

for "next greater/smaller" type questions.

20. QUEUE / DEQUE QUESTIONS

Prepare:

- Queue implementation
- Circular Queue
- BFS

- Sliding Window Maximum
- First negative number in window
- Producer/consumer conceptual questions

21. TREES — VERY HIGH PRIORITY

You should be able to write without hesitation:

DFS traversals

```text
Preorder   !'  Root  Left  Right
Inorder    !'  Left  Root  Right
Postorder  !'  Left  Right  Root
```

BFS

```text
Level Order
```

using queue.

22. TOP TREE QUESTIONS

1. Maximum Depth
2. Same Tree
3. Invert Binary Tree
4. Level Order Traversal
5. Diameter
6. Balanced Binary Tree
7. Lowest Common Ancestor
8. Path Sum
9. Validate BST
10. Kth Smallest in BST
11. Construct tree from traversals

23. BST INTERVIEW RULE

For BST:

```text
left < root < right
```

and:

> **inorder traversal gives sorted order.**

Average search:

\[O(\log n) \]

in balanced BST.

Worst unbalanced:

\[O(n) \]

---

24. HEAPS / PRIORITY QUEUES

Think Heap when question asks:

- kth largest
- kth smallest
- top K
- repeatedly get minimum/maximum
- merge sorted streams
- task scheduling
- median stream

Operations:

```text
p e e k                !'   O ( 1 )
i n s e r t            !'   O ( l o g   n )
e x t r a c t   m i n / m a x  !'   O ( l o g   n )
```

---

25. TOP HEAP QUESTIONS

1. Kth Largest Element
2. Top K Frequent Elements
3. Merge K Sorted Lists
4. Find Median from Data Stream
5. K Closest Points
6. Task Scheduler

---

26. GRAPH QUESTIONS FOR PLACEMENTS

Know how to represent:

```text
Adjacency List
```

Then master:

BFS

DFS

Questions:

1. Number of Islands
2. Flood Fill
3. Connected Components
4. Clone Graph
5. Course Schedule
6. Detect Cycle
7. Topological Sort
8. Shortest path in unweighted graph
9. Dijkstra
10. Minimum Spanning Tree basics

---

27. NUMBER OF ISLANDS PATTERN

Grid:

```text
1 1 0
0 1 0
1 0 1
```

Whenever you find unvisited land:

```text
DFS/BFS
```

mark entire island.

Increment island count.

This is essentially:

\[\boxed{\text{Connected Components}} \]

in disguise.

Placement interviews love converting graphs into grids.

---

28. BACKTRACKING

Think backtracking when asked:

> **Generate all possible valid combinations/arrangements.**

Examples:

- Subsets
- Permutations
- Combination Sum
- N-Queens

- Sudoku
- Word Search

Core template:

```text
choose
explore
undo
```

Memory:

> **Choose !' Explore !' Undo**

---

29. DYNAMIC PROGRAMMING FOR PLACEMENTS

Don't jump immediately to hard DP.

Learn in this order:

Stage 1

Fibonacci

Stage 2

Climbing Stairs

Stage 3

House Robber

Stage 4

Min Cost Climbing Stairs

Stage 5

Coin Change

Stage 6

0/1 Knapsack

Stage 7

LCS

Stage 8

Grid DP

Stage 9

LIS

---

30. HOW TO RECOGNIZE DP IN INTERVIEW

Look for:

- maximum/minimum
- number of ways
- repeated decisions
- take/not take
- overlapping recursive calls
- optimal answer built from smaller answers

Then ask:

> **What is my state?**

---

31. THE MOST IMPORTANT INTERVIEW DP TEMPLATE

Example Climbing Stairs.

To reach stair i:

you could come from:

$$ i-1 $$

or:

$$ i-2 $$

Therefore:

$$ dp[i] = dp[i-1]+dp[i-2] $$

This is the thinking interviewers want to hear.

---

32. TOP 50 MUST-DO PLACEMENT QUESTIONS

If you have limited time, prioritize these categories.

Arrays / Hashing

1. Two Sum
2. Contains Duplicate
3. Best Time to Buy/Sell Stock
4. Maximum Subarray
5. Product Except Self
6. Majority Element
7. Move Zeroes
8. Missing Number
9. 3Sum
10. Longest Consecutive Sequence

Strings

11. Valid Palindrome

12. Valid Anagram
13. Longest Substring Without Repeating Characters
14. Longest Common Prefix
15. Group Anagrams

Binary Search

16. Binary Search
17. First/Last Position
18. Search Rotated Array
19. Find Minimum Rotated Array
20. Search on Answer problem

Linked List

21. Reverse List
22. Cycle Detection
23. Middle Node
24. Merge Two Sorted Lists
25. Remove Nth from End

Stack/Queue

26. Valid Parentheses
27. Min Stack
28. Next Greater Element
29. Daily Temperatures
30. Sliding Window Maximum

Trees

31. Maximum Depth
32. Level Order
33. Same Tree
34. Invert Tree
35. Diameter
36. Validate BST
37. LCA

Heap

38. Kth Largest
39. Top K Frequent
40. Merge K Lists

Graph

41. Number of Islands
42. Flood Fill
43. Connected Components
44. Course Schedule
45. BFS shortest path

DP

46. Climbing Stairs
47. House Robber
48. Coin Change
49. 0/1 Knapsack
50. LCS

This is a very strong base for fresher placement coding rounds.

---

33. DO NOT SOLVE QUESTIONS RANDOMLY

Use this approach:

For every problem, record:

```text
Problem:
Pattern:
Brute Force:
Optimal Approach:
Time:
Space:
Mistake:
One-line memory:
```

Example:

```text
Problem:
Two Sum

Pattern:
Hashing

Brute:
Two loops O(n²)

Optimal:
Store seen numbers

Time:
O(n) expected

Space:
O(n)

Memory:
 N e e d   x  !’   s e a r c h   t a r g e t - x .
```

---

34. INTERVIEW COMMUNICATION TEMPLATE

When the interviewer gives you a coding problem, don't immediately start silently coding.

Use this sequence:

1.

> **"Let me first confirm the input and expected output."**

2.

> **"A straightforward approach would be ..."**

3.

> **"That would take \(O(n^2)\). We can improve it using ..."**

4.

> **"The reason this data structure works is ..."**

5.

Write code.

6.

Walk through an example.

7.

Finish with:

> **"The final time complexity is ... and space complexity is ..."**

This makes you appear much more structured.

---

35. COMMON PLACEMENT INTERVIEW FOLLOW-UPS

Be prepared for:

> **Can you reduce the space?**

> **Can you improve the time?**

> **What if input is sorted?**

> **What if duplicates exist?**

> **What if input is empty?**

> **What happens for \(n=1\)?**

> **Can this overflow?**

> **Why did you choose this data structure?**

> **Is your solution stable/in-place?**

> **What is worst-case complexity?**

These follow-ups are often where candidates struggle.

---

36. LANGUAGE PREPARATION

For placements, pick **one primary coding language** and become comfortable with:

- arrays/lists
- strings
- sorting
- HashMap
- HashSet
- Stack
- Queue
- Priority Queue
- custom comparators
- recursion
- classes
- basic input/output

Do not keep switching languages during placement preparation.

---

37. WHAT YOU SHOULD BE ABLE TO CODE FROM MEMORY

Not entire problem answers—but these templates:

```text
Binary Search
BFS
DFS
Linked List reversal
Fast/Slow pointers
Sliding Window
Two Pointers
Frequency Map
Heap usage
Tree traversals
```

```
Backtracking
Basic 1D DP
```

If these templates are automatic, many interview questions become variations.

38. PLACEMENT COMPLEXITY CHEAT SHEET

Operation	Typical
Array access	\(O(1)\)
Linear scan	\(O(n)\)
Binary Search	\(O(\log n)\)
Hash lookup	expected \(O(1)\)
Sort	\(O(n\log n)\)
Heap insert/delete	\(O(\log n)\)
BFS/DFS	\(O(V+E)\)
Balanced BST search	\(O(\log n)\)
Linked-list search	\(O(n)\)

39. THE PLACEMENT PATTERN MAP

Make this one handwritten page:

```text
PAIR SUM
 s o r t e d     !'  t w o  p o i n t e r s
 u n s o r t e d  !'  h a s h i n g

CONTIGUOUS
 f i x e d  w i n d o w      !'  s l i d i n g  w i n d o w
 m a n y  r a n g e  s u m s  !'  p r e f i x  s u m

SEARCH
 s o r t e d / m o n o t o n i c  !'  b i n a r y  s e a r c h

LINKED LIST
 m i d d l e / c y c l e  !'  f a s t - s l o w

NEXT GREATER/SMALLER
!'  m o n o t o n i c  s t a c k

TOP K / MIN-MAX REPEATEDLY
!'  h e a p

TREE
 l e v e l s  !'  B F S
 p a t h s / s u b t r e e s  !'  D F S
```

```
GRAPH
 c o n n e c t i v i t y  !'  B F S / D F S
 d e p e n d e n c i e s  !'  t o p o l o g i c a l  s o r t
 w e i g h t e d  s h o r t e s t  !'  D i j k s t r a / B e l l m a n

ALL COMBINATIONS
!'  b a c k t r a c k i n g

REPEATED OPTIMAL SUBPROBLEMS
!'  D P
```

This page is extremely valuable for placements.

---

40. THREE LEVELS OF PLACEMENT READINESS

Level 1 — Concept

You know what Hashing is.

Not enough.

Level 2 — Coding

You can solve Two Sum using HashMap.

Better.

Level 3 — Interview

You can say:

> **brute force is \(O(n^2)\), hashing reduces expected time to \(O(n)\) with \(O(n)\) space, explain why, code it correctly, test duplicates and discuss alternatives.**

That's the target.

Ø=ÜÉPLACEMENT NOTES TO WRITE

Core patterns

```TEXT
Hashing
Two Pointers
Sliding Window
Prefix Sum
Binary Search
Fast/Slow Pointers
Monotonic Stack
Heap
BFS/DFS
Backtracking
Greedy
DP
```

Problem-solving order

```text
Understand
!’   E x a m p l e
!’   B r u t e   F o r c e
!’   O p t i m i z e
!’   C o d e
!’   T e s t
!’   C o m p l e x i t y
```

Interview order

```text
Explain
!’   C o d e
!’   D r y   r u n
!’   C o m p l e x i t y
!’   E d g e   c a s e s
```

&β30-SECOND PLACEMENT REVISION

1. Don't memorize solutions; recognize patterns.
2. Always know brute force first.
3. Then optimize.
4. Sorted pair !’ Two Pointers.
5. Fast lookup !’ Hashing.
6. Contiguous subarray !’ Sliding Window.
7. Range sums !’ Prefix Sum.
8. Sorted/monotonic !’ Binary Search.
9. Linked-list middle/cycle !’ Fast/Slow.
10. Next greater !’ Stack.
11. Top-K !’ Heap.
12. Tree/Graph traversal !’ BFS/DFS.
13. All combinations !’ Backtracking.
14. Repeated optimal states !’ DP.
15. Always state time + space complexity.
16. Always test edge cases.
17. Explain before coding.
18. Practice patterns, not random questions.

' Placement Preparation Track Started

From here, we can follow the same `NEXT` system we used for GATE.

Next Placement Module:

PLACEMENT MODULE 2 — ARRAYS: COMPLETE INTERVIEW PREPARATION

It will cover:

- array fundamentals,
- maximum/second maximum,
- reverse/rotate,
- duplicates,
- missing numbers,
- Two Sum,
- Move Zeroes,
- Best Time to Buy/Sell Stock,
- Kadane's Maximum Subarray,
- Majority Element,
- Dutch National Flag / Sort Colors,
- Two Pointers,
- Prefix Sum,
- Sliding Window,
- interview explanation,
- coding patterns,
- edge cases,
- and placement-style practice problems.

Type `NEXT` to continue with Placement Module 2.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 2 — ARRAYS: COMPLETE INTERVIEW PREPARATION

Placement Priority: & & & & &

Arrays are one of the **highest-priority placement topics** because many interview questions that look completely different are actually built on a small number of array patterns.

The goal is not:

> **“Memorize 50 array programs.”**

The goal is:

\[\boxed{\text{Recognize pattern !’ derive solution !’ explain !’ code !’ analyze}} \]

For code examples, I’ll use **simple Python-style code** because it keeps the algorithm visible. The same logic transfers directly to Java/C++.

---

1. WHAT IS AN ARRAY?

An array stores multiple values using indexes.

Example:

```text
A = [10, 20, 30, 40, 50]

index:
     0   1   2   3   4
```

Therefore:

```text
A[0] = 10
A[3] = 40
```

Random access:

\[\boxed{O(1)} \]

2. BASIC ARRAY COMPLEXITIES

Operation	Time
Access by index	\(O(1)\)
Update by index	\(O(1)\)
Search unsorted array	\(O(n)\)
Traverse array	\(O(n)\)
Insert at beginning	\(O(n)\)
Delete from beginning	\(O(n)\)
Binary Search on sorted array	\(O(\log n)\)
Sorting	usually \(O(n\log n)\)

3. THE MOST IMPORTANT ARRAY INTERVIEW PATTERNS

Before individual questions, memorize these.

Signal	Think
Need maximum/minimum	One traversal
Need pair in sorted array	Two Pointers
Need pair in unsorted array	Hashing

Signal	Think
Contiguous fixed-size region	Sliding Window
Many range sums	Prefix Sum
Sorted/monotonic condition	Binary Search
Maximum contiguous sum	Kadane
0,1,2 sorting	Dutch National Flag
Majority \(>n/2\)	Boyer-Moore Voting
Modify array without extra space	Two pointers/in-place

This table is more valuable than memorizing solutions.

PART A — BASIC ARRAY PROBLEMS

4. FIND MAXIMUM ELEMENT

Example:

```text
[7, 2, 9, 4, 5]
```

We need:

\[\boxed{9} \]

Approach:

Start:

```text
maximum = first element
```

Then scan remaining elements.

Whenever:

\[A[i]>maximum \]

update maximum.

Code

```python
def find_max(a):
    maximum = a[0]

    for x in a:
        if x > maximum:
            maximum = x

    return maximum
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(1)} $$

---

5. INTERVIEW EXPLANATION

Say:

> **“Since the array is unsorted, in the worst case I must inspect every element. I maintain the maximum seen so far and update it during one traversal. This takes $O(n)$ time and $O(1)$ extra space.”**

That's a clean placement answer.

---

6. FIND SECOND LARGEST

Example:

```text
[12, 35, 1, 10, 34, 1]
```

Largest:

$$ 35 $$

Second largest:

$$ 34 $$

---

7. BAD APPROACH

Sort:

```text
[1,1,10,12,34,35]
```

Then pick second largest distinct element.

Time:

$$ O(n\log n) $$

Works, but unnecessary.

---

8. OPTIMAL APPROACH

Maintain:

```text
largest
second
```

One traversal.

```python
def second_largest(a):
    largest = float("-inf")
    second = float("-inf")

    for x in a:
        if x > largest:
            second = largest
            largest = x

        elif largest > x > second:
            second = x

    return second
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(1)} $$

---

9. IMPORTANT DUPLICATE TRAP

Array:

```text
[10, 10, 9]
```

If question asks **second largest distinct element**:

Answer:

$$ 9 $$

not:

$$ 10 $$

Always clarify whether duplicates count.

---

10. REVERSE AN ARRAY

Example:

```text
[1,2,3,4,5]
```

Output:

```text
[5,4,3,2,1]
```

Use two pointers:

```text
left = 0
right = n-1
```

Swap.

Then:

```text
left++
right--
```

---

11. CODE

```python
def reverse_array(a):
    left = 0
    right = len(a) - 1

    while left < right:
        a[left], a[right] = a[right], a[left]

        left += 1
        right -= 1
```

Time:

\[\boxed{O(n)} \]

Extra space:

\[\boxed{O(1)} \]

This is an **in-place** solution.

---

12. TWO POINTERS — CORE IDEA

Two pointers means maintaining two indexes that move according to some condition.

Common forms:

Opposite ends

```text
 L  !'                    !•   R
```

Used for:

- reverse array

- palindrome
- pair sum in sorted array
- container problems

Same direction

```text
s l o w  !'
f a s t  !'
```

Used for:

- remove duplicates
- move zeroes
- partitioning

PART B — REMOVE DUPLICATES

---

13. REMOVE DUPLICATES FROM SORTED ARRAY

Input:

```text
[1,1,2,2,2,3,4,4]
```

Unique values:

```text
[1,2,3,4]
```

Because input is sorted, duplicates are adjacent.

Use two pointers.

---

14. IDEA

`write` indicates where next unique value should go.

```text
[1,1,2,2,2,3,4,4]
  !'
write
```

Scan forward.

Whenever current value differs from previous unique value:

> **write it.**

---

15. CODE

```python
def remove_duplicates(a):
    if not a:
        return 0

    write = 1

    for i in range(1, len(a)):
        if a[i] != a[write - 1]:
            a[write] = a[i]
            write += 1

    return write
```

For:

```text
[1,1,2,2,3]
```

the first `write` positions become:

```text
[1,2,3,...]
```

Return:

\[3 \]

Time:

\[O(n) \]

Space:

\[O(1) \]

---

16. PLACEMENT SIGNAL

If array is:

\[\boxed{\text{sorted}} \]

and question involves:

- duplicates
- pairs
- unique values

always check whether **Two Pointers** can help.

PART C — MOVE ZEROES

17. PROBLEM

Input:

```text
[0,1,0,3,12]
```

Output:

```text
[1,3,12,0,0]
```

Requirements often say:

- maintain order of nonzero elements,
- modify in-place.

18. OPTIMAL IDEA

Use `write`.

Whenever nonzero element appears:

> **place it at next valid position.**

Example:

```text
[0,1,0,3,12]
```

Nonzero sequence:

```text
1,3,12
```

Then fill remaining positions with zeros.

19. CODE

```python
def move_zeroes(a):
    write = 0

    for x in a:
        if x != 0:
            a[write] = x
            write += 1

    while write < len(a):
        a[write] = 0
        write += 1
```

Time:

$$\boxed{O(n)}$$

Space:

$$ \boxed{O(1)} $$

---

20. INTERVIEW FOLLOW-UP

Interviewer may ask:

> **Can you do it in-place?**

Yes.

> **Can you preserve order?**

Yes.

> **Can you avoid creating another array?**

Yes.

Those requirements lead directly toward two-pointer compaction.

PART D — MISSING NUMBER

21. PROBLEM

Array contains distinct numbers from:

$$ 0\text{ to }n $$

with one number missing.

Example:

```text
[3,0,1]
```

Numbers should be:

```text
0,1,2,3
```

Missing:

$$ \boxed{2} $$

---

22. SUM APPROACH

Expected total:

$$ \frac{n(n+1)}2 $$

Actual:

$$ \sum A[i] $$

Missing:

$$ \boxed{ \frac{n(n+1)}2-\sum A[i] } $$

Code:

```python
def missing_number(a):
    n = len(a)

    expected = n * (n + 1) // 2
    actual = sum(a)

    return expected - actual
```

Time:

$$ O(n) $$

Space:

$$ O(1) $$

---

23. XOR APPROACH

Useful interview follow-up.

Because:

$$ x\oplus x=0 $$

and:

$$ x\oplus0=x $$

XOR all:

```text
0...n
```

with all array elements.

Everything cancels except missing number.

Time:

$$ O(n) $$

Space:

$$ O(1) $$

This avoids arithmetic overflow concerns in fixed-width languages.

PART E — CONTAINS DUPLICATE

24. PROBLEM

Input:

```text
[1,2,3,1]
```

Return:

```text
True
```

because `1` repeats.

---

25. BRUTE FORCE

Compare every pair.

\[O(n^2) \]

---

26. SORTING

Sort and compare adjacent elements.

\[O(n\log n) \]

---

27. HASH SET — EXPECTED OPTIMAL

Scan each number.

If already seen:

> **duplicate.**

Otherwise add it.

```python
def contains_duplicate(a):
    seen = set()

    for x in a:
        if x in seen:
            return True

        seen.add(x)

    return False
```

Expected time:

\[\boxed{O(n)} \]

Space:

$$ \boxed{O(n)} $$

28. INTERVIEW TRADE-OFF

Hashing:

$$ O(n)\text{ expected time} $$

but:

$$ O(n)\text{ extra space} $$

Sorting:

$$ O(n\log n) $$

but can sometimes reduce extra space depending on sorting method/input constraints.

Always be able to discuss this.

PART F — TWO SUM

29. PROBLEM

```text
nums = [2,7,11,15]
target = 9
```

Return indices:

```text
[0,1]
```

because:

$$ 2+7=9 $$

30. BRUTE FORCE

Check every pair:

```python
for i:
    for j:
```

Time:

$$ O(n^2) $$

Space:

$$ O(1) $$

31. HASHING IDEA

For current number:

$$ x $$

needed value is:

$$ target-x $$

Example:

Current:

$$ 7 $$

Need:

$$ 9-7=2 $$

If 2 has already been seen:

> **solution found.**

---

32. CODE

```python
def two_sum(nums, target):
    seen = {}

    for i in range(len(nums)):
        need = target - nums[i]

        if need in seen:
            return [seen[need], i]

        seen[nums[i]] = i
```

Expected time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(n)} $$

---

33. TWO SUM — SORTED ARRAY VERSION

If array is sorted:

```text
[1,2,4,6,10]
target = 8
```

Use:

```text
left
right
```

If sum:

`< target`

move:

```text
left++
```

`> target`

move:

```text
right--
```

`== target`

found.

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(1)} $$

---

34. CRITICAL INTERVIEW RULE

Two Sum unsorted

Think:

$$ \boxed{\text{Hashing}} $$

Two Sum sorted

Think:

$$ \boxed{\text{Two Pointers}} $$

PART G — BEST TIME TO BUY AND SELL STOCK

35. PROBLEM

Prices:

```text
[7,1,5,3,6,4]
```

Buy once.

Sell once after buying.

Maximum profit?

Buy:

\[1 \]

Sell:

\[6 \]

Profit:

\[\boxed{5} \]

---

36. BRUTE FORCE

Try every:

```text
buy day
sell day
```

\[O(n^2) \]

---

37. OPTIMAL OBSERVATION

When at current price, the best buy price is:

> **minimum price seen before today.**

Maintain:

```text
min_price
max_profit
```

---

38. TRACE

```text
prices = [7,1,5,3,6,4]
```

Start:

```text
min = 7
profit = 0
```

At 1:

```text
min = 1
```

At 5:

\[5-1=4 \]

profit = 4.

At 6:

\[6-1=5 \]

profit = 5.

Answer:

\[\boxed{5} \]

---

39. CODE

```python
def max_profit(prices):
    min_price = prices[0]
    best = 0

    for price in prices:
        min_price = min(min_price, price)
        best = max(best, price - min_price)

    return best
```

Time:

\[\boxed{O(n)} \]

Space:

\[\boxed{O(1)} \]

---

40. PATTERN

This is essentially:

> **maintain best historical information while scanning.**

Very common interview technique.

PART H — MAXIMUM SUBARRAY / KADANE'S ALGORITHM

41. PROBLEM

Find maximum sum of a **contiguous subarray**.

Example:

```text
[-2,1,-3,4,-1,2,1,-5,4]
```

Best subarray:

```text
[4,-1,2,1]
```

Sum:

$$ \boxed{6} $$

---

42. BRUTE FORCE

Try all subarrays.

Naive:

$$ O(n^3) $$

with repeated summation.

Improved brute:

$$ O(n^2) $$

using running sum.

But we can do:

$$ \boxed{O(n)} $$

---

43. KADANE'S IDEA

At every index ask:

> **Should I continue my previous subarray, or start fresh from this element?**

Thus:

$$ current= \max(A[i],current+A[i]) $$

Then:

$$ best=\max(best,current) $$

---

44. CODE

```python
def max_subarray(a):
    current = a[0]
    best = a[0]
```

```
    for i in range(1, len(a)):
        current = max(a[i], current + a[i])
        best = max(best, current)

    return best
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(1)} $$

---

45. KADANE MEMORY TRICK

> **Continue or Restart.**

At every element:

```TEXT
continue old sum
OR
start from current element
```

Pick maximum.

---

46. IMPORTANT ALL-NEGATIVE TRAP

Array:

```TEXT
[-5,-2,-8]
```

Answer:

$$ \boxed{-2} $$

not:

$$ 0 $$

if the subarray must contain at least one element.

That's why initialization with:

```TEXT
current = a[0]
best = a[0]
```

is safer.

PART I — MAJORITY ELEMENT

47. PROBLEM

Majority element appears more than:

$$ \frac n2 $$

times.

Example:

```
TEXT
[2,2,1,1,1,2,2]
```

Majority:

$$ \boxed{2} $$

---

48. HASHING SOLUTION

Count frequencies.

Time:

$$ O(n) $$

Space:

$$ O(n) $$

But interviewers may ask for:

$$ O(1) $$

space.

---

49. BOYER-MOORE VOTING

Maintain:

```
TEXT
candidate
count
```

Rules:

If:

$$ count=0 $$

choose current element as candidate.

If current equals candidate:

```text
count++
```

otherwise:

```text
count--
```

---

50. INTUITION

Majority element appears more than all other elements combined.

Pair each majority occurrence against a different element.

After cancellation:

> **majority survives.**

---

51. CODE

```python
def majority_element(a):
    candidate = None
    count = 0

    for x in a:
        if count == 0:
            candidate = x

        if x == candidate:
            count += 1
        else:
            count -= 1

    return candidate
```

Time:

\[\boxed{O(n)} \]

Space:

\[\boxed{O(1)} \]

---

52. IMPORTANT ASSUMPTION

This simple version assumes a majority element is guaranteed.

If not guaranteed:

> **perform another pass to verify candidate frequency \(>n/2\).**

PART J — SORT COLORS / DUTCH NATIONAL FLAG

53. PROBLEM

Array contains only:

```text
0, 1, 2
```

Example:

```text
[2,0,2,1,1,0]
```

Output:

```text
[0,0,1,1,2,2]
```

Could count frequencies.

But famous in-place solution:

\[\boxed{\text{Dutch National Flag}} \]

54. THREE POINTERS

Maintain:

```text
low
mid
high
```

Invariant:

```text
 0   . . .   l o w - 1            !'   0 s
 l o w   . . .   m i d - 1        !'   1 s
 m i d   . . .   h i g h          !'   u n k n o w n
 h i g h + 1   . . .   e n d      !'   2 s
```

55. RULES

If:

\[A[mid]=0 \]

swap with `low`.

Then:

```text
low++
mid++
```

If:

$$ A[mid]=1 $$

just:

```text
mid++
```

If:

$$ A[mid]=2 $$

swap with `high`.

Then:

```text
high--
```

Do **not** immediately increment `mid`, because swapped element from right has not yet been examined.

---

56. CODE

```python
def sort_colors(a):
    low = 0
    mid = 0
    high = len(a) - 1

    while mid <= high:

        if a[mid] == 0:
            a[low], a[mid] = a[mid], a[low]
            low += 1
            mid += 1

        elif a[mid] == 1:
            mid += 1

        else:
            a[mid], a[high] = a[high], a[mid]
            high -= 1
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(1)} $$

PART K — ROTATE ARRAY

57. PROBLEM

Rotate right by:

$$ k=2 $$

Input:

```text
[1,2,3,4,5]
```

Output:

```text
[4,5,1,2,3]
```

---

58. IMPORTANT FIRST STEP

If:

$$ k>n $$

use:

$$ \boxed{k=k\bmod n} $$

Example:

$$ n=5,\quad k=7 $$

Equivalent rotation:

$$ 7\bmod5=2 $$

---

59. REVERSAL METHOD

To rotate right by k:

1. Reverse entire array.
2. Reverse first k.
3. Reverse remaining $n-k$.

Example:

```text
1 2 3 4 5
```

Reverse:

```text
5 4 3 2 1
```

Reverse first 2:

```
TEXT
4 5 3 2 1
```

Reverse remaining:

```
TEXT
4 5 1 2 3
```

Done.

Time:

$$ O(n) $$

Space:

$$ O(1) $$

PART L — PREFIX SUM

60. PROBLEM

Array:

```
TEXT
[2,4,1,5,3]
```

We may receive many queries:

> **Sum from index l to r.**

Without preprocessing each query:

$$ O(n) $$

could be costly.

---

61. PREFIX ARRAY

Define:

$$ prefix[i] = A[0]+\cdots+A[i] $$

So:

```
TEXT
A      = [2,4,1,5,3]
prefix = [2,6,7,12,15]
```

---

62. RANGE SUM

For:

$$ l>0 $$

$$ \boxed{ sum(l,r)=prefix[r]-prefix[l-1] } $$

Example:

indices 1 to 3:

```
TEXT
4 + 1 + 5 = 10
```

Using prefix:

$$ prefix[3]-prefix[0] $$

$$ 12-2=10 $$

Query:

$$ \boxed{O(1)} $$

after $O(n)$ preprocessing.

---

63. PREFIX SUM SIGNALS

Think Prefix Sum when question says:

- many range queries,
- sum between indices,
- cumulative sum,
- subarray sum,
- pivot/equilibrium index.

PART M — SLIDING WINDOW

64. FIXED WINDOW PROBLEM

Find maximum sum of any subarray of length $k=3$:

```
TEXT
[2,1,5,1,3,2]
```

Windows:

```
TEXT
 [ 2 , 1 , 5 ]  !'   8
 [ 1 , 5 , 1 ]  !'   7
 [ 5 , 1 , 3 ]  !'   9
 [ 1 , 3 , 2 ]  !'   6
```

Answer:

$$ \boxed{9} $$

65. BRUTE FORCE

Calculate each window from scratch.

$$ O(nk) $$

66. SLIDE IN O(1)

When moving one position:

```TEXT
remove left old value
add new right value
```

Example:

$$ 8-2+1=7 $$

67. CODE

```PYTHON
def max_window_sum(a, k):
    window = sum(a[:k])
    best = window

    for i in range(k, len(a)):
        window += a[i]
        window -= a[i - k]

        best = max(best, window)

    return best
```

Time:

$$ \boxed{O(n)} $$

68. SLIDING WINDOW SIGNALS

Look for:

$$ \boxed{\text{contiguous}} $$

plus:

- maximum
- minimum
- longest
- shortest
- fixed size

- substring/subarray

Then consider sliding window.

PART N — MERGE INTERVALS

69. PROBLEM

Input:

```TEXT
[[1,3], [2,6], [8,10], [15,18]]
```

Output:

```TEXT
[[1,6], [8,10], [15,18]]
```

Because:

```TEXT
[1,3]
[2,6]
```

overlap.

---

70. IDEA

1. Sort intervals by start.
2. Maintain last merged interval.
3. If current start:

\[\le \]

previous end:

> **merge.**

Otherwise:

> **start new interval.**

Complexity:

Sorting dominates:

\[\boxed{O(n\log n)} \]

PART O — PRODUCT OF ARRAY EXCEPT SELF

71. PROBLEM

Input:

```text
[1,2,3,4]
```

Output:

```text
[24,12,8,6]
```

Because:

```text
24 = 2×3×4
12 = 1×3×4
...
```

Often interviewer says:

> **Do not use division.**

---

72. PREFIX × SUFFIX IDEA

For each index:

\[answer[i] = (\text{product left of i}) \times (\text{product right of i}) \]

Use one pass for prefix product and one reverse pass for suffix product.

Time:

\[\boxed{O(n)} \]

Output array aside, extra space can be:

\[\boxed{O(1)} \]

PART P — 3SUM

73. PROBLEM

Find unique triplets whose sum is zero.

Example:

```text
[-1,0,1,2,-1,-4]
```

Output:

```text
[-1,-1,2]
[-1,0,1]
```

74. PATTERN

Sort first.

For each index i:

> **solve Two Sum on remaining portion using two pointers.**

Complexity:

$$\boxed{O(n^2)}$$

instead of brute-force:

$$O(n^3)$$

Important challenge:

> **avoid duplicate triplets.**

PART Q — LONGEST CONSECUTIVE SEQUENCE

75. PROBLEM

Input:

```text
[100,4,200,1,3,2]
```

Longest consecutive sequence:

```text
1,2,3,4
```

Length:

$$\boxed{4}$$

76. HASH-SET INSIGHT

Put all numbers in a set.

Only start counting from x if:

$$x-1$$

does not exist.

That means \(x\) is the beginning of a sequence.

For `1`:

```
TEXT
0 not present
```

so count:

```
TEXT
1,2,3,4
```

Time expected:

\[\boxed{O(n)} \]

---

77. WHY NOT START FROM EVERY NUMBER?

If you start scanning from every number, repeated work can cause:

\[O(n^2) \]

The condition:

\[x-1\notin set \]

ensures only sequence beginnings are expanded.

This is a very interview-style optimization insight.

PART R — ARRAY PATTERN CLASSIFICATION

78. IF YOU SEE THIS, THINK THIS

Sorted array + pair target

\[\boxed{\text{Two Pointers}} \]

Unsorted array + pair target

\[\boxed{\text{Hashing}} \]

Duplicate detection

\[\boxed{\text{HashSet}} \]

Contiguous max sum

\[\boxed{\text{Kadane}} \]

Fixed-size contiguous window

\[\boxed{\text{Sliding Window}} \]

Many range sums

\[\boxed{\text{Prefix Sum}} \]

Majority \(>n/2\)

\[\boxed{\text{Boyer-Moore}} \]

0,1,2 sorting

\[\boxed{\text{Dutch National Flag}} \]

Merge overlapping ranges

\[\boxed{\text{Sort + scan}} \]

Top K values

Usually:

\[\boxed{\text{Heap}} \]

PART S — COMMON ARRAY INTERVIEW TRAPS

79. TRAP 1 — EMPTY ARRAY

Before using:

```text
a[0]
```

consider whether input may be empty.

---

80. TRAP 2 — DUPLICATES

Questions may ask:

- second largest,
- second largest **distinct**,
- pair values,
- pair indices.

Read precisely.

---

81. TRAP 3 — INTEGER OVERFLOW

In Java/C++:

```text
int
```

may overflow for large:

- sums,
- products,
- \(n(n+1)/2\).

Use larger type such as `long long` / `long` when needed.

82. TRAP 4 — SORTING CHANGES INDICES

For Two Sum, sorting may make pair finding easy, but if original indices are required:

> **sorting loses direct original index information unless preserved.**

Hashing is often better.

83. TRAP 5 — CONTIGUOUS VS SUBSEQUENCE

Subarray:

$$ \boxed{\text{contiguous}} $$

Subsequence:

> **elements may be skipped.**

Kadane solves maximum **subarray**, not arbitrary subsequence.

84. TRAP 6 — ALL NEGATIVE KADANE

Do not initialize answer to zero unless empty subarray is allowed.

85. TRAP 7 — TWO POINTERS NEED STRUCTURE

For target sum, opposite-end Two Pointers usually relies on:

$$ \boxed{\text{sorted array}} $$

Don't blindly use it on unsorted input.

PART T — HOW TO ANSWER AN ARRAY QUESTION IN INTERVIEW

86. STANDARD SCRIPT

Suppose interviewer asks Two Sum.

A strong answer:

> **“The brute-force solution checks every pair, which is $O(n^2)$. Since I need repeated lookup of the complementary value, I can use a hash map. While scanning each value x, I check whether `target - x` has already appeared. If so, I have the required pair; otherwise I store the current value and index. This gives expected $O(n)$ time and $O(n)$ space.”**

Then code.

Then dry-run.

Then discuss edge cases.

That structure should become automatic.

87. INTERVIEW CHECKLIST

For every problem ask:

```text
1. Input?
2. Output?
3. Constraints?
4. Sorted?
5. Duplicates?
6. Need indices or values?
7. Can modify input?
8. Extra space allowed?
9. Brute force?
10. Better pattern?
11. Complexity?
12. Edge cases?
```

PART U — MUST-DO ARRAY QUESTIONS

88. LEVEL 1 — FOUNDATION

You should be able to solve these without help:

1. Find Largest Element
2. Find Second Largest
3. Reverse Array
4. Check if Array Sorted
5. Remove Duplicates from Sorted Array
6. Move Zeroes
7. Missing Number
8. Contains Duplicate
9. Linear Search
10. Rotate Array

89. LEVEL 2 — CORE PLACEMENT

11. Two Sum
12. Best Time to Buy/Sell Stock
13. Maximum Subarray
14. Majority Element
15. Sort Colors
16. Maximum Consecutive Ones
17. Merge Sorted Arrays

18. Intersection of Two Arrays
19. Prefix Range Sum
20. Maximum Sum Window of Size K

90. LEVEL 3 — INTERVIEW PATTERNS

21. 3Sum
22. Product Except Self
23. Longest Consecutive Sequence
24. Merge Intervals
25. Subarray Sum Equals K
26. Container With Most Water
27. Find Duplicate Number
28. Next Permutation
29. Search in Rotated Sorted Array
30. Sliding Window Maximum

Don't try to memorize all 30 solutions.

Learn why each belongs to a pattern.

PART V — PLACEMENT PRACTICE SET

Try before reading solutions.

Q1 — MCQ

For an unsorted array, finding the maximum element requires worst-case:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n\log n)$

Q2 — NAT

What is the second largest **distinct** element?

```text
[10,5,10,8,9]
```

Q3 — MCQ

For a sorted array and a target pair sum, the best $O(1)$-extra-space pattern is:

A. Hashing B. Two Pointers C. DP D. BFS

Q4 — MCQ

Which pattern is most directly suggested by:

> **“maximum sum of a contiguous subarray”?**

A. BFS B. Kadane's Algorithm C. Binary Search D. Heap

Q5 — NAT

For:

```text
[7,1,5,3,6,4]
```

what is the maximum profit from one buy followed by one sell?

Q6 — MSQ

Which statements are true?

A. HashSet can detect duplicates in expected \(O(n)\).

B. Kadane's Algorithm solves maximum contiguous subarray sum.

C. Two Pointers for pair sum normally requires sorted order.

D. Binary Search works on every unsorted array.

Q7 — NAT

For:

```text
[2,1,5,1,3,2]
```

what is the maximum sum of any subarray of size 3?

Q8 — MCQ

Majority Element with \(O(1)\) extra space is classically solved by:

A. Prim B. Boyer-Moore Voting C. Bellman-Ford D. Merge Sort

Q9 — MCQ

Array contains only `0`, `1`, and `2`.

The classic \(O(n)\), \(O(1)\)-space algorithm is:

A. Floyd-Warshall B. Dutch National Flag C. Huffman D. KMP

Q10 — MSQ

Which problems strongly suggest hashing?

A. Contains Duplicate

B. Two Sum on an unsorted array

C. Frequency counting

D. BFS traversal

Q11 — NAT

Reverse-sorted array:

```text
[5,4,3,2,1]
```

How many inversions?

Q12 — MCQ

If many range-sum queries are performed on a fixed array, the best basic preprocessing technique is:

A. DFS B. Prefix Sum C. Heap D. Selection Sort

PRACTICE SOLUTIONS

Q1

Need inspect every element in worst case.

$$\boxed{C}$$

Q2

Distinct elements:

```text
10,9,8,5
```

Second largest:

$$\boxed{9}$$

Q3

Sorted pair-sum:

$$\boxed{\text{Two Pointers}}$$

Answer:

$$\boxed{B}$$

Q4

Maximum contiguous subarray:

$$\boxed{\text{Kadane}}$$

Answer:

$$\boxed{B}$$

Q5

Buy:

$$1$$

sell:

$$6$$

profit:

$$\boxed{5}$$

Q6

A true.

B true.

C true.

D false.

Answer:

$$\boxed{A,B,C}$$

Q7

Windows:

$$2+1+5=8$$

$$1+5+1=7$$

$$5+1+3=9$$

$$1+3+2=6$$

Maximum:

$$\boxed{9}$$

Q8

$$\boxed{B}$$

Q9

$$\boxed{B}$$

Q10

A, B and C.

$$\boxed{A,B,C}$$

Q11

Maximum inversions for 5 elements:

$$\frac{5(4)}{2} = \boxed{10}$$

Q12

$$\boxed{B}$$

Prefix Sum.

Ø=ÜÝPLACEMENT NOTES TO WRITE

Array patterns

```TEXT
Sorted pair                !' Two Pointers
Unsorted pair              !' Hashing
Duplicates                 !' HashSet
Max contiguous sum         !' Kadane
Fixed window               !' Sliding Window
Many range sums            !' Prefix Sum
Majority > n/2             !' Boyer-Moore
0/1/2 array                !' Dutch National Flag
Intervals                  !' Sort + Merge
```

MUST REMEMBER COMPLEXITIES

```TEXT
Array access            O(1)
Linear traversal        O(n)
Two Pointers            O(n)
Hash lookup             Expected O(1)
```

```
Two Sum with HashMap      Expected O(n)
Sliding Window            O(n)
Prefix creation           O(n)
Prefix range query        O(1)
Kadane                    O(n)
Boyer-Moore               O(n)
Dutch National Flag       O(n)
Merge Intervals           O(n log n)
3Sum                      O(n²)
```

&β30-SECOND ARRAY REVISION

1. Arrays provide \(O(1)\) indexed access.
2. Unsorted maximum requires \(O(n)\).
3. Two pointers are essential for sorted arrays.
4. Unsorted lookup problems often suggest hashing.
5. Remove Duplicates !’ slow/write pointer.
6. Move Zeroes !’ compaction pointer.
7. Missing Number !’ sum or XOR.
8. Two Sum unsorted !’ HashMap.
9. Two Sum sorted !’ Two Pointers.
10. Stock !’ minimum-so-far.
11. Max Subarray !’ Kadane.
12. Kadane = continue or restart.
13. Majority !’ Boyer-Moore.
14. 0/1/2 sort !’ Dutch National Flag.
15. Range queries !’ Prefix Sum.
16. Contiguous fixed range !’ Sliding Window.
17. Rotation !’ reversal trick.
18. Merge Intervals !’ sort + scan.
19. Longest Consecutive !’ HashSet + sequence start.
20. Always explain brute !’ optimal !’ complexity.

' Placement Module 2 Complete

Next Placement Module:

MODULE 3 — STRINGS + HASHING + TWO POINTERS

We’ll cover:

- string fundamentals,
- character frequency,
- palindrome,
- anagram,
- reverse words,

- longest common prefix,
- first unique character,
- valid parentheses connection,
- longest substring without repeating characters,
- sliding window on strings,
- Group Anagrams,
- string interview traps,
- and the most important placement string problems.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 3 — STRINGS + HASHING + TWO POINTERS

Placement Priority: & & & & &

Strings are among the most common coding-round topics because they combine:

- arrays
- hashing
- two pointers
- sliding window
- stacks
- frequency counting
- pattern recognition

The goal is not to memorize dozens of string programs.

The goal is:

\[\boxed{\text{Recognize the pattern behind the string problem}} \]

---

1. WHAT IS A STRING?

A string is a sequence of characters.

Example:

```
TEXT
"HELLO"
```

Indexes:

```text
H  E  L  L  O
0  1  2  3  4
```

Length:

\[5 \]

2. IMPORTANT STRING OPERATIONS

Typical complexities depend on language and representation, but conceptually:

Operation	Typical
Access character	\(O(1)\)
Traverse string	\(O(n)\)
Compare strings	up to \(O(n)\)
Reverse	\(O(n)\)
Search naive pattern	up to \(O(nm)\)
Character frequency count	\(O(n)\)

3. IMMUTABLE STRING TRAP

In languages such as Python and Java:

> **Strings are immutable.**

That means repeated concatenation like:

```text
result = result + character
```

inside a large loop can be inefficient.

For interviews:

- Java !' use `StringBuilder`
- Python !' collect pieces and `"".join(...)`

This matters especially for large inputs.

4. STRING PATTERN MASTER TABLE

Memorize this:

Problem signal	Think
Palindrome	Two Pointers
Character counts	Hashing/Frequency Array
Anagram	Frequency comparison
Unique/repeated character	Hashing
Longest substring	Sliding Window
Reverse string	Two Pointers
Parentheses/brackets	Stack
Group words by same letters	Hashing + canonical signature
Prefix shared by strings	Scan characters
Pattern search	String matching algorithms when required

PART A — REVERSE STRING

5. PROBLEM

Input:

```text
"hello"
```

Output:

```text
"olleh"
```

If represented as character array:

```text
['h','e','l','l','o']
```

use two pointers.

---

6. TWO-POINTER APPROACH

```text
left = 0
right = n - 1
```

Swap:

```text
 s [ l e f t ]  !”   s [ r i g h t ]
```

then:

```text
left++
right--
```

---

7. CODE

```python
def reverse_string(s):
    s = list(s)

    left = 0
    right = len(s) - 1

    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

    return "".join(s)
```

Time:

\[\boxed{O(n)} \]

Extra space in this Python form:

\[O(n) \]

because strings are immutable and converted to list.

If interviewer gives mutable char array:

\[\boxed{O(1)} \]

extra space.

PART B — PALINDROME

8. WHAT IS A PALINDROME?

A string that reads the same forward and backward.

Examples:

```text
"madam"
"racecar"
"level"
```

---

9. TWO POINTERS

For:

```text
racecar
```

Compare:

```text
r == r
a == a
c == c
```

Move inward.

If any mismatch:

\[\boxed{\text{not palindrome}} \]

---

10. CODE

```python
def is_palindrome(s):
    left = 0
    right = len(s) - 1

    while left < right:
        if s[left] != s[right]:
            return False

        left += 1
        right -= 1

    return True
```

Time:

\[\boxed{O(n)} \]

Space:

\[\boxed{O(1)} \]

---

11. VALID PALINDROME INTERVIEW VERSION

Sometimes question says:

> **Ignore spaces, punctuation and case.**

Example:

```text
"A man, a plan, a canal: Panama"
```

Normalize or skip non-alphanumeric characters while moving pointers.

Concept:

```text
move left until alphanumeric
move right until alphanumeric
compare lowercase versions
```

Still:

$$ O(n) $$

---

12. PALINDROME MEMORY RULE

> **Mirror characters must match.**

Therefore:

$$ \boxed{\text{Two Pointers}} $$

PART C — VALID ANAGRAM

13. PROBLEM

Two strings are anagrams if they contain the same characters with same frequencies.

Example:

```text
"listen"
"silent"
```

Yes.

---

14. SORTING APPROACH

Sort both strings.

```text
eilnst
eilnst
```

Compare.

Time:

$$ \boxed{O(n\log n)} $$

15. HASHING/FREQUENCY APPROACH

Count characters.

For:

```text
anagram
nagaram
```

frequency of every character must match.

16. CODE

```python
def is_anagram(s, t):
    if len(s) != len(t):
        return False

    freq = {}

    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1

    for ch in t:
        if ch not in freq:
            return False

        freq[ch] -= 1

        if freq[ch] < 0:
            return False

    return True
```

Expected time:

\[\boxed{O(n)} \]

Space:

\[O(k) \]

where \(k\) is number of distinct characters.

For lowercase English letters only:

\[k\le26 \]

so effectively constant auxiliary size.

17. INTERVIEW FOLLOW-UP

If interviewer says:

> **Only lowercase `a-z`.**

You can use:

```
TEXT
freq[26]
```

instead of HashMap.

Index:

\[ord(ch)-ord('a') \]

Time:

\[O(n) \]

Space:

\[O(1) \]

with fixed alphabet.

PART D — CHARACTER FREQUENCY

18. FREQUENCY COUNTING

Input:

```
TEXT
"banana"
```

Counts:

```
TEXT
 b  !'   1
 a  !'   3
 n  !'   2
```

HashMap pattern:

```
PYTHON
freq = {}

for ch in s:
    freq[ch] = freq.get(ch, 0) + 1
```

This simple pattern solves many interview problems.

---

19. WHEN TO THINK FREQUENCY MAP

Trigger words:

- count
- occurrence
- duplicate

- unique
- anagram
- most frequent
- frequency
- character count

Think:

$$ \boxed{\text{HashMap / frequency array}} $$

PART E — FIRST UNIQUE CHARACTER

20. PROBLEM

Input:

```text
"leetcode"
```

First unique character:

```text
'l'
```

index:

$$ \boxed{0} $$

For:

```text
"loveleetcode"
```

first unique:

```text
'v'
```

index:

$$ 2 $$

---

21. APPROACH

Pass 1:

> **Count frequency.**

Pass 2:

> **Return first character with frequency 1.**

---

22. CODE

```python
def first_unique_char(s):
    freq = {}

    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1

    for i in range(len(s)):
        if freq[s[i]] == 1:
            return i

    return -1
```

Time:

\[\boxed{O(n)} \]

Space:

\[O(k) \]

PART F — LONGEST COMMON PREFIX

23. PROBLEM

Input:

```text
["flower", "flow", "flight"]
```

Longest common prefix:

```text
"fl"
```

---

24. SIMPLE IDEA

Take first string as candidate prefix.

Compare character positions across all strings.

At index 0:

```text
f f f
```

match.

Index 1:

```text
l l l
```

match.

Index 2:

```text
o o i
```

mismatch.

Answer:

```text
fl
```

---

25. COMPLEXITY

If total number of examined characters is \(S\):

\[\boxed{O(S)} \]

In rough worst-case form with \(m\) strings each of length \(n\):

\[O(mn) \]

---

26. IMPORTANT EDGE CASES

Input:

```text
[]
```

or:

```text
[""]
```

or:

```text
["dog","racecar","car"]
```

Answer:

```text
""
```

PART G — REVERSE WORDS

27. PROBLEM

Input:

```text
"the sky is blue"
```

Output:

```text
"blue is sky the"
```

Important:

This is **reverse word order**, not reverse every character.

28. SIMPLE APPROACH

Split words:

```text
["the","sky","is","blue"]
```

Reverse:

```text
["blue","is","sky","the"]
```

Join:

```text
"blue is sky the"
```

29. INTERVIEW TRAP

The interviewer may ask:

> **Do it in-place on a character array.**

Classic strategy:

1. Reverse entire string.
2. Reverse each word individually.

Example:

```text
the sky
```

Reverse all:

```text
yks eht
```

Reverse each word:

```text
sky the
```

Useful pattern.

PART H — LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS

This is one of the most important string-placement problems.

30. PROBLEM

Input:

```text
"abcabcbb"
```

Longest substring without repeated characters:

```text
"abc"
```

Length:

$$ \boxed{3} $$

---

31. BRUTE FORCE

Check every substring and test uniqueness.

Can be:

$$ O(n^3) $$

or optimized brute:

$$ O(n^2) $$

But optimal expected:

$$ \boxed{O(n)} $$

using Sliding Window.

---

32. SLIDING WINDOW IDEA

Maintain window:

```text
[left ... right]
```

such that:

> **all characters inside are unique.**

Expand right.

If duplicate appears:

> **shrink from left until valid again.**

---

33. TRACE

String:

```text
a b c a b c b b
```

Start:

```text
a
```

length 1.

Then:

```text
ab
```

length 2.

Then:

```text
abc
```

length 3.

Next `a` repeats.

Remove from left until old `a` leaves.

Window becomes:

```text
bca
```

Still length 3.

Best remains:

\[3 \]

---

34. SET-BASED CODE

```python
def longest_unique_substring(s):
    seen = set()

    left = 0
    best = 0

    for right in range(len(s)):

        while s[right] in seen:
            seen.remove(s[left])
            left += 1

        seen.add(s[right])

        best = max(best, right - left + 1)

    return best
```

Time:

\[\boxed{O(n)} \]

expected.

Why not \(O(n^2)\)?

Each character enters and leaves the window at most once.

Space:

\[O(k) \]

---

35. FASTER LAST-SEEN VERSION

Instead of removing one by one, store last seen index.

If current character appeared within window:

\[left=\max(left,last[ch]+1) \]

Then update:

```text
last[ch] = right
```

Still:

$$ O(n) $$

expected.

---

36. SLIDING WINDOW SIGNAL

Question says:

> **Longest/shortest substring satisfying a condition.**

Think immediately:

$$ \boxed{\text{Sliding Window}} $$

Examples:

- longest substring without repeating
- longest substring with at most K distinct chars
- minimum window containing characters
- longest repeating-character replacement

PART I — GROUP ANAGRAMS

37. PROBLEM

Input:

```text
["eat","tea","tan","ate","nat","bat"]
```

Output groups:

```text
["eat","tea","ate"]
["tan","nat"]
["bat"]
```

---

38. KEY INSIGHT

Anagrams share the same **signature**.

For example:

```text
eat
tea
ate
```

sorted signature:

```text
aet
```

Use signature as HashMap key.

39. APPROACH 1 — SORTED SIGNATURE

For every word:

```text
key = sorted(word)
```

Group words by key.

If average word length is \(k\):

\[O(nk\log k) \]

for \(n\) words.

40. APPROACH 2 — FREQUENCY SIGNATURE

For lowercase English letters:

Create:

```text
[frequency of a, b, c, ..., z]
```

Words with same frequency vector are anagrams.

Complexity:

\[\boxed{O(nk)} \]

PART J — VALID PARENTHESES

41. PROBLEM

Input:

```text
"()[]{}"
```

Valid.

Input:

```text
"(]"
```

Invalid.

Input:

```text
"([{}])"
```

Valid.

42. WHY THIS IS A STACK PROBLEM

When closing bracket appears, it must match the **most recently opened unmatched bracket**.

That is:

$$ \boxed{\text{LIFO}} $$

Therefore:

$$ \boxed{\text{Stack}} $$

43. ALGORITHM

For opening bracket:

```text
push
```

For closing bracket:

1. stack must not be empty,
2. top must be matching opening bracket,
3. pop.

At end:

$$ \boxed{\text{stack must be empty}} $$

44. CODE

```python
def valid_parentheses(s):
    stack = []

    match = {
        ')': '(',
        ']': '[',
        '}': '{'
    }

    for ch in s:
        if ch in "([{":
            stack.append(ch)

        else:
```

```
            if not stack or stack[-1] != match[ch]:
                return False

            stack.pop()

    return len(stack) == 0
```

Time:

\[\boxed{O(n)} \]

Space:

\[\boxed{O(n)} \]

worst case.

---

45. INTERVIEW PATTERN SIGNAL

Words such as:

- balanced
- nested
- matching brackets
- next greater
- previous smaller
- expression evaluation

often indicate:

\[\boxed{\text{Stack}} \]

PART K — VALID PALINDROME II

46. PROBLEM

Can string become palindrome after deleting at most one character?

Example:

```text
"abca"
```

Remove:

```text
b
```

gives:

```text
aca
```

or remove `c`:

```text
aba
```

So:

$$\boxed{\text{True}}$$

47. TWO-POINTER INSIGHT

Compare inward.

At first mismatch:

```text
s[left] != s[right]
```

you get only two possible useful choices:

1. skip left,
2. skip right.

Then check whether remaining substring is palindrome.

48. COMPLEXITY

Initial scan:

$$O(n)$$

At most two palindrome checks of remaining section, but only once at first mismatch.

Total:

$$\boxed{O(n)}$$

not exponential.

PART L — STRING COMPRESSION

49. PROBLEM

Input:

```text
["a","a","b","b","c","c","c"]
```

Compressed:

```text
["a","2","b","2","c","3"]
```

Use:

- read pointer
- write pointer

Count consecutive groups.

Pattern:

$$ \boxed{\text{Two Pointers / Run-Length Encoding}} $$

---

50. IDEA

For each group:

```TEXT
aaa
```

write:

```TEXT
a3
```

For single character:

```TEXT
b
```

write only:

```TEXT
b
```

Time:

$$ O(n) $$

PART M — ISOMORPHIC STRINGS

51. PROBLEM

```TEXT
"egg"
"add"
```

Isomorphic:

```TEXT
 e   !′    a
 g   !′    d
```

Yes.

But:

```TEXT
"foo"
"bar"
```

No.

---

52. KEY RULE

Mapping must be consistent in both directions.

Why both directions?

Suppose:

```TEXT
"ab"
"cc"
```

If only map f i r s t!' s e c o n d :

```TEXT
 a!' c
 b!' c
```

looks possible.

But two different characters should not map to the same target if mapping must be one-to-one.

So maintain:

```TEXT
map1
map2
```

This is a hashing pattern.

PART N — RANSOM NOTE / CHARACTER CONSUMPTION

53. PROBLEM TYPE

Can one string be constructed from characters of another?

Example:

```TEXT
ransom = "aa"
magazine = "aab"
```

Yes.

Frequency technique:

1. count source characters,
2. consume for target,
3. fail if count becomes unavailable.

This pattern appears frequently.

PART O — LONGEST PALINDROMIC SUBSTRING: IMPORTANT DISTINCTION

54. PROBLEM

Input:

```text
"babad"
```

One answer:

```text
"bab"
```

or:

```text
"aba"
```

This is different from simple palindrome check.

---

55. EXPAND AROUND CENTER

A palindrome has a center.

Two possibilities:

Odd length

```text
racecar
   ^
```

one center character.

Even length

```text
abba
  ^^
```

center between two characters.

For each possible center:

> **expand while characters match.**

Time:

$$ \boxed{O(n^2)} $$

Space:

$$ O(1) $$

This is a common medium interview problem.

PART P — SUBSTRING VS SUBSEQUENCE

56. CRITICAL DISTINCTION

String:

```text
ABCDE
```

Substring

Contiguous.

Examples:

```text
ABC
BCD
DE
```

Subsequence

Need not be contiguous.

Examples:

```text
ACE
ABD
AE
```

This distinction determines algorithm patterns.

---

57. INTERVIEW SIGNAL

Substring

Think:

- sliding window
- two pointers

- pattern matching

Subsequence

Think:

- DP
- greedy
- two pointers depending problem

Example:

LCS = Dynamic Programming.

PART Q — STRING HASHING VS FIXED ARRAY

58. WHEN TO USE HASHMAP

Use HashMap if:

- arbitrary Unicode/general characters,
- keys aren't limited to small alphabet,
- need dynamic mappings.

---

59. WHEN TO USE FIXED ARRAY

If problem guarantees:

```TEXT
'a' to 'z'
```

use:

```TEXT
freq[26]
```

Advantages:

- simpler
- faster constants
- fixed \(O(1)\) space

Interviewers like when you exploit constraints.

PART R — COMMON STRING INTERVIEW TRAPS

60. CASE SENSITIVITY

Are:

```text
"A"
"a"
```

considered same?

Do not assume.

Read/clarify.

61. SPACES AND PUNCTUATION

Palindrome questions may or may not ignore:

```text
spaces
commas
punctuation
```

Clarify or follow statement exactly.

62. EMPTY STRING

Is empty string valid?

For many definitions, yes.

But always respect problem specification.

63. CHARACTER ENCODING

Do not assume only 26 characters unless constraints say so.

64. SUBSTRING "`SUBSEQUENCE

One of the most frequent conceptual mistakes.

65. LONGEST SUBSTRING SLIDING WINDOW INVARIANT

Your window should always satisfy the condition.

Example:

> **no repeating characters.**

If duplicate appears:

> **shrink until valid.**

This **window invariant** is the key idea.

PART S — INTERVIEW EXPLANATION EXAMPLE

Suppose interviewer asks:

> **Longest substring without repeating characters.**

A strong explanation:

> **"A brute-force approach would examine every substring and check whether all characters are unique, which is at least quadratic. Since the problem asks for the longest valid contiguous substring, I can use a sliding window. I maintain a set of characters currently in the window. I expand the right pointer, and whenever the new character already exists, I move the left pointer forward while removing characters until the window becomes valid again. Each character enters and leaves the set at most once, so the expected time complexity is $O(n)$ and space is $O(k)$, where k is the number of distinct characters in the window."**

That's interview-level communication.

PART T — MUST-DO STRING QUESTIONS

66. LEVEL 1 — FOUNDATION

1. Reverse String
2. Check Palindrome
3. Character Frequency
4. Valid Anagram
5. First Unique Character
6. Longest Common Prefix
7. Reverse Words
8. Ransom Note
9. Find the Difference
10. Isomorphic Strings

67. LEVEL 2 — CORE PLACEMENT

11. Valid Parentheses
12. Longest Substring Without Repeating Characters
13. Group Anagrams
14. Valid Palindrome II
15. String Compression
16. Longest Palindromic Substring
17. Check Subsequence
18. Remove Adjacent Duplicates

19. Count Palindromic Substrings
20. Roman-to-Integer type parsing

68. LEVEL 3 — ADVANCED PATTERNS

21. Minimum Window Substring
22. Longest Repeating Character Replacement
23. Find All Anagrams in a String
24. Decode String
25. Word Break
26. Edit Distance
27. Pattern Matching
28. KMP basics
29. Rabin-Karp basics
30. Trie-based prefix problems

Do not jump to these until foundation is comfortable.

PART U — PLACEMENT PRACTICE SET

Try them mentally first.

Q1 — MCQ

The most natural pattern for checking whether a string is a palindrome is:

A. Heap B. Two Pointers C. BFS D. Dynamic Programming

Q2 — MCQ

Two lowercase strings are anagrams. Best basic linear-time technique:

A. BFS B. Frequency array C. Heap Sort D. Dijkstra

Q3 — NAT

For:

```text
"abcabcbb"
```

what is the length of the longest substring without repeating characters?

Q4 — MSQ

Which problems strongly suggest hashing?

A. Character frequency

B. Group Anagrams

C. First Unique Character

D. Binary-tree level order

Q5 — MCQ

Balanced parentheses are naturally solved using:

A. Queue B. Stack C. Heap D. Union-Find

Q6 — NAT

For:

```text
"pwwkew"
```

what is the length of the longest substring without repeating characters?

Q7 — MCQ

Which statement is true?

A. Every substring is a subsequence.

B. Every subsequence is a substring.

C. Substrings need not be contiguous.

D. Subsequence characters must always be adjacent.

Q8 — MSQ

Which are typical Two-Pointer string problems?

A. Reverse String

B. Palindrome Check

C. Valid Palindrome II

D. Dijkstra

Q9 — MCQ

For lowercase letters only, character frequencies can be stored in:

A. array of size 26

B. adjacency matrix

C. priority queue only

D. recursion stack only

Q10 — NAT

Input:

```text
["flower","flow","flight"]
```

What is the length of the longest common prefix?

Q11 — MCQ

Longest substring without repeating characters is most naturally:

A. Sliding Window

B. Merge Sort

C. Union-Find

D. Prim

Q12 — MSQ

Which statements are correct?

A. Valid Anagram can be solved by sorting.

B. Valid Anagram can be solved using frequency counting.

C. Group Anagrams can use sorted word as a HashMap key.

D. Valid Parentheses is mainly a queue problem.

PRACTICE SOLUTIONS

Q1

Palindrome compares characters from both ends.

$$\boxed{B}$$

Q2

Fixed lowercase alphabet:

$$\boxed{\text{frequency array}}$$

Answer:

$$\boxed{B}$$

Q3

Longest valid substrings include:

```text
abc
bca
cab
```

Length:

$$ \boxed{3} $$

Q4

A, B, C use hashing naturally.

$$ \boxed{A,B,C} $$

Q5

Nested matching requires LIFO.

$$ \boxed{B} $$

Q6

For:

```text
pwwkew
```

longest examples:

```text
wke
kew
```

Length:

$$ \boxed{3} $$

Q7

Every contiguous substring preserves order, so it is also a subsequence.

Answer:

$$ \boxed{A} $$

Q8

Reverse String, Palindrome Check, Valid Palindrome II.

$$\boxed{A,B,C}$$

Q9

$$\boxed{A}$$

Q10

Common prefix:

```
TEXT
fl
```

Length:

$$\boxed{2}$$

Q11

$$\boxed{A}$$

Q12

A true.

B true.

C true.

D false.

Answer:

$$\boxed{A,B,C}$$

Ø=ÜÓTES TO WRITE

String Pattern Map

```
TEXT
Palindrome                    !'  Two Pointers
Anagram                       !'  Frequency
Character Count               !'  Hashing
First Unique                  !'  Frequency + scan
Longest substring             !'  Sliding Window
Valid Parentheses             !'  Stack
Group Anagrams                !'  Hashing + signature
```

```
Reverse Words              !' Split/reverse or reversal trick
Palindrome II              !' Two pointers + skip one
```

COMPLEXITIES TO REMEMBER

```TEXT
Reverse String                          O(n)
Palindrome                              O(n)
Anagram frequency                       O(n)
First Unique Character                  O(n)
Longest Common Prefix                   O(total chars)
Longest Unique Substring                O(n)
Valid Parentheses                       O(n)
Group Anagrams sorted-signature         O(nk log k)
Group Anagrams frequency-signature      O(nk)
Longest Palindromic Substring
  expand around center                  O(n²)
```

&β30-SECOND STRING REVISION

1. Palindrome !' Two Pointers.
2. Reverse !' Two Pointers.
3. Anagram !' frequencies.
4. Lowercase only !' array[26].
5. General chars !' HashMap.
6. First unique !' count then scan.
7. Longest substring !' Sliding Window.
8. Sliding Window maintains a valid contiguous region.
9. Longest unique substring !' HashSet/last-seen map.
10. Parentheses !' Stack.
11. Group Anagrams !' canonical signature.
12. Substring is contiguous.
13. Subsequence need not be contiguous.
14. Valid Palindrome II !' skip left or right at first mismatch.
15. Reverse Words "`reverse characters.
16. Group Anagrams can use sorted letters or frequency vector.
17. Expand-around-center solves longest palindrome in \(O(n^2)\).
18. Always use constraints to choose HashMap vs fixed array.

' Placement Module 3 Complete

Next Placement Module:

MODULE 4 — LINKED LISTS + FAST/SLOW POINTER PATTERNS

We'll cover:

- linked-list fundamentals,
- node structure,
- array vs linked list,
- insertion/deletion,
- reverse linked list,
- middle node,
- Floyd cycle detection,
- merge two sorted lists,
- remove Nth node from end,
- palindrome linked list,
- intersection of two linked lists,
- dummy-node technique,
- pointer interview traps,
- and placement-style coding questions.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 4 — LINKED LISTS + FAST/SLOW POINTER PATTERNS

Placement Priority: & & & & &

Linked Lists are important because they test whether you truly understand **references, pointers, node manipulation, and edge cases**.

The biggest placement mistake is trying to memorize pointer code. Instead, learn a small set of patterns:

\[\boxed{\text{Dummy Node + Fast/Slow + Pointer Reversal + Merge}} \]

---

1. WHAT IS A LINKED LIST?

A linked list consists of nodes.

Each node stores:

TEXT

```
data
next
```

Example:

```TEXT
 1 0  !’   2 0  !’   3 0  !’   4 0  !’   N U L L
```

Unlike arrays, nodes do not need to be stored next to each other in memory.

2. BASIC NODE STRUCTURE

Python-style:

```PYTHON
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None
```

A list:

```TEXT
head
  !“
 1 0  !’   2 0  !’   3 0  !’   N U L L
```

`head` points to the first node.

3. ARRAY VS LINKED LIST

Operation	Array	Linked List
Access by index	\(O(1)\)	\(O(n)\)
Search	\(O(n)\)	\(O(n)\)
Insert at beginning	\(O(n)\)	\(O(1)\)
Delete beginning	\(O(n)\)	\(O(1)\)
Random access	Yes	No
Extra pointer memory	No	Yes

Important interview fact:

> **Linked Lists are good at pointer-based insertion/deletion, but bad at random access.**

4. TYPES OF LINKED LISTS

Singly Linked List

```TEXT
 A  !’   B  !’   C  !’   N U L L
```

Each node stores only `next`.

Doubly Linked List

```TEXT
 N U L L  !•   A  !Ä   B  !Ä   C  !’   N U L L
```

Each node stores:

```TEXT
prev
next
```

Circular Linked List

```TEXT
 A  !’   B  !’   C
!’              !“
% % % % % % % % %
```

Last node points back instead of `NULL`.

---

5. TRAVERSING A LINKED LIST

```PYTHON
current = head

while current:
    print(current.val)
    current = current.next
```

Time:

\[\boxed{O(n)} \]

---

6. INSERT AT BEGINNING

Suppose:

```TEXT
 1 0  !’   2 0  !’   3 0
```

Insert `5`.

Create node:

```text
5
```

Point new node to old head:

```text
 5  !’   1 0  !’   2 0  !’   3 0
```

Then update head.

```python
new_node.next = head
head = new_node
```

Time:

\[\boxed{O(1)} \]

---

7. INSERT AT END

If no tail pointer exists:

> **traverse until last node.**

Time:

\[O(n) \]

If a tail pointer is maintained:

\[\boxed{O(1)} \]

---

8. DELETE FIRST NODE

```text
 1 0  !’   2 0  !’   3 0
```

Set:

```text
head = head.next
```

Result:

```text
 2 0  !’   3 0
```

Time:

\[\boxed{O(1)} \]

PART A — REVERSE LINKED LIST

This is one of the most important placement problems.

9. PROBLEM

Input:

```text
 1  !’   2  !’   3  !’   4  !’   N U L L
```

Output:

```text
 4  !’   3  !’   2  !’   1  !’   N U L L
```

10. THE THREE-POINTER PATTERN

Maintain:

```text
prev
current
next_node
```

Initially:

```text
prev = NULL
current = head
```

At every node:

1. Save next node.
2. Reverse current pointer.
3. Move `prev`.
4. Move `current`.

11. TRACE

Start:

```text
 N U L L       1  !’   2  !’   3
  !’          !’
prev   current
```

Save:

```TEXT
next = 2
```

Reverse:

```TEXT
 1  !’   N U L L
```

Move:

```TEXT
prev = 1
current = 2
```

Next:

```TEXT
 1  !•   2  !’   3
```

Eventually:

```TEXT
 4  !’   3  !’   2  !’   1  !’   N U L L
```

New head:

\[\boxed{prev} \]

---

12. CODE

```PYTHON
def reverse_list(head):
    prev = None
    current = head

    while current:
        next_node = current.next

        current.next = prev

        prev = current
        current = next_node

    return prev
```

Time:

\[\boxed{O(n)} \]

Space:

\[\boxed{O(1)} \]

---

13. MEMORY TRICK

> **Save !’ Reverse !’ Move !’ Move**

Never reverse `current.next` before saving the original next node, or you may lose the remaining list.

PART B — FIND MIDDLE OF LINKED LIST

14. PROBLEM

```TEXT
 1  !’   2  !’   3  !’   4  !’   5
```

Middle:

\[\boxed{3} \]

For:

```TEXT
 1  !’   2  !’   3  !’   4  !’   5  !’   6
```

many interview definitions return the second middle:

\[\boxed{4} \]

---

15. FAST/SLOW POINTERS

Use:

```TEXT
 s l o w  !’   1   s t e p
 f a s t  !’   2   s t e p s
```

When fast reaches end:

> **slow is in the middle.**

---

16. TRACE

```TEXT
 1  !’   2  !’   3  !’   4  !’   5
S
F
```

Move:

```text
slow = 2
fast = 3
```

Again:

```text
slow = 3
fast = 5
```

Fast finishes.

Answer:

\[3 \]

---

17. CODE

```python
def middle_node(head):
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    return slow
```

Time:

\[\boxed{O(n)} \]

Space:

\[\boxed{O(1)} \]

---

18. FAST/SLOW SIGNAL

Think fast/slow when question asks:

- middle node
- cycle detection
- cycle starting point
- palindrome linked list
- kth-related gap techniques

PART C — LINKED LIST CYCLE

19. PROBLEM

```text
 1  !′   2  !′   3  !′   4
                !′          !“
                % % % % % % %
```

Determine whether a cycle exists.

---

20. BRUTE APPROACH — HASHSET

Store visited node addresses.

If same node appears again:

> **cycle.**

Time:

\[O(n) \]

Space:

\[O(n) \]

Good first solution.

---

21. OPTIMAL — FLOYD'S CYCLE DETECTION

Use:

```text
slow = 1 step
fast = 2 steps
```

If no cycle:

> **fast eventually reaches `NULL`.**

If cycle:

> **fast eventually meets slow.**

This is also called:

\[\boxed{\text{Tortoise and Hare}} \]

---

22. WHY DO THEY MEET?

Inside a cycle, fast gains one node per iteration relative to slow.

Eventually the distance between them becomes zero modulo cycle length.

Therefore:

$$ \boxed{slow==fast} $$

23. CODE

```python
def has_cycle(head):
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

        if slow == fast:
            return True

    return False
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(1)} $$

PART D — FIND START OF CYCLE

24. ADVANCED FOLLOW-UP

After slow and fast meet inside cycle:

1. move one pointer back to head,
2. keep other at meeting point,
3. move both one step at a time,
4. where they meet is cycle entry.

25. CODE IDEA

```python
slow = head

while slow != fast:
    slow = slow.next
    fast = fast.next

return slow
```

This is a very common follow-up after cycle detection.

PART E — MERGE TWO SORTED LINKED LISTS

26. PROBLEM

```TEXT
 L 1 :   1  !′   3  !′   5
 L 2 :   2  !′   4  !′   6
```

Output:

```TEXT
 1  !′   2  !′   3  !′   4  !′   5  !′   6
```

27. PATTERN

This is similar to Merge Sort's merge step.

Compare current nodes:

```TEXT
L1.val
L2.val
```

Attach smaller node.

Move that pointer.

28. DUMMY NODE TECHNIQUE

Instead of special handling for the first node, create:

```TEXT
 d u m m y  !′   . . .
tail
```

This simplifies code dramatically.

29. CODE

```PYTHON
def merge_two_lists(a, b):
    dummy = Node(0)
    tail = dummy

    while a and b:
        if a.val <= b.val:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next

        tail = tail.next
```

```
    if a:
        tail.next = a
    else:
        tail.next = b

    return dummy.next
```

Time:

\[\boxed{O(n+m)} \]

Extra space:

\[\boxed{O(1)} \]

excluding existing nodes.

---

30. DUMMY NODE MEMORY TRICK

> **Dummy removes head special cases.**

Very useful for:

- merge lists
- delete nodes
- remove nth from end
- partition list

PART F — REMOVE NTH NODE FROM END

31. PROBLEM

```TEXT
1 !' 2 !' 3 !' 4 !' 5
```

Remove:

\[n=2 \]

from end.

Second from end is `4`.

Output:

```TEXT
1 !' 2 !' 3 !' 5
```

---

32. TWO-PASS APPROACH

First count length.

Suppose:

\[L=5 \]

Nth from end corresponds to:

\[L-n \]

steps before target.

Works:

\[O(n) \]

but traverses twice.

---

33. ONE-PASS GAP TECHNIQUE

Use two pointers:

```text
fast
slow
```

Create a gap of \(n+1\) nodes between them using dummy node.

Then move both together.

When fast reaches end:

> **slow is immediately before node to remove.**

---

34. WHY DUMMY NODE?

Suppose removing first node.

Example:

```text
 1  !'   2
```

remove second from end !' remove `1`.

Dummy makes deletion uniform:

```text
 D  !'   1  !'   2
```

No special-case head handling.

---

35. CODE

```python
def remove_nth_from_end(head, n):
    dummy = Node(0)
    dummy.next = head

    slow = dummy
    fast = dummy

    for _ in range(n + 1):
        fast = fast.next

    while fast:
        slow = slow.next
        fast = fast.next

    slow.next = slow.next.next

    return dummy.next
```

Time:

\[\boxed{O(n)} \]

Space:

\[\boxed{O(1)} \]

---

36. GAP PATTERN

Whenever interviewer says:

> **kth/nth node from end**

think:

\[\boxed{\text{Two pointers with fixed gap}} \]

PART G — PALINDROME LINKED LIST

37. PROBLEM

```text
1 !' 2 !' 2 !' 1
```

Palindrome:

\[\boxed{True} \]

---

38. EASY APPROACH

Copy values to array:

```text
[1,2,2,1]
```

Then two-pointer palindrome test.

Time:

\[O(n) \]

Space:

\[O(n) \]

---

39. OPTIMAL APPROACH

1. Find middle using fast/slow.
2. Reverse second half.
3. Compare first half and reversed second half.
4. Optionally restore list.

Time:

\[\boxed{O(n)} \]

Space:

\[\boxed{O(1)} \]

---

40. BEAUTIFUL PATTERN COMBINATION

Palindrome Linked List combines two patterns:

\[\boxed{\text{Fast/Slow + Reverse List}} \]

Interviewers love such combinations.

PART H — INTERSECTION OF TWO LINKED LISTS

41. PROBLEM

Two linked lists may physically share nodes.

Example:

```text
 A :   1  !'   2
         \
                    8  !'   9
         /
 B :   4  !'   5
```

Intersection node:

\[\boxed{8} \]

Important:

> **Intersection means same node reference, not merely same value.**

---

42. HASHSET APPROACH

Store all nodes of first list.

Traverse second.

First shared node is intersection.

Time:

\[O(n+m) \]

Space:

\[O(n) \]

---

43. OPTIMAL TWO-POINTER TRICK

Use:

```text
p = headA
q = headB
```

When `p` reaches end:

```text
p = headB
```

When `q` reaches end:

```text
q = headA
```

Eventually:

- they meet at intersection, or
- both become `None`.

---

44. WHY DOES SWITCHING HEADS WORK?

Suppose lengths:

\[A=a+c \]

\[B=b+c \]

where \(c\) is shared suffix.

Pointer A walks:

$$ a+c+b $$

Pointer B walks:

$$ b+c+a $$

Same total distance.

Their unequal starting lengths cancel.

---

45. CODE

```python
def get_intersection(a, b):
    p = a
    q = b

    while p != q:
        p = p.next if p else b
        q = q.next if q else a

    return p
```

Time:

$$ \boxed{O(n+m)} $$

Space:

$$ \boxed{O(1)} $$

PART I — DELETE NODE GIVEN ONLY THAT NODE

46. INTERVIEW TRICK

Suppose list:

```text
4 !' 5 !' 1 !' 9
```

You are given node `5`, but not head.

Can you delete it?

You cannot access previous node.

Trick:

1. copy next node's value into current node,
2. bypass next node.

```text
4 !' 1 !' 9
```

Conceptually, the node object remains but its value changes.

---

47. CODE

```python
node.val = node.next.val
node.next = node.next.next
```

Time:

\[O(1) \]

But cannot use this method to delete last node.

PART J — REMOVE DUPLICATES FROM SORTED LIST

48. PROBLEM

```text
 1  !'   1  !'   2  !'   3  !'   3
```

Output:

```text
 1  !'   2  !'   3
```

Because sorted list places duplicates together.

---

49. CODE IDEA

```python
current = head

while current and current.next:
    if current.val == current.next.val:
        current.next = current.next.next
    else:
        current = current.next
```

Time:

\[O(n) \]

Space:

\[O(1) \]

PART K — SWAP PAIRS

50. PROBLEM

```text
1 !' 2 !' 3 !' 4
```

Output:

```text
2 !' 1 !' 4 !' 3
```

This is a good pointer-manipulation exercise.

A dummy node is extremely useful.

You need to carefully reconnect:

```text
p r e v !' f i r s t !' s e c o n d !' n e x t
```

into:

```text
p r e v !' s e c o n d !' f i r s t !' n e x t
```

PART L — REORDER LIST

51. PROBLEM

```text
1 !' 2 !' 3 !' 4 !' 5
```

Reorder:

```text
1 !' 5 !' 2 !' 4 !' 3
```

Pattern combination:

1. Find middle.
2. Reverse second half.
3. Merge alternating nodes.

Again:

\[\boxed{\text{Fast/Slow + Reverse + Merge}} \]

PART M — SORT LINKED LIST

52. BEST NATURAL SORT

Linked Lists work very well with Merge Sort.

Why?

Splitting can use fast/slow pointers.

Merging does not require shifting elements.

Complexity:

$$\boxed{O(n\log n)}$$

Additional recursive stack:

$$O(\log n)$$

PART N — WHY BINARY SEARCH IS NOT NATURAL ON LINKED LIST

In an array:

$$A[mid]$$

is:

$$O(1)$$

In linked list:

> **reaching middle requires traversal.**

So even though the list could be sorted, normal Binary Search loses its major advantage.

This is a common conceptual interview question.

PART O — DUMMY NODE MASTER PATTERN

53. DUMMY NODE

Create:

```TEXT
d u m m y  !’  h e a d
```

Benefits:

- deleting head becomes ordinary deletion,
- merging is easier,
- building new list is easier,
- fewer `if head is None` cases.

Use it aggressively when pointer code becomes messy.

PART P — POINTER SAFETY RULES

54. NEVER LOSE THE NEXT NODE

Before doing:

```text
current.next = prev
```

save:

```text
next_node = current.next
```

Otherwise remaining list may become unreachable.

---

55. CHECK NULL BEFORE `.next`

Bad:

```python
while fast.next.next:
```

if `fast.next` could be `None`.

Safer:

```python
while fast and fast.next:
```

---

56. NODE VALUE VS NODE IDENTITY

Two nodes can both contain:

```text
5
```

but be different nodes.

For intersection and cycles, compare:

\[\boxed{\text{node references}} \]

not just values.

PART Q — LINKED LIST PATTERN MAP

Memorize:

Problem	Pattern
Reverse List	prev/current/next
Middle	Fast/Slow

Problem	Pattern
Cycle	Fast/Slow
Cycle Entry	Fast/Slow reset
Merge sorted lists	Dummy + Merge
Nth from end	Fixed gap
Palindrome	Middle + Reverse
Intersection	Pointer switching
Remove node	Dummy often
Sort list	Merge Sort
Reorder list	Middle + Reverse + Merge

PART R — INTERVIEW EXPLANATION EXAMPLE

Suppose interviewer asks:

> **Detect a cycle in linked list using constant space.**

Strong answer:

> **"A HashSet solution can store visited node references and detect repetition in \(O(n)\) time, but that uses \(O(n)\) extra space. To achieve constant space, I can use Floyd's slow and fast pointers. Slow advances one node and fast advances two. If there is no cycle, fast reaches the end. If a cycle exists, the faster pointer eventually catches the slower pointer inside the cycle. This gives \(O(n)\) time and \(O(1)\) extra space."**

That is exactly the kind of comparison interviewers want.

PART S — MUST-DO LINKED LIST QUESTIONS

Foundation

1. Traverse Linked List
2. Insert at Beginning
3. Delete First Node
4. Reverse Linked List
5. Middle of Linked List

Core Placement

6. Detect Cycle
7. Find Cycle Start
8. Merge Two Sorted Lists
9. Remove Nth Node from End

10. Palindrome Linked List
11. Intersection of Two Linked Lists
12. Remove Duplicates from Sorted List
13. Delete Node Given Node

Medium

14. Swap Nodes in Pairs
15. Reorder List
16. Odd-Even Linked List
17. Add Two Numbers
18. Sort List
19. Reverse Between Positions
20. Reverse Nodes in K Group

PART T — PLACEMENT PRACTICE SET

Q1 — MCQ

Random access of the \(i\)-th node in a singly linked list takes:

A. \(O(1)\) B. \(O(\log n)\) C. \(O(n)\) D. \(O(n\log n)\)

Q2 — MCQ

Which pointer pattern is most suitable for finding middle node?

A. Two independent queues B. Fast/Slow pointers C. Binary Search D. Heap

---

Q3 — MCQ

Floyd's cycle detection uses:

A. HashMap only B. Slow and Fast pointers C. Priority Queue D. Dynamic Programming

---

Q4 — MSQ

Which problems commonly use Fast/Slow pointers?

A. Middle of Linked List

B. Cycle Detection

C. Finding cycle entry

D. Heap Sort

Q5 — MCQ

Best extra-space complexity for iterative Reverse Linked List:

A. $O(n)$ B. $O(\log n)$ C. $O(1)$ D. $O(n^2)$

Q6 — NAT

A linked list contains 100 nodes.

In the worst case, how many nodes may need to be traversed to access the last node from `head`?

Q7 — MCQ

A dummy node is particularly useful because:

A. it sorts the list automatically

B. it reduces special handling around the head

C. it gives random access

D. it detects cycles automatically

Q8 — MSQ

Which are $O(n)$-time problems with optimal linked-list techniques?

A. Reverse List

B. Find Middle

C. Detect Cycle

D. Merge two lists of total length n

Q9 — MCQ

To find the Nth node from end in one traversal, use:

A. two pointers separated by a fixed gap

B. Heap Sort

C. BFS

D. Prefix Sum

Q10 — MCQ

Palindrome Linked List with $O(1)$ extra space typically combines:

A. Hashing + Heap

B. Fast/Slow + Reverse List

C. BFS + DFS

D. Binary Search + Sorting

Q11 — MCQ

Two lists intersect if:

A. they contain equal values somewhere

B. they share the same node object/reference

C. both are sorted

D. both have equal length

Q12 — MSQ

Which statements are true?

A. Linked List insertion at head can be $O(1)$.

B. Random indexing is $O(1)$.

C. Merge Sort is a natural sorting algorithm for Linked Lists.

D. Binary Search is usually less attractive on Linked Lists.

PRACTICE SOLUTIONS

Q1

Must traverse from head.

$$\boxed{C}$$

Q2

$$\boxed{B}$$

Q3

$$\boxed{B}$$

Q4

A, B, C.

$$\boxed{A,B,C}$$

Q5

Iterative reversal uses three pointers.

$$\boxed{C}$$

Q6

From first to last can require visiting all 100 nodes.

$$ \boxed{100} $$

Q7

Dummy simplifies head-related cases.

$$ \boxed{B} $$

Q8

All four can be linear.

$$ \boxed{A,B,C,D} $$

Q9

$$ \boxed{A} $$

Q10

$$ \boxed{B} $$

Q11

Intersection means identity/reference.

$$ \boxed{B} $$

Q12

A true.

B false.

C true.

D true.

$$ \boxed{A,C,D} $$

Ø=ÜÓTES TO WRITE

```text
LINKED LIST PATTERNS

Reverse:
 p r e v  !'  c u r r e n t  !'  n e x t
 S a v e  !'  R e v e r s e  !'  M o v e  !'  M o v e

Middle:
slow = 1 step
fast = 2 steps

Cycle:
slow/fast meet
```

```
Cycle Entry:
after meeting,
 o n e   p o i n t e r  !’   h e a d
both move 1 step

Merge:
dummy + tail

Nth from end:
fixed gap

Palindrome:
middle + reverse + compare

Intersection:
switch heads

Sort List:
Merge Sort
```

COMPLEXITIES TO REMEMBER

Operation	Time
Access kth node	\(O(n)\)
Search	\(O(n)\)
Insert at head	\(O(1)\)
Reverse	\(O(n)\)
Middle	\(O(n)\)
Cycle detection	\(O(n)\)
Merge sorted lists	\(O(n+m)\)
Intersection	\(O(n+m)\)
Linked-list Merge Sort	\(O(n\log n)\)

&β30-SECOND LINKED LIST REVISION

1. Linked List has nodes connected by pointers.
2. Random access is \(O(n)\).
3. Insert/delete at head can be \(O(1)\).
4. Reverse uses `prev-current-next`.
5. Save `next` before breaking pointer.
6. Middle !’ slow/fast.
7. Cycle !’ Floyd slow/fast.
8. Cycle entry !’ reset one pointer to head.
9. Merge sorted lists !’ dummy node.

10. Nth from end !' fixed pointer gap.
11. Palindrome !' middle + reverse + compare.
12. Intersection means same node reference.
13. Dummy node removes head special cases.
14. Merge Sort fits Linked Lists well.
15. Binary Search is less useful because middle access is not \(O(1)\).

' Placement Module 4 Complete

Next Placement Module:

MODULE 5 — STACKS, QUEUES, DEQUES + MONOTONIC STACK

We'll cover:

- Stack and Queue fundamentals,
- implementation and complexity,
- Valid Parentheses,
- Min Stack,
- Next Greater Element,
- Previous Smaller Element,
- Daily Temperatures,
- Monotonic Stack,
- Queue using Stacks,
- Stack using Queues,
- Circular Queue,
- Deque,
- Sliding Window Maximum,
- and how to recognize Stack vs Queue questions.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 5 — STACKS, QUEUES, DEQUES + MONOTONIC STACK

Placement Priority: & & & & &

This module is extremely important because many questions that look unrelated are actually based on only a few patterns:

\[\boxed{\text{LIFO, FIFO, Monotonic Stack, Deque}} \]

The goal is to recognize:

- when order of processing matters,
- when we need the most recent unresolved item,
- when we need the oldest pending item,
- when we need next greater/smaller,
- when we need max/min inside a moving window.

---

1. STACK

A Stack follows:

\[\boxed{\text{LIFO}} \]

Last In, First Out

Example:

```text
push 10
push 20
push 30
```

Stack:

```text
TOP
  !“
30
20
10
```

`pop()` removes:

\[30 \]

first.

---

2. STACK OPERATIONS

Operation	Meaning	Time
Push	Insert at top	\(O(1)\)
Pop	Remove top	\(O(1)\)
Peek/Top	View top	\(O(1)\)
Empty check	Is stack empty?	\(O(1)\)

---

3. WHEN TO THINK STACK

Trigger words:

- nested
- matching
- most recent
- undo
- backtracking
- next greater
- previous greater
- next smaller
- expression evaluation

Think:

\[\boxed{\text{Stack}} \]

---

4. QUEUE

A Queue follows:

\[\boxed{\text{FIFO}} \]

First In, First Out

Example:

```text
enqueue 10
enqueue 20
enqueue 30
```

Queue:

```text
FRONT           REAR
  !“                                !“
10   20   30
```

`dequeue()` removes:

\[10 \]

first.

---

5. QUEUE OPERATIONS

Operation	Meaning	Time
Enqueue	Insert rear	\(O(1)\)
Dequeue	Remove front	\(O(1)\)
Front	See oldest item	\(O(1)\)
Empty	Check empty	\(O(1)\)

Typical placement use:

$$ \boxed{\text{BFS}} $$

---

6. STACK VS QUEUE

Stack	Queue
LIFO	FIFO
Most recent first	Oldest first
DFS-like behavior	BFS-like behavior
Parentheses	Level traversal
Undo	Scheduling

Memory:

> **Stack = Plates**

> **Queue = People in line**

PART A — VALID PARENTHESES

7. PROBLEM

Input:

```text
"({[]})"
```

Valid.

Input:

```text
"([)]"
```

Invalid.

Why Stack?

Because closing bracket must match the:

$$ \boxed{\text{most recently opened unmatched bracket}} $$

That's LIFO.

---

8. ALGORITHM

For opening:

```text
( [ {
```

push.

For closing:

```text
) ] }
```

check top.

Example:

```text
(
[
]
)
```

Stack evolution:

```text
(
( [
(
empty
```

Valid.

---

9. CODE

```python
def valid_parentheses(s):
    stack = []
    match = {')': '(', ']': '[', '}': '{'}

    for ch in s:
        if ch in "([{":
            stack.append(ch)
        else:
            if not stack or stack[-1] != match[ch]:
                return False
            stack.pop()

    return not stack
```

Time:

\[\boxed{O(n)} \]

Space:

$$ \boxed{O(n)} $$

PART B — MIN STACK

10. PROBLEM

Design stack supporting:

```text
push
pop
top
getMin
```

with all operations:

$$ \boxed{O(1)} $$

---

11. NAIVE APPROACH

To get minimum:

> **scan whole stack.**

That costs:

$$ O(n) $$

Not acceptable.

---

12. OPTIMAL IDEA

Maintain another stack:

```text
main stack
min stack
```

Example push:

```text
5
3
7
2
```

Main:

```text
2
7
```

```
3
5
```

Min stack can track:

```TEXT
2
3
3
5
```

or store only new minima depending implementation.

Then:

\[getMin=O(1) \]

---

13. SIMPLE PAIR APPROACH

Store:

```TEXT
(value, current_min)
```

Example:

```TEXT
 p u s h ( 5 )  !'   ( 5 , 5 )
 p u s h ( 3 )  !'   ( 3 , 3 )
 p u s h ( 7 )  !'   ( 7 , 3 )
 p u s h ( 2 )  !'   ( 2 , 2 )
```

Top always knows minimum.

PART C — NEXT GREATER ELEMENT

One of the most important Stack patterns.

14. PROBLEM

Input:

```TEXT
[2,1,2,4,3]
```

For each element find first greater element to its right.

Answers:

```TEXT
 2  !'   4
 1  !'   2
```

```
2  !'   4
4  !'   - 1
3  !'   - 1
```

Result:

```text
[4,2,4,-1,-1]
```

---

15. BRUTE FORCE

For every element, scan right.

Worst:

\[O(n^2) \]

---

16. MONOTONIC STACK

Maintain a stack whose elements follow an ordered pattern.

For Next Greater, one common method scans from right.

Maintain decreasing stack of candidate values.

For each current value \(x\):

1. Remove values \(\le x\).
2. Top becomes next greater.
3. Push \(x\).

---

17. TRACE

Input:

```text
[2,1,2,4,3]
```

Scan right to left.

3

Stack empty.

Answer:

```text
-1
```

Push 3.

4

Pop 3 because:

\[3\le4 \]

Stack empty.

Answer:

```
TEXT
-1
```

Push 4.

2

Top:

\[4 \]

Answer:

\[4 \]

Push 2.

1

Top:

\[2 \]

Answer:

\[2 \]

Push 1.

2

Pop:

```
TEXT
1
2
```

Top:

\[4 \]

Answer:

\[4 \]

Final:

```
TEXT
[4,2,4,-1,-1]
```

18. CODE

```python
def next_greater(a):
    stack = []
    ans = [-1] * len(a)

    for i in range(len(a) - 1, -1, -1):
        while stack and stack[-1] <= a[i]:
            stack.pop()

        if stack:
            ans[i] = stack[-1]

        stack.append(a[i])

    return ans
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(n)} $$

19. WHY IS IT O(n), NOT O(n²)?

There is a `while` loop inside a `for` loop.

But every element is:

- pushed once,
- popped at most once.

Total stack operations:

$$ O(n) $$

This is a classic amortized-analysis interview question.

20. MONOTONIC STACK SIGNAL

If question asks:

> **next greater**

> **next smaller**

> **previous greater**

> **previous smaller**

think immediately:

\[\boxed{\text{Monotonic Stack}} \]

PART D — PREVIOUS SMALLER ELEMENT

21. EXAMPLE

Input:

```text
[4,5,2,10,8]
```

Previous smaller:

```text
 4    !'   - 1
 5    !'   4
 2    !'   - 1
 1 0  !'   2
 8    !'   2
```

Output:

```text
[-1,4,-1,2,2]
```

Scan left to right.

Remove stack values:

\[\ge current \]

Top becomes previous smaller.

PART E — DAILY TEMPERATURES

Classic interview problem.

22. PROBLEM

Temperatures:

```text
[73,74,75,71,69,72,76,73]
```

For every day, find number of days until a warmer temperature.

Output:

```text
[1,1,4,2,1,1,0,0]
```

23. PATTERN

This is:

> **Next Greater Element, but return distance between indices.**

Therefore store indexes in stack, not values.

24. ALGORITHM

Scan left to right.

Maintain indices of unresolved temperatures.

If current temperature is greater than temperature at stack top:

> **current day resolves that earlier day.**

```python
def daily_temperatures(t):
    ans = [0] * len(t)
    stack = []

    for i in range(len(t)):
        while stack and t[i] > t[stack[-1]]:
            j = stack.pop()
            ans[j] = i - j

        stack.append(i)

    return ans
```

Time:

$$\boxed{O(n)}$$

25. WHY STORE INDEX?

Because answer asks:

$$\boxed{\text{distance}}$$

not just next greater temperature.

General rule:

> **If final answer needs position/distance, store indexes.**

PART F — LARGEST RECTANGLE IN HISTOGRAM

Important medium/hard monotonic-stack problem.

26. PROBLEM

Heights:

```text
[2,1,5,6,2,3]
```

Maximum rectangle area:

$$ \boxed{10} $$

using heights 5 and 6:

```text
5,6
```

minimum height:

$$ 5 $$

width:

$$ 2 $$

area:

$$ 10 $$

---

27. CORE IDEA

For each bar, determine:

- first smaller bar on left,
- first smaller bar on right.

Then width over which bar can extend.

Area:

$$ height \times width $$

Monotonic Stack makes this:

$$ O(n) $$

instead of:

$$ O(n^2) $$

---

28. INTERVIEW SIGNAL

Histogram + nearest smaller:

$$ \boxed{\text{Monotonic Stack}} $$

This question is a very common extension of Next Smaller Element.

PART G — QUEUE USING TWO STACKS

29. PROBLEM

Implement Queue using only stacks.

Need FIFO from LIFO structures.

Use:

```text
input stack
output stack
```

---

30. ENQUEUE

Push into:

```text
input
```

Example:

```text
input = [1,2,3]
```

---

31. DEQUEUE

If output empty:

move everything from input to output.

```text
input:
1 2 3

 m o v e  !'

output:
3
2
1
```

Now pop output:

\[1 \]

FIFO restored.

---

32. COMPLEXITY

A single transfer may cost:

$$ O(n) $$

but each element moves from input to output only once.

Therefore amortized:

$$ \boxed{O(1)} $$

per operation.

This is another important amortized-analysis example.

PART H — STACK USING QUEUES

33. PROBLEM

Implement LIFO using FIFO queue(s).

One common method:

After pushing a new element:

> **rotate previous elements behind it.**

Example queue before:

```text
1,2
```

Push 3:

```text
1,2,3
```

Rotate first two:

```text
3,1,2
```

Now front acts as stack top.

Push may cost:

$$ O(n) $$

while pop:

$$ O(1) $$

PART I — CIRCULAR QUEUE

34. PROBLEM WITH NORMAL ARRAY QUEUE

Suppose fixed array:

```text
[10,20,30,_,_]
```

After removing:

```text
10,20
```

front moves forward.

There may be unused cells at beginning.

Circular Queue reuses them.

---

35. MODULO FORMULA

For capacity \(N\):

```text
next index = (index + 1) % N
```

Example capacity 5:

From:

\[4 \]

next:

\[(4+1)\bmod5=0 \]

Wrap around.

---

36. CIRCULAR QUEUE APPLICATIONS

- buffers
- CPU scheduling
- streaming
- producer-consumer systems
- fixed-size queues

PART J — DEQUE

37. WHAT IS DEQUE?

Deque = Double Ended Queue.

Supports:

```text
insert front
insert back
remove front
remove back
```

all efficiently.

Pronounced:

> **“deck”**

---

38. WHY DEQUE MATTERS

Most important placement use:

\[\boxed{\text{Sliding Window Maximum}} \]

PART K — SLIDING WINDOW MAXIMUM

39. PROBLEM

Input:

```text
nums = [1,3,-1,-3,5,3,6,7]
k = 3
```

Windows:

```text
 [ 1 , 3 , - 1 ]      !’   3
 [ 3 , - 1 , - 3 ]    !’   3
 [ - 1 , - 3 , 5 ]    !’   5
 [ - 3 , 5 , 3 ]      !’   5
 [ 5 , 3 , 6 ]        !’   6
 [ 3 , 6 , 7 ]        !’   7
```

Output:

```text
[3,3,5,5,6,7]
```

---

40. BRUTE FORCE

For every window, find max.

\[O(nk) \]

Can we do:

$$ O(n)? $$

Yes.

Use:

$$ \boxed{\text{Monotonic Deque}} $$

---

41. DEQUE INVARIANT

Store indexes in decreasing order of values.

Therefore:

$$ nums[deque[0]] $$

is always maximum of current window.

---

42. FOR EACH INDEX i

Step 1

Remove indexes outside window.

If:

$$ deque[0]\le i-k $$

remove front.

Step 2

While current value is greater than or equal to values at deque back:

> **pop back.**

Why?

Those smaller values can never become maximum while current larger value remains in the window.

Step 3

Append current index.

Step 4

Once window size reaches k:

front gives maximum.

---

43. CODE

```python
from collections import deque

def max_sliding_window(nums, k):
    dq = deque()
    ans = []

    for i in range(len(nums)):

        while dq and dq[0] <= i - k:
            dq.popleft()

        while dq and nums[dq[-1]] <= nums[i]:
            dq.pop()

        dq.append(i)

        if i >= k - 1:
            ans.append(nums[dq[0]])

    return ans
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(k)} $$

---

44. WHY O(n)?

Each index:

- enters deque once,
- leaves deque at most once.

Therefore total:

$$ O(n) $$

Again amortized reasoning.

PART L — MONOTONIC STACK VS MONOTONIC DEQUE

Problem	Structure
Next Greater	Monotonic Stack
Previous Smaller	Monotonic Stack
Daily Temperatures	Monotonic Stack
Histogram	Monotonic Stack
Sliding Window Maximum	Monotonic Deque

Memory:

Nearest greater/smaller !' Stack

Window max/min !' Deque

PART M — INFIX / POSTFIX BASICS

45. EXPRESSIONS

Infix:

```text
A + B
```

Postfix:

```text
A B +
```

Prefix:

```text
+ A B
```

Stacks are commonly used for:

- i n f i x!' p o s t f i x conversion,
- postfix evaluation,
- expression parsing.

---

46. POSTFIX EVALUATION

Expression:

```text
2 3 + 4 *
```

Push:

```text
2
3
```

Encounter `+`:

pop:

\[2,3 \]

compute:

\[5 \]

push 5.

Push 4.

Encounter `*`:

\[5\times4=20 \]

Answer:

\[\boxed{20} \]

PART N — REMOVE ADJACENT DUPLICATES

47. PROBLEM

Input:

```text
"abbaca"
```

Remove adjacent duplicate pairs repeatedly.

Process:

```text
abbaca
!“
aaca
!“
ca
```

Output:

```text
"ca"
```

---

48. STACK APPROACH

For each character:

If top equals current:

> **pop.**

Else:

> **push.**

Time:

\[O(n) \]

This is another "most recent unresolved element" pattern.

PART O — BACKSPACE STRING COMPARE

49. PROBLEM

`#` means backspace.

```text
"ab#c"
```

becomes:

```text
"ac"
```

Stack is a natural first approach.

For normal char:

> **push.**

For `#`:

> **pop if possible.**

There is also an \(O(1)\)-space reverse two-pointer solution as follow-up.

PART P — STOCK SPAN

50. PROBLEM

Prices:

```text
[100,80,60,70,60,75,85]
```

Span asks number of consecutive previous days with price:

\[\le current \]

Output:

```text
[1,1,1,2,1,4,6]
```

This is another:

\[\boxed{\text{Monotonic Stack}} \]

problem.

Think:

> **Previous greater element.**

PART Q — STACK / QUEUE RECOGNITION MAP

51. MASTER TABLE

Signal	Pattern
Matching brackets	Stack
Undo / most recent	Stack
Next greater	Monotonic Stack
Next smaller	Monotonic Stack
Daily Temperatures	Monotonic Stack
Histogram	Monotonic Stack
BFS / level order	Queue
Scheduling oldest first	Queue
Sliding window max/min	Deque
Fixed circular buffer	Circular Queue
Expression evaluation	Stack

PART R — COMMON INTERVIEW TRAPS

52. STACK EMPTY CHECK

Before:

```text
stack[-1]
```

always ensure:

```text
stack not empty
```

---

53. QUEUE IN PYTHON

Do not repeatedly use:

```python
list.pop(0)
```

for large queues.

It can cost:

\[O(n) \]

because remaining elements shift.

Use:

```python
collections.deque
```

where:

```text
append     O(1)
popleft    O(1)
```

---

54. MONOTONIC STACK DOES NOT MEAN SORTED INPUT

Input itself may be random.

The **stack** maintains monotonic order.

Example input:

```text
5,1,7,3
```

still valid.

---

55. STORE VALUES OR INDICES?

Store values if answer needs:

> **next greater value.**

Store indices if answer needs:

- distance,
- number of days,
- window boundaries,
- positions.

This is a powerful decision rule.

---

56. WHILE INSIDE FOR DOES NOT ALWAYS MEAN $O(n^2)$

Monotonic Stack and Deque are classic examples.

If every item can be removed only once:

$$\boxed{O(n)}$$

PART S — INTERVIEW EXPLANATION EXAMPLE

Suppose interviewer asks Daily Temperatures.

Strong explanation:

> **"The brute-force approach checks future days for each temperature, which can take $O(n^2)$. I can use a monotonic decreasing stack of unresolved day indices. When I encounter a warmer temperature, I pop all smaller temperatures from the stack; for each popped index, the difference in indices gives the wait. Every index is pushed once and popped at most once, so the total time is $O(n)$, with $O(n)$ extra space."**

That is a strong placement explanation.

PART T — MUST-DO QUESTIONS

Foundation

1. Implement Stack
2. Implement Queue
3. Valid Parentheses
4. Queue Using Stacks
5. Stack Using Queues
6. Circular Queue

Core Placement

7. Min Stack
8. Next Greater Element
9. Previous Smaller Element
10. Daily Temperatures
11. Remove Adjacent Duplicates
12. Backspace String Compare
13. Stock Span

Medium/Advanced

14. Largest Rectangle in Histogram
15. Sliding Window Maximum
16. Next Greater Element II
17. Asteroid Collision

18. Decode String
19. Evaluate Reverse Polish Notation
20. Basic Calculator

PART U — PLACEMENT PRACTICE SET

Q1 — MCQ

Stack follows:

A. FIFO B. LIFO C. Random order D. Priority order

Q2 — MCQ

Queue follows:

A. LIFO B. FIFO C. Heap order D. Sorted order

Q3 — MCQ

Valid Parentheses is most naturally solved using:

A. Stack B. Queue C. Heap D. Union-Find

Q4 — MSQ

Which strongly suggest a Monotonic Stack?

A. Next Greater Element

B. Daily Temperatures

C. Previous Smaller Element

D. BFS

Q5 — MCQ

Sliding Window Maximum is optimally associated with:

A. Monotonic Deque B. DFS C. Binary Search Tree D. Union-Find

Q6 — MCQ

Why is Next Greater Element with a monotonic stack $O(n)$ even though it has nested loops?

A. Stack has maximum size 1.

B. Each element is pushed once and popped at most once.

C. The input is always sorted.

D. Inner loop never executes.

Q7 — NAT

For:

```text
[4,5,2,10,8]
```

what is the previous smaller element for `10`?

Q8 — NAT

Evaluate postfix:

```text
2 3 + 4 *
```

Q9 — MSQ

Which statements are correct?

A. BFS uses a queue.

B. DFS naturally uses a stack/recursion.

C. `deque.popleft()` is suitable for queue operations.

D. `list.pop(0)` is always \(O(1)\) in Python.

Q10 — MCQ

If answer requires number of positions between current element and next greater element, monotonic stack should usually store:

A. only values B. indices C. strings D. graph edges

Q11 — NAT

For:

```text
[2,1,2,4,3]
```

what is the next greater value for the first `2`?

Q12 — MSQ

Which are typical \(O(n)\) amortized solutions?

A. Next Greater Element with Monotonic Stack

B. Daily Temperatures with Monotonic Stack

C. Sliding Window Maximum with Monotonic Deque

D. Naively checking every pair

PRACTICE SOLUTIONS

Q1

$$\boxed{B}$$

Q2

$$\boxed{B}$$

Q3

$$\boxed{A}$$

Q4

A, B, C.

$$\boxed{A,B,C}$$

Q5

$$\boxed{A}$$

Q6

Each element enters and leaves at most once.

$$\boxed{B}$$

Q7

Previous smaller before 10:

```text
2
```

$$\boxed{2}$$

Q8

\[(2+3)\times4=20 \]

\[\boxed{20} \]

Q9

A true.

B true.

C true.

D false.

\[\boxed{A,B,C} \]

Q10

Distance requires positions.

\[\boxed{B} \]

Q11

First greater to right of first 2:

\[4 \]

\[\boxed{4} \]

Q12

A, B, C.

\[\boxed{A,B,C} \]

Ø=ÑÓTES TO WRITE

```text
STACK = LIFO
QUEUE = FIFO

Stack signals:
parentheses
undo
most recent
next greater/smaller
expressions

Queue signals:
BFS
level order
```

```
oldest pending

MONOTONIC STACK:
next greater
next smaller
previous greater
previous smaller
daily temperatures
histogram

MONOTONIC DEQUE:
sliding window maximum/minimum

Important:
Store INDEX if answer needs distance.

Amortized O(n):
each item pushed once,
popped at most once.
```

COMPLEXITIES TO MEMORIZE

Operation/Problem	Complexity
Stack push/pop	\(O(1)\)
Queue enqueue/dequeue	\(O(1)\)
Valid Parentheses	\(O(n)\)
Min Stack operations	\(O(1)\)
Next Greater	\(O(n)\)
Daily Temperatures	\(O(n)\)
Stock Span	\(O(n)\)
Histogram	\(O(n)\)
Sliding Window Maximum	\(O(n)\)
Queue via two stacks	amortized \(O(1)\)

&β30-SECOND REVISION

1. Stack = LIFO.
2. Queue = FIFO.
3. Parentheses !' Stack.
4. BFS !' Queue.
5. Next greater/smaller !' Monotonic Stack.
6. Daily Temperatures = Next Greater with distance.
7. Need distance !' store indices.

8. Histogram !’ nearest smaller boundaries.
9. Deque supports both ends.
10. Sliding Window Maximum !’ Monotonic Deque.
11. Min Stack !’ track current minimum.
12. Queue using stacks can have amortized \(O(1)\).
13. Circular Queue uses modulo.
14. `deque.popleft()` is efficient for Python queues.
15. Nested `while` in monotonic stack can still be \(O(n)\).
16. Each element pushed once/popped once !’ linear total.

' Placement Module 5 Complete

Next Placement Module:

MODULE 6 — BINARY SEARCH + SEARCH-ON-ANSWER PATTERNS

We’ll cover:

- standard Binary Search,
- first/last occurrence,
- lower/upper bound,
- count occurrences,
- Search Insert Position,
- floor/ceil,
- rotated sorted arrays,
- find minimum in rotated array,
- peak element,
- binary search on monotonic answers,
- square root,
- Koko Eating Bananas-type problems,
- minimum feasible capacity problems,
- and the most important Binary Search interview traps.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 6 — BINARY SEARCH + SEARCH-ON-ANSWER PATTERNS

Placement Priority: & & & & &

Binary Search is much more than:

> **“Find an element in a sorted array.”**

In placements, Binary Search appears in two major forms:

$$ \boxed{\text{Binary Search on Array}} $$

and

$$ \boxed{\text{Binary Search on Answer}} $$

The second one is where many interview candidates get stuck.

---

1. CORE IDEA OF BINARY SEARCH

Binary Search works when the search space has an order or monotonic structure.

Basic sorted array:

```text
[1, 3, 5, 7, 9, 11]
```

Target:

$$ 7 $$

Instead of checking all elements, check middle.

Every comparison eliminates roughly half the search space.

Therefore:

$$ \boxed{O(\log n)} $$

---

2. STANDARD BINARY SEARCH

Use:

```text
low = 0
high = n - 1
```

While:

```text
low <= high
```

compute:

$$ mid=low+\frac{high-low}{2} $$

---

3. WHY NOT `(low + high) / 2`?

In fixed-width languages, this may overflow:

```text
low + high
```

Safer:

$$ \boxed{ mid=low+(high-low)/2 } $$

4. STANDARD CODE

```python
def binary_search(a, target):
    low = 0
    high = len(a) - 1

    while low <= high:
        mid = low + (high - low) // 2

        if a[mid] == target:
            return mid

        elif a[mid] < target:
            low = mid + 1

        else:
            high = mid - 1

    return -1
```

Time:

$$ \boxed{O(\log n)} $$

Space:

$$ \boxed{O(1)} $$

5. FIRST INTERVIEW TRAP — LOOP CONDITION

For closed interval:

$$ [low,high] $$

use:

```text
while low <= high
```

Because:

```text
low == high
```

still represents one valid candidate.

6. BINARY SEARCH MEMORY RULE

```TEXT
 t a r g e t   >   m i d d l e  !'   g o   R I G H T
 t a r g e t   <   m i d d l e  !'   g o   L E F T
```

Meaning:

```TEXT
 r i g h t  !'   l o w   =   m i d   +   1
 l e f t    !'   h i g h   =   m i d   -   1
```

PART A — FIRST OCCURRENCE

7. PROBLEM

Sorted array:

```TEXT
[1,2,2,2,3,4]
```

Target:

\[2 \]

Ordinary Binary Search might return:

```TEXT
index 2
```

But first occurrence is:

\[\boxed{1} \]

8. KEY IDEA

When target found:

> **Do not stop.**

Store:

```TEXT
answer = mid
```

and continue searching:

\[\boxed{\text{LEFT}} \]

9. CODE

```python
def first_occurrence(a, target):
    low = 0
    high = len(a) - 1
    ans = -1

    while low <= high:
        mid = low + (high - low) // 2

        if a[mid] == target:
            ans = mid
            high = mid - 1

        elif a[mid] < target:
            low = mid + 1

        else:
            high = mid - 1

    return ans
```

Memory:

> **Found + First !' go Left**

PART B — LAST OCCURRENCE

10. PROBLEM

Same array:

```text
[1,2,2,2,3,4]
```

Last occurrence:

\[\boxed{3} \]

When found:

> **save and go right.**

11. CODE

```python
def last_occurrence(a, target):
    low = 0
    high = len(a) - 1
```

```
    ans = -1

    while low <= high:
        mid = low + (high - low) // 2

        if a[mid] == target:
            ans = mid
            low = mid + 1

        elif a[mid] < target:
            low = mid + 1

        else:
            high = mid - 1

    return ans
```

Memory:

> **Found + Last !' go Right**

---

12. COUNT OCCURRENCES

If:

```text
first = f
last = l
```

then count:

$$ \boxed{l-f+1} $$

Example:

```text
first = 1
last = 3
```

$$ 3-1+1=3 $$

PART C — LOWER BOUND

13. DEFINITION

Lower Bound of `x`:

$$ \boxed{\text{first index }i\text{ such that }A[i]\ge x} $$

Example:

```text
A = [1,3,3,5,7]
x = 3
```

First element \(\ge3\):

```text
index 1
```

So:

\[\boxed{LB(3)=1} \]

---

14. ANOTHER EXAMPLE

```text
A = [1,3,3,5,7]
x = 4
```

First value:

\[\ge4 \]

is 5.

Index:

\[\boxed{3} \]

---

15. LOWER BOUND CODE

```python
def lower_bound(a, x):
    low = 0
    high = len(a) - 1
    ans = len(a)

    while low <= high:
        mid = low + (high - low) // 2

        if a[mid] >= x:
            ans = mid
            high = mid - 1
        else:
            low = mid + 1

    return ans
```

Core condition:

\[\boxed{A[mid]\ge x} \]

When true:

> **store candidate, move left.**

PART D — UPPER BOUND

16. DEFINITION

Upper Bound of `x`:

$$ \boxed{\text{first index }i\text{ such that }A[i]>x} $$

Example:

```text
A = [1,3,3,3,5,7]
x = 3
```

First value >3 is 5.

Index:

$$ \boxed{4} $$

17. UPPER BOUND CONDITION

Lower bound:

$$ A[mid]\ge x $$

Upper bound:

$$ A[mid]>x $$

Only one symbol changes.

18. CODE

```python
def upper_bound(a, x):
    low = 0
    high = len(a) - 1
    ans = len(a)

    while low <= high:
        mid = low + (high - low) // 2

        if a[mid] > x:
            ans = mid
            high = mid - 1
        else:
            low = mid + 1

    return ans
```

19. COUNT USING BOUNDS

For sorted array:

$$ \boxed{ count(x)=upperBound(x)-lowerBound(x) } $$

Example:

```
TEXT
[1,3,3,3,5]
```

For 3:

```
TEXT
LB = 1
UB = 4
```

Count:

$$ 4-1=3 $$

PART E — SEARCH INSERT POSITION

20. PROBLEM

Input:

```
TEXT
[1,3,5,6]
```

Target:

$$ 5 $$

Answer:

$$ 2 $$

Target:

$$ 2 $$

Insert at:

$$ 1 $$

Target:

$$ 7 $$

Insert at:

$$ 4 $$

This is exactly:

$$ \boxed{\text{Lower Bound}} $$

PART F — FLOOR AND CEIL

21. FLOOR

Floor of \(x\):

> **greatest array value \(\le x\)**

Example:

```
TEXT
[1,3,5,7]
x = 6
```

Floor:

\[\boxed{5} \]

---

22. CEIL

Ceil of \(x\):

> **smallest array value \(\ge x\)**

For:

\[x=6 \]

Ceil:

\[\boxed{7} \]

---

23. MEMORY

```
TEXT
 F l o o r  !'  b i g g e s t  "d  x
 C e i l   !'  s m a l l e s t  "e  x
```

Ceil is closely related to:

\[\boxed{\text{Lower Bound}} \]

PART G — SEARCH IN ROTATED SORTED ARRAY

One of the most important placement Binary Search questions.

24. PROBLEM

Originally sorted:

```text
[0,1,2,4,5,6,7]
```

Rotated:

```text
[4,5,6,7,0,1,2]
```

Find:

\[0 \]

Answer index:

\[\boxed{4} \]

---

25. KEY OBSERVATION

After rotation, at every binary-search step:

\[\boxed{\text{at least one half is sorted}} \]

Example:

```text
[4,5,6,7,0,1,2]
```

Middle:

```text
7
```

Left half:

```text
4,5,6,7
```

sorted.

Right:

```text
0,1,2
```

also sorted here, but in general one side is guaranteed sorted.

---

26. HOW TO IDENTIFY SORTED HALF

If:

\[A[low]\le A[mid] \]

then left half is sorted.

Otherwise:

> **right half is sorted.**

---

27. IF LEFT HALF SORTED

Check whether target lies inside:

$$ A[low]\le target<A[mid] $$

If yes:

> **search left.**

Otherwise:

> **search right.**

---

28. CODE

```python
def search_rotated(a, target):
    low = 0
    high = len(a) - 1

    while low <= high:
        mid = low + (high - low) // 2

        if a[mid] == target:
            return mid

        if a[low] <= a[mid]:
            if a[low] <= target < a[mid]:
                high = mid - 1
            else:
                low = mid + 1

        else:
            if a[mid] < target <= a[high]:
                low = mid + 1
            else:
                high = mid - 1

    return -1
```

Time:

$$ \boxed{O(\log n)} $$

assuming distinct elements.

---

29. DUPLICATE TRAP

If rotated array contains many duplicates:

```text
[1,1,1,1,1,0,1]
```

it may become difficult to determine which half is sorted.

Sometimes you must shrink boundaries:

```text
low++
high--
```

Worst case can degrade to:

$$ \boxed{O(n)} $$

PART H — FIND MINIMUM IN ROTATED SORTED ARRAY

30. PROBLEM

```text
[4,5,6,7,0,1,2]
```

Minimum:

$$ \boxed{0} $$

---

31. KEY OBSERVATION

Compare middle with:

$$ A[high] $$

If:

$$ A[mid]>A[high] $$

minimum lies to the right of mid.

So:

```text
low = mid + 1
```

Otherwise:

$$ A[mid]\le A[high] $$

minimum is at mid or left.

So:

```text
high = mid
```

32. CODE

```python
def find_min_rotated(a):
    low = 0
    high = len(a) - 1

    while low < high:
        mid = low + (high - low) // 2

        if a[mid] > a[high]:
            low = mid + 1
        else:
            high = mid

    return a[low]
```

Note loop:

```text
low < high
```

because when they meet:

> **answer found.**

PART I — FIND PEAK ELEMENT

33. PROBLEM

A peak is an element greater than its neighbors.

Example:

```text
[1,2,3,1]
```

Peak:

\[3 \]

index:

\[2 \]

34. BINARY SEARCH IDEA

Compare:

$$ A[mid] $$

with:

$$ A[mid+1] $$

If:

$$ A[mid]<A[mid+1] $$

you are on an upward slope.

A peak exists on right.

So:

```text
low = mid + 1
```

Otherwise:

> **peak exists at mid or left.**

```text
high = mid
```

---

35. CODE

```python
def find_peak(a):
    low = 0
    high = len(a) - 1

    while low < high:
        mid = low + (high - low) // 2

        if a[mid] < a[mid + 1]:
            low = mid + 1
        else:
            high = mid

    return low
```

Time:

$$ \boxed{O(\log n)} $$

PART J — INTEGER SQUARE ROOT

36. PROBLEM

Find:

$$ \lfloor\sqrt n\rfloor $$

Example:

$$ n=17 $$

Answer:

$$ \boxed{4} $$

because:

$$ 4^2\le17 $$

but:

$$ 5^2>17 $$

---

37. SEARCH SPACE

Potential answer lies between:

$$ 0 $$

and:

$$ n $$

or tighter:

$$ 0\text{ to }n/2+1 $$

Binary Search over possible answer.

---

38. MONOTONIC CONDITION

For candidate x:

```
TEXT
x*x <= n
```

For small x:

```
TEXT
True
```

Eventually becomes:

```
TEXT
False
```

Pattern:

```
TEXT
T T T T F F F
```

We want:

$$ \boxed{\text{last True}} $$

PART K — THE BIG IDEA: BINARY SEARCH ON ANSWER

This is one of the most important placement concepts.

39. NORMAL BINARY SEARCH

Search among array values/indexes.

Example:

```text
Is target 17 present?
```

---

40. SEARCH ON ANSWER

We don't have sorted array.

Instead, possible answers themselves form a numeric range.

Example:

> **What is the minimum speed needed to finish work within H hours?**

Possible speeds:

```text
1,2,3,4,5,6,...
```

For each speed ask:

```text
Can I finish within H hours?
```

The answer might look like:

```text
False False False True True True ...
```

Once it becomes feasible, every larger speed remains feasible.

That monotonic property allows Binary Search.

---

41. THE GOLDEN CONDITION

Binary Search on Answer requires a predicate:

$$ \boxed{P(x)} $$

that changes monotonically:

Either:

```text
F F F F T T T T
```

or:

```text
T T T T F F F F
```

Then find transition boundary.

---

42. MOST IMPORTANT RECOGNITION SIGNAL

If question says:

> **minimum possible maximum**

> **maximum possible minimum**

> **smallest capacity**

> **minimum speed**

> **minimum days**

> **maximum feasible value**

think:

\[\boxed{\text{Binary Search on Answer}} \]

especially when feasibility for a guessed answer can be checked efficiently.

PART L — KOKO EATING BANANAS TYPE

43. PROBLEM IDEA

Pile sizes:

```text
[3,6,7,11]
```

Need finish in:

\[H=8 \]

hours.

Find minimum eating speed k.

44. FEASIBILITY CHECK

At speed k, pile p needs:

$$ \left\lceil\frac{p}{k}\right\rceil $$

hours.

Total:

$$ \sum \left\lceil \frac{p_i}{k} \right\rceil $$

If total:

$$ \le H $$

speed is feasible.

45. SEARCH RANGE

Minimum possible speed:

$$ 1 $$

Maximum needed:

$$ \max(piles) $$

So binary search:

$$ [1,\max(piles)] $$

46. MONOTONICITY

Small speed:

> **maybe impossible.**

Larger speed:

> **easier.**

Pattern:

```TEXT
F F F T T T T
```

We want:

$$ \boxed{\text{first True}} $$

47. CEILING DIVISION

Instead of floating point:

$$ \left\lceil\frac{p}{k}\right\rceil $$

use:

$$ \boxed{ \frac{p+k-1}{k} } $$

with integer division.

Very useful interview formula.

PART M — SHIP PACKAGES WITHIN D DAYS TYPE

48. PROBLEM

Weights:

```text
[1,2,3,4,5,6,7,8,9,10]
```

Need ship in D days.

Find minimum ship capacity.

---

49. ANSWER RANGE

Capacity cannot be smaller than largest package:

$$ low=\max(weights) $$

Why?

A package cannot be split.

Maximum possible capacity:

$$ high=\sum weights $$

because carrying everything in one day always works.

---

50. CHECK FUNCTION

Given candidate capacity:

> **simulate loading packages in order.**

If current package doesn't fit:

```text
day++
start new load
```

At end, if days needed:

$$ \le D $$

candidate feasible.

---

51. MONOTONICITY

Small capacity:

```text
Not feasible
```

Large capacity:

```text
Feasible
```

Again:

```text
F F F F T T T
```

Find first True.

PART N — SPLIT ARRAY / PAINTER'S PARTITION PATTERN

52. PROBLEM FAMILY

Typical statements:

> **Split array into k parts so that largest subarray sum is minimized.**

or:

> **Painters paint contiguous boards, minimize completion time.**

or:

> **Allocate books among students, minimize maximum pages.**

These look different.

But underlying pattern:

$$ \boxed{\text{Minimize Maximum}} $$

---

53. SEARCH RANGE

Minimum possible maximum:

$$ \boxed{\max(array)} $$

Maximum:

$$ \boxed{\sum array} $$

Then feasibility:

> **Can we divide into at most k groups with each group sum $\le mid$?**

This is Binary Search on Answer.

---

54. MEMORY

```TEXT
Minimize maximum load
!’   B i n a r y   S e a r c h   A n s w e r

low  = largest single item
high = total sum
```

PART O — AGGRESSIVE COWS TYPE

Another famous Binary Search pattern.

55. PROBLEM IDEA

Positions of stalls:

```TEXT
[1,2,4,8,9]
```

Place k cows such that minimum distance between any two is maximized.

This is:

$$ \boxed{\text{maximize minimum distance}} $$

---

56. SEARCH SPACE

Possible minimum distance:

$$ 1\text{ to }maxPos-minPos $$

For guessed distance d:

> **Can we place all cows while keeping at least distance d?**

---

57. MONOTONICITY

Small distance:

> **feasible.**

Too large distance:

> **infeasible.**

Pattern:

```text
T T T T F F F
```

We want:

$$\boxed{\text{last True}}$$

---

58. CRUCIAL DISTINCTION

Minimize something

Usually seek:

$$\boxed{\text{first feasible}}$$

Pattern:

```text
F F F T T T
```

Maximize something

Usually seek:

$$\boxed{\text{last feasible}}$$

Pattern:

```text
T T T F F F
```

This is a powerful interview shortcut.

PART P — GENERIC FIRST-TRUE TEMPLATE

59. FIRST TRUE

For:

```text
F F F F T T T
```

want first `T`.

```python
def first_true(low, high):
    ans = high

    while low <= high:
        mid = low + (high - low) // 2

        if feasible(mid):
            ans = mid
            high = mid - 1
        else:
            low = mid + 1

    return ans
```

Used for:

- minimum speed
- minimum capacity
- minimum maximum load

---

60. GENERIC LAST-TRUE TEMPLATE

Pattern:

```text
T T T T F F F
```

```python
def last_true(low, high):
    ans = low

    while low <= high:
        mid = low + (high - low) // 2

        if feasible(mid):
            ans = mid
            low = mid + 1
        else:
            high = mid - 1

    return ans
```

Used for:

- maximum minimum distance
- largest feasible threshold

PART Q — BINARY SEARCH TRAPS

61. TRAP 1 — UNSORTED ARRAY

Ordinary Binary Search does not work directly on arbitrary unsorted data.

Need:

\[\boxed{\text{sorted/monotonic structure}} \]

62. TRAP 2 — INFINITE LOOP

Bad updates:

```text
low = mid
```

with:

```text
while low <= high
```

may fail to shrink range.

Safer closed-interval updates:

```text
low = mid + 1
high = mid - 1
```

unless deliberately using another invariant.

63. TRAP 3 — FIRST/LAST OCCURRENCE

Finding target is not enough.

```text
 f i r s t  !'   m o v e   l e f t
 l a s t    !'   m o v e   r i g h t
```

64. TRAP 4 — SEARCH ON ANSWER WITHOUT MONOTONICITY

You cannot Binary Search simply because answer is numerical.

You need:

\[\boxed{\text{monotonic feasibility}} \]

65. TRAP 5 — WRONG SEARCH RANGE

For search-on-answer, finding correct:

```text
low
high
```

is half the solution.

Examples:

Koko

```text
1 ... max pile
```

Ship Capacity

```text
max weight ... sum weights
```

Aggressive Cows

```text
1 ... max position - min position
```

---

66. TRAP 6 — OVERFLOW

Candidates like:

```text
mid * mid
```

can overflow.

Instead compare safely where possible:

```text
mid <= n // mid
```

in fixed-width settings.

---

67. TRAP 7 — DUPLICATES IN ROTATED ARRAY

Distinct-element reasoning may break.

Worst-case:

\[O(n) \]

can occur.

PART R — BINARY SEARCH PATTERN MAP

68. MASTER TABLE

Problem	Pattern
Find target sorted array	Standard BS
First occurrence	Found ! left
Last occurrence	Found ! right
Lower bound	First \(\ge x\)
Upper bound	First \(>x\)
Search Insert Position	Lower bound
Floor	Last \(\le x\)
Ceil	First \(\ge x\)
Rotated Search	Find sorted half
Rotated Minimum	Compare with high
Peak	Follow rising slope
Integer sqrt	Last \(x^2\le n\)
Minimum feasible speed	First True
Minimum capacity	First True
Maximum minimum distance	Last True

PART S — INTERVIEW EXPLANATION EXAMPLE

Suppose interviewer asks:

> **Find minimum ship capacity needed to ship all packages within D days.**

A strong answer:

> **"Trying every capacity would be too slow. The possible capacity lies between the largest single package and the sum of all package weights. For any guessed capacity, I can greedily simulate how many days are needed in \(O(n)\). The feasibility is monotonic: if capacity \(C\) works, every capacity larger than \(C\) also works. So I can Binary Search for the first feasible capacity. This gives \(O(n\log S)\) time, where \(S\) is the numeric search range."**

This is exactly how Search-on-Answer should be explained.

PART T — COMPLEXITY OF SEARCH ON ANSWER

If:

- feasibility check takes $O(n)$,
- answer range size is R,

then Binary Search performs:

$$ O(\log R) $$

checks.

Total:

$$ \boxed{O(n\log R)} $$

Important:

Here R may be based on numeric values, not number of array elements.

PART U — MUST-DO BINARY SEARCH QUESTIONS

Foundation

1. Standard Binary Search
2. First Occurrence
3. Last Occurrence
4. Count Occurrences
5. Lower Bound
6. Upper Bound
7. Search Insert Position
8. Floor/Ceil

Core Placement

9. Search Rotated Sorted Array
10. Find Minimum in Rotated Array
11. Find Peak Element
12. Integer Square Root
13. Single Element in Sorted Array
14. Search 2D Matrix

Search on Answer

15. Koko Eating Bananas
16. Ship Packages Within D Days
17. Split Array Largest Sum
18. Allocate Books
19. Painter's Partition
20. Aggressive Cows
21. Minimum Days to Make Bouquets
22. Smallest Divisor Given Threshold

These problems train the real placement Binary Search skill.

PART V — PLACEMENT PRACTICE SET

Q1 — MCQ

Binary Search on a sorted array has worst-case:

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n\log n)$

Q2 — MCQ

If target is found but we need the first occurrence, we should:

A. return immediately B. continue left C. continue right D. sort again

Q3 — MCQ

Lower Bound of x means:

A. first value $>x$

B. first value $\ge x$

C. last value $<x$

D. last value $>x$

Q4 — MCQ

Upper Bound means:

A. first value $>x$

B. first value $\ge x$

C. last value $\le x$

D. exact match only

Q5 — NAT

For:

```text
[1,2,2,2,4,5]
```

what is the first index of `2`?

Use 0-based indexing.

Q6 — NAT

For the same array, what is the last index of `2`?

Q7 — NAT

How many times does `2` occur?

Q8 — MCQ

In a rotated sorted array with distinct elements, at every search step:

A. neither half is sorted

B. at least one half is sorted

C. both halves are always unsorted

D. Binary Search is impossible

Q9 — MCQ

For Binary Search on Answer, the most important requirement is:

A. HashMap

B. monotonic feasibility condition

C. linked list

D. DFS traversal

Q10 — MSQ

Which commonly suggest Binary Search on Answer?

A. Minimum feasible speed

B. Minimum possible maximum load

C. Maximum possible minimum distance

D. Character frequency

Q11 — MCQ

For minimum ship capacity, a valid lower bound is:

A. 0

B. minimum package weight

C. maximum package weight

D. total weight + 1

Q12 — MCQ

For minimize-maximum partition problems, typical answer range is:

A. 0 to n

B. `min(array)` to `max(array)`

C. `max(array)` to `sum(array)`

D. $-\infty$ to $+\infty$

Q13 — MCQ

Pattern:

```text
F F F T T T
```

To find minimum feasible answer, search for:

A. first True B. last True C. first False D. arbitrary True

Q14 — MCQ

Pattern:

```text
T T T T F F
```

To maximize a feasible answer, search for:

A. first True B. last True C. first False only D. middle True

Q15 — NAT

For:

```text
A = [1,3,5,7]
```

what is the lower-bound index of:

$$ 6 $$

?

PRACTICE SOLUTIONS

Q1

$$ \boxed{B} $$

Q2

First occurrence:

$$ \boxed{\text{go left}} $$

Answer:

$$ \boxed{B} $$

Q3

$$ \boxed{B} $$

Q4

$$ \boxed{A} $$

Q5

Array:

```
TEXT
[1,2,2,2,4,5]
```

First `2`:

$$ \boxed{1} $$

Q6

Last `2`:

$$ \boxed{3} $$

Q7

$$ 3-1+1 = \boxed{3} $$

Q8

$$ \boxed{B} $$

Q9

$$ \boxed{B} $$

Q10

A, B, C.

$$\boxed{A,B,C}$$

Q11

Capacity must fit largest package.

$$\boxed{C}$$

Q12

$$\boxed{C}$$

Q13

Minimum feasible:

$$\boxed{\text{first True}}$$

Answer:

$$\boxed{A}$$

Q14

Maximum feasible:

$$\boxed{\text{last True}}$$

Answer:

$$\boxed{B}$$

Q15

```text
[1,3,5,7]
```

First value ≥6:

```text
7
```

index:

$$\boxed{3}$$

Ø=ÜÓ NOTES TO WRITE

```text
BINARY SEARCH

 S o r t e d  /  m o n o t o n i c  !’  B i n a r y  S e a r c h

Standard:
low = 0
high = n-1
mid = low + (high-low)/2

Found first occurrence:
save + go LEFT

Found last occurrence:
save + go RIGHT

Lower Bound:
first >= x

Upper Bound:
first > x

Count:
UB - LB

Rotated array:
identify sorted half

Search on Answer:
need monotonic predicate

F F F T T T
!’  f i r s t  T r u e
!’  m i n i m u m  f e a s i b l e

T T T F F F
!’  l a s t  T r u e
!’  m a x i m u m  f e a s i b l e

Minimize maximum:
low = max item
high = sum items

Common:
Koko
Ship Capacity
Book Allocation
Painter Partition
Aggressive Cows
```

&β30-SECOND REVISION

1. Binary Search needs sorted/monotonic structure.
2. Time = \(O(\log n)\).
3. Use safe midpoint.
4. First occurrence !’ found then left.
5. Last occurrence !’ found then right.

6. Lower Bound = first \(\ge x\).
7. Upper Bound = first \(>x\).
8. Count = upper " lower.
9. Search Insert Position = Lower Bound.
10. Rotated array !' one half is sorted.
11. Find rotated minimum !' compare mid with high.
12. Peak !' follow upward slope.
13. Search on Answer needs monotonic feasibility.
14. Minimum feasible !' first True.
15. Maximum feasible !' last True.
16. Minimize maximum problems often use `max(array)` to `sum(array)`.
17. Search-on-answer total complexity often \(O(n\log R)\).
18. Correct search boundaries are as important as Binary Search itself.

' Placement Module 6 Complete

Next Placement Module:

MODULE 7 — RECURSION + BACKTRACKING

We'll cover:

- recursion fundamentals,
- call stack,
- base/recursive cases,
- recursion trees,
- subsets,
- subsequences,
- permutations,
- combination sum,
- N-Queens,
- Sudoku,
- Word Search,
- choose/explore/undo pattern,
- pruning,
- backtracking complexity,
- and how to distinguish **Recursion vs Backtracking vs DP**.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 7 — RECURSION + BACKTRACKING

Placement Priority: & & & & &

Recursion and Backtracking are important because many medium-level placement problems are based on:

$$ \boxed{\text{Choose !' Explore !' Undo}} $$

The biggest difficulty is not syntax.

It is learning:

> **what state to carry, what the base case is, what choices are available, and when to backtrack.**

---

1. WHAT IS RECURSION?

Recursion means a function calls itself to solve a smaller version of the same problem.

Example:

$$ 5! = 5\times4\times3\times2\times1 $$

Recursive definition:

$$ n! = n\times(n-1)! $$

Base case:

$$ 0! = 1 $$

---

2. RECURSION HAS TWO ESSENTIAL PARTS

Every recursive solution needs:

1. Base Case

When should recursion stop?

2. Recursive Case

How do I reduce the problem?

Memory:

$$ \boxed{\text{STOP + SHRINK}} $$

---

3. FACTORIAL EXAMPLE

PYTHON

```python
def factorial(n):
    if n == 0:
        return 1

    return n * factorial(n - 1)
```

For:

\[factorial(4) \]

calls:

```
TEXT
factorial(4)
!’   4   ×   f a c t o r i a l ( 3 )
!’   4   ×   3   ×   f a c t o r i a l ( 2 )
!’   4   ×   3   ×   2   ×   f a c t o r i a l ( 1 )
!’   4   ×   3   ×   2   ×   1   ×   f a c t o r i a l ( 0 )
```

Then values return backward.

---

4. CALL STACK

Recursive calls are stored in the call stack.

For:

```
TEXT
factorial(4)
```

stack grows:

```
TEXT
factorial(4)
factorial(3)
factorial(2)
factorial(1)
factorial(0)
```

Then unwinds.

If recursion depth is \(n\):

\[\boxed{\text{Stack Space }=O(n)} \]

even if you did not explicitly create an array.

---

5. FIRST RECURSION TRAP

Recursive code:

```
PYTHON
def f(n):
    if n == 0:
        return
    f(n - 1)
```

Time:

\[O(n) \]

Space:

\[O(n) \]

because of recursion stack.

Interviewers often ask both.

PART A — PRINT 1 TO N USING RECURSION

6. PROBLEM

Print:

```text
1 2 3 4 5
```

for:

\[n=5 \]

One way:

```python
def print_numbers(n):
    if n == 0:
        return

    print_numbers(n - 1)
    print(n)
```

Notice:

> **print happens after recursive call.**

---

7. PRINT N TO 1

```python
def print_reverse(n):
    if n == 0:
        return

    print(n)
    print_reverse(n - 1)
```

Difference:

Before recursion

```text
5 4 3 2 1
```

After recursion

```text
1 2 3 4 5
```

This is a useful recursion-order concept.

PART B — SUM OF N NUMBERS

8. RECURRENCE

\[sum(n)=n+sum(n-1) \]

Base:

\[sum(0)=0 \]

Code:

```python
def total(n):
    if n == 0:
        return 0

    return n + total(n - 1)
```

Time:

\[O(n) \]

Stack:

\[O(n) \]

PART C — FIBONACCI

9. NAIVE RECURSION

\[F(n)=F(n-1)+F(n-2) \]

Code:

```python
def fib(n):
    if n <= 1:
        return n

    return fib(n - 1) + fib(n - 2)
```

Looks simple.

But very inefficient.

---

10. WHY?

For:

\[fib(5) \]

same subproblems repeat:

```text
fib(3)
fib(2)
fib(3)
fib(2)
...
```

Time roughly:

\[\boxed{O(2^n)} \]

This repeated work is exactly why Dynamic Programming becomes useful.

---

11. RECURSION !' DP SIGNAL

If recursion repeatedly computes same state:

\[\boxed{\text{Memoization / DP}} \]

Remember:

> **Recursion describes the choices.**

> **DP avoids repeating them.**

PART D — REVERSE ARRAY RECURSIVELY

12. IDEA

Use:

```text
left
right
```

Swap ends.

Then solve smaller interval:

\[left+1,\ right-1 \]

Code:

```python
def reverse(a, left, right):
    if left >= right:
        return
```

```
    a[left], a[right] = a[right], a[left]

    reverse(a, left + 1, right - 1)
```

Time:

\[O(n) \]

Stack:

\[O(n) \]

PART E — RECURSION VS BACKTRACKING

This distinction is extremely important.

13. RECURSION

Recursion simply means:

> **function calls itself.**

Example:

```TEXT
factorial
binary tree traversal
merge sort
```

---

14. BACKTRACKING

Backtracking means:

> **explore one possible choice, then undo it, then explore another.**

Pattern:

\[\boxed{\text{Choose !' Explore !' Undo}} \]

Used for:

- subsets
- permutations
- combinations
- N-Queens
- Sudoku
- Word Search

---

15. SIMPLE STORY

Imagine a maze.

At a junction:

```text
Left
Right
Straight
```

You choose Left.

If dead end:

> **return to junction.**

Undo that choice.

Try Right.

That's Backtracking.

PART F — SUBSETS

This is one of the most important recursion patterns.

16. PROBLEM

Given:

```text
[1,2,3]
```

generate all subsets.

Possible subsets:

```text
[]
[1]
[2]
[3]
[1,2]
[1,3]
[2,3]
[1,2,3]
```

Number of subsets:

$$\boxed{2^n}$$

because every element has two choices:

```text
take
don't take
```

17. DECISION TREE

For `1`:

```text
Take 1
Don't take 1
```

For `2`:

again:

```text
Take 2
Don't take 2
```

So every element creates two branches.

Depth:

\[n \]

Leaves:

\[2^n \]

18. CODE

```python
def subsets(nums):
    ans = []
    current = []

    def solve(i):
        if i == len(nums):
            ans.append(current.copy())
            return

        current.append(nums[i])
        solve(i + 1)

        current.pop()
        solve(i + 1)

    solve(0)
    return ans
```

19. WHERE IS BACKTRACKING?

This line:

```python
current.pop()
```

undoes:

```python
current.append(nums[i])
```

So:

```text
 C h o o s e   !'  a p p e n d
 E x p l o r e !'  r e c u r s i v e  c a l l
 U n d o       !'  p o p
```

---

20. COMPLEXITY

Number of subsets:

\[2^n \]

Copying each subset may take up to \(O(n)\).

So output-generation complexity:

\[\boxed{O(n2^n)} \]

Space excluding output:

\[O(n) \]

recursion/current path.

PART G — SUBSEQUENCES

21. SUBSET VS SUBSEQUENCE

For array/string:

```text
ABC
```

subsequences include:

```text
""
A
B
C
AB
AC
BC
ABC
```

Again each character:

```text
take
skip
```

Same recursion tree.

---

22. IMPORTANT INTERVIEW SIGNAL

If question says:

> **generate all subsets**

> **generate all subsequences**

think:

$$\boxed{\text{Take / Not Take}}$$

PART H — PERMUTATIONS

23. PROBLEM

Given:

```text
[1,2,3]
```

generate:

```text
123
132
213
231
312
321
```

Number of permutations:

$$\boxed{n!}$$

---

24. DIFFERENCE FROM SUBSETS

Subsets ask:

> **include or exclude?**

Permutations ask:

> **which unused element should go next?**

---

25. USED-ARRAY APPROACH

Maintain:

```text
used[i]
```

At every position:

> **try each unused number.**

---

26. CODE

```python
def permutations(nums):
    ans = []
    current = []
    used = [False] * len(nums)

    def solve():
        if len(current) == len(nums):
            ans.append(current.copy())
            return

        for i in range(len(nums)):
            if used[i]:
                continue

            used[i] = True
            current.append(nums[i])

            solve()

            current.pop()
            used[i] = False

    solve()
    return ans
```

Again:

\[\boxed{\text{Choose !' Explore !' Undo}} \]

---

27. COMPLEXITY

Number of outputs:

\[n! \]

Each output length:

$$ n $$

So roughly:

$$ \boxed{O(n\cdot n!)} $$

PART I — PERMUTATIONS USING SWAPPING

Alternative:

At position `index`, swap each candidate into that position.

Example:

```text
[1,2,3]
```

Position 0 can become:

```text
1
2
3
```

After recursive call:

> **swap back.**

This is also Backtracking.

PART J — COMBINATION SUM

28. PROBLEM IDEA

Candidates:

```text
[2,3,6,7]
```

Target:

$$ 7 $$

Possible combinations:

```text
[2,2,3]
[7]
```

---

29. CORE CHOICE

For candidate at index i:

Take it

Target becomes:

$$ target-candidate $$

If reuse allowed:

> **stay at same index.**

Skip it

Move to:

$$ i+1 $$

---

30. IMPORTANT PRUNING

If:

$$ target<0 $$

stop.

If:

$$ target=0 $$

valid solution found.

This reduces unnecessary exploration.

PART K — WHAT IS PRUNING?

31. DEFINITION

Pruning means:

> **stop exploring a branch when we know it cannot lead to a valid answer.**

Example:

Target:

$$ 7 $$

Current sum already:

$$ 10 $$

and all numbers are positive.

No need to continue.

Stop branch.

---

32. WHY PRUNING MATTERS

Backtracking can be exponential.

Pruning may dramatically reduce practical runtime.

Interviewers often ask:

> **Can you reduce the search space?**

Think:

$$ \boxed{\text{Pruning}} $$

PART L — COMBINATIONS

33. PROBLEM

Generate all combinations choosing:

$$ k $$

numbers from:

$$ 1...n $$

Example:

$$ n=4,\ k=2 $$

Output:

```text
[1,2]
[1,3]
[1,4]
[2,3]
[2,4]
[3,4]
```

Number:

$$ \binom nk $$

---

34. PATTERN

Maintain `start`.

At each level:

```text
for i from start...
```

After choosing i:

> **next recursive call starts at \(i+1\).**

This prevents duplicates and maintains order.

PART M — N-QUEENS

Classic Backtracking problem.

35. PROBLEM

Place \(N\) queens on \(N\times N\) chessboard so that no two queens attack each other.

Queens attack along:

- row
- column
- diagonal

---

36. BACKTRACKING STRATEGY

Place one queen row by row.

For current row:

> **try every column.**

If position safe:

```TEXT
place queen
recurse next row
remove queen
```

---

37. PATTERN

```TEXT
for every possible column:
    if safe:
        choose
        explore
        undo
```

That's pure Backtracking.

---

38. HOW TO CHECK SAFETY

Before placing at:

\[(row,col) \]

check:

- same column
- upper-left diagonal
- upper-right diagonal

Optimized implementations maintain sets for:

```TEXT
columns
row-col diagonal
row+col diagonal
```

This can make safety check:

\[O(1) \]

average.

---

39. DIAGONAL TRICK

Two cells share main diagonal if:

\[row-col \]

same.

They share anti-diagonal if:

\[row+col \]

same.

Very useful interview trick.

PART N — SUDOKU SOLVER

40. PROBLEM

Fill empty Sudoku cells satisfying:

- each row unique 1–9
- each column unique
- each 3×3 box unique

---

41. BACKTRACKING

Find an empty cell.

Try:

```TEXT
1..9
```

For each digit:

If valid:

```text
place
recurse
```

If recursion fails:

```text
remove
```

Memory:

\[\boxed{\text{Try !' Validate !' Recurse !' Undo}} \]

---

42. IMPORTANT SUDOKU PRUNING

Before placing digit:

> **check constraints immediately.**

Do not recursively continue with invalid state.

That's pruning.

PART O — WORD SEARCH

Very common grid Backtracking problem.

43. PROBLEM

Given grid:

```text
A B C E
S F C S
A D E E
```

Can word:

```text
ABCCED
```

be formed by adjacent cells?

Rules often:

- up/down/left/right
- same cell cannot be reused

---

44. BACKTRACKING STATE

You need:

```text
row
col
word_index
visited state
```

At each character:

1. Check bounds.
2. Check character match.
3. Mark current cell used.
4. Explore 4 directions.
5. Unmark before returning.

45. WHY UNMARK?

Because same cell may be used in a different search path.

So:

```text
mark
explore
unmark
```

Again:

\[\boxed{\text{Choose !' Explore !' Undo}} \]

46. WORD SEARCH CODE SKELETON

```python
def exist(board, word):
    rows = len(board)
    cols = len(board[0])

    def dfs(r, c, i):
        if i == len(word):
            return True

        if r < 0 or r >= rows or c < 0 or c >= cols:
            return False

        if board[r][c] != word[i]:
            return False

        temp = board[r][c]
        board[r][c] = '#'

        found = (
            dfs(r + 1, c, i + 1) or
```

```
            dfs(r - 1, c, i + 1) or
            dfs(r, c + 1, i + 1) or
            dfs(r, c - 1, i + 1)
        )

        board[r][c] = temp

        return found

    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0):
                return True

    return False
```

PART P — BACKTRACKING TEMPLATE

47. MASTER TEMPLATE

PYTHON

```python
def backtrack(state):
    if solution_complete:
        save_answer()
        return

    for choice in possible_choices:

        if not valid(choice):
            continue

        make_choice(choice)

        backtrack(new_state)

        undo_choice(choice)
```

Memorize the logic, not exact syntax.

---

48. THE FIVE QUESTIONS TO ASK

For any Backtracking problem:

1. What is my current state?

Example:

TEXT

```text
current subset
current permutation
row in N-Queens
```

2. What choices do I have?

3. What is invalid?

4. What is the base case?

5. What must I undo?

This framework solves many problems.

PART Q — RECURSION TREE

---

49. TAKE / NOT TAKE

For every element:

$$2$$

choices.

Depth:

$$n$$

Leaves:

$$2^n$$

Hence exponential.

---

50. PERMUTATION TREE

At first level:

$$n$$

choices.

Second:

$$n-1$$

Third:

$$n-2$$

Therefore leaves:

$$n!$$

This helps estimate complexity.

PART R — BACKTRACKING COMPLEXITY

Backtracking complexity is often based on number of states explored.

Examples:

Subsets

$$O(2^n)$$

states/output count.

Permutations

$$ O(n!) $$

N-Queens

roughly exponential/factorial search space.

Do not automatically say:

$$ O(n) $$

because recursion depth is n.

Depth and number of calls are different.

PART S — RECURSION VS BACKTRACKING VS DP VS D&C

51. MASTER COMPARISON

Technique	Main Idea
Recursion	Function solves smaller version
Backtracking	Try choices + undo
DP	Store repeated subproblem answers
Divide & Conquer	Solve mostly independent subproblems
Greedy	Commit to safe local choice

---

52. EXAMPLES

Factorial

Recursion.

Merge Sort

Divide & Conquer.

Subsets

Backtracking.

Fibonacci optimized

DP.

Activity Selection

Greedy.

---

53. CRITICAL DIFFERENCE — BACKTRACKING VS DP

Suppose recursion explores states.

If same state appears repeatedly:

> **DP may cache it.**

If states represent different partial choices and you must generate all possibilities:

> **Backtracking is often natural.**

PART T — MEMOIZATION

54. WHAT IS MEMOIZATION?

Store answer for a state.

Example Fibonacci:

```python
def fib(n, memo):
    if n <= 1:
        return n

    if n in memo:
        return memo[n]

    memo[n] = fib(n - 1, memo) + fib(n - 2, memo)
    return memo[n]
```

Naive:

\[O(2^n) \]

Memoized:

\[\boxed{O(n)} \]

---

55. SIGNAL FOR MEMOIZATION

If you can describe recursive state using small parameters like:

```text
(index)
(index, remaining)
(i, j)
```

and same state repeats:

\[\boxed{\text{Memoize}} \]

PART U — COMMON RECURSION INTERVIEW TRAPS

56. TRAP 1 — NO BASE CASE

Without proper stopping condition:

> **infinite recursion / stack overflow.**

57. TRAP 2 — PROBLEM DOES NOT SHRINK

Bad:

```python
solve(n)
```

calls:

```python
solve(n)
```

again unchanged.

No progress.

58. TRAP 3 — FORGETTING TO UNDO

Example subsets:

```python
current.append(x)
solve(...)
```

but no:

```python
current.pop()
```

Then choices leak into later branches.

59. TRAP 4 — SAVING SAME LIST REFERENCE

Bad:

```python
ans.append(current)
```

if `current` is later modified.

Often need:

```python
ans.append(current.copy())
```

This is a very common Python Backtracking bug.

60. TRAP 5 — DUPLICATE ANSWERS

If input contains duplicates:

```text
[1,2,2]
```

naive Backtracking may generate duplicate subsets/permutations.

Typical technique:

1. sort input,
2. skip duplicate choices at same recursion level.

Condition often conceptually:

```text
if i > start and nums[i] == nums[i-1]:
    continue
```

61. TRAP 6 — EXPONENTIAL COMPLEXITY

Backtracking may be correct but still too slow.

Use:

- pruning,
- memoization where applicable,
- sorting to prune,
- constraints to eliminate choices.

62. TRAP 7 — MUTABLE GLOBAL STATE

Be careful with shared:

```text
current
visited
board
```

Every choice must be properly restored.

PART V — DUPLICATE SUBSET PATTERN

63. EXAMPLE

Input:

```text
[1,2,2]
```

Valid unique subsets:

```text
[]
[1]
[2]
[1,2]
[2,2]
[1,2,2]
```

not repeated versions.

Sort first.

At same recursion depth:

> **skip identical candidate after first.**

This pattern is important for:

- Subsets II
- Combination Sum II
- unique permutations

PART W — PHONE LETTER COMBINATIONS

64. PROBLEM

Digits:

```text
"23"
```

Mapping:

```text
 2  !'   a b c
 3  !'   d e f
```

Outputs:

```text
ad ae af
bd be bf
cd ce cf
```

At every digit:

> **choose one letter.**

This is a clean Backtracking problem.

Number of possibilities approximately:

\[3^n \]

or \(4^n\) depending digits.

PART X — GENERATE PARENTHESES

65. PROBLEM

For:

\[n=3 \]

generate valid:

```text
((()))
(()())
(())()
()(())
()()()
```

---

66. PRUNING RULES

Maintain:

```text
open
close
```

Can add `(` if:

\[open<n \]

Can add `)` only if:

\[close<open \]

This ensures prefix never becomes invalid.

Excellent Backtracking example because pruning prevents invalid combinations.

PART Y — INTERVIEW EXPLANATION EXAMPLE

Suppose interviewer asks:

> **Generate all subsets.**

A strong answer:

> **“For every element I have two choices: include it in the current subset or exclude it. This naturally forms a binary recursion tree of depth n. At a leaf, I copy the current subset into the answer. When I take an element, I append it, recursively explore that branch, and then remove it before exploring the skip branch. There are 2^n subsets, and copying each subset can take up to $O(n)$, so the total output-generation complexity is $O(n2^n)$, with $O(n)$ recursion/path space excluding the output.”**

That is placement-quality reasoning.

PART Z — MUST-DO PROBLEMS

Foundation

1. Factorial
2. Sum of N
3. Fibonacci
4. Reverse Array Recursively
5. Palindrome Recursion
6. Binary Search Recursively

Core Backtracking

7. Subsets
8. Subsequences
9. Permutations
10. Combinations
11. Combination Sum
12. Letter Combinations of Phone Number
13. Generate Parentheses
14. Subsets II
15. Permutations II

Medium / Important

16. Word Search
17. N-Queens
18. Sudoku Solver
19. Palindrome Partitioning
20. Combination Sum II

PLACEMENT PRACTICE SET

Q1 — MCQ

Every recursive function must have:

A. Heap

B. Base case

C. HashMap

D. Queue

Q2 — MCQ

The Backtracking pattern is:

A. Sort !’ Merge !’ Return

B. Choose !’ Explore !’ Undo

C. BFS !’ DFS !’ Heap

D. Push !’ Pop only

Q3 — NAT

How many subsets exist for a set with:

\[5 \]

elements?

Q4 — NAT

How many permutations exist for:

\[4 \]

distinct elements?

Q5 — MSQ

Which are classic Backtracking problems?

A. N-Queens

B. Sudoku

C. Generate Subsets

D. Dijkstra

Q6 — MCQ

If a recursive Fibonacci solution repeatedly solves identical states, the natural optimization is:

A. Heapify

B. Memoization

C. BFS

D. Binary Search

Q7 — MCQ

In subset generation, forgetting to `pop()` after exploring a take-branch causes:

A. faster execution

B. state from one branch to leak into another

C. Binary Search failure

D. queue overflow

Q8 — MSQ

Which statements are true?

A. All Backtracking uses recursion in typical implementations.

B. Backtracking explores alternatives.

C. Pruning avoids exploring impossible branches.

D. Every Backtracking problem runs in $O(n)$.

Q9 — MCQ

For generating permutations of n distinct values, the number of solutions is:

A. n

B. 2^n

C. $n!$

D. n^2

Q10 — MCQ

Which problem most directly fits a Take/Not-Takerecursion?

A. Subsets

B. Dijkstra

C. Heap Sort

D. BFS

Q11 — MSQ

In Word Search Backtracking, we typically need to:

A. mark current cell

B. explore neighboring cells

C. unmark current cell after branch

D. sort the entire matrix after every step

Q12 — MCQ

If the same recursive state appears multiple times and only the optimal/count answer is needed, think:

A. DP/Memoization

B. MST

C. Queue

D. Selection Sort

PRACTICE SOLUTIONS

Q1

$$\boxed{B}$$

Without stopping condition recursion may continue indefinitely.

Q2

$$\boxed{B}$$

Q3

$$2^5 = \boxed{32}$$

Q4

$$4! = 4\times3\times2\times1 = \boxed{24}$$

Q5

N-Queens, Sudoku, and Subsets.

$$\boxed{A,B,C}$$

Q6

$$\boxed{B}$$

Memoization stores already computed results.

Q7

\[\boxed{B} \]

Q8

A, B, C are true.

D false.

\[\boxed{A,B,C} \]

Q9

\[\boxed{C} \]

Q10

\[\boxed{A} \]

Q11

A, B, C.

\[\boxed{A,B,C} \]

Q12

\[\boxed{A} \]

Ø=ÑÓTES TO WRITE

```text
RECURSION
= solve smaller version

Must have:
1. Base Case
2. Problem Shrinks

BACKTRACKING:
Choose
!’   E x p l o r e
!’   U n d o

SUBSETS:
Take / Don't Take
```

```
2^n possibilities

PERMUTATIONS:
Choose unused element
n! possibilities

PRUNING:
stop branch early if impossible

MEMOIZATION:
 s a m e  s t a t e  r e p e a t s !'  c a c h e  i t

Common:
Subsets
Permutations
Combination Sum
Generate Parentheses
Word Search
N-Queens
Sudoku
```

BACKTRACKING MASTER CHECKLIST

Whenever you see a new question, ask:

```text
1. What is my state?
2. What choices exist?
3. What is the base case?
4. What makes a choice invalid?
5. What do I change before recursion?
6. What do I undo after recursion?
7. Can I prune?
8. Are duplicate answers possible?
9. Does the same state repeat?
10. If yes, should I memoize?
```

COMPLEXITIES TO REMEMBER

Problem	Typical Complexity
Factorial recursion	\(O(n)\)
Fibonacci naive	\(O(2^n)\)
Fibonacci memoized	\(O(n)\)
Generate Subsets	\(O(n2^n)\) incl. output copies
Generate Permutations	\(O(n\cdot n!)\)
Word Search	exponential worst case
N-Queens	exponential/factorial search
Recursion depth	depends on tree depth

&β30-SECOND REVISION

1. Recursion = function calls itself.
2. Every recursion needs a base case.
3. Problem must shrink.
4. Recursion uses call-stack space.
5. Backtracking = Choose !’ Explore !’ Undo.
6. Subsets = Take / Don't Take.
7. \(n\) elements !’ \(2^n\) subsets.
8. \(n\) distinct elements !’ \(n!\) permutations.
9. `pop()` /unmark restores state.
10. Pruning kills impossible branches early.
11. Repeated states !’ Memoization/DP.
12. Word Search !’ DFS + Backtracking.
13. N-Queens !’ place one queen, validate, recurse, undo.
14. Sudoku !’ try valid digit, recurse, undo.
15. Generate Parentheses !’ use validity rules while building.
16. Depth \(n\) does not mean total time \(O(n)\).
17. Backtracking may explore exponentially many states.

' Placement Module 7 Complete

Next Placement Module:

MODULE 8 — TREES + BINARY TREES + BST INTERVIEW PREPARATION

Next we’ll cover:

- tree terminology,
- binary trees,
- recursive DFS traversals,
- iterative traversals,
- level-order BFS,
- height/depth,
- maximum depth,
- diameter,
- balanced tree,
- same/invert tree,
- path sum,
- BST search/insert/delete,
- validate BST,
- kth smallest,
- Lowest Common Ancestor,

- tree construction,
- and the most important tree interview patterns.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 8 — TREES + BINARY TREES + BST INTERVIEW PREPARATION

Placement Priority: & & & & &

Trees are one of the most important placement topics because they test:

- recursion
- BFS
- DFS
- divide-and-conquer thinking
- hierarchical data
- BST properties
- path-based reasoning

The biggest goal is to recognize:

\[\boxed{\text{Tree problem !' traversal + state}} \]

---

1. WHAT IS A TREE?

A tree is a hierarchical data structure.

Example:

TEXT

```
        1
      /   \
     2     3
    / \
   4   5
```

Here:

- `1` is root
- `2,3` are children of `1`
- `4,5` are children of `2`

---

2. BASIC TREE TERMS

Root

Topmost node.

```
TEXT
1
```

Parent

A node directly above another node.

`2` is parent of `4`.

Child

Node directly below.

`4` is child of `2`.

Leaf

Node with no children.

In above tree:

```
TEXT
4, 5, 3
```

are leaves.

Sibling

Nodes having same parent.

```
TEXT
2 and 3
```

are siblings.

Subtree

Any node along with all its descendants.

---

3. DEPTH VS HEIGHT

These are often confused.

Depth of node

Number of edges from root to node.

Root depth:

\[0 \]

Example:

```TEXT
        1      depth 0
       /
      2        depth 1
     /
    4          depth 2
```

Height of node

Number of edges on longest downward path from that node to a leaf.

For root above:

\[height=2 \]

Definitions sometimes count nodes instead of edges, so always follow problem statement.

---

4. BINARY TREE

A Binary Tree is a tree in which each node has at most:

\[\boxed{2} \]

children.

Usually called:

```TEXT
left
right
```

Example:

```TEXT
        10
       /  \
      5   20
```

---

5. TREE NODE STRUCTURE

Python-style:

```PYTHON
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
```

---

6. MOST IMPORTANT TREE IDEA

For many tree problems:

> **solve left subtree solve right subtree combine results**

This naturally matches recursion.

Generic pattern:

```python
def solve(root):
    if root is None:
        return ...

    left = solve(root.left)
    right = solve(root.right)

    return combine(left, right, root)
```

This template appears everywhere.

PART A — TREE TRAVERSALS

There are four must-know traversals.

7. PREORDER

Order:

\[\boxed{\text{Root !' Left !' Right}} \]

For:

```text
        1
       / \
      2   3
     / \
    4   5
```

Preorder:

```text
1 2 4 5 3
```

Memory:

> **PRE = Root first**

---

8. PREORDER CODE

```python
def preorder(root):
    if root is None:
        return

    print(root.val)
    preorder(root.left)
    preorder(root.right)
```

Time:

\[O(n) \]

Space:

\[O(h) \]

recursion stack.

Worst skewed tree:

\[O(n) \]

Balanced tree:

\[O(\log n) \]

---

9. INORDER

Order:

\[\boxed{\text{Left !' Root !' Right}} \]

For same tree:

```text
4 2 5 1 3
```

Memory:

> **IN = Root in middle**

---

10. INORDER CODE

```python
def inorder(root):
    if root is None:
        return

    inorder(root.left)
    print(root.val)
    inorder(root.right)
```

Critical BST fact:

\[\boxed{\text{Inorder of BST gives sorted order}} \]

11. POSTORDER

Order:

\[\boxed{\text{Left !' Right !' Root}} \]

Same tree:

```text
4 5 2 3 1
```

Memory:

> **POST = Root last**

12. POSTORDER CODE

```python
def postorder(root):
    if root is None:
        return

    postorder(root.left)
    postorder(root.right)
    print(root.val)
```

Postorder is useful when parent answer depends on child answers.

Examples:

- height
- diameter
- balance
- deleting tree

13. LEVEL ORDER

Level Order traverses tree level by level.

Same tree:

```text
1
2 3
4 5
```

Output:

```text
1 2 3 4 5
```

This is:

$$ \boxed{\text{BFS}} $$

Uses:

$$ \boxed{\text{Queue}} $$

---

14. LEVEL ORDER CODE

```python
from collections import deque

def level_order(root):
    if root is None:
        return []

    q = deque([root])
    ans = []

    while q:
        node = q.popleft()
        ans.append(node.val)

        if node.left:
            q.append(node.left)

        if node.right:
            q.append(node.right)

    return ans
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(w)} $$

where w = maximum tree width.

---

15. TRAVERSAL MASTER TABLE

Traversal	Order	Common Use
Preorder	Root-L-R	copy/serialize
Inorder	L-Root-R	BST sorted order
Postorder	L-R-Root	subtree results
Level Order	level-wise	BFS

Memory:

```text
 P R E   !'  r o o t  f i r s t
 I N     !'  r o o t  m i d d l e
 P O S T !'  r o o t  l a s t
```

PART B — MAXIMUM DEPTH

16. PROBLEM

```text
        1
       / \
      2   3
     /
    4
```

Maximum depth in node-count definition:

\[3 \]

17. RECURSIVE THINKING

Depth of tree:

\[1+\max(depth(left), depth(right)) \]

Base case:

\[depth(NULL)=0 \]

18. CODE

```python
def max_depth(root):
    if root is None:
        return 0

    left = max_depth(root.left)
    right = max_depth(root.right)

    return 1 + max(left, right)
```

Time:

\[\boxed{O(n)} \]

Space:

\[O(h) \]

19. TREE RECURSION STORY

At every node ask:

> **What answer do I need from my children?**

For depth:

```text
left height
right height
```

Then combine:

```text
1 + max(left, right)
```

This mindset is extremely important.

PART C — SAME TREE

20. PROBLEM

Determine whether two trees are structurally identical and contain same values.

Need:

1. same node value
2. left subtrees same
3. right subtrees same

---

21. BASE CASES

If both:

```text
None
```

return True.

If only one is `None`:

return False.

If values differ:

return False.

Else compare children recursively.

---

22. CODE

```python
def same_tree(a, b):
    if a is None and b is None:
        return True

    if a is None or b is None:
        return False

    if a.val != b.val:
        return False

    return (
        same_tree(a.left, b.left)
        and
        same_tree(a.right, b.right)
    )
```

Time:

\[O(n) \]

PART D — INVERT BINARY TREE

---

23. PROBLEM

Input:

```text
      1
     / \
    2   3
```

Output:

```text
      1
     / \
    3   2
```

At every node:

> **swap left and right.**

---

24. CODE

```python
def invert(root):
    if root is None:
        return None

    root.left, root.right = root.right, root.left

    invert(root.left)
```

```
    invert(root.right)

    return root
```

Time:

$$ \boxed{O(n)} $$

PART E — BALANCED BINARY TREE

25. DEFINITION

A height-balanced binary tree satisfies for every node:

$$ \boxed{ |height(left)-height(right)|\le1 } $$

Important:

Checking only root is not enough.

Condition must hold for:

$$ \boxed{\text{every node}} $$

---

26. BAD APPROACH

For every node:

1. compute left height
2. compute right height
3. recursively check children

If height is recalculated repeatedly:

Worst:

$$ O(n^2) $$

---

27. OPTIMAL POSTORDER APPROACH

Return height from subtree.

If subtree already unbalanced:

return sentinel like:

$$ -1 $$

At node:

```TEXT
left height
right height
```

If:

\[|left-right|>1 \]

return `-1`.

Otherwise:

\[1+\max(left,right) \]

Total:

\[\boxed{O(n)} \]

PART F — DIAMETER OF BINARY TREE

28. WHAT IS DIAMETER?

Longest path between any two nodes.

Important:

> **It does not have to pass through root.**

Example:

```text
        1
       /
      2
     / \
    4   5
```

Longest path:

```text
 4  !'   2  !'   5
```

---

29. CORE FORMULA

At node:

\[diameterThroughNode = height(left)+height(right) \]

if height measured in edges appropriately.

Need maximum over all nodes.

---

30. POSTORDER PATTERN

For each node:

1. get left height
2. get right height
3. update global maximum

4. return height to parent

Code concept:

```python
def diameter(root):
    best = 0

    def height(node):
        nonlocal best

        if node is None:
            return 0

        left = height(node.left)
        right = height(node.right)

        best = max(best, left + right)

        return 1 + max(left, right)

    height(root)
    return best
```

Time:

$$ \boxed{O(n)} $$

---

31. IMPORTANT TREE PATTERN

Many problems require two outputs:

1. something returned to parent
2. global answer updated locally

Diameter is classic example.

Return:

```text
height
```

Global:

```text
maximum diameter
```

PART G — PATHSUM

32. PROBLEM

Is there a root-to-leaf path whose values sum to target?

Example:

```text
      5
     / \
    4   8
   /
  11
```

Suppose target:

\[20 \]

Path:

\[5+4+11=20 \]

True.

---

33. IDEA

At each node:

\[remaining = target - node.val \]

When leaf reached:

check whether:

\[remaining=node.val \]

or after subtraction remaining is zero depending implementation.

---

34. CODE

```python
def has_path_sum(root, target):
    if root is None:
        return False

    if root.left is None and root.right is None:
        return target == root.val

    remaining = target - root.val

    return (
        has_path_sum(root.left, remaining)
        or
        has_path_sum(root.right, remaining)
    )
```

---

35. ROOT-TO-LEAF TRAP

A path such as:

```text
 r o o t  !’  i n t e r n a l  n o d e
```

is not valid if question specifically says:

> **root-to-leaf.**

Always check leaf condition.

PART H — BINARY SEARCH TREE

36. BST PROPERTY

For every node:

$$ \boxed{\text{all left values < node < all right values}} $$

Example:

```text
        8
       / \
      3   10
     / \
    1   6
```

---

37. IMPORTANT GLOBAL PROPERTY

It is not enough that:

```text
left child < root
right child > root
```

Every node in entire left subtree must be smaller.

Every node in entire right subtree must be larger.

---

38. BST SEARCH

Suppose searching:

$$ 6 $$

At root 8:

$$ 6<8 $$

go left.

At 3:

$$ 6>3 $$

go right.

Found.

Balanced BST average/worst depending balance:

$$ O(\log n) $$

Skewed BST worst:

$$ \boxed{O(n)} $$

---

39. SEARCH CODE

```python
def search_bst(root, target):
    while root:
        if root.val == target:
            return root

        if target < root.val:
            root = root.left
        else:
            root = root.right

    return None
```

PART I — BST INSERT

40. RULE

If value smaller:

> **go left.**

If greater:

> **go right.**

Continue until null position.

---

41. CODE

```python
def insert_bst(root, val):
    if root is None:
        return TreeNode(val)

    if val < root.val:
        root.left = insert_bst(root.left, val)
    else:
        root.right = insert_bst(root.right, val)

    return root
```

Complexity:

$$ O(h) $$

Balanced:

$$ O(\log n) $$

Skewed:

$$ O(n) $$

PART J — BST DELETE

This is a favorite conceptual interview topic.

42. THREE CASES

Delete node with:

Case 1 — No children

Simply remove it.

Case 2 — One child

Replace node with its child.

Case 3 — Two children

Common strategy:

Find:

$$ \boxed{\text{Inorder Successor}} $$

smallest node in right subtree.

Copy its value.

Then delete successor from right subtree.

---

43. MEMORY

```TEXT
0   c h i l d  !'   r e m o v e
1   c h i l d  !'   r e p l a c e
2   c h i l d  !'   s u c c e s s o r / p r e d e c e s s o r
```

PART K — VALIDATE BST

44. WRONG APPROACH

Only checking:

```TEXT
node.left.val < node.val
node.right.val > node.val
```

is insufficient.

Counterexample:

```TEXT
        10
       /  \
      5   15
         /  \
        6   20
```

At node 15:

\[6<15 \]

looks locally correct.

But 6 lies in right subtree of 10 and:

\[6<10 \]

So tree is not BST.

---

45. CORRECT RANGE APPROACH

Every node must lie within allowed range.

Root:

```TEXT
 ( -"  ,   +"  )
```

For left child:

```TEXT
 ( -"  ,   r o o t . v a l )
```

For right:

```TEXT
 ( r o o t . v a l ,   +"  )
```

---

46. CODE

```PYTHON
def valid_bst(root):
    def check(node, low, high):
        if node is None:
            return True
```

```
        if not (low < node.val < high):
            return False

        return (
            check(node.left, low, node.val)
            and
            check(node.right, node.val, high)
        )

    return check(root, float("-inf"), float("inf"))
```

Time:

\[\boxed{O(n)} \]

PART L — KTH SMALLEST IN BST

47. KEY PROPERTY

BST inorder traversal gives:

\[\boxed{\text{sorted order}} \]

Example:

```TEXT
        5
       / \
      3   7
     / \
    2   4
```

Inorder:

```TEXT
2 3 4 5 7
```

3rd smallest:

\[\boxed{4} \]

---

48. PATTERN

Kth smallest BST:

\[\boxed{\text{Inorder traversal}} \]

Time:

\[O(h+k) \]

with optimized iterative traversal, roughly up to \(O(n)\).

PART M — LOWEST COMMON ANCESTOR

49. WHAT IS LCA?

Lowest node that is ancestor of both given nodes.

Example:

```text
        3
       / \
      5   1
     / \
    6   2
```

LCA of 6 and 2:

$$ \boxed{5} $$

LCA of 5 and 1:

$$ \boxed{3} $$

50. LCA IN GENERAL BINARY TREE

At node:

If node is:

```text
p or q
```

return node.

Search:

```text
left
right
```

If both sides return non-null:

> **current node is LCA.**

If only one side non-null:

> **propagate that upward.**

51. CODE

```python
def lca(root, p, q):
    if root is None or root == p or root == q:
        return root

    left = lca(root.left, p, q)
    right = lca(root.right, p, q)

    if left and right:
        return root

    return left if left else right
```

Time:

\[O(n) \]

PART N — LCA IN BST

BST property makes it easier.

Suppose values:

```text
p < root
q < root
```

Both are left:

> **go left.**

If:

```text
p > root
q > root
```

Both right:

> **go right.**

Otherwise they split around root.

Then root is:

\[\boxed{\text{LCA}} \]

---

52. BST LCA CODE

```python
def lca_bst(root, p, q):
    while root:
        if p.val < root.val and q.val < root.val:
```

```
            root = root.left

        elif p.val > root.val and q.val > root.val:
            root = root.right

        else:
            return root
```

Time:

\[O(h) \]

PART O — LEVEL ORDER BY LEVELS

53. PROBLEM

Instead of:

```TEXT
1 2 3 4 5
```

return:

```TEXT
[
 [1],
 [2,3],
 [4,5]
]
```

---

54. QUEUE LEVEL-SIZE TRICK

At beginning of each loop:

```TEXT
level_size = len(queue)
```

Process exactly that many nodes.

Those nodes belong to same level.

---

55. CODE

```PYTHON
from collections import deque

def levels(root):
    if not root:
        return []

    q = deque([root])
```

```
    ans = []

    while q:
        size = len(q)
        level = []

        for _ in range(size):
            node = q.popleft()
            level.append(node.val)

            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)

        ans.append(level)

    return ans
```

PART P — RIGHT SIDE VIEW

56. PROBLEM

Return node visible from right side.

Example:

```text
        1
       / \
      2   3
       \
        5
```

View:

```text
1, 3, 5
```

Simple BFS idea:

> **for each level, take last node.**

Pattern:

\[\boxed{\text{Level Order}} \]

PART Q — ZIGZAG LEVEL ORDER

57. PROBLEM

Levels alternate direction.

Example:

```text
[
 [1],
 [3,2],
 [4,5,6]
]
```

Still BFS.

Maintain boolean:

```text
left_to_right
```

Flip after each level.

PART R — COUNT NODES

58. GENERAL TREE

\[count(root) = 1+count(left)+count(right) \]

Code:

```python
def count_nodes(root):
    if root is None:
        return 0

    return 1 + count_nodes(root.left) + count_nodes(root.right)
```

Time:

\[O(n) \]

PART S — CHECK SYMMETRIC TREE

59. PROBLEM

Tree:

```text
        1
       / \
      2   2
     / \ / \
    3  4 4  3
```

Symmetric.

Need compare:

```text
left.left with right.right
left.right with right.left
```

This is mirror comparison.

60. RECURSION

```python
def symmetric(root):
    def mirror(a, b):
        if not a and not b:
            return True

        if not a or not b:
            return False

        return (
            a.val == b.val
            and mirror(a.left, b.right)
            and mirror(a.right, b.left)
        )

    return mirror(root.left, root.right)
```

PART T — BUILD TREE FROM TRAVERSALS

Important medium interview topic.

61. PREORDER + INORDER

Preorder tells us:

\[\boxed{\text{Root first}} \]

Inorder tells us:

```text
left subtree | root | right subtree
```

Therefore:

1. first preorder element = root
2. find root in inorder
3. split inorder into left/right
4. recursively construct.

62. EXAMPLE

Preorder:

```text
3 9 20 15 7
```

Inorder:

```text
9 3 15 20 7
```

Root:

\[3 \]

In inorder:

```text
9 | 3 | 15 20 7
```

Left subtree:

```text
9
```

Right:

```text
15 20 7
```

Continue recursively.

---

63. HASHMAP OPTIMIZATION

Finding root index by scanning inorder every time could lead to:

\[O(n^2) \]

Store:

```text
 v a l u e  !'  i n o r d e r  i n d e x
```

using HashMap.

Then construction can be:

\[\boxed{O(n)} \]

assuming unique values.

PART U — SERIALIZE / DESERIALIZE TREE

64. IDEA

Convert tree into a string/list representation and reconstruct it later.

Need represent null children too.

Example preorder serialization:

```text
1,2,#,#,3,#,#
```

where `#` means null.

Why nulls needed?

Without them, structure may be ambiguous.

This is a common advanced interview topic.

PART V — TREE BFS VS DFS

65. WHEN TO USE BFS

Think BFS for:

- level order
- minimum depth
- nearest node by levels
- right side view
- zigzag
- level averages

Memory:

> **Levels !’ Queue**

---

66. WHEN TO USE DFS

Think DFS for:

- paths
- subtree calculations
- height
- diameter
- balance
- LCA
- root-to-leaf problems

Memory:

> **Subtree/Path !’ Recursion/Stack**

PART W — ITERATIVE DFS

Recursive inorder is simple, but interviewers may ask iterative.

67. ITERATIVE INORDER

Use Stack.

Process:

1. go as far left as possible
2. pop node
3. visit
4. move right

```python
def inorder_iterative(root):
    stack = []
    ans = []
    current = root

    while current or stack:
        while current:
            stack.append(current)
            current = current.left

        current = stack.pop()
        ans.append(current.val)

        current = current.right

    return ans
```

Time:

$$ O(n) $$

Space:

$$ O(h) $$

PART X — BST VS HEAP

Common interview comparison.

BST	Heap
Left/right ordering	Parent-child ordering
Search can be $O(\log n)$ balanced	Arbitrary search $O(n)$
Inorder sorted	Traversal not sorted
Good for ordered searching	Good for repeated min/max
Supports predecessor/successor	Fast root min/max

Do not confuse them.

PART Y — COMMON TREE INTERVIEW TRAPS

68. TRAP 1 — NULL TREE

Always handle:

```text
root is None
```

69. TRAP 2 — LEAF VS NULL

Leaf:

```text
left = None
right = None
```

But leaf itself is a real node.

70. TRAP 3 — BALANCED CHECK ONLY AT ROOT

Wrong.

Need condition at every node.

71. TRAP 4 — DIAMETER MUST PASS ROOT

False.

Diameter may exist entirely inside a subtree.

72. TRAP 5 — BST LOCAL CHECK

Checking only immediate children is insufficient.

Need global range.

73. TRAP 6 — BST ALWAYS O(log n)

False.

Skewed:

```text
1
 \
```

```
  2
   \
    3
```

height:

\[n \]

Search:

\[O(n) \]

---

74. TRAP 7 — HEIGHT SPACE

Recursive tree traversal uses:

\[O(h) \]

stack, not automatically \(O(n)\).

Balanced:

\[h=O(\log n) \]

Skewed:

\[h=O(n) \]

---

75. TRAP 8 — NODE COUNT VS EDGE COUNT

Some problems define height as:

- number of nodes
- number of edges

Check carefully.

PART Z — TREE PATTERNMASTER MAP

Signal	Pattern
Traverse hierarchy	DFS/BFS
Levels	BFS
Height	Postorder DFS
Diameter	heights + global max
Balanced	height + sentinel
Same tree	paired DFS
Invert	swap children recursively
Root-to-leaf path	DFS

Signal	Pattern
BST search	compare and choose side
Validate BST	range constraints
Kth smallest BST	inorder
LCA binary tree	postorder
LCA BST	ordering
Construct tree	traversal properties
Right side view	BFS levels

---

76. THE MOST IMPORTANT TREE INTERVIEW QUESTION

Whenever you see a recursive tree problem, ask:

> **What information should the left subtree return?**

> **What information should the right subtree return?**

> **How do I combine them at the current node?**

For height:

```text
left height
right height
!'   1   +   m a x
```

For balance:

```text
left height
right height
!'   c o m p a r e   d i f f e r e n c e
```

For diameter:

```text
left height
right height
!'   l e f t   +   r i g h t
```

This is the core tree-solving skill.

PART AA — MUST-DO TREE QUESTIONS

Foundation

1. Preorder Traversal
2. Inorder Traversal
3. Postorder Traversal
4. Level Order Traversal
5. Maximum Depth
6. Count Nodes
7. Same Tree
8. Invert Binary Tree

Core Placement

9. Balanced Binary Tree
10. Diameter
11. Path Sum
12. Symmetric Tree
13. Right Side View
14. Zigzag Level Order
15. Validate BST
16. Search BST
17. Insert BST
18. Kth Smallest BST
19. Lowest Common Ancestor
20. BST LCA

Medium / Important

21. Delete Node in BST
22. Construct Tree from Preorder + Inorder
23. Serialize/Deserialize
24. Maximum Path Sum
25. Path Sum II
26. Flatten Binary Tree
27. Boundary/View questions
28. Vertical Traversal

PLACEMENT PRACTICE SET

Q1 — MCQ

Inorder traversal is:

A. Root-Left-Right B. Left-Root-Right C. Left-Right-Root D. Level-by-level

Q2 — MCQ

Which traversal of a BST gives sorted order?

A. Preorder B. Inorder C. Postorder D. BFS only

Q3 — MCQ

Level Order Traversal naturally uses:

A. Stack B. Queue C. Heap D. Union-Find

Q4 — NAT

For:

```text
        1
       / \
      2   3
     /
    4
```

If depth is measured as number of nodes on the longest root-to-leaf path, what is maximum depth?

Q5 — MSQ

Which are commonly solved using postorder-style subtree information?

A. Height

B. Diameter

C. Balanced Tree

D. Binary Search on array

Q6 — MCQ

Balanced binary tree requires:

A. root's children have equal values

B. every node has exactly two children

C. height difference of left/right subtrees is at most 1 at every node

D. tree must be BST

Q7 — MCQ

Diameter of a binary tree:

A. must pass through root

B. never passes through root

C. may or may not pass through root

D. equals number of leaves

Q8 — MCQ

BST search complexity in a completely skewed tree can be:

A. $O(1)$

B. $O(\log n)$ always

C. $O(n)$

D. $O(n\log n)$

Q9 — MCQ

For validating a BST, which method is robust?

A. compare only immediate children

B. propagate valid min/max range

C. BFS only without values

D. sort nodes physically

Q10 — MCQ

Kth smallest element in BST most naturally uses:

A. preorder

B. inorder

C. postorder

D. heap sort only

Q11 — MSQ

Which statements are true?

A. Inorder of BST is sorted.

B. Level order uses Queue.

C. Recursive DFS uses call stack.

D. Every binary tree is a BST.

Q12 — MCQ

In BST deletion, deleting a node with two children commonly uses:

A. random leaf

B. inorder successor/predecessor

C. BFS root

D. Heapify

Q13 — MCQ

General Binary Tree LCA uses:

A. information returned from left and right subtrees

B. only sorting

C. Prefix Sum

D. Dijkstra

Q14 — MSQ

Which are good BFS-tree problems?

A. Level Order

B. Right Side View

C. Zigzag Level Order

D. Maximum Subarray

Q15 — MCQ

Constructing a binary tree from preorder and inorder can be optimized using a map from:

A. preorder value to height

B. inorder value to index

C. node to queue size

D. edge to weight

PRACTICE SOLUTIONS

Q1

$$\boxed{B}$$

Q2

$$\boxed{B}$$

Q3

$$\boxed{B}$$

Q4

Path:

```TEXT
1  !′   2  !′   4
```

contains 3 nodes.

$$\boxed{3}$$

Q5

Height, Diameter and Balanced Tree all depend naturally on subtree results.

$$\boxed{A,B,C}$$

Q6

$$\boxed{C}$$

Q7

$$\boxed{C}$$

Q8

Skewed BST behaves like linked list.

$$\boxed{C}$$

Q9

$$\boxed{B}$$

Q10

$$\boxed{B}$$

Q11

A, B, C true.

D false.

$$\boxed{A,B,C}$$

Q12

$$\boxed{B}$$

Q13

$$\boxed{A}$$

Q14

A, B, C.

$$\boxed{A,B,C}$$

Q15

$$\boxed{B}$$

Ø=ÑÓTES TO WRITE

```TEXT
TREE TRAVERSALS

Preorder:
Root Left Right

Inorder:
Left Root Right

Postorder:
Left Right Root

Level Order:
BFS + Queue

TREE RECURSION:
left = solve(left)
right = solve(right)
combine

Depth:
1 + max(left, right)

Diameter:
leftHeight + rightHeight

Balanced:
|leftHeight-rightHeight| <= 1
```

```
at EVERY node

BST:
all left < root < all right

BST Inorder:
sorted

Validate BST:
allowed range

Kth smallest:
Inorder

LCA Binary Tree:
left/right subtree returns

LCA BST:
use ordering
```

COMPLEXITIES TO REMEMBER

Problem	Complexity
DFS traversal	\(O(n)\)
BFS traversal	\(O(n)\)
Height	\(O(n)\)
Diameter optimal	\(O(n)\)
Balanced optimal	\(O(n)\)
Validate BST	\(O(n)\)
BST search	\(O(h)\)
BST insert	\(O(h)\)
BST delete	\(O(h)\)
Build from preorder+inorder	\(O(n)\) with map

For balanced BST:

\[h=O(\log n) \]

For skewed:

\[h=O(n) \]

&β30-SECOND TREE REVISION

1. Tree = hierarchical nodes.
2. Binary tree !’ at most two children.
3. Preorder = Root-L-R.

4. Inorder = L-Root-R.
5. Postorder = L-R-Root.
6. Level Order = BFS + Queue.
7. Inorder BST = sorted.
8. Height = 1 + max(left,right).
9. Diameter uses left height + right height.
10. Diameter need not pass through root.
11. Balanced condition applies at every node.
12. Path problems !’ DFS.
13. Level problems !’ BFS.
14. BST search follows ordering.
15. BST search is \(O(h)\), not always \(O(\log n)\).
16. Validate BST with ranges.
17. Kth smallest BST !’ inorder.
18. General LCA !’ combine left/right results.
19. BST LCA !’ exploit ordering.
20. Tree recursion = ask children for information and combine.

' Placement Module 8 Complete

Next Placement Module:

MODULE 9 — HEAPS + PRIORITY QUEUES + TOP-K PATTERNS

We’ll cover:

- min heap / max heap,
- heap properties,
- insertion and deletion,
- priority queues,
- Kth Largest / Kth Smallest,
- Top K Frequent Elements,
- K Closest Points,
- Merge K Sorted Lists,
- Median from Data Stream,
- task scheduling,
- when to use a heap vs sorting,
- and the most important heap interview patterns.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 9 — HEAPS + PRIORITY QUEUES + TOP-K PATTERNS

Placement Priority: & & & & &

Heaps are extremely important in placements because whenever a question says:

- smallest/largest repeatedly
- top K
- kth largest
- kth smallest
- merge many sorted structures
- streaming median
- schedule by priority

you should immediately consider:

$$\boxed{\text{Heap / Priority Queue}}$$

The main idea:

> **A heap gives fast access to the most important element, not full sorted order.**

---

1. WHAT IS A HEAP?

A heap is a complete binary tree with a special ordering property.

Two common types:

Min Heap

Parent is smaller than or equal to children.

Example:

```
TEXT
          2
        /   \
      4       5
     / \    /
    9   7  8
```

Minimum element is at root:

$$\boxed{2}$$

Max Heap

Parent is greater than or equal to children.

Example:

```TEXT
        10
       /  \
      8    9
     / \
    3   5
```

Maximum element:

$$ \boxed{10} $$

2. COMPLETE BINARY TREE PROPERTY

A heap is usually stored as an array because it is complete.

Example heap:

```TEXT
        10
       /  \
      8    9
     / \
    3   5
```

Array:

```TEXT
[10, 8, 9, 3, 5]
```

3. INDEX FORMULAS

For 0-based indexing:

For node at index i:

$$ left=2i+1 $$

$$ right=2i+2 $$

$$ parent=\left\lfloor\frac{i-1}{2}\right\rfloor $$

These formulas are very common in interviews.

4. HEAP OPERATIONS

Operation	Time
Get min/max	$O(1)$
Insert	$O(\log n)$
Remove min/max	$O(\log n)$
Build Heap	$\Theta(n)$

Operation	Time
Arbitrary search	$O(n)$

Important:

> **Heap is not a structure for fast arbitrary search.**

---

5. WHY INSERT IS O(log n)

Insert new element at next free position.

Then restore heap property by moving upward.

This is called:

$$ \boxed{\text{Heapify Up / Bubble Up}} $$

Height of heap:

$$ O(\log n) $$

Therefore insertion:

$$ \boxed{O(\log n)} $$

---

6. REMOVE ROOT

For min heap, remove minimum.

Steps:

1. Replace root with last element.
2. Remove last.
3. Move new root downward until heap property restored.

Called:

$$ \boxed{\text{Heapify Down}} $$

Time:

$$ O(\log n) $$

---

7. HEAP VS SORTING

Suppose you need the smallest element once.

Sorting:

$$ O(n\log n) $$

But scanning minimum:

$$ O(n) $$

Heap may be unnecessary.

Suppose you repeatedly need smallest element while data changes.

Then heap becomes much more useful.

Key question:

> **Do I need fully sorted order, or only repeated access to min/max?**

PART A — PRIORITY QUEUE

8. WHAT IS A PRIORITY QUEUE?

A Priority Queue removes elements based on priority, not insertion order.

Example hospital:

```text
Patient A !' priority 2
Patient B !' priority 10
Patient C !' priority 5
```

Highest-priority patient may be handled first regardless of arrival time.

A heap is the standard implementation.

---

9. HEAP VS PRIORITY QUEUE

Heap:

> **underlying data structure.**

Priority Queue:

> **abstract behavior/interface.**

A Priority Queue is commonly implemented using a Heap.

---

10. PYTHON HEAP

Python's:

```python
import heapq
```

implements a:

$$ \boxed{\text{Min Heap}} $$

Example:

```python
import heapq

heap = []

heapq.heappush(heap, 5)
heapq.heappush(heap, 2)
heapq.heappush(heap, 8)

print(heapq.heappop(heap))
```

Output:

```text
2
```

---

11. MAX HEAP IN PYTHON

Common trick:

Store negative numbers.

Instead of:

```text
5
2
8
```

store:

```text
-5
-2
-8
```

Minimum negative value corresponds to largest original value.

Example:

```python
heapq.heappush(heap, -8)
```

When pop:

```python
largest = -heapq.heappop(heap)
```

PART B — KTH LARGEST ELEMENT

One of the most common Heap questions.

12. PROBLEM

Input:

```text
[3,2,1,5,6,4]
```

\[k=2 \]

Sorted descending:

```text
6,5,4,3,2,1
```

2nd largest:

\[\boxed{5} \]

---

13. APPROACH 1 — SORT

Sort:

\[O(n\log n) \]

Then pick.

Simple, but not always optimal.

---

14. APPROACH 2 — MIN HEAP OF SIZE K

Keep only the \(k\) largest elements seen so far.

For:

\[k=2 \]

Heap size never exceeds 2.

Process each value.

If heap size exceeds \(k\):

> **remove smallest.**

At end:

> **root is kth largest.**

---

15. WHY MIN HEAP FOR KTH LARGEST?

This confuses many students.

We want \(k\) largest elements.

Among those k elements, the smallest is exactly the kth largest overall.

Therefore:

$$\boxed{\text{Kth Largest !' Min Heap of size K}}$$

16. CODE

```python
import heapq

def kth_largest(nums, k):
    heap = []

    for x in nums:
        heapq.heappush(heap, x)

        if len(heap) > k:
            heapq.heappop(heap)

    return heap[0]
```

Time:

$$\boxed{O(n\log k)}$$

Space:

$$\boxed{O(k)}$$

Better than full sort when:

$$k\ll n$$

17. KTH SMALLEST

Symmetric logic:

$$\boxed{\text{Kth Smallest !' Max Heap of size K}}$$

Keep k smallest elements.

Largest among them is kth smallest.

Memory:

```text
 K t h   L a r g e s t   !'  M i n   H e a p   K
 K t h   S m a l l e s t !'  M a x   H e a p   K
```

PART C — TOP K FREQUENT ELEMENTS

18. PROBLEM

Input:

```text
[1,1,1,2,2,3]
```

\[k=2 \]

Frequencies:

```text
 1  !'   3
 2  !'   2
 3  !'   1
```

Top 2:

```text
[1,2]
```

---

19. TWO-PHASE PATTERN

Step 1:

\[\boxed{\text{HashMap for frequency}} \]

Step 2:

\[\boxed{\text{Heap for Top K}} \]

This combination appears constantly.

---

20. APPROACH

Build map:

```text
 v a l u e  !'  f r e q u e n c y
```

Maintain min heap of:

```text
(frequency, value)
```

size \(k\).

If size becomes \(k+1\):

> **remove least frequent.**

At end heap contains top \(k\).

Complexity:

\[O(n\log k) \]

approximately after counting.

21. IMPORTANT COMBINATION PATTERN

When question says:

> **frequency + Top K**

think:

$$\boxed{\text{HashMap + Heap}}$$

PART D — K CLOSEST POINTS TO ORIGIN

22. PROBLEM

Points:

```TEXT
(1,3)
(-2,2)
(5,8)
```

Need closest k to origin.

Distance:

$$\sqrt{x^2+y^2}$$

But square root unnecessary for comparison.

Compare:

$$\boxed{x^2+y^2}$$

23. HEAP PATTERN

For K closest:

> **keep only best K candidates.**

Use max heap of size k, where heap root represents farthest among current closest candidates.

If a closer point appears:

> **replace root.**

Complexity:

$$O(n\log k)$$

PART E — MERGE K SORTED LISTS

Very important interview problem.

24. PROBLEM

Suppose:

```text
L 1 :   1  !′   4  !′   7
L 2 :   2  !′   5  !′   8
L 3 :   3  !′   6  !′   9
```

Need:

```text
1  !′   2  !′   3  !′   4  !′   5  !′   6  !′   7  !′   8  !′   9
```

---

25. NAIVE IDEA

Compare heads of all \(k\) lists every time.

Each selection costs:

\[O(k) \]

For total \(N\) nodes:

\[O(Nk) \]

---

26. HEAP IDEA

Put first node from each nonempty list into min heap.

Heap contains at most:

\[k \]

nodes.

Repeatedly:

1. pop smallest node
2. append it to result
3. push that node's next node

Each heap operation:

\[O(\log k) \]

For \(N\) nodes:

\[\boxed{O(N\log k)} \]

---

27. WHY THIS WORKS

Each list is already sorted.

So only the current head of each list can possibly be the next global smallest element.

We do not need every node in heap.

Only:

$$ \boxed{\text{one candidate per list}} $$

This is a beautiful priority-queue pattern.

PART F — MERGE K SORTED ARRAYS

Same concept.

Suppose:

```text
[1,4,7]
[2,5,8]
[3,6,9]
```

Store heap entry like:

```text
(value, array_index, element_index)
```

Pop smallest.

Then push next value from same array.

Same complexity:

$$ O(N\log k) $$

PART G — FIND MEDIAN FROM DATA STREAM

Very important advanced Heap problem.

28. PROBLEM

Numbers arrive one by one.

After each insertion, return median efficiently.

Example stream:

```text
5
2
10
3
```

Sorted states:

```TEXT
[5]           median 5
[2,5]         median 3.5
[2,5,10]      median 5
[2,3,5,10]    median 4
```

Sorting entire stream after each insertion is expensive.

---

29. TWO-HEAP IDEA

Divide numbers into:

Lower half

Store in:

$$ \boxed{\text{Max Heap}} $$

Upper half

Store in:

$$ \boxed{\text{Min Heap}} $$

Example:

```TEXT
lower: 2,3
upper: 5,10
```

---

30. BALANCE CONDITION

Heap sizes should differ by at most:

$$ 1 $$

If odd total:

> **larger heap's root is median.**

If even:

> **average of both roots.**

---

31. WHY TWO HEAPS?

Need:

- maximum of lower half
- minimum of upper half

Those are exactly what heaps give quickly.

Insertion:

$$ O(\log n) $$

Median lookup:

$$ \boxed{O(1)} $$

---

32. MEMORY

```text
Streaming Median:
 L o w e r   H a l f  !'   M a x   H e a p
 U p p e r   H a l f  !'   M i n   H e a p
```

PART H — TASK SCHEDULER

33. PROBLEM IDEA

Tasks:

```text
A A A B B B
```

Cooldown $n=2$.

Same task must have at least 2 intervals between repeated executions.

Heap can help choose:

> **currently most frequent available task.**

---

34. PATTERN

1. Count frequencies.
2. Use max heap of remaining frequencies.
3. Track tasks in cooldown.
4. Reinsert when available.

This combines:

$$ \boxed{\text{Hashing + Heap + Queue}} $$

Good example of multiple structures working together.

PART I — REORGANIZE STRING

35. PROBLEM IDEA

Rearrange string so no adjacent characters are same.

Example:

```text
"aab"
```

Possible:

```text
"aba"
```

Strategy:

1. Count character frequencies.
2. Always choose highest-frequency valid character.
3. Temporarily hold previous character to avoid immediate reuse.

Use max heap.

This is another:

\[\boxed{\text{Frequency + Priority Queue}} \]

pattern.

PART J — KTH LARGEST IN A STREAM

36. PROBLEM

Numbers keep arriving.

After each insertion, return kth largest.

Maintain:

\[\boxed{\text{Min Heap of size K}} \]

Exactly same logic as static kth largest, but continuously updated.

Add new number:

```text
push
if size > k:
    pop
```

Root always:

\[\boxed{\text{kth largest}} \]

PART K — CONNECT ROPES / MINIMUM COST

Classic Greedy + Heap problem.

37. PROBLEM

Rope lengths:

```text
4,3,2,6
```

Connecting two ropes costs their total length.

Need minimum total cost.

---

38. GREEDY RULE

Always combine:

$$ \boxed{\text{two smallest ropes}} $$

Why?

Same underlying idea as Huffman coding.

---

39. TRACE

Heap:

```text
2,3,4,6
```

Take:

$$ 2+3=5 $$

Cost:

$$ 5 $$

Heap:

```text
4,5,6
```

Take:

$$ 4+5=9 $$

Total:

$$ 5+9=14 $$

Heap:

```text
6,9
```

Take:

$$ 6+9=15 $$

Total:

$$ 14+15=29 $$

Answer:

$$ \boxed{29} $$

---

40. COMPLEXITY

Each merge uses heap operations.

For n ropes:

$$ \boxed{O(n\log n)} $$

PART L — HEAPIFY VS BUILD HEAP

Important from both GATE and placements.

41. HEAPIFY ONE NODE

Restoring heap property downward:

$$ \boxed{O(\log n)} $$

---

42. BUILD HEAP

Building a heap bottom-up from entire array:

$$ \boxed{\Theta(n)} $$

Not:

$$ O(n\log n) $$

This remains one of the most common heap traps.

PART M — HEAP SORT

43. IDEA

For ascending order:

1. Build Max Heap.
2. Swap root with last.
3. Reduce heap size.
4. Heapify root.
5. Repeat.

Complexity:

$$ \boxed{O(n\log n)} $$

worst-case.

Extra array space:

\[O(1) \]

for standard in-place version.

Not stable.

PART N — HEAP VS BST

Very common interview comparison.

Need	Heap	Balanced BST
Get min/max root	\(O(1)\)	\(O(\log n)\) or min pointer
Insert	\(O(\log n)\)	\(O(\log n)\)
Delete root min/max	\(O(\log n)\)	\(O(\log n)\)
Search arbitrary key	\(O(n)\)	\(O(\log n)\)
Sorted traversal	No	Yes
Top priority	Excellent	Possible

Use Heap when:

> **priority matters.**

Use BST when:

> **ordered search/range queries matter.**

PART O — HEAP VS SORTING DECISION

44. WHEN SORTING IS BETTER

If you need:

- entire sorted output
- many ordered comparisons
- all elements in order

then sorting may be simpler.

---

45. WHEN HEAP IS BETTER

If you need:

- top K only
- kth largest/smallest
- repeated min/max
- stream processing
- merging sorted sources

then heap is often better.

---

46. COMPLEXITY COMPARISON

Top K from n elements:

Sort all

$$ O(n\log n) $$

Heap of size K

$$ O(n\log k) $$

If:

$$ k\ll n $$

heap can be better.

PART P — TOP K MASTER PATTERN

Whenever you see:

> **K largest**

> **K smallest**

> **K frequent**

> **K closest**

think:

$$ \boxed{\text{Heap size K}} $$

Then ask:

Need K largest?

Use min heap.

Need K smallest?

Use max heap.

This opposite relationship is crucial.

PART Q — WHY O(n log k)?

Suppose heap never exceeds size k.

Each insertion/removal costs:

$$O(\log k)$$

You process n elements.

Therefore:

$$\boxed{O(n\log k)}$$

not:

$$O(n\log n)$$

PART R — COMMON HEAP INTERVIEW TRAPS

47. TRAP 1 — HEAP IS FULLY SORTED

False.

In Min Heap:

> **parent $\le$ children.**

But siblings and deeper nodes need not be sorted relative to each other.

Example:

```TEXT
      1
     / \
    5   2
```

Valid Min Heap.

5 > 2 is fine.

48. TRAP 2 — SEARCH IN HEAP IS O(log n)

False.

Heap ordering isn't enough to locate arbitrary value.

Worst:

$$\boxed{O(n)}$$

49. TRAP 3 — KTH LARGEST USES MAX HEAP SIZE K

Possible, but the common space-efficient top-K method is:

$$ \boxed{\text{Min Heap size K}} $$

because root is smallest among current K largest.

50. TRAP 4 — BUILD HEAP

$$ \boxed{\Theta(n)} $$

not $n\log n$.

51. TRAP 5 — MAX HEAP PYTHON

`heapq` is Min Heap.

For max behavior often use:

```text
-negative value
```

52. TRAP 6 — STORE ENOUGH INFORMATION

For merging K arrays, value alone may not be enough.

Need know:

```text
which array
which index
```

so next element can be pushed.

53. TRAP 7 — PRIORITY TIES

In some languages/library heaps, if tuple first fields tie, comparator may examine next fields.

Be careful if the next object itself is not comparable.

In interviews, define a proper comparator or include a stable tie-break key when needed.

PART S — INTERVIEW EXPLANATION EXAMPLE

Suppose interviewer asks:

> **Find kth largest element without sorting entire array.**

A strong answer:

> **“Sorting takes $O(n\log n)$. Since I only need the kth largest, I can maintain a min heap containing the k largest elements seen so far. For every number I push it into the heap, and if the heap size exceeds k, I remove the smallest. At the end, the heap contains exactly the k largest values, and its root is the smallest among them, which is the kth largest overall. The time complexity is $O(n\log k)$ and the extra space is $O(k)$.”**

That's placement-quality reasoning.

PART T — HEAP RECOGNITION TABLE

Signal	Pattern
Repeated minimum	Min Heap
Repeated maximum	Max Heap
Kth largest	Min Heap size K
Kth smallest	Max Heap size K
Top K frequent	Frequency + Heap
K closest	Heap size K
Merge K sorted lists	Min Heap of heads
Streaming median	Two Heaps
Minimum rope cost	Two smallest repeatedly
Priority scheduling	Priority Queue

PART U — MUST-DO HEAP QUESTIONS

Foundation

1. Min Heap basics
2. Max Heap basics
3. Insert into Heap
4. Heapify
5. Build Heap
6. Heap Sort

Core Placement

7. Kth Largest Element
8. Kth Smallest Element
9. Top K Frequent Elements
10. K Closest Points
11. Kth Largest in Stream

12. Merge K Sorted Lists
13. Merge K Sorted Arrays

Medium / Important

14. Find Median from Data Stream
15. Task Scheduler
16. Reorganize String
17. Connect Ropes / Minimum Cost
18. Smallest Range Covering K Lists
19. Find K Pairs with Smallest Sums
20. Meeting Rooms / scheduling variants

PLACEMENT PRACTICE SET

Q1 — MCQ

In a Min Heap, which is guaranteed?

A. Entire array is sorted.

B. Root is minimum.

C. Every left child is smaller than every right child.

D. Search is always $O(\log n)$.

Q2 — MCQ

Heap insertion takes:

A. $O(1)$ always B. $O(\log n)$ C. $O(n)$ D. $O(n\log n)$

Q3 — MCQ

Bottom-up Build Heap takes:

A. $O(\log n)$ B. $O(n)$ C. $O(n\log n)$ D. $O(n^2)$

Q4 — MCQ

To find kth largest using $O(k)$ extra space, use:

A. Min Heap of size K

B. Max Heap of size K

C. Queue

D. DFS

Q5 — MCQ

To find kth smallest using size-K heap, commonly use:

A. Min Heap

B. Max Heap

C. Stack

D. HashSet only

Q6 — NAT

For:

```text
[3,2,1,5,6,4]
```

what is the 2nd largest?

Q7 — MSQ

Which strongly suggest a Heap?

A. Top K Frequent Elements

B. K Closest Points

C. Repeatedly extract minimum

D. Check palindrome

Q8 — MCQ

Merge K sorted lists with total \(N\) nodes using heap takes:

A. \(O(N)\)

B. \(O(N\log k)\)

C. \(O(k^N)\)

D. \(O(N^2)\)

Q9 — MCQ

Streaming median is commonly solved using:

A. one stack

B. two heaps

C. BFS only

D. linked list cycle detection

Q10 — MSQ

For streaming median:

A. lower half can be stored in Max Heap

B. upper half can be stored in Min Heap

C. heap sizes should be balanced

D. arbitrary search in heap is always $O(\log n)$

Q11 — NAT

Rope lengths:

```text
1,2,3
```

Using minimum-cost rope connection, what is total minimum cost?

Q12 — MCQ

If you need only Top 10 elements from 1,000,000 values, which is often preferable to sorting everything?

A. Heap of size 10

B. DFS

C. Linked List reversal

D. Bellman-Ford

Q13 — MSQ

Which statements are true?

A. Heap is a complete binary tree.

B. Heap arbitrary search can be $O(n)$.

C. Heap Sort is stable.

D. Heap gives fast root min/max access.

Q14 — MCQ

For Top K Frequent Elements, a common combination is:

A. HashMap + Heap

B. DFS + Binary Search

C. Stack + Queue only

D. Union-Find + MST

Q15 — MCQ

Python `heapq` is naturally:

A. Max Heap

B. Min Heap

C. BST

D. Stack

PRACTICE SOLUTIONS

Q1

Root of Min Heap is minimum.

$$\boxed{B}$$

Q2

$$\boxed{B}$$

Q3

Build Heap:

$$\boxed{O(n)}$$

Answer:

$$\boxed{B}$$

Q4

Kth largest:

$$\boxed{\text{Min Heap size K}}$$

Answer:

$$\boxed{A}$$

Q5

Kth smallest:

$$\boxed{\text{Max Heap size K}}$$

Answer:

$$ \boxed{B} $$

Q6

Sorted descending:

```
TEXT
6,5,4,3,2,1
```

2nd largest:

$$ \boxed{5} $$

Q7

A, B, C.

$$ \boxed{A,B,C} $$

Q8

Heap has at most k list heads.

$$ \boxed{O(N\log k)} $$

Answer:

$$ \boxed{B} $$

Q9

$$ \boxed{B} $$

Q10

A, B, C true.

D false.

$$ \boxed{A,B,C} $$

Q11

Take:

$$ 1+2=3 $$

Cost:

$$ 3 $$

Now:

```text
3,3
```

Connect:

\[3+3=6 \]

Total:

\[3+6 = \boxed{9} \]

Q12

\[\boxed{A} \]

Q13

A true.

B true.

C false.

D true.

\[\boxed{A,B,D} \]

Q14

\[\boxed{A} \]

Q15

\[\boxed{B} \]

Ø=ÜÓTES TO WRITE

```text
HEAP

Min Heap:
root = minimum

Max Heap:
root = maximum

Operations:
peek      O(1)
insert    O(log n)
remove    O(log n)
build     O(n)
search    O(n)
```

```
TOP K:
 K t h   L a r g e s t   !’   M i n   H e a p   s i z e   K
 K t h   S m a l l e s t  !’   M a x   H e a p   s i z e   K

Frequency + Top K:
HashMap + Heap

Merge K sorted:
Min Heap of current heads

Streaming Median:
 L o w e r   h a l f  !’   M a x   H e a p
 U p p e r   h a l f  !’   M i n   H e a p

Connect ropes:
combine two smallest repeatedly
```

COMPLEXITIES TO MEMORIZE

Problem	Complexity
Heap insert	\(O(\log n)\)
Heap delete root	\(O(\log n)\)
Peek min/max	\(O(1)\)
Build Heap	\(O(n)\)
Heap Sort	\(O(n\log n)\)
Kth largest size-K heap	\(O(n\log k)\)
Top K frequent	\(O(n\log k)\) typical
Merge K sorted lists	\(O(N\log k)\)
Streaming median insertion	\(O(\log n)\)
Median lookup	\(O(1)\)

&β30-SECOND HEAP REVISION

1. Min Heap !’ minimum at root.
2. Max Heap !’ maximum at root.
3. Heap is complete binary tree.
4. Insert/delete root !’ \(O(\log n)\).
5. Peek !’ \(O(1)\).
6. Build Heap !’ \(O(n)\).
7. Heap arbitrary search !’ \(O(n)\).
8. Kth largest !’ Min Heap K.
9. Kth smallest !’ Max Heap K.

10. Top K !’ think Heap.
11. Frequency + Top K !’ HashMap + Heap.
12. Merge K sorted !’ heap of current heads.
13. Streaming Median !’ two heaps.
14. Connect Ropes !’ repeatedly combine two smallest.
15. Heap is not fully sorted.
16. Heap vs sorting: use Heap when only priority/top-K matters.

' Placement Module 9 Complete

Next Placement Module:

MODULE 10 — GRAPHS FOR PLACEMENTS: BFS, DFS, GRIDS, CYCLES + TOPOLOGICAL SORT

We’ll cover:

- graph representation,
- adjacency lists,
- BFS/DFS coding templates,
- connected components,
- Number of Islands,
- Flood Fill,
- cycle detection in undirected graphs,
- cycle detection in directed graphs,
- bipartite checking,
- topological sorting,
- Course Schedule,
- multi-source BFS,
- shortest path in unweighted graphs,
- and the most important graph interview patterns.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 10 — GRAPHS FOR PLACEMENTS: BFS, DFS, GRIDS, CYCLES + TOPOLOGICAL SORT

Placement Priority: & & & & &

Graphs are one of the most important placement topics because many interview problems that do **not look like**

graphs are actually graph problems.

Examples:

- cities connected by roads,
- people connected as friends,
- islands in a grid,
- course prerequisites,
- network connectivity,
- shortest path in a maze,
- dependency resolution.

The first skill is:

$$\boxed{\text{Recognize the graph hidden inside the problem}}$$

---

1. WHAT IS A GRAPH?

A graph contains:

- **Vertices / Nodes**
- **Edges / Connections**

Example:

```text
A ---- B
|      |
|      |
C ---- D
```

Vertices:

```text
A, B, C, D
```

Edges:

```text
A-B
A-C
B-D
C-D
```

---

2. DIRECTED VS UNDIRECTED GRAPH

Undirected

Connection works both ways.

```text
A ----- B
```

Means:

```text
 A   !′    B
 B   !′    A
```

Examples:

- friendship
- two-way roads

Directed

Edge has direction.

```text
 A   !′    B
```

Does not automatically mean:

```text
 B   !′    A
```

Examples:

- course prerequisites
- Twitter-style following
- dependency graphs

---

3. WEIGHTED VS UNWEIGHTED

Unweighted

```text
A -- B
```

All edges treated equally.

Often shortest path means:

> **minimum number of edges.**

Use:

\[\boxed{\text{BFS}} \]

Weighted

```text
A --5-- B
```

Edge has cost/distance/time.

Shortest path may require:

- Dijkstra
- Bellman-Ford
- DAG shortest path

depending on weights.

---

4. GRAPH REPRESENTATION

Two common ways:

Adjacency Matrix

Adjacency List

For placements, **Adjacency List** is usually most useful.

---

5. ADJACENCY LIST

Graph:

```text
0 -- 1
|    |
2 -- 3
```

Adjacency list:

```text
 0  !'   [ 1 , 2 ]
 1  !'   [ 0 , 3 ]
 2  !'   [ 0 , 3 ]
 3  !'   [ 1 , 2 ]
```

Python:

```python
graph = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
    3: [1, 2]
}
```

---

6. BUILDING FROM EDGE LIST

Suppose:

```text
edges = [
  [0,1],
  [0,2],
  [1,3]
]
```

Undirected graph:

```python
graph = [[] for _ in range(4)]

for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)
```

Directed graph:

```python
for u, v in edges:
    graph[u].append(v)
```

Only one direction.

---

7. COMPLEXITY OF ADJACENCY LIST

Space:

$$ \boxed{O(V+E)} $$

For undirected graph, each edge appears twice, but asymptotically still:

$$ O(V+E) $$

PART A — BFS

8. WHAT IS BFS?

BFS = Breadth-First Search.

It explores:

$$ \boxed{\text{level by level}} $$

Uses:

$$ \boxed{\text{Queue}} $$

---

9. EXAMPLE

Graph:

```text
        0
      /   \
     1     2
    /       \
   3         4
```

Starting from 0:

```text
Level 0: 0
Level 1: 1,2
Level 2: 3,4
```

Possible BFS:

```text
0 1 2 3 4
```

---

10. BFS TEMPLATE

```python
from collections import deque

def bfs(graph, start):
    visited = set([start])
    q = deque([start])

    while q:
        node = q.popleft()

        for nei in graph[node]:
            if nei not in visited:
                visited.add(nei)
                q.append(nei)
```

Time:

\[\boxed{O(V+E)} \]

Space:

\[\boxed{O(V)} \]

---

11. IMPORTANT RULE — MARK WHEN ENQUEUED

Preferred:

```text
mark visited
then enqueue
```

not:

> **enqueue several times and mark only much later.**

Why?

If multiple neighbors discover same node before it is processed, duplicate queue entries may occur.

So:

```python
if nei not in visited:
    visited.add(nei)
    q.append(nei)
```

---

12. WHEN TO THINK BFS

Trigger concepts:

- shortest path in unweighted graph
- minimum number of moves
- levels
- nearest
- minimum steps
- spreading process
- grid shortest path
- multi-source expansion

Memory:

> **Minimum edges / levels !' BFS**

PART B — DFS

13. WHAT IS DFS?

DFS = Depth-First Search.

It explores one path deeply before backtracking.

Uses:

- recursion
- explicit stack

---

14. RECURSIVE DFS

```python
def dfs(node, graph, visited):
    visited.add(node)

    for nei in graph[node]:
        if nei not in visited:
            dfs(nei, graph, visited)
```

Time:

$$ \boxed{O(V+E)} $$

Space:

$$ O(V) $$

worst case due to visited + recursion stack.

---

15. ITERATIVE DFS

```python
def dfs_iterative(graph, start):
    stack = [start]
    visited = set()

    while stack:
        node = stack.pop()

        if node in visited:
            continue

        visited.add(node)

        for nei in graph[node]:
            if nei not in visited:
                stack.append(nei)
```

---

16. BFS VS DFS

BFS	DFS
Queue	Stack/recursion
Level-wise	Depth-wise
Unweighted shortest path	Connectivity/path exploration
Often shortest steps	Often simpler recursion
Useful for multi-source	Useful for cycle/components

Both:

$$ \boxed{O(V+E)} $$

with adjacency list.

PART C — CONNECTED COMPONENTS

17. PROBLEM

Graph:

```text
0 -- 1

2 -- 3 -- 4

5
```

Components:

```text
{0,1}
{2,3,4}
{5}
```

Number:

\[\boxed{3} \]

18. APPROACH

Maintain visited.

For every vertex:

```text
if unvisited:
    BFS/DFS from it
    component_count++
```

19. CODE

```python
def count_components(n, graph):
    visited = set()
    count = 0

    def dfs(node):
        visited.add(node)

        for nei in graph[node]:
            if nei not in visited:
                dfs(nei)

    for node in range(n):
        if node not in visited:
            count += 1
```

```
            dfs(node)

    return count
```

Time:

\[\boxed{O(V+E)} \]

---

20. INTERVIEW SIGNAL

If question says:

- groups
- clusters
- provinces
- isolated networks
- number of connected regions

think:

\[\boxed{\text{Connected Components}} \]

using BFS/DFS.

PART D — NUMBER OF ISLANDS

This is one of the highest-frequency graph/grid interview problems.

21. PROBLEM

Grid:

```text
1 1 0 0
1 0 0 1
0 0 1 1
```

Assume `1 = land`.

Islands:

```text
top-left group
right-bottom group
```

Answer:

\[\boxed{2} \]

---

22. HIDDEN GRAPH

Each land cell is a node.

Edges connect neighboring land cells.

Usually neighbors:

```text
up
down
left
right
```

Therefore:

> **Number of Islands = Number of Connected Components in a grid.**

---

23. DFS APPROACH

Whenever unvisited land found:

```text
count++
DFS and mark entire island
```

---

24. CODE

```python
def num_islands(grid):
    rows = len(grid)
    cols = len(grid[0])

    count = 0

    def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols:
            return

        if grid[r][c] != '1':
            return

        grid[r][c] = '0'

        dfs(r + 1, c)
        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)

    return count
```

Time:

\[\boxed{O(RC)} \]

because each cell processed at most once.

25. GRID DIRECTION TRICK

For 4-direction movement:

```python
directions = [
    (1, 0),
    (-1, 0),
    (0, 1),
    (0, -1)
]
```

Then:

```python
for dr, dc in directions:
    nr = r + dr
    nc = c + dc
```

Useful in many grid problems.

PART E — FLOOD FILL

26. PROBLEM

Similar to paint bucket tool.

Grid:

```text
1 1 1
1 1 0
1 0 1
```

Starting at center with new color 2.

Change all connected cells having original color.

This is simply:

\[\boxed{\text{DFS/BFS on grid}} \]

27. IMPORTANT TRAP

If:

```text
newColor == oldColor
```

return immediately in some implementations.

Otherwise you may repeatedly revisit unchanged cells unless visited tracking is separate.

PART F — SHORTEST PATH IN UNWEIGHTED GRAPH

28. WHY BFS?

Starting source:

```text
distance = 0
```

Neighbors:

```text
distance = 1
```

Their unvisited neighbors:

```text
distance = 2
```

Thus first time BFS reaches a node:

\[\boxed{\text{shortest number of edges}} \]

29. DISTANCE ARRAY

```python
from collections import deque

def shortest_unweighted(graph, source):
    n = len(graph)
    dist = [-1] * n

    dist[source] = 0
    q = deque([source])

    while q:
        node = q.popleft()

        for nei in graph[node]:
            if dist[nei] == -1:
                dist[nei] = dist[node] + 1
                q.append(nei)

    return dist
```

PART G — SHORTEST PATH IN GRID

30. PROBLEM FAMILY

Examples:

- shortest path in maze
- minimum moves
- minimum steps from start to end
- binary matrix shortest path

If:

- each move has same cost,
- movement is grid adjacency,

use:

\[\boxed{\text{BFS}} \]

---

31. GRID BFS STATE

Queue stores:

```
TEXT
(row, col, distance)
```

or maintain distance separately.

Mark cell visited when enqueued.

PART H — MULTI-SOURCE BFS

Extremely important placement pattern.

32. NORMAL BFS

Starts from one source.

```
TEXT
 s o u r c e  !'  s p r e a d  o u t w a r d
```

---

33. MULTI-SOURCE BFS

Start queue with **all sources simultaneously**.

Example:

```
TEXT
0 . . 0
. . . .
0 . . .
```

Suppose every `0` is a source.

Put all zero cells in queue initially.

Then BFS expands them together.

34. WHEN TO THINK MULTI-SOURCE BFS

Problems like:

- Rotten Oranges
- nearest zero
- nearest hospital
- fire spread
- contamination spread
- distance to nearest source

Memory:

> **Many starting points spreading simultaneously !' Multi-source BFS**

PART I — ROTTING ORANGES

35. PROBLEM

Grid values:

```text
0 = empty
1 = fresh
2 = rotten
```

Every minute, rotten orange infects adjacent fresh oranges.

Find minimum minutes.

36. KEY INSIGHT

All initially rotten oranges spread **at same time**.

Therefore queue begins with:

$$\boxed{\text{all rotten oranges}}$$

This is Multi-source BFS.

37. LEVEL = TIME

Each BFS level corresponds to:

$$\boxed{1\text{ minute}}$$

This "BFS levels = elapsed time" idea appears often.

PART J — UNDIRECTED GRAPH CYCLE DETECTION

38. PROBLEM

Graph:

```text
0 -- 1
|    |
2 -- 3
```

contains a cycle:

```text
 0  !'   1  !'   3  !'   2  !'   0
```

39. WHY VISITED NEIGHBOR IS NOT ENOUGH

Suppose:

```text
0 -- 1
```

DFS from 0 visits 1.

At 1, neighbor 0 is already visited.

But that's just the parent edge, not a cycle.

Therefore track:

\[\boxed{\text{parent}} \]

40. RULE

During DFS in undirected graph:

If neighbor is:

- unvisited !' DFS
- visited and `neighbor != parent` !' cycle

41. CODE

```python
def has_cycle_undirected(graph):
    visited = set()

    def dfs(node, parent):
        visited.add(node)

        for nei in graph[node]:
```

```
            if nei not in visited:
                if dfs(nei, node):
                    return True

            elif nei != parent:
                return True

        return False

    for node in range(len(graph)):
        if node not in visited:
            if dfs(node, -1):
                return True

    return False
```

PART K — DIRECTED GRAPH CYCLE DETECTION

This is different.

42. WHY PARENT METHOD DOESN'T WORK

Directed edges have direction.

Need to know whether we reached a node currently in the active DFS path.

Use:

```text
visited
recursion_path
```

or 3 colors.

---

43. THREE-COLOR METHOD

```text
WHITE = unvisited
GRAY  = currently exploring
BLACK = completely finished
```

If DFS reaches:

\[\boxed{\text{GRAY}} \]

node:

> **directed cycle exists.**

---

44. INTUITION

GRAY means:

> **this node is one of my ancestors in the current recursion chain.**

An edge back to ancestor creates:

$$ \boxed{\text{cycle}} $$

45. CODE

```python
def has_cycle_directed(graph):
    n = len(graph)

    color = [0] * n
    # 0 = white
    # 1 = gray
    # 2 = black

    def dfs(node):
        color[node] = 1

        for nei in graph[node]:
            if color[nei] == 1:
                return True

            if color[nei] == 0 and dfs(nei):
                return True

        color[node] = 2
        return False

    for node in range(n):
        if color[node] == 0:
            if dfs(node):
                return True

    return False
```

PART L — TOPOLOGICAL SORT

Very important for prerequisites and dependencies.

46. WHAT IS TOPOLOGICAL ORDER?

A linear order of vertices such that for every directed edge:

$$ u\rightarrow v $$

u appears before v.

47. EXAMPLE

Dependencies:

```TEXT
A  !′   C
B  !′   C
C  !′   D
```

Possible topological order:

```TEXT
A B C D
```

or:

```TEXT
B A C D
```

Topological order may not be unique.

---

48. WHEN DOES TOPOLOGICAL ORDER EXIST?

Only for:

\[\boxed{\text{DAG}} \]

Directed Acyclic Graph.

If directed cycle exists:

> **no topological ordering.**

PART M — KAHN'S ALGORITHM

49. INDEGREE

Indegree of node:

> **number of incoming edges.**

Example:

```TEXT
A  !′   C
B  !′   C
```

Then:

\[indegree(C)=2 \]

---

50. KAHN'S ALGORITHM

1. Compute indegrees.
2. Put all indegree-0 nodes into queue.
3. Remove one.
4. Add to topological order.
5. Reduce indegree of its outgoing neighbors.
6. If neighbor reaches 0, enqueue it.

51. CODE

```python
from collections import deque

def topo_sort(n, graph):
    indegree = [0] * n

    for u in range(n):
        for v in graph[u]:
            indegree[v] += 1

    q = deque()

    for i in range(n):
        if indegree[i] == 0:
            q.append(i)

    order = []

    while q:
        node = q.popleft()
        order.append(node)

        for nei in graph[node]:
            indegree[nei] -= 1

            if indegree[nei] == 0:
                q.append(nei)

    return order
```

Time:

$$\boxed{O(V+E)}$$

52. CYCLE DETECTION USING KAHN

If:

```text
len(order) < V
```

then some nodes never reached indegree zero.

That indicates:

$$\boxed{\text{directed cycle}}$$

PART N — COURSE SCHEDULE

Very common interview problem.

53. PROBLEM

Suppose:

```text
course 1 requires course 0
```

Represent:

```text
 0  !′   1
```

Question:

> **Can all courses be completed?**

This means:

> **Is prerequisite graph acyclic?**

Therefore solve using:

- directed cycle detection, or
- Kahn's topological sort.

---

54. COURSE SCHEDULE II

Follow-up:

> **Return a valid order.**

Now directly return:

\[\boxed{\text{Topological Sort}} \]

PART O — DFS TOPOLOGICAL SORT

Another method.

55. IDEA

In DFS:

> **add node after all descendants are processed.**

This is:

\[\boxed{\text{postorder}} \]

Then reverse finishing order.

---

56. CODE IDEA

```python
visited = set()
order = []

def dfs(node):
    visited.add(node)

    for nei in graph[node]:
        if nei not in visited:
            dfs(nei)

    order.append(node)

# run from all nodes
order.reverse()
```

But if graph might contain cycles:

> **cycle detection must also be included.**

Kahn's algorithm often makes cycle handling very clear in interviews.

PART P — BIPARTITE GRAPH

57. WHAT IS BIPARTITE?

Can divide vertices into two groups such that every edge connects opposite groups.

Equivalent:

> **graph can be colored using 2 colors with no adjacent equal colors.**

---

58. BFS/DFS COLORING

Assign:

```text
color[start] = 0
```

For neighbor:

```text
color[neighbor] = 1 - color[node]
```

If already colored and same as current:

> **not bipartite.**

---

59. IMPORTANT THEOREM

An undirected graph is bipartite iff it has:

\[\boxed{\text{no odd-length cycle}} \]

Example triangle:

```text
A
|\
| \
B--C
```

3-cycle.

Cannot 2-color.

---

60. CODE

```python
from collections import deque

def is_bipartite(graph):
    n = len(graph)
    color = [-1] * n

    for start in range(n):
        if color[start] != -1:
            continue

        q = deque([start])
        color[start] = 0

        while q:
            node = q.popleft()

            for nei in graph[node]:
                if color[nei] == -1:
                    color[nei] = 1 - color[node]
                    q.append(nei)

                elif color[nei] == color[node]:
                    return False

    return True
```

PART Q — CLONE GRAPH

61. PROBLEM

Given node in a graph, create deep copy.

Main challenge:

- cycles
- repeated references

Need map:

```text
original node !' copied node
```

---

62. HASHMAP + DFS/BFS

When visiting original node:

If already copied:

> **return existing clone.**

Otherwise:

1. create clone
2. store in map immediately
3. recursively clone neighbors

This avoids infinite recursion.

---

63. PATTERN

Graph copying:

$$\boxed{\text{Traversal + HashMap}}$$

HashMap preserves one-to-one mapping.

PART R — SURROUNDED REGIONS / BOUNDARY DFS

Common grid pattern.

64. PROBLEM IDEA

Grid has:

```text
X
O
```

Any `O` region completely surrounded by X should become X.

Key insight:

> **O s connected to boundary cannot be captured.**

So:

1. Start DFS/BFS from boundary O's.
2. Mark them safe.
3. Convert remaining O's.

This is a very useful reverse-thinking pattern.

PART S — PACIFIC ATLANTIC TYPE

65. REVERSE GRAPH THINKING

Sometimes asking:

> **From which cells can water flow to both oceans?**

Instead of starting from every cell and trying to reach oceans:

> **start from ocean boundaries and traverse in reverse condition.**

Compute:

```text
reachable from Pacific
reachable from Atlantic
```

Intersection is answer.

This is an important graph optimization technique:

\[\boxed{\text{Reverse the direction of the question}} \]

PART T — WORD LADDER

66. PROBLEM FAMILY

Transform:

```text
 h i t  !’   c o g
```

changing one letter at a time, each intermediate word valid.

Each word is a node.

Edge exists if two words differ by one character.

Need minimum number of transformations.

That's:

$$ \boxed{\text{Unweighted shortest path}} $$

Therefore:

$$ \boxed{\text{BFS}} $$

PART U — GRAPH PATTERN RECOGNITION

67. MASTER TABLE

Problem signal	Think
Connectivity	DFS/BFS
Number of groups/components	DFS/BFS
Grid islands	DFS/BFS
Minimum steps, equal edge cost	BFS
Many simultaneous sources	Multi-source BFS
Undirected cycle	DFS/BFS + parent
Directed cycle	DFS colors / Kahn
Dependencies	Topological Sort
Course prerequisites	Topological Sort
Two-coloring	Bipartite
Copy graph	DFS/BFS + HashMap
Grid region	DFS/BFS
Shortest transformation	BFS

PART V — BFS VS DFS DECISION RULE

68. USE BFS WHEN

Question emphasizes:

- minimum number of moves,
- closest,
- shortest unweighted path,
- levels,

- spreading over time.

69. USE DFS WHEN

Question emphasizes:

- explore region,
- count components,
- recursive path exploration,
- cycle via recursion,
- process all connected nodes.

70. EITHER OFTEN WORKS FOR

- connected components
- islands
- flood fill
- reachability

Then choose whichever is simpler.

PART W — COMMON GRAPH INTERVIEW TRAPS

71. TRAP 1 — FORGETTING DISCONNECTED COMPONENTS

Bad:

```text
DFS only from node 0
```

This may miss disconnected vertices.

If entire graph must be processed:

```text
for every vertex:
    if unvisited:
        DFS/BFS
```

72. TRAP 2 — UNDIRECTED EDGE INSERTION

For undirected:

```python
graph[u].append(v)
graph[v].append(u)
```

Need both.

73. TRAP 3 — MARK VISITED TOO LATE

In BFS, prefer marking when enqueuing to avoid duplicate queue entries.

74. TRAP 4 — USING BFS SHORTEST PATH ON WEIGHTED GRAPH

Ordinary BFS does not solve arbitrary weighted shortest path.

It works for:

\[\boxed{\text{equal edge weights}} \]

75. TRAP 5 — CYCLE LOGIC SAME FOR DIRECTED/UNDIRECTED

False.

```text
Undirected !' parent tracking
Directed   !' recursion stack/colors
```

76. TRAP 6 — TOPOLOGICAL SORT FOR UNDIRECTED GRAPH

Topological sorting is for:

\[\boxed{\text{Directed Acyclic Graphs}} \]

77. TRAP 7 — MODIFYING GRID

If you overwrite grid cells to mark visited, ensure:

> **problem allows mutation.**

Otherwise use separate visited matrix/set.

78. TRAP 8 — RECURSION DEPTH

Deep DFS recursion can overflow recursion stack.

Iterative DFS may be safer for very large graphs.

79. TRAP 9 — BFS $O(V^2)$

With adjacency list:

\[\boxed{O(V+E)} \]

not \(O(V^2)\) unless using matrix or dense assumptions.

80. TRAP 10 — TOPOLOGICAL ORDER UNIQUE

Not necessarily.

Example:

```text
A  !'   C
B  !'   C
```

Both:

```text
A,B,C
```

and:

```text
B,A,C
```

are valid.

PART X — INTERVIEW EXPLANATION EXAMPLE

Suppose interviewer asks:

> **Find number of islands.**

Strong answer:

> **"I can model each land cell as a graph node connected to its adjacent land cells. Then the number of islands is exactly the number of connected components. I scan every grid cell, and whenever I find unvisited land, I increment the island count and run DFS or BFS to mark the entire connected region as visited. Since every cell is processed at most once, the time complexity is \(O(RC)\), and the auxiliary space is \(O(RC)\) in the worst case due to the recursion stack or queue."**

That's placement-quality communication.

PART Y — MUST-DO GRAPH QUESTIONS

Foundation

1. Build Adjacency List
2. BFS
3. DFS
4. Connected Components
5. Reachability

Grid Core

6. Number of Islands
7. Flood Fill
8. Max Area of Island
9. Rotting Oranges
10. Shortest Path in Binary Matrix
11. Number of Enclaves
12. Surrounded Regions

Graph Core

13. Undirected Cycle Detection
14. Directed Cycle Detection
15. Is Graph Bipartite?
16. Clone Graph
17. Course Schedule
18. Course Schedule II
19. Topological Sort

Important Medium

20. Word Ladder
21. Pacific Atlantic Water Flow
22. All Paths Source to Target
23. Eventual Safe States
24. Minimum Height Trees
25. Network connectivity variants

PART Z — PLACEMENT PRACTICE SET

Q1 — MCQ

BFS naturally uses:

A. Stack B. Queue C. Heap D. Union-Find

Q2 — MCQ

DFS naturally uses:

A. Stack/recursion B. Queue only C. Priority Queue only D. Prefix Sum

Q3 — MCQ

Shortest path in an unweighted graph is naturally solved using:

A. DFS B. BFS C. Heap Sort D. Kruskal

Q4 — MSQ

Which can be solved using BFS or DFS?

A. Connected Components

B. Number of Islands

C. Flood Fill

D. Merge Sort

Q5 — NAT

A graph has:

```text
Component 1: {0,1,2}
Component 2: {3,4}
Component 3: {5}
```

How many connected components?

Q6 — MCQ

In undirected DFS cycle detection, if a visited neighbor is found, it indicates a cycle only when:

A. neighbor is parent

B. neighbor is not parent

C. graph has weights

D. neighbor has larger value

Q7 — MCQ

For directed cycle detection with DFS colors, an edge to which color indicates a cycle?

A. WHITE B. GRAY C. BLACK only D. Any color

Q8 — MCQ

Topological ordering exists only for:

A. every undirected graph

B. DAG

C. every weighted graph

D. complete graphs only

Q9 — MSQ

Which are good Topological Sort problems?

A. Course prerequisites

B. Dependency ordering

C. Build order

D. Maximum Subarray

Q10 — MCQ

Kahn's algorithm primarily uses:

A. indegree + queue

B. heap + DFS

C. stack + sorting

D. Binary Search

Q11 — MCQ

If Kahn's algorithm processes fewer than V nodes:

A. graph must be tree

B. directed cycle exists

C. graph has no edges

D. graph is bipartite

Q12 — MCQ

Rotting Oranges is best recognized as:

A. Binary Search

B. Multi-source BFS

C. Merge Sort

D. Dynamic Programming only

Q13 — MSQ

Which statements about bipartite graphs are true?

A. They can be colored using two colors.

B. Adjacent vertices must have different colors.

C. An odd cycle prevents bipartiteness.

D. Every graph with a cycle is non-bipartite.

Q14 — MCQ

In graph cloning, HashMap usually stores:

A. node degree only

B. original node !’ copied node

C. edge weight !’ index

D. source !’ distance only

Q15 — MCQ

Time complexity of BFS/DFS using adjacency lists is:

A. \(O(V)\)

B. \(O(E)\)

C. \(O(V+E)\)

D. \(O(VE)\)

PRACTICE SOLUTIONS

Q1

\[\boxed{B} \]

Q2

\[\boxed{A} \]

Q3

\[\boxed{B} \]

Q4

Connected Components, Number of Islands and Flood Fill.

\[\boxed{A,B,C} \]

Q5

\[\boxed{3} \]

Q6

Visited neighbor different from parent implies cycle.

$$\boxed{B}$$

Q7

GRAY means currently active DFS path.

$$\boxed{B}$$

Q8

$$\boxed{B}$$

Q9

A, B, C.

$$\boxed{A,B,C}$$

Q10

$$\boxed{A}$$

Q11

Unprocessed vertices indicate dependency cycle.

$$\boxed{B}$$

Q12

All rotten oranges start simultaneously.

$$\boxed{B}$$

Q13

A true.

B true.

C true.

D false because even cycles are bipartite.

$$\boxed{A,B,C}$$

Q14

\[\boxed{B} \]

Q15

\[\boxed{C} \]

Ø=ÑÓTES TO WRITE

```text
GRAPH REPRESENTATION:
 A d j a c e n c y  L i s t  !’  O ( V + E )  s p a c e

BFS:
Queue
Level-wise
O(V+E)
Unweighted shortest path

DFS:
Stack/Recursion
O(V+E)
Connectivity

CONNECTED COMPONENTS:
for every unvisited node:
    count++
    BFS/DFS

GRID:
cells = graph nodes
4-direction neighbors

Number of Islands:
Connected Components

Multi-source BFS:
many starting points at once
Examples:
Rotting Oranges
Nearest source

UNDIRECTED CYCLE:
visited neighbor != parent

DIRECTED CYCLE:
GRAY / recursion-path node

TOPOLOGICAL SORT:
DAG only

Kahn:
indegree + queue

processed < V:
cycle exists

BIPARTITE:
2-color graph
```

```
odd cycle !’ not bipartite
```

&β30-SECOND GRAPH REVISION

1. Graph = vertices + edges.
2. Adjacency List = \(O(V+E)\).
3. BFS !’ Queue.
4. DFS !’ Stack/recursion.
5. BFS/DFS = \(O(V+E)\).
6. BFS gives unweighted shortest path.
7. Components !’ repeated BFS/DFS.
8. Number of Islands = grid components.
9. Grid problems are often hidden graphs.
10. Many simultaneous sources !’ Multi-source BFS.
11. Rotten Oranges !’ Multi-source BFS.
12. Undirected cycle !’ visited non-parent neighbor.
13. Directed cycle !’ GRAY/active-path neighbor.
14. Topological sort !’ DAG only.
15. Kahn !’ indegree + queue.
16. Fewer than V processed !’ directed cycle.
17. Course Schedule = cycle/topological problem.
18. Bipartite = 2-colorable.
19. Odd cycle !’ not bipartite.
20. Graph cloning !’ traversal + original-to-clone map.

' Placement Module 10 Complete

Next Placement Module:

MODULE 11 — SHORTEST PATHS+ DIJKSTRA + MST + UNION-FIND

We’ll cover:

- weighted graph shortest paths,
- Dijkstra coding pattern,
- Bellman-Ford interview use,
- DAG shortest path,
- 0-1 BFS,
- Prim,
- Kruskal,
- Disjoint Set Union,
- path compression,
- union by rank/size,

- redundant connections,
- network connectivity,
- MST interview problems,
- and the exact rule for choosing **BFS vs Dijkstra vs Bellman-Ford vs DSU vs MST**.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 11 — SHORTEST PATHS+ DIJKSTRA + MST + UNION-FIND

Placement Priority: & & & & &

This module connects several graph ideas that interviewers often mix together.

The most important skill is choosing the correct tool:

\[\boxed{\text{BFS vs Dijkstra vs Bellman-Ford vs MST vs DSU}} \]

Do not choose an algorithm just because the problem says "graph."

---

1. FIRST DECISION: WHAT IS THE PROBLEM ASKING?

Ask:

Is it asking shortest path?

Then think:

- BFS
- Dijkstra
- Bellman-Ford
- DAG shortest path

Is it asking minimum cost to connect all vertices?

Then think:

\[\boxed{\text{MST}} \]

using:

- Prim
- Kruskal

Is it asking connectivity / whether two nodes belong to same group?

Think:

\[\boxed{\text{DSU / Union-Find}} \]

2. MASTER ALGORITHM SELECTION TABLE

Situation	Algorithm
Unweighted graph	BFS
All edge weights equal	BFS
Non-negative weighted graph	Dijkstra
Negative edges possible	Bellman-Ford
DAG weighted graph	Topological relaxation
Edges only 0 or 1	0-1 BFS
Connect all vertices with minimum total edge cost	MST
Repeated connectivity/merge queries	DSU

Memorize this table.

PART A — SHORTEST PATH BASICS

3. WHAT IS SHORTEST PATH?

Given source \(S\), find minimum total path cost to other vertices.

Example:

```text id="m11a01" A --4-- B

```

1 2

```

C --1-- D

```

```
A path from A to B could be:

```text id="m11a02"
 A  !’   B
cost = 4
```

or:

```text id="m11a03" A !’ C !’ D !’ B cost = 1 + 1 + 2 = 4

```

```
Shortest path cost:
```
```

```
\[
4
\]

---

# 4. RELAXATION

This is the heart of most shortest-path algorithms.

Suppose:

\[
dist[u]=5
\]

edge:

\[
u\rightarrow v
\]

weight:

\[
3
\]

Current:

\[
dist[v]=12
\]

Going through \(u\):

\[
5+3=8
\]

Since:

\[
8<12
\]

update:

\[
dist[v]=8
\]

Formula:

\[
\boxed{
dist[v]=\min(dist[v],dist[u]+w(u,v))
}
\]

This is called:

\[
\boxed{\text{Relaxation}}
\]
```

```
---

# PART B — DIJKSTRA'S ALGORITHM

# 5. WHEN TO USE DIJKSTRA

Use Dijkstra for:

\[
\boxed{\text{weighted shortest path with non-negative edge weights}}
\]

Zero-weight edges are fine.

Negative edges are not.

---

# 6. MAIN IDEA

Maintain current best known distance.

Always process the vertex with the:

\[
\boxed{\text{smallest tentative distance}}
\]

Use:

\[
\boxed{\text{Min Heap / Priority Queue}}
\]

This is why your Heap module directly connects to Dijkstra.

---

# 7. EXAMPLE

Graph:

```text id="m11a04"
A --4--> B
|        ^
1        |
v        2
C --1--> D
```
```

Source A.

Initial:

```text id="m11a05" A = 0 B = " C = " D = "

```
From A:

```text id="m11a06"
B = 4
C = 1
```
```

Next smallest:
```

\[C=1 \]

From C:

\[D=1+1=2 \]

Next:

\[D=2 \]

From D:

\[B=\min(4,2+2) \]

\[=4 \]

Final:

```text id="m11a07" A = 0 C = 1 D = 2 B = 4

---

# 8. DIJKSTRA TEMPLATE

```python id="m11a08"
import heapq

def dijkstra(n, graph, source):
    dist = [float("inf")] * n
    dist[source] = 0

    heap = [(0, source)]

    while heap:
        current_dist, node = heapq.heappop(heap)

        if current_dist != dist[node]:
            continue

        for nei, weight in graph[node]:
            new_dist = current_dist + weight

            if new_dist < dist[nei]:
                dist[nei] = new_dist
                heapq.heappush(heap, (new_dist, nei))

    return dist
```

---

9. GRAPH FORMAT

For weighted adjacency list:

```text id="m11a09" graph[u] = [(v, weight), ...]

Example:

```python id="m11a10"
graph = [
    [(1, 4), (2, 1)],
    [],
    [(3, 1)],
```

```
    [(1, 2)]
]
```

---

10. WHY THIS CHECK?

```python id="m11a11" if current_dist != dist[node]: continue

```

```
Because Python `heapq` does not decrease an existing key directly.

Instead, we may push a better copy.

Old outdated entries can remain in heap.

When popped:

> ignore stale one.

This is an important placement detail.

---

# 11. DIJKSTRA COMPLEXITY

Using adjacency list + binary heap:

\[
\boxed{O((V+E)\log V)}
\]

Often simplified for connected sparse graphs as:

\[
\boxed{O(E\log V)}
\]

Space:

\[
O(V+E)
\]

including graph and auxiliary structures.

---

# 12. DIJKSTRA INTERVIEW SIGNAL

Words like:

- minimum travel cost
- shortest road distance
- network delay
- cheapest non-negative weighted path
- minimum time where all weights are non-negative

should trigger:

\[
\boxed{\text{Dijkstra}}
\]
```

```
---

# 13. CRITICAL DIJKSTRA TRAP

Negative edge:

```text id="m11a12"
 A  !’   B   =   5
 A  !’   C   =   2
 B  !’   C   =   - 1 0
```
```

Dijkstra's greedy finalization assumption may fail.

So:

\[\boxed{\text{Negative edge !’ do not blindly use Dijkstra}} \]

PART C — NETWORK DELAY TIME PATTERN

14. PROBLEM FAMILY

Given directed weighted network:

> **signal starts at source. How long until every node receives it?**

Run Dijkstra.

Get distances to all nodes.

If any remain infinity:

> **unreachable.**

Otherwise answer:

\[\boxed{\max(dist)} \]

Because the last node receives the signal at the largest shortest-path time.

Pattern:

\[\boxed{\text{Dijkstra + maximum final distance}} \]

PART D — BELLMAN-FORD

15. WHEN TO USE

Use when:

\[\boxed{\text{negative edge weights may exist}} \]

It can also detect reachable negative cycles.

---

16. MAIN IDEA

Relax every edge repeatedly.

For \(V\) vertices:

\[\boxed{V-1} \]

main passes are sufficient when no negative cycle is reachable.

---

17. WHY V-1?

A simple path can contain at most:

\[V-1 \]

edges.

After pass \(k\), shortest paths using at most \(k\) edges can be correctly propagated.

---

18. TEMPLATE

```python id="m11a13" def bellman_ford(n, edges, source): dist = [float("inf")] * n dist[source] = 0

for _ in range(n - 1): changed = False

for u, v, w in edges: if dist[u] != float("inf") and dist[u] + w < dist[v]: dist[v] = dist[u] + w changed = True

if not changed: break

return dist

```

---

19. NEGATIVE CYCLE DETECTION

After \(V-1\) passes, do one more scan.

If some edge can still relax:

\[
dist[u]+w<dist[v]
\]

then a reachable:

\[
\boxed{\text{negative cycle}}
\]

exists.

---

20. COMPLEXITY

\[
\boxed{O(VE)}
\]

```
Space:

\[
O(V)
\]

So Bellman-Ford is slower than Dijkstra, but handles negative edges.

---

# 21. DIJKSTRA VS BELLMAN-FORD

| Feature | Dijkstra | Bellman-Ford |
|---|---|---|
| Negative edges | No | Yes |
| Negative cycle detection | No | Yes |
| Typical speed | Faster | Slower |
| Heap | Usually | Not required |
| Complexity | \(O(E\log V)\) typical | \(O(VE)\) |

---

# PART E — DAG SHORTEST PATH

# 22. SPECIAL CASE

If graph is a:

\[
\boxed{\text{DAG}}
\]

we can find shortest paths even with negative weights.

Steps:

1. Topological sort.
2. Process nodes in topo order.
3. Relax outgoing edges once.

---

# 23. COMPLEXITY

Topological sort:

\[
O(V+E)
\]

Relax edges:

\[
O(E)
\]

Total:

\[
\boxed{O(V+E)}
\]

Very efficient.

---
```

24. WHY NEGATIVE EDGES ARE FINE?

No cycles exist.

So once predecessors are processed in topological order, no later path can loop back and change them indefinitely.

PART F — 0-1 BFS

25. SPECIAL WEIGHT CASE

If every edge weight is only:

$$
\boxed{0\text{ or }1}
$$

you can use:

$$
\boxed{\text{0-1 BFS}}
$$

with a deque.

---

26. RULE

For edge weight 0:

> push neighbor to front.

For edge weight 1:

> push neighbor to back.

Why?

Weight-0 transitions should be processed earlier.

---

27. TEMPLATE IDEA

```python id="m11a14"
from collections import deque

def zero_one_bfs(n, graph, source):
    dist = [float("inf")] * n
    dist[source] = 0

    dq = deque([source])

    while dq:
        node = dq.popleft()

        for nei, w in graph[node]:
            if dist[node] + w < dist[nei]:
                dist[nei] = dist[node] + w

                if w == 0:
                    dq.appendleft(nei)
                else:
```

```
                    dq.append(nei)

    return dist
```

Time:

\[\boxed{O(V+E)} \]

---

28. SHORTEST PATHMASTER MAP

```text id="m11a15" Unweighted / Equal Weight !’ BFS 0 or 1 weights !’ 0-1 BFS Non-negative weights !’ Dijkstra Negative edges !’ Bellman-Ford DAG weighted !’ Topological shortest path

This is one of the most important placement notes in this subject.

---

# PART G — MINIMUM SPANNING TREE

Now shortest path and MST must be clearly separated.

# 29. WHAT IS MST?

Given a connected undirected weighted graph, find a set of edges that:

- connects all vertices,
- has no cycles,
- has minimum total weight.

Such a tree contains:

\[
\boxed{V-1}
\]

edges.

---

# 30. MST IS NOT SHORTEST PATH TREE

Suppose you need:

> minimum total cost to connect every building.

Use:

\[
\boxed{\text{MST}}
\]

If you need:

> shortest route from building A to building B.

Use shortest path algorithm.

Different objectives.

---
```

```
# 31. SIMPLE DISTINCTION

### Shortest Path

Optimize:

\[
\boxed{\text{path cost from source}}
\]

### MST

Optimize:

\[
\boxed{\text{total cost of chosen edges connecting everyone}}
\]

---

# PART H — PRIM'S ALGORITHM

# 32. MAIN IDEA

Prim grows one tree.

Start with any vertex.

Repeatedly choose minimum-weight edge that connects:

```text id="m11a16"
 c u r r e n t   t r e e  !’   o u t s i d e   v e r t e x
```
```

33. HEAP-BASED PRIM

Use Min Heap of candidate edges.

Store:

```text id="m11a17" (weight, node)

```

```
When smallest candidate gives an unvisited node:

1. add node to MST,
2. add edge weight to total,
3. push its outgoing edges.

---

# 34. TEMPLATE

```python id="m11a18"
import heapq

def prim(n, graph):
    visited = [False] * n
    heap = [(0, 0)]
    total = 0

    while heap:
        weight, node = heapq.heappop(heap)
```
```

```
        if visited[node]:
            continue

        visited[node] = True
        total += weight

        for nei, w in graph[node]:
            if not visited[nei]:
                heapq.heappush(heap, (w, nei))

    return total
```

Assuming connected graph.

---

35. WHY START WITH `(0,0)`?

First node has no connecting edge.

We give artificial cost:

\[0 \]

so it contributes nothing to MST total.

---

36. PRIM COMPLEXITY

Using binary heap:

\[\boxed{O(E\log V)} \]

roughly.

---

37. PRIM VS DIJKSTRA — MAJOR INTERVIEW TRAP

Both use:

- Min Heap
- visited-like processing
- adjacency lists

But the heap key differs.

Prim

Heap priority:

\[\boxed{\text{individual edge weight}} \]

Goal:

> **connect tree cheaply.**

Dijkstra

Heap priority:

\[\boxed{\text{total source-to-node distance}} \]

Goal:

> **shortest path from source.**

Memory:

```text id="m11a19" Prim  !’ cheapest EDGE Dijkstra !’ cheapest PATH

---

# PART I — KRUSKAL'S ALGORITHM

# 38. MAIN IDEA

Kruskal does not grow one tree from a chosen source.

Instead:

1. sort all edges by weight,
2. take smallest edge,
3. add it if it does not create a cycle,
4. continue until \(V-1\) edges chosen.

---

# 39. WHAT DO WE NEED?

We need fast way to ask:

> Are \(u\) and \(v\) already connected?

If yes:

> adding edge creates cycle.

If no:

> safe to join.

This is exactly:

\[
\boxed{\text{DSU / Union-Find}}
\]

---

# PART J — DISJOINT SET UNION

# 40. WHAT IS DSU?

DSU maintains groups of connected elements.

Supports:

### Find

Which component does this element belong to?

### Union
```

```
Merge two components.

---

# 41. SIMPLE EXAMPLE

Initially:

```text id="m11a20"
{0} {1} {2} {3} {4}
```
```

Union:

```text id="m11a21" union(0,1)

```
Now:

```text id="m11a22"
{0,1} {2} {3} {4}
```
```

Then:

```text id="m11a23" union(1,2)

```
Now:

```text id="m11a24"
{0,1,2} {3} {4}
```
```

Now:

```text id="m11a25" find(0) == find(2)

```
True.

They are connected.

---

42. PARENT ARRAY

Initially:

```text id="m11a26"
parent[i] = i
```
```

Example:

```text id="m11a27" parent = [0,1,2,3,4]

```
Each node is its own representative.

---

# 43. FIND

Basic:
```

```python id="m11a28"
def find(x):
    if parent[x] == x:
        return x

    return find(parent[x])
```

But this can become slow if tree becomes tall.

PART K — PATH COMPRESSION

44. IDEA

During `find(x)`, directly attach node to root representative.

```python id="m11a29" def find(x): if parent[x] != x: parent[x] = find(parent[x])

return parent[x]

```
This dramatically flattens structure.

---

45. BEFORE PATH COMPRESSION

```text id="m11a30"
0
|
1
|
2
|
3
```

Find(3) follows:

```text id="m11a31" 3 !' 2 !' 1 !' 0

```
After compression:

```text id="m11a32"
    0
  / | \
 1  2  3
```

Future finds are very fast.

PART L — UNION BY RANK / SIZE

46. PROBLEM

If we attach arbitrary root under another repeatedly, DSU tree can become tall.

Solution:

\[\boxed{\text{attach smaller tree under larger tree}} \]

Using:

- rank,
- or size.

---

47. UNION BY SIZE

```python id="m11a33" def union(a, b): ra = find(a) rb = find(b)

if ra == rb: return False

if size[ra] < size[rb]: ra, rb = rb, ra

parent[rb] = ra size[ra] += size[rb]

return True

```
Return `False` can indicate:

> already connected.

---

48. FULL DSU CLASS

```python id="m11a34"
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])

        return self.parent[x]

    def union(self, a, b):
        ra = self.find(a)
        rb = self.find(b)

        if ra == rb:
            return False

        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra

        self.parent[rb] = ra
        self.size[ra] += self.size[rb]

        return True
```

---

49. DSU COMPLEXITY

With:

- path compression
- union by rank/size

operations are almost constant amortized:

\[\boxed{O(\alpha(n))} \]

where \(\alpha(n)\) is inverse Ackermann function.

For practical input sizes:

> **essentially constant.**

PART M — KRUSKAL WITH DSU

50. TEMPLATE

```python id="m11a35" def kruskal(n, edges): edges.sort(key=lambda x: x[2])

dsu = DSU(n)

total = 0 used = 0

for u, v, w in edges: if dsu.union(u, v): total += w used += 1

if used == n - 1: break

return total

```

```
---

# 51. WHY `union()` TELLS US CYCLE?

If:

\[
find(u)=find(v)
\]

then \(u\) and \(v\) are already connected.

Adding edge:

\[
u-v
\]

would create a cycle.

So skip.

---

# 52. KRUSKAL COMPLEXITY

Sorting edges dominates:

\[
\boxed{O(E\log E)}
\]
```

```
DSU operations are almost constant.

---

# 53. PRIM VS KRUSKAL

| Prim | Kruskal |
|---|---|
| Grow one tree | Grow forest and merge |
| Heap common | Sorting + DSU |
| Works from start vertex | No start needed |
| Chooses crossing edge | Chooses globally smallest cycle-safe edge |
| \(O(E\log V)\) typical | \(O(E\log E)\) |

---

# PART N — REDUNDANT CONNECTION

Very common DSU interview problem.

# 54. PROBLEM

Edges are added to initially disconnected nodes.

One extra edge creates a cycle.

Find that redundant edge.

Example:

```text id="m11a36"
[1,2]
[1,3]
[2,3]
```
```

When processing:

```text id="m11a37" [2,3]

```
2 and 3 are already connected through 1.

So:

\[
\boxed{[2,3]}
\]

is redundant.

---

55. DSU SOLUTION

For each edge:

```text id="m11a38"
if find(u) == find(v):
    redundant
else:
    union(u,v)
```
```

Time:
```

\[\boxed{O(E\alpha(V))} \]

approximately.

PART O — NUMBER OF PROVINCES WITH DSU

56. PROBLEM

Find number of connected components.

Initially:

\[components=n \]

Whenever successful union joins two different groups:

```text id="m11a39" components--

```
At end:

\[
\boxed{components}
\]

This gives another way to solve connected-components problems.

PART P — NETWORK CONNECTIVITY

57. PROBLEM FAMILY

Examples:

> Can all computers be connected?

> How many extra operations are required?

DSU can maintain components while edges are processed.

Useful variables:

```text id="m11a40"
components
extra_edges
```
```

This appears in network-connectivity interview questions.

---

# PART Q — ACCOUNTS MERGE

A famous DSU / Hashing problem.

## 58. PROBLEM IDEA

Accounts may share emails.

If two accounts share an email:
```

> **they belong to same person/group.**

Use:

- HashMap email !' account index
- DSU to merge account indices

Then collect emails by representative.

Pattern:

\[\boxed{\text{Hashing + DSU}} \]

PART R — NUMBER OF ISLANDS II

Dynamic version of Number of Islands.

Land cells are added one by one.

After each addition:

> **return number of islands.**

BFS/DFS from scratch each time can be expensive.

DSU is ideal.

For new land:

```text id="m11a41" islands++

```
For each neighboring land cell:

If union merges two separate components:

```text id="m11a42"
islands--
```
```

This is an important advanced DSU pattern.

---

## PART S — MINIMUM COST TO CONNECT POINTS

## 59. PROBLEM FAMILY

Given points/cities and pairwise connection costs.

Need:

> **connect every point with minimum total cost.**

This is:

\[ \boxed{\text{MST}} \]
```

Possible approaches:

- Prim
- Kruskal

Do not use Dijkstra just because edges have costs.

PART T — MST NEGATIVE WEIGHTS

MST can absolutely contain negative edges.

Example:

```text id="m11a43" A --(-5)-- B

```

```
If it safely helps connect graph, taking it is beneficial.

Negative weights are not a problem for:

- Prim
- Kruskal

They are a problem for Dijkstra.

---

# PART U — MST UNIQUENESS

Useful conceptual interview facts:

If all edge weights are distinct:

\[
\boxed{\text{MST is unique}}
\]

But equal weights do **not** automatically imply multiple MSTs.

---

# PART V — DSU VS BFS/DFS

Both can answer connectivity.

So when should you prefer each?

### BFS/DFS

Good when graph is already built and you want:

- traverse components,
- inspect nodes,
- paths,
- reachability.

### DSU

Great when:

- edges arrive dynamically,
- repeated union/connectivity queries,
- cycle detection while adding edges,
```

```
- Kruskal,
- dynamic components.

Memory:

 >   * * T r a v e r s a l  !’   B F S / D F S * *

 >   * * M e r g e / C o n n e c t i v i t y   u p d a t e s  !’   D S U * *

---

# PART W — SHORTEST PATH VS MST EXAMPLE

Suppose:

```text id="m11a44"
A --1-- B
|       |
10      1
|       |
C --1-- D
```
```

MST may use:

```text id="m11a45" A-B B-D D-C

```
total:

\[
3
\]

But shortest path from A to C inside MST is:

\[
A\to B\to D\to C=3
\]

whereas original direct edge A-C is:

\[
10
\]

Here MST path happens to be better, but in general:

> MST is not built to preserve shortest paths.

Never substitute one for the other.

PART X — INTERVIEW DECISION TREE

When a graph problem comes:

```text id="m11a46"
Need shortest number of edges?
!’   B F S

Need weighted shortest path?
    negative edges?
                y e s  !’   B e l l m a n - F o r d
```

```
                n o   !'  D i j k s t r a

DAG?
!'  t o p o  s h o r t e s t  p a t h

Weights only 0/1?
!'  0 - 1  B F S

Need connect all vertices with minimum total edge weight?
!'  M S T

MST:
    edge sorting natural?
        !'  K r u s k a l  +  D S U

    adjacency graph + grow tree?
        !'  P r i m  +  H e a p

Need repeated connectivity/merging?
!'  D S U
```

PART Y — COMMON INTERVIEW TRAPS

60. TRAP 1 — DIJKSTRA WITH NEGATIVE EDGES

Wrong.

Use Bellman-Ford or exploit DAG structure when applicable.

61. TRAP 2 — BELLMAN-FORD PASSES

Main passes:

$$\boxed{V-1}$$

not:

$$E-1$$

62. TRAP 3 — BFS FOR ARBITRARY WEIGHTS

BFS ignores weight magnitudes.

Use it only when edge costs are equal/unweighted.

63. TRAP 4 — MST = SHORTEST PATH TREE

False.

Objectives differ.

64. TRAP 5 — PRIM HEAP KEY

Prim stores cheapest connecting edge cost.

Dijkstra stores total path distance.

65. TRAP 6 — KRUSKAL WITHOUT CYCLE CHECK

Taking smallest edges blindly can create cycles.

Need:

$$ \boxed{\text{DSU}} $$

or another cycle-detection mechanism.

66. TRAP 7 — DSU WITHOUT PATH COMPRESSION/RANK

Still correct, but may become slow.

Use both optimizations.

67. TRAP 8 — UNION ORIGINAL NODES DIRECTLY

You must union:

$$ \boxed{\text{representatives/roots}} $$

not blindly assign arbitrary parent relationships.

68. TRAP 9 — DISCONNECTED GRAPH MST

A disconnected graph has no single spanning tree.

Instead:

$$ \boxed{\text{Minimum Spanning Forest}} $$

69. TRAP 10 — DIJKSTRA STALE HEAP ENTRIES

If pushing improved distances without decrease-key:

> **old heap entries remain.**

Ignore them when their distance no longer matches current best.

PART Z — INTERVIEW EXPLANATION EXAMPLE

Suppose interviewer asks:

> **Find the minimum cost to connect all cities.**

A strong answer:

> **“This is not a shortest-path problem because we are minimizing the total cost of the selected connections while ensuring all cities are connected. That is a Minimum Spanning Tree problem. I can use Kruskal's algorithm by sorting edges by cost and adding each edge only when its endpoints are currently in different components. A Disjoint Set Union structure with path compression and union by size lets me check and merge components efficiently. Sorting dominates the runtime, giving \(O(E\log E)\).”**

That is a strong placement explanation.

MUST-DO QUESTIONS

Shortest Path

1. Dijkstra implementation
2. Network Delay Time
3. Cheapest weighted path with non-negative weights
4. Bellman-Ford
5. Detect Negative Cycle
6. Shortest Path in DAG
7. 0-1 BFS basics

MST

8. Prim's Algorithm
9. Kruskal's Algorithm
10. Minimum Cost to Connect Points
11. Connecting Cities With Minimum Cost

DSU

12. Implement DSU
13. Redundant Connection
14. Number of Provinces
15. Number of Connected Components
16. Network Connection
17. Accounts Merge
18. Number of Islands II

PLACEMENT PRACTICE SET

Q1 — MCQ

A weighted graph has only non-negative edge weights. Standard SSSP choice:

A. DFS B. Dijkstra C. Kruskal D. Prim

Q2 — MCQ

A graph contains negative edge weights and may contain cycles. Appropriate SSSP algorithm:

A. Dijkstra B. Bellman-Ford C. BFS D. Prim

Q3 — MCQ

All edges have equal cost. Most natural shortest-path algorithm:

A. BFS B. Kruskal C. Heap Sort D. DFS only

Q4 — MCQ

Edges have weights only 0 and 1. Specialized optimal technique:

A. 0-1 BFS B. Selection Sort C. Prim only D. Standard DFS

Q5 — MSQ

Which statements are true?

A. Dijkstra requires non-negative edges.

B. Bellman-Ford can detect reachable negative cycles.

C. DAG shortest paths can handle negative edges.

D. BFS handles arbitrary weighted shortest paths.

Q6 — MCQ

Prim chooses based on:

A. current cheapest source-to-node path

B. cheapest edge connecting current tree to outside

C. highest-weight cycle edge

D. indegree

Q7 — MCQ

Dijkstra chooses based on:

A. minimum total tentative distance from source

B. globally smallest unused edge regardless of source distance

C. number of components

D. node degree

Q8 — MCQ

Kruskal's algorithm commonly uses:

A. Queue

B. DSU / Union-Find

C. Binary Search only

D. Dynamic Programming

Q9 — NAT

A spanning tree has:

\[V=12 \]

vertices.

How many edges?

Q10 — MCQ

Which DSU optimization flattens trees during `find()`?

A. Heapify

B. Path Compression

C. Topological Sort

D. Relaxation

Q11 — MCQ

Union by size means:

A. attach larger component under smaller

B. attach smaller component under larger

C. sort all nodes

D. union only equal-sized sets

Q12 — MSQ

Which are good DSU problems?

A. Redundant Connection

B. Kruskal MST

C. Dynamic Connectivity

D. Longest Common Subsequence

Q13 — MCQ

If `find(u) == find(v)` while processing a new undirected edge in Kruskal:

A. edge safely joins two components

B. edge would create a cycle

C. graph becomes directed

D. shortest path is found

Q14 — MCQ

MST minimizes:

A. shortest source-to-destination path

B. total weight needed to connect all vertices

C. maximum vertex degree

D. number of vertices

Q15 — MSQ

Which algorithms can handle negative edge weights?

A. Bellman-Ford

B. DAG shortest-path algorithm

C. Dijkstra in general

D. Prim for MST

PRACTICE SOLUTIONS

Q1

$$\boxed{B}$$

Q2

$$\boxed{B}$$

Q3

$$\boxed{A}$$

Q4

$$ \boxed{A} $$

Q5

A true.

B true.

C true.

D false.

$$ \boxed{A,B,C} $$

Q6

$$ \boxed{B} $$

Q7

$$ \boxed{A} $$

Q8

$$ \boxed{B} $$

Q9

$$ V-1 $$

$$ 12-1 = \boxed{11} $$

Q10

$$ \boxed{B} $$

Q11

$$ \boxed{B} $$

Q12

A, B, C.

$$ \boxed{A,B,C} $$

Q13

Already connected means adding another edge closes a cycle.

\[\boxed{B} \]

Q14

\[\boxed{B} \]

Q15

Bellman-Ford handles negative shortest-path edges.

DAG shortest path handles negative edges because no cycles.

Dijkstra generally does not.

Prim is an MST algorithm and negative edge weights are allowed.

\[\boxed{A,B,D} \]

Ø=ÑØTES TO WRITE

```text id="m11a47" SHORTEST PATH

Unweighted / equal weight !’ BFS

0/1 weights !’ 0-1 BFS

Non-negative weights !’ Dijkstra

Negative edges !’ Bellman-Ford

DAG !’ Topological relaxation

DIJKSTRA: Min Heap priority = total source distance O(E log V) typical

BELLMAN-FORD: V-1 passes extra pass !’ negative cycle O(VE)

MST: minimum total cost to connect all V-1 edges

PRIM: grow one tree cheapest crossing EDGE Heap

KRUSKAL: sort edges take smallest cycle-safe edge DSU

DSU: find !’ representative union !’ merge components

Optimizations: Path Compression Union by Size/Rank

Kruskal: find(u) == find(v) !’ adding edge creates cycle

---

# &¡ 30-SECOND REVISION

1. BFS !’ unweighted shortest path.
2. 0-1 BFS !’ weights only 0/1.
3. Dijkstra !’ non-negative weights.
```

```
 4 .  B e l l m a n - F o r d  !’  n e g a t i v e  e d g e s .
5. Bellman-Ford main rounds = \(V-1\).
 6 .  E x t r a  i m p r o v i n g  r o u n d  !’  n e g a t i v e  c y c l e .
 7 .  D A G  s h o r t e s t  p a t h  !’  t o p o  o r d e r .
8. Dijkstra uses Min Heap.
9. Relaxation = improve `dist[v]` through `u`.
10. MST minimizes total connection cost.
11. MST has \(V-1\) edges.
12. Prim grows one tree.
 1 3 .  P r i m  !’  c h e a p e s t  c r o s s i n g  e d g e .
 1 4 .  K r u s k a l  !’  g l o b a l l y  s m a l l e s t  c y c l e - s a f e  e d g e .
15. Kruskal uses DSU.
16. DSU supports `find` and `union`.
17. Path compression flattens DSU.
18. Union by size keeps tree shallow.
19. MST supports negative edge weights.
 2 0 .  P r i m  "`  D i j k s t r a :  e d g e  c o s t  v s  p a t h  c o s t .

 #  '  P l a c e m e n t  M o d u l e  1 1  C o m p l e t e

## Next Placement Module:

# **MODULE 12 — DYNAMIC PROGRAMMING FOR PLACEMENTS: 1D DP + TAKE/NOT-TAKE**

We’ll cover:

- how to recognize DP,
 -  r e c u r s i o n  !’  m e m o i z a t i o n  !’  t a b u l a t i o n ,
- Fibonacci,
- Climbing Stairs,
- Min Cost Climbing Stairs,
- House Robber,
- Frog Jump style,
- Coin Change,
- 0/1 Knapsack,
- Subset Sum,
- Partition Equal Subset Sum,
- target-style DP,
- state/transition/base-case design,
- and the most important DP interview mistakes.

**Type `NEXT` to continue.**
```

USER

NEXT

ASSISTANT

PLACEMENT MODULE 12 — DYNAMIC PROGRAMMING FOR PLACEMENTS: 1D DP + TAKE / NOT-TAKE

Placement Priority: & & & & &

Dynamic Programming is one of the most feared placement topics, but the core idea is much smaller than it looks.

The main skill is learning to identify:

\[\boxed{\text{State !’ Transition !’ Base Case !’ Order}} \]

and to recognize when recursion is recomputing the same work.

---

1. WHAT IS DYNAMIC PROGRAMMING?

Dynamic Programming is useful when:

1. The problem can be divided into smaller subproblems.
2. The same subproblems occur repeatedly.
3. Their answers can be stored and reused.

The two classic properties are:

$$\boxed{\text{Overlapping Subproblems}}$$

and

$$\boxed{\text{Optimal Substructure}}$$

---

2. THE BIGGEST DP SIGNAL

Suppose recursion does this:

```text
solve(5)
% % %    s o l v e ( 4 )
%       % % %    s o l v e ( 3 )
%       % % %    s o l v e ( 2 )
% % %    s o l v e ( 3 )
```

Notice:

```text
solve(3)
```

is repeated.

Instead of solving it again:

$$\boxed{\text{Store the answer}}$$

That's DP.

---

3. DP IS NOT JUST A TABLE

A common misconception:

> **"If I create a matrix, it is DP."**

No.

DP is about:

$$\boxed{\text{Remembering states so repeated work is avoided}}$$

The table/list is only the storage mechanism.

4. FOUR STEPS FOR EVERY DP PROBLEM

Use:

STBO

S — State

What does `dp[i]` or `dp[i][j]` mean?

T — Transition

How do smaller states produce the current state?

B — Base Case

What answers do we already know?

O — Order

In what order should states be calculated?

Memory:

\[\boxed{\text{State !' Transition !' Base !' Order}} \]

PART A — RECURSION !' MEMOIZATION !' TABULATION

5. THREE LEVELS

Take Fibonacci:

\[F(n)=F(n-1)+F(n-2) \]

We can solve it three ways.

---

6. LEVEL 1 — NAIVE RECURSION

```python
def fib(n):
    if n <= 1:
        return n

    return fib(n - 1) + fib(n - 2)
```

Time:

\[O(2^n) \]

roughly.

Reason:

> **same states are recalculated repeatedly.**

7. LEVEL 2 — MEMOIZATION

Memoization = Top-down DP.

```python
def fib(n):
    memo = {}

    def solve(x):
        if x <= 1:
            return x

        if x in memo:
            return memo[x]

        memo[x] = solve(x - 1) + solve(x - 2)
        return memo[x]

    return solve(n)
```

Each state computed once.

States:

```text
0,1,2,...,n
```

Therefore:

$$ \boxed{O(n)} $$

Time.

Space:

$$ O(n) $$

memo + recursion stack.

8. LEVEL 3 — TABULATION

Tabulation = Bottom-up DP.

```python
def fib(n):
    if n <= 1:
        return n

    dp = [0] * (n + 1)

    dp[0] = 0
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

Time:

\[O(n) \]

Space:

\[O(n) \]

---

9. SPACE OPTIMIZATION

We only need:

```text
dp[i-1]
dp[i-2]
```

So no need to store whole array.

```python
def fib(n):
    if n <= 1:
        return n

    prev2 = 0
    prev1 = 1

    for _ in range(2, n + 1):
        current = prev1 + prev2
        prev2 = prev1
        prev1 = current

    return prev1
```

Time:

\[O(n) \]

Space:

\[\boxed{O(1)} \]

---

10. IMPORTANT INTERVIEW PROGRESSION

When interviewer gives DP problem, a strong process is:

```text
Recursive solution
!'  identify  repeated  states
!'  Memoization
!'  Tabulation
!'  Space  optimization  if  possible
```

You do not always need to code all four, but understand the progression.

PART B — CLIMBING STAIRS

One of the easiest DP patterns.

11. PROBLEM

You can climb:

```text
1 step
or
2 steps
```

How many ways to reach stair n?

Example:

$$ n=3 $$

Ways:

```text
1+1+1
1+2
2+1
```

Answer:

$$ \boxed{3} $$

12. THINK FROM THE LAST STEP

To reach stair n, your last move came from:

$$ n-1 $$

or:

$$ n-2 $$

Therefore:

$$ \boxed{ dp[n]=dp[n-1]+dp[n-2] } $$

13. STATE

$$ dp[i] $$

means:

> **number of ways to reach stair i.**

14. BASE CASE

Common setup:

$$ dp[0]=1 $$

$$ dp[1]=1 $$

Why $dp[0]=1$?

There is one way to remain at ground: take no steps.

Then:

$$ dp[2]=2 $$

---

15. CODE

```python
def climb_stairs(n):
    if n <= 1:
        return 1

    prev2 = 1
    prev1 = 1

    for _ in range(2, n + 1):
        current = prev1 + prev2
        prev2 = prev1
        prev1 = current

    return prev1
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(1)} $$

---

16. INTERVIEW SIGNAL

If question asks:

> **number of ways to reach position n using allowed jumps**

think:

$$ \boxed{\text{1D DP}} $$

Transition usually sums possible previous states.

PART C — FROG JUMP / MINIMUM ENERGY

17. PROBLEM FAMILY

A frog stands at index 0.

It can jump:

```TEXT
i  !'   i + 1
i  !'   i + 2
```

Cost may depend on height difference.

Example cost:

\[

```
height[i]-height[j]
```

\]

Find minimum total cost.

---

18. STATE

\[dp[i] \]

means:

> **minimum cost to reach index \(i\).**

---

19. TRANSITION

To reach \(i\):

From \(i-1\)

\[dp[i-1]+|h[i]-h[i-1]| \]

From \(i-2\)

\[dp[i-2]+|h[i]-h[i-2]| \]

Take minimum:

\[\boxed{ dp[i] = \min(dp[i-1]+cost_1, dp[i-2]+cost_2) } \]

---

20. DP SIGNAL

Compare Climbing Stairs:

> **count ways !' add.**

Frog Jump:

> **minimum cost !' take min.**

This distinction appears everywhere.

PART D — MIN COST CLIMBING STAIRS

21. PROBLEM IDEA

Each stair has a cost.

You can move 1 or 2 steps.

Find minimum cost to reach the top.

Example:

```text
cost = [10,15,20]
```

You can start at index 0 or 1.

Best:

```text
start at 1
pay 15
jump to top
```

Answer:

$$ \boxed{15} $$

22. STATE IDEA

One possible state:

$$ dp[i] $$

= minimum cost to reach stair i.

Then transition depends on two previous stairs.

Another equivalent interpretation:

> **minimum cost starting from each stair.**

Both are valid if consistent.

23. COMMON TRANSITION

For cost paid at current stair:

$$ dp[i] = cost[i] + \min(dp[i-1],dp[i-2]) $$

Final answer:

$$ \min(dp[n-1],dp[n-2]) $$

because top is beyond final stair.

PART E — HOUSE ROBBER

Extremely important 1D DP problem.

24. PROBLEM

Houses contain money:

```text
[2,7,9,3,1]
```

Cannot rob adjacent houses.

Find maximum amount.

Best:

```text
2 + 9 + 1 = 12
```

Answer:

$$ \boxed{12} $$

25. THINK AT HOUSE i

Two choices:

Skip house i

Then:

$$ dp[i-1] $$

Rob house i

Then previous house cannot be robbed:

$$ nums[i]+dp[i-2] $$

Take maximum:

$$ \boxed{ dp[i]=\max(dp[i-1],nums[i]+dp[i-2]) } $$

26. THIS IS A TAKE / NOT-TAKE PATTERN

At each index:

```text
Take current
or
Skip current
```

This becomes one of the most important DP families.

Memory:

\[\boxed{\text{Take / Not Take}} \]

---

27. CODE

```python
def house_robber(nums):
    if not nums:
        return 0

    prev2 = 0
    prev1 = 0

    for money in nums:
        take = money + prev2
        skip = prev1

        current = max(take, skip)

        prev2 = prev1
        prev1 = current

    return prev1
```

Time:

\[O(n) \]

Space:

\[O(1) \]

---

28. INTERVIEW EXPLANATION

Say:

> **"At each house, I either rob it or skip it. If I rob it, I can only combine it with the best answer up to two houses before. If I skip it, the best answer remains the previous state's answer. Therefore `dp[i] = max(dp[i-1], nums[i] + dp[i-2])`."**

That explains DP very clearly.

PART F — HOUSE ROBBER II

29. CIRCULAR VARIANT

Houses arranged in circle.

First and last are adjacent.

So cannot choose both.

Solve two cases:

Case 1

Exclude first house.

Solve:

```
TEXT
1 ... n-1
```

Case 2

Exclude last house.

Solve:

```
TEXT
0 ... n-2
```

Answer:

\[\boxed{\max(case1,case2)} \]

---

30. PATTERN

Circular constraint often becomes:

\[\boxed{\text{split into linear cases}} \]

This appears elsewhere too.

PART G — DELETE AND EARN CONNECTION

A useful medium pattern.

Suppose selecting value \(x\) prevents selecting:

\[x-1,\ x+1 \]

First aggregate total contribution per value.

Example:

```
TEXT
[2,2,3,3,3,4]
```

Totals:

```
TEXT
 2  !'   4
 3  !'   9
 4  !'   4
```

Now choosing value 3 prevents 2 and 4.

This becomes:

\[\boxed{\text{House Robber on values}} \]

Important lesson:

> **Some problems can be transformed into a known DP pattern.**

PART H — COIN CHANGE: MINIMUM COINS

One of the most important DP questions.

31. PROBLEM

Coins:

```text
[1,2,5]
```

Amount:

$$ 11 $$

Minimum coins:

```text
5 + 5 + 1
```

Answer:

$$ \boxed{3} $$

---

32. STATE

$$ dp[x] $$

means:

> **minimum number of coins needed to form amount x.**

---

33. BASE CASE

$$ dp[0]=0 $$

To form amount 0:

> **need zero coins.**

---

34. TRANSITION

For every coin c:

If:

$$ c\le x $$

then:

$$ dp[x] = \min(dp[x],1+dp[x-c]) $$

---

35. CODE

```python
def coin_change(coins, amount):
    inf = amount + 1
    dp = [inf] * (amount + 1)

    dp[0] = 0

    for x in range(1, amount + 1):
        for coin in coins:
            if coin <= x:
                dp[x] = min(dp[x], 1 + dp[x - coin])

    return dp[amount] if dp[amount] != inf else -1
```

Time:

$$ \boxed{O(amount\times k)} $$

where k = number of coin types.

Space:

$$ O(amount) $$

---

36. WHY `amount + 1` AS INFINITY?

Maximum number of coins, if coin 1 exists, cannot exceed amount.

So:

```text
amount + 1
```

is safely impossible/large.

You can also use infinity.

PART I — COIN CHANGE: NUMBER OF WAYS

Very important distinction.

Now question may ask:

> **How many combinations form amount?**

This is not minimum.

Now transition uses:

$$ \boxed{+} $$

instead of `min`.

---

37. CORE DIFFERENCE

Minimum Coins

$$ dp[x] = \min(...) $$

Number of Ways

$$ dp[x] = dp[x]+... $$

Read the required output carefully.

PART J — 0/1 KNAPSACK

One of the fundamental DP patterns.

38. PROBLEM

Each item has:

- weight
- value

Capacity:

$$ W $$

Each item may be used:

$$ \boxed{\text{at most once}} $$

Find maximum value.

---

39. EXAMPLE

Items:

Item	Weight	Value
A	1	15
B	3	20
C	4	30

Capacity:

$$ W=4 $$

Options:

```text
A + B:
weight = 4
value = 35
```

or C:

```text
value = 30
```

Answer:

$$ \boxed{35} $$

40. STATE

Classic:

$$ dp[i][w] $$

means:

> **maximum value using first i items with capacity w.**

41. CHOICES

For item i:

Do not take

$$ dp[i-1][w] $$

Take

Only if:

$$ weight_i\le w $$

Then:

$$ value_i+dp[i-1][w-weight_i] $$

Thus:

$$ \boxed{ dp[i][w] = \max(dp[i-1][w], value_i+dp[i-1][w-weight_i]) } $$

42. WHY `i-1` AFTER TAKING?

Because each item can only be used once.

That's why:

$$ \boxed{\text{0/1}} $$

Knapsack.

43. COMPLEXITY

States:

$$ nW $$

Time:

$$ \boxed{O(nW)} $$

Space:

$$ O(nW) $$

Can optimize to:

$$ O(W) $$

44. PSEUDO-POLYNOMIAL TRAP

$O(nW)$ is not polynomial in the input bit-length of W.

So it's called:

$$ \boxed{\text{pseudo-polynomial}} $$

Useful for both GATE and interviews.

PART K — 1D KNAPSACK OPTIMIZATION

45. KEY QUESTION

If using:

```text
dp[w]
```

for 0/1 Knapsack, should capacity iterate:

forward?

or

backward?

Correct:

$$ \boxed{\text{BACKWARD}} $$

46. WHY BACKWARD?

Suppose item has:

```text
weight = 2
value = 10
```

If you loop forward:

```text
w = 2,3,4,...
```

then after updating:

\[dp[2] \]

later `dp[4]` might use the newly updated `dp[2]`.

That effectively uses same item twice.

Not allowed in 0/1 Knapsack.

So iterate:

```python
for w in range(W, weight - 1, -1):
```

---

47. CODE

```python
def knapsack(weights, values, W):
    dp = [0] * (W + 1)

    for i in range(len(weights)):
        wt = weights[i]
        val = values[i]

        for w in range(W, wt - 1, -1):
            dp[w] = max(dp[w], val + dp[w - wt])

    return dp[W]
```

---

48. MAJOR MEMORY RULE

```text
 0 / 1   K n a p s a c k      !'   c a p a c i t y   B A C K W A R D
 U n b o u n d e d            !'   c a p a c i t y   F O R W A R D
```

This is extremely important.

PART L — UNBOUNDED KNAPSACK

49. DIFFERENCE

In unbounded Knapsack:

> **each item can be used unlimited times.**

Therefore after taking an item, same item may still be used again.

In 1D DP:

$$ \boxed{\text{iterate capacity forward}} $$

Example:

```python
for item:
    for w from weight to W:
```

---

50. COMMON EXAMPLES

Unbounded-type problems:

- Coin Change
- Rod Cutting
- Unbounded Knapsack

0/1-type:

- each object selected once
- subset selection
- partition problems

PART M — SUBSET SUM

Another foundational DP pattern.

51. PROBLEM

Given:

```text
[3,34,4,12,5,2]
```

Target:

$$ 9 $$

Does some subset sum to 9?

Yes:

```text
4 + 5
```

Return:

$$ \boxed{True} $$

52. STATE

$$ dp[s] $$

means:

> **Is it possible to form sum s?**

Base:

$$ dp[0]=True $$

because empty subset forms zero.

53. TRANSITION

For each number x:

```TEXT
if dp[s-x] was True
then dp[s] becomes True
```

Because number can only be used once:

$$ \boxed{\text{iterate sums backward}} $$

54. CODE

```PYTHON
def subset_sum(nums, target):
    dp = [False] * (target + 1)
    dp[0] = True

    for x in nums:
        for s in range(target, x - 1, -1):
            dp[s] = dp[s] or dp[s - x]

    return dp[target]
```

Time:

$$ \boxed{O(n\times target)} $$

Space:

$$ O(target) $$

55. SUBSET SUM = 0/1 KNAPSACK FAMILY

Each number:

```text
Take
or
Skip
```

and can be used once.

So:

$$ \boxed{\text{Subset Sum is a 0/1 DP pattern}} $$

PART N — PARTITION EQUAL SUBSET SUM

Very common placement problem.

56. PROBLEM

Can array be divided into two subsets with equal sum?

Example:

```text
[1,5,11,5]
```

Total:

$$ 22 $$

Need each side:

$$ 11 $$

Question becomes:

> **Does any subset sum to 11?**

That's:

$$ \boxed{\text{Subset Sum}} $$

---

57. FIRST CHECK

If total sum is odd:

$$ \boxed{\text{Impossible}} $$

because equal halves would require fractional sum.

If even:

$$ target=\frac{sum}{2} $$

Run subset-sum DP.

PART O — TARGET SUM

Another common transformation problem.

58. PROBLEM

Given numbers:

```text
[1,1,1,1,1]
```

Assign `+` or `-` signs to reach:

\[3 \]

Number of ways:

\[\boxed{5} \]

---

59. DP VIEW

Each number has two choices:

```text
+x
-x
```

State may be:

```text
(index, current_sum)
```

Memoization works naturally.

This also has a subset-sum transformation under certain assumptions, but for placements start with state reasoning first.

PART P — COUNTING VS BOOLEAN VS MIN/MAX DP

60. VERY IMPORTANT

The same structure can ask different outputs.

Boolean DP

> **Is it possible?**

Use:

```text
True / False
```

Examples:

- Subset Sum
- Partition Equal Subset Sum

Count DP

> **How many ways?**

Use:

\[+ \]

Examples:

- Climbing Stairs
- Coin Change combinations

Min DP

> **Minimum cost / minimum items?**

Use:

\[\min \]

Examples:

- Coin Change minimum
- Min Cost Climbing Stairs

Max DP

> **Maximum value?**

Use:

\[\max \]

Examples:

- House Robber
- 0/1 Knapsack

---

61. OUTPUT WORD !’ OPERATION

Memorize:

```text
Possible?              !'  Boolean
How many ways?         !'  Sum
Minimum?               !'  min
Maximum?               !'  max
```

This helps derive transitions.

PART Q — DP STATE DESIGN

62. MOST IMPORTANT QUESTION

Ask:

> **What information uniquely determines the remaining subproblem?**

Examples:

Fibonacci

```text
n
```

House Robber

```text
index
```

Knapsack

```text
index + remaining capacity
```

LCS

```text
i + j
```

Grid

```text
row + col
```

That's your:

\[\boxed{\text{DP state}} \]

---

63. DON'T STORE UNNECESSARY INFORMATION

Bad state:

```text
(index, current array, all previous choices, current answer...)
```

Good DP state should be minimal.

Smaller state:

> **fewer possible states !' faster algorithm.**

PART R — MEMOIZATION TEMPLATE

64. GENERIC

```python
def solve(state):
    if base_case:
        return base_answer

    if state in memo:
        return memo[state]

    answer = combine(
        solve(next_state1),
        solve(next_state2)
    )

    memo[state] = answer
    return answer
```

---

65. TAKE / SKIP TEMPLATE

For index \(i\):

```python
def solve(i):
    if i == n:
        return ...

    skip = solve(i + 1)

    take = value[i] + solve(i + 2)

    return max(skip, take)
```

House Robber is exactly this family.

PART S — TABULATION ORDER

66. RULE

If:

\[dp[i] \]

depends on:

\[dp[i-1] \]

then compute left to right.

If:

\[dp[i][j] \]

depends on previous row:

> **previous row must already exist.**

Order must respect dependencies.

---

67. WHY ORDER MATTERS IN 1D KNAPSACK

0/1:

```text
backward
```

because current item should not feed itself.

Unbounded:

```text
forward
```

because reusing current item is allowed.

This is not just syntax—it encodes problem rules.

PART T — COMMON DP INTERVIEW TRAPS

68. TRAP 1 — GREEDY INSTEAD OF DP

0/1 Knapsack:

> **choosing highest value/weight ratio is not always optimal.**

Fractional Knapsack:

> **ratio Greedy works.**

Do not confuse.

69. TRAP 2 — WRONG BASE CASE

If base states are wrong, entire DP is wrong.

Always derive them logically.

Example subset sum:

$$ dp[0]=True $$

not False.

70. TRAP 3 — FORWARD LOOP IN 0/1 KNAPSACK

Can reuse same item multiple times accidentally.

Use:

$$ \boxed{\text{backward capacity}} $$

71. TRAP 4 — TREATING DP AS MAGIC

Always be able to explain:

```text
What does dp[i] mean?
```

If you cannot define state in one sentence, you probably do not fully understand the solution.

72. TRAP 5 — IGNORING SPACE COMPLEXITY

Memoization may use:

- memo
- recursion stack

Tabulation may avoid recursion stack.

Interviewers may ask to optimize memory.

73. TRAP 6 — INFINITE VALUE OVERFLOW

For min DP, writing:

```text
1 + INF
```

may overflow in fixed-width languages if INF is maximum integer.

Use a safe large number and ensure previous state is reachable.

74. TRAP 7 — SAME OUTPUT, DIFFERENT ORDER

Coin Change can ask:

> **combinations**

or:

> **permutations/order matters.**

Loop ordering may change the meaning.

Read the question carefully.

PART U — DP RECOGNITION MAP

75. IF YOU SEE...

"How many ways?"

Think:

$$\boxed{\text{Counting DP}}$$

"Minimum cost"

Think:

$$\boxed{\text{Min DP}}$$

"Maximum profit/value"

Think:

$$\boxed{\text{Max DP}}$$

"Can we form...?"

Think:

$$\boxed{\text{Boolean DP}}$$

"Take or skip each item"

Think:

$$\boxed{\text{0/1 DP}}$$

"Items reusable"

Think:

$$\boxed{\text{Unbounded DP}}$$

PART V — DP VS GREEDY VS BACKTRACKING

76. BACKTRACKING

Need to generate/explore possibilities.

Example:

```text
all subsets
all permutations
```

---

77. DP

Need:

> **best/count/feasibility answer**

and states repeat.

Examples:

- Knapsack
- Coin Change
- House Robber

---

78. GREEDY

Make a locally optimal choice that can be proven safe.

Example:

- Activity Selection
- Fractional Knapsack

---

79. DECISION SHORTCUT

```text
Need ALL valid answers?
!’   B a c k t r a c k i n g

Need optimal/count and states repeat?
!’   D P

Safe local choice exists?
!’   G r e e d y
```

PART W — INTERVIEW EXPLANATION EXAMPLE

Suppose interviewer asks House Robber.

A strong explanation:

> **"At every house I have two choices. If I skip the current house, the best answer remains the best value up to the previous house. If I rob it, I cannot rob the previous house, so I add its money to the best value from two positions earlier. This gives the recurrence `dp[i] = max(dp[i-1], nums[i] + dp[i-2])`. Since each state depends only on the previous two states, I can optimize the DP array to two variables, giving \(O(n)\) time and \(O(1)\) extra space."**

PART X — MUST-DO DP QUESTIONS

DP Foundations

1. Fibonacci
2. Climbing Stairs
3. Min Cost Climbing Stairs
4. Frog Jump
5. House Robber
6. House Robber II

Take / Not-Take

7. 0/1 Knapsack
8. Subset Sum
9. Partition Equal Subset Sum
10. Target Sum
11. Count Subsets with Target Sum

Coin / Unbounded

12. Coin Change Minimum Coins
13. Coin Change Number of Ways
14. Unbounded Knapsack
15. Rod Cutting

Important Follow-Ups

16. Delete and Earn
17. Perfect Squares
18. Decode Ways
19. Combination Sum IV
20. Maximum Product style state variants

PLACEMENT PRACTICE SET

Q1 — MCQ

The strongest signal for DP is:

A. Every subproblem is independent B. Same subproblems repeat C. Input is sorted D. Graph is complete

Q2 — MCQ

Memoization is usually:

A. Bottom-up B. Top-down C. Greedy D. BFS

Q3 — MCQ

Tabulation is usually:

A. Bottom-up B. Top-down recursion C. Backtracking only D. Heap-based

Q4 — NAT

How many ways are there to climb 4 stairs using jumps of 1 or 2?

Q5 — NAT

For:

```text
[2,7,9,3,1]
```

what is maximum House Robber amount?

Q6 — MCQ

House Robber transition is:

A.

\[dp[i]=dp[i-1]+dp[i-2] \]

B.

\[dp[i]=\max(dp[i-1],nums[i]+dp[i-2]) \]

C.

\[dp[i]=\min(dp[i-1],dp[i-2]) \]

D.

\[dp[i]=nums[i] \]

Q7 — MSQ

Which are classic DP problems?

A. 0/1 Knapsack

B. Coin Change

C. Subset Sum

D. Fractional Knapsack

Q8 — MCQ

For 0/1 Knapsack space optimization, capacity is generally iterated:

A. forward

B. backward

C. randomly

D. only once total

Q9 — MCQ

For unbounded Knapsack, capacity is commonly iterated:

A. backward

B. forward

C. using BFS

D. using DFS only

Q10 — MCQ

Subset Sum state usually stores:

A. minimum vertex degree

B. whether a sum is achievable

C. heap priority

D. graph distance

Q11 — NAT

Array:

```text
[1,5,11,5]
```

For Equal Partition, what subset target should be checked?

Q12 — MSQ

Which statements are true?

A. `dp[0] = True` is natural for Subset Sum.

B. $O(nW)$ 0/1 Knapsack is pseudo-polynomial.

C. Forward capacity iteration in 0/1 Knapsack may accidentally reuse an item.

D. DP always requires a 2D table.

Q13 — MCQ

If a problem asks "minimum number of coins," transition usually combines candidates using:

A. max

B. min

C. multiplication

D. DFS color

Q14 — MCQ

If a problem asks "number of ways," DP transitions often combine solutions using:

A. addition

B. minimum only

C. maximum only

D. division

Q15 — MSQ

Which can often be space-optimized from $O(n)$ to $O(1)$?

A. Fibonacci

B. Climbing Stairs

C. House Robber

D. Full 0/1 Knapsack capacity dimension always to $O(1)$

PRACTICE SOLUTIONS

Q1

Repeated states are the major DP signal.

$$\boxed{B}$$

Q2

$$\boxed{B}$$

Q3

$$\boxed{A}$$

Q4

Ways:

```TEXT
1+1+1+1
1+1+2
1+2+1
2+1+1
2+2
```

Therefore:

$$\boxed{5}$$

Q5

Best:

$$2+9+1 = \boxed{12}$$

Q6

$$\boxed{B}$$

Q7

0/1 Knapsack, Coin Change and Subset Sum are DP classics.

Fractional Knapsack is Greedy.

$$\boxed{A,B,C}$$

Q8

0/1 means item only once.

$$\boxed{B}$$

Q9

Reuse allowed:

$$\boxed{B}$$

Q10

$$ \boxed{B} $$

Q11

Total:

$$ 1+5+11+5=22 $$

Target:

$$ 22/2 = \boxed{11} $$

Q12

A true.

B true.

C true.

D false.

$$ \boxed{A,B,C} $$

Q13

Minimum:

$$ \boxed{B} $$

Q14

Number of ways:

$$ \boxed{A} $$

Q15

Fibonacci, Climbing Stairs and House Robber depend only on a few previous states.

$$ \boxed{A,B,C} $$

Ø=ÜÓTES TO WRITE

```text
DP = repeated states + store answers

STBO:
State
Transition
```

```
Base Case
Order

Memoization:
Top-down

Tabulation:
Bottom-up

Fibonacci:
dp[i] = dp[i-1] + dp[i-2]

Climbing Stairs:
 w a y s  !’   A D D

House Robber:
dp[i] = max(
    dp[i-1],
    nums[i] + dp[i-2]
)

Coin Change minimum:
dp[x] = min(
    dp[x],
    1 + dp[x-coin]
)

0/1 Knapsack:
Take / Skip
Each item once

1D 0/1:
capacity BACKWARD

Unbounded:
capacity FORWARD

Subset Sum:
dp[sum] = possible?
dp[0] = True

Equal Partition:
 i f   t o t a l   o d d  !’   f a l s e
else target = total/2
```

DP MASTER MEMORY TABLE

Question asks	DP operation
Possible?	Boolean
Number of ways?	Add
Minimum?	Min
Maximum?	Max

And:

```text
Item  once        !'  0/1 !'  backward
Item  reusable !'  Unbounded !'  forward
```

&β30-SECOND DP REVISION

1. DP stores repeated-state answers.
2. Define the state first.
3. State must have a clear meaning.
4. Derive transition from choices.
5. Define base cases carefully.
6. Memoization = top-down.
7. Tabulation = bottom-up.
8. Fibonacci naive recursion is exponential.
9. Climbing Stairs = ways from previous states.
10. House Robber = take vs skip.
11. Coin Change minimum !' min.
12. Count ways !' addition.
13. 0/1 Knapsack = each item once.
14. 0/1 capacity loop !' backward.
15. Unbounded !' forward.
16. Subset Sum = Boolean 0/1 DP.
17. Equal Partition reduces to target sum = total/2.
18. \(O(nW)\) Knapsack is pseudo-polynomial.
19. Optimize space when only previous states matter.
20. If you cannot explain what `dp[i]` means, do not start coding yet.

' Placement Module 12 Complete

Next Placement Module:

MODULE 13 — 2D DP: LCS, EDIT DISTANCE, GRID DP, LIS + INTERVIEW PATTERNS

Next we'll cover:

- 2D DP state design,
- Longest Common Subsequence,
- Longest Common Substring distinction,
- Edit Distance,
- Minimum Insertions/Deletions,
- Grid Unique Paths,
- Minimum Path Sum,
- obstacle grids,

- Longest Increasing Subsequence,
- LIS $O(n^2)$ and $O(n\log n)$,
- Matrix Chain style interval DP basics,
- and how to identify **1D vs 2D vs interval DP**.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 13 — 2D DP: LCS, EDIT DISTANCE, GRID DP, LIS + INTERVIEW PATTERNS

Placement Priority: & & & & &

This module moves from simple 1D DP to problems where the state usually depends on **two dimensions**, such as:

$$\boxed{(i,j)}$$

Typical signals:

- two strings
- two sequences
- row + column
- interval boundaries
- compare prefixes
- move through a grid

The main goal is still the same:

$$\boxed{\text{State !' Transition !' Base Case !' Order}}$$

---

1. WHEN DO WE NEED 2D DP?

Suppose one variable is not enough to uniquely describe the subproblem.

Example:

> **Find LCS of two strings.**

We need to know:

- how much of string 1 we are considering,
- how much of string 2 we are considering.

Therefore state:

$$\boxed{dp[i][j]}$$

---

2. COMMON 2D DP STATES

Two strings

$$ dp[i][j] $$

might represent answer using:

```text
first i characters of X
first j characters of Y
```

Grid

$$ dp[r][c] $$

might represent:

> **best/number of ways to reach cell (r,c).**

Interval

$$ dp[i][j] $$

might mean:

> **best answer for subarray/string interval from i to j.**

PART A — LONGEST COMMON SUBSEQUENCE

3. WHAT IS A SUBSEQUENCE?

Characters remain in the same relative order, but need not be contiguous.

Example:

```text
X = ABCDE
```

Valid subsequences:

```text
ACE
ABD
AE
BCD
```

---

4. LCS PROBLEM

Given:

```text
X = "abcde"
Y = "ace"
```

Longest Common Subsequence:

```text
"ace"
```

Length:

$$ \boxed{3} $$

---

5. STATE

Define:

$$ dp[i][j] $$

as:

> **length of LCS of first i characters of X and first j characters of Y.**

This definition is extremely important.

---

6. BASE CASE

If either string contributes zero characters:

$$ dp[0][j]=0 $$

and:

$$ dp[i][0]=0 $$

because LCS with empty string has length 0.

---

7. CASE 1 — CHARACTERS MATCH

If:

$$ X[i-1]=Y[j-1] $$

then this character can extend an LCS of the previous prefixes.

Therefore:

$$ \boxed{ dp[i][j] = 1+dp[i-1][j-1] } $$

Memory:

> **Match !' Diagonal + 1**

---

8. CASE 2 — CHARACTERS DO NOT MATCH

If:

$$ X[i-1]\neq Y[j-1] $$

we must skip one side.

Possibilities:

$$ dp[i-1][j] $$

or:

$$ dp[i][j-1] $$

Take maximum:

$$ \boxed{ dp[i][j] = \max(dp[i-1][j],dp[i][j-1]) } $$

Memory:

> **Mismatch !' Max of Top and Left**

---

9. LCS CODE

PYTHON

```python
def lcs(x, y):
    m = len(x)
    n = len(y)

    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):

            if x[i - 1] == y[j - 1]:
                dp[i][j] = 1 + dp[i - 1][j - 1]

            else:
                dp[i][j] = max(
                    dp[i - 1][j],
                    dp[i][j - 1]
                )

    return dp[m][n]
```

Time:

$$ \boxed{O(mn)} $$

Space:

$$ \boxed{O(mn)} $$

---

10. LCS EXAMPLE

```text
X = "abc"
Y = "ac"
```

Table idea:

```text
      ""  a  c
""     0  0  0
a      0  1  1
b      0  1  1
c      0  1  2
```

Answer:

\[\boxed{2} \]

LCS:

```text
ac
```

---

11. LCS SPACE OPTIMIZATION

Current row depends only on:

```text
previous row
current row
```

Therefore space can be reduced to:

\[\boxed{O(n)} \]

where \(n\) is length of smaller string if we choose dimensions carefully.

PART B — LONGEST COMMON SUBSTRING

This is frequently confused with LCS.

12. SUBSTRING MUST BE CONTIGUOUS

Example:

```text
X = "abcdxyz"
Y = "xyzabcd"
```

Longest common substring:

```text
"abcd"
```

length:

\[4 \]

13. TRANSITION

If characters match:

\[\boxed{ dp[i][j] = 1+dp[i-1][j-1] } \]

If they do not match:

\[\boxed{ dp[i][j]=0 } \]

Why zero?

Because substring must remain contiguous.

Mismatch breaks the current common substring.

14. LCS VS COMMON SUBSTRING

LCS	Longest Common Substring
Need not be contiguous	Must be contiguous
`Mismatch!' max(top,left)`	`Mismatch!' 0`
Answer usually dp[m][n]	Track max over all cells

This distinction is extremely important.

PART C — EDIT DISTANCE

One of the most important 2D DP placement questions.

15. PROBLEM

Convert string A into B using minimum operations:

- insert
- delete
- replace

Example:

```text
A = "horse"
B = "ros"
```

Minimum edit distance:

$$ \boxed{3} $$

---

16. STATE

$$ dp[i][j] $$

means:

> **minimum operations required to convert first i characters of A into first j characters of B.**

---

17. BASE CASES

Convert first i characters into empty string:

Need delete all:

$$ \boxed{dp[i][0]=i} $$

Convert empty string into first j characters:

Need insert all:

$$ \boxed{dp[0][j]=j} $$

---

18. CHARACTERS MATCH

If:

$$ A[i-1]=B[j-1] $$

no new operation needed:

$$ \boxed{ dp[i][j] = dp[i-1][j-1] } $$

---

19. CHARACTERS DIFFER

Three choices.

Delete from A

$$ dp[i-1][j] $$

Insert into A

$$ dp[i][j-1] $$

Replace

$$ dp[i-1][j-1] $$

Add one operation:

$$ \boxed{ dp[i][j] = 1+\min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) } $$

20. EDIT DISTANCE MEMORY

```text
 D e l e t e    !'   t o p
 I n s e r t    !'   l e f t
 R e p l a c e  !'   d i a g o n a l
```

Then:

\[1+\min(...) \]

21. CODE

```python
def edit_distance(a, b):
    m = len(a)
    n = len(b)

    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(m + 1):
        dp[i][0] = i

    for j in range(n + 1):
        dp[0][j] = j

    for i in range(1, m + 1):
        for j in range(1, n + 1):

            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]

            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j],
                    dp[i][j - 1],
                    dp[i - 1][j - 1]
                )

    return dp[m][n]
```

Time:

\[\boxed{O(mn)} \]

PART D — MIN INSERTIONS/DELETIONS TO CONVERT STRINGS

22. USE LCS

Suppose:

```text
A length = m
B length = n
```

```
LCS length = L
```

Characters of A not part of LCS must be deleted:

$$ \boxed{m-L} $$

Characters needed from B must be inserted:

$$ \boxed{n-L} $$

Total operations:

$$ \boxed{m+n-2L} $$

when only insertion and deletion are allowed.

---

23. EXAMPLE

```text
A = "heap"
B = "pea"
```

One LCS:

```text
"ea"
```

$$ L=2 $$

Delete:

$$ 4-2=2 $$

Insert:

$$ 3-2=1 $$

Total:

$$ \boxed{3} $$

PART E — LONGEST PALINDROMIC SUBSEQUENCE

Another important transformation.

24. KEY IDEA

For string:

```text
s
```

reverse it:

```text
rev(s)
```

Then:

$$\boxed{ LPS(s)=LCS(s,reverse(s)) }$$

25. EXAMPLE

```text
s = "bbbab"
```

Longest Palindromic Subsequence:

```text
"bbbb"
```

Length:

$$\boxed{4}$$

26. MIN INSERTIONS TO MAKE PALINDROME

If:

$$n=|s|$$

and LPS length is L:

$$\boxed{ Minimum\ insertions=n-L }$$

Because existing longest palindromic subsequence can remain.

PART F — GRID DP

Now the second dimension is row + column.

27. UNIQUE PATHS

Grid:

$$m\times n$$

Robot starts top-left.

Can move only:

```text
right
down
```

Find number of ways to reach bottom-right.

28. STATE

$$ dp[r][c] $$

means:

> **number of ways to reach cell (r,c).**

29. TRANSITION

You can arrive from:

- top
- left

Therefore:

$$ \boxed{ dp[r][c] = dp[r-1][c]+dp[r][c-1] } $$

30. BASE CASE

Starting point:

$$ dp[0][0]=1 $$

First row has only one way:

> **keep moving right.**

First column has only one way:

> **keep moving down.**

31. CODE

```python
def unique_paths(m, n):
    dp = [[1] * n for _ in range(m)]

    for r in range(1, m):
        for c in range(1, n):
            dp[r][c] = dp[r - 1][c] + dp[r][c - 1]

    return dp[m - 1][n - 1]
```

Time:

$$ \boxed{O(mn)} $$

Space:

$$ O(mn) $$

Can optimize to:

$$ O(n) $$

PART G — UNIQUE PATHS WITH OBSTACLES

32. PROBLEM

Some cells blocked.

If blocked:

$$ \boxed{dp[r][c]=0} $$

because no path can pass through that cell.

Otherwise:

$$ dp[r][c] = top+left $$

---

33. IMPORTANT START TRAP

If starting cell itself is blocked:

$$ \boxed{\text{answer = 0}} $$

PART H — MINIMUM PATH SUM

34. PROBLEM

Grid contains costs.

Move only:

```text
right
down
```

Find minimum total path cost.

Example:

```text
1 3 1
1 5 1
4 2 1
```

Best path cost:

$$ \boxed{7} $$

using:

```text
1  !′  3  !′  1  !′  1  !′  1
```

35. STATE

\[dp[r][c] \]

means:

> **minimum cost to reach cell \((r,c)\).**

36. TRANSITION

Arrive from:

- top
- left.

Therefore:

\[\boxed{ dp[r][c] = grid[r][c] + \min(dp[r-1][c], dp[r][c-1]) } \]

37. COUNT VS MIN GRID DP

Compare:

Unique Paths

\[top+left \]

because counting paths.

Minimum Path Sum

\[value+\min(top,left) \]

because minimizing cost.

Again:

> **output type determines combination operation.**

PART I — TRIANGLE DP

38. PROBLEM FAMILY

Triangle:

```TEXT
      2
     3 4
    6 5 7
   4 1 8 3
```

Find minimum path from top to bottom, moving to adjacent entries.

---

39. BOTTOM-UP TRICK

Start from last row.

For every cell:

$$ \boxed{ dp[r][c] = triangle[r][c] + \min(dp[r+1][c],dp[r+1][c+1]) } $$

This is a clean example where bottom-up order starts from bottom.

PART J — LONGEST INCREASING SUBSEQUENCE

One of the most important placement DP questions.

40. PROBLEM

Input:

```TEXT
[10,9,2,5,3,7,101,18]
```

One LIS:

```TEXT
2,3,7,101
```

Length:

$$ \boxed{4} $$

Remember:

> **subsequence need not be contiguous.**

---

41. O(n²) DP STATE

Define:

$$ dp[i] $$

as:

> **length of longest increasing subsequence ending exactly at index i.**

Initialize:

$$ dp[i]=1 $$

because every element alone is an increasing subsequence.

---

42. TRANSITION

For every earlier index $j<i$:

If:

$$ nums[j]<nums[i] $$

then:

$$ \boxed{ dp[i] = \max(dp[i],1+dp[j]) } $$

Final answer:

$$ \boxed{\max_i dp[i]} $$

---

43. CODE

```python
def lis(nums):
    n = len(nums)

    if n == 0:
        return 0

    dp = [1] * n

    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], 1 + dp[j])

    return max(dp)
```

Time:

$$ \boxed{O(n^2)} $$

Space:

$$ O(n) $$

---

44. IMPORTANT LIS STATE TRAP

`dp[i]` does **not** mean:

> **LIS anywhere in first i elements.**

It specifically means:

$$ \boxed{\text{LIS ending at i}} $$

That makes the transition easy.

State definition matters.

PART K — LIS O(n log n)

Important interview optimization.

45. CORE IDEA

Maintain an array often called:

```text
tails
```

where:

\[tails[k] \]

stores the smallest possible ending value of an increasing subsequence of length \(k+1\).

---

46. EXAMPLE

Process:

```text
10, 9, 2, 5, 3, 7, 101, 18
```

Evolution:

```text
[10]
[9]
[2]
[2,5]
[2,3]
[2,3,7]
[2,3,7,101]
[2,3,7,18]
```

Length:

\[4 \]

---

47. WHY REPLACE?

When processing 3:

```text
[2,5]
```

replace 5 with 3:

```text
[2,3]
```

Both represent subsequence length 2.

But ending at 3 is better because it leaves more room for future larger values.

---

48. BINARY SEARCH

Find first position in `tails` such that:

\[tails[pos]\ge x \]

This is:

\[\boxed{\text{Lower Bound}} \]

If none exists:

> **append x.**

Therefore:

\[\boxed{O(n\log n)} \]

---

49. IMPORTANT WARNING

The `tails` array is not necessarily the actual LIS itself.

Its length gives correct LIS length.

This is a frequent interview misconception.

PART L — LIS VS LONGEST CONTIGUOUS INCREASING SUBARRAY

Input:

```text
1,3,5,4,7
```

Longest contiguous increasing run:

```text
1,3,5
```

length 3.

But LIS can be:

```text
1,3,4,7
```

length 4.

Again:

\[\boxed{\text{Subsequence "` Subarray}} \]

PART M — NUMBER OF LIS

Medium interview extension.

Maintain two arrays:

```text
length[i]
count[i]
```

`length[i]`:

> **LIS length ending at i.**

`count[i]`:

> **number of LIS of that length ending at i.**

When a longer predecessor found:

> **replace length and copy count.**

When equal-length predecessor found:

> **add counts.**

This shows how DP can track more than one piece of information.

PART N — MATRIX CHAIN MULTIPLICATION / INTERVAL DP

You already saw MCM for GATE; now focus on placement recognition.

50. INTERVAL DP SIGNAL

If problem asks:

> **best way to split an interval**

and answer depends on trying every split point:

\[k \]

think:

\[\boxed{\text{Interval DP}} \]

State:

$$ dp[i][j] $$

= best answer for interval $(i...j)$.

---

51. MCM TRANSITION

For matrices (i) through (j):

Try all split points:

$$ i\le k<j $$

Cost:

$$ dp[i][k] + dp[k+1][j] + p_{i-1}p_kp_j $$

Take minimum:

$$ \boxed{ dp[i][j] = \min_k(...) } $$

Time:

$$ O(n^3) $$

Space:

$$ O(n^2) $$

PART O — PALINDROME PARTITIONING II TYPE

Another interval-style DP problem.

Question:

> **minimum cuts needed so every substring is palindrome.**

Possible state:

$$ dp[i] $$

= minimum cuts for prefix ending at (i).

Need palindrome-precomputation or repeated checks.

Pattern:

```text
Try partition points
!’   v a l i d a t e   s e g m e n t
!’   c o m b i n e   p r e v i o u s   o p t i m a l   a n s w e r
```

PART P — 1D VS 2D VS INTERVAL DP

52. MASTER DECISION

One changing index/state

Examples:

- Fibonacci
- House Robber
- Climbing Stairs

Think:

$$\boxed{\text{1D DP}}$$

Two sequences / row-column

Examples:

- LCS
- Edit Distance
- Grid paths

Think:

$$\boxed{\text{2D DP}}$$

Subarray/string interval with split

Examples:

- MCM
- Burst Balloons
- palindrome interval problems

Think:

$$\boxed{\text{Interval DP}}$$

PART Q — RECONSTRUCTING THE ACTUAL LCS

Interview follow-up:

> **Not just length; print the LCS.**

Start from:

```text
i = m
j = n
```

If characters match:

> **include character and move diagonal.**

If mismatch:

> **move toward larger DP value.**

```text
if dp[i-1][j] > dp[i][j-1]:
    i--
else:
    j--
```

Collected characters come in reverse.

Reverse at end.

PART R — RECONSTRUCTING LIS

For \(O(n^2)\) LIS, maintain:

```text
parent[i]
```

When:

\[dp[i] \]

improves through \(j\):

```text
parent[i] = j
```

After finding index with maximum `dp`, follow parent pointers backward.

This is a general DP lesson:

> **To reconstruct solution, store choices/predecessors.**

PART S — COMMON 2D DP INTERVIEW TRAPS

53. TRAP 1 — WRONG INDEX OFFSET

If `dp` has size:

\[(m+1)\times(n+1) \]

then string characters often use:

```text
x[i-1]
y[j-1]
```

not:

```text
x[i]
```

---

54. TRAP 2 — LCS VS SUBSTRING

Mismatch:

```TEXT
 L C S            !'   m a x ( t o p , l e f t )
 S u b s t r i n g  !'   0
```

Lock this.

55. TRAP 3 — WRONG EDIT-DISTANCEBASE CASE

Empty !' length-\(j\) string needs:

\[j \]

insertions.

Length-\(i\) !' empty needs:

\[i \]

deletions.

56. TRAP 4 — GRID BOUNDARIES

Top row and left column need careful initialization.

Obstacle grids make this even more important.

57. TRAP 5 — LIS STRICT VS NON-DECREASING

Strictly increasing means:

\[nums[j]<nums[i] \]

Non-decreasing would allow:

\[nums[j]\le nums[i] \]

Read problem exactly.

58. TRAP 6 — LIS `tails` IS ACTUAL SUBSEQUENCE

Not necessarily.

It is a structure maintaining best ending values.

59. TRAP 7 — DP ANSWER LOCATION

For LCS:

\[dp[m][n] \]

For longest common substring:

> **maximum over all cells.**

For LIS:

> **maximum over `dp[i]`.**

Do not assume answer is always last state.

---

60. TRAP 8 — SPACE OPTIMIZATION TOO EARLY

First build correct 2D recurrence.

Only then optimize.

Trying to write compressed DP before understanding dependencies causes bugs.

PART T — INTERVIEW EXPLANATION EXAMPLE: LCS

A strong explanation:

> **"I define `dp[i][j]` as the LCS length using the first `i` characters of the first string and the first `j` characters of the second. If the current characters match, they can extend the LCS of the previous prefixes, so I take `1 + dp[i-1][j-1]`. If they differ, one of the two characters must be excluded, so I take the maximum of `dp[i-1][j]` and `dp[i][j-1]`. There are \(mn\) states with constant work per state, so the time complexity is \(O(mn)\)."**

That's placement-quality reasoning.

PART U — INTERVIEW EXPLANATION EXAMPLE: LIS

> **"I define `dp[i]` as the length of the longest increasing subsequence ending exactly at index `i`. Every element can form a subsequence of length 1 by itself. For each earlier index `j`, if `nums[j] < nums[i]`, I can append the current value to a subsequence ending at `j`, giving `1 + dp[j]`. I take the best such value. This gives \(O(n^2)\) time. It can be optimized to \(O(n\log n)\) using a `tails` array and lower-bound Binary Search."**

PART V — MUST-DO 2D DP QUESTIONS

String DP

1. Longest Common Subsequence
2. Longest Common Substring
3. Edit Distance
4. Longest Palindromic Subsequence

5. Minimum Insertions to Make Palindrome
6. Minimum Insertions/Deletions to Convert Strings
7. Distinct Subsequences
8. Word Break

Grid DP

9. Unique Paths
10. Unique Paths II
11. Minimum Path Sum
12. Triangle Minimum Path
13. Maximum Falling Path Sum
14. Dungeon-type grid DP basics

LIS Family

15. Longest Increasing Subsequence
16. Print LIS
17. Number of LIS
18. Largest Divisible Subset
19. Longest Bitonic Subsequence

Interval DP

20. Matrix Chain Multiplication
21. Palindrome Partitioning II
22. Burst Balloons basics

PLACEMENT PRACTICE SET

Q1 — MCQ

For LCS, if:

$$ X[i-1]=Y[j-1] $$

transition is:

A.

$$ dp[i][j]=dp[i-1][j] $$

B.

$$ dp[i][j]=1+dp[i-1][j-1] $$

C.

$$ dp[i][j]=0 $$

D.

$$ dp[i][j]=dp[i][j-1] $$

Q2 — MCQ

For LCS mismatch:

A.

$$ dp[i][j]=0 $$

B.

$$ dp[i][j]=1+dp[i-1][j-1] $$

C.

$$ dp[i][j]=\max(dp[i-1][j],dp[i][j-1]) $$

D.

$$ dp[i][j]=i+j $$

Q3 — MCQ

For Longest Common Substring mismatch:

A. take max(top,left)

B. set current state to 0

C. add 1

D. use heap

Q4 — NAT

Find LCS length:

```text
X = "ABCD"
Y = "ACBD"
```

Q5 — MCQ

Edit Distance supports:

A. insert, delete, replace

B. only sorting

C. BFS, DFS, heapify

D. only deletion

Q6 — MSQ

Which are typical 2D DP problems?

A. LCS

B. Edit Distance

C. Unique Paths

D. Heap Sort

Q7 — NAT

A 3×3 grid has no obstacles. Robot can move only right/down.

How many unique paths from top-left to bottom-right?

Q8 — MCQ

Minimum Path Sum transition is:

A.

$$ grid[r][c]+\min(top,left) $$

B.

$$ top+left $$

C.

$$ \max(top,left) $$

D.

$$ grid[r][c] $$

only.

Q9 — MCQ

For LIS $O(n^2)$, `dp[i]` commonly means:

A. LIS ending at index i

B. total sum up to i

C. number of graph edges

D. heap size

Q10 — NAT

Find LIS length:

```text
[10,9,2,5,3,7,101,18]
```

Q11 — MCQ

The $O(n\log n)$ LIS optimization uses:

A. Binary Search on tails

B. BFS

C. DSU

D. Floyd-Warshall

Q12 — MSQ

Which statements are true?

A. Substring must be contiguous.

B. Subsequence need not be contiguous.

C. LCS standard complexity is $O(mn)$.

D. Longest Common Substring mismatch uses `max(top,left)`.

Q13 — MCQ

Longest Palindromic Subsequence can be computed as:

A. LCS of string and reverse

B. BFS of string

C. Heap Sort of characters

D. MST

Q14 — NAT

If string length is 7 and its longest palindromic subsequence length is 5, minimum insertions to make it palindrome are:

Q15 — MSQ

Which are interval-DP style problems?

A. Matrix Chain Multiplication

B. Burst Balloons

C. Some palindrome partitioning problems

D. Standard BFS

PRACTICE SOLUTIONS

Q1

Match:

$$\boxed{B}$$

Q2

Mismatch:

$$\boxed{C}$$

Q3

Contiguous match breaks on mismatch.

$$\boxed{B}$$

Q4

Possible LCS:

```text
ABD
```

or:

```text
ACD
```

Length:

$$\boxed{3}$$

Q5

$$\boxed{A}$$

Q6

A, B, C.

$$\boxed{A,B,C}$$

Q7

Paths for 3×3:

Need 2 right and 2 down moves.

Number:

$$\binom42 = \boxed{6}$$

Q8

$$ \boxed{A} $$

Q9

$$ \boxed{A} $$

Q10

Example:

```TEXT
2,3,7,101
```

length:

$$ \boxed{4} $$

Q11

$$ \boxed{A} $$

Q12

A true.

B true.

C true.

D false.

$$ \boxed{A,B,C} $$

Q13

$$ \boxed{A} $$

Q14

$$ 7-5 = \boxed{2} $$

Q15

A, B, C.

\[\boxed{A,B,C} \]

Ø=ÜÓTES TO WRITE

```text
2D DP

LCS:
dp[i][j]
= LCS of first i and j chars

Match:
diagonal + 1

Mismatch:
max(top, left)

LCS:
O(mn)

LONGEST COMMON SUBSTRING:
 M a t c h  !’  d i a g o n a l  +  1
 M i s m a t c h  !’  0
Track global max


EDIT DISTANCE:
State:
 c o n v e r t  f i r s t  i  c h a r s  !’  f i r s t  j  c h a r s

Base:
dp[i][0] = i
dp[0][j] = j

Match:
diagonal

Mismatch:
1 + min(
  delete = top,
  insert = left,
  replace = diagonal
)


GRID DP:
Unique Paths:
top + left

Min Path:
cell + min(top,left)


LIS:
dp[i] = LIS ending at i

If nums[j] < nums[i]:
dp[i] = max(dp[i], 1 + dp[j])

O(n²)

Optimized LIS:
tails + lower_bound
```

```
O(n log n)


INTERVAL DP:
dp[i][j]
try all split points k
```

DP PATTERNMASTER TABLE

Problem Type	Typical State
Fibonacci/House Robber	dp[i]
Knapsack	dp[i][capacity]
LCS/Edit Distance	dp[i][j]
Grid	dp[row][col]
LIS	dp[i] ending at i
Interval splitting	dp[i][j]

&β30-SECOND REVISION

1. Two sequences often !’ 2D DP.
2. LCS state = prefixes of both strings.
3. LCS match !’ diagonal + 1.
4. LCS mismatch !’ max(top,left).
5. Longest Common Substring mismatch !’ 0.
6. Edit Distance base row/column = number of operations.
7. Edit Distance mismatch !’ 1 + min(delete, insert, replace).
8. LPS = LCS(string, reversed string).
9. Min insertions palindrome = \(n-LPS\).
10. Grid paths !’ top + left.
11. Minimum grid cost !’ cell + min(top,left).
12. LIS `dp[i]` = best sequence ending at i.
13. LIS simple = \(O(n^2)\).
14. LIS optimized = Binary Search + tails = \(O(n\log n)\).
15. `tails` is not necessarily the actual LIS.
16. Interval DP uses state \((i,j)\).
17. Interval DP often tries every split \(k\).
18. First understand full DP; optimize space later.
19. Substring is contiguous; subsequence is not.
20. Always define what `dp[i][j]` means before coding.

' Placement Module 13 Complete

Next Placement Module:

MODULE 14 — GREEDY ALGORITHMS + INTERVALS + SCHEDULING FOR PLACEMENTS

Next we'll cover:

- how to recognize Greedy,
- Greedy vs DP,
- Activity/Interval Selection,
- Meeting Rooms,
- Merge Intervals,
- Non-overlapping Intervals,
- Minimum Arrows,
- Job Scheduling patterns,
- Fractional Knapsack,
- Jump Game,
- Gas Station,
- Candy,
- interval sorting rules,
- proof intuition,
- and the most common Greedy interview traps.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 14 — GREEDY ALGORITHMS + INTERVALS + SCHEDULING

Placement Priority: & & & & &

Greedy problems often look deceptively simple.

The basic idea is:

$$ \boxed{\text{Make the best local choice now}} $$

But the difficult part is:

> **proving that this local choice will still lead to a globally optimal answer.**

So never assume:

> **“Looks greedy, so greedy must work.”**

---

1. WHAT IS A GREEDY ALGORITHM?

A Greedy algorithm repeatedly makes the best immediate choice without reconsidering previous decisions.

Typical structure:

```text
Choose locally best option
!’   C o m m i t   t o   i t
!’   R e d u c e   r e m a i n i n g   p r o b l e m
!’   R e p e a t
```

Unlike Backtracking:

> **Greedy does not usually undo choices.**

---

2. GREEDY VS DP

Greedy	Dynamic Programming
Commits to one choice	Explores/computes multiple states
Usually simpler	Usually more state-heavy
Needs greedy-choice property	Uses overlapping subproblems
Often faster	May be \(O(nW), O(n^2)\), etc.
Does not reconsider	Combines optimal subproblems

---

3. CLASSIC EXAMPLE

Fractional Knapsack

Take items by maximum:

\[\frac{value}{weight} \]

Greedy works because fractions are allowed.

0/1 Knapsack

Items cannot be split.

Same ratio strategy can fail.

Therefore:

```TEXT
 F r a c t i o n a l  K n a p s a c k !'  G r e e d y
 0 / 1  K n a p s a c k              !'  D P
```

4. WHEN TO SUSPECT GREEDY

Signals:

- choose maximum/minimum repeatedly
- schedule maximum number of activities
- minimize overlaps
- minimum number of intervals/resources
- minimum cost by taking cheapest available choice
- maximize reach
- sort then scan
- “earliest finishing”
- “furthest reachable”
- “smallest ending point”

Think:

\[\boxed{\text{Sort + Greedy Decision}} \]

very often.

PART A — ACTIVITY SELECTION

5. PROBLEM

Each activity has:

```TEXT
start time
finish time
```

Choose maximum number of non-overlapping activities.

Example:

```TEXT
Activity    Start   Finish
A             1       2
B             3       4
C             0       6
D             5       7
E             8       9
F             5       9
```

6. GREEDY RULE

Always choose activity with:

$$\boxed{\text{earliest finishing time}}$$

Why?

Because finishing early leaves maximum room for future activities.

---

7. ALGORITHM

1. Sort by finishing time.
2. Select first activity.
3. Keep `last_end`.
4. For each next activity:
   - if `start >= last_end`
   - select it
   - update `last_end`.

---

8. CODE

```python
def max_activities(intervals):
    intervals.sort(key=lambda x: x[1])

    count = 0
    last_end = float("-inf")

    for start, end in intervals:
        if start >= last_end:
            count += 1
            last_end = end

    return count
```

Time:

$$\boxed{O(n\log n)}$$

because sorting dominates.

---

9. MEMORY TRICK

> **Maximum number of intervals !' earliest finish**

Not:

- earliest start
- shortest duration
- longest interval

Those can fail.

PART B — NON-OVERLAPPING INTERVALS

10. PROBLEM

Given intervals, remove minimum number so remaining intervals do not overlap.

Example:

```text
[1,2]
[2,3]
[3,4]
[1,3]
```

Remove:

```text
[1,3]
```

Answer:

$$\boxed{1}$$

---

11. TRANSFORMATION

Instead of:

> **minimum removals**

think:

> **maximum number of non-overlapping intervals.**

If maximum kept is k:

$$\boxed{removals=n-k}$$

Then use Activity Selection:

> **earliest finishing interval.**

---

12. DIRECT GREEDY

Sort by end time.

Whenever an interval overlaps:

> **remove one.**

Which one should survive?

The one with smaller end time.

Because it leaves more future space.

PART C — MERGE INTERVALS

You saw this in Arrays, but it belongs strongly to the interval family.

13. PROBLEM

Input:

```text
[1,3]
[2,6]
[8,10]
[15,18]
```

Output:

```text
[1,6]
[8,10]
[15,18]
```

---

14. STRATEGY

Sort by:

$$ \boxed{\text{start time}} $$

Then compare current interval with last merged interval.

If:

$$ current.start \le last.end $$

overlap exists.

Merge:

$$ last.end=\max(last.end,current.end) $$

---

15. CODE

```python
def merge_intervals(intervals):
    if not intervals:
        return []
```

```
    intervals.sort(key=lambda x: x[0])
    ans = [intervals[0]]

    for start, end in intervals[1:]:
        if start <= ans[-1][1]:
            ans[-1][1] = max(ans[-1][1], end)
        else:
            ans.append([start, end])

    return ans
```

Time:

\[\boxed{O(n\log n)} \]

---

16. INTERVAL SORTING RULES

This is important:

```TEXT
Merge Intervals
!’   s o r t   b y   S T A R T

Maximum non-overlapping intervals
!’   s o r t   b y   E N D

Minimum removals
!’   s o r t   b y   E N D
```

Do not use the same sorting rule for every interval problem.

PART D — MEETING ROOMS I

17. PROBLEM

Can one person attend all meetings?

Example:

```TEXT
[0,30]
[5,10]
[15,20]
```

No.

---

18. SOLUTION

Sort by start time.

Check adjacent meetings.

If:

$$ current.start < previous.end $$

overlap.

Return False.

Otherwise True.

Time:

$$ O(n\log n) $$

PART E — MEETING ROOMS II

Very common placement problem.

19. PROBLEM

Find minimum number of meeting rooms required.

Example:

```text
[0,30]
[5,10]
[15,20]
```

Timeline:

```text
0---------30
   5--10
        15--20
```

Maximum simultaneous meetings:

$$ \boxed{2} $$

Need 2 rooms.

---

20. HEAP APPROACH

Sort meetings by start time.

Maintain Min Heap of ending times.

Heap represents rooms currently occupied.

For each meeting:

If earliest ending room satisfies:

$$ heap[0]\le current.start $$

reuse that room.

Pop old end.

Then push current end.

Maximum heap size:

$$ \boxed{\text{minimum rooms}} $$

---

21. CODE

```python
import heapq

def min_meeting_rooms(intervals):
    intervals.sort(key=lambda x: x[0])

    heap = []

    for start, end in intervals:
        if heap and heap[0] <= start:
            heapq.heappop(heap)

        heapq.heappush(heap, end)

    return len(heap)
```

Time:

$$ \boxed{O(n\log n)} $$

---

22. IMPORTANT CONNECTION

Meeting Rooms II combines:

$$ \boxed{\text{Intervals + Min Heap}} $$

Because we repeatedly need:

> **earliest room to become free.**

PART F — MINIMUM ARROWS TO BURST BALLOONS

23. PROBLEM

Each balloon is an interval.

Example:

```text
[10,16]
[2,8]
[1,6]
[7,12]
```

One arrow at a coordinate bursts all balloons containing that coordinate.

Find minimum arrows.

24. GREEDY RULE

Sort balloons by:

$$ \boxed{\text{ending coordinate}} $$

Shoot arrow at earliest possible end.

Why?

It maximizes chance of hitting future overlapping balloons.

25. TRACE

Sorted by end:

```text
[1,6]
[2,8]
[7,12]
[10,16]
```

Arrow at:

$$ 6 $$

bursts first two.

Next interval starts 7 > 6.

Shoot new arrow at:

$$ 12 $$

bursts remaining two.

Answer:

$$ \boxed{2} $$

26. MEMORY

```text
Minimum points/arrows to hit intervals
!'   s o r t   b y   E N D
!'   c h o o s e   e n d i n g   p o i n t
```

PART G — JOB SEQUENCING WITH DEADLINES

Classic Greedy.

27. PROBLEM

Each job has:

```text
deadline
profit
```

Each job takes one unit of time.

Choose jobs to maximize profit.

28. GREEDY RULE

Sort jobs by:

\[\boxed{\text{profit descending}} \]

For each job:

> **schedule it in the latest available slot before its deadline.**

29. WHY LATEST SLOT?

Suppose a job can run before deadline 4.

If you put it at slot 1 unnecessarily, you may block another job whose deadline is 1.

So place it as late as possible.

Memory:

> **High profit first, latest legal slot**

30. COMPLEXITY

Simple implementation:

\[O(n^2) \]

Can be improved using DSU in advanced versions.

PART H — FRACTIONAL KNAPSACK

31. PROBLEM

Each item:

```text
weight
value
```

Capacity \(W\).

Fractions allowed.

32. GREEDY RULE

Compute:

$$ \boxed{\frac{value}{weight}} $$

Sort descending.

Take highest ratio first.

If whole item fits:

> **take all.**

Otherwise:

> **take required fraction.**

33. WHY GREEDY WORKS HERE

Because fractional splitting allows you to continuously allocate capacity to the most valuable weight first.

This property disappears in 0/1 Knapsack.

34. EXAMPLE

Items:

```TEXT
Weight   Value   Ratio
10        60       6
20       100       5
30       120       4
```

Capacity:

$$ 50 $$

Take first:

$$ 10,\ value=60 $$

Take second:

$$ 20,\ value=100 $$

Remaining:

$$ 20 $$

Take:

$$ 20/30 $$

of third.

Value:

$$ 120\times\frac{20}{30}=80 $$

Total:

$$ 60+100+80 = \boxed{240} $$

PART I — JUMP GAME

A very important Greedy array problem.

35. PROBLEM

Input:

```
TEXT
[2,3,1,1,4]
```

At index i, value tells maximum jump length.

Can you reach last index?

---

36. GREEDY IDEA

Maintain:

$$ \boxed{maxReach} $$

furthest index reachable so far.

For each index i:

If:

$$ i>maxReach $$

we cannot even reach current index.

Return False.

Otherwise update:

$$ maxReach=\max(maxReach,i+nums[i]) $$

---

37. CODE

```
PYTHON
def can_jump(nums):
    max_reach = 0

    for i in range(len(nums)):
        if i > max_reach:
            return False
```

```
        max_reach = max(max_reach, i + nums[i])

    return True
```

Time:

\[\boxed{O(n)} \]

Space:

\[O(1) \]

---

38. MEMORY

> **Jump Game feasibility !' furthest reachable position**

You do not need to explore every jump path.

PART J — JUMP GAME II

39. PROBLEM

Find minimum number of jumps to reach end.

Example:

```text
[2,3,1,1,4]
```

Minimum:

\[\boxed{2} \]

Path:

```text
 0  !'   1  !'   4
```

---

40. GREEDY LEVEL IDEA

Think of reachable index ranges like BFS levels.

Maintain:

```text
current_end
farthest
jumps
```

For each index:

\[farthest=\max(farthest,i+nums[i]) \]

When:

\[i=current_end \]

you have finished current jump range.

So:

```text
jumps++
current_end = farthest
```

---

41. CODE

```python
def min_jumps(nums):
    jumps = 0
    current_end = 0
    farthest = 0

    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])

        if i == current_end:
            jumps += 1
            current_end = farthest

    return jumps
```

This assumes end is reachable.

Time:

\[O(n) \]

PART K — GAS STATION

Classic tricky Greedy problem.

42. PROBLEM

At station \(i\):

```text
gas[i]
cost[i]
```

Need complete circular route.

Find valid starting station.

---

43. NECESSARY CONDITION

If:

$$ \sum gas < \sum cost $$

impossible.

Return:

$$ -1 $$

---

44. GREEDY INSIGHT

Maintain current tank.

If starting from `start` fails at station i:

$$ tank<0 $$

then no station between `start` and i can be a valid start.

So reset:

```
TEXT
start = i + 1
tank = 0
```

---

45. CODE

```
PYTHON
def gas_station(gas, cost):
    total = 0
    tank = 0
    start = 0

    for i in range(len(gas)):
        diff = gas[i] - cost[i]

        total += diff
        tank += diff

        if tank < 0:
            start = i + 1
            tank = 0

    return start if total >= 0 else -1
```

Time:

$$ \boxed{O(n)} $$

PART L — CANDY

Common Greedy challenge.

46. PROBLEM

Each child has a rating.

Rules:

1. everyone gets at least 1 candy.
2. higher-rated child than adjacent neighbor gets more candy.

Find minimum total candies.

---

47. TWO-PASS GREEDY

Initialize:

```text
candies = 1 for everyone
```

Left-to-right:

If:

$$ rating[i]>rating[i-1] $$

then:

$$ candies[i]=candies[i-1]+1 $$

Right-to-left:

If:

$$ rating[i]>rating[i+1] $$

then:

$$ candies[i]=\max(candies[i],candies[i+1]+1) $$

---

48. WHY TWO PASSES?

One pass satisfies left-neighbor constraint.

Other satisfies right-neighbor constraint.

Need both.

Time:

$$ O(n) $$

Space:

$$ O(n) $$

There are advanced constant-space variants, but this is placement-safe.

PART M — ASSIGN COOKIES

49. PROBLEM

Children have greed factors.

Cookies have sizes.

Each child gets at most one cookie.

Maximize number of satisfied children.

---

50. GREEDY RULE

Sort:

```TEXT
children
cookies
```

Use two pointers.

Try to satisfy the least-demanding remaining child with the smallest cookie that works.

Why?

Don't waste a large cookie on a child who could accept a small one.

---

51. PATTERN

\[\boxed{\text{Sort both + smallest feasible pairing}} \]

This appears in many matching problems.

PART N — PARTITION LABELS

Interesting Greedy string problem.

52. PROBLEM IDEA

Split string into as many parts as possible such that every character appears in at most one part.

Example:

```TEXT
ababcbacadefegdehijhklij
```

Need know each character's last occurrence.

---

53. GREEDY IDEA

While scanning:

\[end=\max(end,lastOccurrence[ch]) \]

When:

\[i=end \]

all characters seen in current partition end within that boundary.

So cut partition.

Pattern:

\[\boxed{\text{Expand boundary using farthest requirement}} \]

PART O — TASK SCHEDULING GREEDY

Some scheduling problems ask:

> **execute tasks under deadlines, intervals or cooldowns.**

Be careful: not all are pure Greedy.

Examples:

```text
Activity Selection
!’   G r e e d y

Meeting Rooms II
!’   H e a p

Job Sequencing
!’   G r e e d y   +   s l o t s

Task Scheduler with cooldown
!’   H e a p / c o u n t i n g
```

Always identify exact objective.

PART P — GREEDY PROOF INTUITION

You usually don't need formal mathematical proof in interviews, but you should explain why the local choice is safe.

Three useful proof intuitions:

---

54. EXCHANGE ARGUMENT

Show that any optimal solution can be changed to include your greedy choice without making it worse.

Example Activity Selection:

If an optimal schedule starts with an activity that finishes later than the greedy earliest-finishing activity, replace it with the greedy one.

You do not reduce remaining available time.

Therefore greedy choice is safe.

---

55. STAY-AHEAD ARGUMENT

Show greedy solution is always at least as good as alternatives after every step.

Often used in scheduling/reachability reasoning.

56. CUT / LOCAL-SAFE PROPERTY

For MST:

> **cheapest safe crossing edge can belong to an MST.**

That's the cut property.

Same general mindset:

> **prove the choice doesn't destroy optimality.**

PART Q — GREEDY FAILURE EXAMPLE

57. COIN CHANGE

Suppose coins:

```text
[1,3,4]
```

Amount:

\[6 \]

Greedy chooses largest first:

```text
4 + 1 + 1
```

3 coins.

Optimal:

```text
3 + 3
```

2 coins.

Therefore arbitrary coin systems do not support simple largest-first Greedy.

Use DP.

58. LESSON

Just because a problem says:

> **minimum number**

doesn't mean Greedy works.

Need structural reason/proof.

PART R — INTERVAL MASTER MAP

Memorize:

Problem	Sort By	Main Idea
Merge Intervals	Start	Extend current merged interval
Meeting Rooms I	Start	Detect overlap
Activity Selection	End	Take earliest finish
Non-overlap removal	End	Keep earliest finish
Minimum Arrows	End	Shoot at current end
Meeting Rooms II	Start	Min Heap of end times

This table is extremely placement-useful.

PART S — GREEDY VS HEAP

Sometimes Greedy strategy needs a Heap to efficiently select the next greedy choice.

Examples:

Connect Ropes

Greedy choice:

> **choose two smallest.**

Data structure:

$$\boxed{\text{Min Heap}}$$

Task Scheduler

Greedy choice:

> **choose most frequent available task.**

Data structure:

$$\boxed{\text{Max Heap}}$$

So:

> **Greedy = strategy.**

> **Heap = data structure helping implement it.**

PART T — COMMON GREEDY INTERVIEW TRAPS

59. TRAP 1 — "TAKE THE BIGGEST"

Greedy does not always mean choose largest value.

Activity Selection chooses:

$$ \boxed{\text{earliest finish}} $$

not largest anything.

---

60. TRAP 2 — WRONG INTERVAL SORT KEY

```text
Merge !' Start
Select maximum count !' End
Minimum arrows !' End
```

Sorting choice determines correctness.

---

61. TRAP 3 — TOUCHING INTERVALS

Suppose:

```text
[1,2]
[2,3]
```

Are these overlapping?

Depends on problem definition.

For meeting scheduling, one ending at 2 and next starting at 2 is usually allowed:

$$ start\ge previousEnd $$

Read specification carefully.

---

62. TRAP 4 — 0/1 VS FRACTIONAL KNAPSACK

```text
Fractional !' Greedy
0/1 !' DP
```

Lock it.

63. TRAP 5 — JUMP GAME BRUTE FORCE

You don't need to recursively try every jump for reachability.

Track:

\[\boxed{maxReach} \]

64. TRAP 6 — GAS STATION ONLY CHECKING TOTAL

Total gas "etotal cost means some valid starting point exists under the standard problem, but you still need find which one.

Use local reset logic.

65. TRAP 7 — GREEDY WITHOUT PROOF

Always ask:

> **Why can this choice never hurt a future optimal solution?**

If you cannot answer, consider DP/backtracking.

PART U — INTERVIEW EXPLANATION EXAMPLE

Suppose interviewer asks:

> **Remove minimum intervals to eliminate overlaps.**

Strong explanation:

> **“Minimizing removals is equivalent to maximizing the number of intervals we keep without overlap. For that, I use the classic activity-selection greedy strategy: sort intervals by ending time and repeatedly keep the interval that finishes earliest. Finishing earlier leaves the maximum possible room for future intervals. If an interval starts before the current selected interval ends, I remove that interval. Sorting costs \(O(n\log n)\), and the scan is \(O(n)\).”**

PART V — GREEDY RECOGNITION MAP

```text
Maximum non-overlapping intervals
!’   E a r l i e s t   f i n i s h

Minimum overlap removals
!’   E a r l i e s t   f i n i s h
```

```
Merge intervals
!’  S o r t  b y  s t a r t

Minimum arrows / points covering intervals
!’  E a r l i e s t  e n d

Meeting rooms
!’  S t a r t  +  h e a p  o f  e n d s

Fractional Knapsack
!’  H i g h e s t  v a l u e / w e i g h t

Jump Game
!’  F a r t h e s t  r e a c h a b l e

Gas Station
!’  R u n n i n g  b a l a n c e  +  r e s e t

Candy
!’  T w o  d i r e c t i o n a l  g r e e d y  p a s s e s

Assign Cookies
!’  S m a l l e s t  f e a s i b l e  m a t c h
```

PART W — MUST-DO GREEDY QUESTIONS

Foundation

1. Activity Selection
2. Fractional Knapsack
3. Assign Cookies
4. Lemonade Change

Interval Core

5. Merge Intervals
6. Insert Interval
7. Meeting Rooms
8. Meeting Rooms II
9. Non-overlapping Intervals
10. Minimum Arrows to Burst Balloons

Core Greedy

11. Jump Game
12. Jump Game II
13. Gas Station
14. Candy
15. Partition Labels
16. Job Sequencing with Deadlines

Important Follow-Ups

17. Task Scheduler
18. Reorganize String
19. Minimum Platforms
20. Connect Ropes
21. Valid Parenthesis String
22. Boats to Save People

PLACEMENT PRACTICE SET

Q1 — MCQ

To maximize number of non-overlapping activities, sort by:

A. earliest start B. earliest finish C. longest duration D. largest start

Q2 — MCQ

Merge Intervals is typically sorted by:

A. start time B. end time only C. interval length D. midpoint

Q3 — MCQ

Minimum arrows to burst intervals is naturally solved by sorting by:

A. earliest ending point B. longest interval C. latest start only D. random endpoint

Q4 — MSQ

Which are classic Greedy problems?

A. Fractional Knapsack

B. Activity Selection

C. Jump Game

D. 0/1 Knapsack

Q5 — MCQ

Why is earliest finish correct for Activity Selection?

A. It maximizes remaining room for future activities.

B. It always has maximum duration.

C. It guarantees maximum profit.

D. It uses a heap.

Q6 — NAT

Intervals:

```text
[1,2]
[2,3]
[3,4]
[1,3]
```

Minimum removals to make them non-overlapping?

Q7 — MCQ

Meeting Rooms II commonly uses:

A. Min Heap of end times

B. DFS recursion

C. DSU

D. LCS

Q8 — MCQ

Fractional Knapsack sorts by:

A. weight only

B. value only

C. value/weight ratio

D. deadline

Q9 — MCQ

Which statement is correct?

A. Greedy ratio always solves 0/1 Knapsack.

B. Greedy ratio solves Fractional Knapsack.

C. Both always need DP.

D. Neither can be optimized.

Q10 — MCQ

Jump Game reachability can be solved by tracking:

A. maximum reachable index

B. minimum heap

C. indegree

D. LCS table

Q11 — MSQ

Which interval rules are correct?

A. Merge Intervals !' sort by start.

B. Maximum non-overlapping intervals !' sort by end.

C. Minimum arrows !' sort by end.

D. Meeting Rooms II !' heap of ending times.

Q12 — MCQ

For Gas Station, if:

$$ \sum gas < \sum cost $$

then:

A. some start always exists

B. no complete route exists

C. use Binary Search

D. choose station 0

Q13 — NAT

Coins:

```text
[1,3,4]
```

Amount:

$$ 6 $$

How many coins does naive largest-first Greedy use?

Q14 — NAT

For the same coin system, what is the optimal minimum number of coins?

Q15 — MSQ

Which statements are true?

A. Greedy solutions need correctness justification.

B. Greedy usually commits to choices.

C. Heap can be used to implement a Greedy strategy.

D. Every optimization problem has a correct Greedy algorithm.

PRACTICE SOLUTIONS

Q1

$$\boxed{B}$$

Q2

$$\boxed{A}$$

Q3

$$\boxed{A}$$

Q4

Fractional Knapsack, Activity Selection and Jump Game have standard greedy solutions.

0/1 Knapsack needs DP in the general case.

$$\boxed{A,B,C}$$

Q5

$$\boxed{A}$$

Q6

Keep:

```text
[1,2]
[2,3]
[3,4]
```

Remove:

```text
[1,3]
```

Answer:

$$\boxed{1}$$

Q7

$$\boxed{A}$$

Q8

$$\boxed{C}$$

Q9

$$\boxed{B}$$

Q10

$$\boxed{A}$$

Q11

All four.

$$\boxed{A,B,C,D}$$

Q12

$$\boxed{B}$$

Q13

Greedy:

```text
4 + 1 + 1
```

$$\boxed{3}$$

Q14

Optimal:

```text
3 + 3
```

$$\boxed{2}$$

This demonstrates Greedy failure.

Q15

A, B, C true.

D false.

\[\boxed{A,B,C} \]

Ø=ÑÓTES TO WRITE

```TEXT
GREEDY
= make locally best safe choice
= usually don't undo

Greedy must be justified.

GREEDY vs DP:
 G r e e d y  !’  c o m m i t
 D P         !’  c o m p a r e / s t o r e  s t a t e s

INTERVALS:

Merge Intervals
!’  s o r t  b y  S T A R T

Maximum non-overlap
!’  s o r t  b y  E N D

Minimum removals
!’  k e e p  m a x i m u m  n o n - o v e r l a p
!’  s o r t  b y  E N D

Minimum arrows
!’  s o r t  b y  E N D
!’  s h o o t  a t  e a r l i e s t  e n d

Meeting Rooms II
!’  s o r t  b y  S T A R T
!’  M i n  H e a p  o f  E N D  t i m e s

Fractional Knapsack
!’  v a l u e / w e i g h t  d e s c e n d i n g

0/1 Knapsack
!’  D P

Jump Game
!’  m a x R e a c h

Gas Station
!’  t o t a l  b a l a n c e  +  r e s e t  s t a r t

Candy
!’  l e f t  p a s s  +  r i g h t  p a s s

Assign Cookies
!’  s m a l l e s t  f e a s i b l e  p a i r i n g
```

GREEDY MASTER CHECKLIST

Before using Greedy, ask:

```text
1. What exactly am I optimizing?
2. What local choice looks best?
3. What should I sort by?
4. Why does this choice preserve future options?
5. Can I give an exchange/stay-ahead argument?
6. Is there a counterexample?
7. Does DP seem necessary instead?
```

COMPLEXITIES TO REMEMBER

Problem	Complexity
Activity Selection	\(O(n\log n)\)
Merge Intervals	\(O(n\log n)\)
Non-overlap removal	\(O(n\log n)\)
Minimum Arrows	\(O(n\log n)\)
Meeting Rooms II	\(O(n\log n)\)
Fractional Knapsack	\(O(n\log n)\)
Jump Game	\(O(n)\)
Jump Game II	\(O(n)\)
Gas Station	\(O(n)\)
Candy	\(O(n)\)
Assign Cookies	\(O(n\log n)\)

&β30-SECOND GREEDY REVISION

1. Greedy = local safe choice.
2. A greedy-looking choice still needs justification.
3. Activity Selection !’ earliest finish.
4. Merge Intervals !’ sort by start.
5. Minimum removals !’ maximize intervals kept.
6. Minimum arrows !’ earliest end.
7. Meeting Rooms II !’ Min Heap of ending times.
8. Fractional Knapsack !’ value/weight.
9. 0/1 Knapsack !’ not ratio Greedy.
10. Jump Game !’ farthest reachable.

11. Jump Game II !’ greedy range expansion.
12. Gas Station !’ running balance + restart.
13. Candy !’ left and right constraints.
14. Assign Cookies !’ smallest feasible match.
15. Heap can implement a Greedy choice.
16. Greedy "`always “pick maximum.”
17. Counterexamples are important.
18. If choices interact heavily and states repeat, consider DP.

' Placement Module 14 Complete

Next Placement Module:

MODULE 15 — TRIES + ADVANCED STRING PATTERNS + KMP / RABIN-KARP

Next we'll cover:

- Trie fundamentals,
- insert/search/prefix,
- Word Dictionary,
- prefix autocomplete,
- Trie + DFS,
- Word Search II intuition,
- pattern matching,
- naive string matching,
- KMP,
- LPS/prefix-function array,
- Rabin-Karp,
- rolling hash,
- when to use hashing vs Trie vs KMP,
- and the most important advanced string interview patterns.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 15 — TRIES + ADVANCED STRING PATTERNS + KMP / RABIN-KARP

Placement Priority: & & & & &

This module is especially useful when questions involve:

- prefixes,
- dictionaries,
- autocomplete,
- repeated string lookup,
- many words,
- pattern matching,
- searching one string inside another efficiently.

The three big ideas are:

\[\boxed{\text{Trie}} \]

\[\boxed{\text{KMP}} \]

\[\boxed{\text{Rolling Hash / Rabin-Karp}} \]

---

1. WHEN SHOULD YOU THINK OF A TRIE?

Suppose you have words:

```text
apple
app
apply
apt
bat
ball
```

Now questions repeatedly ask:

> **Does `"apple"` exist?**

> **Does any word start with `"app"`?**

> **Find words with prefix `"ap"`.**

Using a Trie makes prefix operations natural.

Memory:

\[\boxed{\text{Many words + Prefix queries !' Trie}} \]

PART A — TRIE FUNDAMENTALS

2. WHAT IS A TRIE?

Trie is a tree designed for strings.

Also called:

\[\boxed{\text{Prefix Tree}} \]

Each edge/node represents a character.

Example words:

```text
app
apple
apt
```

Conceptually:

```text
root
 |
 a
 |
 p
 / \
p   t
|
l
|
e
```

---

3. WHY DO WORDS SHARE NODES?

Words:

```text
app
apple
application
```

all share prefix:

```text
app
```

Trie stores that common prefix once.

That's the main benefit.

---

4. TRIE NODE

Each node normally contains:

1. children
2. end-of-word marker

Example Python:

```python
class TrieNode:
    def __init__(self):
```

```
        self.children = {}
        self.end = False
```

---

5. WHY DO WE NEED `end`?

Suppose Trie contains:

```TEXT
apple
```

Then path:

```TEXT
 a  !’   p  !’   p
```

exists.

But that does **not** automatically mean:

```TEXT
"app"
```

was inserted as a complete word.

Therefore we need:

```TEXT
end = True
```

on nodes where a word actually ends.

PART B — INSERT INTO TRIE

6. EXAMPLE

Insert:

```TEXT
"cat"
```

Start at root.

For:

```TEXT
c
```

if child doesn't exist:

> **create it.**

Then:

```text
a
```

Then:

```text
t
```

At final node:

```text
end = True
```

---

7. CODE

```python
class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root

        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()

            node = node.children[ch]

        node.end = True
```

If word length is \(L\):

\[\boxed{O(L)} \]

time.

PART C — SEARCH COMPLETE WORD

8. PROBLEM

Trie contains:

```text
apple
```

Search:

```text
apple
```

Return True.

Search:

```text
app
```

Return False unless `app` itself was inserted.

---

9. CODE

```python
def search(self, word):
    node = self.root

    for ch in word:
        if ch not in node.children:
            return False

        node = node.children[ch]

    return node.end
```

Time:

\[\boxed{O(L)} \]

PART D — PREFIX SEARCH

10. PROBLEM

Does any word begin with:

```text
"app"
```

Here we do **not** require `end=True`.

We only need path to exist.

---

11. CODE

```python
def starts_with(self, prefix):
    node = self.root

    for ch in prefix:
        if ch not in node.children:
```

```
            return False

        node = node.children[ch]

    return True
```

Time:

\[\boxed{O(L)} \]

---

12. SEARCH VS PREFIX

Important:

```text
search("app")
```

asks:

> **Is `"app"` a complete stored word?**

But:

```text
startsWith("app")
```

asks:

> **Does at least one stored word begin with `"app"`?**

Do not confuse them.

PART E — ARRAY CHILDREN VS HASHMAP CHILDREN

If alphabet is known and small, such as lowercase English letters:

```text
a-z
```

each node can use:

```text
children[26]
```

Then character index:

\[ord(ch)-ord('a') \]

---

13. WHICH IS BETTER?

Array children

Pros:

- predictable $O(1)$ access
- fast

Cons:

- potentially wastes memory

HashMap children

Pros:

- stores only existing children
- flexible alphabet

Cons:

- hashing overhead

14. INTERVIEW ANSWER

For lowercase-only dictionary:

> **Array of size 26 is simple and fast.**

For arbitrary characters:

> **HashMap is more flexible.**

PART F — TRIE COMPLEXITY

Suppose word length L.

Operation	Time
Insert	$O(L)$
Search	$O(L)$
Prefix search	$O(L)$

Space depends on total inserted characters.

Upper bound roughly:

$$ \boxed{O(\text{total characters inserted})} $$

PART G — AUTOCOMPLETE

15. PROBLEM

Stored words:

```TEXT
apple
app
apply
apt
bat
```

Prefix:

```TEXT
"app"
```

Possible suggestions:

```TEXT
app
apple
apply
```

---

16. SOLUTION

Step 1:

Traverse Trie to prefix node.

Step 2:

From that node, DFS all descendants.

Whenever:

```TEXT
end == True
```

record current word.

Pattern:

\[\boxed{\text{Trie + DFS}} \]

PART H — WORD DICTIONARY WITH `.`

Classic interview question.

17. PROBLEM

Support:

```text
addWord("bad")
addWord("dad")
addWord("mad")
```

Search:

```text
 " p a d "  !’   F a l s e
 " b a d "  !’   T r u e
 " . a d "  !’   T r u e
 " b . . "  !’   T r u e
```

`.` can match any character.

---

18. WHY NORMAL TRIE SEARCH IS NOT ENOUGH

If character is normal:

> **follow one child.**

If character is `.`:

> **try every child.**

Therefore wildcard introduces branching.

Use:

\[\boxed{\text{Trie + DFS/Backtracking}} \]

---

19. CORE LOGIC

```python
def dfs(i, node):
    if i == len(word):
        return node.end

    ch = word[i]

    if ch == '.':
        for child in node.children.values():
            if dfs(i + 1, child):
                return True
        return False

    if ch not in node.children:
        return False

    return dfs(i + 1, node.children[ch])
```

PART I — WORD SEARCH II

Important advanced pattern.

20. PROBLEM

Given:

- character board
- many dictionary words

Find all dictionary words that exist in board.

Naively:

> **run Word Search separately for every word.**

That can be expensive.

---

21. OPTIMAL IDEA

Put all words into Trie.

Then DFS board once while simultaneously walking Trie paths.

If current board path doesn't exist in Trie:

\[\boxed{\text{stop immediately}} \]

This is powerful pruning.

Pattern:

\[\boxed{\text{Trie + Grid DFS + Backtracking}} \]

---

22. WHY TRIE HELPS

Suppose dictionary contains:

```text
cat
car
care
card
dog
```

While board DFS currently spells:

```text
cx
```

Trie immediately tells us:

> **no dictionary word starts with `"cx"`.**

So stop exploring that path.

PART J — LONGEST COMMON PREFIX USING TRIE

You can insert words and follow single-child path until:

- node has multiple children,
- or a word ends.

That path forms common prefix.

But for one small input list, simple scanning is often easier.

Important interview lesson:

> **A Trie is not automatically the best solution just because strings are involved.**

PART K — WHEN NOT TO USE A TRIE

Avoid overengineering.

If problem asks:

> **Compare only two strings once.**

Trie is unnecessary.

If problem asks:

> **find palindrome.**

Use Two Pointers.

If problem asks:

> **count character frequency.**

Use HashMap/frequency array.

Use Trie mainly when:

\[\boxed{\text{many prefix/dictionary operations}} \]

matter.

PART L — STRING PATTERN MATCHING

Now consider:

Text:

```text
ABABDABACDABABCABAB
```

Pattern:

```text
ABABCABAB
```

Question:

> **At what index does pattern occur in text?**

---

23. NAIVE STRING MATCHING

For each starting position in text:

> **compare pattern characters.**

If text length:

$$ n $$

pattern length:

$$ m $$

Worst-case:

$$ \boxed{O(nm)} $$

---

24. EXAMPLE OF REPEATED WORK

Text:

```text
aaaaaaaaab
```

Pattern:

```text
aaaab
```

At many positions we compare several `a`s again and again.

KMP avoids rechecking information already known.

PART M — KMP ALGORITHM

KMP = Knuth-Morris-Pratt.

Goal:

$$ \boxed{\text{Pattern matching in }O(n+m)} $$

The key structure is:

$$ \boxed{\text{LPS array}} $$

---

25. WHAT IS LPS?

Here LPS means:

$$ \boxed{\text{Longest Proper Prefix which is also Suffix}} $$

Do not confuse it with:

> **Longest Palindromic Subsequence.**

Same abbreviation, completely different concept.

---

26. WHAT IS A PROPER PREFIX?

For string:

```text
ABAB
```

Prefixes:

```text
A
AB
ABA
ABAB
```

Proper prefixes exclude entire string:

```text
A
AB
ABA
```

Suffixes:

```text
B
AB
BAB
ABAB
```

Longest proper prefix also suffix:

```text
AB
```

Length:

$$ \boxed{2} $$

PART N — LPS ARRAY EXAMPLE

Pattern:

```text
ABABCABAB
```

LPS values:

```text
Index:   0 1 2 3 4 5 6 7 8
Char:    A B A B C A B A B
LPS:     0 0 1 2 0 1 2 3 4
```

Interpretation:

At index 8, prefix/suffix:

```text
ABAB
```

length 4.

---

27. WHY LPS HELPS

Suppose pattern partially matched:

```text
ABAB...
```

then mismatch occurs.

Instead of restarting pattern at index 0:

> **use previous prefix/suffix information.**

Because some suffix already equals a prefix.

So we reuse that knowledge.

PART O — BUILDING LPS ARRAY

Maintain:

```text
length
i
```

where `length` = length of current proper prefix-suffix.

Start:

```text
lps[0] = 0
length = 0
i = 1
```

---

28. IF CHARACTERS MATCH

If:

```text
pattern[i] == pattern[length]
```

then:

```text
length += 1
lps[i] = length
i += 1
```

---

29. IF MISMATCH AND length > 0

Do not immediately move `i`.

Fallback:

```text
length = lps[length - 1]
```

Then try again.

---

30. IF MISMATCH AND length == 0

Then:

```text
lps[i] = 0
i += 1
```

---

31. LPS CODE

```python
def build_lps(pattern):
    lps = [0] * len(pattern)
```

```
    length = 0
    i = 1

    while i < len(pattern):

        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1

        elif length > 0:
            length = lps[length - 1]

        else:
            lps[i] = 0
            i += 1

    return lps
```

Time:

\[\boxed{O(m)} \]

PART P — KMP SEARCH

Use:

```TEXT
 i  !'   t e x t   i n d e x
 j  !'   p a t t e r n   i n d e x
```

If characters match:

```TEXT
i++
j++
```

If:

\[j=m \]

pattern found.

---

32. MISMATCH

If mismatch and:

```TEXT
j > 0
```

do:

```TEXT
j = lps[j - 1]
```

Do **not** move text pointer backward.

That is key.

---

33. KMP CODE

```python
def kmp_search(text, pattern):
    if pattern == "":
        return 0

    lps = build_lps(pattern)

    i = 0
    j = 0

    while i < len(text):

        if text[i] == pattern[j]:
            i += 1
            j += 1

            if j == len(pattern):
                return i - j

        elif j > 0:
            j = lps[j - 1]

        else:
            i += 1

    return -1
```

Time:

$$ \boxed{O(n+m)} $$

Space:

$$ \boxed{O(m)} $$

---

34. KMP MEMORY TRICK

```text
Match:
advance both

Mismatch:
if j > 0
    jump j using LPS

else
    advance text
```

Most important idea:

$$ \boxed{\text{Never restart text scanning}} $$

PART Q — WHY KMP IS LINEAR

Although there are fallback steps, indices do not repeatedly restart from scratch.

Building LPS:

$$ O(m) $$

Searching:

$$ O(n) $$

Total:

$$ \boxed{O(n+m)} $$

PART R — KMP INTERVIEW SIGNAL

Use KMP when:

- exact substring search
- repeated prefix/suffix structure
- pattern matching needs guaranteed linear time
- find pattern occurrences
- string periodicity problems
- prefix-function/LPS-based problems

PART S — RABIN-KARP

Rabin-Karp uses hashing.

Instead of comparing full substring each time:

> **compare hash of pattern with hash of current window.**

Pattern:

$$ \boxed{\text{Sliding Window + Rolling Hash}} $$

---

35. BASIC IDEA

Text:

```text
abcdefghi
```

Pattern length:

$$ 3 $$

Windows:

```text
abc
bcd
cde
def
...
```

Instead of recomputing each window hash from scratch, update hash efficiently.

That's:

$$ \boxed{\text{Rolling Hash}} $$

PART T — HASH IDEA

Represent string numerically.

Conceptually:

$$ hash("abc") = a\cdot B^2+b\cdot B+c $$

where B is some base.

Usually calculations use modulo:

$$ \boxed{\text{mod prime}} $$

to keep numbers manageable.

---

36. ROLLING HASH

When window slides:

```text
 a b c  !′   b c d
```

we:

1. remove contribution of `a`,
2. shift remaining values,
3. add `d`.

This can update hash in:

$$ \boxed{O(1)} $$

per window.

---

37. RABIN-KARP COMPLEXITY

Average/expected:

$$ \boxed{O(n+m)} $$

But hashing can have collisions.

Worst case with repeated verification:

$$ \boxed{O(nm)} $$

38. HASH COLLISION

Two different strings can theoretically produce same hash.

Therefore:

> **equal hash does not prove strings are equal.**

If hashes match, often compare actual substring to confirm.

PART U — KMP VS RABIN-KARP

KMP	Rabin-Karp
Prefix-function based	Hash based
Deterministic linear search	Average linear
No collision issue	Collision possible
Great for one exact pattern	Useful for hashing/window patterns
$O(n+m)$ worst-case	Worst can degrade

PART V — KMP VS TRIE

These solve different problem families.

Trie

Use for:

> **many words/prefix queries.**

KMP

Use for:

> **one pattern searched efficiently inside text.**

Example:

```text
Dictionary autocomplete ! Trie

Find "abcab" inside huge text ! KMP
```

PART W — HASHING VS TRIE

Suppose storing many whole words and only exact membership is needed.

HashSet:

$$ \boxed{\text{excellent}} $$

Expected search:

$$ O(1) $$

But prefix query:

```text
Does any word start with "app"?
```

HashSet does not naturally support this.

Trie does.

Memory:

```text
 E x a c t   w o r d   l o o k u p  !'  H a s h S e t
 P r e f i x   l o o k u p          !'  T r i e
```

PART X — STRING PERIODICITY USING KMP

Interesting interview application.

Suppose:

```text
s = "ababab"
```

Can it be formed by repeating smaller substring?

Build LPS.

Let:

$$ n=|s| $$

and:

$$ L=lps[n-1] $$

Candidate repeating block length:

$$ n-L $$

If:

$$ L>0 $$

and:

$$ n \bmod (n-L)=0 $$

then string is periodic.

Example:

```text
ababab
```

$$ n=6 $$

LPS final:

$$ 4 $$

Block length:

$$ 6-4=2 $$

$$ 6\bmod2=0 $$

So:

```text
ab
```

repeats 3 times.

PART Y — FIND ALL PATTERN OCCURRENCES WITH KMP

Instead of stopping at first match:

After finding pattern:

```text
answer.append(i-j)
```

then set:

```text
j = lps[j - 1]
```

to continue looking for overlapping matches.

Example:

```text
Text = "aaaaa"
Pattern = "aaa"
```

matches at:

```text
0, 1, 2
```

KMP can find overlapping matches efficiently.

PART Z — Z-ALGORITHM BRIEF AWARENESS

Some interviews also use Z-function.

For string s:

$$ Z[i] $$

= length of longest substring starting at i matching prefix of s.

Pattern matching can combine:

```text
pattern + "#" + text
```

Then inspect Z values.

You do **not** need to prioritize Z over KMP unless your interview preparation specifically demands it.

For most placement preparation:

$$ \boxed{\text{Know KMP well first}} $$

PART AA — MANACHER'S ALGORITHM AWARENESS

There is an $O(n)$ algorithm for Longest Palindromic Substring called:

$$ \boxed{\text{Manacher's Algorithm}} $$

But for most entry-level placements:

> **Expand Around Center at $O(n^2)$ is usually sufficient unless interviewer explicitly demands better.**

Don't spend excessive time memorizing Manacher before mastering core patterns.

PART AB — ADVANCED STRING PATTERN MAP

Problem	Technique
Palindrome	Two Pointers
Anagram	Frequency array
Longest unique substring	Sliding Window
Parentheses	Stack
Many prefix queries	Trie
Autocomplete	Trie + DFS
Wildcard dictionary	Trie + DFS
Many words on board	Trie + Backtracking
Exact pattern matching	KMP

Problem	Technique
Sliding substring hash	Rabin-Karp
String periodicity	KMP/LPS
Longest palindrome substring	Expand Around Center

PART AC — COMMON TRIE INTERVIEW TRAPS

39. TRAP 1 — PREFIX PATH= COMPLETE WORD

False.

Need:

\[\boxed{\text{end-of-word marker}} \]

---

40. TRAP 2 — RETURN FALSE AFTER PREFIX EXISTS

For `startsWith`, once all prefix chars are traversed:

\[\boxed{True} \]

regardless of `end`.

---

41. TRAP 3 — WORD SEARCH II WITHOUT PRUNING

Trie exists specifically so invalid prefixes can stop immediately.

Use that.

---

42. TRAP 4 — HUGE FIXED CHILD ARRAY

If character set is huge/sparse:

> **HashMap children may save memory.**

PART AD — COMMON KMP TRAPS

43. TRAP 1 — LPS MEANING

LPS in KMP:

> **Longest Proper Prefix also Suffix.**

Not Longest Palindromic Subsequence.

44. TRAP 2 — MISMATCH RESTART AT ZERO

If:

```text
j > 0
```

don't reset `j=0`.

Use:

```text
j = lps[j-1]
```

45. TRAP 3 — MOVE TEXT BACKWARD

KMP does not move `i` backward.

That's the whole advantage.

46. TRAP 4 — LPS INCLUDES WHOLE STRING

No.

"Proper prefix" excludes the complete string itself.

47. TRAP 5 — EMPTY PATTERN

Define behavior explicitly.

Common convention:

```text
empty pattern found at index 0
```

PART AE — COMMON RABIN-KARP TRAPS

48. TRAP 1 — HASH EQUALITY = STRING EQUALITY

Not guaranteed.

Collision possible.

49. TRAP 2 — OVERFLOW

Use modular arithmetic carefully in fixed-width languages.

50. TRAP 3 — NEGATIVE MODULO

When removing old-character contribution, value may become negative depending language.

Normalize:

$$ (hash+MOD)\%MOD $$

PART AF — INTERVIEW EXPLANATION: TRIE

Suppose interviewer asks:

> **Design autocomplete for dictionary words.**

Strong answer:

> **"Because the main operation is prefix lookup across many words, I would use a Trie. Each node represents a prefix and stores links to possible next characters plus an end-of-word marker. To answer a prefix query, I traverse the prefix in $O(L)$, where L is the prefix length. From that prefix node, I can DFS through descendants to generate suggestions. Shared prefixes are stored once, which makes a Trie a natural structure for autocomplete."**

PART AG — INTERVIEW EXPLANATION: KMP

Suppose interviewer asks:

> **Find a pattern in text in guaranteed linear time.**

Strong answer:

> **"A naive approach may restart matching after every mismatch, which can take $O(nm)$. KMP preprocesses the pattern into an LPS array, where each position stores the longest proper prefix that is also a suffix. On a mismatch after a partial match, I use the LPS value to move the pattern pointer to the next possible matching prefix instead of moving the text pointer backward. Building LPS takes $O(m)$ and searching takes $O(n)$, so total time is $O(n+m)$."**

PART AH — MUST-DO TRIE + STRING QUESTIONS

Trie Foundation

1. Implement Trie
2. Insert Word
3. Search Word
4. Starts With Prefix
5. Count Prefixes

6. Autocomplete basics

Trie Core

7. Design Add and Search Words
8. Replace Words
9. Longest Word in Dictionary
10. Map Sum Pairs
11. Word Search II

Pattern Matching

12. Naive Pattern Matching
13. Build KMP LPS Array
14. KMP Search
15. Find All Occurrences
16. Repeated Substring Pattern
17. Rabin-Karp basics
18. Rolling Hash
19. Longest Duplicate Substring — advanced
20. Z-function awareness

PLACEMENT PRACTICE SET

Q1 — MCQ

Trie is most naturally designed for:

A. shortest weighted paths

B. prefix operations on strings

C. MST

D. matrix multiplication

Q2 — MCQ

Why does a Trie node need an end-of-word flag?

A. To store graph weight

B. Because an existing prefix path does not prove a complete word exists

C. To perform Heap Sort

D. To detect cycles

Q3 — MCQ

If stored word is only:

```text
apple
```

then `startsWith("app")` should return:

A. False B. True C. Error D. Depends on sorting

Q4 — MCQ

If stored word is only:

```text
apple
```

then `search("app")` should return:

A. True B. False C. Always 1 D. Heap root

Q5 — MSQ

Which are good Trie use cases?

A. Autocomplete

B. Prefix dictionary

C. Word Search II optimization

D. Dijkstra

Q6 — MCQ

Naive string pattern matching worst-case complexity is:

A. \(O(1)\)

B. \(O(n+m)\)

C. \(O(nm)\)

D. \(O(\log n)\)

Q7 — MCQ

KMP preprocesses:

A. graph indegree

B. LPS array

C. heap

D. DSU parent

Q8 — MCQ

In KMP, LPS means:

A. Longest Palindromic Subsequence

B. Longest Proper Prefix also Suffix

C. Least Possible String

D. Longest Path Search

Q9 — MCQ

For pattern:

```text
ABAB
```

length of longest proper prefix that is also suffix is:

A. 0 B. 1 C. 2 D. 4

Q10 — MCQ

If KMP gets mismatch while `j > 0`, it generally updates:

A. `i = 0`

B. `j = lps[j-1]`

C. sort the pattern

D. `j = pattern length`

Q11 — MSQ

Which statements about KMP are true?

A. Text pointer need not move backward.

B. LPS preprocessing is \(O(m)\).

C. Search can be \(O(n)\).

D. KMP relies on hash collisions.

Q12 — MCQ

Rabin-Karp primarily uses:

A. DSU

B. Rolling Hash

C. BFS

D. Heapify

Q13 — MCQ

A danger in Rabin-Karp is:

A. negative edges

B. hash collisions

C. tree imbalance

D. stack underflow only

Q14 — MCQ

For exact membership of many whole words with no prefix queries, which can be simpler than Trie?

A. HashSet

B. Bellman-Ford

C. Prim

D. Matrix Chain DP

Q15 — MSQ

Which mappings are correct?

A. Prefix lookup !’ Trie

B. Exact substring matching !’ KMP

C. Rolling-window string hash !’ Rabin-Karp

D. Unweighted shortest path !’ Trie

PRACTICE SOLUTIONS

Q1

\[\boxed{B} \]

Q2

Prefix existing does not mean a full word ended there.

\[\boxed{B} \]

Q3

`apple` begins with `app`.

\[\boxed{B} \]

Q4

`app` was not inserted as a complete word.

$$\boxed{B}$$

Q5

A, B, C.

$$\boxed{A,B,C}$$

Q6

$$\boxed{C}$$

Q7

$$\boxed{B}$$

Q8

$$\boxed{B}$$

Q9

For:

```
TEXT
ABAB
```

proper prefix/suffix:

```
TEXT
AB
```

length:

$$\boxed{2}$$

Answer:

$$\boxed{C}$$

Q10

$$\boxed{B}$$

Q11

A, B, C true.

D false.

$$\boxed{A,B,C}$$

Q12

$$\boxed{B}$$

Q13

$$\boxed{B}$$

Q14

$$\boxed{A}$$

Q15

A, B, C true.

D false.

$$\boxed{A,B,C}$$

Ø=ÑÓTES TO WRITE

```text
TRIE = Prefix Tree

Trie Node:
children
end_of_word

Insert:
follow/create characters
mark end

Search:
path must exist
AND end = True

Prefix:
path only needs to exist

Complexity:
Insert  O(L)
Search  O(L)
Prefix  O(L)
```

```
Use Trie:
many words
prefix queries
autocomplete
dictionary
Word Search II


KMP:
pattern matching
O(n + m)

LPS:
Longest Proper Prefix
which is also Suffix

KMP mismatch:
if j > 0:
    j = lps[j-1]
else:
    i++

Text pointer does NOT move backward.


RABIN-KARP:
Rolling Hash
average O(n+m)
collision possible
```

ADVANCED STRING MASTER MAP

TEXT

```
Palindrome
!’   T w o   P o i n t e r s

Anagram
!’   F r e q u e n c y

Longest unique substring
!’   S l i d i n g   W i n d o w

Parentheses
!’   S t a c k

Many prefix queries
!’   T r i e

Autocomplete
!’   T r i e   +   D F S

Wildcard dictionary
!’   T r i e   +   D F S

Dictionary + Grid
!’   T r i e   +   B a c k t r a c k i n g

Exact pattern search
!’   K M P

Rolling substring comparison
```

```
!’   R a b i n - K a r p

Repeated substring pattern
!’   K M P / L P S
```

&β30-SECOND REVISION

1. Trie = Prefix Tree.
2. Trie is best for many prefix queries.
3. Each node stores children + end marker.
4. Prefix path existing does not mean whole word exists.
5. Insert/Search/Prefix take \(O(L)\).
6. Autocomplete = Trie + DFS.
7. Wildcard dictionary = Trie + DFS.
8. Word Search II = Trie + Grid Backtracking.
9. Naive pattern search can be \(O(nm)\).
10. KMP = \(O(n+m)\).
11. KMP uses LPS.
12. KMP-LPS = Longest Proper Prefix also Suffix.
13. Match !’ advance text and pattern.
14. Mismatch with partial match !’ use `lps[j-1]`.
15. KMP never needs to move text pointer backward.
16. Rabin-Karp = Rolling Hash.
17. Hash collisions are possible.
18. Equal hashes do not always guarantee equal strings.
19. HashSet !’ exact words.
20. Trie !’ prefix words.
21. KMP !’ exact pattern in text.
22. Rabin-Karp !’ hash-based substring matching.

' Placement Module 15 Complete

Next Placement Module:

MODULE 16 — BIT MANIPULATION + MATH PATTERNS FOR CODING INTERVIEWS

Next we'll cover:

- binary representation,
- AND / OR / XOR / NOT,
- left and right shifts,
- check/set/clear/toggle bit,
- power of two,

- count set bits,
- Brian Kernighan trick,
- Single Number,
- XOR cancellation,
- missing number with XOR,
- subsets using bitmasks,
- GCD / Euclidean algorithm,
- prime checking,
- sieve,
- modular arithmetic,
- fast exponentiation,
- overflow traps,
- and the most useful math shortcuts for placements.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 16 — BIT MANIPULATION + MATH PATTERNS FOR CODING INTERVIEWS

Placement Priority: & & & & &

Bit manipulation looks difficult initially because binary numbers feel unfamiliar.

But for placements, most questions reduce to a few operations:

\[\boxed{\text{AND, OR, XOR, Shifts}} \]

and a few identities:

\[\boxed

USER

NEXT

ASSISTANT

PLACEMENT MODULE 16 — BIT MANIPULATION + MATH PATTERNS FOR CODING INTERVIEWS

Placement Priority: & & & & &

Bit manipulation looks difficult initially because binary numbers feel unfamiliar.

For placements, however, most questions reduce to a few operations:

\[\boxed{\text{AND, OR, XOR, NOT, Left Shift, Right Shift}} \]

and a few identities worth memorizing.

---

1. BINARY REPRESENTATION

Computers represent integers using bits:

```text
0
1
```

Example:

\[13_{10}=1101_2 \]

because:

\[13=8+4+1 \]

or:

\[13=2^3+2^2+2^0 \]

Bit positions:

```text
Binary:  1  1  0  1
Index:   3  2  1  0
```

---

2. POWERS OF TWO

Memorize:

Power	Value
\(2^0\)	1
\(2^1\)	2
\(2^2\)	4
\(2^3\)	8
\(2^4\)	16
\(2^5\)	32
\(2^6\)	64
\(2^7\)	128
\(2^8\)	256
\(2^{10}\)	1024

PART A — BITWISE AND

3. AND `&`

Truth table:

```text
0 & 0 = 0
0 & 1 = 0
1 & 0 = 0
1 & 1 = 1
```

Memory:

> **AND gives 1 only when both bits are 1.**

Example:

```text
5 = 101
3 = 011
--------
    001
```

Therefore:

\[5\&3=1 \]

4. IMPORTANT AND USE

AND is very useful for:

- checking a bit,
- clearing bits,
- testing powers of two,
- extracting masks.

PART B — BITWISE OR

5. OR `|`

```text
0 | 0 = 0
0 | 1 = 1
1 | 0 = 1
1 | 1 = 1
```

Example:

```text
5 = 101
3 = 011
--------
    111
```

\[5|3=7 \]

Memory:

> **OR gives 1 if at least one bit is 1.**

Common use:

\[\boxed{\text{Set a bit}} \]

PART C — XOR

---

6. XOR `^`

XOR gives 1 when bits are different.

```text
0 ^ 0 = 0
0 ^ 1 = 1
1 ^ 0 = 1
1 ^ 1 = 0
```

Example:

```text
5 = 101
3 = 011
--------
    110
```

\[5\oplus3=6 \]

---

7. XOR IDENTITIES — MUST MEMORIZE

\[\boxed{x\oplus x=0} \]

\[\boxed{x\oplus0=x} \]

\[\boxed{x\oplus y=y\oplus x} \]

XOR is associative too:

\[(a\oplus b)\oplus c = a\oplus(b\oplus c) \]

This allows duplicates to cancel.

PART D — NOT

8. BITWISE NOT `~`

It flips bits:

```text
 0  !’   1
 1  !’   0
```

But signed integers use two's complement, so in Python:

$$ \boxed{\sim x=-x-1} $$

Example:

```python
~5
```

returns:

$$ -6 $$

For placements, focus more on AND/OR/XOR/shifts unless NOT is specifically needed.

PART E — LEFT SHIFT

9. `x << k`

Shifts bits left by k.

For non-overflowing nonnegative integers:

$$ \boxed{x<<k=x\times2^k} $$

Example:

$$ 5=101 $$

Shift left once:

```text
1010
```

which is:

$$ 10 $$

So:

$$ 5<<1=10 $$

10. LEFT SHIFT SHORTCUT

$$ 1<<k=2^k $$

Example:

$$ 1<<4=16 $$

This is extremely useful when creating bit masks.

PART F — RIGHT SHIFT

11. `x >> k`

For nonnegative integers:

$$ \boxed{x>>k=\left\lfloor\frac{x}{2^k}\right\rfloor} $$

Example:

$$ 20=10100 $$

$$ 20>>2=00101=5 $$

PART G — CHECK THE i-th BIT

Suppose number:

```text
n = 13 = 1101
```

Want to know whether bit at position i is set.

Create mask:

$$ 1<<i $$

Then:

$$ \boxed{n\&(1<<i)} $$

If nonzero:

> **bit is set.**

---

12. EXAMPLE

Check bit 2 of 13.

```text
13      = 1101
1 << 2  = 0100
----------------
AND     = 0100
```

Nonzero.

Therefore bit 2 is:

$$ \boxed{1} $$

13. CODE

```python
def is_set(n, i):
    return (n & (1 << i)) != 0
```

Memory:

> **CHECK !' AND**

PART H — SET THE i-th BIT

Want to make bit i:

$$ 1 $$

Use:

$$ \boxed{n|(1<<i)} $$

Example:

```text
n       = 1001
mask    = 0100
OR      = 1101
```

Memory:

> **SET !' OR**

PART I — CLEAR THE i-th BIT

Make bit i:

$$ 0 $$

Use:

$$ \boxed{n\ \&\ \sim(1<<i)} $$

Memory:

> **CLEAR !' AND with inverted mask**

PART J — TOGGLE THE i-th BIT

Change:

```TEXT
 0  !'   1
 1  !'   0
```

Use:

\[\boxed{n\oplus(1<<i)} \]

Memory:

> **TOGGLE !' XOR**

---

14. FOUR OPERATIONS TO LOCK

```TEXT
 C h e c k    !'   n   &   ( 1   < <   i )

 S e t        !'   n   |   ( 1   < <   i )

 C l e a r    !'   n   &   ~ ( 1   < <   i )

 T o g g l e  !'   n   ^   ( 1   < <   i )
```

PART K — ODD OR EVEN USING BITS

15. OBSERVATION

Last binary bit tells parity.

Even numbers end with:

```TEXT
0
```

Odd numbers end with:

```TEXT
1
```

Therefore:

\[\boxed{n\&1} \]

If:

```TEXT
 0  !'   e v e n
 1  !'   o d d
```

Code:

```python
if n & 1:
    print("Odd")
else:
    print("Even")
```

PART L — POWER OF TWO

Extremely common interview question.

16. BINARY PROPERTY

Powers of two contain exactly one set bit.

Examples:

```text
1  = 0001
2  = 0010
4  = 0100
8  = 1000
16 = 10000
```

---

17. MAGIC IDENTITY

For power of two \(n\):

\[\boxed{n\&(n-1)=0} \]

Example:

\[8=1000 \]

\[7=0111 \]

```text
1000
0111
----
0000
```

Therefore:

```python
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0
```

---

18. WHY `n > 0`?

Because:

\[0\&(0-1) \]

may also appear zero under fixed-width representation logic.

But zero is not a power of two.

Always include:

$$ \boxed{n>0} $$

PART M — REMOVE LOWEST SET BIT

One of the most useful identities:

$$ \boxed{n\&(n-1)} $$

removes the rightmost set bit.

Example:

```text
n     = 101100
n-1   = 101011
AND   = 101000
```

The lowest `1` disappeared.

PART N — COUNT SET BITS

Problem:

> **How many 1s are in binary representation?**

Example:

$$ 13=1101 $$

Answer:

$$ \boxed{3} $$

---

19. SIMPLE APPROACH

Repeatedly inspect last bit:

```python
def count_bits(n):
    count = 0

    while n:
        count += n & 1
        n >>= 1

    return count
```

Time:

$$ O(\log n) $$

20. BRIAN KERNIGHAN'S ALGORITHM

Instead of removing one binary position per iteration:

> **remove one set bit per iteration.**

```python
def count_bits(n):
    count = 0

    while n:
        n = n & (n - 1)
        count += 1

    return count
```

If number contains k set bits:

$$ \boxed{O(k)} $$

Memory trick:

> **`n & (n-1)` kills one 1.**

PART O — SINGLE NUMBER

Extremely important XOR question.

21. PROBLEM

Every number occurs twice except one.

Example:

```text
[4,1,2,1,2]
```

Unique number:

$$ \boxed{4} $$

---

22. XOR SOLUTION

XOR everything:

$$ 4\oplus1\oplus2\oplus1\oplus2 $$

Duplicates cancel:

$$ 1\oplus1=0 $$

$$ 2\oplus2=0 $$

Remaining:

$$ \boxed{4} $$

23. CODE

```python
def single_number(nums):
    ans = 0

    for x in nums:
        ans ^= x

    return ans
```

Time:

$$ \boxed{O(n)} $$

Space:

$$ \boxed{O(1)} $$

24. XOR INTERVIEW SIGNAL

If problem says:

> **every element appears twice except one**

think immediately:

$$ \boxed{\text{XOR}} $$

PART P — MISSING NUMBER USING XOR

25. PROBLEM

Array contains distinct values from:

$$ 0...n $$

with one missing.

Example:

```text
[3,0,1]
```

Expected:

```text
0,1,2,3
```

Missing:

$$ \boxed{2} $$

26. XOR IDEA

XOR all expected numbers:

$$ 0\oplus1\oplus2\oplus3 $$

XOR array:

$$ 3\oplus0\oplus1 $$

Everything occurring twice cancels.

Remaining:

$$ \boxed{2} $$

27. CODE

```python
def missing_number(nums):
    ans = len(nums)

    for i in range(len(nums)):
        ans ^= i
        ans ^= nums[i]

    return ans
```

Time:

$$ O(n) $$

Space:

$$ O(1) $$

PART Q — FIND TWO UNIQUE NUMBERS

Advanced XOR pattern.

Suppose every number occurs twice except exactly two values.

Example:

```text
[1,2,1,3,2,5]
```

Unique:

```text
3,5
```

28. STEP 1 — XOR EVERYTHING

Duplicates vanish:

\[xor=3\oplus5 \]

This is nonzero because 3 and 5 differ somewhere.

29. STEP 2 — FIND DIFFERING BIT

Find rightmost set bit:

\[\boxed{mask=xor\&(-xor)} \]

This bit differs between the two unique numbers.

30. STEP 3 — SPLIT NUMBERS INTO TWO GROUPS

Use that bit.

One unique number goes to each group.

Duplicates remain in same group and cancel.

This is a useful medium interview technique.

PART R — SUBSETS USING BITMASK

31. WHY 2^n SUBSETS?

Each element has two states:

```text
 0  !'   e x c l u d e
 1  !'   i n c l u d e
```

For \(n\) elements, an \(n\)-bit number can represent one subset.

Example:

```text
nums = [A,B,C]
```

Mask:

```text
101
```

means:

```text
A included
B excluded
C included
```

subset:

```text
[A,C]
```

---

32. ALL MASKS

For \(n\) elements:

\[0\text{ to }2^n-1 \]

Code:

```python
def subsets(nums):
    ans = []
    n = len(nums)

    for mask in range(1 << n):
        current = []

        for i in range(n):
            if mask & (1 << i):
                current.append(nums[i])

        ans.append(current)

    return ans
```

Time:

\[\boxed{O(n2^n)} \]

including output construction.

PART S — BITMASKING INTERVIEW SIGNAL

Think bitmask when:

- \(n\) is small,
- each element has yes/no state,
- subsets need compact representation,
- visited states fit in bits,
- combinations of features/permissions.

PART T — SWAP USING XOR

Classic trick:

```text
a ^= b
b ^= a
a ^= b
```

swaps values.

But in real interviews/code:

```python
a, b = b, a
```

or temporary variable is clearer.

Know XOR-swap conceptually, but don't force it into production code.

PART U — GCD

Now math patterns.

33. GREATEST COMMON DIVISOR

$$ gcd(a,b) $$

= largest integer dividing both.

Example:

$$ gcd(48,18)=6 $$

---

34. EUCLIDEAN ALGORITHM

Key identity:

$$ \boxed{gcd(a,b)=gcd(b,a\bmod b)} $$

until:

$$ b=0 $$

Then answer is:

$$ a $$

---

35. TRACE

$$ gcd(48,18) $$

$$ 48\bmod18=12 $$

So:

$$ gcd(18,12) $$

Then:

$$ 18\bmod12=6 $$

$$ gcd(12,6) $$

Then:

$$ 12\bmod6=0 $$

Answer:

$$ \boxed{6} $$

---

36. CODE

```python
def gcd(a, b):
    while b:
        a, b = b, a % b

    return a
```

Time:

$$ \boxed{O(\log\min(a,b))} $$

PART V — LCM

Relationship:

$$ \boxed{ lcm(a,b)=\frac{|ab|}{gcd(a,b)} } $$

Safer fixed-width implementation:

$$ \boxed{ lcm=(a/gcd(a,b))\times b } $$

because multiplying first can overflow.

PART W — PRIME CHECKING

37. NAIVE

Try divisors:

$$ 2...n-1 $$

Time:

$$ O(n) $$

Unnecessary.

---

38. KEY OBSERVATION

If n has a factor larger than $\sqrt n$, the paired factor must be smaller than $\sqrt n$.

Therefore only test:

$$ 2...\sqrt n $$

Time:

$$ \boxed{O(\sqrt n)} $$

39. CODE

```python
def is_prime(n):
    if n < 2:
        return False

    i = 2

    while i * i <= n:
        if n % i == 0:
            return False

        i += 1

    return True
```

40. SLIGHT OPTIMIZATION

After checking 2, test only odd numbers.

```python
def is_prime(n):
    if n < 2:
        return False

    if n == 2:
        return True

    if n % 2 == 0:
        return False

    i = 3

    while i * i <= n:
        if n % i == 0:
            return False
        i += 2

    return True
```

PART X — SIEVE OF ERATOSTHENES

41. PROBLEM

Find all primes up to:

$$ N $$

Checking every number separately costs roughly:

$$ O(N\sqrt N) $$

Sieve is much better.

---

42. IDEA

Initially assume:

```text
2...N
```

are prime.

Start with 2.

Mark:

```text
4,6,8,10,...
```

composite.

Next unmarked:

```text
3
```

mark:

```text
6,9,12,...
```

Continue.

---

43. OPTIMIZATION

For prime p, start marking at:

$$ \boxed{p^2} $$

because smaller multiples were already marked by smaller primes.

---

44. CODE

```python
def sieve(n):
    prime = [True] * (n + 1)

    if n >= 0:
        prime[0] = False
    if n >= 1:
        prime[1] = False
```

```
    p = 2

    while p * p <= n:
        if prime[p]:
            for multiple in range(p * p, n + 1, p):
                prime[multiple] = False

        p += 1

    return prime
```

Time:

$$ \boxed{O(n\log\log n)} $$

Space:

$$ \boxed{O(n)} $$

PART Y — FAST EXPONENTIATION

Very important math pattern.

45. PROBLEM

Compute:

$$ a^n $$

Naively multiply a:

$$ n $$

times.

Time:

$$ O(n) $$

Can do:

$$ \boxed{O(\log n)} $$

---

46. BINARY EXPONENTIATION

If n is even:

$$ a^n=(a^{n/2})^2 $$

If n is odd:

$$ a^n=a\times a^{n-1} $$

---

47. ITERATIVE VERSION

```python
def power(a, n):
    result = 1

    while n > 0:
        if n & 1:
            result *= a

        a *= a
        n >>= 1

    return result
```

Time:

$$ \boxed{O(\log n)} $$

48. WHY?

Every iteration divides exponent roughly by 2:

$$ n\rightarrow n/2 $$

Therefore:

$$ O(\log n) $$

PART Z — MODULAR ARITHMETIC

Competitive coding and placements often ask results modulo:

$$ 10^9+7 $$

or another modulus.

Important identities:

$$ (a+b)\bmod M = ((a\bmod M)+(b\bmod M))\bmod M $$

$$ (a\times b)\bmod M = ((a\bmod M)(b\bmod M))\bmod M $$

49. MODULAR SUBTRACTION

To avoid negative result:

$$ \boxed{ (a-b+M)\bmod M } $$

when (a,b) are already reduced appropriately.

50. MODULAR EXPONENTIATION

Combine fast power with modulo:

```python
def mod_pow(a, n, mod):
    result = 1
    a %= mod

    while n > 0:
        if n & 1:
            result = (result * a) % mod

        a = (a * a) % mod
        n >>= 1

    return result
```

Time:

\[O(\log n) \]

PART AA — DIGIT EXTRACTION

Very common aptitude/coding technique.

Given number:

\[n \]

Last digit:

\[\boxed{n\bmod10} \]

Remove last digit:

\[\boxed{n//10} \]

---

51. REVERSE INTEGER

Example:

\[1234 \]

Process digits:

```text
4
3
2
1
```

Update:

\[rev=rev\times10+digit \]

---

52. CODE

```python
def reverse_number(n):
    rev = 0

    while n > 0:
        digit = n % 10
        rev = rev * 10 + digit
        n //= 10

    return rev
```

PART AB — PALINDROME NUMBER

Example:

$$ 121 $$

Reverse and compare.

But be aware of:

- negative numbers,
- trailing zeroes,
- integer overflow in fixed-width languages.

Alternative advanced solution reverses only half the digits.

PART AC — COUNT DIGITS

For positive integer:

Simple loop:

```python
count = 0

while n:
    count += 1
    n //= 10
```

Time:

$$ O(\log_{10}n) $$

Special case:

$$ n=0 $$

has:

$$ \boxed{1} $$

digit.

PART AD — TRAILING ZEROES IN FACTORIAL

Classic math interview question.

53. PROBLEM

How many trailing zeroes does:

$$ n! $$

have?

A zero comes from factor:

$$ 10=2\times5 $$

There are more 2s than 5s in factorial.

So count factors of 5.

---

54. FORMULA

$$ \boxed{ \left\lfloor\frac n5\right\rfloor+ \left\lfloor\frac n{25}\right\rfloor+ \left\lfloor\frac n{125}\right\rfloor+\cdots } $$

Example:

$$ 25! $$

$$ 25/5=5 $$

$$ 25/25=1 $$

Total:

$$ \boxed{6} $$

PART AE — HAPPY NUMBER PATTERN

Repeatedly replace number with sum of squares of digits.

Either:

- reach 1,
- or enter cycle.

This can be solved using:

HashSet

or

Floyd cycle detection

Notice the connection to Linked Lists:

$$ \boxed{\text{Repeated deterministic state !' cycle detection}} $$

PART AF — MAJORITY / FREQUENCY MATH REMINDER

If question says:

> **element appears more than $n/2$**

think:

$$ \boxed{\text{Boyer-Moore}} $$

not Bit Manipulation.

Not every numeric-looking problem is a bit problem.

PART AG — BIT VS HASHING

Example:

> **Every number appears twice except one.**

HashSet:

$$ O(n) $$

space.

XOR:

$$ O(1) $$

space.

Use XOR because duplication structure is special.

But if frequencies are arbitrary:

> **HashMap is more general.**

PART AH — BITMASK DP AWARENESS

Advanced problems with small n, such as:

```
TEXT
n <= 20
```

may represent selected elements using a mask:

$$ 0...2^n-1 $$

Then state can be:

$$ \boxed{dp[mask]} $$

Examples:

- assignment problems,
- traveling salesman variants,
- matching small sets.

Complexity may be:

$$ O(n2^n) $$

or:

\[O(n^22^n) \]

This is advanced but useful to recognize.

PART AI — COMMON BIT INTERVIEW TRAPS

55. TRAP 1 — OPERATOR PRECEDENCE

Write parentheses clearly:

```python
(n & (1 << i)) != 0
```

Don't rely on remembering precedence.

---

56. TRAP 2 — NEGATIVE NUMBERS

Right shifts and NOT can behave differently conceptually because of signed representation.

Unless problem needs negatives, first master nonnegative bit manipulation.

---

57. TRAP 3 — POWER OF TWO CHECK WITH ZERO

Always:

```python
n > 0 and (n & (n - 1)) == 0
```

---

58. TRAP 4 — XOR ONLY WORKS BECAUSE OF FREQUENCY STRUCTURE

For:

```text
every number twice except one
```

XOR works.

If numbers occur arbitrary counts:

> **simple XOR may not solve it.**

---

59. TRAP 5 — `2^n` IN CODE

In many programming languages:

```text
^
```

means XOR, not exponentiation.

For Python:

```python
2 ** n
```

means exponentiation.

And:

```python
1 << n
```

equals \(2^n\) for nonnegative integer \(n\).

PART AJ — COMMON MATH INTERVIEW TRAPS

60. OVERFLOW

In Java/C++:

```text
a * b
```

can overflow before division/modulo.

Strategies:

- cast to larger type,
- divide before multiply where mathematically valid,
- apply modulo carefully.

---

61. PRIME CHECK BOUND

Use:

```text
i * i <= n
```

or to avoid multiplication overflow:

```text
i <= n // i
```

---

62. INTEGER DIVISION

Remember:

```text
7 // 2 = 3
```

not 3.5.

Important in Binary Search and mathematical formulas.

63. MODULO WITH NEGATIVES

Languages differ in handling negative remainder.

Normalize if needed.

PART AK — INTERVIEW EXPLANATION: SINGLE NUMBER

A strong answer:

> **"A HashSet could solve this in \(O(n)\) time but needs \(O(n)\) extra space. Since every value occurs exactly twice except one, XOR is ideal because `x ^ x = 0` and `x ^ 0 = x`. XOR is associative and commutative, so all duplicate pairs cancel regardless of order, leaving only the unique value. Thus the solution takes \(O(n)\) time and \(O(1)\) extra space."**

PART AL — INTERVIEW EXPLANATION: POWER OF TWO

> **"A positive power of two has exactly one set bit. Subtracting one clears that bit and sets all lower bits, so the original number and `n-1` have no common set bits. Therefore `n & (n-1)` becomes zero. I also explicitly require `n > 0`."**

PART AM — BIT/MATH RECOGNITION MAP

Signal	Technique
Check bit	AND
Set bit	OR
Clear bit	AND + NOT
Toggle bit	XOR
Odd/even	n & 1
Power of two	n & (n-1)

Signal	Technique
Count set bits	Kernighan
Duplicate pairs + one unique	XOR
Missing 0..n	XOR / Sum
Generate subsets	Bitmask
GCD	Euclidean algorithm
Many primes up to N	Sieve
\(a^n\) quickly	Binary exponentiation
Trailing zeros in n!	Count factors of 5

PART AN — MUST-DO QUESTIONS

Bit Foundation

1. Check Odd/Even
2. Check ith Bit
3. Set ith Bit
4. Clear ith Bit
5. Toggle ith Bit
6. Count Set Bits
7. Power of Two
8. Power of Four
9. Number of 1 Bits

XOR Core

10. Single Number
11. Missing Number
12. Find Difference
13. Two Unique Numbers
14. XOR from 1 to N basics

Bitmask

15. Generate Subsets
16. Small-set state representation
17. Bitmask DP awareness

Math Core

18. GCD
19. LCM

20. Prime Check
21. Sieve of Eratosthenes
22. Reverse Integer
23. Palindrome Number
24. Count Digits
25. Trailing Zeroes
26. Fast Power
27. Modular Exponentiation
28. Happy Number

PLACEMENT PRACTICE SET

Q1 — MCQ

What is:

\[5\&3 \]

A. 1 B. 2 C. 6 D. 7

Q2 — MCQ

What is:

\[5\oplus3 \]

A. 1 B. 6 C. 7 D. 8

Q3 — MCQ

Which operation is commonly used to set bit \(i\)?

A. AND

B. OR

C. Division

D. Modulo

Q4 — MCQ

Which operation toggles a bit?

A. XOR

B. AND only

C. Multiplication

D. Right shift only

Q5 — NAT

How many set bits are present in:

\[13 \]

?

Q6 — MCQ

Which expression removes the lowest set bit?

A. `n | (n-1)`

B. `n & (n-1)`

C. `n ^ 1`

D. `n << 1`

Q7 — MCQ

A positive integer is a power of two if:

A. `(n & (n-1)) == 0`

B. `n % 2 == 0` only

C. `n & 1 == 1`

D. `n == 0`

Q8 — MSQ

Which XOR identities are correct?

A. \(x\oplus x=0\)

B. \(x\oplus0=x\)

C. XOR is commutative

D. \(x\oplus x=x\)

Q9 — NAT

For:

```text
[4,1,2,1,2]
```

what is the unique value?

Q10 — MCQ

For n elements, number of possible subsets is:

A. n

B. n^2

C. 2^n

D. $n!$

Q11 — MCQ

Euclidean GCD uses:

A. `gcd(a,b) = gcd(b, a % b)`

B. sorting

C. BFS

D. XOR only

Q12 — MCQ

Primality of one number can be checked in:

A. $O(\sqrt n)$

B. $O(n^2)$ necessarily

C. $O(2^n)$

D. $O(n!)$

Q13 — MCQ

Sieve of Eratosthenes is useful for:

A. all primes up to N

B. shortest paths

C. MST

D. LCS

Q14 — MCQ

Fast exponentiation computes a^n in:

A. $O(n)$ only

B. $O(\log n)$

C. $O(n^2)$

D. $O(2^n)$

Q15 — NAT

How many trailing zeroes are in:

$$ 25! $$

?

PRACTICE SOLUTIONS

Q1

```text
5 = 101
3 = 011
    ---
    001
```

$$ \boxed{1} $$

Answer: **A**

Q2

```text
101
011
---
110
```

$$ \boxed{6} $$

Answer: **B**

Q3

Set:

$$ n|(1<<i) $$

$$ \boxed{B} $$

Q4

Toggle:

$$ n\oplus(1<<i) $$

$$ \boxed{A} $$

Q5

$$ 13=1101 $$

contains:

$$ \boxed{3} $$

set bits.

Q6

$$ \boxed{B} $$

Q7

With $n>0$:

$$ \boxed{A} $$

Q8

A, B, C true.

D false.

$$ \boxed{A,B,C} $$

Q9

Duplicates cancel:

$$ \boxed{4} $$

Q10

$$ \boxed{2^n} $$

Answer: **C**

Q11

$$ \boxed{A} $$

Q12

$$ \boxed{A} $$

Q13

$$ \boxed{A} $$

Q14

$$ \boxed{B} $$

Q15

$$ \left\lfloor\frac{25}{5}\right\rfloor + \left\lfloor\frac{25}{25}\right\rfloor = 5+1 = \boxed{6} $$

Ø=ÑÓTES TO WRITE

```TEXT
BIT BASICS

CHECK:
n & (1 << i)

SET:
n | (1 << i)

CLEAR:
n & ~(1 << i)

TOGGLE:
n ^ (1 << i)

Odd:
n & 1

Power of Two:
n > 0 &&
(n & (n-1)) == 0

Remove lowest set bit:
n & (n-1)

Count set bits:
while n:
    n &= n-1


XOR:
x ^ x = 0
x ^ 0 = x

Duplicate pairs + one unique
!’   X O R


LEFT SHIFT:
x << k
"H   x   *   2 ^ k

RIGHT SHIFT:
x >> k
"H   x   /   2 ^ k


MATH:
```

```
GCD:
gcd(a,b)
= gcd(b,a%b)

LCM:
(a/gcd) * b

Prime check:
up to sqrt(n)

Many primes:
Sieve
O(n log log n)

Fast Power:
square base
halve exponent
O(log n)

Trailing zeroes n!:
n/5 + n/25 + n/125 + ...
```

&β30-SECOND REVISION

1. Binary uses 0 and 1.
2. AND checks common set bits.
3. OR is useful for setting bits.
4. XOR is useful for toggling/cancellation.
5. Check bit !' AND.
6. Set bit !' OR.
7. Clear bit !' AND with inverted mask.
8. Toggle !' XOR.
9. Odd/even !' `n & 1`.
10. `1 << i = 2^i`.
11. Power of two has one set bit.
12. Power of two !' `n > 0 && n&(n-1)==0`.
13. `n&(n-1)` removes lowest set bit.
14. Kernighan counts set bits efficiently.
15. Duplicate pairs + one unique !' XOR.
16. Subsets can be represented using bitmasks.
17. GCD !' Euclidean algorithm.
18. Prime check !' up to \(\sqrt n\).
19. Many primes !' Sieve.
20. Fast exponentiation !' \(O(\log n)\).
21. Trailing zeros in factorial !' count factors of 5.
22. Be careful with overflow and negative bit operations.

' Placement Module 16 Complete

Next Placement Module:

MODULE 17 — TWO POINTERS + SLIDING WINDOW + PREFIX SUM: INTERVIEW PATTERNMASTERY

We touched these techniques earlier, but this module will make them **placement-interview complete**, including:

- opposite-direction two pointers,
- same-direction pointers,
- fast/slow array pointers,
- pair sum and 3Sum patterns,
- fixed-size windows,
- variable-size windows,
- Longest Substring,
- Minimum Window Substring,
- Character Replacement,
- Find All Anagrams,
- Subarray Sum Equals K,
- Prefix Sum + HashMap,
- range-query techniques,
- difference arrays,
- and exactly how to decide **Two Pointers vs Sliding Window vs Prefix Sum + Hashing**.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 17 — TWO POINTERS + SLIDING WINDOW + PREFIX SUM: INTERVIEW PATTERNMASTERY

Placement Priority: & & & & &

These three patterns solve a huge percentage of array/string interview questions:

\[\boxed{\text{Two Pointers}} \]

\[\boxed{\text{Sliding Window}} \]

\[\boxed{\text{Prefix Sum + Hashing}} \]

The most important skill is not memorizing code.

It is deciding:

> **Which one should I use?**

1. MASTER DECISION

Use this first:

```text id="m17a1" Need pair/relationship between positions? !’ Two Pointers

Need answer over CONTIGUOUS window? !’ Sliding Window

Need many range sums? !’ Prefix Sum

Need count of subarrays with exact sum? !’ Prefix Sum + HashMap

```

PART A — TWO POINTERS

2. WHAT ARE TWO POINTERS?

Instead of using nested loops, maintain two indexes that move intelligently.

Typical forms:

Opposite direction

```text id="m17a2"
 l e f t  !’             !•   r i g h t
```

Same direction

```text id="m17a3" slow !’ fast !’

```
---

3. WHEN TO USE OPPOSITE-DIRECTION POINTERS

Common signals:

- sorted array pair sum
- palindrome
- reverse array/string
- 3Sum after sorting
- container problems

Memory:

\[
 \ b o x e d { \ t e x t { S o r t e d + P a i r !’ T w o P o i n t e r s } }
\]

PART B — TWO SUM IN SORTED ARRAY

4. PROBLEM

```text id="m17a4"
[1,2,4,6,10]
target = 8
```

Need pair:

\[2+6=8 \]

5. BRUTE FORCE

Try every pair:

\[O(n^2) \]

6. TWO-POINTER IDEA

Start:

```text id="m17a5" left = 0 right = n-1

```
Current sum:

\[
A[left]+A[right]
\]

If equal:

> found.

If sum too small:

> increase left.

If sum too large:

> decrease right.

---

7. WHY?

Because array is sorted.

If:

\[
sum<target
\]

moving right leftward only makes value smaller.

So we need a larger value:

\[
\boxed{left++}
\]

Likewise if:

\[
sum>target
\]

we need smaller value:
```
```

```
\[
\boxed{right--}
\]

---

# 8. CODE

```python id="m17a6"
def two_sum_sorted(nums, target):
    left = 0
    right = len(nums) - 1

    while left < right:
        total = nums[left] + nums[right]

        if total == target:
            return [left, right]

        if total < target:
            left += 1
        else:
            right -= 1

    return [-1, -1]
```
```

Time:

\[\boxed{O(n)} \]

Space:

\[\boxed{O(1)} \]

---

9. UNSORTED TWO SUM

For unsorted:

```text id="m17a7" [2,7,11,15]

```
target 9.

Best standard approach:

\[
\boxed{\text{HashMap}}
\]

Time expected:

\[
O(n)
\]

So:

```text id="m17a8"
 S o r t e d   T w o   S u m     !’   T w o   P o i n t e r s
 U n s o r t e d   T w o   S u m  !’   H a s h M a p
```
```
```

PART C — PALINDROME

10. PATTERN

```text id="m17a9" left !’ "racecar" !• right

```

```
Compare ends.

If different:

> not palindrome.

Move inward.

Time:

\[
O(n)
\]

Space:

\[
O(1)
\]

---

# PART D — REMOVE DUPLICATES FROM SORTED ARRAY

This is **same-direction two pointers**.

# 11. IDEA

Use:

```text id="m17a10"
read pointer
write pointer
```
```

`read` explores every element.

`write` marks where next unique element should go.

Example:

```text id="m17a11" [1,1,2,2,3]

```

```
Result front becomes:

```text id="m17a12"
[1,2,3,...]
```
```

12. GENERAL SAME-DIRECTION PATTERN

```text id="m17a13" fast/read: scans data

slow/write: tracks valid result boundary
```

```
Common questions:

- remove duplicates
- move zeroes
- remove element
- partition arrays

---

# PART E — 3SUM

# 13. PROBLEM

Find triplets summing to zero.

Example:

```text id="m17a14"
[-1,0,1,2,-1,-4]
```
```

Possible:

```text id="m17a15" [-1,-1,2] [-1,0,1]

```
---

14. STRATEGY

Step 1:

Sort.

```text id="m17a16"
[-4,-1,-1,0,1,2]
```
```

Step 2:

Fix one index \(i\).

Step 3:

Run Two Pointers on remaining section.

So:

\[ \boxed{\text{Sort + Fix One + Two Pointers}} \]

---

## 15. COMPLEXITY

Outer loop:

\[ O(n) \]

Inner two pointers:

\[ O(n) \]

Total:

\[ \boxed{O(n^2)} \]
```

Better than brute force:

\[O(n^3) \]

---

16. 3SUM TRAP — DUPLICATES

Need skip duplicate fixed values and duplicate pointer values.

Otherwise repeated triplets may appear.

PART F — CONTAINER WITH MOST WATER

17. PROBLEM IDEA

Given heights, choose two lines maximizing:

\[area= width\times\min(leftHeight,rightHeight) \]

---

18. GREEDY TWO-POINTER RULE

Start at both ends.

Area uses shorter line.

Move the shorter side inward.

Why?

Moving taller side decreases width while shorter height still limits area.

So only replacing shorter line gives chance to improve.

Pattern:

\[\boxed{\text{Two Pointers + Greedy reasoning}} \]

Time:

\[O(n) \]

PART G — SLIDING WINDOW

19. WHAT IS A WINDOW?

A window is a contiguous segment:

```text id="m17a17" [a b c d e f] % % % % % % window

```
Sliding Window avoids recomputing information from scratch when moving between nearby contiguous segments.

```
```

```
# 20. TWO TYPES

### Fixed-size window

Window length given:

\[
k
\]

### Variable-size window

Window expands/shrinks depending on condition.

---

# PART H — FIXED-SIZE WINDOW

# 21. PROBLEM

Find maximum sum of any subarray of size \(k=3\).

Input:

```text id="m17a18"
[2,1,5,1,3,2]
```
```

Windows:

```text id="m17a19" [2,1,5] !’ 8 [1,5,1] !’ 7 [5,1,3] !’ 9 [1,3,2] !’ 6

```
Answer:

\[
\boxed{9}
\]

---

22. BRUTE FORCE

Compute every window sum from scratch:

\[
O(nk)
\]

---

23. SLIDING IDEA

First window:

\[
2+1+5=8
\]

To move one step:

> subtract outgoing element
> add incoming element.

\[
```
```

```
newSum=oldSum-outgoing+incoming
\]

---

# 24. CODE

```python id="m17a20"
def max_sum_k(nums, k):
    window_sum = sum(nums[:k])
    best = window_sum

    for right in range(k, len(nums)):
        window_sum += nums[right]
        window_sum -= nums[right - k]

        best = max(best, window_sum)

    return best
```
```

Time:

\[\boxed{O(n)} \]

---

25. FIXED WINDOW SIGNAL

If question says:

> **subarray/substring of exactly K length**

think:

\[\boxed{\text{Fixed Sliding Window}} \]

PART I — VARIABLE WINDOW

26. MASTER IDEA

Maintain:

```text id="m17a21" left right

Expand `right`.

When window violates condition:

> move `left` until valid again.

Generic:

```python id="m17a22"
left = 0

for right in range(n):
    add(nums[right])
```
```

```
    while window_invalid:
        remove(nums[left])
        left += 1

    update_answer()
```

This pattern is extremely important.

PART J — LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS

You saw this earlier, but now focus on the pattern.

27. PROBLEM

```text id="m17a23" "abcabcbb"

```

```
Longest substring without repeated characters:

```text id="m17a24"
"abc"
```
```

Length:

\[\boxed{3} \]

---

28. WINDOW INVARIANT

Maintain:

\[\boxed{\text{all characters inside window are unique}} \]

When duplicate enters:

> **shrink from left until valid again.**

---

29. CODE

```python id="m17a25" def longest_unique(s): seen = set() left = 0 best = 0

for right in range(len(s)): while s[right] in seen: seen.remove(s[left]) left += 1

seen.add(s[right])

best = max(best, right - left + 1)

return best

```

```
Time:

\[
\boxed{O(n)}
```

```
\]

Why?

Each character enters and leaves set at most once.

---

# PART K — LAST-SEEN OPTIMIZATION

Instead of shrinking character by character, store:

```text id="m17a26"
 c h a r a c t e r  !’   l a t e s t   i n d e x
```
```

If duplicate:

\[left=\max(left,last[ch]+1) \]

Code:

```python id="m17a27" def longest_unique(s): last = {} left = 0 best = 0

for right, ch in enumerate(s): if ch in last: left = max(left, last[ch] + 1)

last[ch] = right best = max(best, right - left + 1)

return best

```
```

---

# 30. WHY `max`?

Suppose duplicate's old occurrence lies before current window.

We should never move `left` backward.

Hence:

\[
\boxed{left=\max(left,last[ch]+1)}
\]

---

# PART L — MINIMUM SIZE SUBARRAY SUM

# 31. PROBLEM

Positive numbers:

```text id="m17a28"
[2,3,1,2,4,3]
```
```

Target:

\[7 \]

Smallest contiguous subarray with sum "e7:

```text id="m17a29" [4,3]
```

```
length:

\[
\boxed{2}
\]

---

# 32. VARIABLE WINDOW

Expand right and increase sum.

While:

\[
sum\ge target
\]

record answer and shrink left.

---

# 33. CODE

```python id="m17a30"
def min_subarray_len(target, nums):
    left = 0
    total = 0
    best = float("inf")

    for right in range(len(nums)):
        total += nums[right]

        while total >= target:
            best = min(best, right - left + 1)

            total -= nums[left]
            left += 1

    return 0 if best == float("inf") else best
```
```

Time:

\[\boxed{O(n)} \]

---

34. CRITICAL CONDITION — POSITIVE NUMBERS

Why does normal sliding window work?

Because with positive numbers:

- expanding increases sum,
- shrinking decreases sum.

This gives monotonic behavior.

If negative numbers exist:

> **this simple approach may fail.**

Very important trap.

PART M — LONGEST REPEATING CHARACTER REPLACEMENT

Classic variable-window problem.

35. PROBLEM

```text id="m17a31" s = "AABABBA" k = 1

```
You can replace at most one character.

Longest substring that can become all same character:

\[
\boxed{4}
\]

Example:

```text id="m17a32"
AABA
```

replace B with A.

36. KEY CONDITION

Window length:

\[R-L+1 \]

Most frequent character count:

\[maxFreq \]

Characters needing replacement:

\[\boxed{ windowLength-maxFreq } \]

Window valid if:

\[windowLength-maxFreq\le k \]

37. CORE WINDOW RULE

```text id="m17a33" if replacements needed > k: shrink

```
This is a powerful example of designing a window invariant.

PART N — FIND ALL ANAGRAMS

38. PROBLEM

```text id="m17a34"
```

```
s = "cbaebabacd"
p = "abc"
```

Find starting indices where substring is an anagram of `abc`.

Answer:

```text id="m17a35" [0,6]

---

# 39. PATTERN

Anagram length is fixed:

\[
|p|
\]

So use:

\[
\boxed{\text{Fixed Window + Frequency}}
\]

Maintain window frequency and compare to target frequency.

Time can be:

\[
O(n)
\]

for fixed alphabet.

---

# PART O — PERMUTATION IN STRING

Same pattern.

Question:

> Does string `s2` contain any permutation of `s1`?

Use window length:

\[
|s1|
\]

and compare frequency counts.

Pattern:

\[
\boxed{\text{Fixed Window + Frequency}}
\]

---

# PART P — MINIMUM WINDOW SUBSTRING

This is one of the most important advanced sliding-window problems.
```

```
# 40. PROBLEM

```text id="m17a36"
s = "ADOBECODEBANC"
t = "ABC"
```
```

Smallest substring containing all characters of `t`:

```text id="m17a37" "BANC"

```
---

41. MAIN IDEA

Maintain target frequency:

```text id="m17a38"
need
```
```

Window frequency:

```text id="m17a39" have

```
Expand until window contains all required characters.

Then shrink left as much as possible while remaining valid.

Pattern:

```text id="m17a40"
Expand until valid
!’   s h r i n k   w h i l e   v a l i d
!’   r e c o r d   m i n i m u m
```
```

This is opposite of longest-valid-window style.

---

42. LONGEST VS MINIMUM WINDOW

Longest valid window

```text id="m17a41" Expand if invalid !’ shrink record longest

```
Minimum covering window

```text id="m17a42"
Expand until valid
while valid:
    record minimum
    shrink
```
```

Important conceptual distinction.

---
```

PART Q — WHEN SLIDING WINDOW DOES NOT WORK

43. EXAMPLE

Find subarray sum exactly \(K\) when numbers can be:

```text id="m17a43" positive zero negative

```
Simple sliding window fails.

Why?

With negatives:

- adding a number may decrease sum,
- removing a number may increase sum.

No monotonic behavior.

Instead use:

\[
\boxed{\text{Prefix Sum + HashMap}}
\]

PART R — PREFIX SUM

44. BASIC IDEA

Input:

```text id="m17a44"
[2,4,1,3]
```

Build:

```text id="m17a45" prefix = [2,6,7,10]

```
where:

\[
prefix[i]
=
A[0]+A[1]+\cdots+A[i]
\]

---

45. RANGE SUM

Want sum from \(L\) to \(R\).

If \(L>0\):

\[
\boxed{
sum(L,R)=prefix[R]-prefix[L-1]
}
\]
```
```

```
If \(L=0\):

\[
sum=prefix[R]
\]

---

# 46. CLEANER PREFIX FORMAT

Often create:

```text id="m17a46"
prefix[0] = 0
```
```

and:

\[prefix[i+1]=prefix[i]+A[i] \]

Then range:

\[\boxed{ sum(L,R)=prefix[R+1]-prefix[L] } \]

This avoids special case for \(L=0\).

---

47. CODE

```python id="m17a47" def build_prefix(nums): prefix = [0] * (len(nums) + 1)

for i in range(len(nums)): prefix[i + 1] = prefix[i] + nums[i]

return prefix

```
Range query:

```python id="m17a48"
def range_sum(prefix, left, right):
    return prefix[right + 1] - prefix[left]
```
```

Preprocessing:

\[ O(n) \]

Each query:

\[ \boxed{O(1)} \]

---

## PART S — SUBARRAY SUM EQUALS K

Extremely important placement problem.

## 48. PROBLEM

Input:

```text id="m17a49" [1,1,1]

```
\[
k=2
\]

Subarrays:

```text id="m17a50"
[1,1] positions 0-1
[1,1] positions 1-2
```
```

Answer:

\[\boxed{2} \]

---

49. BRUTE FORCE

Try every subarray.

With incremental sum:

\[O(n^2) \]

Can we do:

\[O(n) \]

expected?

Yes.

---

50. PREFIX SUM EQUATION

Suppose current prefix sum is:

\[P \]

We need a previous prefix sum \(Q\) such that subarray between them sums to \(k\):

\[P-Q=k \]

Therefore:

\[\boxed{Q=P-k} \]

So while scanning:

> **count how many times `prefix - k` has appeared earlier.**

---

51. HASHMAP

Store:

```text id="m17a51" prefix_sum !' frequency
```

```
Initialize:

```text id="m17a52"
 0  !′   1
```
```

Why?

So a subarray starting at index 0 can be counted.

---

52. CODE

```python id="m17a53" def subarray_sum(nums, k): freq = {0: 1}

prefix = 0 count = 0

for x in nums: prefix += x

count += freq.get(prefix - k, 0)

freq[prefix] = freq.get(prefix, 0) + 1

return count

```
Time:

\[
\boxed{O(n)}
\]

expected.

Space:

\[
O(n)
\]

---

53. MOST IMPORTANT PREFIX HASH IDENTITY

\[
\boxed{
currentPrefix - previousPrefix = target
}
\]

Rearrange:

\[
\boxed{
previousPrefix = currentPrefix-target
}
\]

This single equation solves many subarray-counting problems.

PART T — WHY `freq = {0:1}`?
```
```

```
Example:

```text id="m17a54"
nums = [3]
k = 3
```
```

Current prefix:

\[3 \]

Need:

\[3-3=0 \]

If prefix 0 already has frequency 1:

> **we correctly count subarray `[3]`.**

So virtual prefix before array is:

\[\boxed{0} \]

PART U — LONGEST SUBARRAY SUM K

Similar equation, different data stored.

Instead of prefix frequency, store:

```text id="m17a55" prefix_sum !’ earliest index

```
Why earliest?

To maximize:

\[
currentIndex-earliestIndex
\]

Pattern:

```text id="m17a56"
 C o u n t   s u b a r r a y s  !’  p r e f i x  f r e q u e n c y
 L o n g e s t  s u b a r r a y  !’  e a r l i e s t  p r e f i x  i n d e x
```
```

Important distinction.

---

## PART V — EQUAL 0s AND 1s

Classic Prefix Sum transformation.

## 54. PROBLEM

Find longest subarray with equal number of:

```text id="m17a57" 0s and 1s

```
Transform:

```text id="m17a58"
 0  !′   - 1
 1  !′   + 1
```
```

Then equal 0s and 1s means transformed sum:

$$ 0 $$

Now solve:

> **longest zero-sum subarray.**

Use:

$$ \boxed{\text{Prefix Sum + first occurrence map}} $$

PART W — SUBARRAY SUM DIVISIBLE BY K

55. KEY IDEA

Two prefix sums with same remainder mod k have difference divisible by k.

If:

$$ P_i\bmod k=P_j\bmod k $$

then:

$$ (P_i-P_j)\bmod k=0 $$

Therefore track remainder frequencies.

Pattern:

$$ \boxed{\text{Prefix Remainder + HashMap}} $$

PART X — PREFIX XOR

Prefix idea works beyond sums.

Define:

$$ prefixXor[i] = A[0]\oplus\cdots\oplus A[i] $$

Then XOR of range can be obtained similarly because:

$$ x\oplus x=0 $$

For prefix format:

$$ \boxed{ rangeXor(L,R) = prefixXor[R]\oplus prefixXor[L-1] } $$

Useful in XOR subarray problems.

PART Y — 2D PREFIX SUM

56. PROBLEM FAMILY

Many sum queries over rectangular submatrices.

Build 2D prefix:

\[prefix[r][c] \]

Then rectangle can be answered using inclusion-exclusion.

Formula conceptually:

\[\boxed{ rectangle = whole -top -left +overlap } \]

This is worth recognizing even if less frequent than 1D prefix sum.

PART Z — DIFFERENCE ARRAY

Useful for many range updates.

57. PROBLEM

Suppose repeatedly asked:

> **Add value \(x\) to every index from \(L\) to \(R\).**

Doing each range directly:

\[O(n) \]

per update.

Difference array makes each update:

\[O(1) \]

---

58. RANGE UPDATE

For:

\[[L,R] += x \]

do:

```text id="m17a59" diff[L] += x diff[R+1] -= x

if `R+1` exists.

After all updates:

> prefix sum of `diff` gives final values.

---
```

```
# 59. WHY?

Adding at \(L\) starts effect.

Subtracting after \(R\) ends effect.

Pattern:

\[
 \ b o x e d { \ t e x t { M a n y  r a n g e  u p d a t e s  !'  D i f f e r e n c e  A r r a y } }
\]

---

# PART AA — PREFIX SUM VS DIFFERENCE ARRAY

### Prefix Sum

Optimizes:

> many range queries.

### Difference Array

Optimizes:

> many range updates.

Memory:

```text id="m17a60"
 Q u e r i e s  !'  P r e f i x
 U p d a t e s  !'  D i f f e r e n c e
```
```

PART AB — TWO POINTERS VS SLIDING WINDOW

This is a common confusion.

60. TWO POINTERS

Often pointers have different roles:

- opposite ends,
- read/write,
- sorted relationship.

Example:

```text id="m17a61" Two Sum sorted Palindrome Remove duplicates

```
---

61. SLIDING WINDOW

Pointers define a:

\[
\boxed{\text{contiguous active range}}
\]
```
```

```
and maintain information about that range.

Example:

```text id="m17a62"
Longest unique substring
 M i n i m u m   s u b a r r a y   s u m  "e   t a r g e t
```
```

Sliding Window is technically a two-pointer technique, but conceptually you should recognize it separately.

PART AC — SLIDING WINDOW VS PREFIX HASHING

This decision is extremely important.

62. QUESTION

> **Find subarray sum \(K\).**

If all values are positive:

Sliding window may work for some formulations.

If negatives can exist:

Use:

\[\boxed{\text{Prefix Sum + HashMap}} \]

because window sum no longer changes monotonically.

---

63. EXAMPLE

```text id="m17a63" [5, -4, 5]

```
Target:

\[
6
\]

Expanding can make sum go down.

Shrinking can make sum go up.

Therefore simple:

```text id="m17a64"
 s u m   t o o   l a r g e  !’   s h r i n k
 s u m   t o o   s m a l l  !’   e x p a n d
```
```

is invalid.

---
```

PART AD — VARIABLE WINDOW INVARIANT

The biggest Sliding Window skill is defining:

\[\boxed{\text{What makes my current window valid?}} \]

Examples:

Longest unique substring

Valid:

```text id="m17a65" no duplicate characters

### Character Replacement

Valid:

\[
windowSize-maxFreq\le k
\]

### Minimum Window Substring

Valid:

```text id="m17a66"
contains all required counts
```

Once you define validity, pointer movement becomes much easier.

PART AE — COMMON WINDOW TEMPLATES

64. LONGEST VALID WINDOW

```python id="m17a67" left = 0 best = 0

for right in range(n): add_right()

while invalid(): remove_left() left += 1

best = max(best, right - left + 1)

```
---

65. MINIMUM VALID WINDOW

```python id="m17a68"
left = 0
best = float("inf")

for right in range(n):
    add_right()

    while valid():
        best = min(best, right - left + 1)

        remove_left()
        left += 1
```
```

Memorize the **idea**, not exact function names.

PART AF — COMMON INTERVIEW TRAPS

66. TRAP 1 — TWO POINTERS ON UNSORTED PAIR SUM

For arbitrary unsorted Two Sum, moving pointers based on sum has no valid direction.

Sort first if indices do not matter, or use HashMap.

---

67. TRAP 2 — SORTING LOSES ORIGINAL INDICES

If problem asks original indices and you sort:

> **you may need to carry indices with values.**

Often HashMap is simpler.

---

68. TRAP 3 — SUBARRAY VS SUBSEQUENCE

Sliding Window works on:

\[\boxed{\text{contiguous}} \]

ranges.

That's:

- substring
- subarray

Not arbitrary subsequences.

---

69. TRAP 4 — NEGATIVE VALUES

Simple sum-based variable window often relies on positive/nonnegative numbers.

Always inspect constraints.

---

70. TRAP 5 — OFF-BY-ONE WINDOW LENGTH

For inclusive:

```text id="m17a69" left ... right

```

```
length:

\[
\boxed{right-left+1}
\]
```

```
---

# 71. TRAP 6 — UPDATE ANSWER AT WRONG TIME

For longest valid window:

> restore validity first, then update answer.

For minimum covering window:

> once valid, update while shrinking.

---

# 72. TRAP 7 — REMOVE FREQUENCY INCORRECTLY

When shrinking window:

```text id="m17a70"
freq[s[left]] -= 1
```
```

before incrementing left.

Order matters.

---

73. TRAP 8 — PREFIX MAP UPDATE ORDER

For Subarray Sum = K:

Do:

```text id="m17a71" count prefix-k first then insert current prefix

```
Otherwise for \(k=0\), you can accidentally count empty/current-state artifacts incorrectly.

---

74. TRAP 9 — PREFIX FREQUENCY VS SET

If counting number of subarrays, a set is insufficient.

Need:

\[
\boxed{\text{frequency}}
\]

because same prefix value may have occurred multiple times.

---

75. TRAP 10 — FIXED WINDOW OF INVALID K

Check:

```text id="m17a72"
k <= n
```
```

and problem-specific behavior for:
```

```text id="m17a73" k = 0

---

# PART AG — INTERVIEW EXPLANATION: SUBARRAY SUM K

A strong answer:

> “Because the array may contain negative values, a normal sliding window is not reliable. I use prefix sums instead. If the current prefix sum is `P`, then a previous prefix sum `P-k` means the elements between those two prefixes sum exactly to `k`. I keep a HashMap storing how many times each prefix sum has appeared. For every current prefix, I add the frequency of `prefix-k` to the answer, then record the current prefix. This gives expected \(O(n)\) time and \(O(n)\) space.”

---

# PART AH — INTERVIEW EXPLANATION: LONGEST UNIQUE SUBSTRING

> “I maintain a sliding window with the invariant that every character inside it is unique. The right pointer expands the window. If a duplicate enters, I move the left pointer and remove characters until the window becomes valid again. Since each character enters and leaves the window at most once, the total complexity is \(O(n)\), not \(O(n^2)\).”

---

# PART AI — MASTER PATTERN MAP

| Signal | Pattern |
|---|---|
| Sorted pair sum | Two Pointers |
| Palindrome | Opposite pointers |
| Remove duplicates | Read/Write pointers |
| 3Sum | Sort + Two Pointers |
| Exact K-length contiguous | Fixed Window |
| Longest valid substring | Variable Window |
| Minimum valid covering substring | Variable Window |
| Character counts in window | Window + HashMap/Frequency |
| Many range sum queries | Prefix Sum |
| Count subarrays sum K | Prefix + HashMap |
| Longest subarray sum K | Prefix + earliest index |
| Equal 0/1 longest subarray | Transform + Prefix Map |
| Divisible subarray count | Prefix remainder map |
| Many range updates | Difference Array |

---

# PART AJ — MUST-DO QUESTIONS

## Two Pointers

1. Two Sum II
2. Valid Palindrome
3. Reverse String
4. Remove Duplicates Sorted Array
5. Move Zeroes
6. 3Sum
7. Container With Most Water
8. Trapping Rain Water — advanced two-pointer version

## Fixed Sliding Window

9. Maximum Sum Subarray Size K
10. Find All Anagrams
11. Permutation in String
12. Maximum Average Subarray
```

Variable Sliding Window

13. Longest Substring Without Repeating
14. Minimum Size Subarray Sum
15. Longest Repeating Character Replacement
16. Minimum Window Substring
17. Fruit Into Baskets
18. Max Consecutive Ones III
19. Longest Subarray with At Most K Distinct

Prefix Sum + Hashing

20. Subarray Sum Equals K
21. Longest Subarray Sum K
22. Contiguous Array
23. Subarray Sums Divisible by K
24. Count Number of Nice Subarrays
25. Binary Subarrays With Sum

Range Techniques

26. Range Sum Query
27. 2D Range Sum
28. Difference Array
29. Range Addition
30. Corporate Flight Bookings-style updates

PLACEMENT PRACTICE SET

Q1 — MCQ

For pair sum in a sorted array, best common pattern is:

A. Two Pointers
B. DFS
C. DSU
D. Trie

Q2 — MCQ

A fixed-size contiguous segment problem strongly suggests:

A. Sliding Window
B. MST
C. Backtracking
D. Heapify

Q3 — MCQ

For longest substring without repeating characters, window invariant is:

A. window is sorted

B. all characters are unique

C. sum is always zero

D. first character is largest

```
---

## Q4 — NAT

For:

```text id="m17a74"
[2,1,5,1,3,2]
```
```

and:

\[k=3 \]

what is maximum window sum?

Q5 — MCQ

For:

```text id="m17a75" "abcabcbb"

```
```
longest substring without repeating characters has length:

A. 2
B. 3
C. 4
D. 8

---

## Q6 — MSQ

Which are typical Sliding Window problems?

A. Longest unique substring

B. Minimum size contiguous subarray

C. Find all anagrams

D. Kruskal MST

---

## Q7 — MCQ

A basic sum Sliding Window can fail when array contains negative values because:

A. numbers cannot be stored

B. expanding/shrinking no longer changes sum monotonically

C. HashMaps cannot store negatives

D. arrays become unsorted

---

## Q8 — MCQ

To count subarrays with sum K for arbitrary integers, commonly use:
```

```
A. Prefix Sum + HashMap

B. Heap

C. Trie

D. DSU

---

## Q9 — MCQ

In Subarray Sum = K, if current prefix is \(P\), we search previous prefix:

A. \(P+K\)

B. \(P-K\)

C. \(K-P\)

D. \(P/K\)

---

## Q10 — MCQ

Why initialize prefix frequency with:

```text id="m17a76"
 0  !’   1
```
```

?

A. To count subarrays starting from index 0

B. To sort the array

C. To avoid hashing

D. To detect cycles

Q11 — MSQ

Which statements are correct?

A. Prefix sums make range-sum queries \(O(1)\) after preprocessing.

B. Difference arrays are useful for many range updates.

C. Sliding Window always works for arbitrary negative numbers.

D. 3Sum commonly uses sorting + Two Pointers.

Q12 — NAT

For:

```text id="m17a77" [1,1,1]
```

```
and:

\[
k=2
\]

how many subarrays sum to 2?

---

## Q13 — MCQ

For longest subarray sum K, prefix HashMap usually stores:

A. latest index only always

B. earliest index

C. heap priority

D. edge count

---

## Q14 — MCQ

To find equal 0s and 1s in a subarray, useful transformation is:

 A .   0  !’   - 1 ,   1  !’   + 1

 B .   0  !’   1 0 ,   1  !’   2 0

C. sort first

D. multiply by zero

---

## Q15 — MCQ

For an inclusive window `[left, right]`, its length is:

A. `right-left`

B. `right-left+1`

C. `left+right`

D. `right+1`

---

# PRACTICE SOLUTIONS

## Q1

\[
\boxed{A}
\]

---

## Q2

\[
```

```
\boxed{A}
\]

---

## Q3

\[
\boxed{B}
\]

---

## Q4

Windows:

```text id="m17a78"
2+1+5 = 8
1+5+1 = 7
5+1+3 = 9
1+3+2 = 6
```
```

Answer:

\[\boxed{9} \]

Q5

`abc` is one longest valid substring.

\[\boxed{3} \]

Answer:

\[\boxed{B} \]

Q6

A, B, C.

\[\boxed{A,B,C} \]

Q7

\[\boxed{B} \]

Q8

\[\boxed{A} \]

Q9

From:

\[P-Q=K \]

we get:

\[Q=P-K \]

Answer:

\[\boxed{B} \]

Q10

\[\boxed{A} \]

Q11

A, B, D true.

C false.

\[\boxed{A,B,D} \]

Q12

Subarrays:

```text id="m17a79" indices 0..1 indices 1..2

```
\[
\boxed{2}
\]

Q13

To maximize distance, preserve first/earliest occurrence.

\[
\boxed{B}
\]

Q14

Then equal zeroes and ones produce transformed sum 0.

\[
\boxed{A}
\]

Q15

\[
\boxed{B}
\]
```
```

Ø=ÜÝ N O T E S T O W R I T E

```text id="m17a80"
TWO POINTERS

Sorted pair
!’ o p p o s i t e p o i n t e r s

Palindrome
!’ o p p o s i t e p o i n t e r s

Remove duplicates
!’ r e a d / w r i t e

3Sum
!’ s o r t + f i x o n e + t w o p o i n t e r s


SLIDING WINDOW

Fixed K:
add incoming
remove outgoing

Variable:
expand right
shrink left based on validity

Longest valid:
w h i l e i n v a l i d !’ s h r i n k
then update maximum

Minimum covering:
o n c e v a l i d !’ u p d a t e m i n i m u m + s h r i n k


PREFIX SUM

prefix[0] = 0

prefix[i+1]
= prefix[i] + nums[i]

Range L..R:
prefix[R+1] - prefix[L]


SUBARRAY SUM K

currentPrefix - oldPrefix = K

oldPrefix
= currentPrefix - K

HashMap:
p r e f i x !’ f r e q u e n c y

Initialize:
0 !’ 1


LONGEST SUM K:
p r e f i x !’ e a r l i e s t i n d e x
```

```
DIFFERENCE ARRAY:
range [L,R] += x

diff[L] += x
diff[R+1] -= x
```

&β30-SECOND REVISION

1. Sorted pair !’ Two Pointers.
2. Palindrome !’ left/right.
3. Read/write pointers !’ in-place filtering.
4. 3Sum !’ Sort + Two Pointers.
5. Contiguous fixed K !’ Fixed Sliding Window.
6. Variable contiguous condition !’ Variable Window.
7. Define the window invariant first.
8. Longest valid window !’ shrink when invalid.
9. Minimum valid window !’ shrink while valid.
10. Negative numbers can break simple sum Sliding Window.
11. Many range queries !’ Prefix Sum.
12. Range sum = difference of prefixes.
13. Subarray Sum K !’ Prefix Sum + HashMap.
14. Equation: previous prefix = current prefix " K.
15. Initialize prefix frequency `{0:1}`.
16. Counting !’ prefix frequency.
17. Longest !’ earliest prefix index.
18. Equal 0/1 !’ transform 0 to -1.
19. Same prefix remainder !’ difference divisible by K.
20. Many range updates !’ Difference Array.

' Placement Module 17 Complete

Next Placement Module:

MODULE 18 — SORTING + CUSTOM COMPARATORS+ INTERVIEW SORTING PATTERNS

Next we'll cover:

- when sorting itself is the solution,
- custom comparator thinking,
- sorting pairs/objects,
- interval sorting,
- largest number problem,

- relative sorting,
- frequency sorting,
- Dutch National Flag revision,
- counting/radix sort placement relevance,
- Quickselect,
- kth-element selection,
- inversion counting,
- merge-sort based counting problems,
- and how to decide **Sorting vs Heap vs Hashing vs Quickselect**.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 18 — SORTING + CUSTOM COMPARATORS+ INTERVIEW SORTING PATTERNS

Placement Priority: & & & & &

Sorting is often not the final answer—it is the step that makes a better pattern possible.

Very common transformation:

\[\boxed{\text{Unstructured input !' Sort !' Easy scan / Two Pointers / Greedy}} \]

The main interview skill is recognizing:

> **Would sorting make the structure useful enough to simplify the problem?**

---

1. WHEN SHOULD YOU THINK SORTING?

Typical signals:

- intervals
- duplicates
- pair/triplet problems
- kth element
- arranging numbers/strings
- grouping equal values
- custom priority
- merging
- inversion counting
- greedy scheduling

Common pattern:

```text id="sort1801" Sort !’ exploit order !’ scan once

```
---

2. BASIC SORTING COMPLEXITIES

Algorithm	Best	Average	Worst	Stable?
Bubble	\(O(n)\)*	\(O(n^2)\)	\(O(n^2)\)	Yes
Selection	\(O(n^2)\)	\(O(n^2)\)	\(O(n^2)\)	No
Insertion	\(O(n)\)	\(O(n^2)\)	\(O(n^2)\)	Yes
Merge Sort	\(O(n\log n)\)	\(O(n\log n)\)	\(O(n\log n)\)	Yes
Quick Sort	\(O(n\log n)\)	\(O(n\log n)\)	\(O(n^2)\)	Usually No
Heap Sort	\(O(n\log n)\)	\(O(n\log n)\)	\(O(n\log n)\)	No

*Bubble Sort best \(O(n)\) only with early-exit optimization.

---

3. WHAT MATTERS MOST IN PLACEMENTS?

You usually do **not** need to manually implement Bubble/Selection Sort in interview problems.

More important:

- know complexities,
- know when sorting helps,
- know custom sorting,
- know comparator logic,
- know Quickselect,
- know merge-sort counting techniques.

PART A — SORTING PAIRS / OBJECTS

4. EXAMPLE

Suppose:

```text id="sort1802"
[(3, 8), (1, 5), (2, 4)]
```
```

Sort by first value:

```text id="sort1803" [(1,5), (2,4), (3,8)]

```
Python:

```python id="sort1804"
pairs.sort(key=lambda x: x[0])
```
```

Sort by second value:

```python id="sort1805" pairs.sort(key=lambda x: x[1])

```

```
```

```
# 5. MULTIPLE SORT KEYS

Suppose:

> sort by first ascending, and for equal first sort second descending.

Use conceptually:

```python id="sort1806"
pairs.sort(key=lambda x: (x[0], -x[1]))
```
```

Example:

```text id="sort1807" (1,5) (1,8) (2,3)

```
becomes:

```text id="sort1808"
(1,8)
(1,5)
(2,3)
```
```

This kind of tie-breaking is extremely important.

---

## PART B — INTERVAL SORTING

You have already seen intervals, but sort key decides the whole algorithm.

## 6. MERGE INTERVALS

Sort by:

\[ \boxed{\text{start}} \]

Then merge overlaps.

---

## 7. ACTIVITY SELECTION / NON-OVERLAP

Sort by:

\[ \boxed{\text{end}} \]

because earliest finishing interval preserves more future space.

---

## 8. MEETING ROOMS II

Sort meetings by start.

Use Min Heap of ending times.

---

## 9. INTERVAL MEMORY TABLE

```text id="sort1809" Merge intervals !' START Meeting overlap !' START Maximum activities !' END Minimum

removals !’ END Minimum arrows !’ END

```

PART C — SORTING + TWO POINTERS

10. CLASSIC PATTERN

For unsorted problems like:

> find triplets summing to zero

sorting creates order.

Then:

```text id="sort1810"
fix one
left
right
```
```

This reduces:

\[O(n^3) \]

to:

\[\boxed{O(n^2)} \]

for 3Sum.

---

11. TRADE-OFF

Sorting costs:

\[O(n\log n) \]

and may destroy:

- original index order,
- original relative positions.

So always ask:

> **Does the answer require original indices?**

For Two Sum, HashMap is often better if original indices matter.

PART D — SORTING FOR DUPLICATES

12. CONTAINS DUPLICATE

One approach:

Sort:

```text id="sort1811" [4,2,1,2] !’ [1,2,2,4]
```

```
Then check adjacent elements.

Time:

\[
O(n\log n)
\]

Space may be low depending sort implementation.

HashSet gives expected:

\[
O(n)
\]

but uses extra space.

---

# 13. WHEN SORTING IS ATTRACTIVE

Use sorting if:

- modifying order is allowed,
- extra hash memory is undesirable,
- later operations also benefit from sorted order.

---

# PART E — CUSTOM COMPARATOR

This is a very important placement concept.

# 14. WHAT IS A COMPARATOR?

A comparator defines:

> Which of two objects should come first?

For ordinary numbers:

```text id="sort1812"
2 < 5
```
```

Simple.

But some problems need a nonstandard ordering.

PART F — LARGEST NUMBER PROBLEM

---

15. PROBLEM

Given:

```text id="sort1813" [3,30,34,5,9]

```

```
Arrange them to form largest possible number.
```

```
Answer:

```text id="sort1814"
9534330
```
```

16. WHY NORMAL DESCENDING SORT FAILS

Descending numeric order:

```text id="sort1815" 34,30,9,5,3

```
gives something wrong.

The correct question for two numbers \(a,b\) is:

> Should `a` come before `b`?

Compare:

```text id="sort1816"
str(a) + str(b)
```
```

with:

```text id="sort1817" str(b) + str(a)

```
If:

```text id="sort1818"
ab > ba
```
```

put `a` first.

17. EXAMPLE

Compare:

```text id="sort1819" 3 30

```
Possibilities:

```text id="sort1820"
330
303
```
```

Since:

\[ 330>303 \]

3 should come before 30.
```

18. CORE COMPARATOR

```text id="sort1821" a before b if:

a+b > b+a

```
where `a`, `b` are strings.

This is a classic custom comparator pattern.

---

19. IMPORTANT ZERO CASE

Input:

```text id="sort1822"
[0,0]
```
```

Naively concatenating gives:

```text id="sort1823" "00"

```
Expected:

```text id="sort1824"
"0"
```
```

So if first sorted value is `"0"`:

> **return `"0"`.**

PART G — RELATIVE SORTING

20. PROBLEM FAMILY

Sort array `A` according to the ordering specified by array `B`.

Example:

```text id="sort1825" A = [2,1,2,5,7,1,9,3,6,8,8] B = [2,1,8,3]

```
Numbers from B should appear first in B's order.

Remaining values sorted normally.

---

21. PATTERN

Use:

\[
\boxed{\text{Frequency Map + Custom Order}}
```
```

```
\]

1. count values in A,
2. output B values according to frequency,
3. sort remaining keys,
4. output them.

---

# PART H — FREQUENCY SORTING

# 22. PROBLEM

Sort elements by frequency.

Example:

```text id="sort1826"
[1,1,2,2,2,3]
```
```

Frequencies:

```text id="sort1827" 1 !’ 2 2 !’ 3 3 !’ 1

```
Sort may require:

- frequency descending,
- value ascending for ties.

Use:

\[
\boxed{\text{HashMap + Custom Sort}}
\]

PART I — DUTCH NATIONAL FLAG

You already saw this, but it belongs to sorting patterns.

23. PROBLEM

Sort array containing only:

```text id="sort1828"
0,1,2
```
```

in one pass.

Use:

```text id="sort1829" low mid high

```
Regions:

```text id="sort1830"
0 ... low-1        = 0s
low ... mid-1      = 1s
mid ... high       = unknown
high+1 ... end     = 2s
```
```

24. RULES

If:

\[A[mid]=0 \]

swap low/mid:

```text id="sort1831" low++ mid++

```
If:

\[
A[mid]=1
\]

```text id="sort1832"
mid++
```

If:

\[A[mid]=2 \]

swap mid/high:

```text id="sort1833" high--

```
Do **not** increment `mid` yet.

Why?

The element swapped from high has not been examined.

---

25. COMPLEXITY

\[
\boxed{O(n)}
\]

time.

\[
\boxed{O(1)}
\]

space.

PART J — COUNTING SORT

26. WHEN DOES IT WORK?

When values lie in a reasonably small integer range.

Suppose:

```text id="sort1834"
```

```
0 ... k
```

Count each value.

Then reconstruct sorted array.

Complexity:

\[\boxed{O(n+k)} \]

---

27. WHEN IS COUNTING SORT BAD?

If:

```text id="sort1835" n = 1000 values up to 10^12

creating enormous frequency array is impractical.

Use comparison sorting/hash structure instead.

---

# PART K — RADIX SORT

# 28. IDEA

Sort numbers digit by digit.

For base \(b\) and \(d\) digits:

\[
\boxed{O(d(n+b))}
\]

Often useful conceptually, but less common to manually implement in general placement interviews.

Know when comparison lower bound can be bypassed:

> by exploiting numeric structure.

---

# PART L — COMPARISON SORTING LOWER BOUND

Any general comparison-based sorting algorithm has worst-case lower bound:

\[
\boxed{\Omega(n\log n)}
\]

Therefore you cannot build a general comparison sort with guaranteed:

\[
O(n)
\]

time.

But Counting/Radix Sort escape this because they are not purely comparison-based.

---
```

```
# PART M — QUICKSELECT

Very important placement topic.

# 29. PROBLEM

Find kth smallest/largest element.

Sorting everything:

\[
O(n\log n)
\]

Heap of size K:

\[
O(n\log k)
\]

Quickselect can achieve average:

\[
\boxed{O(n)}
\]

---

# 30. QUICKSELECT IDEA

Similar to Quick Sort partition.

Choose pivot.

Partition:

```text id="sort1836"
smaller | pivot | larger
```
```

Suppose pivot lands at index \(p\).

If:

\[p=k \]

done.

If:

\[k<p \]

only search left partition.

If:

\[k>p \]

only search right.

Unlike Quick Sort:

> **recurse into only one side.**

31. AVERAGE COMPLEXITY

\[\boxed{O(n)} \]

Expected/average with good/random pivots.

Worst case:

\[\boxed{O(n^2)} \]

---

32. KTH LARGEST INDEX CONVERSION

If array length:

\[n \]

kth largest corresponds to sorted ascending index:

\[\boxed{n-k} \]

Example:

```text id="sort1837" n = 6 k = 2

```

```
index:

\[
6-2=4
\]

0-based.

---

# PART N — SORTING VS HEAP VS QUICKSELECT

# 33. DECISION TABLE

| Need | Good Choice |
|---|---|
| Entire sorted array | Sorting |
| Kth only, static array | Quickselect |
| Kth/top K with streaming | Heap |
| Top K values | Heap |
| Need worst-case \(O(n\log n)\) simple | Sorting |
| Need average \(O(n)\) selection | Quickselect |

---

# 34. IMPORTANT INTERVIEW ANSWER

If interviewer asks kth largest:

You can discuss:

### Sorting

\[
O(n\log n)
\]

### Heap
```

```
\[
O(n\log k)
\]

### Quickselect

Average:

\[
O(n)
\]

Then choose based on constraints.

That shows strong problem-solving depth.

---

# PART O — MERGE SORT FOR COUNTING

Merge Sort is not just a sorting algorithm.

Its merge phase can count relationships between left and right halves.

Classic problem:

\[
\boxed{\text{Count Inversions}}
\]

---

# PART P — INVERSIONS

# 35. DEFINITION

An inversion is pair:

\[
(i,j)
\]

such that:

\[
i<j
\]

but:

\[
A[i]>A[j]
\]

Example:

```text id="sort1838"
[2,4,1,3,5]
```
```

Inversions:

```text id="sort1839" (2,1) (4,1) (4,3)
```

```
Answer:

\[
\boxed{3}
\]

---

# 36. BRUTE FORCE

Check every pair:

\[
O(n^2)
\]

---

# 37. MERGE SORT IDEA

Suppose left and right halves are already sorted.

If:

```text id="sort1840"
left[i] > right[j]
```
```

then because left is sorted:

> **every remaining element from `left[i]` onward is also greater than `right[j]`.**

So add:

\[\boxed{len(left)-i} \]

inversions at once.

---

38. COMPLEXITY

Merge-sort counting:

\[\boxed{O(n\log n)} \]

This is a crucial interview pattern:

> **Count cross-pairs using Merge Sort.**

PART Q — REVERSE PAIRS

Advanced extension.

Question may ask count:

\[i<j \]

and:

\[A[i]>2A[j] \]

Again use modified Merge Sort.

Before normal merge:

> **use two pointers across sorted halves to count valid cross pairs.**

Time:

\[\boxed{O(n\log n)} \]

PART R — COUNT SMALLER AFTER SELF

Another advanced counting problem.

Possible approaches:

- modified Merge Sort,
- Fenwick Tree,
- Segment Tree.

For placement fundamentals:

> **recognize Merge Sort can count cross-order relationships.**

PART S — SORTED SQUARES

39. PROBLEM

Sorted input:

```text id="sort1841" [-4,-1,0,3,10]

```
Square:

```text id="sort1842"
[16,1,0,9,100]
```

Need sorted result:

```text id="sort1843" [0,1,9,16,100]

```
Naive:

> square + sort.

\[
O(n\log n)
\]

Better:
```
```

```
> Two Pointers from both ends.

Largest absolute value is at an end.

Fill answer backward.

Time:

\[
\boxed{O(n)}
\]

This reinforces:

> sorted order enables two-pointer optimization.

---

# PART T — MERGE SORTED ARRAYS

# 40. PROBLEM

Two sorted arrays.

Use two pointers to merge in:

\[
\boxed{O(n+m)}
\]

If first array has empty buffer at end, merge from the **back** to avoid shifting.

Pattern:

\[
 \ b o x e d { \ t e x t { S o r t e d   +   m e r g e  !’   T w o   P o i n t e r s } }
\]

---

# PART U — SORT COLORS VS COUNTING

For `0,1,2`, two approaches:

### Counting

Count each:

\[
O(n)
\]

then rewrite.

### Dutch National Flag

One pass, in-place:

\[
O(n)
\]

Both valid.

Interview follow-up often asks:
```

```
> Can you do it in one pass?

Then DNF is preferable.

---

# PART V — SORT BY DISTANCE / CUSTOM METRIC

Example:

> sort points by distance to origin.

Metric:

\[
x^2+y^2
\]

No need square root.

```python id="sort1844"
points.sort(key=lambda p: p[0]*p[0] + p[1]*p[1])
```
```

If only K closest are needed:

> **Heap/Quickselect may beat sorting all.**

Again ask:

\[\boxed{\text{Do I need all sorted or only K?}} \]

PART W — STABILITY

41. WHAT IS STABLE SORT?

If equal-key elements preserve original relative order.

Example:

```text id="sort1845"
(A,5) (B,3) (C,5)
```

```
After stable sort by second:

```text id="sort1846"
(B,3)
(A,5)
(C,5)
```
```

A remains before C.

---

42. WHEN STABILITY MATTERS

Suppose objects have multiple fields.

You may sort by secondary key first, then primary key using stable sort.

Or application may require original order among equal priorities.

PART X — IN-PLACE VS EXTRA SPACE

In-place-ish algorithms

- Selection
- Insertion
- Heap Sort
- standard array Quick Sort

Extra memory

Standard Merge Sort:

\[O(n) \]

auxiliary array.

Know this for interview complexity discussions.

PART Y — CUSTOM COMPARATOR TRANSITIVITY TRAP

A comparator must define a consistent ordering.

If comparator says:

```text id="sort1847" A before B B before C C before A

```

```
sorting behavior becomes invalid/unreliable.

Your custom comparison should satisfy consistent ordering properties.

In interviews, explain comparator carefully.

---

# PART Z — SORTING + GREEDY

Many Greedy algorithms become easy only after sorting.

Examples:

```text id="sort1848"
Activity selection
!’   s o r t   b y   f i n i s h

Assign cookies
!’   s o r t   b o t h

Minimum arrows
!’   s o r t   b y   e n d

Fractional knapsack
!’   s o r t   b y   r a t i o
```
```

So a common pattern is:

\[\boxed{\text{Sort !’ Greedy Scan}} \]

PART AA — SORTING + SWEEP LINE AWARENESS

Advanced interval/event problems often convert intervals into events:

```text id="sort1849" (start, +1) (end, -1)

Sort events by coordinate/time.

Maintain current active count.

Useful for:

- maximum overlapping intervals,
- number of active events,
- meeting-room variants.

This is called:

\[
\boxed{\text{Sweep Line}}
\]

You don't need to master advanced geometry yet, but recognize the pattern.

---

# PART AB — INTERVIEW EXPLANATION: QUICKSELECT

A strong answer:

> “Sorting would solve kth largest in \(O(n\log n)\), but since I only need one order statistic, Quickselect is more appropriate. I partition around a pivot as in Quick Sort. Once the pivot reaches its final sorted position, I only recurse into the side containing the target index rather than sorting both halves. With randomized pivots, the expected time is \(O(n)\), though the worst case is \(O(n^2)\).”

---

# PART AC — INTERVIEW EXPLANATION: INVERSIONS

> “The brute-force approach checks every pair and costs \(O(n^2)\). During Merge Sort, both halves are sorted. If `left[i] > right[j]`, then every remaining element from `left[i]` onward is also greater than `right[j]`, so I can count all those inversions at once. The merge process remains linear at each level, giving total \(O(n\log n)\).”

---

# PART AD — MASTER DECISION: SORT / HASH / HEAP / QUICKSELECT

Use:

```text id="sort1850"
Need exact membership / frequency?
!’   H a s h i n g

Need whole ordered output?
!’   S o r t

Need Top K / repeated min-max?
!’   H e a p

Need only kth element in static array?
```
```

```
!'   Q u i c k s e l e c t

Need pair/triplet after ordering?
!'   S o r t   +   T w o   P o i n t e r s

Need interval scheduling?
!'   S o r t   +   G r e e d y

Need count cross-order pairs?
!'   M e r g e   S o r t
```

PART AE — COMMON INTERVIEW TRAPS

43. TRAP 1 — SORTING WHEN ORIGINAL INDICES MATTER

Sorting changes positions.

Keep:

```text id="sort1851" (value, original_index)

```
if required.

---

44. TRAP 2 — WRONG CUSTOM SORT KEY

Example Activity Selection:

Sort by start is not optimal for maximum number of activities.

Need:

\[
\boxed{\text{finish time}}
\]

---

45. TRAP 3 — QUICKSELECT WORST CASE

Do not claim:

\[
O(n)
\]

worst case.

Correct:

\[
\boxed{\text{Expected/Average }O(n),\ Worst\ O(n^2)}
\]

---

46. TRAP 4 — BUILD HEAP VS SORTING

If you need full sorted array, simply building heap:
```
```

```
\[
O(n)
\]

does **not** mean heap sort is \(O(n)\).

Repeated removals still cost:

\[
O(n\log n)
\]

---

# 47. TRAP 5 — STABILITY

Do not say every \(O(n\log n)\) sort is stable.

Merge Sort can be stable.

Heap Sort generally isn't.

Quick Sort generally isn't.

---

# 48. TRAP 6 — COUNTING SORT RANGE

Complexity depends on \(k\):

\[
O(n+k)
\]

If \(k\) is enormous, it may be impractical.

---

# 49. TRAP 7 — COMPARISON LOWER BOUND

\[
\Omega(n\log n)
\]

applies to general **comparison-based** sorting.

Not automatically Counting/Radix.

---

# 50. TRAP 8 — CUSTOM COMPARATOR OVERFLOW

In Java/C++, avoid comparator like:

```text id="sort1852"
return a - b
```
```

when integers may overflow.

Prefer safe comparison functions such as `Integer.compare(a,b)` in Java.

PART AF — MUST-DO QUESTIONS

Sorting Foundation

1. Sort Array
2. Merge Sort
3. Quick Sort
4. Heap Sort
5. Sort Colors
6. Merge Sorted Arrays

Sorting Patterns

7. Merge Intervals
8. 3Sum
9. Sorted Squares
10. Contains Duplicate via sorting
11. Relative Sort Array
12. Sort Characters by Frequency

Custom Comparator

13. Largest Number
14. Sort pairs by multiple keys
15. Arrange intervals by end
16. Custom object sorting

Selection

17. Kth Largest
18. Kth Smallest
19. Quickselect
20. K Closest Points

Merge-Sort Counting

21. Count Inversions
22. Reverse Pairs
23. Count Smaller After Self — advanced

Sweep/Intervals

24. Meeting Rooms
25. Meeting Rooms II
26. Minimum Platforms
27. Maximum Overlapping Intervals

PLACEMENT PRACTICE SET

Q1 — MCQ

If you need the entire array sorted, a natural choice is:

A. Sorting B. DSU C. BFS D. Trie

Q2 — MCQ

For 3Sum, common optimized pattern is:

A. Sort + Fix One + Two Pointers

B. DFS only

C. Heap Sort only

D. Prefix Trie

Q3 — MCQ

Merge Intervals is typically sorted by:

A. start

B. end only

C. duration

D. value/weight

Q4 — MCQ

Activity Selection is typically sorted by:

A. earliest finish

B. earliest start

C. largest duration

D. largest index

Q5 — MCQ

For Largest Number, compare strings \(a,b\) using:

A. `a < b` only

B. `a+b` versus `b+a`

C. numeric difference only

D. string length only

Q6 — NAT

For:

```text id="sort1853" [2,4,1,3,5]

how many inversions exist?

---

## Q7 — MCQ

Quickselect expected complexity is:

A. \(O(\log n)\)

B. \(O(n)\)

C. \(O(n\log n)\)

D. \(O(n^2)\) always

---

## Q8 — MCQ

Quickselect worst-case complexity is:

A. \(O(n)\)

B. \(O(n\log n)\)

C. \(O(n^2)\)

D. \(O(1)\)

---

## Q9 — MSQ

Which are valid uses of custom sorting?

A. Sort intervals by end

B. Sort objects by multiple keys

C. Arrange numbers for largest concatenated number

D. BFS shortest path

---

## Q10 — MCQ

To find kth largest from a continuous stream, best common structure is:

A. Heap

B. Quickselect repeatedly

C. DFS

D. Trie

---

## Q11 — MCQ

To find kth largest once in a static array with good average performance and without fully
```

sorting, use:

A. Quickselect

B. KMP

C. Bellman-Ford

D. Trie

Q12 — MSQ

Which statements are true?

A. Standard Merge Sort can be stable.

B. Heap Sort is generally not stable.

C. Quick Sort worst case can be $O(n^2)$.

D. Comparison sorting has general lower bound $\Omega(n\log n)$.

Q13 — MCQ

Counting Sort runs in:

A. $O(n+k)$

B. $O(n\log n)$ always

C. $O(n^2)$ always

D. $O(\log n)$

Q14 — MCQ

Modified Merge Sort can count inversions in:

A. $O(n)$

B. $O(n\log n)$

C. $O(n^2)$

D. $O(2^n)$

Q15 — MSQ

Which are often solved using Sort + Greedy?

A. Activity Selection

B. Minimum Arrows

C. Assign Cookies

D. Longest Common Subsequence

```
---

# PRACTICE SOLUTIONS

## Q1

\[
\boxed{A}
\]

---

## Q2

\[
\boxed{A}
\]

---

## Q3

\[
\boxed{A}
\]

---

## Q4

\[
\boxed{A}
\]

---

## Q5

\[
\boxed{B}
\]

---

## Q6

Inversions:

```text id="sort1854"
(2,1)
(4,1)
(4,3)
```
```

Total:

\[\boxed{3} \]

Q7

Expected:

\[\boxed{O(n)} \]

Answer:

$$\boxed{B}$$

Q8

Worst:

$$\boxed{O(n^2)}$$

Answer:

$$\boxed{C}$$

Q9

A, B, C.

$$\boxed{A,B,C}$$

Q10

Streaming Top-K:

$$\boxed{A}$$

Q11

$$\boxed{A}$$

Q12

All are true.

$$\boxed{A,B,C,D}$$

Q13

$$\boxed{A}$$

Q14

$$\boxed{B}$$

Q15

A, B, C.

$$\boxed{A,B,C}$$

Ø=ÑØTES TO WRITE

```text id="sort1855" SORTING PATTERNS

Need whole order !’ Sort

Sorted pair/triplet !’ Two Pointers

Intervals: Merge !’ sort START Activity !’ sort END Min removals !’ sort END Min arrows !’ sort END

Custom Comparator: define which object comes first

Largest Number: a before b if a+b > b+a

Top K: !’ Heap

Kth only, static: !’ Quickselect

Quickselect: Expected O(n) Worst O(n²)

Inversions: Modified Merge Sort O(n log n)

Counting Sort: O(n + k)

Comparison sorting lower bound: © (log n)

Dutch National Flag: 0 / 1 / 2 in O(n), O(1)

---

# &¡ 30-SECOND REVISION

1. Sorting often exposes structure.
2. Sort + scan is a major interview pattern.
3. Sort + Two Pointers solves many pair/triplet questions.
4. Sorting changes original indices.
5. Merge Intervals !’ sort by start.
6. Maximum non-overlap !’ sort by end.
7. Custom comparator defines ordering.
8. Largest Number !’ compare `ab` vs `ba`.
9. Top K !’ Heap.
10. One kth element !’ Quickselect.
11. Quickselect expected \(O(n)\), worst \(O(n^2)\).
12. Merge Sort can count inversions.
13. Inversions can be counted in \(O(n\log n)\).
14. Counting Sort = \(O(n+k)\).
15. Comparison sorting lower bound = \(\Omega(n\log n)\).
16. Build Heap \(O(n)\), Heap Sort \(O(n\log n)\).
17. Stable sort preserves relative order among equal keys.
18. Merge Sort can be stable.
19. Heap Sort generally isn't stable.
20. Sort + Greedy is common in scheduling.

# ' Placement Module 18 Complete

## Next Placement Module:

# **MODULE 19 — ADVANCED DATA STRUCTURES: FENWICK TREE + SEGMENT TREE + RANGE QUERY PATTERNS**

Next we'll cover:

- why Prefix Sum fails when updates occur,
- Fenwick Tree / Binary Indexed Tree,
- lowbit trick,
- point update + prefix query,
```

```
- range-sum queries,
- Segment Tree,
- build/query/update,
- range minimum/maximum,
- lazy propagation intuition,
- coordinate compression,
- inversion counting with Fenwick Tree,
- and exactly when to choose **Prefix Sum vs Fenwick Tree vs Segment Tree**.

**Type `NEXT` to continue.**
```

USER

NEXT

ASSISTANT

PLACEMENT MODULE 19 — ADVANCED DATA STRUCTURES: FENWICK TREE + SEGMENT TREE + RANGE QUERY PATTERNS

Placement Priority: & & & & & & & & & &

This module is more advanced than Arrays, Hashing, Trees, Graphs, and DP.

But for stronger product-company interviews and competitive coding, these ideas are extremely useful.

The core question is:

> **What if I need many range queries and also many updates?**

That is where:

\[\boxed{\text{Fenwick Tree}} \]

and

\[\boxed{\text{Segment Tree}} \]

become important.

---

1. WHY PREFIX SUM IS NOT ENOUGH

Suppose:

```text id="r1901" nums = [2,4,1,3,5]

We build prefix sums.

Then range-sum query can be answered in:

\[
\boxed{O(1)}
\]
```

```
Great.

But now suppose:

```text id="r1902"
nums[2] changes from 1 to 10
```
```

All prefix sums after index 2 become incorrect.

Updating them may require:

\[\boxed{O(n)} \]

So:

```text id="r1903" Prefix Sum: Query !’ O(1) Update !’ O(n)

```
If updates are frequent, this is inefficient.

---

2. THE RANGE QUERY DECISION TABLE

Situation	Good Choice
Static array, many sum queries	Prefix Sum
Point updates + prefix/range sums	Fenwick Tree
Point updates + sum/min/max/etc.	Segment Tree
Many simple offline range additions	Difference Array
Complex range updates + queries	Segment Tree + Lazy Propagation

Memorize:

```text id="r1904"
Static queries
!’   P r e f i x

Dynamic sums
!’   F e n w i c k

Dynamic general ranges
!’   S e g m e n t   T r e e
```

PART A — FENWICK TREE / BIT

3. WHAT IS A FENWICK TREE?

Fenwick Tree is also called:

\[\boxed{\text{Binary Indexed Tree}} \]

or:

\[\boxed{\text{BIT}} \]

It efficiently supports:

- point update
- prefix sum query

- range sum query

with:

\[\boxed{O(\log n)} \]

time per operation.

Space:

\[\boxed{O(n)} \]

---

4. MAIN OPERATIONS

Update

Add value \(x\) at position \(i\).

\[O(\log n) \]

Prefix Query

Find:

\[A[1]+A[2]+\cdots+A[i] \]

in:

\[O(\log n) \]

Then range sum:

\[\boxed{ sum(L,R)=prefix(R)-prefix(L-1) } \]

---

5. IMPORTANT INDEXING RULE

Fenwick Tree is easiest with:

\[\boxed{\text{1-based indexing}} \]

So if original array is 0-based:

```text id="r1905" array index 0 !' BIT index 1 array index 1 !' BIT index 2 ...

```
This avoids many bugs.

PART B — THE `LOWBIT` TRICK

Fenwick Tree relies on:

\[
\boxed{x\&(-x)}
\]

This extracts the lowest set bit.

Example:

\[
```
```

```
12=1100_2
\]

Lowest set bit represents:

\[
4
\]

And:

\[
12\&(-12)=4
\]

---

# 6. WHY LOWBIT MATTERS

Each Fenwick node stores sum for a block whose length is:

\[
\boxed{lowbit(i)}
\]

Example:

```text id="r1906"
i = 8
lowbit(8) = 8

i = 6
binary 110
lowbit(6) = 2

i = 12
binary 1100
lowbit(12) = 4
```

---

7. WHAT DOES `tree[i]` STORE?

It stores sum of a specific ending range.

Formula:

\[\boxed{ tree[i] = A[i-lowbit(i)+1]+\cdots+A[i] } \]

Example:

For \(i=6\):

\[lowbit(6)=2 \]

So `tree[6]` stores:

\[A[5]+A[6] \]

For \(i=8\):

\[lowbit(8)=8 \]

So `tree[8]` stores:

\[A[1]+\cdots+A[8] \]
```

PART C — FENWICK PREFIX QUERY

8. GOAL

Find prefix sum up to index \(i\).

Algorithm:

```text id="r1907" sum = 0

while i > 0: sum += tree[i] i -= lowbit(i)

```
That is:

\[
\boxed{i=i-(i\&-i)}
\]

---

9. WHY SUBTRACT LOWBIT?

Because `tree[i]` already covers a block ending at \(i\).

After adding that block:

> jump to the previous uncovered block.

Example query prefix 7:

```text id="r1908"
7 = 111
lowbit = 1

 7  !’   6
6 = 110
lowbit = 2

 6  !’   4
4 = 100
lowbit = 4

 4  !’   0
```

So only a few nodes are visited.

Hence:

\[\boxed{O(\log n)} \]

10. QUERY CODE

```python id="r1909" def query(tree, i): total = 0

while i > 0: total += tree[i] i -= i & -i

return total
```

```
---

# PART D — FENWICK UPDATE

# 11. GOAL

Add `delta` to array position \(i\).

All Fenwick nodes containing that position must be updated.

Algorithm:

```text id="r1910"
while i <= n:
    tree[i] += delta
    i += lowbit(i)
```
```

or:

\[\boxed{i=i+(i\&-i)} \]

---

12. UPDATE CODE

```python id="r1911" def update(tree, i, delta): n = len(tree) - 1

while i <= n: tree[i] += delta i += i & -i

```
Time:

\[
\boxed{O(\log n)}
\]

---

13. MEMORY TRICK

Fenwick:

```text id="r1912"
 Q u e r y   !’  m o v e  D O W N :
i -= lowbit(i)

 U p d a t e  !’  m o v e  U P :
i += lowbit(i)
```

Lock this.

PART E — FULL FENWICK TREE CLASS

```python id="r1913" class FenwickTree: def **init**(self, n): self.n = n self.tree = [0] * (n + 1)

def update(self, i, delta): while i <= self.n: self.tree[i] += delta i += i & -i

def prefix_sum(self, i): total = 0

while i > 0: total += self.tree[i] i -= i & -i
```

return total

def range_sum(self, left, right): return (self.prefix_sum(right)

- self.prefix_sum(left - 1)

)

```
Indices here are 1-based.

---

# PART F — BUILDING FENWICK TREE

Simple build:

For each array value:

```python id="r1914"
for i in range(1, n + 1):
    bit.update(i, arr[i - 1])
```
```

This costs:

\[O(n\log n) \]

There is also an \(O(n)\) construction, but for placements the simpler build is usually enough unless specifically asked.

PART G — ASSIGNMENT UPDATE VS ADDITIVE UPDATE

Very common bug.

Fenwick update usually means:

\[\boxed{\text{add delta}} \]

not:

> **set value directly.**

Suppose old value:

\[5 \]

new value:

\[9 \]

Then:

\[delta=9-5=4 \]

Call:

```text id="r1915" update(index, 4)

```
```

```
not:

```text id="r1916"
update(index, 9)
```
```
```

unless your implementation specifically supports assignment.

PART H — RANGE SUM USING FENWICK

For range:

\[L...R \]

use:

\[\boxed{ prefix(R)-prefix(L-1) } \]

Same mathematical idea as Prefix Sum arrays.

Difference:

> **prefix values are dynamically maintained.**

PART I — WHEN FENWICK TREE IS BEST

Use Fenwick when:

- operation is mainly sum/count,
- point updates,
- prefix queries,
- range sum,
- inversion counting,
- frequency tables with updates.

It is often simpler than Segment Tree.

PART J — INVERSION COUNT WITH FENWICK TREE

You saw inversion counting using Merge Sort.

Fenwick gives another method.

14. PROBLEM

```text id="r1917" [2,4,1,3,5]

```

```
An inversion:

\[
i<j,\ A[i]>A[j]
\]

Answer:

\[
3
\]
```

```
---

# 15. IDEA

Process elements from right to left.

For current value \(x\):

> how many previously processed elements are smaller than \(x\)?

Those processed elements originally appeared to the right.

That count contributes inversions.

Fenwick stores frequency of values.

---

# 16. PROBLEM — VALUES MAY BE HUGE

Example:

```text id="r1918"
[1000000000, 5, 90000000]
```
```

Cannot allocate Fenwick array of size \(10^9\).

Solution:

\[\boxed{\text{Coordinate Compression}} \]

PART K — COORDINATE COMPRESSION

17. IDEA

We usually care about relative ordering, not exact magnitude.

Values:

```text id="r1919" [100, 5000, -20, 100]

```
```
Distinct sorted:

```text id="r1920"
[-20,100,5000]
```
```

Assign ranks:

```text id="r1921" -20 !' 1 100 !' 2 5000 !' 3

```
```
Original becomes:

```text id="r1922"
[2,3,1,2]
```
```

Relative ordering is preserved.

18. COMPLEXITY

Sorting distinct values:

\[O(n\log n) \]

Then Fenwick operations:

\[O(\log n) \]

each.

Coordinate compression appears frequently with:

- Fenwick Tree,
- Segment Tree,
- sweep line,
- large sparse values.

PART L — SEGMENT TREE

19. WHAT IS SEGMENT TREE?

Segment Tree is a binary tree where each node represents an interval.

Example array:

```text id="r1923" [2,4,1,3]

```
Root covers:

```text id="r1924"
[0,3]
```
```

Its children:

```text id="r1925" [0,1]    [2,3]

```
Then:

```text id="r1926"
[0,0] [1,1] [2,2] [3,3]
```
```

20. WHAT CAN EACH NODE STORE?

Depending on problem:

- sum
- minimum
- maximum
- GCD

- frequency/count
- other mergeable information

This is why Segment Tree is more flexible than Fenwick Tree.

PART M — SEGMENT TREE OPERATIONS

Typical:

Build

\[O(n) \]

Range Query

\[\boxed{O(\log n)} \]

for standard queries.

Point Update

\[\boxed{O(\log n)} \]

Space:

\[\boxed{O(n)} \]

typically allocated around:

\[4n \]

for recursive array implementation.

PART N — BUILD SEGMENT TREE

21. RECURSIVE IDEA

For node representing interval:

\[[l,r] \]

If:

\[l=r \]

it's a leaf.

Otherwise:

\[mid=(l+r)//2 \]

Build:

```text id="r1927" left child  !' [l,mid] right child !' [mid+1,r]

```
Then combine children.

For sum:
```
```

```
\[
\boxed{
tree[node]=tree[left]+tree[right]
}
\]

---

# 22. CODE

```python id="r1928"
def build(arr, tree, node, left, right):
    if left == right:
        tree[node] = arr[left]
        return

    mid = (left + right) // 2

    build(arr, tree, node * 2, left, mid)
    build(arr, tree, node * 2 + 1, mid + 1, right)

    tree[node] = (
        tree[node * 2]
        + tree[node * 2 + 1]
    )
```
```

PART O — RANGE QUERY

Suppose node covers:

\[[l,r] \]

and query asks:

\[[ql,qr] \]

There are 3 cases.

---

23. CASE 1 — NO OVERLAP

```text id="r1929" node interval: [0,2]

query: [4,6]

```
No intersection.

For sum query return:

\[
\boxed{0}
\]

because 0 is identity for addition.

---

24. CASE 2 — COMPLETE OVERLAP

Node interval completely inside query.
```
```

```
Example:

```text id="r1930"
node = [2,4]
query = [1,6]
```
```

Return stored node value directly.

No need to go deeper.

---

25. CASE 3 — PARTIAL OVERLAP

Need query both children and combine.

For sum:

\[\boxed{ leftResult+rightResult } \]

---

26. QUERY TEMPLATE

```python id="r1931" def query(tree, node, left, right, ql, qr): if qr < left or right < ql: return 0

if ql <= left and right <= qr: return tree[node]

mid = (left + right) // 2

return ( query(tree, node *2, left, mid, ql, qr) + query(tree, node* 2 + 1, mid + 1, right, ql, qr) )

```
Time:

\[
\boxed{O(\log n)}
\]

for standard range query.

PART P — POINT UPDATE

Suppose:

```text id="r1932"
arr[index] = new_value
```
```

Navigate from root to the corresponding leaf.

Update leaf.

Then on return:

> **recompute every ancestor.**

---
```

27. CODE

```python id="r1933" def update(tree, node, left, right, index, value): if left == right: tree[node] = value return

mid = (left + right) // 2

if index <= mid: update(tree, node *2, left, mid, index, value) else: update(tree, node* 2 + 1, mid + 1, right, index, value)

tree[node] = ( tree[node * 2]

  - tree[node * 2 + 1]

)

```

```
Time:

\[
\boxed{O(\log n)}
\]

---

# PART Q — WHY SEGMENT TREE IS LOGARITHMIC

Height of balanced segment tree:

\[
\boxed{O(\log n)}
\]

A point update follows one path from root to leaf.

So:

\[
O(\log n)
\]

Range queries visit only relevant nodes, also around logarithmic for standard aggregation queries.

---

# PART R — RANGE MINIMUM QUERY

Suppose each node stores:

\[
\boxed{\text{minimum in its interval}}
\]

Combine using:

\[
\min(left,right)
\]

For no overlap, identity should be:

\[
\boxed{+\infty}
\]

not 0.

Why?
```

```
If values are positive, returning 0 would incorrectly dominate minimum.

---

# 28. IDENTITY ELEMENT MATTERS

For sum:

```text id="r1934"
 N o   o v e r l a p  !’   0
```
```

For min:

```text id="r1935" No overlap !’  +"

```
For max:

```text id="r1936"
 N o   o v e r l a p  !’   -"
```
```

For GCD:

```text id="r1937" No overlap !’ 0

```
because:

\[
gcd(x,0)=x
\]

This is an important Segment Tree concept.

---

# PART S — SEGMENT TREE GENERALIZATION

Segment Tree works whenever results can be merged associatively.

Examples:

\[
sum(a,b)
\]

\[
min(a,b)
\]

\[
max(a,b)
\]

\[
gcd(a,b)
\]

You can think:

> each node summarizes its interval.
```

```
---

# PART T — FENWICK VS SEGMENT TREE

| Fenwick Tree | Segment Tree |
|---|---|
| Simpler | More flexible |
| Great for sums/counts | Sum/min/max/GCD/etc. |
| \(O(\log n)\) update | \(O(\log n)\) update |
| \(O(\log n)\) prefix query | \(O(\log n)\) range query |
| Smaller implementation | Larger implementation |
| Uses lowbit | Uses interval tree |

Memory shortcut:

```text id="r1938"
Simple dynamic SUM
!’   F e n w i c k

General dynamic RANGE
!’   S e g m e n t   T r e e
```
```

PART U — PREFIX SUM VS FENWICK VS SEGMENT TREE

29. STATIC ARRAY

No updates:

```text id="r1939" many sum queries

```
Use Prefix Sum.

```text id="r1940"
Build  O(n)
Query  O(1)
```

30. POINT UPDATES + SUM QUERIES

Use Fenwick.

```text id="r1941" Update O(log n) Query  O(log n)

```
---

31. POINT UPDATES + MIN/MAX/SUM QUERIES

Use Segment Tree.

```text id="r1942"
Update O(log n)
Query  O(log n)
```

PART V — RANGE UPDATES

What if the question says:

> **Add \(x\) to every element from \(L\) to \(R\), many times.**

There are multiple possibilities.

If all updates happen first and queries only later:

\[\boxed{\text{Difference Array}} \]

is easiest.

But if updates and queries are mixed dynamically:

> **Segment Tree with Lazy Propagation may be needed.**

PART W — LAZY PROPAGATION

32. PROBLEM

Suppose Segment Tree stores range sums.

Update:

```text id="r1943" add 5 to every element in [1000, 900000]

```
Updating every leaf individually is too slow.

Lazy Propagation lets us update an entire covered segment without immediately visiting all descendants.

---

33. CORE IDEA

Each node may store:

```text id="r1944"
tree[node]
lazy[node]
```
```

`lazy[node]` means:

> **this update still needs to be propagated to children later.**

We postpone unnecessary work.

Hence:

\[ \boxed{\text{Lazy = Delay work until needed}} \]

---
```

34. EXAMPLE

Suppose node covers 8 elements and update adds 3 to all of them.

For sum tree:

\[tree[node]+=8\times3 \]

Instead of visiting all 8 leaves.

Then mark children as having pending +3.

---

35. COMPLEXITY

With proper lazy propagation:

Range update

\[\boxed{O(\log n)} \]

Range query

\[\boxed{O(\log n)} \]

for standard use.

---

36. SHOULD YOU MASTER FULL LAZY CODE NOW?

For entry-level placement preparation:

Priority:

1. Understand Prefix Sum.
2. Understand Fenwick.
3. Understand basic Segment Tree.
4. Understand Lazy concept.
5. Implement full lazy tree only if target companies/contests require it.

Do not let advanced trees consume time before mastering core DSA.

PART X — RANGE UPDATE USING FENWICK

Fenwick can also support certain range-update variants using clever transformations.

For example:

- range update + point query
- range update + range sum using two BITs

But this is advanced.

For placement fundamentals:

\[\boxed{\text{Use Segment Tree when requirements become complex}} \]

PART Y — ORDER STATISTICS USING FENWICK

Advanced pattern:

Suppose Fenwick stores frequencies.

You can find:

> **kth smallest active value**

by searching on cumulative frequency.

This appears in competitive programming and advanced interviews.

It connects:

\[\boxed{\text{Fenwick + Binary Search on Prefix Frequency}} \]

or bit-lifting within Fenwick.

Awareness is enough initially.

PART Z — RANGE QUERY EXAMPLE

Suppose:

```text id="r1945" A = [2,4,1,3,5]

```
Need frequent operations:

```text id="r1946"
sum(1,3)
update index 2 to 10
sum(0,4)
update index 4 to 7
sum(2,4)
```

Prefix Sum becomes inconvenient because updates invalidate many prefixes.

Fenwick/Segment Tree handles each operation in:

\[\boxed{O(\log n)} \]

PART AA — WHY NOT JUST RECOMPUTE?

Suppose:

\[n=10^5 \]

and:

\[q=10^5 \]

operations.

If every update/query costs \(O(n)\):

\[10^5\times10^5 = 10^{10} \]

too expensive.

With:

$$ O(\log n) $$

each:

$$ 10^5\log_2(10^5) $$

roughly only a few million operations.

That is why these structures matter.

PART AB — SEGMENT TREE NODE COUNT

A recursive Segment Tree array often allocates:

$$ \boxed{4n} $$

space.

Why?

It is a safe upper bound for arbitrary n.

Actual number of nodes is $O(n)$.

PART AC — ITERATIVE SEGMENT TREE AWARENESS

There is also an iterative Segment Tree stored in array of roughly:

$$ 2n $$

positions.

It can be:

- compact,
- fast,
- less recursive.

But recursive version is easier for learning and interviews.

PART AD — STATIC RANGE MINIMUM QUERY

If array never changes and many minimum queries are required, Segment Tree works.

But advanced alternative:

$$ \boxed{\text{Sparse Table}} $$

For idempotent static operations such as min/max/GCD:

- preprocessing $O(n\log n)$
- query $O(1)$ for RMQ-style operations

This is more competitive-programming oriented.

Know the name; prioritize Segment Tree first.

PART AE — FENWICK TREE RECOGNITION

If problem says:

```text id="r1947" many updates many prefix sums frequencies changing count elements smaller than x dynamically

```
consider:

\[
\boxed{\text{Fenwick Tree}}
\]

PART AF — SEGMENT TREE RECOGNITION

If problem says:

```text id="r1948"
many updates
many range minimum queries

many updates
many range maximum queries

dynamic range aggregate
```

consider:

\[\boxed{\text{Segment Tree}} \]

PART AG — COORDINATE COMPRESSION RECOGNITION

If values are:

```text id="r1949" 10^9 -10^12 999999999 ...

```
but only their ordering matters:

\[
\boxed{\text{Coordinate Compression}}
\]

Example:

```text id="r1950"
 1 0 0  !'   2
 5 0 0  !'   3
 - 5 0  !'   1
```

Preserve:

\[-50<100<500 \]

PART AH — COUNT SMALLER ELEMENTS AFTER SELF

Advanced classic.

Input:

```text id="r1951" [5,2,6,1]

```
For each element, count smaller numbers to its right.

Output:

```text id="r1952"
[2,1,1,0]
```
```

Approaches:

- Merge Sort
- Fenwick + Coordinate Compression

Process right to left.

For each value rank \(r\):

```text id="r1953" query(r - 1)

```
gives number of smaller processed values.

Then:

```text id="r1954"
update(r, +1)
```
```

PART AI — INTERVIEW EXPLANATION: FENWICK TREE

A strong answer:

> **"A normal prefix-sum array gives \(O(1)\) range queries, but a point update can require \(O(n)\) because many prefix values change. A Fenwick Tree balances the two operations: point updates and prefix-sum queries both take \(O(\log n)\). It uses 1-based indexing and the `i & -i` lowbit operation to move between ranges. A range sum is obtained as `prefix(right) - prefix(left-1)`."**

PART AJ — INTERVIEW EXPLANATION: SEGMENT TREE

> **"I would use a Segment Tree because I need dynamic range queries and updates. Each node stores the aggregate value for an interval. A point update modifies one leaf and recomputes the \(O(\log n)\) ancestors. For a range query, I return immediately for complete overlap, ignore no-overlap segments, and recursively combine partial overlaps. This gives \(O(\log n)\) point updates and standard range queries after an \(O(n)\) build."**

PART AK — COMMON FENWICK TREE TRAPS

37. TRAP 1 — USING INDEX 0

Fenwick loops depend on:

```text id="r1955" i & -i

```
At:

\[
i=0
\]

lowbit is 0.

Update loop can become stuck.

Use 1-based indices.

---

38. TRAP 2 — SET VS ADD

Standard Fenwick update adds delta.

For assignment:

\[
delta=new-old
\]

---

39. TRAP 3 — RANGE FORMULA

For 1-based:

\[
\boxed{
range(L,R)=prefix(R)-prefix(L-1)
}
\]

---

40. TRAP 4 — LOWBIT DIRECTION

```text id="r1956"
Query:
i -= i & -i

Update:
i += i & -i
```
```

Don't reverse them.

## PART AL — COMMON SEGMENT TREE TRAPS
```

41. TRAP 1 — WRONG NO-OVERLAP VALUE

Sum:

\[0 \]

Min:

\[+\infty \]

Max:

\[-\infty \]

Choose correct identity.

---

42. TRAP 2 — WRONG MID SEGMENTS

Use:

```text id="r1957" left child  = [l, mid] right child = [mid+1, r]

```
Avoid overlapping or missing indices.

---

43. TRAP 3 — FORGET TO RECOMPUTE PARENT

After updating child:

```text id="r1958"
tree[node] = combine(left, right)
```

must happen.

---

44. TRAP 4 — QUERY CASES

Always reason:

```text id="r1959" No overlap Complete overlap Partial overlap

```
This three-case template is central.

---

45. TRAP 5 — USING SEGMENT TREE FOR EVERYTHING

If array is static and only sums:

> Prefix Sum is simpler and faster.

If only dynamic sums:

> Fenwick is usually simpler.

Choose simplest sufficient structure.

```
```

```
# PART AM — DATA STRUCTURE DECISION MAP

```text id="r1960"
Static Range Sum
!’   P r e f i x   S u m

Many offline Range Additions
!’   D i f f e r e n c e   A r r a y

Point Update + Range Sum
!’   F e n w i c k   /   S e g m e n t   T r e e

Point Update + Range Min/Max
!’   S e g m e n t   T r e e

Range Update + Range Query
!’   L a z y   S e g m e n t   T r e e

Huge values but only rank matters
!’   C o o r d i n a t e   C o m p r e s s i o n

Dynamic frequency + prefix counts
!’   F e n w i c k
```
```

PART AN — COMPLEXITY TABLE

Structure	Build	Query	Point Update	Space
Prefix Sum	\(O(n)\)	\(O(1)\)	\(O(n)\)	\(O(n)\)
Fenwick Tree	\(O(n\log n)\)*	\(O(\log n)\)	\(O(\log n)\)	\(O(n)\)
Segment Tree	\(O(n)\)	\(O(\log n)\)	\(O(\log n)\)	\(O(n)\)
Lazy Segment Tree	\(O(n)\)	\(O(\log n)\)	Range \(O(\log n)\)	\(O(n)\)

*Simple Fenwick build. An \(O(n)\) build exists.

PART AO — MUST-DO QUESTIONS

Fenwick Foundation

1. Implement Binary Indexed Tree
2. Prefix Sum Query
3. Range Sum Query
4. Point Update
5. Range Sum Query Mutable

Fenwick Applications

6. Count Inversions
7. Count Smaller After Self

8. Dynamic Frequency Queries
9. Coordinate Compression + BIT

Segment Tree Foundation

10. Build Segment Tree
11. Range Sum Query
12. Point Update
13. Range Minimum Query
14. Range Maximum Query

Advanced

15. Lazy Propagation Concept
16. Range Addition + Range Sum
17. Count Range Sum — advanced
18. Reverse Pairs alternatives
19. Dynamic kth order statistic awareness
20. Sparse Table awareness

PLACEMENT PRACTICE SET

Q1 — MCQ

Static array with many range-sum queries and no updates:

A. Prefix Sum B. DSU C. Trie D. DFS

Q2 — MCQ

Fenwick Tree supports point updates and prefix queries in:

A. \(O(1)\)

B. \(O(\log n)\)

C. \(O(n)\)

D. \(O(n^2)\)

Q3 — MCQ

Fenwick Tree is commonly implemented using:

A. 0-based indexing only

B. 1-based indexing

C. negative indexing

D. strings only

Q4 — MCQ

`i & -i` gives:

A. highest set bit

B. lowest set bit

C. number of bits

D. parity only

Q5 — MCQ

During Fenwick prefix query, index changes as:

A. `i += lowbit(i)`

B. `i -= lowbit(i)`

C. `i *= 2`

D. `i = 0` immediately

Q6 — MCQ

During Fenwick update:

A. `i -= lowbit(i)`

B. `i += lowbit(i)`

C. `i /= 10`

D. `i--` only

Q7 — MSQ

Fenwick Tree is suitable for:

A. dynamic prefix sums

B. dynamic frequency counts

C. inversion counting

D. arbitrary graph shortest paths

Q8 — MCQ

Segment Tree build complexity is typically:

A. \(O(n)\)

B. \(O(n^2)\)

C. \(O(\log n)\)

D. \(O(2^n)\)

Q9 — MCQ

Standard Segment Tree point update is:

A. $O(1)$

B. $O(\log n)$

C. $O(n)$

D. $O(n\log n)$

Q10 — MCQ

For a Segment Tree sum query, no-overlap should return:

A. 0

B. $+\infty$

C. $-\infty$

D. 1 always

Q11 — MCQ

For a Segment Tree minimum query, no-overlap should return:

A. 0

B. $+\infty$

C. $-\infty$

D. current index

Q12 — MSQ

Which are the three range-query overlap cases?

A. No overlap

B. Complete overlap

C. Partial overlap

D. Binary overlap

Q13 — MCQ

Large sparse values but relative order matters:

A. Coordinate Compression

B. BFS

C. KMP

D. Heap Sort only

Q14 — MCQ

For frequent dynamic range minimum queries, prefer:

A. Prefix Sum

B. Segment Tree

C. XOR only

D. Trie

Q15 — MCQ

Lazy Propagation is mainly used for:

A. efficient range updates in Segment Trees

B. string prefix matching

C. graph cloning

D. binary search

PRACTICE SOLUTIONS

Q1

No updates:

$$\boxed{A}$$

Q2

$$\boxed{B}$$

Q3

$$\boxed{B}$$

Q4

$$\boxed{B}$$

Q5

Query moves toward zero:

$$ \boxed{B} $$

Q6

Update moves to larger responsible nodes:

$$ \boxed{B} $$

Q7

A, B, C.

$$ \boxed{A,B,C} $$

Q8

$$ \boxed{A} $$

Q9

$$ \boxed{B} $$

Q10

0 is additive identity.

$$ \boxed{A} $$

Q11

For minimum, no-overlap should not lower result.

$$ \boxed{B} $$

Q12

$$ \boxed{A,B,C} $$

Q13

$$ \boxed{A} $$

Q14

$$ \boxed{B} $$

Q15

\[\boxed{A} \]

Ø=ÑØTES TO WRITE

```text id="r1961" RANGE QUERY STRUCTURES

STATICSUM: Prefix Sum Build O(n) Query O(1) Update O(n)

FENWICK / BIT: Dynamic prefix/range sums 1-based indexing

lowbit(i): i & -i

Query: while i > 0: sum += tree[i] i -= i & -i

Update: while i <= n: tree[i] += delta i += i & -i

Both: O(log n)

Range: prefix(R) - prefix(L-1)

SEGMENT TREE: Dynamic range aggregates

Build: O(n)

Point Update: O(log n)

Range Query: O(log n)

Three query cases:

1. No overlap
2. Complete overlap
3. Partial overlap

No-overlap identity: Sum !' 0 Min !' +" Max !' -"

LAZY PROPAGATION:range updates delay child updates

COORDINATE COMPRESSION: huge values !' small ranks preserve ordering

---

# &¡ 3 0 - S E C O N D R E V I S I O N

1. Prefix Sum is best for static range sums.
2. Prefix Sum updates can cost \(O(n)\).
3. Fenwick = Binary Indexed Tree.
4. Fenwick usually uses 1-based indexing.
5. `lowbit(i) = i & -i`.
6. Fenwick query subtracts lowbit.
7. Fenwick update adds lowbit.
8. Both Fenwick operations are \(O(\log n)\).
9. F e n w i c k r a n g e = p r e f i x ( R ) " p r e f i x ( L" 1 ) .
10. Standard Fenwick updates by delta.
11. Segment Tree stores interval summaries.
12. Segment Tree build = \(O(n)\).
13. Point update = \(O(\log n)\).
14. Range query = \(O(\log n)\) standard.
15. Query cases: no/complete/partial overlap.
```

```
16. Correct identity value matters.
17. Fenwick is simpler for dynamic sums.
18. Segment Tree is more flexible.
19. Lazy Propagation handles range updates efficiently.
20. Coordinate Compression maps huge values to ranks.
21. Inversion counting can use Merge Sort or Fenwick.
22. Don't use advanced trees when Prefix Sum is enough.

 #  '    P l a c e m e n t   M o d u l e   1 9   C o m p l e t e

## Next Placement Module:

# **MODULE 20 — ADVANCED GRAPH INTERVIEW PATTERNS: SCC, BRIDGES, ARTICULATION POINTS + UNION-FIND
APPLICATIONS**

Next we'll cover:

- strongly connected components,
- Kosaraju intuition,
- Tarjan SCC awareness,
- bridges,
- articulation points,
- discovery time / low-link values,
- critical connections,
- network failure questions,
- advanced DSU applications,
- redundant edges,
- dynamic connectivity,
- and how to recognize when ordinary BFS/DFS is not enough.

**Type `NEXT` to continue.**
```

USER

NEXT

ASSISTANT

PLACEMENT MODULE 20 — ADVANCED GRAPH INTERVIEW PATTERNS: SCC, BRIDGES, ARTICULATION POINTS + ADVANCED DSU

Placement Priority: & & & & & & & & & &

These topics are more advanced than ordinary BFS/DFS, but they matter for stronger coding interviews.

The big idea is:

> **Sometimes knowing that two nodes are connected is not enough.**

We may need to know:

- which directed nodes are mutually reachable,
- which edge is critical,
- which vertex is critical,
- which network connection is redundant,
- how connectivity changes as edges are added.

The key tools are:

\[\boxed{\text{SCC}} \]

\[\boxed{\text{Bridges}} \]

\[\boxed{\text{Articulation Points}} \]

\[\boxed{\text{DSU}} \]

---

1. GRAPH CONNECTIVITY RECAP

In an undirected graph:

If:

```
TEXT
A -- B -- C
```

all three belong to the same connected component.

DFS/BFS is enough.

But directed graphs are different.

Example:

```
TEXT
 A  !’   B  !’   C
!‘                |
% % % % % % % % %
```

Here every node can reach every other node.

This forms a:

\[\boxed{\text{Strongly Connected Component}} \]

PART A — STRONGLY CONNECTED COMPONENTS

2. WHAT IS AN SCC?

In a directed graph, a Strongly Connected Component is a maximal set of vertices such that:

\[\boxed{\text{every vertex can reach every other vertex}} \]

Example:

```
TEXT
 0  !’   1
!‘      !“
% % %    2

 2  !’   3  !’   4
        !‘      !“
        % % % % %
```

Possible SCCs:

```text
{0,1,2}
{3,4}
```

---

3. "STRONGLY" MATTERS

Suppose:

```text
 A  !′   B
```

A can reach B.

But B cannot reach A.

Therefore:

```text
{A,B}
```

is **not** one SCC.

They are separate SCCs.

---

4. WHERE SCC APPEARS

Interview signals:

- mutually reachable nodes,
- strongly connected groups,
- directed network cycles,
- compress a directed graph,
- dependency groups,
- minimum starting points in directed systems,
- deadlock/dependency analysis.

PART B — KOSARAJU'S ALGORITHM

One of the easiest SCC algorithms to understand.

5. HIGH-LEVEL IDEA

Kosaraju uses:

\[\boxed{\text{2 DFS passes}} \]

plus:

\[\boxed{\text{transpose/reverse graph}} \]

Steps:

1. DFS original graph and record finishing order.
2. Reverse every edge.
3. Process vertices in reverse finishing order.
4. Each DFS in reversed graph gives one SCC.

---

6. WHY FINISHING ORDER?

Suppose SCCs form a DAG after compression.

Finishing times help us start from the correct SCC so DFS on the reversed graph cannot leak into an unprocessed component incorrectly.

For interviews, you mainly need this intuition:

> **First DFS determines safe processing order.**

---

7. TRANSPOSE GRAPH

Original:

```text
A  !′   B
B  !′   C
```

Transpose:

```text
A  !•   B
B  !•   C
```

Every edge:

\[u\rightarrow v \]

becomes:

\[v\rightarrow u \]

---

8. KOSARAJU TEMPLATE

```python
def kosaraju(n, graph):
    visited = [False] * n
    order = []

    def dfs1(node):
        visited[node] = True

        for nei in graph[node]:
            if not visited[nei]:
```

```
                dfs1(nei)

        order.append(node)

    for node in range(n):
        if not visited[node]:
            dfs1(node)

    reverse_graph = [[] for _ in range(n)]

    for u in range(n):
        for v in graph[u]:
            reverse_graph[v].append(u)

    visited = [False] * n
    components = []

    def dfs2(node, comp):
        visited[node] = True
        comp.append(node)

        for nei in reverse_graph[node]:
            if not visited[nei]:
                dfs2(nei, comp)

    while order:
        node = order.pop()

        if not visited[node]:
            comp = []
            dfs2(node, comp)
            components.append(comp)

    return components
```

---

9. COMPLEXITY

First DFS:

\[O(V+E) \]

Reverse graph:

\[O(V+E) \]

Second DFS:

\[O(V+E) \]

Total:

\[\boxed{O(V+E)} \]

Space:

\[O(V+E) \]

---

10. KOSARAJU MEMORY TRICK

```text
DFS
!'  F i n i s h  o r d e r
!'  R e v e r s e  e d g e s
!'  D F S  i n  r e v e r s e  f i n i s h  o r d e r
```

Mnemonic:

> **Finish !' Flip !' Find**

PART C — TARJAN SCC AWARENESS

There is another \(O(V+E)\) SCC algorithm:

\[\boxed{\text{Tarjan's Algorithm}} \]

It uses:

- discovery time,
- low-link value,
- stack.

Unlike Kosaraju:

> **Tarjan finds SCCs in one DFS traversal.**

For placement preparation:

- know Tarjan exists,
- understand low-link concepts,
- prioritize Kosaraju first unless asked specifically.

Low-link becomes especially important for Bridges and Articulation Points.

PART D — CONDENSATION GRAPH

11. COMPRESS SCCs

Each SCC can be treated as one super-node.

Example:

```text
S C C 1  !'  S C C 2  !'  S C C 3
```

Important property:

\[\boxed{\text{SCC condensation graph is always a DAG}} \]

Why?

If condensed SCC nodes formed a directed cycle, those SCCs would actually all be mutually reachable and should have been one SCC.

This is a useful interview theorem.

PART E — BRIDGES

Now move to **undirected graphs**.

12. WHAT IS A BRIDGE?

An edge is a bridge if removing it increases the number of connected components.

Example:

```text
0 ---- 1 ---- 2
       |
       3
      / \
     4---5
```

Edge:

```text
0-1
```

may be a bridge.

If removed:

```text
0
```

becomes separated.

---

13. NON-BRIDGE EXAMPLE

Triangle:

```text
0
|\
| \
1--2
```

Remove:

```text
0-1
```

0 can still reach 1 through:

```TEXT
 0  !'   2  !'   1
```

Therefore that edge is not a bridge.

---

14. BRIDGE INTUITION

An edge is critical when:

> **the child's DFS subtree has no alternative way to reach the current node or any ancestor.**

This leads to:

\[\boxed{\text{Discovery Time + Low Value}} \]

PART F — DISCOVERY TIME

15. `tin[u]`

When DFS first visits node \(u\):

```TEXT
tin[u] = current_time
```

Example:

```TEXT
 n o d e   0  !'   t i n   0
 n o d e   1  !'   t i n   1
 n o d e   2  !'   t i n   2
```

It records DFS discovery order.

PART G — LOW-LINK VALUE

16. WHAT IS `low[u]`?

`low[u]` represents the earliest discovery time reachable from:

- node \(u\),
- its DFS subtree,
- using tree edges,
- and at most a back edge upward.

Simplified intuition:

> **How high can this subtree climb without using the parent edge?**

17. INITIALIZATION

When visiting u:

$$\boxed{low[u]=tin[u]}$$

18. BACK EDGE UPDATE

If u has an already visited neighbor v that is not its parent:

$$\boxed{low[u]=\min(low[u],tin[v])}$$

Because u or its subtree can reach ancestor v.

19. CHILD UPDATE

After DFS on child v:

$$\boxed{low[u]=\min(low[u],low[v])}$$

PART H — BRIDGE CONDITION

Suppose DFS tree edge:

$$u-v$$

where u is parent and v child.

If:

$$\boxed{low[v]>tin[u]}$$

then edge:

$$(u,v)$$

is a bridge.

20. WHY `>`?

If:

$$low[v]\le tin[u]$$

then child subtree can reach:

- u, or
- an ancestor of u,

through another path.

So parent edge is not essential.

But if:

$$low[v]>tin[u]$$

there is no alternative route upward.

Removing edge disconnects the subtree.

PART I — BRIDGE CODE

```python
def find_bridges(n, graph):
    timer = 0
    visited = [False] * n
    tin = [-1] * n
    low = [-1] * n
    bridges = []

    def dfs(node, parent):
        nonlocal timer

        visited[node] = True
        tin[node] = low[node] = timer
        timer += 1

        for nei in graph[node]:
            if nei == parent:
                continue

            if visited[nei]:
                low[node] = min(low[node], tin[nei])

            else:
                dfs(nei, node)

                low[node] = min(low[node], low[nei])

                if low[nei] > tin[node]:
                    bridges.append((node, nei))

    for node in range(n):
        if not visited[node]:
            dfs(node, -1)

    return bridges
```

Time:

$$\boxed{O(V+E)}$$

PART J — CRITICAL CONNECTIONS

A common interview problem asks:

> **Find all network connections whose removal disconnects part of the network.**

That is directly:

$$\boxed{\text{Bridge Detection}}$$

The famous "Critical Connections in a Network" pattern is exactly this.

PART K — ARTICULATION POINTS

21. WHAT IS AN ARTICULATION POINT?

A vertex whose removal increases the number of connected components.

Also called:

$$\boxed{\text{Cut Vertex}}$$

Example:

```text
0 --- 1 --- 2
      |
      3
```

Remove node 1:

```text
0    2    3
```

graph breaks apart.

Therefore 1 is an articulation point.

22. BRIDGE VS ARTICULATION

Bridge

Critical:

$$\boxed{\text{edge}}$$

Articulation Point

Critical:

$$\boxed{\text{vertex}}$$

PART L — ARTICULATION CONDITION

For non-root node u:

If child v satisfies:

$$\boxed{low[v]\ge tin[u]}$$

then u is an articulation point.

Notice:

Bridge

$$low[v]>tin[u]$$

Articulation

$$ low[v]\ge tin[u] $$

This distinction is important.

23. WHY `>=` FOR ARTICULATION?

If child subtree can only come back to u, but not above u:

$$ low[v]=tin[u] $$

then removing u still disconnects that subtree.

So equality still makes u critical.

PART M — SPECIAL ROOT CASE

DFS root is special.

Root is an articulation point if it has:

$$ \boxed{\text{more than one DFS child}} $$

Why?

Each child represents a separate DFS subtree.

Removing root separates them.

24. ROOT TRAP

Do **not** apply ordinary:

$$ low[child]\ge tin[root] $$

logic directly to root.

Use:

$$ \boxed{childCount>1} $$

PART N — ARTICULATION CODE

PYTHON

```python
def articulation_points(n, graph):
    timer = 0
    visited = [False] * n
    tin = [-1] * n
    low = [-1] * n
    is_cut = [False] * n

    def dfs(node, parent):
        nonlocal timer

        visited[node] = True
        tin[node] = low[node] = timer
        timer += 1
```

```
        children = 0

        for nei in graph[node]:
            if nei == parent:
                continue

            if visited[nei]:
                low[node] = min(low[node], tin[nei])

            else:
                dfs(nei, node)

                low[node] = min(low[node], low[nei])

                if parent != -1 and low[nei] >= tin[node]:
                    is_cut[node] = True

                children += 1

        if parent == -1 and children > 1:
            is_cut[node] = True

    for node in range(n):
        if not visited[node]:
            dfs(node, -1)

    return [i for i in range(n) if is_cut[i]]
```

Time:

\[\boxed{O(V+E)} \]

PART O — BRIDGES VS ARTICULATION TABLE

Concept	Critical item	Condition
Bridge	Edge \((u,v)\)	low[v] > tin[u]
Articulation	Vertex \(u\)	low[v] >= tin[u]
Articulation root	Root vertex	DFS children > 1

Mnemonic:

```text
B r i d g e             !'   >
A r t i c u l a t i o n  !'   > =
```

PART P — WHAT IS A BACK EDGE?

In undirected DFS, if current node connects to an already visited non-parent node, that's a back edge to an ancestor.

Example:

```text
0
|\
| \
1--2
```

DFS:

```text
 0  !'   1  !'   2
```

At 2, edge:

```text
 2  !'   0
```

is a back edge.

This tells subtree:

> **"I have another route upward."**

Therefore it can lower `low[2]`.

PART Q — WHY LOW-LINK IS POWERFUL

`low[]` answers:

> **Can this subtree reconnect to an ancestor without using the parent connection?**

That single idea powers:

- bridges,
- articulation points,
- Tarjan SCC variants.

Memory:

\[\boxed{\text{LOW = highest/earliest ancestor reachable}} \]

PART R — COMMON LOW-LINK TRAP

For visited neighbor:

Use:

\[tin[nei] \]

not blindly:

\[low[nei] \]

in standard undirected bridge logic.

Typical:

```python
low[node] = min(low[node], tin[nei])
```

for back edge.

For DFS tree child after recursion:

```python
low[node] = min(low[node], low[nei])
```

Keep these two cases separate.

PART S — ADVANCED DSU APPLICATIONS

You already know:

- `find`
- `union`
- path compression
- union by size/rank

Now let's use DSU for interview patterns.

---

25. DSU BEST SIGNAL

Question repeatedly asks:

```text
Are u and v connected?
Merge these groups.
How many groups remain?
Does this new edge create a cycle?
```

Think:

$$ \boxed{\text{DSU}} $$

PART T — REDUNDANT CONNECTION

Edges arrive one by one.

For edge:

$$ (u,v) $$

if:

$$ find(u)=find(v) $$

then endpoints are already connected.

Therefore this edge creates a cycle.

Return it.

Pattern:

$$ \boxed{\text{Online undirected cycle detection !' DSU}} $$

PART U — NUMBER OF COMPONENTS

Start:

$$ components=n $$

Every successful union of two different roots:

$$ components-- $$

At any moment:

$$ \boxed{\text{components = number of DSU sets}} $$

PART V — MAKE NETWORK CONNECTED

26. PROBLEM FAMILY

You have n computers and cables.

Can you reconnect cables so all computers become connected?

Minimum number of edges needed to connect c components:

$$ \boxed{c-1} $$

But you need enough total edges.

A connected graph on n nodes requires at least:

$$ \boxed{n-1} $$

edges.

So first check:

$$ E\ge n-1 $$

Then DSU finds number of components.

Answer:

$$ components-1 $$

PART W — NUMBER OF PROVINCES

Given connectivity matrix.

Can solve using:

- DFS,
- BFS,
- DSU.

Using DSU:

For every connected pair:

```
TEXT
union(i,j)
```

Then count roots.

This is a good interview follow-up:

> **"Can you solve it another way?"**

PART X — ACCOUNTS MERGE

27. RECAP

Accounts sharing an email belong to same group.

Use:

```
TEXT
 e m a i l  !'  a c c o u n t  i n d e x
```

If email already seen:

```
TEXT
union(currentAccount, previousAccount)
```

Then group emails by DSU root.

Pattern:

\[\boxed{\text{HashMap + DSU}} \]

PART Y — DYNAMIC ISLANDS

28. PROBLEM

Grid starts as all water.

Land cells are added one by one.

After each addition, report number of islands.

Re-running DFS after every addition can be expensive.

Use DSU.

When adding new land:

\[islands++ \]

For each neighboring land:

If roots differ:

```text
union
islands--
```

Return island count after every update.

Pattern:

\[\boxed{\text{Dynamic components !’ DSU}} \]

PART Z — KRUSKAL CONNECTION

Remember:

Kruskal:

```text
sort edges
for smallest edge:
    if endpoints in different DSU sets:
        union
        take edge
```

DSU provides fast cycle prevention.

So:

\[\boxed{\text{Kruskal = Greedy + DSU}} \]

PART AA — OFFLINE CONNECTIVITY

Sometimes queries ask connectivity after removing edges or under changing thresholds.

A powerful advanced technique:

> **process operations in reverse.**

Example:

- forward operations delete edges,
- reverse processing adds edges.

DSU handles additions easily.

This is called an:

\[\boxed{\text{Offline DSU technique}} \]

Very useful in competitive programming and stronger interviews.

PART AB — UNION BY SIZE RECAP

```python
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])

        return self.parent[x]

    def union(self, a, b):
        ra = self.find(a)
        rb = self.find(b)

        if ra == rb:
            return False

        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra

        self.parent[rb] = ra
        self.size[ra] += self.size[rb]

        return True
```

With path compression + union by size:

$$ \boxed{O(\alpha(n))} $$

amortized per operation.

Practically:

> **almost constant.**

PART AC — BFS/DFS VS DSU

BFS/DFS	DSU
Traverses graph	Maintains components
Finds paths/regions	Fast connectivity
Good static graph	Excellent dynamic merging
Can inspect neighbors	Doesn't preserve actual paths
Components $O(V+E)$	Near-constant union/find

---

29. IMPORTANT LIMITATION OF DSU

DSU can answer:

> **Are A and B connected?**

But not naturally:

> **What exact path connects A to B?**

For path retrieval, use graph traversal.

PART AD — SCC VS CONNECTED COMPONENTS

Very important.

Undirected Connected Component

Every pair has path because edges are bidirectional.

Directed SCC

Every pair must be mutually reachable respecting edge directions.

So:

```text
Connected Components !' undirected BFS/DFS
SCC                  !' directed specialized algorithms
```

PART AE — BRIDGE VS MST EDGE

Do not confuse these.

Bridge in original graph

Removing it disconnects graph.

Edge in an MST

Chosen because it contributes to minimum spanning structure.

Not every MST edge is necessarily a bridge of the original graph.

Example triangle:

```text
A
|\
| \
B--C
```

An MST uses two edges.

But no individual triangle edge is a bridge in original graph.

PART AF — ARTICULATION POINT VS HIGH DEGREE

A high-degree vertex is not automatically an articulation point.

Example:

```text
   B
  / \
 A---C
  \ /
   D
```

A node may have several neighbors but alternate paths keep graph connected.

Articulation is about:

\[\boxed{\text{connectivity after removal}} \]

not degree alone.

PART AG — SCC APPLICATION: DEADLOCK/DEPENDENCY GROUPS

Suppose directed dependency graph contains:

```text
A  !′   B
B  !′   C
C  !′   A
```

All three form one SCC.

This means they are mutually dependent.

Compressing SCCs can simplify analysis into DAG structure.

PART AH — TOPOLOGICAL SORT + SCC

Remember:

Topological sorting works only on DAG.

If directed graph contains cycles:

1. Find SCCs.
2. Compress each SCC.
3. Resulting condensation graph is DAG.
4. Topological sort that DAG if needed.

This is an advanced but important connection.

PART AI — WHEN ORDINARY DFS IS NOT ENOUGH

Use ordinary DFS if asking:

- reachable?

- component count?
- explore region?

Use SCC if:

> **mutual reachability in directed graph.**

Use bridge algorithm if:

> **which edge is critical?**

Use articulation algorithm if:

> **which vertex is critical?**

Use DSU if:

> **connectivity changes through repeated unions.**

PART AJ — INTERVIEW DECISION TREE

```text
Graph connectivity question
%
%%   Directed?
%      %
%      %%   Need mutually reachable groups?
%      %      !' SCC
%      %
%      %%   Need dependency ordering?
%      %      !' Topological Sort
%      %
%      %%   Need cycle only?
%             !' DFS colors / Kahn
%
%%   Undirected?
       %
       %%   Need components?
       %      !' BFS / DFS / DSU
       %
       %%   Critical edge?
       %      !' Bridges
       %
       %%   Critical vertex?
       %      !' Articulation Point
       %
       %%   Dynamic merges/connectivity?
              !' DSU
```

PART AK — COMMON INTERVIEW TRAPS

30. TRAP 1 — SCC ON UNDIRECTED GRAPH

SCC terminology is specifically useful for directed graphs.

For undirected graphs, use connected components.

31. TRAP 2 — ONE-WAY REACHABILITY

If:

```text
A  !′  B
```

that does not put A and B in same SCC.

Need:

\[A\leadsto B \]

and:

\[B\leadsto A \]

32. TRAP 3 — WRONG KOSARAJU ORDER

Second pass must process vertices by:

\[\boxed{\text{decreasing finishing time}} \]

Using stack pop naturally gives that order.

33. TRAP 4 — FORGETTING TO TRANSPOSE

Kosaraju requires reversing all directed edges before second DFS.

34. TRAP 5 — BRIDGE CONDITION

Correct:

\[\boxed{low[child]>tin[parent]} \]

Not `>=`.

35. TRAP 6 — ARTICULATION CONDITION

For non-root:

\[\boxed{low[child]\ge tin[node]} \]

36. TRAP 7 — ROOT ARTICULATION

Root needs:

\[\boxed{more\ than\ one\ DFS\ child} \]

37. TRAP 8 — PARENT EDGE AS BACK EDGE

In undirected DFS:

```python
if nei == parent:
    continue
```

Otherwise parent connection may incorrectly reduce low values.

38. TRAP 9 — DISCONNECTED GRAPH

Bridge/articulation algorithms must run DFS from every unvisited vertex.

Not only node 0.

39. TRAP 10 — DSU FOR DIRECTED REACHABILITY

DSU ignores direction.

It cannot answer general directed reachability/SCC questions.

40. TRAP 11 — PATH COMPRESSION WITHOUT ASSIGNMENT

Correct:

```python
parent[x] = find(parent[x])
```

not merely:

```python
find(parent[x])
```

if you actually want compression.

PART AL — INTERVIEW EXPLANATION: BRIDGES

Strong answer:

> **"I use DFS with discovery time and low-link values. `tin[u]` records when a node is first visited, while `low[u]` represents the earliest ancestor reachable from `u`'s subtree without relying on the parent edge. After DFSing a child `v`, if `low[v] > tin[u]`, then `v`'s subtree has no alternate route back to `u` or any ancestor, so removing edge `(u,v)` disconnects that subtree. Therefore it is a bridge. The total complexity is \(O(V+E)\)."**

PART AM — INTERVIEW EXPLANATION: SCC

> **"For strongly connected components, I can use Kosaraju's algorithm. First I DFS the original graph and record vertices by finishing time. Then I reverse every edge. Processing vertices in decreasing finishing order on the reversed graph isolates one strongly connected component per DFS. Both DFS passes and graph reversal are linear, so the total complexity is \(O(V+E)\)."**

PART AN — INTERVIEW EXPLANATION: DSU

> **"Since the problem repeatedly merges components and asks connectivity, I would use Disjoint Set Union. `find` returns a component representative, and `union` merges two representatives. Path compression and union by size keep the trees shallow, making each operation \(O(\alpha(n))\) amortized, which is effectively constant for practical inputs."**

PART AO — MASTER GRAPH PATTERNTABLE

Signal	Technique
Reachability	BFS/DFS
Undirected components	BFS/DFS/DSU
Directed mutual reachability	SCC
Directed dependencies	Topological Sort
Directed cycle	DFS colors/Kahn
Critical edge	Bridge algorithm
Critical vertex	Articulation Point
Dynamic merging	DSU
Extra edge causing undirected cycle	DSU
MST	Prim/Kruskal
Kruskal cycle check	DSU
Dynamic islands	DSU
SCC compressed graph	DAG

PART AP — MUST-DO QUESTIONS

SCC

1. Strongly Connected Components

2. Kosaraju Algorithm
3. Tarjan SCC awareness
4. Count SCCs
5. Condensation Graph concept
6. Mother Vertex-related reachability patterns

Bridges

7. Find Bridges
8. Critical Connections in a Network
9. Check whether an edge is a bridge
10. Bridge count

Articulation

11. Find Articulation Points
12. Network cut vertices
13. Critical server/router style problems

DSU

14. Implement DSU
15. Redundant Connection
16. Number of Provinces
17. Number of Connected Components
18. Make Network Connected
19. Accounts Merge
20. Number of Islands II
21. Kruskal MST
22. Dynamic Connectivity
23. Offline edge-query awareness

PLACEMENT PRACTICE SET

Q1 — MCQ

A strongly connected component is mainly defined for:

A. Directed graphs B. Arrays C. Heaps D. Strings

Q2 — MCQ

Nodes \(u,v\) belong to the same SCC if:

A. \(u\) can reach \(v\) only

B. \(v\) can reach \(u\) only

C. each can reach the other

D. their values are equal

Q3 — MCQ

Kosaraju uses:

A. two DFS passes and graph transpose

B. Dijkstra only

C. Binary Search

D. Heap Sort

Q4 — MCQ

Kosaraju's complexity is:

A. $O(V+E)$

B. $O(VE)$

C. $O(V^2E)$

D. $O(2^V)$

Q5 — MCQ

The SCC condensation graph is always:

A. complete

B. DAG

C. undirected

D. tree necessarily

Q6 — MCQ

An edge whose removal increases number of components is called:

A. bridge

B. heap edge

C. shortest edge

D. prefix edge

Q7 — MCQ

For DFS tree edge $u-v$, bridge condition is:

A. `low[v] > tin[u]`

B. `low[v] < tin[u]`

C. `low[v] == 0`

D. `tin[v] > V`

Q8 — MCQ

For a non-root articulation point \(u\), useful condition is:

A. `low[v] >= tin[u]`

B. `low[v] < 0`

C. `low[v] > V`

D. `tin[u] == 0` always

Q9 — MCQ

DFS root is articulation point when it has:

A. at least one neighbor

B. more than one DFS child

C. exactly zero children

D. maximum weight

Q10 — MSQ

Which use discovery time and low-link concepts?

A. Bridge detection

B. Articulation points

C. Tarjan-style algorithms

D. Bubble Sort

Q11 — MCQ

Repeated component merge/query operations strongly suggest:

A. DSU

B. KMP

C. LCS

D. Sliding Window

Q12 — MCQ

In DSU, if:

\[find(u)=find(v) \]

before adding an undirected edge \(u-v\), then:

A. it joins two different components

B. it creates a cycle

C. graph becomes directed

D. it creates an SCC

Q13 — NAT

A graph currently has 7 connected components. Minimum number of successful component-joining edges needed to make it connected?

Q14 — MSQ

Which can be solved naturally with DSU?

A. Redundant Connection

B. Kruskal

C. Dynamic Islands

D. Exact shortest directed path

Q15 — MCQ

A high-degree vertex:

A. must always be an articulation point

B. is never an articulation point

C. may or may not be an articulation point

D. must be in every MST

PRACTICE SOLUTIONS

Q1

\[\boxed{A} \]

Q2

Mutual reachability is required.

\[\boxed{C} \]

Q3

$$ \boxed{A} $$

Q4

$$ \boxed{A} $$

Q5

$$ \boxed{B} $$

Q6

$$ \boxed{A} $$

Q7

Bridge:

$$ \boxed{low[v]>tin[u]} $$

Answer:

$$ \boxed{A} $$

Q8

Non-root articulation:

$$ \boxed{low[v]\ge tin[u]} $$

Answer:

$$ \boxed{A} $$

Q9

$$ \boxed{B} $$

Q10

A, B, C.

$$ \boxed{A,B,C} $$

Q11

$$ \boxed{A} $$

Q12

Already connected means extra edge closes a cycle.

$$\boxed{B}$$

Q13

To connect c components:

$$c-1$$

So:

$$7-1=\boxed{6}$$

Q14

A, B, C.

$$\boxed{A,B,C}$$

Q15

Degree alone does not determine articulation status.

$$\boxed{C}$$

Ø=ÜÓTES TO WRITE

```TEXT
SCC
= Strongly Connected Component
Directed graph
Every node can reach every other

KOSARAJU:
1. DFS original
2. Store finish order
3. Reverse graph
4. DFS in decreasing finish order

Mnemonic:
 F i n i s h  !’   F l i p  !’   F i n d

O(V+E)

SCC condensation graph:
always DAG


LOW-LINK:

tin[u]
= discovery time
```

```
low[u]
= earliest ancestor discovery
reachable from subtree
without relying on parent edge


BRIDGE:
low[child] > tin[parent]

ARTICULATION:
non-root:
low[child] >= tin[node]

root:
DFS children > 1


DSU:
find + union

Path Compression
Union by Size/Rank

Use DSU:
dynamic connectivity
redundant edge
Kruskal
dynamic islands
accounts merge

DSU complexity:
O(alpha(n)) amortized
"H   c o n s t a n t
```

&β30-SECOND REVISION

1. SCC is for directed mutual reachability.
2. Kosaraju = two DFS passes.
3. First pass records finish order.
4. Reverse all edges.
5. Second pass gives SCCs.
6. Kosaraju = \(O(V+E)\).
7. SCC condensation graph is DAG.
8. Bridge = critical edge.
9. Articulation Point = critical vertex.
10. `tin` = discovery time.
11. `low` = earliest reachable ancestor time.
12. Bridge condition = `low[child] > tin[parent]`.
13. Articulation condition = `low[child] >= tin[node]`.
14. Root articulation needs more than one DFS child.
15. Back edges reduce low values.
16. DSU = dynamic connectivity.
17. DSU supports find + union.

18. Path compression flattens trees.
19. Union by size/rank keeps them shallow.
20. Redundant Connection !' DSU.
21. Dynamic Islands !' DSU.
22. Kruskal = Greedy + DSU.
23. DSU cannot recover actual paths.
24. High degree does not automatically mean articulation.
25. BFS/DFS handles ordinary traversal; low-link handles critical connectivity.

' Placement Module 20 Complete

Next Placement Module:

MODULE 21 — DESIGN-STYLE DSA: LRU CACHE + LFU CACHE + HASHMAP/DLL COMBINATIONS

Next we'll cover:

- why ordinary HashMap is not enough,
- LRU Cache,
- HashMap + Doubly Linked List,
- \(O(1)\) get/put design,
- dummy head/tail technique,
- LFU Cache intuition,
- frequency buckets,
- design questions like Min Stack,
- Randomized Set,
- Time-Based Key-Value Store,
- browser-history style structures,
- and how to approach **data-structure design interview questions** systematically.

Type `NEXT` to continue.

USER

NEXT

ASSISTANT

PLACEMENT MODULE 21 — DESIGN-STYLE DSA: LRU CACHE + LFU CACHE + HASHMAP/DLL COMBINATIONS

Placement Priority: & & & & &

Design-style DSA questions are important because interviewers are testing whether you can combine data structures, not just solve isolated algorithm problems.

The key thought process is:

\[\boxed{\text{Required Operations !' Required Time Complexity !' Choose Structures}} \]

Example:

> **Design a cache with `get()` and `put()` in \(O(1)\).**

A HashMap alone gives fast lookup.

But it does **not** maintain usage order efficiently.

So we combine:

\[\boxed{\text{HashMap + Doubly Linked List}} \]

---

1. HOW TO APPROACH DATA-STRUCTURE DESIGN QUESTIONS

Whenever interviewer says:

> **Design a structure supporting these operations...**

Do this:

```text id="dsa2101"

1. List operations
2. Write required complexity
3. Identify what information must be maintained
4. Choose one structure per requirement
5. Combine them

Example:

```text id="dsa2102"
Need:
 l o o k u p   b y   k e y   !'   O ( 1 )
 r e m o v e   a r b i t r a r y   c a c h e d   n o d e   !'   O ( 1 )
 m a i n t a i n   r e c e n t   o r d e r   !'   O ( 1 )

 H a s h M a p   !'   l o o k u p
 D o u b l y   L i n k e d   L i s t   !'   o r d e r / r e m o v a l

Combine them.
```

PART A — LRU CACHE

2. WHAT DOES LRU MEAN?

LRU:

\[\boxed{\text{Least Recently Used}} \]

Suppose cache capacity:
```

\[3 \]

Current items from most recent to least recent:

```text id="dsa2103" A !’ B !’ C

```
If new item D arrives and cache is full:

> remove C.

Because C was used least recently.

---

3. REQUIRED OPERATIONS

Typical API:

```text id="dsa2104"
get(key)
put(key, value)
```
```

Requirement:

\[ \boxed{O(1)} \]

for both.

---

## 4. WHY HASHMAP ALONE FAILS

HashMap provides:

```text id="dsa2105" key !’ value

```
Lookup:

\[
O(1)
\]

But how do we quickly know:

> which item is least recently used?

HashMap doesn't naturally preserve/update recency order in a portable interview design.

---

5. WHY LINKED LIST ALONE FAILS

A linked list can maintain usage order.

Example:

```text id="dsa2106"
Most Recent
      !”
 A  !”   B  !”   C
                !’
Least Recent
```
```

But finding key B requires traversal:

\[O(n) \]

So linked list alone fails lookup requirement.

---

6. COMBINE THEM

Use:

\[\boxed{\text{HashMap + Doubly Linked List}} \]

HashMap:

```text id="dsa2107" key !’ Node

```
Doubly Linked List:

```text id="dsa2108"
 M o s t   R e c e n t  !”  . . .  !”  L e a s t  R e c e n t
```

Now:

- locate node in \(O(1)\),
- remove node in \(O(1)\),
- move node in \(O(1)\),
- remove LRU in \(O(1)\).

PART B — WHY DOUBLY LINKED LIST?

7. WHY NOT SINGLY LINKED LIST?

Suppose we want to remove arbitrary node:

```text id="dsa2109" A !’ B !’ C

```
If we know pointer to B, singly linked list still needs B's previous node to remove it cleanly.

With doubly linked list:

```text id="dsa2110"
 A  !”   B  !”   C
```

B contains:

```text id="dsa2111" prev next

```
So removal is \(O(1)\).

That's why LRU uses:

\[
\boxed{\text{Doubly Linked List}}
```
```

```
\]

---

# PART C — DUMMY HEAD AND TAIL

This makes implementation much easier.

# 8. STRUCTURE

```text id="dsa2112"
 H E A D  !”  m o s t  r e c e n t  !”  . . .  !”  l e a s t  r e c e n t  !”  T A I L
```
```

HEAD and TAIL are dummy nodes.

They do not represent actual cache entries.

---

9. WHY DUMMY NODES?

Without dummy nodes, removing:

- first item,
- last item,
- only item,

needs special cases.

With dummy nodes:

> **every real node has both previous and next nodes.**

This greatly simplifies code.

PART D — LRU NODE

```python id="dsa2113" class Node: def **init**(self, key, value): self.key = key self.value = value self.prev = None self.next = None

```
Why store `key` inside node?

Because when removing LRU node from linked list, we also need to remove its key from HashMap.

PART E — CORE LINKED-LIST OPERATIONS

We need two helper operations:

Remove node

```python id="dsa2114"
def remove(self, node):
    prev_node = node.prev
    next_node = node.next

    prev_node.next = next_node
```

```
    next_node.prev = prev_node
```

---

10. ADD AS MOST RECENT

Place node just after dummy head:

```python id="dsa2115" def add_front(self, node): first = self.head.next

node.prev = self.head node.next = first

self.head.next = node first.prev = node

```
Now:

```text id="dsa2116"
 H E A D !”  n e w N o d e !”  p r e v i o u s M o s t R e c e n t . . .
```

PART F — GET OPERATION

11. `get(key)`

If key absent:

```text id="dsa2117" return -1

```
If present:

1. find node from HashMap,
2. remove it from current position,
3. move it to front,
4. return value.

Because access makes it:

\[
\boxed{\text{Most Recently Used}}
\]

---

12. GET CODE

```python id="dsa2118"
def get(self, key):
    if key not in self.cache:
        return -1

    node = self.cache[key]

    self.remove(node)
    self.add_front(node)

    return node.value
```

Time:

\[\boxed{O(1)} \]

PART G — PUT OPERATION

13. CASE 1 — KEY ALREADY EXISTS

Suppose:

```text id="dsa2119" put(A, newValue)

```
A already exists.

Update value.

Then make A most recent.

---

14. CASE 2 — NEW KEY

Create new node.

Add it to HashMap.

Add it to front.

If capacity exceeded:

> remove node just before tail.

That node is:

\[
\boxed{\text{Least Recently Used}}
\]

---

15. FULL LRU CACHE

```python id="dsa2120"
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = {}

        self.head = Node(0, 0)
        self.tail = Node(0, 0)

        self.head.next = self.tail
        self.tail.prev = self.head

    def remove(self, node):
```

```
        node.prev.next = node.next
        node.next.prev = node.prev

    def add_front(self, node):
        node.next = self.head.next
        node.prev = self.head

        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.cache:
            return -1

        node = self.cache[key]

        self.remove(node)
        self.add_front(node)

        return node.value

    def put(self, key, value):
        if key in self.cache:
            node = self.cache[key]
            node.value = value

            self.remove(node)
            self.add_front(node)
            return

        node = Node(key, value)

        self.cache[key] = node
        self.add_front(node)

        if len(self.cache) > self.capacity:
            lru = self.tail.prev

            self.remove(lru)
            del self.cache[lru.key]
```

Both:

\[\boxed{get=O(1)} \]

\[\boxed{put=O(1)} \]

PART H — LRU TRACE

Capacity:

\[2 \]

Operations:

```text id="dsa2121" put(1,10) put(2,20) get(1) put(3,30)

```

```
After first:

```text id="dsa2122"
MRU
```
```

```
1
LRU
```

After second:

```text id="dsa2123" MRU 2 !’ 1 LRU

```
After `get(1)`:

```text id="dsa2124"
MRU
 1  !’   2
LRU
```
```

Now:

```text id="dsa2125" put(3,30)

```
2 is least recently used.

Evict 2:

```text id="dsa2126"
MRU
 3  !’   1
LRU
```
```

PART I — LRU MEMORY TRICK

```text id="dsa2127" HashMap: WHERE is the node?

DLL: HOW RECENT is the node?

```
Together:

\[
\boxed{\text{Lookup + Order = LRU}}
\]

PART J — LRU COMMON TRAPS

16. TRAP 1 — GET DOES NOT UPDATE RECENCY

Wrong.

A successful `get()` means:

> this key was just used.

So it must move to MRU position.

---

17. TRAP 2 — UPDATE EXISTING KEY WITHOUT MOVING IT
```
```

```
A successful `put()` on existing key also counts as recent use under standard LRU behavior.

Move it.

---

# 18. TRAP 3 — REMOVE FROM LIST BUT NOT HASHMAP

When evicting:

```text id="dsa2128"
remove from DLL
AND
remove from HashMap
```
```

Otherwise stale entry remains.

---

19. TRAP 4 — STORE ONLY VALUE IN MAP

Better map:

```text id="dsa2129" key !’ Node

```
not:

```text id="dsa2130"
 k e y  !’   v a l u e
```
```

because we need immediate linked-list access.

---

## PART K — LFU CACHE

## 20. WHAT DOES LFU MEAN?

LFU:

\[ \boxed{\text{Least Frequently Used}} \]

Unlike LRU, eviction is based primarily on:

> **usage frequency.**

Example:

```text id="dsa2131" A used 5 times B used 2 times C used 2 times

```
A is safe.

Eviction candidate is B or C.

---
```

```
# 21. TIE BREAKER

Standard LFU Cache uses:

> if frequencies tie, evict least recently used among that frequency.

So it needs:

\[
\boxed{\text{Frequency + Recency}}
\]

This makes LFU more complex.

---

# PART L — LFU REQUIRED STRUCTURES

Typical \(O(1)\) LFU design uses:

### 1. Key Map

```text id="dsa2132"
 k e y  !’   n o d e
```
```

2. Frequency Map

```text id="dsa2133" frequency !’ doubly linked list

```
Each frequency bucket maintains LRU ordering.

3. `minFreq`

Tracks minimum frequency currently present.

---

22. EXAMPLE

Suppose:

```text id="dsa2134"
frequency 1:
 D  !”   E

frequency 2:
 B  !”   C

frequency 5:
A
```
```

If capacity full:

> **remove least recent node from `frequency 1`.**

No need to scan all frequencies because:

```text id="dsa2135" minFreq = 1

```

PART M — ACCESS IN LFU

Suppose node has:

```text id="dsa2136"
freq = 2
```
```

After `get(node)`:

1. remove from frequency-2 list,
2. increment frequency to 3,
3. insert into frequency-3 list,
4. update `minFreq` if necessary.

---

23. LFU CORE IDEA

```text id="dsa2137" key !’ node freq !’ ordered nodes minFreq !’ eviction bucket

```
This allows average \(O(1)\) `get` and `put` with careful implementation.

PART N — LRU VS LFU

LRU	LFU
Least recently used	Least frequently used
Tracks recency	Tracks frequency + recency
HashMap + one DLL	HashMap + many DLL buckets
Simpler	More complex
Evict oldest use	Evict lowest frequency

PART O — MIN STACK

You already saw it, but this is a classic **design question**.

Required:

```text id="dsa2138"
push()
pop()
top()
getMin()
```
```

all in:

\[ \boxed{O(1)} \]

---

## 24. SOLUTION 1 — TWO STACKS

Main stack:
```

```text id="dsa2139" values

```
Min stack:

```text id="dsa2140"
minimum values
```
```

When pushing \(x\):

If:

```text id="dsa2141" x <= current_min

```
push onto min stack too.

When popping:

If popped value equals current min:

> pop min stack.

---

25. SOLUTION 2 — PAIRS

Store:

```text id="dsa2142"
(value, minimum_so_far)
```
```

Example:

```text id="dsa2143" push 5 !' (5,5) push 3 !' (3,3) push 7 !' (7,3)

```
Then `getMin()`:

> second value of top pair.

PART P — RANDOMIZED SET

Common design interview question.

Need:

```text id="dsa2144"
insert(value)      O(1)
remove(value)      O(1)
getRandom()        O(1)
```
```

---

## 26. WHY HASHSET ALONE FAILS

Insert/remove expected \(O(1)\).

But uniform random element by index is not naturally \(O(1)\).

---
```

27. WHY ARRAY ALONE FAILS

Random access:

\[O(1) \]

But removing arbitrary value requires finding it:

\[O(n) \]

---

28. COMBINATION

Use:

\[\boxed{\text{Array + HashMap}} \]

HashMap:

```text id="dsa2145" value !' index

```
Array:

```text id="dsa2146"
values
```
```

---

## 29. O(1) REMOVE TRICK

Suppose:

```text id="dsa2147" array = [10,20,30,40]

```
Remove 20.

Deleting index 1 directly shifts elements.

Instead:

1. swap 20 with last value 40,
2. update 40's index in map,
3. pop last element,
4. remove 20 from map.

Becomes:

```text id="dsa2148"
[10,40,30]
```
```

No shifting.

\[\boxed{O(1)} \]

expected.

PART Q — RANDOMIZED SET PATTERN

```text id="dsa2149" HashMap !’ locate value Array !’ random index Swap-last !’ O(1) deletion

This pattern is very interview-important.

---

# PART R — TIME-BASED KEY-VALUE STORE

Common system-like DSA problem.

Operations:

```text id="dsa2150"
set(key, value, timestamp)
get(key, timestamp)
```

`get()` should return latest value whose timestamp:

\[\le requestedTimestamp \]

---

30. DATA STRUCTURE

For each key:

```text id="dsa2151" key !’ list of (timestamp, value)

If timestamps arrive increasing:

> append.

Then `get()` performs Binary Search.

---

# 31. COMPLEXITY

Set:

\[
\boxed{O(1)}
\]

amortized.

Get:

\[
\boxed{O(\log m)}
\]

where \(m\) = versions stored for that key.

Pattern:

\[
\boxed{\text{HashMap + Sorted History + Binary Search}}
\]

---
```
```

```
# PART S — BROWSER HISTORY DESIGN

Typical operations:

```text id="dsa2152"
visit(url)
back(steps)
forward(steps)
```
```

Possible structures:

Two stacks

or

Array + current index

Array version is often simplest.

Maintain:

```text id="dsa2153" history current last_valid

```
When visiting after going back:

> forward history must be discarded.

This tests state maintenance more than algorithm theory.

PART T — HIT COUNTER / RECENT EVENTS

Problem:

> count events in last 5 minutes.

Possible:

- Queue/Deque of timestamps,
- circular buckets for high throughput.

Basic interview approach:

```text id="dsa2154"
append timestamp
while front expired:
    pop left
```
```

Pattern:

\[ \boxed{\text{Time Window !' Queue/Deque}} \]

---

## PART U — MOVING AVERAGE FROM DATA STREAM

Given window size \(k\).

Need average of latest \(k\) values.

Maintain:
```

- Queue,
- running sum.

For each new value:

```text id="dsa2155" add value sum += value

if queue size > k: sum -= oldest

```
Average:

\[
\frac{sum}{queueSize}
\]

Operations:

\[
O(1)
\]

PART V — STOCK PRICE FLUCTUATION STYLE

Suppose operations:

- update timestamp price,
- current price,
- maximum price,
- minimum price.

Possible structures combine:

 - H a s h M a p t i m e s t a m p !’ c u r r e n t p r i c e ,
- ordered multiset / heaps with lazy deletion.

Key design lesson:

> Different queries may require different indexes over the same data.

PART W — DESIGN A STACK WITH INCREMENT

Question may ask:

```text id="dsa2156"
push
pop
increment bottom k elements
```
```

Naively incrementing \(k\) elements costs \(O(k)\).

Advanced solution uses lazy increment array.

Again, core idea:

\[ \boxed{\text{Store deferred information}} \]

similar philosophically to Lazy Propagation.
```

PART X — DESIGN QUESTIONS ARE COMBINATIONS

Many famous design questions are combinations of patterns you already know:

Problem	Main Structures
LRU Cache	HashMap + DLL
LFU Cache	HashMap + Frequency DLLs
Min Stack	Stack + Min tracking
Randomized Set	HashMap + Array
Time Map	HashMap + Binary Search
Moving Average	Queue + Running Sum
Median Stream	Two Heaps
Kth Largest Stream	Min Heap
Autocomplete	Trie
Browser History	Array/Stacks
Hit Counter	Queue / Buckets

PART Y — A UNIVERSAL DESIGN METHOD

32. STEP 1 — WRITE OPERATIONS

Example:

```text id="dsa2157" get(key) put(key,value)

```
---

33. STEP 2 — WRITE TARGET COMPLEXITY

```text id="dsa2158"
 g e t  !’   O ( 1 )
 p u t  !’   O ( 1 )
```
```

This immediately rules out certain structures.

---

## 34. STEP 3 — IDENTIFY REQUIREMENTS

For LRU:

```text id="dsa2159" fast key lookup fast arbitrary deletion fast ordering update fast least-recent retrieval

```
---
```

```
# 35. STEP 4 — ASSIGN STRUCTURE TO REQUIREMENT

```text id="dsa2160"
 l o o k u p  !’   H a s h M a p
 o r d e r i n g / r e m o v a l  !’   D L L
```
```

36. STEP 5 — DEFINE INVARIANTS

For LRU:

```text id="dsa2161" head.next = most recently used tail.prev = least recently used

map[key] = exact DLL node

```
If you can clearly state these invariants, implementation becomes much easier.

PART Z — DESIGN INTERVIEW COMMUNICATION

Before coding LRU, a strong explanation is:

> “A HashMap gives \(O(1)\) key lookup, but I also need to maintain recency and remove arbitrary nodes in \(O(1)\). A doubly linked list gives constant-time removal and insertion when I already have the node reference. I therefore map each key to its linked-list node. The node after the dummy head is most recently used, and the node before the dummy tail is least recently used. Every successful get or put moves that node to the front, and when capacity is exceeded I remove the node before the tail.”

This is better than immediately typing code.

PART AA — COMMON LRU INTERVIEW FOLLOW-UPS

37. WHAT IF CAPACITY = 0?

Then no real entries can remain.

Implementation should handle this without breaking dummy nodes.

---

38. WHY STORE KEY IN NODE?

For eviction:

```text id="dsa2162"
lru = tail.prev
```

We need:

```text id="dsa2163" del map[lru.key]

```
So node must know its key.

```
```

```
# 39. WHY NOT USE AN ORDERED DICTIONARY?

In real Python, `OrderedDict` can simplify implementation.

But interviews often want:

> demonstrate the underlying data structure design.

So learn:

\[
\boxed{\text{HashMap + DLL}}
\]

first.

---

# PART AB — COMMON DESIGN INTERVIEW TRAPS

# 40. TRAP 1 — MEETING TIME BUT NOT SPACE

Always discuss both:

```text id="dsa2164"
Time Complexity
Space Complexity
```
```

For LRU:

\[O(capacity) \]

space.

---

41. TRAP 2 — DATA STRUCTURE DOES ONE REQUIREMENT BUT NOT ALL

Example:

HashMap solves lookup but not recency.

Always test every required operation.

---

42. TRAP 3 — O(n) HIDDEN INSIDE "O(1)" DESIGN

Example:

```text id="dsa2165" list.remove(value)

```

```
may be \(O(n)\).

Even if rest is HashMap.

Interviewers watch for this.

---

# 43. TRAP 4 — ARRAY REMOVE FROM MIDDLE

For Randomized Set:
```

```text id="dsa2166"
pop(index)
```

from middle can shift elements:

\[O(n) \]

Use swap-with-last.

---

44. TRAP 5 — SINGLY LINKED LIST FOR LRU

Knowing exact node isn't enough for \(O(1)\) arbitrary removal unless previous node is also accessible.

Use DLL.

---

45. TRAP 6 — BINARY SEARCH ON UNSORTED TIMESTAMPS

TimeMap's histories must be ordered by timestamp.

If inserts aren't guaranteed ordered:

> **maintain ordering differently.**

PART AC — LRU VS CACHE IMPLEMENTATION REALITY

For interviews, LRU is primarily an algorithm/data-structure problem.

In production systems, real caches also involve:

- concurrency,
- expiration,
- memory limits,
- distributed consistency,
- persistence,
- sharding.

Do not bring these in unless interviewer asks system-design follow-ups.

Keep the DSA answer focused.

PART AD — MUST-DO DESIGN QUESTIONS

Core

1. Min Stack
2. LRU Cache
3. Randomized Set
4. Kth Largest in Stream

5. Median Finder
6. Moving Average

Important

7. Time-Based Key-Value Store
8. Browser History
9. Hit Counter
10. Design Circular Queue
11. Design Circular Deque
12. Trie
13. Word Dictionary

Advanced

14. LFU Cache
15. All O(1) Data Structure awareness
16. Snapshot Array
17. Stock Price Fluctuation
18. Underground System
19. Design Twitter
20. Custom Stack with Increment

PLACEMENT PRACTICE SET

Q1 — MCQ

Standard LRU Cache combines:

A. HashMap + Doubly Linked List

B. BFS + DFS

C. Trie + Heap

D. Segment Tree only

Q2 — MCQ

HashMap in LRU is mainly used for:

A. recency order

B. \(O(1)\) key-to-node lookup

C. sorting keys

D. shortest paths

Q3 — MCQ

Doubly Linked List is useful in LRU because:

A. arbitrary known node can be removed in \(O(1)\)

B. it provides binary search

C. it automatically hashes keys

D. it provides MST

Q4 — MCQ

After successful `get(key)` in LRU, key should:

A. remain unchanged in recency

B. become most recently used

C. become least recently used

D. be deleted

Q5 — MCQ

When capacity is exceeded, LRU evicts:

A. most frequent

B. least recently used

C. largest key

D. newest key

Q6 — MCQ

In the dummy-head/dummy-tail design, LRU node is commonly:

A. `head.next`

B. `tail.prev`

C. `head`

D. `tail`

Q7 — MSQ

Which statements about LRU are true?

A. HashMap maps keys to linked-list nodes.

B. Successful access updates recency.

C. A DLL supports \(O(1)\) arbitrary node removal.

D. Standard LRU `get` must be \(O(n)\).

Q8 — MCQ

LFU primarily evicts the:

A. least frequently used item

B. most recently used item

C. highest key

D. smallest value

Q9 — MCQ

When LFU frequencies tie, standard implementation commonly uses:

A. random choice

B. LRU among tied frequency

C. largest value

D. smallest key always

Q10 — MCQ

Randomized Set usually combines:

A. HashMap + Array

B. Linked List + DFS

C. Trie + Queue

D. Fenwick + KMP

Q11 — MCQ

How do we remove an arbitrary value from Randomized Set in \(O(1)\) expected time?

A. Shift all elements

B. Swap with last, then pop

C. Sort array

D. Use BFS

Q12 — MCQ

Time-Based Key-Value Store typically uses:

A. HashMap + sorted timestamp list + Binary Search

B. DSU

C. Segment Tree only

D. Topological Sort

Q13 — MSQ

Which are design-style questions?

A. LRU Cache

B. Min Stack

C. Randomized Set

D. Time Map

Q14 — MCQ

For moving average over last K values, useful structures are:

A. Queue + Running Sum

B. DFS + recursion

C. Trie

D. Union-Find

Q15 — MCQ

The best first step in a data-structure design question is:

A. start coding immediately

B. list required operations and target complexities

C. always choose HashMap

D. always choose a tree

PRACTICE SOLUTIONS

Q1

$$\boxed{A}$$

Q2

$$\boxed{B}$$

Q3

$$\boxed{A}$$

Q4

A successful access makes it MRU.

\[\boxed{B} \]

Q5

\[\boxed{B} \]

Q6

With:

```text id="dsa2167" HEAD !” MRU ... LRU !” TAIL

```
LRU is:

\[
\boxed{tail.prev}
\]

Answer:

\[
\boxed{B}
\]

Q7

A, B, C true.

D false.

\[
\boxed{A,B,C}
\]

Q8

\[
\boxed{A}
\]

Q9

\[
\boxed{B}
\]

Q10
```
```

```
\[
\boxed{A}
\]

---

## Q11

\[
\boxed{B}
\]

---

## Q12

\[
\boxed{A}
\]

---

## Q13

All four.

\[
\boxed{A,B,C,D}
\]

---

## Q14

\[
\boxed{A}
\]

---

## Q15

\[
\boxed{B}
\]

---

 #  Ø=ÜÝ   N O T E S   T O   W R I T E

```text id="dsa2168"
DESIGN QUESTIONS:

Operations
!'   R e q u i r e d   C o m p l e x i t y
!'   R e q u i r e m e n t s
!'   D a t a   S t r u c t u r e s
!'   I n v a r i a n t s


LRU CACHE:

Need:
get O(1)
put O(1)
```
```

```
Use:
HashMap + Doubly Linked List

HashMap:
 k e y  !’   n o d e

DLL:
 H E A D  !”   M R U   . . .   L R U  !”   T A I L

Successful get/put:
move node to MRU

Capacity exceeded:
remove tail.prev

Dummy head/tail:
remove special cases


LFU:
 k e y  !’   n o d e
 f r e q u e n c y  !’   D L L
minFreq

Evict:
lowest frequency
then LRU tie-break


RANDOMIZED SET:
HashMap + Array

Remove:
swap target with last
update index
pop last


TIME MAP:
 k e y  !’   s o r t e d   l i s t   o f
(timestamp,value)

get:
Binary Search


MOVING AVERAGE:
Queue + Running Sum
```

DESIGN PATTERNMASTER TABLE

Requirement	Structure
Fast key lookup	HashMap
Maintain order + arbitrary removal	DLL
Random element	Array
Min/max repeatedly	Heap

Requirement	Structure
Latest K events	Queue
Time-version lookup	Sorted list + Binary Search
Prefix lookup	Trie
Median stream	Two Heaps

&ß30-SECOND REVISION

1. Design questions begin with operations and target complexities.
2. LRU = Least Recently Used.
3. LRU needs HashMap + DLL.
4. HashMap gives \(O(1)\) node lookup.
5. DLL gives \(O(1)\) removal/insertion.
6. Dummy head/tail simplify edge cases.
7. `head.next` = MRU.
8. `tail.prev` = LRU.
9. Successful `get` changes recency.
10. Existing `put` changes recency.
11. Eviction removes from both DLL and HashMap.
12. LFU = Least Frequently Used.
13. LFU also needs recency tie-breaking.
14. Randomized Set = HashMap + Array.
15. O(1) array deletion !' swap with last.
16. Time Map = HashMap + Binary Search.
17. Moving Average = Queue + running sum.
18. Min Stack = Stack + min tracking.
19. Different operations may require multiple data structures.
20. Always check for hidden \(O(n)\) operations.

' Placement Module 21 Complete

Next Placement Module:

MODULE 22 — MIXED INTERVIEW PATTERNRECOGNITION: HOW TO IDENTIFY THE RIGHT DSA IN 30–60 SECONDS

Next we'll stop learning structures in isolation and train the skill interviewers actually care about:

- translating English questions into patterns,
- constraint-based algorithm selection,
- recognizing hidden graphs,
- choosing HashMap vs Sorting,

- Stack vs Heap,
- BFS vs Dijkstra,
- Greedy vs DP,
- Sliding Window vs Prefix Hashing,
- Binary Search on Answer,
- DSU vs DFS,
- top 50 trigger words,
- brute-force !' optimal reasoning,
- and a large **"Which algorithm should I use?" interview drill**.

Type `NEXT` to continue.